LIC. EN **INTELIGENCIA ARTIFICIAL — LIC-EN-IA**

MATERIA

Algoritmos y Estructura de Datos

AED · PRIMER CUATRIMESTRE

UBP UNIVERSIDAD Blas Pascal

CONTACTO
0810 1223 3827
informes@ubp.edu.ar

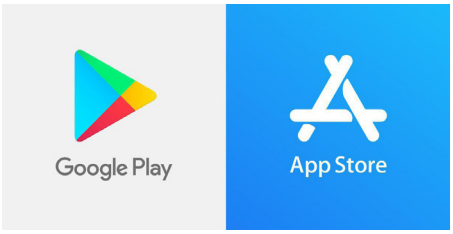
WEB Y REDES
www.**ubp.edu.ar**

Códigos QR

Estimado Alumno:

Esta materia puede contener **Códigos QR** para agilizar el acceso a los contenidos audiovisuales que el docente ha desarrollado especialmente para Usted.

Para poder visualizar los videos utilice el lector de código QR que trae por defecto su teléfono o dispositivo móvil. En caso de no disponer de uno, puede descargarlo desde *Google Play* si su sistema operativo corresponde a Android o desde la *App Store* si utiliza IOS.



Una vez instalado el software en su dispositivo móvil:

1) Abra la aplicación
2) Apunte con la cámara de su dispositivo hacia los códigos QR que encuentre en el interior de su materia.

De esta manera estará visualizando los videos que el docente ha desarrollado y/o seleccionado especialmente para usted.

Índice

Bienvenida 4

Propósito 4

Objetivos 5

Competencias 6

Contenidos mínimos 8

Contenidos 8

Bibliografía 10

Planificación 11

Metodología 13

Evaluación 14

Mapa conceptual 15

Módulos

- *Módulo 1* 16
- *Módulo 2* 58
- *Módulo 3* 102
- *Módulo 4* 139
- *Módulo 5* 202
- *Módulo 6* 298

Importante

Lecturas básicas y complementarias disponibles desde plataforma.

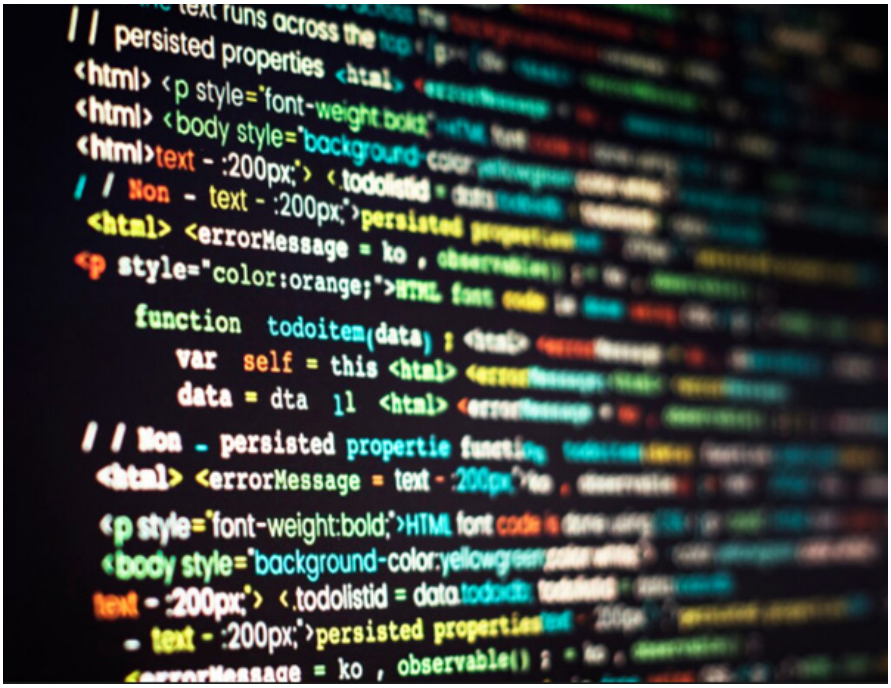
Impresión total del documento 440 páginas

Bienvenida

Bienvenidos a la materia:

"Algoritmos y Estructura de Datos"

Propósito



Fuente: Freepik

¡Bienvenidos/as a Algoritmos y Estructuras de Datos!

Esta asignatura es fundamental en la Licenciatura en Inteligencia Artificial porque introduce los fundamentos de las estructuras de datos y del análisis de eficiencia de los algoritmos. En ella se estudian las estructuras de datos fundamentales —como arrays, listas enlazadas, pilas, colas y árboles— y se implementan en un lenguaje de programación moderno, aprendiendo a definir clases, objetos y métodos. El curso incorpora el análisis de complejidad mediante notación BigO, algoritmos esenciales de búsqueda, ordenamiento, emparejamiento y fusión, así como técnicas avanzadas iterativas y recursivas para resolver problemas clásicos de optimización y grafos.

Estas herramientas les van a permitir evaluar y seleccionar las estructuras de datos y paradigmas algorítmicos adecuados para representar conocimientos, procesar grandes volúmenes de información y diseñar soluciones eficientes a problemas típicos de la Inteligencia Artificial.

En relación con el perfil profesional, la comprensión profunda de algoritmos y estructuras de datos constituye la base sobre la cual se desarrollan los sistemas inteligentes. Esta materia los/las prepara para diseñar y construir soluciones computacionales que hagan uso eficiente de los recursos, aplicando criterios de complejidad para asegurar la escalabilidad de los modelos. A partir de este conocimiento se podrá implementar algoritmos de búsqueda, clasificación, optimización y procesamiento de grafos que sirven de soporte a técnicas de aprendizaje automático, razonamiento heurístico y toma de decisiones.

Asimismo, la asignatura sienta las bases para cursos avanzados de IA —como aprendizaje profundo, procesamiento de lenguaje natural o visión por computadora— al brindar herramientas para manejar estructuras de datos complejas, diseñar algoritmos recursivos o iterativos y evaluar su desempeño.

Así, la materia contribuye a formarlos/las como profesionales capaces de analizar problemas reales de la inteligencia artificial, elegir las estructuras de datos más adecuadas y desarrollar algoritmos eficientes que integren estas estructuras en soluciones robustas y éticas.

¡Comencemos!

Objetivos

- Seleccionar algoritmos y estructuras de datos adecuados para resolver problemas propios de la inteligencia artificial, evaluando alternativas de búsqueda, ordenamiento y optimización según criterios de eficiencia y escalabilidad.

- Implementar y validar estructuras de datos dinámicas con el objetivo de optimizar el desempeño de aplicaciones de inteligencia artificial mediante la codificación y el uso de estructuras como listas, árboles y grafos.

- Diseñar algoritmos recursivos e iterativos para abordar problemas de grafos, teoría de números y programación dinámica, considerando su complejidad y pertinencia en distintos contextos de aplicación.

- Integrar algoritmos y estructuras de datos en programas modulares a fin de desarrollar soluciones de inteligencia artificial robustas, aplicando principios básicos de ingeniería de software y control de calidad del código.

- Comparar y evaluar la eficiencia de distintas soluciones algorítmicas para seleccionar estrategias adecuadas en función de los requerimientos de cada proyecto, utilizando la notación Big O como criterio de análisis.

- Reflexionar sobre los aspectos éticos y de seguridad involucrados en el diseño de algoritmos y el uso de datos con el objetivo de promover prácticas responsables en el desarrollo de soluciones de inteligencia artificial.

Competencias

Relación de la asignatura con las competencias de egreso

En la tabla siguiente se establece la relación de la asignatura con las competencias de egreso: Específicas, Genéricas Tecnológicas, Sociales, Políticas y Actitudinales de la carrera. Se incluyen las competencias de egreso a las que tributa su nivel de aporte (ninguno, bajo, medio y alto).

Competencias Genéricas Tecnológicas (CG)	Nivel
CG1. Identificación, formulación y resolución de problemas complejos mediante técnicas de ciencia de datos y desarrollo de sistemas de IA, empleando modelos matemáticos, algorítmicos y estadísticos para proponer soluciones basadas en datos.	Alta
CG2. Concepción, diseño y desarrollo de proyectos de ciencia de datos y sistemas de inteligencia artificial, integrando metodologías de ingeniería, buenas prácticas de programación y principios éticos aplicados al uso de datos.	Alta
CG3. Gestión, planificación, ejecución y control de proyectos vinculados con la construcción, entrenamiento, validación y despliegue de modelos de IA y soluciones de ciencia de datos, asegurando eficiencia, trazabilidad y colaboración interdisciplinaria.	Alta
CG4. Utilización de técnicas, lenguajes y herramientas especializadas de análisis de datos, aprendizaje automático y desarrollo de modelos de IA, seleccionando las más adecuadas según la naturaleza del problema y la disponibilidad de recursos.	Alta
CG5. Generación de desarrollos tecnológicos y/o innovaciones basadas en inteligencia artificial y ciencia de datos, proponiendo soluciones originales y escalables que respondan a necesidades reales de organizaciones y comunidades.	Alta

Competencias Sociales, Políticas y Actitudinales (CSPA)	Nivel
CSPA6. Desempeño en equipos de trabajo.	Alta
CSPA7. Comunicación efectiva.	Media
CSPA8. Acción ética y responsable.	Media
CSPA9. Evaluación y actuación en relación con el impacto social de su actividad en el contexto global y local.	Media
CSPA10. Aprendizaje continuo.	Alta
CSPA11. Acción emprendedora	Baja

Competencias Específicas (CE)[1]	Nivel
CE 1.1 CD Especificación, proyección y desarrollo de modelos de ciencia de datos y sistemas basados en IA, considerando requerimientos técnicos, éticos y de disponibilidad de datos.	Media

CE1.2 CD Especificación, proyección y desarrollo de infraestructuras y sistemas de comunicación de datos orientados a la adquisición, procesamiento y despliegue de modelos de IA.	Media
CE 1.3 IA Especificación, proyección y desarrollo de software para análisis, entrenamiento, validación y operación de modelos de inteligencia artificial, integrando algoritmos y estructuras de datos avanzadas.	Media
CE 2.1 SI–IA Proyección y dirección de procesos vinculados a la seguridad de datos, resguardo de modelos y protección de información sensible empleada en sistemas de IA y ciencia de datos.	Media
CE 3.1 M–IA Establecimiento de métricas, normas y estándares de calidad para modelos analíticos, procesos de minería de datos y sistemas de inteligencia artificial, garantizando su confiabilidad y desempeño.	Media
CE 4.1 C–IA Certificación del funcionamiento, evaluación de riesgos, validación y condición de uso de modelos de IA, pipelines de ciencia de datos y sistemas de software asociados, incluyendo aspectos de transparencia, auditoría y robustez.	Media
CE 5.1 D–CD/IA Dirección y control de la implementación, operación y mantenimiento de soluciones de ciencia de datos y sistemas de inteligencia artificial, incluyendo infraestructura, monitoreo, mejora continua, seguridad y aseguramiento de calidad.	Media

Resultados de Aprendizaje (RA)

RA N°	Resultado de Aprendizaje
RA1	Modelar y representar datos complejos mediante el uso de estructuras de datos adecuadas (listas, árboles, grafos, pilas y colas), justificando su elección en función de las necesidades de los algoritmos de IA.
RA2	Diseñar e implementar algoritmos de búsqueda, ordenamiento y de grafos que se utilicen comúnmente en aplicaciones de IA, demostrando su correcta funcionalidad mediante pruebas unitarias.
RA3	Analizar la complejidad temporal y espacial de algoritmos y estructuras de datos, empleando notación BigO y comparando diferentes soluciones para seleccionar la más adecuada a un problema de IA, considerando el balance entre tiempo y espacio.
RA4	Implementar y evaluar algoritmos recursivos, iterativos, voraces y de programación dinámica para resolver problemas de optimización y planificación en IA (caminos mínimos, árboles de expansión mínima, flujos máximos y el problema del viajante).
RA5	Utilizar estructuras avanzadas (árboles B, montículos binarios y de Fibonacci) y algoritmos de compresión y codificación de cadenas para procesar grandes volúmenes de datos y mejorar la eficiencia de algoritmos de IA.
RA6	Desarrollar aplicaciones modulares que integren múltiples algoritmos y estructuras de datos para proyectos de IA, empleando buenas prácticas de programación, control de versiones y documentación reproducible.
RA7	Identificar y mitigar sesgos y riesgos éticos relacionados con el uso de algoritmos y estructuras de datos en proyectos de IA, garantizando la privacidad, la equidad y la transparencia en el manejo de datos y en la toma de decisiones.

Contenidos mínimos

- Estructuras de datos fundamentales: Arrays, listas enlazadas, pilas, colas, árboles.
- Complejidad algorítmica: Notación Big-O, análisis de algoritmos.
- Algoritmos Esenciales: Búsqueda - Ordenamiento - Emparejamiento -Fusión.
- Algoritmos avanzados: Iterativos - Recursivos (van los típicos, CMínimo, FMáximo – TSP, etc)

Contenidos

Módulo 1: Introducción a Python y programación orientada a objetos

- Tema 1: Sintaxis básica de Python: variables, tipos de datos primitivos y estructuras de control.
- Tema 2: Contenedores nativos: listas, tuplas, conjuntos y diccionarios.
- Tema 3: Funciones, módulos y paquetes; organización de proyectos.
- Tema 4: Principios de programación orientada a objetos: definición de clases, atributos y métodos, encapsulamiento.
- Tema 5: Herencia, polimorfismo y clases abstractas (abc), con ejemplos de jerarquías de clases.
- Tema 6: Diseño modular y documentación usando docstrings.

Módulo 2: Estructuras de datos lineales y análisis de complejidad

- Tema 1: Concepto de tipo de dato abstracto y su implementación en Python.
- Tema 2: Arrays y listas dinámicas: operaciones, comprensión de listas y manejo de memoria.
- Tema 3: Pilas y colas implementadas con listas y collections.deque; usos en algoritmos recursivos y secuenciales.
- Tema 4: Colas de prioridad: conceptos de heap y uso del módulo heapq.
- Tema 5: Árboles binarios y n-arios: creación de nodos, recorridos (preorden, inorden, postorden) y relación con la recursividad.
- Tema 6: Notación Big-O y análisis temporal y espacial de operaciones básicas; medición de rendimiento con time/timeit.

Módulo 3: Búsqueda, ordenamiento y hashing

- Tema 1: Búsqueda secuencial y binaria; uso del módulo bisect para búsquedas indexadas.
- Tema 2: Árboles de búsqueda binaria (BST): inserción, búsqueda y eliminación; conceptos de indexación y soporte de índices.
- Tema 3: Tablas de dispersión: principios de hashing, uso de diccionarios de Python, funciones de transformación de claves y técnicas de resolución de colisiones.
- Tema 4: Algoritmos de ordenamiento: selección, inserción, burbuja, quicksort y mergesort; comparación con sorted() y list.sort().
- Tema 5: Algoritmos de emparejamiento y fusión de listas ordenadas.

Módulo 4: Árboles y estructuras avanzadas

- Tema 1: Árboles AVL y rojo-negro: equilibrio automático y análisis de rotaciones.
- Tema 2: Árboles B y sus variantes: motivación para indexación en disco, inserción, búsqueda y eliminación.
- Tema 3: Montículos binarios y montículos de Fibonacci: implementaciones con listas y heapq; operaciones y uso en heap sort.
- Tema 4: Conjuntos disjuntos y estructura Union-Find: operaciones de unión y búsqueda, y aplicaciones.
- Tema 5: Recursión avanzada aplicada a estructuras de datos y uso de generadores e iteradores para recorrer grandes estructuras en Python.

Módulo 5: Grafos y algoritmos de grafos

- Tema 1: Representación de grafos: listas de adyacencia con diccionarios de listas y matrices de adyacencia.
- Tema 2: Recorridos de grafos: búsqueda en profundidad (DFS) y búsqueda en amplitud (BFS); análisis de componentes conexas y clausura transitiva mediante Warshall.
- Tema 3: Grafos ponderados: algoritmos de caminos mínimos, incluyendo Dijkstra implementado con heapq.
- Tema 4: Orden topológico y detección de ciclos en grafos dirigidos.
- Tema 5: Árboles de expansión mínima: algoritmos de Kruskal y Prim y su implementación.
- Tema 6: Algoritmos de flujo máximo (por ejemplo, Edmonds–Karp) y resolución del problema del viajante (TSP) mediante búsqueda exhaustiva y heurísticas.

Módulo 6: Grafos y algoritmos de grafos

- Tema 1: Programación dinámica y algoritmos voraces: solución de problemas como mochila, cadena de matrices, caminos mínimos y análisis de su eficiencia.
- Tema 2: Introducción a la programación lineal y entera: método Simplex, problemas de transporte y uso de bibliotecas de optimización en Python.
- Tema 3: Algoritmos de aproximación y complejidad computacional: clase NP, algoritmos deterministas y no deterministas de tiempo polinómico, y ejemplos de problemas NP-completos.
- Tema 4: Transformada rápida de Fourier (FFT) y algoritmos sobre polinomios; aplicaciones en análisis de señales y procesamiento de datos.
- Tema 5: Geometría computacional: cálculos de cercos convexos, inclusión en polígonos, búsqueda por rango e intersección de segmentos; resolución del vecino más próximo y uso de diagramas de Voronoi.
- Tema 6: Algoritmos sobre textos y cadenas: búsqueda de patrones (fuerza bruta y Knuth–Morris–Pratt), compresión (Huffman, LZW) y encriptación basada en teoría de números (RSA).

Bibliografía

Obligatoria

Material didáctico de producción institucional diseñado y desarrollado especialmente para cada módulo.

Complementaria

- Araujo, L., Martínez-Unanue, R., & Rodríguez-Artacho, M. (2011). *Programación y estructuras de datos avanzadas* (2.ª ed., reimp. 2016). Editorial Universidad Ramón Areces.
- Arana Torres, S., & Córdova Neri, T. (2024). *Python desde el laboratorio: Aplicaciones GUI, integración con base de datos e inteligencia artificial*. Marcombo.
- Bel, W. (2020). *Algoritmos y estructuras de datos en Python: Un enfoque ágil y estructurado*. Editorial UADER. http://editorial.uader.edu.ar/libros/amalgama/algoritmos-y-estructuras-de-datos-en-phyton
- Biecek, P., & Kozak, A. (2022). *La guía del viajero al aprendizaje automático responsable: Inteligencia artificial interpretable y explicable*.
- Coeckelbergh, M. (2021). *Ética de la inteligencia artificial*. Ediciones Cátedra.
- GeeksforGeeks. (2023, 22 de marzo). *Introduction to Min-Heap*. https://www.geeksforgeeks.org/dsa/introduction-to-min-heap-data-structure/
- GeeksforGeeks. (2023, 30 de noviembre). *DSA tutorial*. https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/
- GeeksforGeeks. (2024, 24 de enero). *Queue data structure*. https://www.geeksforgeeks.org/dsa/queue-data-structure/
- GeeksforGeeks. (2024, 22 de mayo). *Binary tree data structure*. https://www.geeksforgeeks.org/dsa/binary-tree-data-structure/
- GeeksforGeeks. (2024, 7 de noviembre). *Graph data structure*. https://www.geeksforgeeks.org/dsa/graph-data-structure/
- GeeksforGeeks. (2025). *Linear search algorithm*. https://www.geeksforgeeks.org/dsa/linear-search/
- GeeksforGeeks. (2025, 23 de julio). *Binary heap*. https://www.geeksforgeeks.org/dsa/binary-heap/
- GeeksforGeeks. (2025, 23 de julio). *Fibonacci heap | Set 1 (Introduction)*. https://www.geeksforgeeks.org/dsa/fibonacci-heap-set-1-introduction/

- GeeksforGeeks. (2025, 8 de diciembre). *Introduction to Red-Black tree*. https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/
- GeeksforGeeks. (2025, 20 de diciembre). *Introduction to AVL tree*. https://www.geeksforgeeks.org/dsa/introduction-to-avl-tree/
- GeeksforGeeks. (2025, 22 de diciembre). *Heap sort*. https://www.geeksforgeeks.org/dsa/heap-sort/
- GeeksforGeeks. (2026, 31 de enero). *Binary search*. https://www.geeksforgeeks.org/dsa/binary-search/
- Miller, B., & Ranum, D. (2014). *Solución de problemas con algoritmos y estructuras de datos usando Python*. Runestone Academy. https://runestone.academy/ns/books/published/pythoned/index.html
- Moreno González, P. J. (2026). *MLOps: Del notebook al pipeline*. Edición independiente.
- Python Software Foundation. (s. f.). *bisect — Array bisection algorithm*. En *Python 3 Library Reference*. https://docs.python.org/3/library/bisect.html

Planificación

Actividades	Módulo	Semana	Metodología	Agrupamiento	Carácter	Competencia
Proyecto Integrador 1	1	1 a 3	*Actividades autoasistidas + laboratorio aplicado en Online IDE Pro + mini-proyecto integrador*	*Desarrollo grupal – Entrega individual*	*Formativa Opcional*	CG1–CG2–CG4; CSPA7–CSPA8–CSPA10; CE 1.3 IA (opc.: CE 3.1 M–IA, CE 4.1 C–IA).
Proyecto Integrador 2	2	4 y 5	*Actividades autoasistidas + laboratorio aplicado en Online IDE Pro + mini-proyecto integrador*	*Desarrollo grupal – Entrega individual*	*Formativa Opcional*	CG1–CG2–CG4; CSPA7–CSPA8–CSPA10; CE 1.3 IA (opc.: CE 3.1 M–IA, CE 4.1 C–IA).
Evaluación Integradora – Parte 1 – Semana 6 – Individual – Formativa Obligatoria						

Proyecto Integrador 3	*3*	**7 y 8**	*Actividades autoasistidas + laboratorio aplicado en Online IDE Pro + mini-proyecto integrador*	*Desarrollo grupal – Entrega individual*	*Formativa Opcional*	CG1–CG2–CG4; CSPA7–CSPA8–CSPA10; CE 1.3 IA (opc.: CE 3.1 M–IA, CE 4.1 C–IA).
Proyecto Integrador 4	*4*	**9 y 10**	*Actividades autoasistidas + laboratorio aplicado en Online IDE Pro + mini-proyecto integrador*	*Desarrollo grupal – Entrega individual*	*Formativa Opcional*	CG1–CG2–CG4; CSPA7–CSPA8–CSPA10; CE 1.3 IA (opc.: CE 3.1 M–IA, CE 4.1 C–IA).
Proyecto Integrador 5	*5*	**11 y 12**	*Actividades autoasistidas + laboratorio aplicado en Online IDE Pro + mini-proyecto integrador*	*Desarrollo grupal – Entrega individual*	*Formativa Opcional*	CG1–CG2–CG4; CSPA7–CSPA8–CSPA10; CE 1.3 IA (opc.: CE 3.1 M–IA, CE 4.1 C–IA).
Proyecto Integrador 6	*6*	**13 y 14**	*Actividades autoasistidas + laboratorio aplicado en Online IDE Pro + mini-proyecto integrador*	*Desarrollo grupal – Entrega individual*	*Formativa Opcional*	CG1–CG2–CG4; CSPA7–CSPA8–CSPA10; CE 1.3 IA (opc.: CE 3.1 M–IA, CE 4.1 C–IA).
Evaluación Integradora – Parte 2 – Semana 15 – Individual – Formativa Obligatoria						

Metodología

A) Estrategias de enseñanza: Este espacio curricular fue diseñado según lo previsto en el Sistema Institucional de Educación a Distancia (SIED) de la UBP (Resolución MECCYT 197/2019), las características de los destinatarios, las necesidades de esta disciplina, competencias que se espera promover en esta asignatura y el perfil del egresado.

En relación con lo anterior, para llevar a cabo la enseñanza se cuenta con:

- Contenidos y actividades de aprendizaje desarrollados en la plataforma virtual miUBP.
- Material bibliográfico especialmente seleccionado.
- Clases en video, en las que el docente expone contenidos que considera relevantes y que pueden ser visualizadas en cualquier momento.
- Tutorías virtuales que implican diferentes espacios de interacción.

B) Materiales curriculares: Durante el cursado, se fomenta el estudio sistemático de los contenidos que integran el programa de la asignatura, presentados en diversos soportes y lenguajes (textos teóricos, ejemplos de código, esquemas, material audiovisual y recursos interactivos), así como la resolución progresiva de las actividades de aprendizaje propuestas. De este modo, se introduce al alumnado en los conceptos fundamentales que deben dominarse en relación con las estructuras de datos y los algoritmos, indispensables para el diseño, análisis e implementación de soluciones computacionales en el campo de la Inteligencia Artificial.

Asimismo, se promueve la lectura de bibliografía básica y complementaria que permita profundizar y ampliar los contenidos abordados, con especial énfasis en el uso del lenguaje Python, el análisis de la eficiencia algorítmica, la resolución de problemas de optimización y la aplicación de estructuras de datos en contextos propios de la IA. Por otra parte, se proponen actividades de aprendizaje de carácter obligatorio y optativo, debidamente identificadas en la plataforma, orientadas al desarrollo del pensamiento algorítmico, la experimentación con distintos enfoques de solución y la integración de saberes teóricos y prácticos.

C) Modalidad de agrupamiento: Los alumnos pueden optar por agruparse libremente para estudiar la materia y resolver las actividades de aprendizaje planteadas, de forma individual o grupal. En este sentido, en esta asignatura se propone:

a) Trabajo individual, de modo que el alumno pueda desenvolverse en forma autónoma, reconociendo sus fortalezas y debilidades, a la vez que puede desarrollar sus propias estrategias de aprendizaje.

b) Trabajo colaborativo, de modo que se favorezca el desarrollo de las competencias del trabajo en equipo para resolver eficientemente las futuras situaciones a las cuales puede enfrentarse como profesional.

D) Interacción: La comunicación con el docente tutor se realiza a través de los diferentes medios disponibles en la plataforma miUBP.

E) Formación práctica: En lo que se refiere a la formación práctica se hace foco en la resolución de problemas que implique el uso de los contenidos estudiados en la asignatura contextualizados en situaciones que tengan que ver con la carrera.

Evaluación

La metodología de evaluación propuesta se presenta conforme al Reglamento Académico General de la UBP.

Cada instrumento de evaluación contiene sus criterios de evaluación explicitados y el puntaje otorgado a cada consigna.

En cuanto a los criterios de acreditación, la escala cuantitativa utilizada para evaluar es del 1 al 100 y el puntaje mínimo que se requiere para la aprobación es de 50.

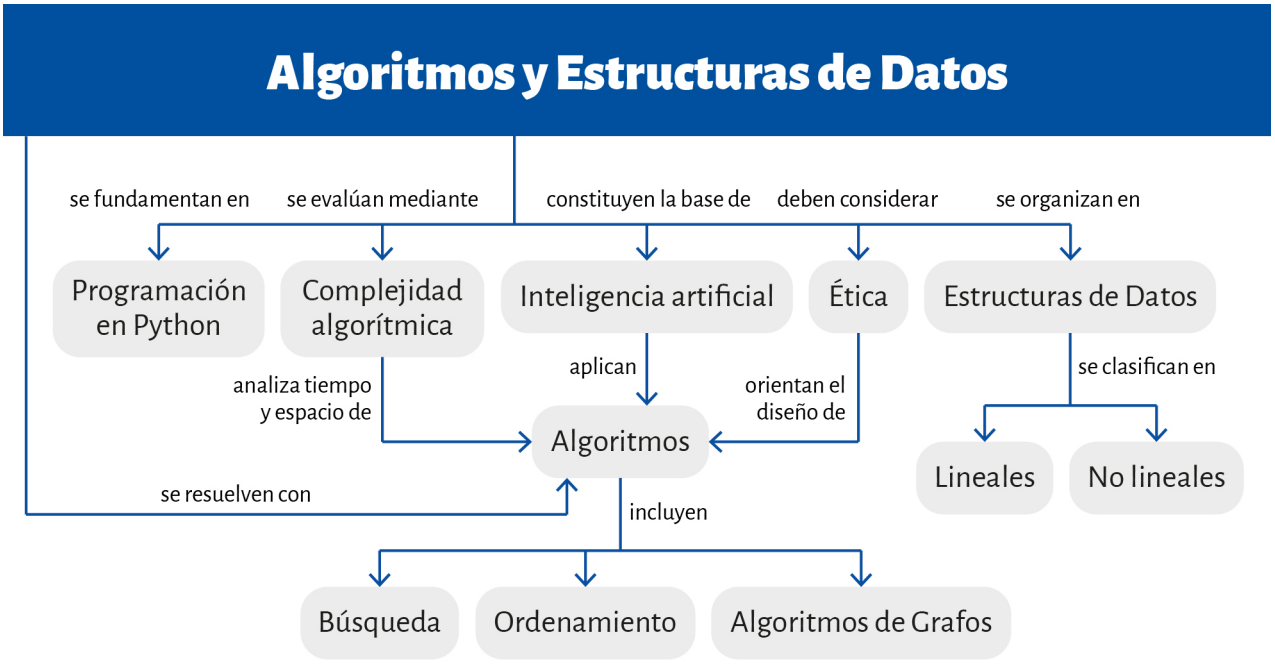
Regularidad:

Para alcanzar la condición de alumno regular usted deberá aprobar la Evaluación Integradora dentro de los plazos previstos para el cursado. Si reprueba esta instancia podrá reelaborar la evaluación según los requerimientos del docente.

Examen Final:

Deberá rendir, en condición de regular, una evaluación final que se realizará a través de un examen escrito integrador, presencial, en el periodo que disponga la Universidad.

Mapa conceptual



Mapa conceptual animado

Módulo 1

Microobjetivos

- Desarrollar pensamiento algorítmico para expresar soluciones claras y ordenadas en la resolución de problemas computacionales propios de la Inteligencia Artificial.
- Aplicar estructuras de datos nativas de Python para representar y organizar información en el procesamiento inicial de datasets simples.
- Implementar programas modulares en Python para favorecer la reutilización y el mantenimiento del código en desarrollos introductorios de análisis de datos.
- Analizar de manera empírica el tiempo y el uso de memoria de algoritmos simples para reconocer criterios básicos de eficiencia en contextos iniciales de IA.
- Integrar buenas prácticas de codificación y documentación para garantizar la legibilidad y trazabilidad del software en el desarrollo académico de sistemas de Inteligencia Artificial

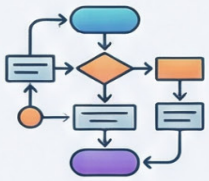
Contenidos

INTRODUCCIÓN A PYTHON Y PROGRAMACIÓN ORIENTADA A OBJETOS

Infografía 1: Esquema integrador de los contenidos del Módulo 1, que articula fundamentos algorítmicos, estructuras de datos, programación orientada a objetos, manejo de datos con pandas y análisis de eficiencia en el contexto de la Inteligencia Artificial. Imagen creada en NotebookLM a partir de información del autor. Puede verla a continuación

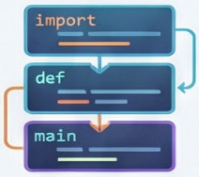
Módulo 1: Programación en Python y Manejo de Datos para IA

Fundamentos Algorítmicos y Estructuras



Pensamiento Algorítmico

Transformar problemas en soluciones computacionales claras, eficientes y reutilizables.



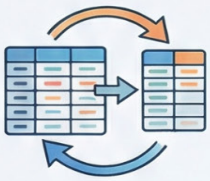
Estructura de un Programa

Organización basada en tres pilares: Importación de modulos, definición de funciones y código principal.

Contenedores Nativos

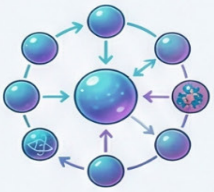
Uso estratégico de listas, tuplas, diccionarios y conjuntos para la representación flexible de datos.

Procesamiento, Objetos y Eficiencia



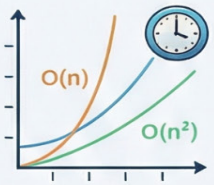
Manejo de Datos y Pandas

Preprocesamiento de datasets mediante lectura de archivos CSV/JSON y el uso de estructuras tabulares (DataFrames).



Programación Orientada a Objetos (POO)

Modelado de entidades de IA mediante clases, encapsulamiento y herencia para crear sistemas modulares.



Análisis de Complejidad (Big-O)

Evaluación empírica y teórica de la eficiencia temporal y espacial de los algorítmos desarrollados.

Pandas

Manipulación y análisis de datos tabulares.

POO

Diseño de modelos y sistemas escalables.

Notación Big-O

Medición de eficiencia en grandes volúmenes de datos.

NotebookLM

1. Introducción

El estudio de los algoritmos y las estructuras de datos constituye uno de los pilares fundamentales en la formación de profesionales de la Licenciatura en Inteligencia Artificial.

Fuente: Freepik

Antes de abordar técnicas avanzadas de modelado, aprendizaje automático u optimización, resulta imprescindible comprender cómo se representan, organizan y procesan los datos, así como de qué manera se diseñan algoritmos eficientes para operar sobre ellos.

En este contexto, el presente módulo introduce los fundamentos de la programación en Python y del manejo estructurado de datos, proporcionando las bases necesarias para el desarrollo del pensamiento algorítmico. A lo largo del mismo, se analizarán los conceptos esenciales que permiten transformar un problema en una solución computacional clara, eficiente y reutilizable, atendiendo tanto a la corrección funcional como al uso adecuado de los recursos.

La apropiación de estos conocimientos permitirá comprender el funcionamiento interno de muchas de las técnicas empleadas en la Inteligencia Artificial, evaluar la eficiencia de distintas estrategias de resolución y sentar las bases para el diseño de sistemas inteligentes robustos, escalables y responsables.

2. Introducción a Python: sintaxis, estructuras de control, funciones y módulos

2.1 Introducción a Python como lenguaje base para la inteligencia artificial

Python se adopta en esta asignatura como el lenguaje principal de trabajo debido a su sintaxis clara, su carácter multiparadigma y su amplia utilización en el ámbito de la Inteligencia Artificial y la Ciencia de Datos.

Estas características permiten que el foco del aprendizaje esté puesto en la comprensión de los algoritmos y de las estructuras de datos, y no en la complejidad sintáctica del lenguaje.

Actividad

A partir de los conceptos introducidos sobre programación en Python y su rol como herramienta para expresar ideas algorítmicas, ya se cuenta con los elementos necesarios para comenzar a aplicar estos fundamentos de manera práctica. La siguiente actividad propone poner en juego el pensamiento algorítmico trabajado, priorizando la claridad del razonamiento por sobre la complejidad sintáctica. **Actividad 1: Pensamiento algorítmico y Python como lenguaje base.**

2.1.1 Características generales del lenguaje

Énfasis

Python es un lenguaje interpretado, de alto nivel y multiparadigma.

Esto significa que el código se ejecuta línea por línea, sin necesidad de compilación previa, y que admite distintos estilos de programación.

2.1.2 Ejemplo 1: Primer programa en Python

```
print("Hola, Inteligencia Artificial")
```

Elemento	Explicación
`print()`	Función incorporada de Python que permite mostrar información por pantalla.
Cadena de texto	Representa un mensaje simple que confirma la correcta ejecución del programa.
Resultado	Verifica el funcionamiento del entorno de desarrollo y la ejecución secuencial de instrucciones.

Captura 1: Este ejemplo permite verificar el entorno de ejecución y comprender la ejecución secuencial de instrucciones. – Propia – Online IDE Pro

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

2.2 Estructura básica de un programa en Python

Ilustración 1: Principios de diseño del código orientados a la claridad, la reutilización y el mantenimiento, fundamentales en el desarrollo de algoritmos y sistemas de Inteligencia Artificial. Imagen creada en Napkin a partir de información del autor.

Énfasis

Un programa en Python suele organizarse en tres partes porque esa estructura ayuda a que el código sea más claro, reutilizable y fácil de mantener, especialmente cuando se trabaja con algoritmos y procesamiento de datos (como ocurre en Inteligencia Artificial).

En primer lugar, se incluye una sección destinada a la importación de módulos y librerías, que permite disponer desde el inicio de las herramientas necesarias para el desarrollo del programa. Luego, se desarrolla la lógica central del procesamiento, donde se especifican las instrucciones y operaciones que resuelven el problema planteado. Finalmente, se encuentra el bloque de ejecución principal, encargado de preparar los datos de entrada, poner en marcha el procesamiento definido y presentar los resultados obtenidos. Como dijimos anteriormente, esta forma de organización no solo ordena el código, sino que también contribuye a su mantenimiento y a la posibilidad de ampliarlo o modificarlo en el futuro.

2.2.1 Importación de módulos

Énfasis

Los módulos son bibliotecas que ya traen funcionalidades listas para usar (por ejemplo, manejo de tiempo, matemáticas, estructuras eficientes, lectura de archivos, etc.).

Importarlos al inicio permite:

- Reutilizar código confiable en lugar de "reinventar" funciones.
- Mantener el programa ordenado (se sabe qué herramientas se usan).
- Acceder a estructuras y algoritmos optimizados.

2.2.2 Definición de funciones

Énfasis

Las funciones permiten encapsular una tarea para poder:

- Reutilizarla sin copiar y pegar código.
- Probarla de manera independiente.
- Mejorar la legibilidad (el programa "se lee" como una secuencia de pasos).
- Separar la lógica del algoritmo del resto del programa.

En IA esto es clave, porque suele repetirse la misma lógica: cargar datos, preparar datos, calcular métricas, aplicar un algoritmo, etc.

2.2.3 Código principal

Énfasis

El código principal es la parte que se ejecuta cuando se corre el archivo.

Su objetivo es:

- "Orquestar" el flujo: qué se hace primero, qué se hace después.
- Llamar a funciones y mostrar resultados.
- Evitar que el archivo se convierta en una lista desordenada de instrucciones.

2.2.4 Ejemplo 2: Programa completo

En este ejemplo se observa que los módulos necesarios se importan al inicio del programa, lo que permite disponer de herramientas externas de manera organizada. La lógica principal del procesamiento se encapsula en funciones reutilizables, favoreciendo la cla-

ridad, el mantenimiento y la reutilización del código. El código principal se encarga de preparar los datos de entrada, invocar dichas funciones y presentar los resultados obtenidos. Y, finalmente, se incorpora una medición básica del tiempo de ejecución como primer acercamiento al análisis de la eficiencia del algoritmo, aspecto que posteriormente se formalizará mediante el estudio de la notación Big-O.

A continuación, el código:

```python
# 1) IMPORTACIÓN DE MÓDULOS (en este ejemplo no es estrictamente necesario,
# pero se incluye para mostrar la estructura)
import time

# 2) DEFINICIÓN DE FUNCIONES
def suma_lista(numeros):
    """Devuelve la suma de todos los elementos de una lista."""
    total = 0
    for n in numeros:
        total += n
    return total

def maximo_lista(numeros):
    """Devuelve el valor máximo de una lista."""
    maximo = numeros[0]
    for n in numeros:
        if n > maximo:
            maximo = n
    return maximo

# 3) CÓDIGO PRINCIPAL
if __name__ == "__main__":
    datos = [3, 7, 2, 9, 4]

    inicio = time.time()

    suma = suma_lista(datos)
    maximo = maximo_lista(datos)

    fin = time.time()

    print("Datos:", datos)
    print("Suma:", suma)
    print("Máximo:", maximo)
    print("Tiempo de ejecución (segundos):", fin - inicio)

```

Captura 2: Este ejemplo muestra la estructura completa de un programa que calcula la suma de una lista y también encuentra el máximo, dos operaciones típicas que luego se generalizan en estructuras más complejas. – Propia – Online IDE Pro

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

El código anterior muestra un ejemplo completo y estructurado de un programa en Python, diseñado para ilustrar buenas prácticas en la implementación de algoritmos y el análisis preliminar de su eficiencia.

Analicemos cada parte del código:

```
# 1) IMPORTACIÓN DE MÓDULOS (en este ejemplo no es estrictamente necesario,
# pero se incluye para mostrar la estructura)
import time
```

En primer lugar, se realiza la **importación del módulo time,** que en este caso no es estrictamente necesaria para el cálculo de la suma o el máximo, pero se incluye para mostrar cómo se integran módulos externos y para permitir la medición del tiempo de ejecución del algoritmo.

```
def suma_lista(numeros):
    """Devuelve la suma de todos los elementos de una lista."""
    total = 0
    for n in numeros:
        total += n
    return total
```

A continuación, se define la **función suma_lista**, cuya finalidad es calcular la suma de todos los elementos de una lista de números. Dentro de la función se inicializa una variable acumuladora (total) y se recorre la lista mediante un ciclo for, agregando cada valor al acumulador. La función devuelve finalmente el resultado obtenido. El uso de un *docstring* permite documentar claramente qué hace la función.

```
def maximo_lista(numeros):
    """Devuelve el valor máximo de una lista."""
    maximo = numeros[0]
    for n in numeros:
        if n > maximo:
            maximo = n
    return maximo
```

Luego se define la **función maximo_lista**, encargada de determinar el valor máximo de una lista. Para ello, se toma inicialmente el primer elemento como valor máximo y se recorre la lista comparando cada elemento con el máximo actual. Si se encuentra un valor mayor, este reemplaza al anterior. Al finalizar el recorrido, la función retorna el valor máximo encontrado. Esta implementación refleja un algoritmo clásico de búsqueda de máximo.

```
# 3) CÓDIGO PRINCIPAL
if __name__ == "__main__":
    datos = [3, 7, 2, 9, 4]

    inicio = time.time()

    suma = suma_lista(datos)
    maximo = maximo_lista(datos)

    fin = time.time()

    print("Datos:", datos)
    print("Suma:", suma)
    print("Máximo:", maximo)
    print("Tiempo de ejecución (segundos):", fin - inicio)

```

Posteriormente se presenta el **código principal**, delimitado por la condición if __name__ == "__main__":, que garantiza que el bloque se ejecute únicamente cuando el archivo se corre directamente. En este bloque se define la lista de datos a procesar, se registra el tiempo inicial, se invocan las funciones definidas previamente para calcular la suma y el máximo, y luego se registra el tiempo final. La diferencia entre ambos tiempos permite obtener una medición básica del tiempo de ejecución.

Finalmente, el programa imprime por pantalla la lista de datos, los resultados obtenidos y el tiempo de ejecución en segundos. De esta manera, este ejemplo integra conceptos clave como modularización, reutilización de funciones, recorrido de estructuras de datos y una primera aproximación al análisis de eficiencia, sentando bases sólidas para el estudio posterior de algoritmos y estructuras de datos en el contexto de la Inteligencia Artificial.

Luego de analizar la estructura general de un programa en Python y de observar ejemplos que integran funciones, módulos y código principal, es posible avanzar hacia una instancia de aplicación. La actividad que se presenta a continuación permite reforzar la importancia de la organización y la modularidad como principios fundamentales en el desarrollo de algoritmos. **Actividad 2: Estructura de programa y modularización.**

2.3 Uso del intérprete y de entornos de desarrollo: Online IDE Pro

Online IDE Pro se utilizará como entorno de desarrollo a lo largo de la asignatura porque permite disponer de un espacio de trabajo accesible, uniforme y orientado al aprendizaje de los algoritmos, sin requerir instalaciones locales ni configuraciones previas.

Al ejecutarse Python como un script completo en un entorno en línea, facilita la escritura, ejecución y análisis inmediato del código, lo que resulta especialmente adecuado para la experimentación algorítmica y la validación progresiva de soluciones.

Además, su interfaz simple favorece la concentración en la lógica de los algoritmos y en el uso de las estructuras de datos, reduciendo las distracciones asociadas a la configuración del entorno.

De este modo, Online IDE Pro se convierte en una herramienta que acompaña el desarrollo del pensamiento algorítmico, permite realizar pruebas rápidas, observar resultados y reflexionar sobre la eficiencia y el comportamiento del código desde las primeras etapas del cursado.

Una vez comprendido el rol del intérprete de Python y las ventajas del uso de un entorno de desarrollo en línea, se está en condiciones de explorar activamente estas herramientas. La siguiente actividad invita a experimentar con la ejecución de código y a reflexionar sobre el comportamiento de los algoritmos en función de los datos utilizados. **Actividad 3: Pensamiento algorítmico y análisis de eficiencia.**

2.4 Buenas prácticas iniciales de escritura de código

Desde el inicio del cursado se promueve la escritura de código legible y ordenado, el uso de nombres descriptivos para variables, funciones y estructuras de datos y la incorporación de comentarios claros que expliquen la finalidad del código y las decisiones tomadas durante su desarrollo.

Estas prácticas favorecen la comprensión de los algoritmos, facilitan el mantenimiento y la reutilización del software y resultan fundamentales en el trabajo colaborativo y en el desarrollo de soluciones propias del ámbito de la Inteligencia Artificial.

2.4.1 Ejemplo 3: Prueba de ideas algorítmicas

El siguiente fragmento de código es un ejemplo básico de cómo funcionan los bucles en Python. Permite ejecutar el mismo bloque de código repetidamente, lo que es fundamental en la programación para realizar tareas que deben repetirse múltiples veces sin necesidad de escribir el mismo código varias veces.

```
1  for i in range(5):
2      print("Iteración:", i)
```

Captura 3: Este fragmento muestra el uso de un ciclo for para repetir una acción, una estructura fundamental en el diseño de algoritmos. – Propia – Online IDE Pro

Línea / Bloque	Elemento del código	Explicación
1	`for i in range(5):`	Define un ciclo que itera 5 veces (valores 0 a 4). Permite repetir una acción de forma controlada.
2	`print("Iteración:", i)`	Imprime el valor actual del contador `i` en cada iteración, evidenciando el recorrido paso a paso.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

2.4.2 Ejemplo 4: Suma de los primeros n números naturales

Este ejemplo introduce el uso de comentarios explicativos que permiten comprender la intención del código, la elección de variables con nombres significativos que reflejan claramente su propósito dentro del algoritmo, y la definición de una función bien delimitada que encapsula una tarea específica.

En conjunto, estos elementos contribuyen a mejorar la claridad, la legibilidad y la organización del programa, aspectos esenciales para el desarrollo de soluciones algorítmicas correctas y mantenibles en el contexto de la Inteligencia Artificial.

```python
# Calcula la suma de los primeros n números naturales
def suma_naturales(n):
    suma = 0
    for i in range(1, n + 1):
        suma += i
    return suma

resultado = suma_naturales(10)
print("Resultado:", resultado)
```
Press Ctrl K to generate code

Captura 4: Ejemplo de implementación en Python de un algoritmo sencillo que calcula la suma de los primeros n números naturales, ilustrando el uso de una función, variables con nombres significativos, un ciclo for y comentarios explicativos como buenas prácticas iniciales de programación. – Propia – Online IDE Pro.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

El siguiente cuadro detalla el análisis del ejemplo anterior:

Línea / Bloque	Elemento del código	Explicación
1	Comentario	Describe la finalidad general del algoritmo para orientar la lectura del código.
2	`def suma_naturales(n):`	Define una función parametrizable que recibe `n` como dato de entrada.
3	`suma = 0`	Inicializa un acumulador que guardará el resultado parcial y final.
4–5	`for i in range(1, n + 1):` `suma += i`	Recorre los números del 1 al *n* y acumula cada valor en `suma`, implementando el algoritmo de sumatoria.
6	`return suma`	Devuelve el resultado final al finalizar el ciclo.
8	`resultado = suma_naturales(10)`	Ejecuta la función con un valor específico (10) y almacena el retorno.
9	`print("Resultado:", resultado)`	Imprime el valor calculado, verificando el funcionamiento del algoritmo.

2.5 Python como herramienta para expresar algoritmos

El objetivo central de este espacio no consiste en memorizar reglas sintácticas de Python ni en reproducir de manera mecánica la forma "correcta" de escribir instrucciones, sino en desarrollar la capacidad de pensar algorítmicamente y expresar ese razonamiento mediante código claro, ordenado y verificable.

En este sentido, Python se utiliza como un medio para transformar una estrategia de resolución —por ejemplo, recorrer una lista, tomar decisiones con una condición, acumular un resultado o buscar un elemento— en una implementación concreta que pueda ejecutarse y evaluarse.

Énfasis

La sintaxis se incorpora de manera progresiva como una herramienta al servicio del algoritmo: se aprende aquello que se necesita para que la idea pueda comunicarse con precisión. Por esta razón, Python resulta especialmente adecuado, ya que permite representar muchos procedimientos de forma directa y legible, favoreciendo que el foco esté puesto en qué hace el algoritmo, por qué lo hace y con qué eficiencia, más que en detalles formales del lenguaje.

2.5.1 Ejemplo 5: Algoritmo simple de decisión

Este ejemplo muestra el uso de estructuras condicionales para evaluar situaciones y tomar decisiones dentro del programa, permitiendo representar de manera directa un algoritmo lógico. A través de las condiciones se controla el flujo de ejecución del código, reflejando cómo una idea basada en reglas y comparaciones puede traducirse de forma clara y precisa a una implementación en Python.

```
1  def es_par(numero):
2      if numero % 2 == 0:
3          return True
4      else:
5          return False
6
7  print(es_par(8))
8  print(es_par(7))
9  Press Ctrl K to generate code
```

Captura 5: Ejemplo de una función en Python que implementa un algoritmo simple de decisión para determinar si un número es par, utilizando una condición if–else y devolviendo un valor booleano, lo que ilustra cómo expresar una lógica algorítmica clara mediante código legible. – Propia – Online IDE Pro

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

2.6 Relación con la Inteligencia Artificial

Estos ejemplos, aunque simples en su planteo, constituyen la base conceptual y práctica sobre la cual se construirán algoritmos de búsqueda, el procesamiento de estructuras de datos, distintas estrategias de optimización y, posteriormente, implementaciones de modelos y sistemas inteligentes.

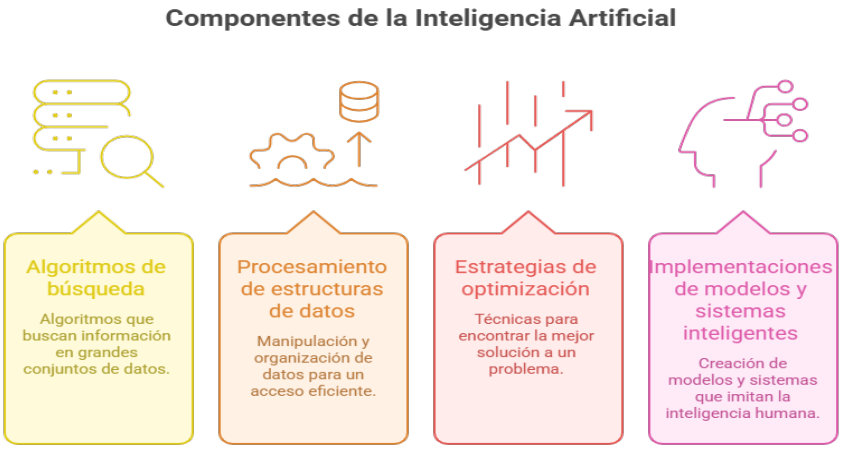


Ilustración 2: Relación entre los fundamentos algorítmicos abordados en el módulo —algoritmos de búsqueda, procesamiento de estructuras de datos y estrategias de optimización— y su proyección hacia la implementación de modelos y sistemas inteligentes propios del campo de la Inteligencia Artificial. – Propia – Napkin

El dominio de Python como lenguaje para expresar algoritmos les permitirá abordar con solvencia los contenidos de los módulos siguientes, en los que se trabajará con estructuras más complejas y con problemas característicos del campo de la Inteligencia Artificial, manteniendo siempre el foco en la claridad, la corrección y la eficiencia de las soluciones desarrolladas.

3. Contenedores nativos (listas, tuplas, diccionarios, conjuntos) y su uso en la representación de datos.

Énfasis

En Python, los contenedores nativos **list, tuple, dict y set** permiten representar y organizar información de manera flexible, lo cual resulta clave para el trabajo posterior con estructuras de datos, algoritmos y problemas propios de la Inteligencia Artificial.

La documentación oficial de Python los define como contenedores integrados de uso general y, a su vez, introduce contenedores especializados (como deque o Counter) que resultan más adecuados cuando se necesitan operaciones específicas con mayor eficiencia.

A continuación, se desarrollan estos contenedores, indicando cuándo conviene usarlos y presentando ejemplos ejecutables en Online IDE Pro.

3.1 Listas (list): secuencias mutables para datos ordenados

Una lista es una secuencia ordenada y mutable, adecuada para almacenar colecciones de elementos y modificarlas (agregar, eliminar, actualizar).

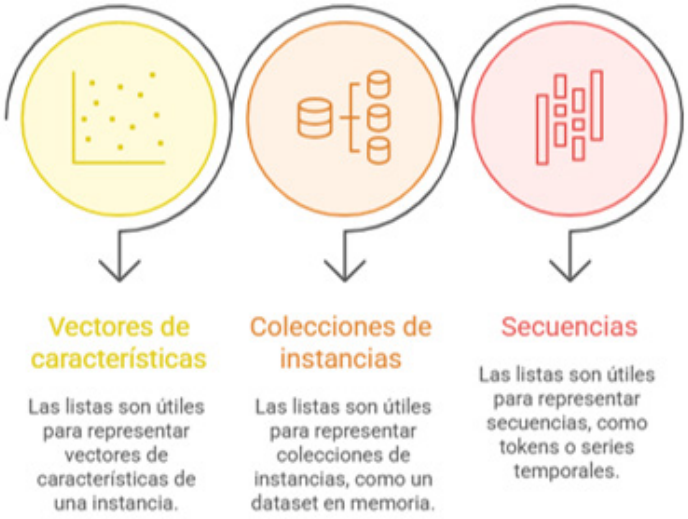


Ilustración 3: Usos de las listas en Python para la representación de vectores de características, colecciones de instancias y secuencias de datos en aplicaciones de Inteligencia Artificial. Imagen creada con Napkin, a partir de información y contenidos elaborados por la autora.

3.1.1 Ejemplo A: Representar un "dataset" pequeño como lista de instancias (filas)

El código que se muestra en la imagen ilustra cómo trabajar con listas anidadas en Python para manipular y acceder a datos específicos.

```python
# Dataset simple: cada instancia tiene [edad, ingresos, etiqueta]
dataset = [
    [22, 35000, 0],
    [45, 82000, 1],
    [36, 62000, 1],
]

print("Cantidad de instancias:", len(dataset))
print("Primera instancia:", dataset[0])

# Acceso a un atributo (edad) de cada instancia
edades = [fila[0] for fila in dataset]
print("Edades:", edades)
```

Captura 6: Ejemplo de representación de un dataset simple en Python mediante una lista de instancias y extracción de un atributo específico, ilustrando el manejo básico de datos en Inteligencia Artificial. Código desarrollado en Online IDE Pro, a partir de información y contenidos elaborados por la autora.

Línea / Bloque	Elemento del código	Explicación
1	Comentario	Explicita la estructura de cada instancia: `[edad, ingresos, etiqueta]`.
2–6	`dataset = [[...], [...], [...]]`	Define un dataset pequeño en memoria como lista de listas, donde cada sublista es una instancia.
8	`len(dataset)`	Obtiene la cantidad de instancias. Resulta útil para validar tamaño del dataset.
9	`dataset[0]`	Accede a la primera instancia para inspección rápida de la estructura.
11	Comentario	Indica que se extraerá un atributo (edad) de cada instancia.
12	`edades = [fila[0] for fila in dataset]`	Recorre el dataset y extrae la primera columna (edad) mediante comprensión de listas.
13	`print("Edades:", edades)`	Imprime la lista resultante para verificar la extracción.

Énfasis

Una lista permite recorrer instancias y extraer columnas/atributos con facilidad, lo que luego se conecta con algoritmos de búsqueda y análisis.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

3.2 Tuplas (tuple): secuencias inmutables para "registros" estables

Una tupla es una secuencia ordenada e inmutable, adecuada cuando los datos no deberían cambiar (por ejemplo, un registro fijo o una coordenada).

Ilustración 4: Usos de las tuplas en Python para representar coordenadas, pares clave y registros estables en el contexto de la Inteligencia Artificial. Imagen creada con Napkin, a partir de información y contenidos elaborados por la autora.

3.2.1 Ejemplo B: Coordenadas como tuplas. Representación típica en algoritmos de IA y geometría

```python
# Coordenadas de puntos (x, y)
p1 = (2, 5)
p2 = (7, 1)

print("Punto 1:", p1)
print("Punto 2:", p2)

# Distancia Manhattan (muy usada en heurísticas como A*)
dist = abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
print("Distancia Manhattan:", dist)
```

Captura 7: Ejemplo de implementación en Python para el cálculo de la distancia Manhattan entre dos puntos, utilizada habitualmente en heurísticas de búsqueda como A*. Código desarrollado en Online IDE Pro, a partir de información y contenidos elaborados por la autora.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Énfasis

Las tuplas son adecuadas para representaciones compactas y estables, especialmente cuando el dato se usa como unidad indivisible dentro de un algoritmo.

Línea / Bloque	Elemento del código	Explicación
1	Comentario	Indica que se trabajará con coordenadas `(x, y)`.
2–3	`p1 = (2, 5)`, `p2 = (7, 1)`	Define dos puntos como tuplas, adecuadas para representar registros inmutables.
5–6	`print(...)`	Muestra ambos puntos para confirmar valores de entrada.
8–9	`dist = abs(p1[0]-p2[0]) + abs(p1[1]-p2[1])`	Calcula distancia Manhattan como suma de diferencias absolutas en cada dimensión.
10	`print("Distancia Manhattan:", dist)`	Presenta el resultado del cálculo para verificación.

3.3 Diccionarios (dict): mapeos clave. Valor para indexación y conteos

Un diccionario es un mapeo mutable que relaciona claves hashables con valores. Es especialmente útil cuando se requiere acceso rápido por clave (por ejemplo, "buscar por ID", "contar ocurrencias", "acceder por nombre de atributo").

Énfasis

Dict es el tipo estándar de "mapping" en Python y establece una restricción clave: las claves deben ser hashables, es decir, valores inmutables que puedan utilizarse de forma segura como identificadores dentro del diccionario.

Ilustración 5: Usos de los diccionarios en Inteligencia Artificial para la representación de registros, la construcción de vocabularios, el conteo de frecuencias y el almacenamiento de grafos. Imagen creada con Napkin, a partir de información y contenidos elaborados por la autora.

3.3.1 Ejemplo C: Registro de una instancia como diccionario–features con nombres

```
instancia = {
    "edad": 36,
    "ingresos": 62000,
    "compra": 1
}

print("Edad:", instancia["edad"])
print("Campos disponibles:", list(instancia.keys()))
Press Ctrl K to generate code
```

Captura 8: Ejemplo de uso de un diccionario en Python para representar una instancia mediante campos nombrados (features), mostrando el acceso a valores y a las claves disponibles. Código desarrollado en Online IDE Pro, a partir de información y contenidos elaborados por la autora.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Elemento del código	Explicación
1–5	`instancia = {"edad":..., "ingresos":..., "compra":...}`	Define un registro con campos nombrados, representando una instancia mediante claves semánticas.
7	`instancia["edad"]`	Accede al valor asociado a la clave `"edad"`, ilustrando indexación por clave.
8	`list(instancia.keys())`	Obtiene el listado de claves disponibles para inspección de campos.

3.3.2 Ejemplo D: Conteo de clases (frecuencia de etiquetas)

Para conteo, Python también ofrece Counter como contenedor especializado.

```
1  from collections import Counter
2
3  etiquetas = [0, 1, 1, 0, 1, 1, 0]
4  conteo = Counter(etiquetas)
5
6  print("Frecuencias:", conteo)
7  print("Cantidad de clase 1:", conteo[1])
8   Press Ctrl K to generate code
```

Captura 9: Ejemplo de uso de la clase Counter de la biblioteca collections para el conteo de frecuencias de clases en Python, aplicado al análisis de datos en Inteligencia Artificial. Código desarrolla.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Elemento del código	Explicación
1	`from collections import Counter`	Importa el tipo `Counter`, especializado para conteo de ocurrencias.
3	`etiquetas = [...]`	Define una lista de etiquetas/clases, típica en datasets supervisados.
4	`conteo = Counter(etiquetas)`	Calcula frecuencias de cada valor en la lista y las almacena como conteo.
6	`print("Frecuencias:", conteo)`	Muestra el conteo completo por clase.
7	`print(..., conteo[1])`	Accede directamente a la frecuencia de la clase `1`.

3.4 Conjuntos (set): colecciones sin duplicados para pertenencia y operaciones

Énfasis

Un conjunto es una colección no ordenada de elementos distintos.

La documentación oficial destaca los usos típicos que se presentan en la imagen a continuación:

Ilustración 6: Operaciones habituales sobre conjuntos en Python —pertenencia, eliminación de duplicados, unión e intersección— aplicadas al procesamiento de datos en Inteligencia Artificial. Imagen creada con Napkin, a partir de información y contenidos elaborados por la autora.

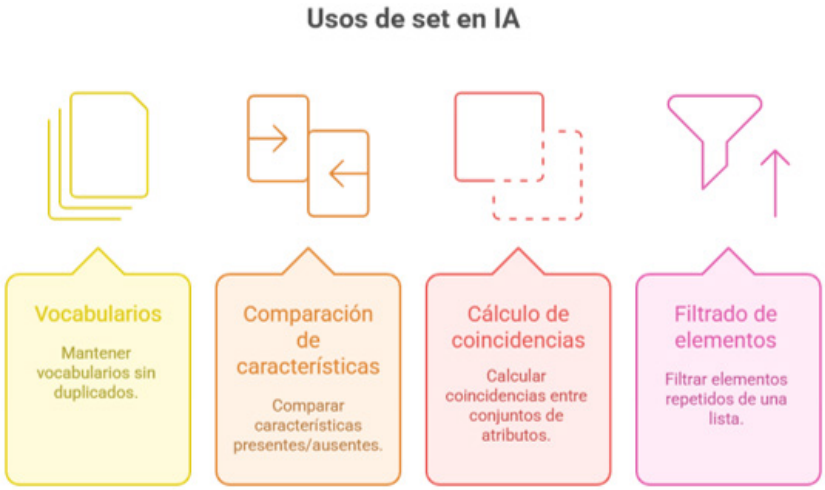


Ilustración 7: Usos de los conjuntos (set) en Inteligencia Artificial para la gestión de vocabularios, la comparación de características, el cálculo de coincidencias y el filtrado de elementos repetidos. Imagen creada con Napkin, a partir de información y contenidos elaborados por la autora.

3.4.1 Ejemplo E: Eliminar duplicados y comprobar pertenencia

```
1  tokens = ["ia", "datos", "ia", "python", "datos", "modelo"]
2  vocabulario = set(tokens)
3
4  print("Vocabulario:", vocabulario)
5  print("¿'python' está en el vocabulario?", "python" in vocabulario)
6  print("Tamaño del vocabulario:", len(vocabulario))
7   Press Ctrl K to generate code
```

Captura 10: Ejemplo de uso de conjuntos (set) en Python para construir un vocabulario sin duplicados, comprobar pertenencia y calcular su tamaño, aplicado al procesamiento básico de texto en Inteligencia Artificial. Código desarrollado en Online IDE Pro, a partir de información y contenidos elaborados por la autora.

Código Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Elemento del código	Explicación
1	`tokens = ["ia", "datos", ...]`	Define una lista con tokens, incluyendo repetidos (situación frecuente en texto).
2	`vocabulario = set(tokens)`	Convierte la lista a conjunto, eliminando duplicados para formar un vocabulario único.
4	`print("Vocabulario:", vocabulario)`	Muestra el conjunto resultante. El orden no está garantizado por ser `set`.
5	`"python" in vocabulario`	Verifica pertenencia en el conjunto, operación típica para filtros o validaciones.
6	`len(vocabulario)`	Calcula el tamaño del vocabulario, indicador básico en procesamiento de texto.

3.4.2 Ejemplo F: Intersección (rasgos compartidos)

```
1  rasgos_a = {"rapido", "preciso", "explicable"}
2  rasgos_b = {"preciso", "robusto"}
3
4  comunes = rasgos_a.intersection(rasgos_b)
5  print("Rasgos comunes:", comunes)
6
```

Captura 11: Ejemplo de uso de la operación de intersección entre conjuntos en Python para identificar rasgos comunes entre dos conjuntos de características, aplicado al análisis comparativo en Inteligencia Artificial. Código desarrollado en Online IDE Pro, a partir de información y contenidos elaborados por la autora.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Elemento del código	Explicación
1–2	`rasgos_a = {...}`, `rasgos_b = {...}`	Define dos conjuntos de características (sin duplicados).
4	`comunes = rasgos_a.intersection(rasgos_b)`	Calcula el conjunto de elementos comunes entre ambos conjuntos.
5	`print("Rasgos comunes:", comunes)`	Imprime el resultado para observar coincidencias.

Actividad

Tras el estudio de las distintas estructuras de datos nativas de Python —listas, tuplas, diccionarios y conjuntos— y de sus posibles aplicaciones, resulta pertinente integrar estos conceptos en una situación práctica. La actividad propuesta permite analizar cómo la elección de una estructura adecuada influye en la representación y el procesamiento de la información. **Actividad 4: Contenedores nativos y representación de datos**

4. Lectura, escritura y preprocesamiento de datos en Inteligencia Artificial

En el ámbito de la Inteligencia Artificial, los datos constituyen el insumo fundamental sobre el cual operan los algoritmos. Antes de aplicar técnicas de modelado, resulta imprescindible leer, interpretar, limpiar y transformar los datos provenientes de distintas fuentes.

Python ofrece bibliotecas estándar, como csv y json, que permiten trabajar con datos estructurados de forma simple, y bibliotecas especializadas como pandas, orientadas al manejo eficiente de estructuras tabulares ampliamente utilizadas en ciencia de datos e IA.

4.1 Lectura y escritura de archivos CSV

4.1.1 Ejemplo 1: Lectura de datos desde un archivo CSV

El formato CSV es ampliamente utilizado para representar datos tabulares simples, donde cada fila corresponde a una instancia y cada columna a un atributo. En Inteligencia Artificial, este formato suele emplearse para datasets iniciales o para intercambiar datos entre herramientas.

La biblioteca estándar CSV permite comprender de forma explícita cómo se leen los datos fila por fila y cómo se representan en memoria como listas, favoreciendo la comprensión de la relación entre la estructura externa del archivo y su procesamiento interno por los algoritmos.

```
1   import csv
2   with open("datos.csv", mode="r", newline="", encoding="utf-8") as archivo:
3       lector = csv.reader(archivo)
4       encabezado = next(lector)
5       filas = list(lector)
6
7   print("Encabezado:", encabezado)
8   print("Primera fila:", filas[0])
9   print("Cantidad de registros:", len(filas))
10  Press Ctrl K to generate code
```

Captura 12: Lectura de un archivo CSV y acceso a sus registros mediante la biblioteca estándar de Python, ejemplificando un primer acercamiento al manejo de datos estructurados. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`import csv`	Importa el módulo estándar para la lectura y escritura de archivos CSV.
`open("datos.csv", ...)`	Abre el archivo CSV en modo lectura, especificando codificación adecuada.
`csv.reader(archivo)`	Crea un lector que interpreta cada fila como una lista de valores.
`next(lector)`	Lee la primera fila del archivo, usualmente utilizada como encabezado.
`list(lector)`	Carga el resto de las filas en memoria como una lista de instancias.
`print(...)`	Permite verificar estructura, contenido y tamaño del dataset leído.

4.1.2 Ejemplo 2: Escritura de datos en un archivo CSV

En muchos procesos de IA es necesario almacenar datos procesados o resultados intermedios. La escritura de archivos CSV permite conservar estos resultados de forma estructurada y reutilizable. Este procedimiento es fundamental para la trazabilidad y reproducibilidad de los experimentos.

```python
import csv

datos = [
    ["edad", "ingresos", "etiqueta"],
    [22, 35000, 0],
    [45, 82000, 1],
    [36, 62000, 1]
]

with open("salida.csv", mode="w", newline="", encoding="utf-8") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerows(datos)

print("Archivo CSV generado correctamente.")
Press Ctrl K to generate code
```

Captura 13: Escritura de un archivo CSV a partir de una estructura de datos en Python, mostrando la generación de un dataset tabular con encabezados y registros. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`datos = [...]`	Define los datos a escribir, incluyendo encabezados y registros.
`open("salida.csv", "w")`	Abre el archivo en modo escritura, creando el archivo si no existe.
`csv.writer(archivo)`	Crea un objeto escritor asociado al archivo.
`writerows(datos)`	Escribe todas las filas en el archivo CSV.
`print(...)`	Confirma la correcta finalización de la operación.

4.1.3 Ejemplo 3: Lectura de datos desde un archivo JSON

El formato JSON permite representar datos con estructura jerárquica y semántica, lo cual es habitual en aplicaciones modernas de Inteligencia Artificial, especialmente en el intercambio de datos entre sistemas y servicios. Python permite convertir automáticamente estos datos en diccionarios y listas, facilitando su manipulación.

```python
import json

with open("instancias.json", "r", encoding="utf-8") as archivo:
    datos = json.load(archivo)

print("Cantidad de instancias:", len(datos))
print("Primera instancia:", datos[0])
```

Captura 14: Lectura de datos desde un archivo JSON y carga de instancias en estructuras nativas de Python, ilustrando el acceso a datos estructurados. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`import json`	Importa el módulo estándar para trabajar con JSON.
`open("instancias.json", "r")`	Abre el archivo JSON en modo lectura.
`json.load(archivo)`	Convierte el contenido del archivo en estructuras nativas de Python.
`print(...)`	Permite inspeccionar el contenido y verificar la correcta lectura.

4.1.4 Ejemplo 4: Escritura de datos en formato JSON

La escritura de archivos JSON resulta especialmente útil cuando se desea preservar la estructura semántica de los datos, como pares atributo–valor. Este enfoque es frecuente en configuraciones de modelos, almacenamiento de resultados y comunicación con APIs.

```python
import json

instancias = [
    {"edad": 22, "ingresos": 35000, "etiqueta": 0},
    {"edad": 45, "ingresos": 82000, "etiqueta": 1}
]

with open("salida.json", "w", encoding="utf-8") as archivo:
    json.dump(instancias, archivo, indent=4)

print("Archivo JSON generado correctamente.")
```

Captura 15: Escritura de datos en formato JSON a partir de una colección de instancias representadas como diccionarios en Python, preservando la estructura semántica de la información. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`instancias = [...]`	Define una colección de registros estructurados como diccionarios.
`json.dump(...)`	Serializa los datos y los guarda en formato JSON legible.
`indent=4`	Mejora la legibilidad del archivo generado.
`print(...)`	Indica que el archivo fue creado correctamente.

Actividad

Habiendo trabajado con distintos formatos de almacenamiento de datos y con operaciones básicas de lectura y escritura, se dispone ahora de los conocimientos necesarios para abordar un flujo simple de preprocesamiento. La actividad siguiente propone aplicar estos conceptos, destacando su relevancia en etapas iniciales de proyectos de Inteligencia Artificial. **Actividad 5: Lectura y Escritura de Datos (CSV y JSON).**

4.2 Introducción a estructuras tabulares con pandas

La biblioteca Pandas introduce una abstracción de alto nivel para el manejo de datos tabulares mediante estructuras como el DataFrame. Esta estructura permite trabajar con filas y columnas etiquetadas, realizar operaciones de selección, filtrado y agregación de manera eficiente y expresiva.

En el ámbito de la Inteligencia Artificial, los DataFrames se utilizan ampliamente para representar datasets de entrenamiento, validación y prueba. Su uso facilita la exploración inicial de los datos, la detección de patrones y la aplicación sistemática de transformaciones previas al modelado.

4.2.1 Ejemplo 5: Introducción a estructuras tabulares con pandas

La biblioteca pandas introduce una abstracción de alto nivel para el manejo de datos tabulares mediante el DataFrame, estructura ampliamente utilizada en ciencia de datos e Inteligencia Artificial. Permite trabajar con filas y columnas etiquetadas y facilita la exploración inicial del dataset.

```
1   import pandas as pd
2
3   datos = {
4       "edad": [22, 45, 36],
5       "ingresos": [35000, 82000, 62000],
6       "etiqueta": [0, 1, 1]
7   }
8
9   df = pd.DataFrame(datos)
10
11  print(df)
12  print("Columnas:", df.columns)
13  print("Cantidad de filas:", len(df))
```

Captura 16: Creación y visualización de un DataFrame en pandas a partir de datos estructurados, ilustrando el manejo básico de información tabular en Inteligencia Artificial. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`import pandas as pd`	Importa la biblioteca pandas con su alias habitual.
`datos = {...}`	Define los datos como un diccionario de listas.
`pd.DataFrame(datos)`	Construye un DataFrame tabular a partir del diccionario.
`df.columns`	Permite inspeccionar los nombres de las columnas.
`len(df)`	Obtiene la cantidad de filas del dataset.

4.3 Operaciones básicas de preprocesamiento con pandas

Énfasis

Las operaciones básicas de preprocesamiento con pandas, como la selección de columnas, el filtrado de filas según condiciones lógicas y la inspección de dimensiones del dataset, constituyen herramientas fundamentales para el análisis de datos. Estas operaciones permiten aislar subconjuntos relevantes de información, reducir ruido y enfocar el análisis en atributos significativos.

El dominio de estas técnicas habilita a comprender cómo los datos evolucionan a lo largo de un pipeline de IA, reforzando la conexión entre la representación de los datos y los algoritmos que posteriormente los utilizarán.

4.3.1 Ejemplo 6: Preprocesamiento básico con pandas

El preprocesamiento es una etapa crítica en cualquier pipeline de IA. Operaciones como la selección de columnas y el filtrado de instancias permiten preparar los datos para su análisis o modelado posterior.

```
1  edades = df["edad"]
2
3  df_filtrado = df[df["ingresos"] > 50000]
4
5  print("Edades:")
6  print(edades)
7  print("\nInstancias filtradas:")
8  print(df_filtrado)
```

Captura 17: Selección de una columna y filtrado de instancias en un DataFrame de pandas mediante condiciones lógicas, como parte del preprocesamiento de datos en Inteligencia Artificial. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`df["edad"]`	Selecciona una columna específica del DataFrame.
`df[df["ingresos"] > 50000]`	Filtra filas que cumplen una condición lógica.
`print(...)`	Muestra los resultados del preprocesamiento realizado.

Actividad

Luego de introducir las estructuras tabulares y las operaciones básicas con la biblioteca pandas, se encuentra el marco adecuado para profundizar su uso mediante la práctica. La siguiente actividad permite consolidar la comprensión de los DataFrame como herramienta central para el análisis y la manipulación de datos. **Actividad 6: Introducción a estructuras tabulares con Pandas.**

5. Programación orientada a objetos en Inteligencia Artificial

La programación orientada a objetos (POO) es un paradigma que organiza el software en torno a objetos que combinan datos (atributos) y comportamientos (métodos).

En el contexto de la Inteligencia Artificial, este enfoque es útil para modelar entidades del dominio (instancias, modelos, datasets, evaluadores), favorecer la reutilización de código y facilitar el mantenimiento y la extensibilidad de los sistemas. La POO promueve principios como encapsulamiento, abstracción, herencia y polimorfismo, ampliamente utilizados en el desarrollo de software complejo y escalable. (Gamma et al., 1995; Lutz, 2013).

5.1 Clases y objetos

Las clases permiten definir un tipo de objeto que representa una entidad del problema, mientras que los objetos son instancias concretas de dicha clase. En IA, este mecanismo se utiliza para representar, por ejemplo, una instancia de datos, un modelo o un componente del sistema.

5.1.1 Ejemplo 1: Definición de una clase simple para una instancia de datos

```python
class Instancia:
    def __init__(self, edad, ingresos, etiqueta):
        self.edad = edad
        self.ingresos = ingresos
        self.etiqueta = etiqueta

    def mostrar(self):
        return f"Edad: {self.edad}, Ingresos: {self.ingresos}, Etiqueta: {self.etiqueta}"

instancia1 = Instancia(36, 62000, 1)
print(instancia1.mostrar())
```

Captura 18: Definición de una clase en Python para modelar una instancia de datos mediante programación orientada a objetos, integrando atributos y métodos asociados. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`class Instancia:`	Define una clase que modela una instancia de datos.
`__init__(...)`	Constructor que inicializa los atributos del objeto.
`self.edad`, `self.ingresos`, `self.etiqueta`	Atributos que representan las características de la instancia.
`def mostrar(self):`	Método que encapsula la forma de presentar los datos del objeto.
`instancia1 = Instancia(...)`	Crea un objeto concreto a partir de la clase definida.
`print(instancia1.mostrar())`	Invoca un método del objeto y muestra su resultado.

5.2 Encapsulamiento y métodos

El encapsulamiento permite agrupar datos y operaciones relacionadas, ocultando detalles internos y exponiendo únicamente una interfaz clara. En IA, este principio facilita controlar cómo se accede y modifica la información asociada a una entidad.

5.2.1 Ejemplo 2: Encapsulamiento mediante métodos

```python
class ContadorClases:
    def __init__(self):
        self.frecuencias = {}

    def agregar(self, etiqueta):
        if etiqueta in self.frecuencias:
            self.frecuencias[etiqueta] += 1
        else:
            self.frecuencias[etiqueta] = 1

    def obtener_frecuencias(self):
        return self.frecuencias

contador = ContadorClases()
for e in [0, 1, 1, 0, 1]:
    contador.agregar(e)

print(contador.obtener_frecuencias())
```

Captura 19: Implementación de una clase en Python que encapsula el conteo de etiquetas mediante un diccionario, ejemplificando el uso de métodos y estado interno en programación orientada a objetos. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`class ContadorClases:`	Define una clase que encapsula el conteo de etiquetas.
`self.frecuencias`	Atributo interno que almacena el estado del objeto.
`agregar(self, etiqueta)`	Método que modifica el estado del objeto de forma controlada.
`obtener_frecuencias()`	Método que expone el resultado sin revelar la implementación interna.
Bucle `for`	Utiliza el objeto para procesar una colección de datos.
`print(...)`	Muestra el resultado del procesamiento encapsulado.

5.3 Herencia y reutilización

Énfasis

La herencia permite definir nuevas clases a partir de clases existentes, reutilizando comportamiento y extendiéndolo.

En IA, este mecanismo se emplea para crear variantes de modelos, evaluadores o estrategias de procesamiento.

5.3.1 Ejemplo 3: Herencia para especializar comportamiento

```
1   class ModeloBase:
2       def entrenar(self):
3           return "Entrenando modelo base"
4
5   class ModeloClasificacion(ModeloBase):
6       def entrenar(self):
7           return "Entrenando modelo de clasificación"
8
9   modelo = ModeloClasificacion()
10  print(modelo.entrenar())
```

Captura 20: Ejemplo de herencia en Python donde una clase derivada redefine el comportamiento de un método, ilustrando la reutilización y especialización de funcionalidades en programación orientada a objetos. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`class ModeloBase:`	Define una clase general con comportamiento común.
`entrenar()`	Método que representa una operación genérica.
`class ModeloClasificacion(ModeloBase):`	Define una subclase que hereda de `ModeloBase`.
Redefinición de `entrenar()`	Especializa el comportamiento heredado.
`modelo = ModeloClasificacion()`	Crea un objeto de la subclase.
`print(modelo.entrenar())`	Ejecuta el método especializado.

5.4 Diseño modular

Énfasis

El diseño modular consiste en dividir un programa en módulos independientes, cada uno con responsabilidades claras.

En proyectos de IA, esta práctica mejora la organización del código, favorece el trabajo colaborativo y facilita la experimentación con distintos componentes (Pressman, 2014).

5.4.1 Ejemplo 4: Separación lógica mediante funciones (base del diseño modular)

```python
def cargar_datos():
    return [22, 45, 36]

def procesar_datos(edades):
    return sum(edades) / len(edades)

datos = cargar_datos()
promedio = procesar_datos(datos)
print("Promedio de edades:", promedio)
```

Captura 21: Separación de responsabilidades mediante funciones para la carga y el procesamiento de datos, ilustrando principios básicos de diseño modular en Python. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`cargar_datos()`	Función responsable de obtener los datos.
`procesar_datos(edades)`	Función que aplica un procesamiento específico.
`datos = cargar_datos()`	Separa la obtención del procesamiento de datos.
`promedio = procesar_datos(datos)`	Aplica el procesamiento de forma modular.
`print(...)`	Muestra el resultado final.

5.5 Integración de POO y diseño modular en IA

La combinación de POO y diseño modular permite construir sistemas de Inteligencia Artificial más robustos, extensibles y comprensibles.

Énfasis

Las clases modelan entidades del dominio, mientras que los módulos organizan responsabilidades a nivel de sistema, facilitando la evolución progresiva del software.

5.5.1 Ejemplo 5: Clase con diseño modular interno

```python
class AnalizadorDatos:
    def __init__(self, datos):
        self.datos = datos

    def promedio(self):
        return sum(self.datos) / len(self.datos)

analizador = AnalizadorDatos([22, 45, 36])
print("Promedio:", analizador.promedio())
```

Captura 22: Definición de una clase en Python que integra datos y métodos asociados para su análisis, ejemplificando la combinación de programación orientada a objetos y diseño modular. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`class AnalizadorDatos:`	Define una clase que agrupa datos y operaciones asociadas.
`__init__`	Inicializa el estado interno del objeto.
`promedio()`	Método que implementa una operación específica sobre los datos.
`analizador = AnalizadorDatos(...)`	Crea un objeto con datos concretos.
`print(...)`	Ejecuta el método y muestra el resultado.

Actividad

Una vez presentados los conceptos fundamentales de la programación orientada a objetos y su integración con el diseño modular, resulta oportuno avanzar hacia una instancia de aplicación concreta. La actividad propuesta busca reforzar el modelado de entidades y procesos, práctica habitual en el desarrollo de sistemas de Inteligencia Artificial. **Actividad 7: Programación orientada a objetos en IA.**

6. Complejidad temporal y espacial en Inteligencia Artificial

En Inteligencia Artificial, la eficiencia de los algoritmos es un factor determinante, especialmente cuando se trabaja con grandes volúmenes de datos o modelos de alto costo computacional.

Énfasis

La complejidad temporal describe cómo crece el tiempo de ejecución de un algoritmo en función del tamaño de la entrada, mientras que la complejidad espacial se refiere al crecimiento del uso de memoria. Si bien el análisis formal se expresa mediante la notación Big-O, en una primera aproximación resulta pedagógicamente valioso realizar mediciones empíricas simples que permitan observar cómo varían el tiempo y la memoria al aumentar el tamaño de los datos.

Python ofrece herramientas básicas que facilitan este análisis inicial y favorecen la comprensión intuitiva de la eficiencia algorítmica (Cormen et al., 2009; Lutz, 2013).

6.1 Medición simple del tiempo de ejecución

Énfasis

La medición del tiempo de ejecución permite observar de manera práctica cuánto tarda un algoritmo en completarse para un conjunto de datos determinado.

Este tipo de medición constituye un primer acercamiento experimental a la complejidad temporal, previo al análisis formal.

6.1.1 Ejemplo 1: Medición de tiempo con el módulo time

```python
import time

def suma_lista(numeros):
    total = 0
    for n in numeros:
        total += n
    return total

datos = list(range(1, 1000001))

inicio = time.time()
resultado = suma_lista(datos)
fin = time.time()

print("Resultado:", resultado)
print("Tiempo de ejecución (segundos):", fin - inicio)

```

Captura 23: Medición empírica del tiempo de ejecución de un algoritmo lineal en Python, utilizando el módulo time como primera aproximación al análisis de complejidad temporal. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`import time`	Importa el módulo estándar para mediciones de tiempo.
`def suma_lista(numeros):`	Define una función que implementa un algoritmo de recorrido lineal.
Bucle `for`	Recorre todos los elementos de la lista, acumulando el resultado.
`datos = list(range(...))`	Genera un conjunto de datos de tamaño grande para observar el tiempo de ejecución.
`inicio = time.time()`	Registra el instante inicial antes de ejecutar el algoritmo.
`resultado = suma_lista(datos)`	Ejecuta el algoritmo a medir.
`fin = time.time()`	Registra el instante final tras la ejecución.
`fin - inicio`	Calcula el tiempo total de ejecución del algoritmo.

6.2 Comparación empírica de algoritmos

Comparar el tiempo de ejecución de dos algoritmos que resuelven el mismo problema permite visualizar diferencias de eficiencia y relacionarlas con su complejidad temporal. Esta práctica resulta fundamental para comprender por qué ciertos algoritmos son preferibles en contextos de IA.

6.2.1 Ejemplo 2: Comparación entre dos enfoques de suma

```python
import time

def suma_for(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

def suma_formula(n):
    return n * (n + 1) // 2

n = 1000000

inicio = time.time()
suma_for(n)
fin = time.time()
print("Tiempo suma con for:", fin - inicio)

inicio = time.time()
suma_formula(n)
fin = time.time()
print("Tiempo suma con fórmula:", fin - inicio)
```

Captura 24: Comparación empírica del tiempo de ejecución entre un algoritmo iterativo y una solución basada en fórmula matemática, evidenciando diferencias de eficiencia temporal. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`suma_for(n)`	Implementa una suma mediante un ciclo, con crecimiento proporcional a `n`.
`suma_formula(n)`	Implementa una suma mediante una fórmula matemática directa.
`n = 1000000`	Define el tamaño del problema a evaluar.
Medición con `time.time()`	Permite registrar el tiempo de ejecución de cada enfoque.
Comparación de tiempos	Evidencia diferencias empíricas de eficiencia entre algoritmos.

6.3 Introducción a la medición de complejidad espacial

Énfasis

La complejidad espacial analiza el uso de memoria adicional requerido por un algoritmo.

En IA, este aspecto es crítico al trabajar con grandes datasets o estructuras complejas. Python permite realizar mediciones simples del tamaño en memoria de objetos mediante funciones básicas.

6.3.1 Ejemplo 3: Medición aproximada del uso de memoria con sys.getsizeof

Este ejemplo muestra cómo estimar el tamaño en memoria de distintas estructuras de datos, permitiendo observar cómo crece el consumo de espacio al aumentar el tamaño de la entrada.

```python
import sys

lista_pequena = list(range(10))
lista_grande = list(range(100000))

print("Tamaño lista pequeña (bytes):", sys.getsizeof(lista_pequena))
print("Tamaño lista grande (bytes):", sys.getsizeof(lista_grande))

```

Captura 25: Estimación del uso de memoria de estructuras de datos en Python mediante sys.getsizeof, ilustrando una primera aproximación al análisis de complejidad espacial. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`import sys`	Importa el módulo que permite acceder a información del sistema.
`lista_pequena`	Define una lista de tamaño reducido.
`lista_grande`	Define una lista significativamente mayor.
`sys.getsizeof(...)`	Devuelve una estimación del tamaño en memoria del objeto.
`print(...)`	Permite comparar el uso de memoria entre estructuras de distinto tamaño.

6.4 Relación entre mediciones empíricas y notación Big-O

Énfasis

Las mediciones empíricas de tiempo y espacio no reemplazan al análisis teórico, pero constituyen una herramienta pedagógica clave para comprender intuitivamente el significado de la notación Big-O.

Al observar cómo varía el tiempo o la memoria con el tamaño de los datos, el estudiantado puede relacionar la experiencia práctica con conceptos como crecimiento lineal, constante o cuadrático.

6.4.1 Ejemplo 4: Observación del crecimiento temporal

```python
import time

def recorrer(n):
    for i in range(n):
        pass

for tamaño in [1000, 10000, 100000]:
    inicio = time.time()
    recorrer(tamaño)
    fin = time.time()
    print(f"Tamaño: {tamaño} - Tiempo:", fin - inicio)
```

Captura 26: Medición empírica del crecimiento del tiempo de ejecución de un algoritmo lineal al variar el tamaño de la entrada, como aproximación práctica a la complejidad temporal. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`def recorrer(n):`	Define una función que ejecuta un bucle vacío `n` veces.
Lista de tamaños	Establece distintos tamaños de entrada para el experimento.
Bucle `for` externo	Ejecuta el algoritmo para cada tamaño.
Medición de tiempo	Registra el tiempo de ejecución para cada caso.
Impresión de resultados	Permite observar el crecimiento del tiempo al aumentar `n`.

Actividad

Después de observar ejemplos de medición empírica del tiempo de ejecución y del uso de memoria, se cuenta con una base suficiente para analizar el comportamiento de los algoritmos frente a distintos tamaños de entrada. La actividad que sigue invita a reflexionar sobre la eficiencia como un criterio relevante en el diseño algorítmico. **Actividad 8: Medición empírica de complejidad.**

La introducción al cálculo de la complejidad temporal y espacial mediante mediciones simples en Python permite desarrollar una comprensión práctica y progresiva de la eficiencia algorítmica. Este enfoque resulta pertinente en la formación en Inteligencia Artificial, ya que articula la experimentación con el análisis teórico y los/las habilita a comparar y seleccionar algoritmos considerando las limitaciones de tiempo y memoria propias de los sistemas inteligentes.

7. Buenas prácticas de codificación en Inteligencia Artificial

Las buenas prácticas de codificación buscan garantizar que el software sea legible, mantenible, reutilizable y confiable. En el ámbito de la Inteligencia Artificial, donde los proyectos suelen evolucionar de manera incremental y colaborativa, estas prácticas resultan

esenciales para comprender algoritmos complejos, reproducir experimentos y facilitar la validación de resultados. Python promueve explícitamente estas prácticas a través de convenciones como el PEP 8, que establece lineamientos para la escritura clara y consistente del código (Python Software Foundation, 2024).

7.1 Ejemplo 1: Código legible y uso de nombres descriptivos

Este ejemplo muestra cómo el uso de nombres descriptivos, indentación adecuada y estructura clara mejora la comprensión de un algoritmo sencillo, facilitando su análisis y mantenimiento.

```python
def calcular_promedio(valores):
    suma = 0
    for valor in valores:
        suma += valor
    promedio = suma / len(valores)
    return promedio

edades = [22, 45, 36]
resultado = calcular_promedio(edades)
print("Promedio de edades:", resultado)
```

Press Ctrl K to generate code

Captura 27: Implementación de una función en Python para calcular el promedio de una colección de valores, ejemplificando buenas prácticas de codificación y claridad algorítmica. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
`def calcular_promedio(valores):`	Define una función con un nombre que describe claramente su propósito.
`suma = 0`	Inicializa una variable acumuladora con un nombre significativo.
Bucle `for valor in valores`	Recorre la colección de datos de forma explícita y legible.
`promedio = suma / len(valores)`	Calcula el resultado final utilizando una expresión clara.
`return promedio`	Devuelve el valor calculado, cerrando la función de forma coherente.
Código principal	Ejecuta la función con datos concretos y muestra el resultado.

7.2 Documentación del código mediante docstrings

La documentación es un componente clave del desarrollo de software profesional.

Énfasis

En Python, los docstrings permiten describir la finalidad de módulos, funciones y clases directamente en el código.

Esta práctica facilita la comprensión del comportamiento esperado, el uso correcto de los componentes y la generación automática de documentación. En proyectos de IA, los

docstrings resultan especialmente útiles para describir supuestos, entradas, salidas y limitaciones de los algoritmos (Lutz, 2013).

7.2.1 Ejemplo 2: Uso de docstrings en una función

Este ejemplo ilustra cómo documentar una función mediante un docstring, especificando su propósito, parámetros y valor de retorno.

```python
def calcular_maximo(numeros):
    """
    Calcula el valor máximo de una lista de números.

    Parámetros:
    numeros (list): lista de valores numéricos.

    Retorna:
    int o float: el valor máximo presente en la lista.
    """
    maximo = numeros[0]
    for n in numeros:
        if n > maximo:
            maximo = n
    return maximo

datos = [3, 7, 2, 9, 4]
print("Máximo:", calcular_maximo(datos))
```

Captura 28: Definición de una función en Python documentada mediante docstring, que calcula el valor máximo de una lista de números, ilustrando buenas prácticas de documentación y claridad algorítmica. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
Docstring	Describe de forma estructurada qué hace la función y cómo debe utilizarse.
`def calcular_maximo(numeros):`	Define una función claramente identificada por su nombre.
`maximo = numeros[0]`	Inicializa la variable con el primer valor de la lista.
Bucle `for`	Recorre la lista comparando cada elemento con el máximo actual.
`return maximo`	Devuelve el resultado final del algoritmo.
Código principal	Ejecuta la función y permite verificar su funcionamiento.

7.3 Comentarios y claridad en la lógica algorítmica

Los comentarios complementan a los docstrings al explicar decisiones puntuales dentro del código. Su uso adecuado ayuda a comprender por qué se adopta una determinada estrategia algorítmica, especialmente en etapas tempranas de formación y en desarrollos colaborativos.

7.3.1 Ejemplo 3: Uso de comentarios explicativos

Este ejemplo muestra cómo los comentarios pueden aclarar pasos clave de un algoritmo, sin reemplazar a la lógica del código.

```python
def contar_clases(etiquetas):
    # Diccionario para almacenar las frecuencias
    frecuencias = {}

    for etiqueta in etiquetas:
        # Incrementa el conteo de la etiqueta actual
        if etiqueta in frecuencias:
            frecuencias[etiqueta] += 1
        else:
            frecuencias[etiqueta] = 1

    return frecuencias

clases = [0, 1, 1, 0, 1]
print(contar_clases(clases))

```

Captura 29: Implementación de una función en Python para el conteo de frecuencias de clases mediante un diccionario, ejemplificando el uso de comentarios explicativos y claridad en la lógica algorítmica. Código desarrollado en Online IDE Pro. Autoría: elaboración propia.

Código

Acceso al código del ejemplo en este enlace https://tinyurl.com/265we34z

Línea / Bloque	Explicación
Comentario inicial	Explica la finalidad de la estructura utilizada.
`frecuencias = {}`	Inicializa un diccionario para almacenar resultados.
Bucle `for`	Recorre la lista de etiquetas.
Comentario interno	Aclara la lógica de actualización del conteo.
`return frecuencias`	Devuelve el resultado final del procesamiento.

7.4 Introducción al control de versiones

El control de versiones es una práctica esencial en el desarrollo de software moderno. Permite registrar cambios, recuperar versiones anteriores y trabajar de manera colaborativa. En proyectos de Inteligencia Artificial, donde los modelos y el código evolucionan constantemente, el uso de herramientas como Git resulta indispensable para garantizar trazabilidad y reproducibilidad (Chacon & Straub, 2014).

7.4.1 Ejemplo 4: Flujo básico de trabajo con Git—conceptual

Este ejemplo describe, de forma conceptual, un flujo básico de control de versiones que acompaña el desarrollo de código en IA.

```
git init
git add main.py
git commit -m "Primer versión del algoritmo de análisis"
```

Captura 30: Ejemplo de comandos básicos de Git para inicializar un repositorio y registrar la primera versión de un algoritmo, ilustrando el uso del control de versiones en el desarrollo de software. Autoría: elaboración propia.

Comando	Explicación
`git init`	Inicializa un repositorio de control de versiones.
`git add main.py`	Marca el archivo para ser incluido en el próximo registro.
`git commit -m "..."`	Guarda un punto de control con un mensaje descriptivo.

7.5 Integración de buenas prácticas en proyectos de IA

La combinación de código legible, documentación adecuada y control de versiones permite desarrollar proyectos de Inteligencia Artificial claros, reproducibles y escalables. Estas prácticas no solo facilitan el aprendizaje y la evaluación académica, sino que también los/las preparan para contextos profesionales y de investigación, donde la calidad del software es tan relevante como la calidad de los modelos.

Las buenas prácticas de codificación, la documentación mediante docstrings y el uso sistemático del control de versiones constituyen pilares fundamentales en la formación en Inteligencia Artificial. Su adopción temprana favorece la comprensión de algoritmos, el trabajo colaborativo y la construcción de soluciones confiables, alineadas con los estándares actuales del desarrollo de software.

Actividad

Tras haber abordado principios de legibilidad, documentación y control de versiones, se está en condiciones de integrar estas buenas prácticas en el desarrollo de código propio. La siguiente actividad propone reflexionar sobre su importancia en contextos profesionales y, particularmente, en proyectos de Inteligencia Artificial. **Actividad 9: Buenas prácticas y control de versiones.**

Bibliografía consultada por el docente para el desarrollo del módulo

- Bel, W. (2020). *Algoritmos y estructuras de datos en Python: Un enfoque ágil y estructurado*. Editorial UADE. https://tinyurl.com/mu2zezrs
- Chacon, S., & Straub, B. (2014). *Pro-Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press. https://tinyurl.com/5dn77jep

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. https://tinyurl.com/3jdkaxmk
- Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media. https://tinyurl.com/9a8p53ew
- Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill. https://tinyurl.com/yc3xjwa8
- Python Software Foundation. (2024). *PEP 8 – Style guide for Python code*. https://peps.python.org/pep-0008/
- Python Software Foundation. (2024). *Tipos de datos* (Documentación de Python 3.12 en español). https://tinyurl.com/5yuuksvw
- Python Software Foundation. (2024). *collections — Container datatypes* (Documentación de Python 3.12 en español). https://tinyurl.com/48hvu9bv
- Python Software Foundation. (2025). *Tipos integrados: dict, set, …* (Documentación de Python 3.13 en español). https://tinyurl.com/yc57pbua
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. https://algs4.cs.princeton.edu/home/

Módulo 1 | Glosario

- **Algoritmo:** Conjunto finito y ordenado de pasos que permiten resolver un problema o realizar una tarea. En el contexto de la Inteligencia Artificial, los algoritmos constituyen la base para el procesamiento de datos, la toma de decisiones y la implementación de modelos.

- **Análisis de eficiencia:** Evaluación del comportamiento de un algoritmo en términos de tiempo de ejecución y uso de memoria. Permite comparar distintas estrategias de resolución y seleccionar aquellas más adecuadas para un problema dado.

- **Big-O (notación):** Notación utilizada para describir el crecimiento del tiempo o del espacio requerido por un algoritmo en función del tamaño de la entrada. En el módulo se introduce de manera intuitiva a partir de mediciones empíricas.

- **Buenas prácticas de codificación:** Conjunto de principios orientados a producir código legible, mantenible, reutilizable y confiable. Incluyen el uso de nombres descriptivos, comentarios claros, documentación adecuada y respeto por convenciones del lenguaje.

- **Clase:** Estructura que define un tipo de objeto en programación orientada a objetos, especificando atributos y métodos. En Inteligencia Artificial se utiliza para modelar entidades del dominio, como instancias de datos o componentes del sistema.

- **Código principal:** Bloque del programa que se ejecuta cuando el archivo se corre directamente. Su función es coordinar el flujo general del programa, invocar funciones y mostrar resultados.

- **Complejidad espacial:** Medida del uso de memoria adicional requerido por un algoritmo en función del tamaño de la entrada. Es un aspecto crítico en sistemas de IA que operan con grandes volúmenes de datos.

- **Complejidad temporal:** Medida de cómo crece el tiempo de ejecución de un algoritmo a medida que aumenta el tamaño de los datos de entrada. Permite analizar la eficiencia de distintas soluciones.

- **Conjunto (set):** Estructura de datos no ordenada que almacena elementos únicos. Se utiliza para eliminar duplicados, comprobar pertenencia y realizar operaciones como unión e intersección, frecuentes en el procesamiento de datos.

- **Contenedores nativos:** Estructuras de datos integradas en Python, como listas, tuplas, diccionarios y conjuntos, que permiten representar y organizar información de manera flexible para su posterior procesamiento algorítmico.

- **Control de versiones:** Práctica que permite registrar cambios en el código a lo largo del tiempo, recuperar versiones anteriores y facilitar el trabajo colaborativo. Herramientas como Git son fundamentales en proyectos de Inteligencia Artificial.

- **CSV (Comma-Separated Values):** Formato de archivo utilizado para representar datos tabulares simples, donde cada fila corresponde a una instancia y cada columna a un atributo. Es comúnmente empleado para el intercambio de datasets.

- **DataFrame:** Estructura tabular de alto nivel provista por la biblioteca pandas, que permite trabajar con datos organizados en filas y columnas etiquetadas. Es ampliamente utilizada en ciencia de datos e Inteligencia Artificial.

- **Dataset:** Conjunto de datos estructurados que sirven como entrada para algoritmos de análisis o modelos de Inteligencia Artificial. Puede representarse mediante contenedores nativos o estructuras tabulares.

- **Diccionario (dict):** Estructura de datos que almacena pares clave–valor, permitiendo acceso rápido a la información mediante una clave. Es especialmente útil para representar instancias con atributos nombrados y para conteo de frecuencias.

- **Diseño modular:** Principio de organización del software que divide un programa en módulos independientes, cada uno con responsabilidades claras. Facilita la reutilización, el mantenimiento y la extensibilidad del código.

- **Docstring:** Cadena de documentación asociada a funciones, clases o módulos en Python, utilizada para describir su finalidad, parámetros y valores de retorno. Contribuye a la claridad y a la generación automática de documentación.

- **Encapsulamiento:** Principio de la programación orientada a objetos que agrupa datos y operaciones relacionadas dentro de una clase, ocultando los detalles internos y exponiendo una interfaz clara.

- **Estructura de control:** Construcción del lenguaje que permite alterar el flujo de ejecución de un programa, como condicionales (if) y bucles (for, while). Son esenciales para expresar decisiones y repeticiones algorítmicas.

- **Estructura de datos:** Forma de organizar y almacenar datos en memoria para facilitar su acceso y procesamiento. La elección de la estructura adecuada influye directamente en la eficiencia del algoritmo.

- **Función:** Bloque de código que encapsula una tarea específica y puede reutilizarse. Favorece la modularidad, la legibilidad y la prueba independiente de los algoritmos.

- **Herencia:** Mecanismo de la programación orientada a objetos que permite definir nuevas clases a partir de clases existentes, reutilizando y extendiendo su comportamiento.

- **Inteligencia artificial:** Campo de la informática dedicado al desarrollo de sistemas capaces de realizar tareas que requieren razonamiento, aprendizaje o toma de decisiones, apoyándose en algoritmos, estructuras de datos y modelos.

- **JSON (JavaScript Object Notation):** Formato de datos estructurado y jerárquico,

ampliamente utilizado para el intercambio de información entre sistemas. En Python se representa mediante diccionarios y listas.

- **Lista (list):** Estructura de datos ordenada y mutable que permite almacenar colecciones de elementos. Se utiliza para representar secuencias de datos, conjuntos de instancias y vectores de características.

- **Módulo:** Archivo o biblioteca que agrupa funciones, clases y variables relacionadas. Su uso permite reutilizar código y mantener los programas organizados.

- **Online IDE Pro:** Entorno de desarrollo en línea utilizado en la asignatura para escribir, ejecutar y analizar código Python sin necesidad de instalaciones locales, favoreciendo la experimentación algorítmica.

- **Pensamiento algorítmico:** Capacidad de formular soluciones a problemas mediante pasos ordenados y lógicos, independientemente del lenguaje de programación utilizado. Constituye un eje central en la formación en Inteligencia Artificial.

- **Preprocesamiento de datos:** Conjunto de operaciones destinadas a limpiar, transformar y seleccionar datos antes de aplicar algoritmos o modelos de IA. Incluye filtrado, selección de atributos y reducción de ruido.

- **Programación orientada a objetos (POO):** Paradigma de programación que organiza el software en torno a objetos que combinan datos y comportamientos. Facilita el modelado de entidades y la construcción de sistemas complejos.

- **Python:** Lenguaje de programación interpretado, de alto nivel y multiparadigma, adoptado como lenguaje base en la asignatura por su claridad sintáctica y su amplia adopción en Inteligencia Artificial y ciencia de datos.

- **Tupla (tuple):** Estructura de datos ordenada e inmutable, utilizada para representar registros o conjuntos de valores que no deben modificarse durante la ejecución del programa.

Módulo 1 | Actividades

ACTIVIDADES

Consigna

Las actividades de este módulo son interactivas y autoasistidas. Para realizarlas, es necesario acceder al enlace ***disponible en plataforma*** y seguir las instrucciones indicadas. Ante cualquier duda o inconveniente durante el desarrollo, se solicita comunicarse con el/la tutor/a para recibir acompañamiento y orientación.

Módulo 2

Microobjetivos

- Analizar estructuras de datos lineales y no lineales para comprender su comportamiento y propósito en la resolución de problemas algorítmicos en Python.
- Implementar listas, pilas, colas y colas de prioridad para modelar distintos patrones de acceso y procesamiento en algoritmos secuenciales, recursivos y heurísticos.
- Construir árboles y grafos junto con sus recorridos fundamentales con el fin de interpretar relaciones jerárquicas y de conectividad en estructuras usadas en Inteligencia Artificial.
- Comparar estrategias algorítmicas mediante BFS, DFS y estructuras de soporte para seleccionar el enfoque más adecuado según el tipo de problema y los requisitos de exploración.
- Evaluar la complejidad temporal y espacial de algoritmos para fundamentar decisiones de diseño eficientes en contextos de procesamiento de datos y optimización computacional

Contenidos

ESTRUCTURAS DE DATOS LINEALES Y ANÁLISIS DE COMPLEJIDAD

Fuente: Freepik

1. Introducción

En este módulo se profundiza el estudio de las estructuras de datos lineales y jerárquicas, junto con el análisis de la eficiencia temporal y espacial de los algoritmos que operan sobre ellas. A partir de los fundamentos algorítmicos trabajados en el módulo anterior, se avanza hacia estructuras más complejas que permiten modelar, organizar y procesar datos de manera eficiente, aspecto central en el desarrollo de sistemas de Inteligencia Artificial.

El abordaje se realiza desde el concepto de ***tipo de dato abstracto***, destacando la separación entre la definición conceptual de una estructura y su implementación concreta en Python. Se analizan arrays, listas dinámicas, pilas, colas y colas de prioridad, así como árboles binarios y n-arios, enfatizando sus operaciones básicas, sus recorridos y su relación con la recursividad.

Asimismo, se introduce de manera sistemática la notación Big-O como herramienta para describir el comportamiento de los algoritmos frente al crecimiento del tamaño de los datos. Este análisis se complementa con mediciones empíricas de rendimiento, utilizando herramientas como time y timeit, con el objetivo de desarrollar una comprensión crítica sobre la eficiencia en contextos reales.

Énfasis

El foco del módulo está puesto en comprender cómo la elección de una estructura de datos y de un algoritmo impacta directamente en el rendimiento, la escalabilidad y la viabilidad de soluciones propias de la Inteligencia Artificial, más allá de la mera implementación sintáctica.

2. Tipos de datos abstractos (TDA) y su implementación en Python

En el desarrollo de algoritmos y estructuras de datos se debe diferenciar entre qué problema se desea resolver y cómo se implementa la solución en un lenguaje de programación concreto. En este contexto, el concepto de *tipo de dato abstracto (TDA)* constituye una herramienta clave para el diseño de soluciones eficientes, claras y reutilizables, especialmente en el ámbito de la Inteligencia Artificial.

2.1 ¿Qué es un tipo de dato abstracto?

Un tipo de dato abstracto es una especificación lógica de una estructura de datos que describe:

- un conjunto de valores posibles,
- un conjunto de operaciones permitidas sobre esos valores,
- y las reglas que rigen dichas operaciones.

Desde esta perspectiva, el énfasis no está en cómo se almacenan los datos en memoria, sino en qué operaciones son posibles y con qué efecto.

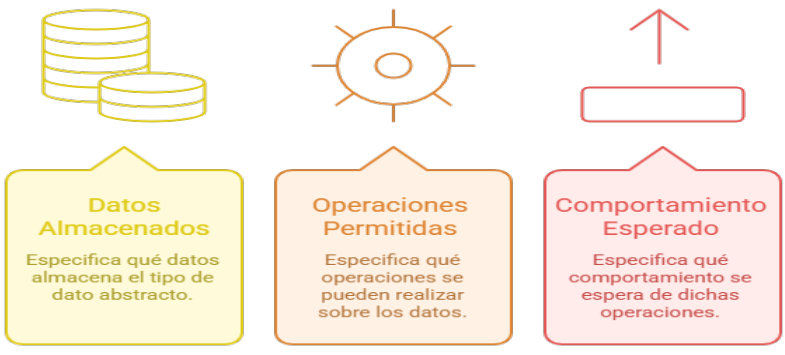


Ilustración 1: Componentes de un tipo de dato abstracto (datos almacenados, operaciones permitidas y comportamiento esperado). Imagen creada en Napkin con información proporcionada por la autora.

Esta forma de pensar resulta especialmente relevante en Inteligencia Artificial, donde un mismo concepto (por ejemplo, una cola, un conjunto de nodos o una frontera de búsqueda) puede implementarse de distintas maneras según los requisitos de eficiencia.

Énfasis

Los tipos de datos abstractos permiten razonar sobre algoritmos y estructuras de datos a un nivel conceptual, independiente de su implementación concreta, facilitando el análisis, la comparación y la reutilización de soluciones.

2.2 Relación entre TDA y Python

Python no implementa tipos de datos abstractos de manera explícita como otras formulaciones teóricas, pero **provee estructuras y mecanismos que permiten modelarlos de forma natural**.

Listas, diccionarios, conjuntos, deque y clases definidas por el usuario pueden utilizarse para representar distintos TDA, siempre que se respeten las operaciones y comportamientos definidos conceptualmente.

De este modo, Python se convierte en una herramienta adecuada para:

- implementar TDA de forma flexible,
- experimentar con distintas estrategias de implementación,
- y analizar el impacto de dichas decisiones en la eficiencia del algoritmo.

2.2.1 Ejemplo conceptual: Lista como TDA

Desde el punto de vista abstracto, una lista puede definirse como un TDA que:

- almacena una colección ordenada de elementos,
- permite acceder a un elemento por su posición,
- admite operaciones como inserción, eliminación y recorrido.

La implementación concreta de este TDA en Python se realiza mediante el tipo list, que proporciona estas operaciones de manera directa.

ejemplo1.py

```python
# Representación de una lista como TDA
datos = [3, 7, 2, 9, 4]

# Operaciones básicas
datos.append(6)      # inserción
valor = datos[0]     # acceso
datos.remove(2)      # eliminación

print("Lista:", datos)
print("Primer elemento:", valor)
```

Press Ctrl K to generate code

Captura 1: Ejemplo de representación de una lista como tipo de dato abstracto (TDA) y operaciones básicas en Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Este ejemplo muestra cómo un TDA definido conceptualmente puede implementarse utilizando estructuras nativas del lenguaje.

Elemento del código	Explicación
`datos = [...]`	Representa la colección de elementos del TDA lista.
`append()`	Implementa la operación de inserción definida por el TDA.
Acceso por índice	Permite recuperar un elemento según su posición.
`remove()`	Elimina un elemento de la colección.
`print()`	Permite observar el estado final de la estructura.

2.3 TDA y diseño algorítmico

Pensar en términos de tipos de datos abstractos permite:

- seleccionar estructuras adecuadas según el problema,
- anticipar costos de tiempo y espacio de las operaciones,
- cambiar la implementación sin modificar la lógica general del algoritmo.

En Inteligencia Artificial, esta forma de diseño resulta esencial para construir algoritmos de búsqueda, estructuras de control, modelos de datos y sistemas escalables.

Énfasis

El diseño algorítmico basado en tipos de datos abstractos favorece soluciones más robustas, comprensibles y adaptables, cualidades indispensables en el desarrollo de sistemas de Inteligencia Artificial.

Actividad

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 1: Tipos de datos abstractos (TDA)**

3. Arrays y listas dinámicas

3.1 Operaciones, comprensión de listas y manejo de memoria

En el desarrollo de algoritmos y sistemas de Inteligencia Artificial, una de las tareas más frecuentes consiste en almacenar, recorrer, transformar y analizar colecciones de datos. Estas colecciones pueden representar desde listas de valores numéricos, hasta conjuntos de características, secuencias temporales, etiquetas de clases o resultados intermedios de un modelo.

Énfasis

En este contexto, las listas dinámicas constituyen una de las estructuras de datos más utilizadas en Python, ya que permiten almacenar colecciones ordenadas de elementos, acceder a ellos de forma indexada y modificarlos dinámicamente durante la ejecución del programa.

Si bien en términos conceptuales pueden asociarse a la idea de *arrays*, las listas de Python incorporan características adicionales que las hacen especialmente flexibles para el prototipado y desarrollo de algoritmos de IA. Comprender el comportamiento de las listas dinámicas y sus operaciones básicas es fundamental para diseñar algoritmos eficientes, evitar errores comunes de rendimiento y seleccionar estructuras adecuadas según el problema a resolver.

3.2 Arrays y listas dinámicas en Python

Énfasis

En Python, las listas (list) son estructuras de datos que permiten almacenar múltiples elementos en una sola variable, manteniendo un orden definido y permitiendo la repetición de valores.

Ilustración 2: Representación conceptual de una lista en Python con orden definido, indexación y posibilidad de valores heterogéneos y repetidos. Imagen creada en Nano Banana. Autoría: elaboración propia.

A diferencia de los arrays estáticos de otros lenguajes, las listas en Python son dinámicas, es decir, pueden crecer o reducirse en tiempo de ejecución. Desde el punto de vista algorítmico, una lista puede interpretarse como una secuencia de posiciones contiguas que permiten acceso directo a cada elemento mediante un índice.

Énfasis

Aunque Python abstrae los detalles de implementación, las listas se apoyan internamente en estructuras similares a arrays dinámicos, lo que tiene implicancias directas en el costo de operaciones como inserciones, eliminaciones y recorridos.

3.3 Operaciones básicas sobre listas

Ilustración 3: Operaciones fundamentales sobre listas en Python: acceso por índice, inserción y eliminación de elementos, recorrido secuencial y cálculo de agregados. Imagen creada en Napkin, a partir de información proporcionada por la autora.

Estas operaciones aparecen de manera recurrente en algoritmos de búsqueda, preprocesamiento de datos y cálculo de métricas en IA.

3.3.1 Ejemplo 1: Acceso y recorrido de una lista

El recorrido secuencial de una lista es una operación básica en la mayoría de los algoritmos de procesamiento de datos. Permite aplicar una acción a cada elemento y constituye la base de múltiples algoritmos lineales.

```
1   datos = [3, 7, 2, 9, 4]
2
3   for valor in datos:
4       print("Valor:", valor)
5   Press Ctrl K to generate code
```

Captura 2: Ejemplo de recorrido secuencial de una lista en Python mediante un bucle for, mostrando cada elemento almacenado. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento del código	Explicación
`datos = [...]`	Se define una lista de valores numéricos.
`for valor in datos`	Se recorre la lista de manera secuencial.
`print(...)`	Se muestra cada elemento individualmente.

Énfasis

El recorrido secuencial tiene un costo proporcional al tamaño de la lista, lo que más adelante se formalizará como complejidad temporal lineal.

3.4 Comprensión de listas (list comprehensions)

La comprensión de listas es una construcción sintáctica que permite generar nuevas listas a partir de listas existentes, de forma compacta y expresiva.

Ilustración 4: Representación conceptual de la comprensión de listas en Python como mecanismo de transformación compacta y expresiva de datos. Imagen creada en Nano Banana con información proporcionada por la autora.

Esta herramienta es ampliamente utilizada en IA para tareas de filtrado, transformación y extracción de características debido a que la comprensión de listas favorece la claridad del código y reduce errores, siempre que se utilice de manera responsable y legible.

3.4.1 Ejemplo 2: Transformación de datos con comprensión de listas

La transformación de datos es una etapa central del preprocesamiento en IA. La comprensión de listas permite expresar estas transformaciones de forma directa y declarativa.

```
1   edades = [22, 45, 36, 28]
2   edades_dobladas = [e * 2 for e in edades]
3
4   print(edades dobladas)
```

Captura 3: Ejemplo de comprensión de listas en Python para transformar datos de entrada. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento del código	Explicación
`edades = [...]`	Lista original de datos.
`[e * 2 for e in edades]`	Se genera una nueva lista aplicando una transformación a cada elemento.
`edades_dobladas`	Lista resultante de la transformación.

Énfasis

Aunque más concisas, las comprensiones de listas siguen recorriendo todos los elementos, por lo que su costo computacional es equivalente al de un ciclo explícito.

3.5 Inserción y eliminación de elementos

Las listas dinámicas permiten agregar y eliminar elementos durante la ejecución. Sin embargo, estas operaciones no siempre tienen el mismo costo computacional.

3.5.1 Ejemplo 3: Inserción y eliminación

En algoritmos de IA, es común construir listas de manera incremental o depurarlas eliminando valores irrelevantes.

```
1   valores = [10, 20, 30]
2
3   valores.append(40)
4   valores.remove(20)
5
6   print(valores)
7
```

Captura 4: Ejemplo de inserción y eliminación de elementos en una lista de Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento del código	Explicación
`append(40)`	Agrega un elemento al final de la lista.
`remove(20)`	Elimina la primera aparición del valor indicado.
`print(valores)`	Muestra la lista actualizada.

Énfasis

Insertar o eliminar elementos en posiciones intermedias puede implicar el desplazamiento de múltiples elementos, lo que impacta en el rendimiento.

3.6 Manejo de memoria y listas dinámicas

Aunque Python gestiona automáticamente la memoria, el uso intensivo de listas grandes puede tener un impacto significativo en el consumo de recursos, especialmente en aplicaciones de IA que trabajan con grandes volúmenes de datos.

3.6.1 Ejemplo 4: Tamaño de una lista en memoria

Comprender el impacto del tamaño de las estructuras de datos permite tomar decisiones más eficientes en el diseño de algoritmos.

```
1  import sys
2
3  lista_pequena = list(range(10))
4  lista_grande = list(range(100000))
5
6  print("Tamaño lista pequeña (bytes):", sys.getsizeof(lista_pequena))
7  print("Tamaño lista grande (bytes):", sys.getsizeof(lista_grande))
8  Press Ctrl K to generate code
```

Captura 8: Ejemplo de medición del consumo de memoria de listas en Python mediante sys.getsizeof. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento del código	Explicación
`sys.getsizeof`	Permite obtener una estimación del tamaño en memoria de la lista.
`lista_pequena`	Lista con pocos elementos.
`lista_grande`	Lista con muchos elementos.

Énfasis

El crecimiento dinámico de las listas implica realocaciones internas de memoria, lo que refuerza la importancia de elegir estructuras adecuadas según el problema.

Las listas dinámicas constituyen una herramienta central en Python para la implementación de algoritmos de Inteligencia Artificial. Su flexibilidad las hace ideales para el prototipado y el procesamiento de datos, pero su uso debe estar acompañado por una comprensión clara de sus costos computacionales y de memoria.

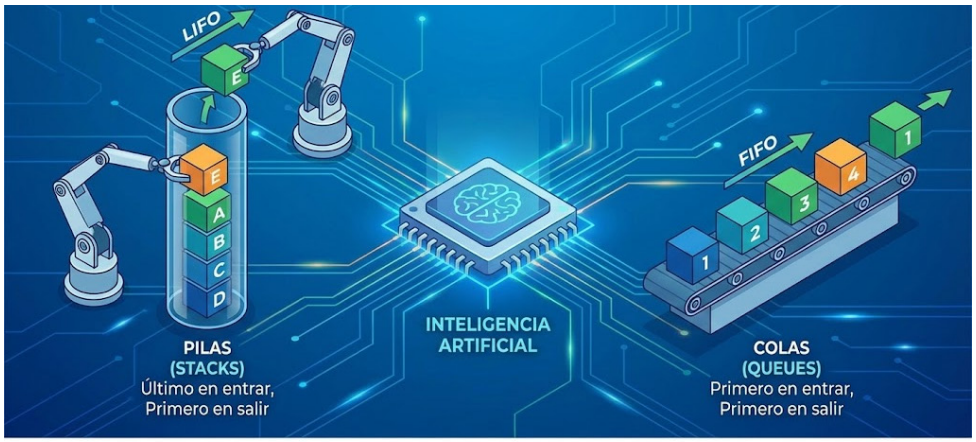
Actividad

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 2: Arrays y listas dinámicas.**

4. Pilas y colas en Python

4.1 Implementación con listas y collections.deque

Las pilas y las colas son estructuras de datos lineales fundamentales en el diseño de algoritmos y en la construcción de sistemas de Inteligencia Artificial.



Las pilas y las colas son estructuras de datos lineales fundamentales en el diseño de algoritmos y en la construcción de sistemas de Inteligencia Artificial.

Ilustración 5: Representación conceptual del funcionamiento de pilas (LIFO) y colas (FIFO) como estructuras de datos lineales aplicadas al diseño de algoritmos en Inteligencia Artificial. Imagen creada en Nano Banana Pro con información proporcionada por la autora.

Se utilizan para modelar procesos secuenciales, gestionar estados intermedios, recorrer estructuras recursivas y coordinar flujos de ejecución.

Énfasis

Comprender el comportamiento LIFO y FIFO de pilas y colas resulta fundamental para modelar procesos computacionales frecuentes en Inteligencia Artificial, tales como la gestión de estados, la exploración sistemática de soluciones y la coordinación eficiente de tareas secuenciales.

En Python, estas estructuras pueden implementarse de manera eficiente mediante **listas** y mediante el módulo **collections.deque**, que ofrece ventajas específicas en términos de rendimiento.

En este tema se estudian los principios conceptuales de ambas estructuras, su implementación práctica y su relación con algoritmos secuenciales y recursivos, con especial atención a su impacto en la eficiencia temporal.

4.2 Pilas (Stack)

Una pila es una estructura de datos que sigue el principio LIFO (Last In, First Out): el último elemento que se inserta es el primero en ser removido.

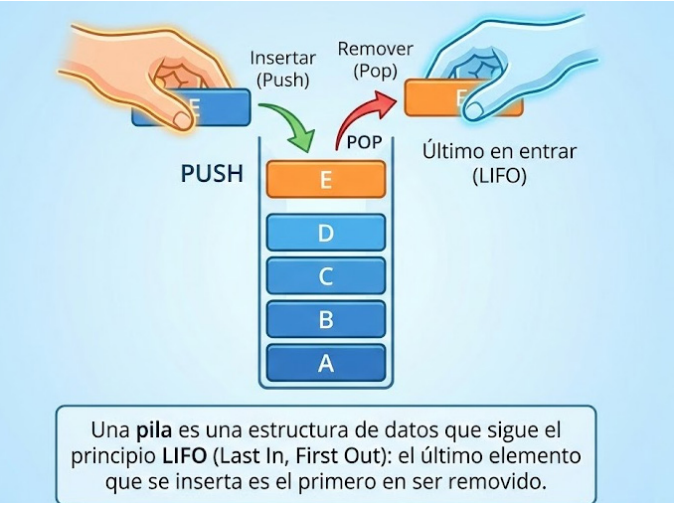


Ilustración 6: Esquema del funcionamiento de una pila (stack) bajo el principio LIFO (Last In, First Out), mostrando las operaciones de inserción (push) y eliminación (pop). Imagen creada en Nano Banana con información proporcionada por la autora.

Este comportamiento es central en numerosos procesos computacionales, como la gestión de llamadas a funciones, el retroceso (backtracking) y la evaluación de expresiones. En Python, una pila puede implementarse de forma sencilla utilizando una **lista**, aprovechando las operaciones *append ()* y *pop ()*.

Énfasis

Las pilas constituyen una abstracción directa del funcionamiento de la recursividad y del manejo de llamadas a funciones, por lo que su comprensión es clave para interpretar el comportamiento interno de algoritmos recursivos y procedimientos de búsqueda en profundidad utilizados en Inteligencia Artificial.

4.2.1 Ejemplo 1: Implementación de una pila con listas

Como se planteó anteriormente, en Python una pila puede implementarse de manera sencilla utilizando una lista, aprovechando las operaciones *append()* para insertar elementos y *pop()* para eliminarlos desde el extremo final de la estructura.

```
1   pila = []
2
3   # Apilar elementos
4   pila.append(10)
5   pila.append(20)
6   pila.append(30)
7
8   # Desapilar elementos
9   print("Elemento extraído:", pila.pop())
10  print("Pila actual:", pila)
11
```

Captura 6: Ejemplo de implementación de una pila (LIFO) en Python utilizando listas y la operación pop. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Línea / Elemento	Explicación
`pila = []`	Se crea una lista vacía que actuará como estructura de pila.
`append(10)`, `append(20)`, `append(30)`	Cada llamada agrega un elemento al final de la lista, representando la operación de apilar.
`pop()`	Elimina y devuelve el último elemento agregado, respetando el principio LIFO.
`print()`	Permite observar el valor extraído y el estado actual de la pila.

Más allá de la implementación concreta, resulta central identificar que las operaciones *append()* y *pop()* modelan de manera directa el principio LIFO, permitiendo expresar algoritmos de forma clara y alineada con el razonamiento lógico subyacente.

4.3 Colas (Queue)

Una cola es una estructura de datos que sigue el principio FIFO (First In, First Out), en el cual el primer elemento en ingresar es el primero en ser retirado.

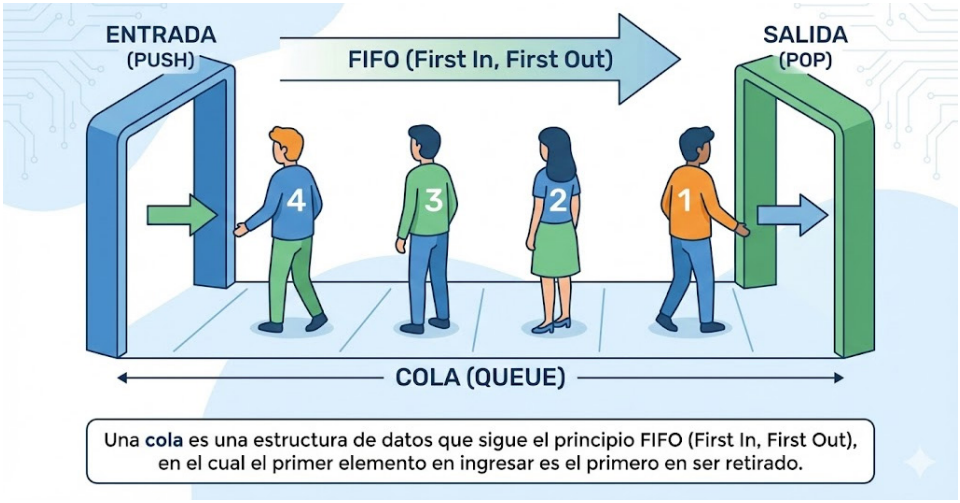


Ilustración 7: Representación del funcionamiento de una cola (queue) bajo el principio FIFO (First In, First Out), mostrando las operaciones de entrada (push) y salida (pop). Imagen creada en Nano Banana con información proporcionada por la autora.

Este comportamiento resulta adecuado para modelar procesos de atención secuencial, planificación de tareas y simulación de eventos.

Énfasis

Las colas permiten representar procesos donde el orden temporal de llegada es determinante, lo cual es habitual en simulaciones, recorridos por niveles y sistemas de planificación utilizados en Inteligencia Artificial.

4.4 Colas implementadas con listas: limitaciones

Una cola puede implementarse utilizando una lista de Python mediante la operación *append ()* para insertar elementos y *pop(0)* para eliminar el primer elemento. Sin embargo, esta implementación presenta limitaciones, ya que eliminar el primer elemento implica desplazar todos los elementos restantes, afectando negativamente la eficiencia temporal.

4.4.1 Ejemplo 2: Implementación de una cola utilizando listas

```
1   cola = []
2
3   cola.append("A")
4   cola.append("B")
5   cola.append("C")
6
7   print("Elemento atendido:", cola.pop(0))
8   print("Cola actual:", cola)
9
```

Captura 7–Ejemplo de implementación de una cola (FIFO) en Python utilizando listas y la operación pop(0). Código desarrollado en Online IDE Pro. Código fuente de la autora.

Línea / Elemento	Explicación
`cola = []`	Se inicializa una lista vacía para representar la cola.
`append("A")`, `append("B")`, `append("C")`	Los elementos se agregan al final de la lista, simulando el ingreso a la cola.
`pop(0)`	Elimina el primer elemento de la lista, representando la salida según el criterio FIFO.
`print()`	Muestra el elemento atendido y el contenido restante de la cola.

Identificar las limitaciones de una implementación es un paso clave en el pensamiento algorítmico, ya que permite anticipar problemas de eficiencia y justificar la elección de estructuras de datos más adecuadas según el contexto del problema.

4.5 Colas eficientes con collections. deque

El módulo collections de Python provee la estructura deque (double-ended queue), diseñada para permitir inserciones y eliminaciones eficientes en ambos extremos de la estructura. Esta característica la convierte en la opción recomendada para implementar colas y pilas cuando se requiere un mejor rendimiento.

El uso de collections.deque introduce tempranamente al análisis de eficiencia temporal, promoviendo decisiones de diseño informadas que impactan directamente en la escalabilidad de los algoritmos desarrollados.

4.5.1 Ejemplo 3: Implementación de una cola eficiente con collections.deque

```python
from collections import deque

cola = deque()

cola.append("A")
cola.append("B")
cola.append("C")

print("Elemento atendido:", cola.popleft())
print("Cola actual:", cola)
```

Captura 8: Ejemplo de implementación de una cola (FIFO) en Python utilizando collections.deque, con extracción eficiente mediante popleft(). Código desarrollado en Online IDE Pro. Código fuente de la autora.

Línea / Elemento	Explicación
`from collections import deque`	Se importa la estructura `deque`, diseñada para operaciones eficientes en ambos extremos.
`cola = deque()`	Se crea una cola doblemente enlazada.
`append()`	Inserta elementos al final de la cola.
`popleft()`	Elimina y retorna el primer elemento en tiempo constante.
`print()`	Permite observar el comportamiento FIFO y el estado actual de la cola.

Este ejemplo pone de manifiesto cómo la elección de la estructura de datos adecuada puede mejorar el rendimiento sin modificar la lógica del algoritmo, principio esencial en el desarrollo de soluciones de Inteligencia Artificial eficientes.

4.6 Relación con algoritmos recursivos y secuenciales

Las pilas y colas se relacionan de forma directa con distintos enfoques algorítmicos. Las pilas modelan naturalmente la recursividad y el retroceso, mientras que las colas se emplean en recorridos secuenciales, simulaciones y algoritmos de búsqueda por niveles.

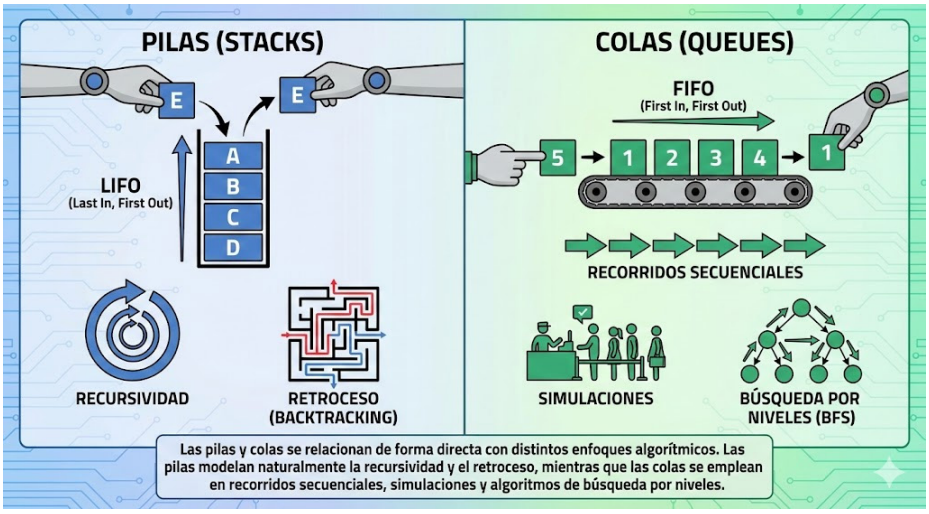


Ilustración 8: Comparación conceptual entre pilas (stacks) y colas (queues), destacando los principios LIFO y FIFO y su relación con la recursividad, el backtracking, los recorridos secuenciales y la búsqueda por niveles (BFS). Imagen creada en Nano Banana con información proporcionada por la autora.

Énfasis

La estructura deque permite combinar ambos comportamientos según las necesidades del algoritmo.

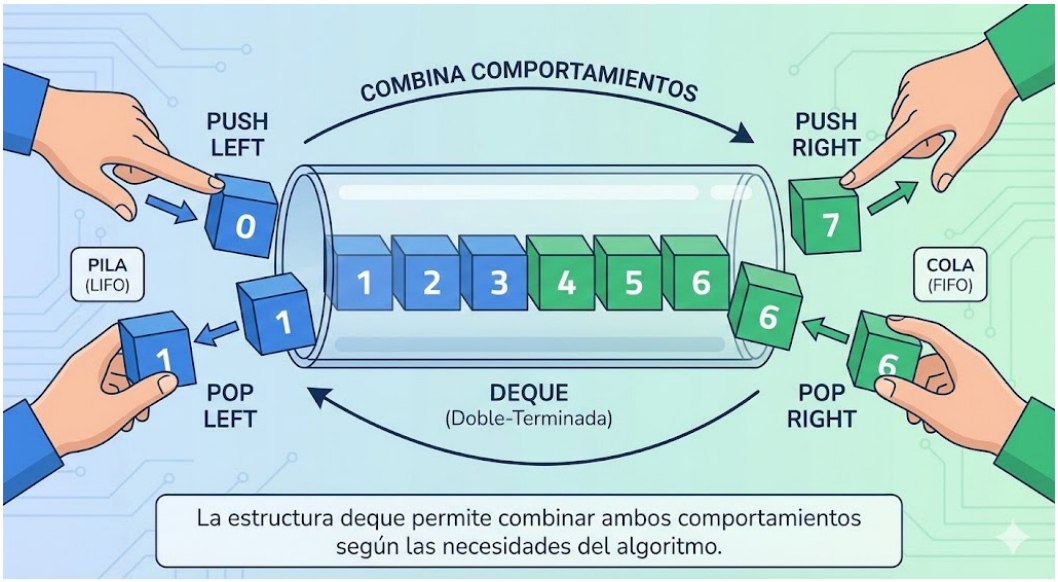


Ilustración 9: Representación conceptual de la estructura deque (doble terminación), que combina comportamientos de pila (LIFO) y cola (FIFO) mediante operaciones de inserción y extracción por ambos extremos. Imagen creada en Nano Banana con información proporcionada por la autora.

Énfasis

La correcta articulación entre pilas, colas y algoritmos recursivos o secuenciales constituye una base estructural para la implementación de búsquedas, recorridos y estrategias de exploración que serán reutilizadas en modelos y sistemas inteligentes más complejos.

Actividad

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 3: Pilas y Colas (LIFO y FIFO).**

5. Colas de prioridad y heaps

5.1 Gestión eficiente de prioridades en algoritmos de Inteligencia Artificial

En numerosos algoritmos y sistemas de Inteligencia Artificial no resulta suficiente procesar los elementos en el orden en que llegan. En su lugar, es necesario priorizar ciertos elementos por sobre otros, atendiendo criterios como costo, distancia, probabilidad o relevancia. Para estos casos, las colas de prioridad constituyen una estructura de datos fundamental.

Énfasis

Una cola de prioridad es una estructura en la que cada elemento posee una prioridad asociada, y el elemento con mayor (o menor) prioridad es el primero en ser atendido.

En Python, estas colas se implementan de manera eficiente mediante la estructura *heap*, disponible a través del módulo estándar heapq. Las colas de prioridad permiten modelar decisiones informadas y eficientes en algoritmos de IA, siendo una pieza clave en búsquedas heurísticas, planificación y optimización.

Ilustración 10: Uso de colas de prioridad en Inteligencia Artificial para la toma de decisiones informadas, aplicadas en búsqueda heurística, planificación y optimización. Imagen creada en Nano Banana con información proporcionada por la autora.

5.2 Colas de prioridad: concepto

Desde el punto de vista abstracto, una cola de prioridad soporta, como mínimo, las siguientes operaciones:

› inserción de elementos con prioridad,

Ilustración 11: Inserción y gestión de elementos en una cola de prioridad, donde las tareas se ordenan según su nivel de prioridad para garantizar decisiones eficientes. Imagen creada en Nano Banana con información proporcionada por la autora.

Esta imagen ilustra la operación de inserción. Un elemento con una alta prioridad (una tarea urgente) es introducido en la cola. La imagen muestra cómo su posición no está determinada por su orden de llegada, sino por su valor de prioridad, ubicándose por delante de otros elementos con menor prioridad que ya estaban en la cola.

› extracción del elemento con mayor o menor prioridad,

Ilustración 12: Operaciones de consulta y extracción en una cola de prioridad, donde siempre se accede al elemento de mayor prioridad sin perder la organización interna. Imagen creada en Nano Banana con información proporcionada por la autora.

Esta imagen muestra las operaciones de extracción y consulta. Se puede observar un brazo robótico extrayendo el elemento con la mayor prioridad (el de prioridad 10), que es siempre el primero en ser atendido. Simultáneamente, una lupa ilustra la acción de consultar el siguiente elemento más prioritario (el de prioridad 5) sin necesidad de eliminarlo de la cola.

› consulta del elemento prioritario sin eliminarlo.

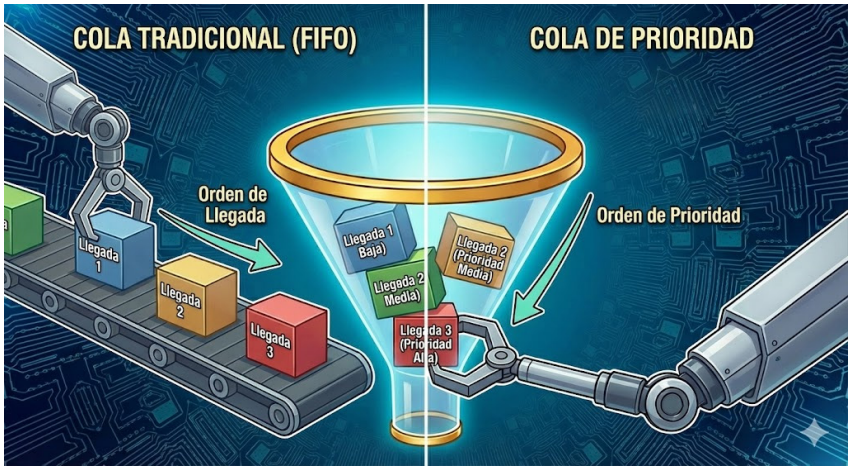


Ilustración 13: Consulta y extracción en una cola de prioridad, destacando el acceso al elemento con mayor prioridad sin alterar el resto de la estructura. Imagen creada en Nano Banana con información proporcionada por la autora.

Esta última imagen compara una cola de prioridad con una cola tradicional (FIFO). A la izquierda, una cinta transportadora muestra cómo los elementos se procesan en el estricto orden en que llegaron. A la derecha, la cola de prioridad demuestra que el elemento con mayor prioridad (aunque haya llegado último) es procesado primero, ilustrando la diferencia fundamental en el criterio de atención.

Énfasis

A diferencia de una cola tradicional (FIFO), el orden de atención no depende del orden de llegada, sino del valor de prioridad. Este tipo de estructura resulta indispensable cuando se requiere seleccionar sistemáticamente la mejor opción disponible en cada paso de un algoritmo.

Lectura complementaria

Para profundizar y ampliar la comprensión sobre el funcionamiento de las colas, se recomienda consultar el material complementario disponible en el enlace provisto. GeeksforGeeks. (2024, enero 24). *Queue Data Structure*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/queue-data-structure/

5.3 Heaps: estructura subyacente

Un *heap* es una estructura de datos basada en un árbol binario casi completo que satisface la propiedad de heap.

Lectura complementaria

Para profundizar y ampliar la comprensión sobre el funcionamiento de los *min-heaps*, se recomienda consultar el material complementario disponible en el enlace provisto. GeeksforGeeks. (2023, marzo 22). *Introduction to Min-Heap*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/introduction-to-min-heap-data-structure/

En el caso de Python, *heapq* implementa un *min-heap*, donde el elemento con menor valor se encuentra siempre en la raíz. Desde el punto de vista práctico, el heap se representa internamente como una lista, pero organizada de manera tal que garantiza eficiencia en las operaciones fundamentales.

Énfasis

El heap combina eficiencia y simplicidad, permitiendo implementar colas de prioridad sin necesidad de estructuras complejas adicionales.

5.4 Uso básico de heapq

5.4.1 Ejemplo 1: Inserción y extracción en una cola de prioridad

Este ejemplo muestra cómo insertar elementos en una cola de prioridad y cómo extraer el elemento con mayor prioridad (menor valor numérico).

```
import heapq

cola_prioridad = []

heapq.heappush(cola_prioridad, 5)
heapq.heappush(cola_prioridad, 2)
heapq.heappush(cola_prioridad, 8)
heapq.heappush(cola_prioridad, 1)

print("Elemento prioritario:", heapq.heappop(cola_prioridad))
print("Cola restante:", cola_prioridad)
```

Captura 9: Ejemplo de implementación de una cola de prioridad en Python utilizando el módulo heapq, donde se extrae el elemento de mayor prioridad según el criterio de mínimo. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Línea / Elemento	Explicación
`import heapq`	Se importa el módulo que permite trabajar con heaps.
`cola_prioridad = []`	Se crea una lista vacía que representará el heap.
`heappush()`	Inserta un elemento manteniendo la propiedad de heap.
`heappop()`	Extrae y retorna el elemento con menor valor.
`print()`	Permite observar el comportamiento de la cola de prioridad.

El heap garantiza que la extracción del elemento prioritario se realice de forma eficiente, independientemente del tamaño de la estructura.

5.5 Colas de prioridad con tuplas (prioridad, elemento)

En muchos problemas de IA, la prioridad no coincide directamente con el valor del elemento. En estos casos, se utilizan *tuplas*, donde el primer valor representa la prioridad y el segundo el dato asociado.

5.5.1 Ejemplo 2: Cola de prioridad con pares (prioridad, dato)

Este ejemplo ilustra cómo asociar una prioridad explícita a cada elemento almacenado en la cola.

```
1   import heapq
2
3   cola_prioridad = []
4
5   heapq.heappush(cola_prioridad, (2, "Nodo B"))
6   heapq.heappush(cola_prioridad, (1, "Nodo A"))
7   heapq.heappush(cola_prioridad, (3, "Nodo C"))
8
9   print("Elemento atendido:", heapq.heappop(cola_prioridad))
```

Captura 10: Ejemplo de cola de prioridad en Python utilizando heapq con tuplas (prioridad, elemento) para gestionar el orden de atención. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Línea / Elemento	Explicación
`(prioridad, dato)`	La prioridad se evalúa primero al ordenar el heap.
`heappush()`	Inserta el par manteniendo el orden por prioridad.
`heappop()`	Extrae el par con menor prioridad numérica.
Resultado	Se obtiene el elemento más prioritario según el criterio definido.

Énfasis

El uso de tuplas permite modelar situaciones reales de IA, donde los datos deben evaluarse en función de métricas o costos asociados.

5.6 Relación con algoritmos de Inteligencia Artificial

Las colas de prioridad son ampliamente utilizadas en algoritmos de búsqueda informada (por ejemplo, búsqueda voraz), planificación de tareas, selección de candidatos óptimos, algoritmos de optimización y ruteo. En todos estos casos, la estructura garantiza que en cada iteración se procese el elemento más prometedor disponible.

Énfasis

La correcta implementación de colas de prioridad impacta directamente en la eficiencia y calidad de los algoritmos de Inteligencia Artificial.

Las colas de prioridad y los heaps constituyen una estructura central para la implementación de algoritmos inteligentes que requieren tomar decisiones basadas en criterios dinámicos. El uso de heapq en Python permite acceder a estas estructuras de manera eficiente y clara, manteniendo una separación adecuada entre la lógica algorítmica y la implementación.

Este tema sienta las bases para el estudio de algoritmos de búsqueda más complejos y estructuras jerárquicas que se abordarán en los temas siguientes del módulo.

Lectura complementaria

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el material complementario disponible en el enlace provisto. GeeksforGeeks. (2023, noviembre 30). *DSA Tutorial*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/

Actividad

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 4: Colas de prioridad y heaps**

6. Árboles binarios y recorridos

6.1 Estructuras jerárquicas para la representación y exploración de datos en IA

En Inteligencia Artificial es frecuente trabajar con **estructuras jerárquicas** para representar decisiones, relaciones entre entidades y procesos de búsqueda. Los **árboles binarios** constituyen una estructura de datos fundamental para modelar este tipo de relaciones, permitiendo organizar la información de manera no lineal y recorrerla siguiendo distintos criterios. Un árbol binario está compuesto por **nodos**, donde cada nodo puede tener como máximo dos hijos: uno izquierdo y uno derecho.

6.1.1 El nodo y la regla de los dos hijos

Esta primera imagen ilustra la regla fundamental de un árbol binario: cada nodo, al que llamamos “Padre”, puede tener como **máximo dos hijos**. Aquí vemos un nodo padre con dos “espacios” para un posible hijo izquierdo y un posible hijo derecho.

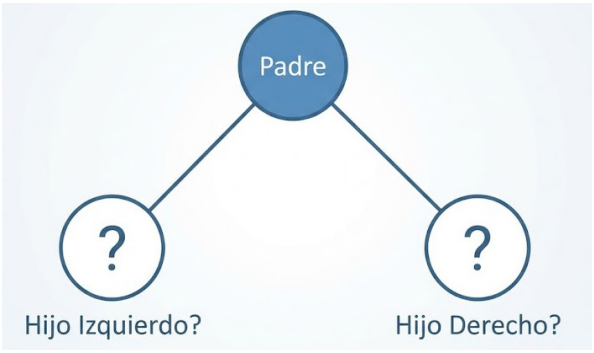


Ilustración 14: Representación conceptual de la relación padre–hijos en un árbol binario. Imagen creada en Nano Banana con información proporcionada por la autora.

6.1.2 Un árbol binario simple

A partir de la estructura anterior, esta segunda imagen muestra un árbol binario simple. La "Raíz" es el nodo principal y sus dos hijos ahora son nodos reales, un "Hijo Izquierdo" y un "Hijo Derecho", cumpliendo con la regla establecida.

Ilustración 15: Estructura básica de un árbol binario con nodo raíz y sus hijos izquierdo y derecho. Imagen creada en Nano Banana con información proporcionada por la autora.

6.1.3 La estructura jerárquica completa

Finalmente, esta imagen expande el árbol para mostrar cómo se construye una estructura de datos más compleja. Los hijos del nivel anterior ("Nodo A" y "Nodo B") se convierten ahora en padres de sus propios hijos. Esto ilustra perfectamente cómo los árboles binarios permiten **organizar la información de manera no lineal y jerárquica**.

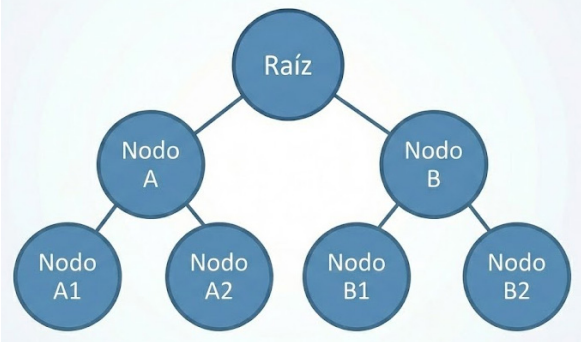


Ilustración 16: Ejemplo de árbol binario con jerarquía de nodos (raíz, nodos internos y hojas). Imagen creada en Nano Banana con información proporcionada por la autora.

Esta estructura resulta especialmente adecuada para representar procesos de decisión, expresiones, jerarquías y espacios de búsqueda.

Énfasis

Los árboles binarios permiten modelar problemas de IA donde la información no se organiza de forma secuencial, sino jerárquica, facilitando la exploración sistemática de alternativas.

6.2 Concepto de árbol binario

Desde un punto de vista abstracto, un árbol binario se define por:

- un nodo raíz,
- subárbol izquierdo,
- subárbol derecho,
- y la relación padre–hijo entre nodos.

Énfasis

Cada nodo almacena un valor y referencias a sus hijos. A diferencia de listas o colas, los árboles no poseen un orden lineal, sino una estructura ramificada. Comprender la estructura conceptual de un árbol binario es clave para interpretar algoritmos de búsqueda, clasificación y toma de decisiones utilizados en sistemas inteligentes.

Lectura complementaria

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el material complementario disponible en el enlace provisto. GeeksforGeeks. (2024, mayo 22). *Binary Tree Data Structure*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/binary-tree-data-structure/

6.3 Representación de un árbol binario en Python

Una forma habitual de representar un árbol binario en Python es mediante programación orientada a objetos, definiendo una clase Nodo que encapsule el valor y las referencias a los hijos.

6.3.1 Ejemplo 1: Definición de un nodo de árbol binario

Este ejemplo muestra cómo definir la estructura básica de un nodo para construir un árbol binario.

```
1   class Nodo:
2       def __init__(self, valor):
3           self.valor = valor
4           self.izquierdo = None
5           self.derecho = None
```

Captura 11: Definición de la clase Nodo para representar un nodo de un árbol binario, con referencias a hijos izquierdo y derecho. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento del código	Explicación
`class Nodo`	Define la estructura de un nodo del árbol.
`valor`	Almacena el dato asociado al nodo.
`izquierdo`	Referencia al hijo izquierdo.
`derecho`	Referencia al hijo derecho.

Énfasis

El uso de clases permite modelar de manera clara y extensible la estructura jerárquica de los árboles.

6.4 Construcción de un árbol binario simple

6.4.1 Ejemplo 2: Creación manual de un árbol binario

Este ejemplo construye un árbol binario simple para ilustrar las relaciones entre nodos.

```
raiz = Nodo(10)
raiz.izquierdo = Nodo(5)
raiz.derecho = Nodo(15)

raiz.izquierdo.izquierdo = Nodo(3)
raiz.izquierdo.derecho = Nodo(7)
```

Captura 12: Construcción manual de un árbol binario mediante la creación de nodos y la asignación de hijos izquierdo y derecho. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`raiz = Nodo(10)`	Se crea el nodo raíz del árbol.
`raiz.izquierdo`	Se asigna el hijo izquierdo.
`raiz.derecho`	Se asigna el hijo derecho.
Nodos hoja	Representan nodos sin hijos.

Énfasis

La estructura del árbol refleja relaciones jerárquicas que luego serán exploradas mediante recorridos sistemáticos.

6.5 Recorridos de árboles binarios

Los recorridos definen el orden en que se visitan los nodos de un árbol. En árboles binarios, los recorridos más habituales son:

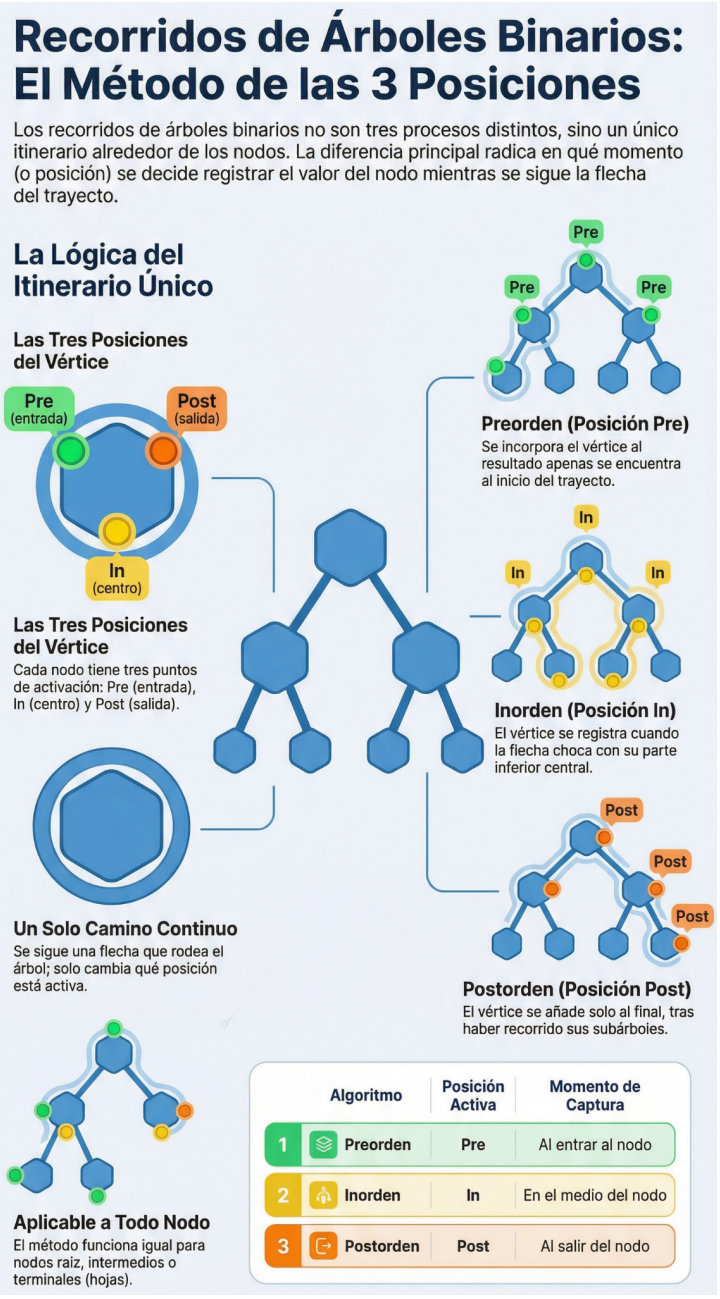
- Preorden
- Inorden
- Postorden

Cada recorrido responde a una lógica distinta y resulta útil según el problema a resolver.

Énfasis

Los recorridos permiten transformar una estructura jerárquica en una secuencia procesable por algoritmos.

En la siguiente infografía, se puede un método sencillo para obtener los caminos de cada recorrido:



Infografía 1: Esquema conceptual de los recorridos en árboles binarios (preorden, inorden y postorden) y su relación con el momento de visita de cada nodo. Imagen creada en NotebookLM con información proporcionada por la autora.

6.6 Recorrido en preorden

En el recorrido preorden, se visita primero el nodo actual, luego el subárbol izquierdo y finalmente el subárbol derecho.

6.6.1 Ejemplo 3: Recorrido en preorden

Este ejemplo implementa un recorrido preorden utilizando recursividad.

```python
def preorden(nodo):
    if nodo is not None:
        print(nodo.valor)
        preorden(nodo.izquierdo)
        preorden(nodo.derecho)
```

Captura 13: Implementación del recorrido preorden en un árbol binario mediante una función recursiva. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`if nodo is not None`	Condición base de la recursión.
`print(nodo.valor)`	Visita el nodo actual.
Llamadas recursivas	Recorren los subárboles izquierdo y derecho.

Este recorrido es utilizado en procesos donde se requiere procesar un nodo antes que sus descendientes, como en la copia de estructuras.

6.7 Recorrido en inorden

En el recorrido inorden, se visita primero el subárbol izquierdo, luego el nodo actual y finalmente el subárbol derecho.

6.7.1 Ejemplo 4: Recorrido en inorden

```python
def inorden(nodo):
    if nodo is not None:
        inorden(nodo.izquierdo)
        print(nodo.valor)
        inorden(nodo.derecho)
```

Captura 14: Implementación del recorrido inorden en un árbol binario mediante una función recursiva. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Subárbol izquierdo	Se recorre antes de visitar el nodo.
Nodo actual	Se procesa luego del hijo izquierdo.
Subárbol derecho	Se recorre al final.

Énfasis

En árboles binarios de búsqueda, el recorrido inorden permite obtener los valores ordenados.

6.8 Recorrido en postorden

En el recorrido postorden, se visitan primero los subárboles izquierdo y derecho, y luego el nodo actual.

6.8.1 Ejemplo 5: Recorrido en postorden

```
1 def postorden(nodo):
2     if nodo is not None:
3         postorden(nodo.izquierdo)
4         postorden(nodo.derecho)
5         print(nodo.valor)
```

Captura 15: Implementación del recorrido postorden en un árbol binario mediante una función recursiva. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Subárboles	Se recorren antes de visitar el nodo.
Nodo actual	Se procesa al final.
Uso típico	Eliminación o evaluación de expresiones.

Énfasis

Este recorrido resulta adecuado cuando se requiere procesar primero los descendientes, como en la liberación de recursos.

Multimedia

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el material complementario disponible en el enlace provisto. Espinas, A. S. [@AprendeSinEspinas]. (2022, septiembre 12). *Algoritmos PREORDEN, INORDEN y POSTORDEN en 3 minutos*. Youtube. https://www.youtube.com/watch?v=Jo2euX89Oz8

6.9 Relación con Inteligencia Artificial

Los árboles binarios y sus recorridos son utilizados en árboles de decisión, estructuras de búsqueda, representación de expresiones y modelado de procesos jerárquicos. Estas estructuras constituyen la base de modelos más complejos que se estudiarán en módulos posteriores.

Énfasis

Dominar árboles y recorridos permite comprender y diseñar algoritmos de IA que exploran espacios de soluciones de forma estructurada y eficiente.

6.10 Árboles n-arios y relación con la recursividad

Un árbol n-ario es una generalización del árbol binario en la cual cada nodo puede tener un número arbitrario de hijos. Este tipo de estructura resulta especialmente útil para representar jerarquías complejas, árboles de decisión con múltiples alternativas y estructuras de conocimiento en Inteligencia Artificial.

Énfasis

Los árboles n-arios permiten modelar espacios de decisión más ricos que los árboles binarios, manteniendo la estructura jerárquica necesaria para la exploración recursiva.

6.11 Representación de un árbol n-ario en Python

6.11.1 Ejemplo 6: Nodo de árbol n-ario

```
1  class Nodo:
2      def __init__(self, valor):
3          self.valor = valor
4          self.hijos = []
5
6      def agregar_hijo(self, nodo):
7          self.hijos.append(nodo)
8  Press Ctrl K to generate code
```

Captura 16: Definición de la clase Nodo para un árbol general, con estructura dinámica de hijos. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`hijos`	Lista que almacena referencias a los nodos descendientes.
`agregar_hijo()`	Permite construir dinámicamente la estructura del árbol.

Énfasis

El uso de listas para representar los hijos mantiene la flexibilidad necesaria para modelar relaciones complejas en IA.

6.11.2 Recorrido recursivo en árboles n-arios

```
def recorrer(nodo):
    if nodo is not None:
        print(nodo.valor)
        for hijo in nodo.hijos:
            recorrer(hijo)
```

Captura 17: Recorrido de un árbol general mediante iteración sobre la lista de hijos de cada nodo. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Recursión	Permite recorrer toda la jerarquía del árbol.
`for hijo in nodo.hijos`	Itera sobre todos los descendientes del nodo actual.

La recursividad es el mecanismo natural para recorrer estructuras jerárquicas, tanto binarias como n-arias.

Los árboles binarios introducen una forma no lineal de organizar y procesar datos, ampliando significativamente el repertorio de estructuras disponibles para el diseño algorítmico aplicables a problemas reales de Inteligencia Artificial.

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 5: Árboles binarios y recorridos.**

7. Grafos y recorridos (BFS y DFS)

7.1 Modelado de relaciones y exploración sistemática de estructuras en IA

En Inteligencia Artificial, una gran cantidad de problemas puede formularse como un conjunto de **entidades** y **relaciones**: páginas conectadas por enlaces, usuarios vinculados por interacciones, ciudades unidas por rutas, conceptos relacionados en una ontología, estados conectados por acciones posibles o nodos conectados en una red neuronal computacional a nivel estructural. La estructura matemática y computacional adecuada para representar estos escenarios es el **grafo**.

Un grafo permite modelar conectividad, vecindad, alcance, caminos y costos. Sobre esa representación, los recorridos BFS (Breadth-First Search, búsqueda en anchura) y DFS (Depth-First Search, búsqueda en profundidad) constituyen estrategias fundamentales para explorar el grafo, construir recorridos, detectar componentes conexas y servir como base para algoritmos más avanzados.

Énfasis

Comprender grafos y recorridos no solo permite implementar exploraciones; permite pensar problemas de IA como espacios de estados y analizar qué estrategia de exploración es más adecuada según el objetivo (cobertura, profundidad, optimalidad por niveles, detección de ciclos, etc.).

7.2 ¿Qué es un grafo?

Un grafo se define como un par $G = (V, E)$, donde:

- V es el conjunto de **vértices** o **nodos** (entidades).
- E es el conjunto de **aristas** (relaciones) que conectan pares de nodos.

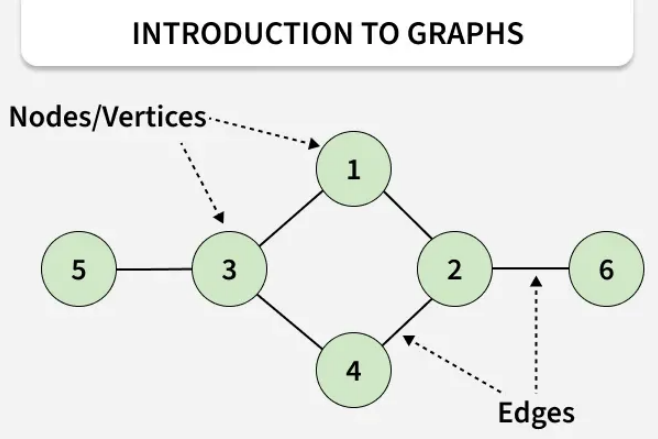


Ilustración 17: Representación básica de un grafo, identificando nodos (vértices) y aristas. Fuente: GeeksforGeeks, Introduction to Graphs – Data Structure and Algorithm Tutorials. Disponible en: https://www.geeksforgeeks.org/dsa/introduction-to-graphs-data-structure-and-algorithm-tutorials/

7.3 Tipos relevantes (en términos computacionales)

- No dirigido: una arista (u, v) implica conexión en ambos sentidos.
- Dirigido: una arista $(u \to v)$ representa una relación con dirección.

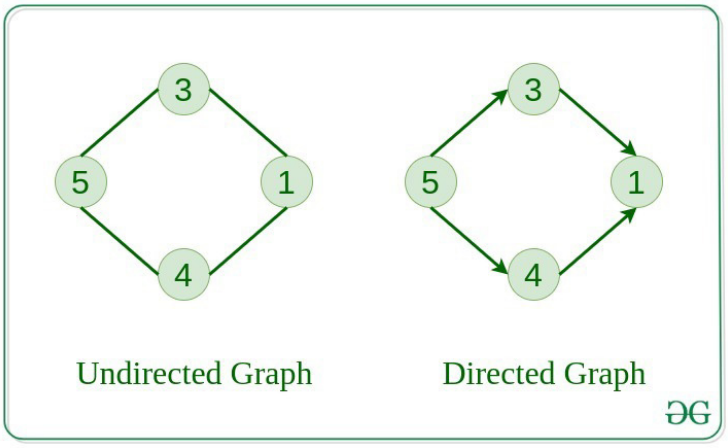


Ilustración 18: Comparación entre grafo no dirigido y grafo dirigido, destacando la ausencia o presencia de orientación en las aristas. Fuente: GeeksforGeeks, Introduction to Graphs – Data Structure and Algorithm Tutorials. Disponible en: https://www.geeksforgeeks.org/dsa/introduction-to-graphs-data-structure-and-algorithm-tutorials/

› ***Ponderado:*** cada arista tiene un peso (costo, distancia, probabilidad, etc.).

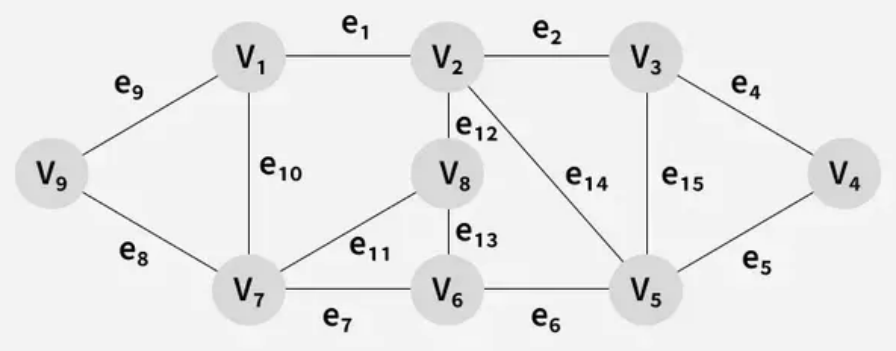


Ilustración 19: Ejemplo de grafo con múltiples vértices y aristas etiquetadas, utilizado para ilustrar la estructura y conectividad de un grafo. Fuente: GeeksforGeeks. Disponible en: https://media.geeksforgeeks.org/wp-content/uploads/20250220130152745825/w-660.webp

› ***Con ciclos / acíclico:*** según existan caminos que vuelvan al mismo nodo.

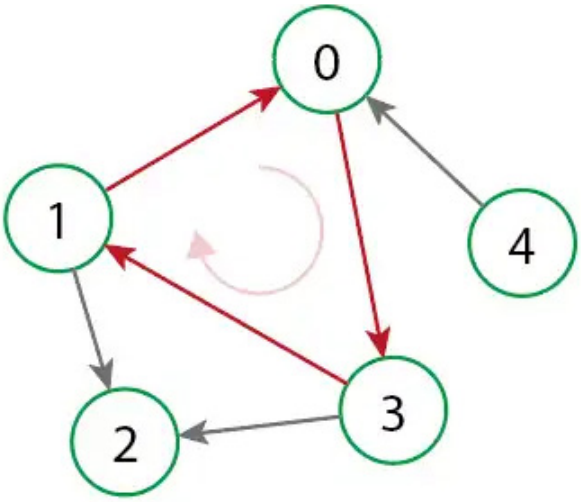


Ilustración 20: Ejemplo de grafo dirigido que ilustra la orientación de las aristas y los posibles recorridos entre nodos. Fuente: Luis Llamas. Disponible en: https://www.luisllamas.es/que-es-un-grafo/

Énfasis

En IA es común trabajar con grafos dirigidos (transiciones de estado, enlaces) y ponderados (costos en ruteo, funciones de evaluación). BFS y DFS son el punto de partida para recorrerlos, aun cuando luego se incorporen pesos o heurísticas.

7.4 Representación de grafos en Python

Para implementar grafos, las representaciones más utilizadas son:

7.4.1 Lista de adyacencia (recomendada en la mayoría de los casos)

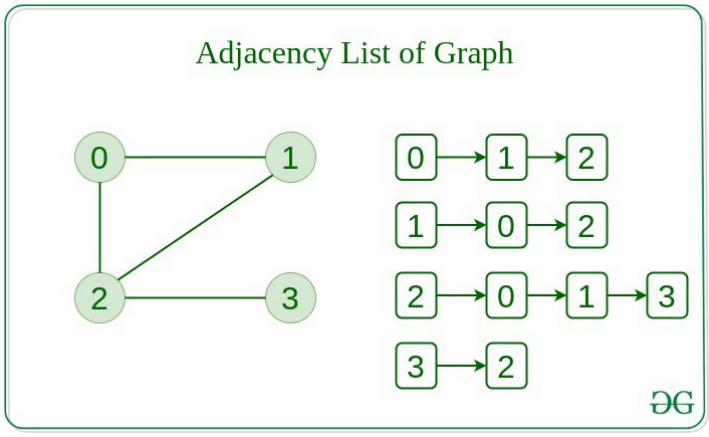


Ilustración 21: Representación de un grafo y su correspondiente matriz de adyacencia, que indica la existencia de aristas entre los vértices. Fuente: GeeksforGeeks. Disponible en: https://media.geeksforgeeks.org/wp-content/uploads/20240502183320/adjacency_mat1-(1)-copy.webp

Estructura típica:

- dict donde la clave es el nodo,
- y el valor es la lista de vecinos.

Ejemplo conceptual:

```
grafo = {
  "A": ["B", "C"],
  "B": ["D"],
  "C": [],
  "D": []
}
```

Ilustración 22: Ejemplo de representación de un grafo mediante una lista de adyacencia en Python. Desarrollado en Online IDE Pro. Código fuente de la autora.

Infografía 2: Ventajas y desventajas de la representación de grafos mediante listas de adyacencia. Creada en Napkin con información proporcionada por la autora.

7.4.2 Matriz de adyacencia

La matriz de adyacencia es una estructura de datos representada mediante una matriz *n×n*, en la que cada elemento *mat[i][j]* indica la existencia de una arista entre los nodos correspondientes, o bien el peso asociado a dicha arista.

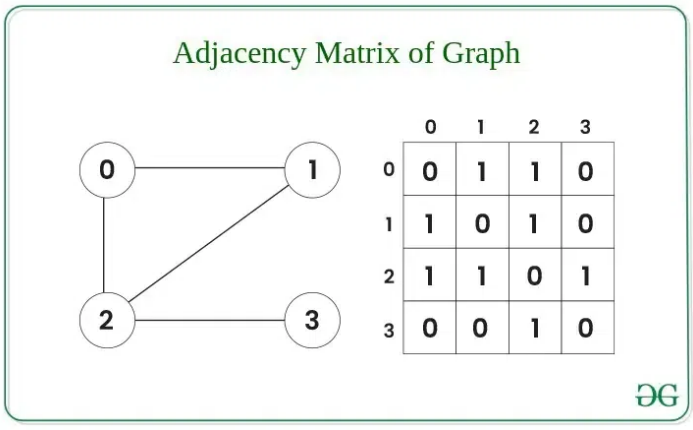


Ilustración 23: Ejemplo de grafo y su representación mediante matriz de adyacencia.
Fuente: https://media.geeksforgeeks.org/wp-content/uploads/20240502183320/adjacency_mat1-(1)-copy.webp

Infografía 3: Ventajas y desventajas de las estructuras de datos de grafos.
Imagen creada en Napkin con información proporcionada por la autora.

Énfasis

En problemas típicos de IA (espacios de estados, redes de relaciones), los grafos suelen ser grandes y relativamente dispersos, por lo que la lista de adyacencia es la elección estándar.

7.5 Principio clave: evitar visitas repetidas

En grafos con ciclos, un recorrido sin control puede:

- repetir nodos indefinidamente,
- degradar el rendimiento,
- o producir exploraciones incorrectas.

Por ello, BFS y DFS se implementan con un conjunto visitados (por ejemplo, set) para garantizar que cada nodo se procese como máximo una vez.

Énfasis

El conjunto de visitados no es un detalle técnico accesorio; es el mecanismo que transforma una exploración potencialmente infinita en un algoritmo correcto y eficiente.

7.6 BFS: Búsqueda en anchura

BFS explora el grafo por capas (niveles):

1. Visita el nodo inicial.
2. Visita todos sus vecinos.
3. Luego los vecinos de los vecinos, y así sucesivamente.

Esto implica que BFS:

- utiliza una cola (FIFO) para mantener el orden de exploración,
- y es especialmente útil para encontrar caminos mínimos en número de aristas en grafos no ponderados.

Énfasis

BFS es la estrategia natural cuando interesa la cercanía por pasos (mínimo número de transiciones), criterio que aparece con frecuencia en espacios de estados y problemas de planificación no ponderada.

7.6.1 Ejemplo 1: BFS con lista de adyacencia y collections.deque

Este ejemplo implementa BFS sobre un grafo no dirigido (modelado como lista de adyacencia) e imprime el orden de visita.

```python
from collections import deque

grafo = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"]
}

def bfs(grafo, inicio):
    visitados = set()
    cola = deque([inicio])
    visitados.add(inicio)

    orden = []

    while cola:
        actual = cola.popleft()
        orden.append(actual)

        for vecino in grafo.get(actual, []):
            if vecino not in visitados:
                visitados.add(vecino)
                cola.append(vecino)

    return orden

print("BFS desde A:", bfs(grafo, "A"))
```

Captura 18: Implementación del recorrido en anchura (BFS) sobre un grafo utilizando una cola (deque). Código desarrollado en Online IDE Pro. Código fuente de la autora.

Bloque / Línea	Explicación
`dist = {inicio: 0}`	Diccionario de distancias; además funciona como "visitados" implícito.
`if vecino not in dist`	Asegura asignar distancia solo la primera vez (que es la mínima en BFS).
`dist[vecino] = dist[actual] + 1`	Propaga distancia por niveles: cada vecino está a un paso más que el nodo actual.
Resultado	Se obtiene un mapa de distancias mínimas por número de aristas.

BFS garantiza que la primera vez que se alcanza un nodo se lo hace con el menor número de pasos posible, propiedad crucial para razonamiento en espacios de estados no ponderados.

7.7 BFS para distancias mínimas (en número de pasos)

BFS puede extenderse para calcular la distancia mínima (cantidad de aristas) desde el nodo inicial a cada nodo alcanzable.

7.7.1 Ejemplo 2: BFS para calcular distancias mínimas

Este ejemplo calcula la distancia mínima (en pasos) desde un nodo de inicio a cada nodo.

```python
from collections import deque

def bfs_distancias(grafo, inicio):
    dist = {inicio: 0}
    cola = deque([inicio])

    while cola:
        actual = cola.popleft()
        for vecino in grafo.get(actual, []):
            if vecino not in dist:
                dist[vecino] = dist[actual] + 1
                cola.append(vecino)

    return dist

print("Distancias desde A:", bfs_distancias(grafo, "A"))

```

Captura 19: Cálculo de distancias mínimas desde un nodo inicial mediante BFS utilizando una cola (deque). Código desarrollado en Online IDE Pro. Código fuente de la autora.

Bloque / Línea	Explicación
`dist = {inicio: 0}`	Diccionario de distancias; además funciona como "visitados" implícito.
`if vecino not in dist`	Asegura asignar distancia solo la primera vez (que es la mínima en BFS).
`dist[vecino] = dist[actual] + 1`	Propaga distancia por niveles: cada vecino está a un paso más que el nodo actual.
Resultado	Se obtiene un mapa de distancias mínimas por número de aristas.

Énfasis

BFS garantiza que la primera vez que se alcanza un nodo se lo hace con el menor número de pasos posible, propiedad crucial para razonamiento en espacios de estados no ponderados.

Multimedia

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el material complementario disponible en el enlace provisto https://youtu.be/FzUrccrk2yc Video creado con NotebookLM con información proporcionada por la autora.

7.8 DFS: Búsqueda en profundidad

DFS explora el grafo avanzando lo más profundo posible antes de retroceder:

1. Visita un nodo.
2. Elige un vecino no visitado y continúa.
3. Si no hay vecinos no visitados, retrocede (backtracking).

DFS se implementa naturalmente con:

- recursión (pila de llamadas) o
- una pila explícita (stack).

Énfasis

DFS es especialmente útil cuando interesa la exploración profunda, la detección de ciclos, la construcción de recorridos y la estructura de "retroceso" típica de problemas combinatorios y búsqueda.

7.8.1 Ejemplo 3: DFS recursivo

Este ejemplo implementa DFS recursivo e imprime el orden de visita.

```python
def dfs_recursivo(grafo, inicio):
    visitados = set()
    orden = []

    def visitar(nodo):
        visitados.add(nodo)
        orden.append(nodo)

        for vecino in grafo.get(nodo, []):
            if vecino not in visitados:
                visitar(vecino)

    visitar(inicio)
    return orden

print("DFS recursivo desde A:", dfs_recursivo(grafo, "A"))
```

Captura 20: Implementación del recorrido DFS recursivo sobre un grafo, utilizando un conjunto de nodos visitados para evitar repeticiones. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Bloque / Línea	Explicación
`visitados = set()`	Controla nodos visitados para evitar ciclos y repeticiones.
`def visitar(nodo)`	Función interna que encapsula la lógica recursiva.
`orden.append(nodo)`	Registra el recorrido en el momento en que se visita el nodo.
`visitar(vecino)`	Avanza en profundidad: procesa el vecino antes de continuar con otros vecinos del nodo actual.
Retorno	Devuelve el orden de visita, útil para ver el patrón de exploración.

La recursión implementa DFS de manera directa, ya que la pila de llamadas actúa como estructura implícita de "pendientes".

7.8.2 Ejemplo 4: DFS iterativo con pila explícita (evita límites de recursión)

En grafos grandes, la recursión puede ser limitada por el stack del lenguaje. Este ejemplo implementa DFS con una pila explícita.

```python
def dfs_iterativo(grafo, inicio):
    visitados = set()
    pila = [inicio]
    orden = []

    while pila:
        actual = pila.pop()

        if actual in visitados:
            continue

        visitados.add(actual)
        orden.append(actual)

        vecinos = grafo.get(actual, [])
        for v in reversed(vecinos):
            if v not in visitados:
                pila.append(v)

    return orden

print("DFS iterativo desde A:", dfs_iterativo(grafo, "A"))
```

Captura 21: Implementación del recorrido DFS iterativo sobre un grafo mediante una pila explícita, controlando los nodos visitados. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Bloque / Línea	Explicación
`pila = [inicio]`	Estructura LIFO: controla la exploración en profundidad.
`actual = pila.pop()`	Extrae el último agregado: esto fuerza profundidad antes que anchura.
`if actual in visitados: continue`	Evita reprocesar nodos que pudieron agregarse varias veces desde distintos caminos.
`reversed(vecinos)`	Mantiene un orden de recorrido más predecible (similar al recursivo).
`pila.append(v)`	Encola "para profundizar" (en realidad apila) vecinos que se visitarán antes que otros niveles.

Énfasis

DFS iterativo hace explícito el rol de la pila, facilitando el control del algoritmo y evitando problemas de recursión en grafos grandes.

Multimedia

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el material complementario disponible en el enlace provisto https://youtu.be/FidAlbL1u_8 Video creado con NotebookLM con información proporcionada por la autora.

7.9 BFS vs. DFS

Aspecto	BFS	DFS
Estructura base	Cola (FIFO)	Pila (LIFO) / recursión
Estrategia	Explora por niveles	Explora por profundidad
Garantía de camino mínimo (no ponderado)	Sí (por número de aristas)	No
Uso típico en IA	Búsqueda por niveles, distancias mínimas, expansión controlada	Backtracking, exploración profunda, detección de ciclos, componentes
Memoria (intuición)	Puede crecer mucho si el frente es grande	Puede ser más acotada si la profundidad domina
Patrón de visita	"Cercanos primero"	"Profundos primero"

Tabla 1: Comparación BFS vs. DFS

Énfasis

La elección entre BFS y DFS no es estilística; define el tipo de exploración del espacio de soluciones y condiciona rendimiento y propiedades del resultado.

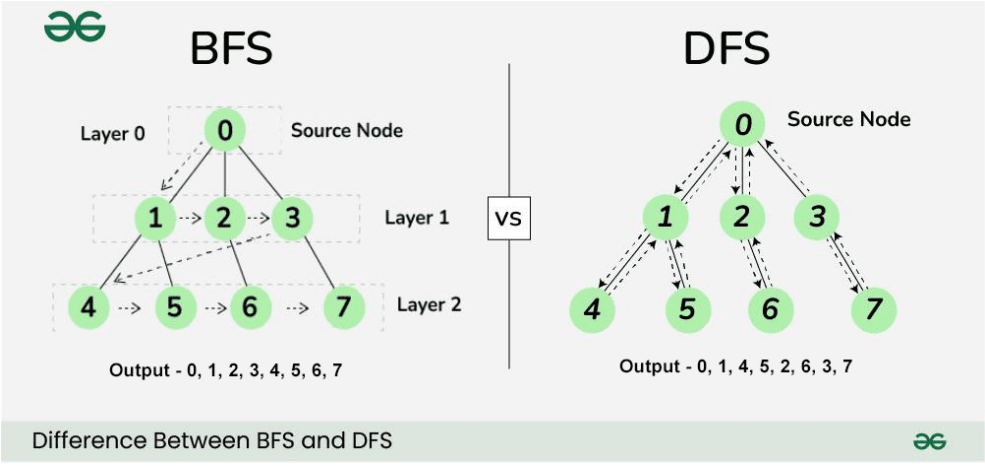


Ilustración 24: Comparación entre los recorridos en anchura (BFS) y en profundidad (DFS) sobre un grafo. Fuente: GeeksforGeeks. https://media.geeksforgeeks.org/wp-content/uploads/20240216084522/bfs-vs-dfs-(1).png

7.10 Complejidad (tiempo y espacio)

Para una representación por lista de adyacencia:

- Tiempo de BFS y DFS: O (|V| + |E|) (cada nodo se visita a lo sumo una vez y cada arista se considera un número acotado de veces).
- Espacio:
  - BFS: cola y estructuras auxiliares pueden crecer con el tamaño de la frontera.
  - DFS: la pila (implícita o explícita) crece con la profundidad máxima explorada.

Énfasis

La complejidad formal explica por qué grafos grandes exigen estructuras eficientes y por qué el control de visitados es determinante para garantizar escalabilidad.

7.11 Aplicación en IA: espacio de estados

Un problema clásico en IA puede verse como:

- estado = nodo
- acción = arista/transición
- objetivo = alcanzar un estado meta

BFS y DFS se transforman así en estrategias de exploración del espacio:

- BFS: es útil cuando las soluciones más cortas son preferibles (en pasos).
- DFS: es útil cuando interesa explorar rápidamente rutas completas o aplicar retroceso.

Énfasis

Esta interpretación convierte grafos y recorridos en una base conceptual para algoritmos posteriores como búsqueda informada, A*, planificación y razonamiento sobre caminos.

Los grafos permiten modelar relaciones complejas y espacios de estados, mientras que BFS y DFS proveen estrategias fundamentales para explorarlos de forma correcta y eficiente.

Lectura complementaria

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el material complementario disponible en el enlace provisto. GeeksforGeeks. (2024, noviembre 7). *Graph Data Structure*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/graph-data-structure/

Actividad

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 6: Grafos y recorridos (BFS y DFS).**

8. Notación Big-O y análisis temporal y espacial

8.1 Introducción al análisis de complejidad

La **notación Big-O** permite describir cómo crece el tiempo de ejecución o el consumo de memoria de un algoritmo en función del tamaño de la entrada. En Inteligencia Artificial, este análisis resulta esencial para evaluar la **escalabilidad** de los algoritmos.

Énfasis

Analizar complejidad no busca medir tiempos exactos, sino comparar comportamientos de crecimiento ante entradas cada vez mayores.

8.2 Complejidad temporal

Énfasis

La complejidad temporal indica cómo crece el tiempo de ejecución de un algoritmo a medida que aumenta el tamaño de la entrada (n).

La notación Big O permite comparar algoritmos según su eficiencia, independientemente del hardware o del lenguaje utilizado.

- **O (1) – Tiempo constante:** El tiempo de ejecución no depende del tamaño de la entrada. La operación siempre tarda lo mismo, por ejemplo, acceder a un elemento por índice en una lista.
- **O(n) – Crecimiento lineal:** El tiempo de ejecución crece de manera proporcional al tamaño de la entrada. A mayor cantidad de datos, mayor tiempo, como al recorrer una lista completa.
- **O(n^2) – Crecimiento cuadrático:** El tiempo aumenta de forma cuadrática respecto al tamaño de la entrada. Suele aparecer cuando se utilizan bucles anidados, como en comparaciones entre todos los pares de elementos.
- **O (log n) – Crecimiento logarítmico:** El tiempo crece lentamente a medida que aumenta la entrada, ya que en cada paso se reduce el problema (por ejemplo, a la mitad). Es típico de algoritmos eficientes como la búsqueda binaria.

Notación	Significado
O (1)	Tiempo constante
O(n)	Crecimiento lineal
O(n^2)	Crecimiento cuadrático
O (log n)	Crecimiento logarítmico

Tabla 2: Resumen de la complejidad temporal

8.3 Medición empírica con time

8.3.1 Ejemplo 1: Medición de tiempo de ejecución

```python
import time

def suma_lista(numeros):
    total = 0
    for n in numeros:
        total += n
    return total

datos = list(range(1, 1000001))

inicio = time.time()
suma_lista(datos)
fin = time.time()

print("Tiempo de ejecución (segundos):", fin - inicio)
```

Captura 22: Medición del tiempo de ejecución de una función que suma los elementos de una lista, como ejemplo de análisis de complejidad temporal. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`time.time()`	Obtiene el tiempo actual en segundos.
Diferencia	Aproxima el tiempo de ejecución del algoritmo.

Esta medición permite observar empíricamente el impacto del tamaño de entrada sobre el tiempo de ejecución.

8.4 Comparación de estrategias algorítmicas

8.4.1 Ejemplo 2: Comparación O(n) vs O (1)

```python
import time

def suma_for(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

def suma_formula(n):
    return n * (n + 1) // 2

n = 1000000

inicio = time.time()
suma_for(n)
print("Tiempo suma con for:", time.time() - inicio)

inicio = time.time()
suma_formula(n)
print("Tiempo suma con fórmula:", time.time() - inicio)
```

Captura 23: Comparación del tiempo de ejecución entre una suma iterativa y una suma mediante fórmula cerrada, ilustrando diferencias de complejidad temporal. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`suma_for`	Implementación de complejidad O(n).
`suma_formula`	Implementación de complejidad O(1).
Resultado	Evidencia empírica de la diferencia de crecimiento.

Énfasis

Elegir un algoritmo más eficiente puede producir mejoras significativas sin modificar el resultado lógico.

8.5 Complejidad espacial

Énfasis

La complejidad espacial describe cómo crece el consumo de memoria de un algoritmo a medida que aumenta el tamaño de la entrada (n).

No solo considera el espacio ocupado por los datos de entrada, sino también la memoria adicional utilizada durante la ejecución, como variables auxiliares, estructuras temporales, listas intermedias o llamadas recursivas. Por ejemplo, un algoritmo que procesa los datos *in situ* (sin crear copias) tendrá un menor costo espacial que otro que genera nuevas listas o estructuras para almacenar resultados parciales. Asimismo, el uso de recursividad puede incrementar la complejidad espacial debido a la memoria requerida por la pila de llamadas.

Analizar la complejidad espacial es fundamental para evaluar la viabilidad de un algoritmo en contextos donde la memoria es un recurso limitado, como en sistemas embebidos, aplicaciones en tiempo real o procesamiento de grandes volúmenes de datos. En IA, la eficiencia espacial es tan importante como la temporal, especialmente al trabajar con grandes volúmenes de datos.

Actividad

A partir de los conceptos trabajados sobre Tipos de Datos Abstractos y su implementación en Python, ya se cuenta con los elementos necesarios para desarrollar la **Actividad 7: Notación Big-O y análisis de complejidad.**

Evaluación

Luego de haber estudiado los módulos 1 y 2 y de haber resuelto las actividades, Ud. está en condiciones de resolver la primera parte de la evaluación integradora.

Palabras finales

A lo largo de este módulo se ha consolidado una comprensión profunda y articulada de las estructuras de datos lineales y jerárquicas, junto con los principios fundamentales del análisis de complejidad temporal y espacial, pilares esenciales para el diseño de algoritmos eficientes en Inteligencia Artificial. El recorrido comenzó desde el enfoque conceptual de los Tipos de Datos Abstractos, que permite separar claramente el *qué* del *cómo*, favoreciendo un pensamiento algorítmico independiente del lenguaje y orientado a la reutilización y comparación de soluciones.

El estudio de listas dinámicas, pilas, colas y colas de prioridad permitió comprender cómo distintas estructuras modelan comportamientos computacionales específicos (secuenciales, LIFO, FIFO, priorizados) y cómo su correcta elección impacta directamente en el rendimiento y la escalabilidad de los algoritmos. En este sentido, el uso de herramientas como collections.deque y heapq evidenció que pequeñas decisiones de implementación pueden producir mejoras significativas sin alterar la lógica general del algoritmo.

La introducción de árboles binarios, árboles n-arios y sus recorridos amplió la perspectiva desde estructuras lineales hacia organizaciones jerárquicas, fundamentales para representar decisiones, procesos recursivos y espacios de búsqueda. Del mismo modo, el abordaje de grafos y recorridos BFS y DFS permitió modelar relaciones complejas y explorar sistemáticamente espacios de estados, sentando las bases conceptuales de algoritmos de búsqueda más avanzados utilizados en IA.

Finalmente, la notación Big-O y el análisis de complejidad temporal y espacial, complementados con mediciones empíricas mediante time y timeit, brindaron un marco riguroso para evaluar la eficiencia real de los algoritmos. Este análisis no solo permite comparar estrategias, sino también anticipar límites prácticos de tiempo y memoria en contextos reales.

En conjunto, el módulo no se limita a presentar estructuras y algoritmos, sino que promueve una mirada crítica e integrada: **elegir una estructura de datos implica elegir un modelo de razonamiento, un patrón de exploración y un compromiso entre tiempo, espacio y escalabilidad**. Esta perspectiva resulta indispensable para el desarrollo de soluciones robustas y eficientes en Inteligencia Artificial, y constituye una base sólida para los módulos posteriores, donde estas estructuras serán reutilizadas y extendidas en algoritmos de mayor complejidad.

Módulo 2 | Glosario

- **Árbol:** Estructura de datos no lineal organizada de forma jerárquica, compuesta por nodos conectados mediante relaciones padre–hijo, utilizada para representar jerarquías y procesos recursivos.

- **Árbol binario:** Tipo de árbol en el cual cada nodo puede tener como máximo dos hijos, denominados hijo izquierdo e hijo derecho, ampliamente empleado en búsquedas, ordenamientos y representación de expresiones.

- **BFS (Breadth-First Search):** Algoritmo de recorrido de grafos que explora los vértices por niveles, utilizando una cola, y que garantiza encontrar el camino más corto en grafos no ponderados.

- **Cola (Queue):** Estructura de datos lineal que sigue el principio FIFO (First In, First Out), donde el primer elemento en ingresar es el primero en salir.

- **Cola de prioridad:** Estructura de datos en la que cada elemento posee una prioridad asociada, y la extracción se realiza según dicha prioridad y no según el orden de inserción.

- **Complejidad espacial:** Medida que analiza cómo crece el consumo de memoria de un algoritmo en función del tamaño de la entrada y de las estructuras auxiliares utilizadas.

- **Complejidad temporal:** Medida que evalúa el crecimiento del tiempo de ejecución de un algoritmo según el tamaño de la entrada, independientemente del hardware utilizado.

- **DFS (Depth-First Search):** Algoritmo de recorrido de grafos que explora profundamente cada rama antes de retroceder, generalmente implementado mediante recursión o pilas.

- **Deque (Double-Ended Queue)**: Estructura de datos que permite inserciones y eliminaciones eficientes en ambos extremos, combinando comportamientos de pila y cola.

- **Grafo:** Estructura de datos compuesta por vértices (nodos) y aristas (conexiones), utilizada para modelar relaciones complejas como redes, caminos o dependencias.

- **Heap:** Estructura de datos basada en árboles binarios que mantiene una propiedad de orden (mínimo o máximo), utilizada para implementar colas de prioridad de forma eficiente.

- **Lista:** Estructura de datos lineal y dinámica que almacena una secuencia ordenada de elementos, permitiendo acceso por índice y modificación de su tamaño.

- **Notación Big-O:** Herramienta matemática utilizada para describir el comportamiento asintótico de la complejidad temporal o espacial de un algoritmo.

- **Pila (Stack):** Estructura de datos lineal que sigue el principio LIFO (Last In, First Out), donde el último elemento en ingresar es el primero en salir.
- **Recorrido de árbol:** Proceso sistemático de visita de los nodos de un árbol siguiendo un orden determinado, como preorden, inorden o postorden.
- **Recursividad:** Técnica de programación en la que una función se define en términos de sí misma, comúnmente utilizada en árboles, grafos y algoritmos de backtracking.
- **Tipo de Dato Abstracto (TDA):** Modelo conceptual que define un conjunto de datos y las operaciones permitidas sobre ellos, sin especificar su implementación concreta.

Módulo 2 | Actividades

ACTIVIDADES

Consigna

Las actividades de este módulo son interactivas y autoasistidas. Para realizarlas, es necesario acceder al enlace ***disponible en plataforma*** y seguir las instrucciones indicadas. Ante cualquier duda o inconveniente durante el desarrollo, se solicita comunicarse con el/la tutor/a para recibir acompañamiento y orientación.

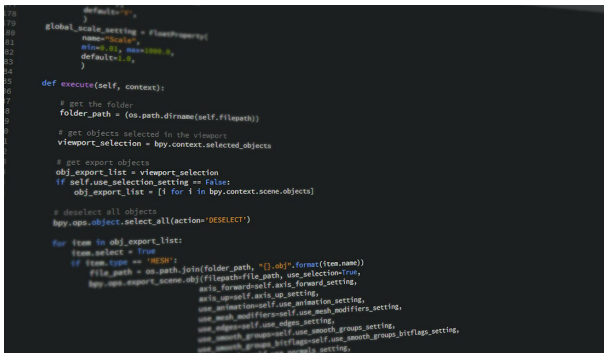
Módulo 3

Microobjetivos

- Reconocer las características y diferencias entre la búsqueda secuencial y la búsqueda binaria para seleccionar el algoritmo más adecuado en problemas de acceso a datos dentro de sistemas de Inteligencia Artificial.
- Aplicar técnicas de búsqueda indexada y árboles de búsqueda binaria para saber optimizar el acceso eficiente a información estructurada en escenarios de procesamiento de datos utilizados en IA.
- Implementar y comparar algoritmos de ordenamiento clásicos y optimizados para evaluar su eficiencia, estabilidad y costo computacional en conjuntos de datos propios de aplicaciones de Inteligencia Artificial.
- Utilizar tablas de dispersión y funciones hash para garantizar operaciones de búsqueda, inserción y actualización en tiempo promedio constante en el manejo de grandes volúmenes de datos en sistemas de IA.
- Integrar algoritmos de emparejamiento y fusión de listas ordenadas para facilitar la organización y consolidación eficiente de información en procesos algorítmicos y pipelines de Inteligencia Artificial.

Contenidos

BÚSQUEDA, ORDENAMIENTO Y HASHING



Fuente: Pexels

1. Introducción

En este módulo se abordan estructuras y algoritmos fundamentales que permiten acceder, organizar y recuperar información de manera eficiente, competencias centrales en el desarrollo de sistemas de Inteligencia Artificial. A medida que los volúmenes de datos crecen y los modelos requieren respuestas cada vez más rápidas, la elección adecuada de estrategias de búsqueda, métodos de ordenamiento y estructuras de indexación se vuelve determinante para el desempeño y la escalabilidad de las soluciones.

Desde una perspectiva algorítmica, el módulo analiza los mecanismos clásicos de búsqueda secuencial y binaria, y los extiende hacia enfoques más estructurados mediante árboles de búsqueda y tablas de dispersión, que constituyen la base de numerosos sistemas de indexación y recuperación utilizados en IA, ciencia de datos y aprendizaje automático. Asimismo, se estudian algoritmos de ordenamiento ampliamente conocidos y sus implementaciones optimizadas en Python, poniendo el foco en la comparación de eficiencia y en los criterios que guían su selección en contextos reales.

Este recorrido adopta un enfoque teórico–práctico, combinando el análisis de complejidad computacional con la experimentación en Python, para que se comprendan no solo los algoritmos en sí, sino también las razones por las cuales determinadas estructuras resultan más adecuadas que otras según el problema planteado. De este modo, se promueve una comprensión profunda del vínculo entre datos, algoritmos y rendimiento, preparándolos para tomar decisiones informadas en el diseño e implementación de soluciones de Inteligencia Artificial eficientes, robustas y escalables.

1.1 Búsqueda secuencial (linear search)

Énfasis

La búsqueda secuencial es un algoritmo elemental que consiste en recorrer una estructura de datos lineal, como una lista o un arreglo, comparando cada elemento con la clave buscada hasta encontrarla o hasta que se hayan examinado todos los elementos.

En este sentido, no requiere que los datos estén ordenados y su simplicidad la hace adecuada para conjuntos pequeños o situaciones donde no es posible imponer un orden previo. Desde el punto de vista de la complejidad computacional, la búsqueda secuencial presenta un costo O(n) tanto en el peor caso como en el caso promedio, ya que puede ser necesario examinar todos los elementos.

En contextos de Inteligencia Artificial, este tipo de búsqueda aparece en fases exploratorias, validaciones iniciales o en estructuras transitorias de tamaño reducido (Cormen et al., 2022).

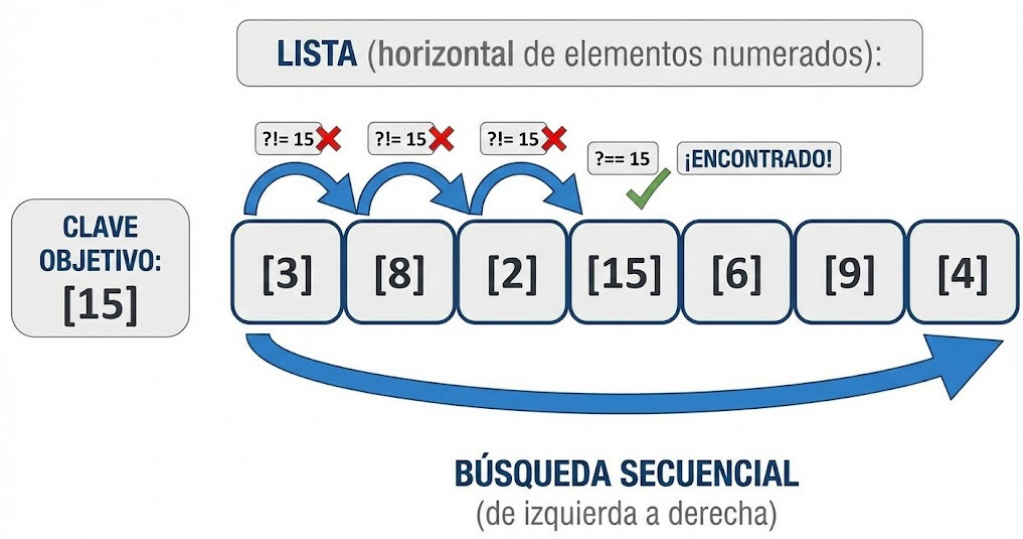


Ilustración 1: Esquema del funcionamiento de la búsqueda secuencial sobre una lista, evaluando elementos de izquierda a derecha hasta encontrar la clave objetivo. Imagen creada en Nano Banana con información proporcionada por la autora.

1.1.1 Ejemplo 1: Implementación de búsqueda secuencial en Python

Este ejemplo permite observar cómo el algoritmo compara cada elemento de la lista hasta encontrar el valor buscado. Es representativo de situaciones frecuentes en IA cuando se trabaja con listas no ordenadas, por ejemplo, conjuntos de etiquetas o resultados intermedios.

```python
def busqueda_secuencial(lista, clave):
    for indice, elemento in enumerate(lista):
        if elemento == clave:
            return indice
    return -1

datos = [42, 7, 19, 23, 5, 88]
resultado = busqueda_secuencial(datos, 23)

print("Se encuentra en la posición: ", resultado+1)
```

Captura 1: Implementación de la búsqueda secuencial sobre una lista en Python, con recorrido elemento a elemento hasta hallar la clave objetivo. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`def busqueda_secuencial(lista, clave):`	Define una función que recibe una lista y la clave a buscar.
`for indice, elemento in enumerate(lista):`	Recorre la lista elemento por elemento, obteniendo tanto el índice como el valor actual.
`if elemento == clave:`	Compara el elemento actual con la clave buscada.
`return indice`	Si hay coincidencia, devuelve la posición donde se encontró la clave.
`return -1`	Indica que la clave no se encuentra en la lista tras recorrerla por completo.
Ejemplo de uso	Se aplica la función a una lista de enteros y se imprime el índice encontrado.

Lectura complementaria

Para profundizar en el concepto de **búsqueda secuencial**, se sugiere consultar el siguiente recurso complementario disponible en GeeksforGeeks: https://www.geeksforgeeks.org/dsa/linear-search/.

1.2 Búsqueda binaria (Binary Search)

Énfasis

La búsqueda binaria es un algoritmo de búsqueda eficiente que opera sobre estructuras previamente ordenadas.

En cada paso, compara la clave buscada con el elemento central de la lista y, a partir de esa comparación, descarta la mitad del espacio de búsqueda que no puede contenerla.

Este proceso se repite de forma iterativa o recursiva hasta encontrar el valor o agotar las posibilidades.

Su complejidad temporal es O(log n), lo que representa una mejora sustancial frente a la búsqueda secuencial. En Inteligencia Artificial, la búsqueda binaria se utiliza en procesos de discretización, selección de umbrales, validación de rangos y recuperación eficiente de datos ordenados (Knuth, 1998).

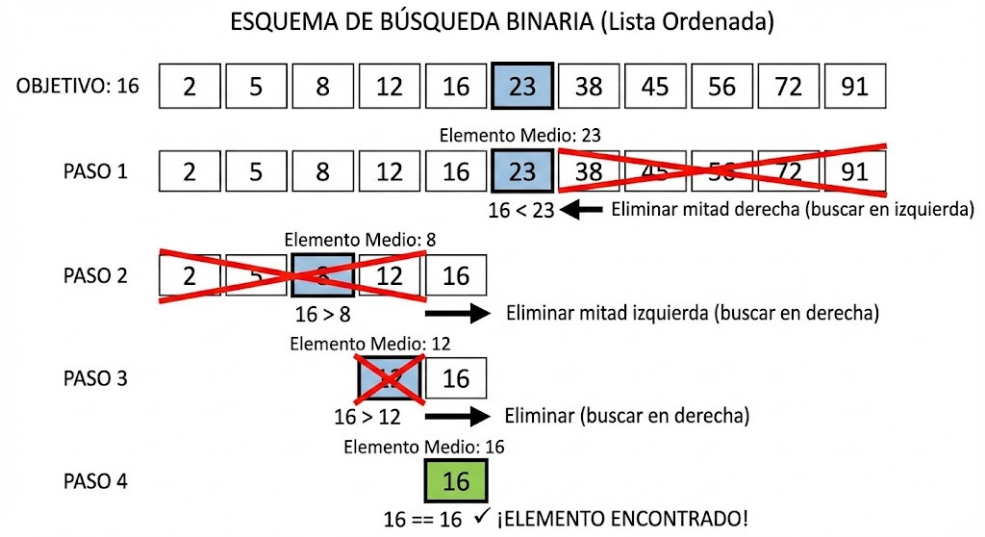


Ilustración 2: Esquema del funcionamiento de la búsqueda binaria sobre una lista ordenada, mostrando la reducción progresiva del espacio de búsqueda hasta encontrar el elemento objetivo. Imagen creada en Nano Banana con información proporcionada por la autora.

1.2.1 Ejemplo 2: Implementación de búsqueda binaria iterativa

Este ejemplo muestra de forma explícita la lógica de partición del espacio de búsqueda, facilitando la comprensión del salto conceptual desde la búsqueda secuencial hacia algoritmos logarítmicos.

```python
def busqueda_binaria(lista, clave):
    inicio = 0
    fin = len(lista) - 1

    while inicio <= fin:
        medio = (inicio + fin) // 2
        if lista[medio] == clave:
            return medio
        elif lista[medio] < clave:
            inicio = medio + 1
        else:
            fin = medio - 1
    return -1

datos = [3, 7, 12, 19, 25, 31, 42]
resultado = busqueda_binaria(datos, 19)

print("Resultado:", resultado)
```

Captura 2: Implementación del algoritmo de búsqueda binaria sobre una lista ordenada en Python. Código fuente de la autora, desarrollado en Online IDE Pro.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`inicio` y `fin`	Delimitan el rango actual de búsqueda dentro de la lista.
`medio`	Calcula el índice central del rango considerado.
Comparación central	Determina si la clave está en el centro, a la izquierda o a la derecha.
Actualización de límites	Reduce el espacio de búsqueda descartando la mitad innecesaria.
`return -1`	Indica que la clave no está presente en la lista.

Lectura complementaria

Para profundizar en el concepto de búsqueda binaria, se sugiere consultar el siguiente recurso complementario disponible en GeeksforGeeks: https://www.geeksforgeeks.org/dsa/binary-search/

1.3 Búsquedas indexadas con el módulo bisect

Énfasis

La búsqueda indexada se apoya en estructuras ordenadas y en índices que permiten localizar rápidamente posiciones relevantes sin recorrer la lista completa. En este sentido, el módulo *bisect* de Python implementa algoritmos de búsqueda binaria para determinar posiciones de inserción en listas ordenadas.

A diferencia de otras estrategias, no devuelve directamente si un elemento existe, sino el índice donde debería insertarse para mantener el orden. Esto resulta especialmente útil en IA para discretización de valores, asignación de rangos y mantenimiento de estructuras ordenadas dinámicas (Python Software Foundation, s. f.).

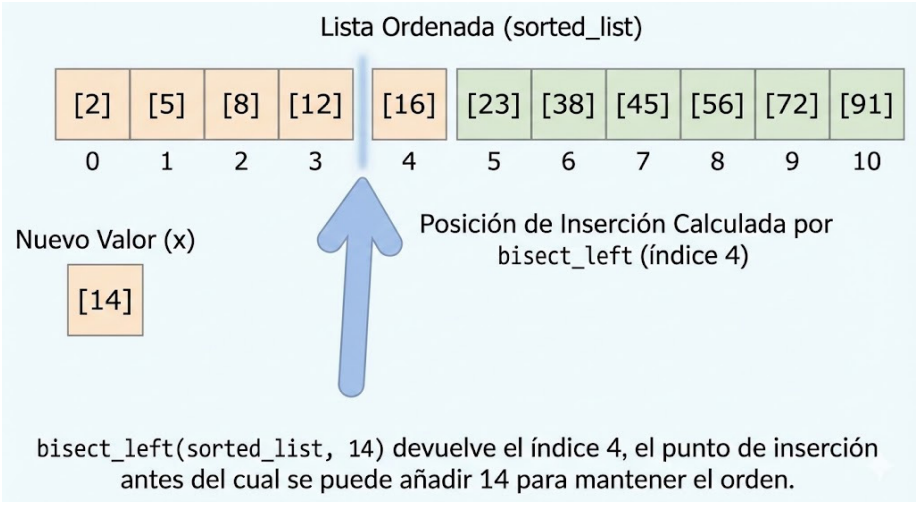


Ilustración 3: Esquema del uso de bisect_left para calcular la posición de inserción de un nuevo valor en una lista ordenada, manteniendo el orden. Imagen creada en Nano Banana con información proporcionada por la autora.

1.3.1 Ejemplo 3: Uso de bisect_left para búsquedas indexadas

Este ejemplo muestra la conexión de la búsqueda binaria con una aplicación práctica frecuente en IA: determinar rápidamente en qué intervalo o segmento cae un valor continuo, como un score o una probabilidad.

```
1  from bisect import bisect_left
2
3  umbrales = [0.2, 0.4, 0.6, 0.8]
4  valor = 0.65
5
6  indice = bisect_left(umbrales, valor)
7
8  print("El valor debe ubicarse en la posición:", indice)
9
```

Captura 3: Ejemplo de uso del módulo bisect para calcular la posición de inserción en una lista ordenada. Código fuente de la autora, desarrollado en Online IDE Pro.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`from bisect import bisect_left`	Importa la función de búsqueda binaria para inserción.
`umbrales`	Lista ordenada que actúa como índice de referencia.
`bisect_left(umbrales, valor)`	Devuelve el índice donde `valor` debería insertarse manteniendo el orden.
Resultado	El índice indica el intervalo o segmento correspondiente al valor analizado.

Lectura complementaria

Para profundizar en el concepto de búsqueda binaria, se sugiere consultar el siguiente recurso complementario disponible en GeeksforGeeks: https://docs.python.org/3/library/bisect.html

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 1: Sistema interactivo de búsquedas.**

2. Árboles de búsqueda binaria (BST)

Énfasis

Un árbol de búsqueda binaria es una estructura de datos jerárquica en la cual cada nodo contiene una clave y cumple la siguiente propiedad de orden:

- Todas las claves del subárbol izquierdo son menores que la clave del nodo,
- Todas las claves del subárbol derecho son mayores que la clave del nodo.

Esta propiedad permite realizar operaciones de búsqueda, inserción y eliminación de manera eficiente, siguiendo una lógica comparable a la búsqueda binaria sobre arreglos ordenados, pero aplicada a una estructura dinámica basada en nodos enlazados (Cormen et al., 2022).

En el contexto de la Inteligencia Artificial, los BST resultan relevantes como estructuras conceptuales de indexación, especialmente en escenarios donde los datos deben mantenerse ordenados dinámicamente y consultarse por clave, como en sistemas de recuperación de información, rankings o almacenamiento intermedio de resultados.

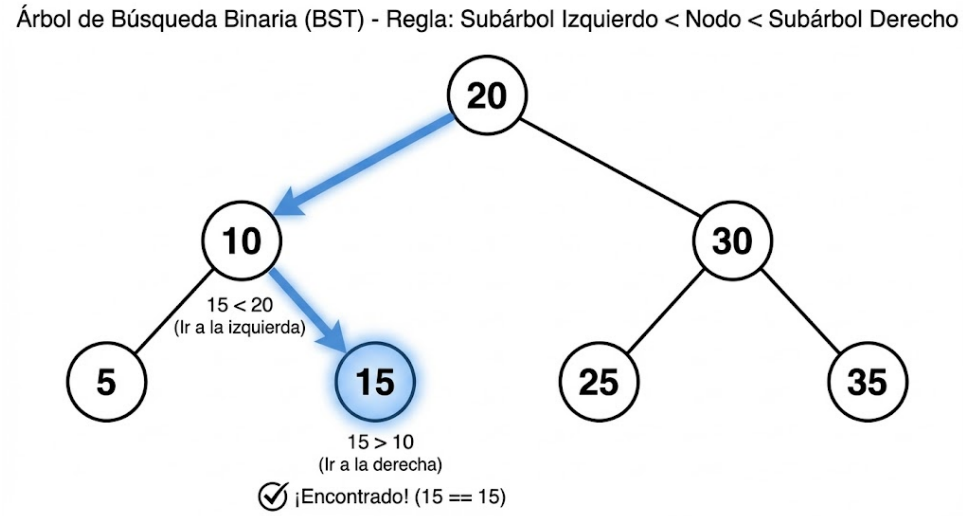


Ilustración 4: Árbol de búsqueda binaria (BST): esquema ilustrativo del criterio de orden y recorrido para la búsqueda de elementos. Imagen creada en Nano Banana con información proporcionada por la autora.

2.1 Inserción en un árbol de búsqueda binaria

Énfasis

La inserción en un BST consiste en ubicar una nueva clave respetando la propiedad de orden del árbol.

El proceso comienza en la raíz y avanza recursiva o iterativamente hacia el subárbol izquierdo o derecho según la comparación entre la clave a insertar y la clave del nodo actual.

Desde el punto de vista del costo computacional, la complejidad de la inserción depende de la altura del árbol. En un árbol balanceado, la inserción tiene un costo O (log n), mientras que en el peor caso (árbol degenerado), puede alcanzar O(n) (Knuth, 1998).

2.1.1 Ejemplo 1: Inserción en un BST

Este ejemplo muestra cómo se construye un BST a partir de inserciones sucesivas, permitiendo observar cómo la estructura del árbol depende del orden de llegada de las claves, un aspecto crítico para comprender sus limitaciones en aplicaciones de IA.

```python
class Nodo:
    def __init__(self, clave):
        self.clave = clave
        self.izquierda = None
        self.derecha = None

def insertar(raiz, clave):
    if raiz is None:
        return Nodo(clave)
    if clave < raiz.clave:
        raiz.izquierda = insertar(raiz.izquierda, clave)
    elif clave > raiz.clave:
        raiz.derecha = insertar(raiz.derecha, clave)
    return raiz

# Construcción del árbol
valores = [50, 30, 70, 20, 40, 60, 80]
raiz = None
for v in valores:
    raiz = insertar(raiz, v)
```

Captura 4: Inserción en un árbol de búsqueda binaria (BST): ejemplo de construcción del árbol a partir de una lista de valores. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`class Nodo`	Define la estructura básica del nodo con una clave y referencias a los hijos.
`insertar`	Función recursiva que ubica la nueva clave respetando el orden del BST.
`raiz is None`	Caso base: se crea un nuevo nodo cuando se alcanza una posición vacía.
Comparaciones `<` y `>`	Determinan si se continúa por el subárbol izquierdo o derecho.
Construcción del árbol	Inserta múltiples valores para formar el BST completo.

2.2 Búsqueda en un árbol de búsqueda binaria

Énfasis

La búsqueda en un BST se basa en la propiedad de orden del árbol. A partir de la raíz, se compara la clave buscada con la clave del nodo actual y se decide continuar por el subárbol izquierdo o derecho.

Este mecanismo reduce el espacio de búsqueda de manera progresiva, lo que permite un rendimiento eficiente en árboles balanceados. En sistemas de IA, esta operación es análoga a consultar un índice por clave.

2.2.1 Ejemplo 2: Búsqueda de una clave en un BST

El ejemplo permite observar cómo la estructura jerárquica del BST guía la búsqueda, reforzando la idea de reducción del espacio de búsqueda sin recorrer todos los elementos.

```python
def buscar(raiz, clave):
    if raiz is None:
        return False
    if raiz.clave == clave:
        return True
    if clave < raiz.clave:
        return buscar(raiz.izquierda, clave)
    else:
        return buscar(raiz.derecha, clave)
```

Captura 5: Búsqueda en un árbol de búsqueda binaria (BST): función recursiva para localizar una clave siguiendo la regla de orden del árbol. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`buscar`	Función recursiva que localiza una clave en el árbol.
`raiz is None`	Indica que la clave no se encuentra en el árbol.
Comparación de claves	Determina el subárbol por el cual continuar la búsqueda.
Valores devueltos	`True` si la clave existe, `False` en caso contrario.

2.3 Eliminación en un árbol de búsqueda binaria

Énfasis

La eliminación es la operación más compleja en un BST, ya que debe preservar la propiedad de orden del árbol.

Existen tres casos principales:

1. El nodo por eliminar es una hoja.
2. El nodo tiene un solo hijo.
3. El nodo tiene dos hijos, y debe ser reemplazado por su sucesor o predecesor inorden.

Esta operación es relevante en IA cuando se actualizan índices dinámicos o se eliminan registros obsoletos.

2.3.1 Ejemplo 3: Eliminación de un nodo en un BST

Este ejemplo muestra el caso más complejo (nodo con dos hijos), permitiendo comprender cómo se mantiene la propiedad del BST tras la eliminación.

```python
def minimo(nodo):
    actual = nodo
    while actual.izquierda is not None:
        actual = actual.izquierda
    return actual

def eliminar(raiz, clave):
    if raiz is None:
        return raiz
    if clave < raiz.clave:
        raiz.izquierda = eliminar(raiz.izquierda, clave)
    elif clave > raiz.clave:
        raiz.derecha = eliminar(raiz.derecha, clave)
    else:
        if raiz.izquierda is None:
            return raiz.derecha
        if raiz.derecha is None:
            return raiz.izquierda
        sucesor = minimo(raiz.derecha)
        raiz.clave = sucesor.clave
        raiz.derecha = eliminar(raiz.derecha, sucesor.clave)
    return raiz
```

Captura 6: Eliminación de un nodo en un árbol de búsqueda binaria (BST): implementación recursiva que contempla los distintos casos estructurales y el uso del sucesor inorden. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`minimo`	Localiza el nodo con la clave mínima en un subárbol.
Casos base	Maneja nodos hoja y nodos con un solo hijo.
Sucesor inorden	Reemplaza la clave del nodo eliminado cuando tiene dos hijos.
Preservación del orden	Garantiza que el BST conserve su propiedad estructural.

2.4 BST como estructura de indexación y soporte de índices

Énfasis

La indexación consiste en organizar datos de manera que puedan recuperarse eficientemente mediante una clave.

Un BST puede entenderse como una forma básica de índice en memoria, donde cada clave conduce a un valor asociado.

En sistemas de Inteligencia Artificial, los índices permiten acelerar búsquedas sobre grandes volúmenes de datos, como identificadores, rankings o resultados intermedios. Aunque en sistemas productivos se utilizan estructuras más avanzadas (B-trees, índices hash), el BST constituye un modelo conceptual fundamental para comprender estos mecanismos (Cormen et al., 2022).

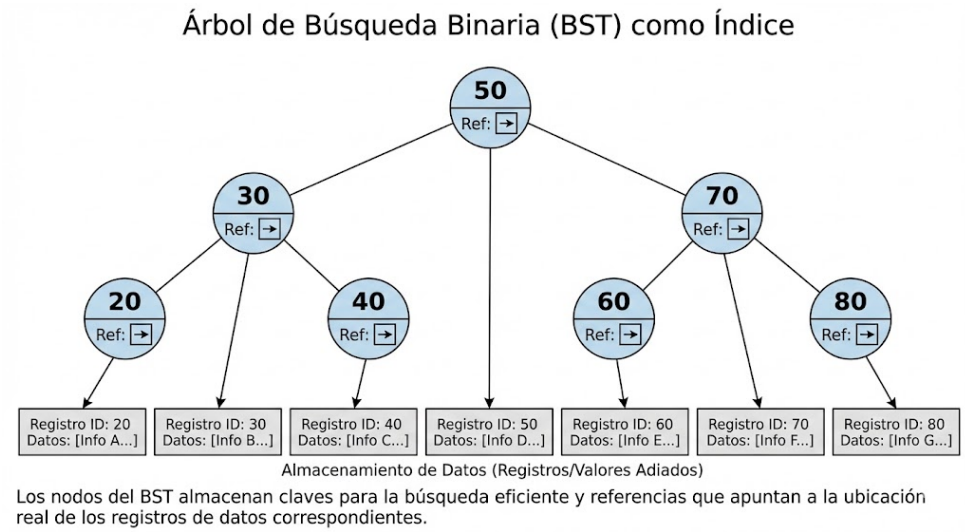


Ilustración 5: Árbol de búsqueda binaria (BST) utilizado como estructura de indexación para el acceso eficiente a registros de datos. Imagen creada en Nano Banana con información proporcionada por la autora.

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 2: Árboles binarios de búsqueda.**

3. Principios de hashing y tablas de dispersión

3.1 Tabla de dispersión

Énfasis

Una tabla de dispersión es una estructura de datos que permite almacenar y recuperar pares (clave, valor) mediante el uso de una función hash, la cual transforma una clave en una posición (índice) dentro de una estructura subyacente, típicamente un arreglo.

Desde el punto de vista algorítmico, el objetivo del hashing es lograr operaciones de inserción, búsqueda y eliminación en tiempo promedio O (1), lo que resulta crítico en sistemas de IA donde se gestionan grandes volúmenes de datos, caches de modelos, diccionarios de características o tablas de conteo.

3.2 Fusión hash

Énfasis

Una función hash es una función determinística que transforma una clave perteneciente a un dominio amplio y potencialmente infinito (por ejemplo, cadenas de texto, números grandes, tuplas) en un valor entero perteneciente a un rango finito de índices, generalmente asociado a las posiciones de una tabla de dispersión.

El carácter determinístico implica que **una misma clave siempre produce el mismo valor hash**, condición indispensable para garantizar la corrección de las operaciones de búsqueda, inserción y eliminación en una tabla hash. Sin esta propiedad, no sería posible localizar de manera consistente un elemento previamente almacenado.

Bajo la óptica algorítmica, la función hash constituye el núcleo lógico de una tabla de dispersión, ya que de su calidad depende directamente el rendimiento global de la estructura.

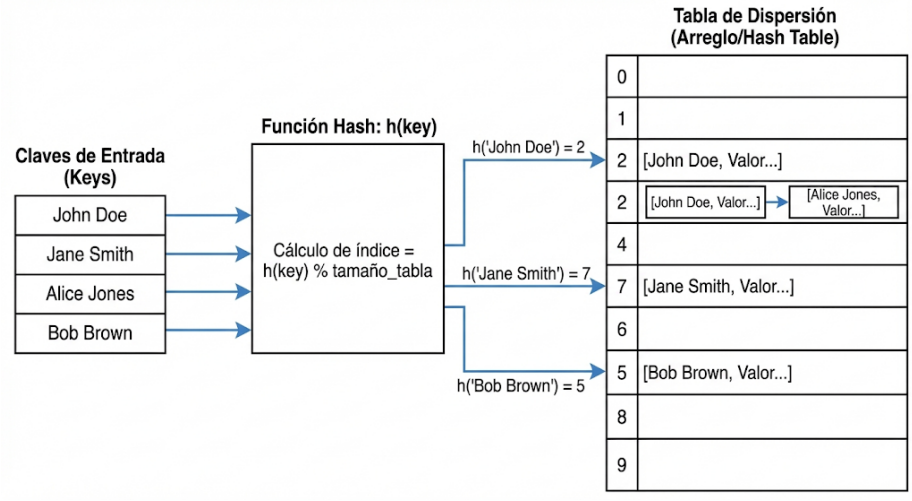


Ilustración 6: Esquema conceptual del funcionamiento de una tabla de dispersión (hash table) y la aplicación de la función hash sobre claves de entrada. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.3 Rapidez de cómputo

Una función hash debe ser computacionalmente eficiente, es decir, su cálculo debe realizarse en tiempo constante o cuasi constante, incluso cuando las claves son complejas. Esta propiedad es crítica porque la función hash se evalúa en cada operación sobre la tabla.

Énfasis

En contextos de Inteligencia Artificial —por ejemplo, durante el procesamiento masivo de datos, la indexación de características o la inferencia en tiempo real— una función hash costosa puede convertirse en un cuello de botella y terminar anulando las ventajas teóricas de la tabla de dispersión.

Por este motivo, en la práctica se privilegian funciones hash basadas en operaciones aritméticas simples y combinaciones bit a bit, evitando cálculos innecesariamente complejos que afecten el rendimiento.

3.4 Distribución uniforme de las claves

Una propiedad central de una buena función hash es la distribución uniforme de las claves sobre el rango de índices disponible. Idealmente, cada posición de la tabla debería recibir aproximadamente la misma cantidad de claves. Esta distribución equilibrada permite reducir la probabilidad de colisiones, mantener el tiempo promedio de acceso cercano a O(1) y garantizar un uso más eficiente de la memoria.

Cuando la función hash no distribuye adecuadamente las claves, se generan zonas de congestión dentro de la tabla, lo que degrada el rendimiento y puede llevar a comportamientos cercanos a O(n), especialmente en escenarios de gran escala habituales en IA.

3.5 Minimización de colisiones

Dado que el dominio de claves suele ser mucho mayor que el rango de índices, las colisiones son inevitables desde un punto de vista matemático. Sin embargo, una función hash bien diseñada debe minimizar la frecuencia de colisiones y evitar patrones sistemáticos que las favorezcan.

La minimización de colisiones no implica su eliminación total, sino su **control estadístico**, de modo que las técnicas de resolución de colisiones (encadenamiento o direccionamiento abierto) puedan operar de manera eficiente.

En sistemas de Inteligencia Artificial, una alta tasa de colisiones puede traducirse en un incremento de la latencia en las búsquedas, un mayor consumo de memoria y dificultades de escalabilidad en entornos de producción.

3.6 Relación entre función hash y diseño del sistema

La elección de una función hash adecuada no constituye una decisión meramente técnica, sino una definición clave dentro del diseño de la arquitectura algorítmica. En aplicaciones de Inteligencia Artificial, donde confluyen grandes volúmenes de datos, exigencias de tiempo real y restricciones de recursos, la función hash influye directamente en la eficiencia global del sistema, en la estabilidad del rendimiento y en la previsibilidad de su comportamiento ante cargas elevadas.

Por esta razón, los lenguajes y bibliotecas modernas —como Python— incorporan implementaciones de funciones hash robustas y optimizadas, sin embargo, comprender sus principios resulta esencial para reconocer sus límites y emplearlas de manera adecuada.

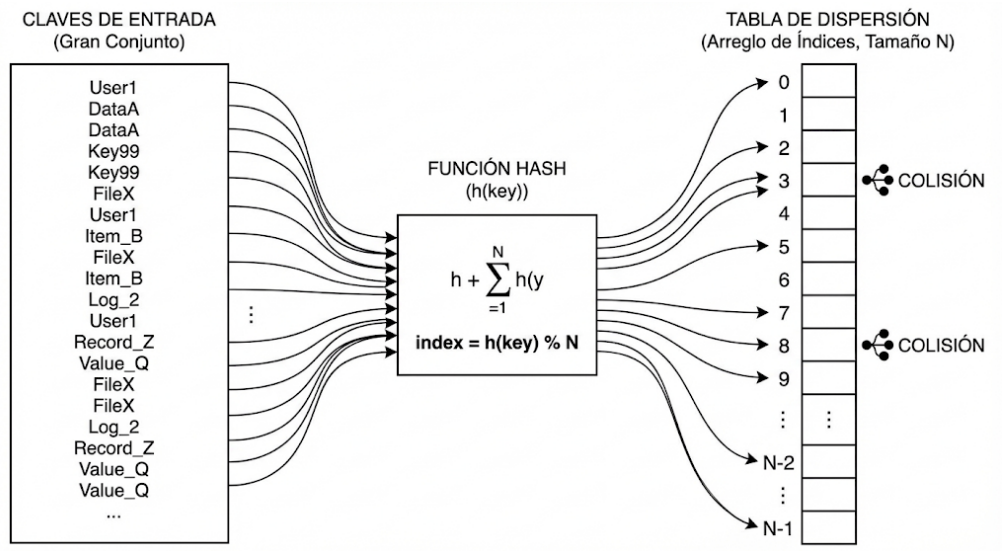


Ilustración 7: Esquema conceptual del proceso de hashing y distribución de claves en una tabla de dispersión, incluyendo la aparición de colisiones. Imagen creada usando herramientas de IA, a partir de información proporcionada por la autora.

3.7 Función hash: ejemplos prácticos y análisis comparativo

Desde el punto de vista del diseño de estructuras de datos, no todas las funciones hash son equivalentes. Una función hash puede ser correcta desde el punto de vista funcional, pero resultar deficiente desde el punto de vista algorítmico si no distribuye adecuadamente las claves o genera un número elevado de colisiones.

Este contraste puede observarse con mayor claridad en los siguientes ejemplos, que permiten analizar su impacto en el rendimiento:

3.7.1 Ejemplo 1: Función hash inadecuada

```python
def bad_hash(key):
    return key % 10

table = [[] for _ in range(10)]

values = [10, 20, 30, 40, 50]

for v in values:
    index = bad_hash(v)
    table[index].append(v)

print(table)

```

Captura 7: Ejemplo de función hash deficiente y colisiones resultantes en una tabla de dispersión. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`bad_hash`	Función hash simplista basada en módulo fijo
`key % 10`	Produce siempre el mismo índice para múltiplos de 10
`table`	Tabla hash con encadenamiento
`append`	Inserta todos los valores en un único bucket
Resultado	Alta concentración de colisiones

Esta función hash viola el principio de distribución uniforme, ya que distintas claves se asignan sistemáticamente al mismo índice. En un sistema de IA, este comportamiento podría generar accesos lineales repetidos, anulando la ventaja del hashing.

3.7.2 Ejemplo 2: Función hash más adecuada

Este ejemplo muestra una función hash mejor diseñada, que introduce mayor variabilidad y favorece una distribución más uniforme de las claves.

```python
def better_hash(key, size):
    return (key * 31) % size

table = [[] for _ in range(10)]

values = [10, 20, 30, 40, 50]

for v in values:
    index = better_hash(v, 10)
    table[index].append(v)

print(table)
```

Captura 8: Ejemplo de función hash mejorada para una distribución más uniforme de claves. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`better_hash`	Función hash con factor multiplicativo
`key * 31`	Introduce dispersión adicional
`% size`	Ajusta el índice al tamaño de la tabla
`table`	Tabla hash con encadenamiento
Resultado	Menor concentración de colisiones

La introducción de un factor multiplicativo mejora la distribución estadística de las claves, reduciendo colisiones. Este principio se utiliza ampliamente en implementaciones reales de funciones hash.

3.7.3 Ejemplo 3: Función hash incorporada en Python

Python implementa internamente funciones hash robustas, diseñadas para equilibrar eficiencia, seguridad y distribución uniforme.

```python
keys = ["model", "dataset", "epoch", "loss", "accuracy"]

for k in keys:
    print(k, "→", hash(k))

```

Captura 9: Ejemplo de cálculo de valores hash para distintas claves en Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`keys`	Claves típicas en sistemas de IA
`hash(k)`	Función hash nativa de Python
Resultado	Valores enteros distribuidos
Propósito	Indexación eficiente en diccionarios

Énfasis

Python aplica técnicas adicionales (como hash randomization) para prevenir ataques y garantizar estabilidad en entornos reales.

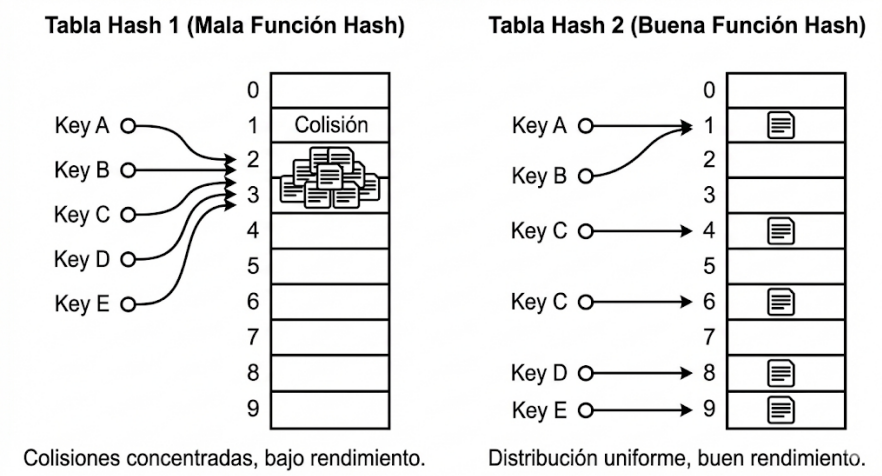


Ilustración 8: Comparación entre una tabla hash con mala función de dispersión y otra con buena función hash, evidenciando el impacto en colisiones y rendimiento. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.8 Uso de diccionarios en Python como tablas hash

En Python, el tipo de dato *dict* implementa internamente una tabla de dispersión altamente optimizada, lo que lo convierte en una herramienta central para el desarrollo de soluciones en IA, como el manejo de frecuencias, mapeos, cachés o embeddings indexados.

Énfasis

En términos estructurales, un diccionario es una colección mutable que almacena pares (clave, valor), donde las claves deben ser hashables, es decir, inmutables y con una función hash definida.

3.8.1 Ejemplo 4: Creación y acceso a un diccionario

Este ejemplo muestra el uso básico de una tabla hash: inserción y acceso en tiempo constante promedio, principio fundamental del hashing.

```python
# Ejemplo básico de diccionario en Python
model_metrics = {
    "accuracy": 0.91,
    "precision": 0.88,
    "recall": 0.86
}

print(model_metrics["accuracy"])

```

Captura 10: Ejemplo básico de uso de un diccionario en Python para almacenar y acceder a métricas de un modelo. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`model_metrics`	Diccionario que actúa como tabla de dispersión
Claves (`"accuracy"` , etc.)	Cadenas hashables usadas como claves
Valores numéricos	Métricas asociadas a cada clave
Acceso `["accuracy"]`	Búsqueda O(1) promedio mediante hashing

3.9 Funciones de transformación de claves

Énfasis

Una clave es hashable si es inmutable, mantiene un valor hash constante durante toda su vida útil e implementa correctamente los métodos *hash()* y *eq()*.

En este sentido, Python proporciona la función hash(), que permite obtener el valor hash de un objeto y verificar su comportamiento dentro de estructuras como los diccionarios.

3.9.1 Ejemplo 5: Uso de la función hash ()

Este ejemplo muestra cómo Python transforma claves en valores enteros utilizados internamente por la tabla hash.

```python
key = "neural_network"
print(hash(key))

```

Captura 11: Ejemplo de aplicación de la función hash () en Python para obtener el valor hash de una clave. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`key`	Clave candidata para una tabla hash
`hash(key)`	Devuelve un entero que representa el hash
Resultado	Usado internamente para indexar la tabla

Observación: el valor puede variar entre ejecuciones por razones de seguridad (hash randomization).

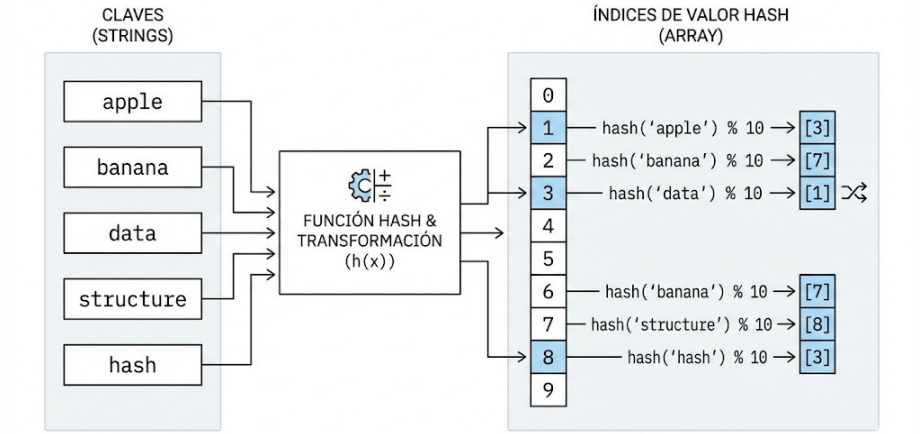


Ilustración 9: Esquema conceptual del funcionamiento de una función hash aplicada a claves tipo string y su asignación a índices en una tabla. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.10 Colisiones y técnicas de resolución

Énfasis

Una colisión ocurre cuando dos claves distintas producen el mismo índice en la tabla hash. La gestión eficiente de colisiones es esencial para preservar el rendimiento.

Las técnicas clásicas incluyen:

- Encadenamiento (chaining).
- Dirección abierta (open addressing).

Python utiliza una variante avanzada de direccionamiento abierto.

3.10.1 Ejemplo 6: Simulación simple de colisiones con encadenamiento

Este ejemplo muestra explícitamente cómo varias claves pueden compartir una misma posición mediante listas.

```python
# Tabla hash simple con encadenamiento
table = [[] for _ in range(5)]

def simple_hash(key):
    return key % 5

values = [10, 15, 20]

for v in values:
    index = simple_hash(v)
    table[index].append(v)

print(table)
```

Captura 12: Ejemplo de implementación de una tabla hash simple con encadenamiento para la resolución de colisiones. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`table`	Arreglo de listas (encadenamiento)
`simple_hash`	Función hash simplificada
`key % 5`	Genera colisiones deliberadas
`append`	Inserta en la lista del índice
Resultado	Múltiples valores en un mismo índice

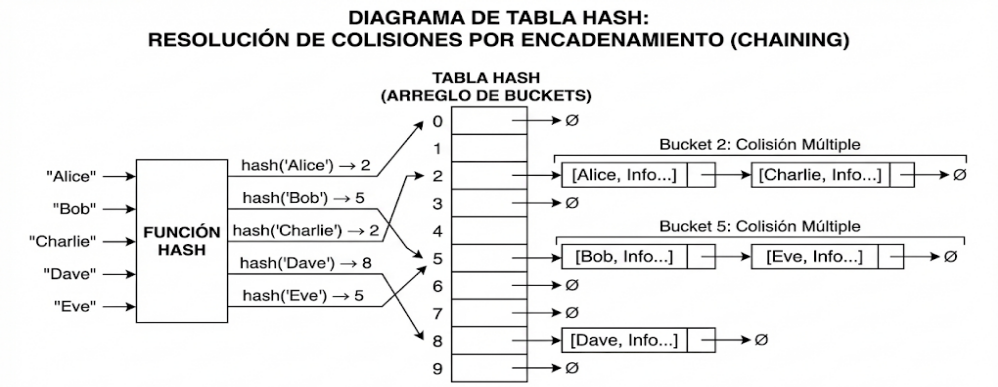


Figura 1: Estructura de tabla hash utilizando listas enlazadas para almacenar múltiples elementos en el mismo índice (bucket) debido a colisiones.

Ilustración 10: Diagrama de una tabla hash con resolución de colisiones por encadenamiento (chaining). Imagen creada usando herramientas de IA, a partir de información proporcionada por la autora.

3.11 Relevancia del hashing en Inteligencia Artificial

En el ámbito de la Inteligencia Artificial, el hashing constituye un mecanismo estructural clave para garantizar eficiencia computacional, escalabilidad y viabilidad práctica de los sistemas inteligentes. Dado que los modelos de IA suelen operar sobre grandes volúmenes de datos, con altas exigencias de acceso rápido y repetido, las tablas de dispersión permiten resolver uno de los problemas centrales del cómputo: la localización eficiente de información relevante.

Desde una perspectiva algorítmica, el hashing permite reducir el costo de operaciones críticas —búsqueda, inserción y actualización— a un tiempo promedio constante O (1), lo cual resulta esencial en contextos donde estas operaciones se ejecutan millones de veces durante el entrenamiento o la inferencia.

3.12 Conteo de frecuencias y representación de texto (Bag of Words)

En tareas de Procesamiento de Lenguaje Natural (PLN), el hashing se utiliza extensivamente para el conteo de frecuencias de términos, como en el modelo *bag of words*. En este enfoque, cada palabra se asocia a una clave dentro de una tabla hash, y su frecuencia se incrementa dinámicamente.

El uso de hashing permite procesar corpus extensos sin necesidad de mantener estructuras ordenadas, actualizar frecuencias en tiempo constante y reducir el impacto del crecimiento del vocabulario.

Este principio se mantiene incluso en variantes más avanzadas, como el hashing trick, donde se evita explícitamente almacenar el vocabulario completo, sacrificando interpretabilidad en favor de eficiencia (Weinberger et al., 2009).

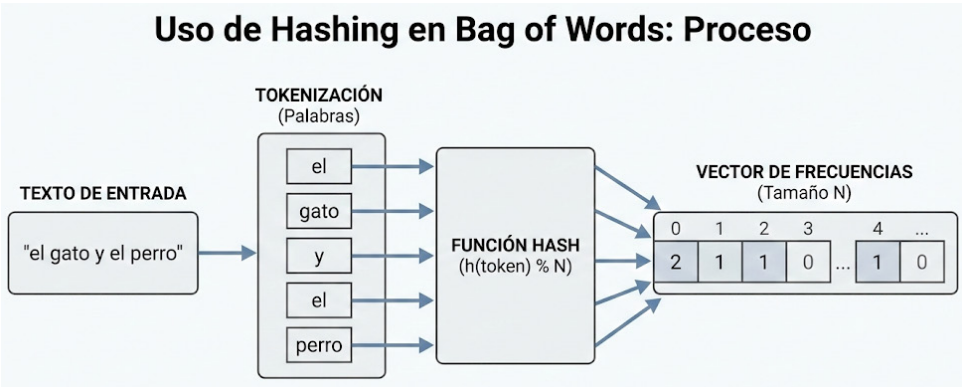


Ilustración 11: Proceso de uso de hashing en el modelo Bag of Words, desde la tokenización del texto hasta la construcción del vector de frecuencias. Imagen creada usando herramientas de IA, a partir de información proporcionada por la autora.

3.13 Indexación y recuperación de embeddings

En sistemas modernos de IA, especialmente aquellos basados en modelos de lenguaje y visión, los datos suelen representarse como embeddings vectoriales de alta dimensión. En este contexto, el hashing cumple aquí un rol fundamental en la indexación eficiente de estos vectores para tareas de búsqueda y recuperación.

Las tablas hash permiten asociar identificadores simbólicos a embeddings, construir estructuras auxiliares para búsqueda aproximada e implementar mecanismos de acceso rápido a memorias vectoriales.

De este modo, en sistemas de recuperación semántica, el hashing contribuye a reducir la latencia en la búsqueda de representaciones relevantes, aspecto crítico en aplicaciones interactivas.

3.14 Cachés de inferencia y reutilización de resultados

Durante la inferencia de modelos de IA, especialmente en sistemas en producción, es frecuente que consultas similares se repitan. En estos casos, el hashing se utiliza para implementar cachés de inferencia, donde la entrada (o una representación de ella) se transforma mediante una función hash y se asocia al resultado previamente calculado.

Este mecanismo permite evitar cómputos redundantes, reducir el uso de recursos de hardware y mejorar tiempos de respuesta del sistema. La eficiencia del caché depende directamente de la calidad de la función hash y del manejo adecuado de colisiones.

3.15 Tablas de símbolos y memoria de agentes inteligentes

En sistemas de agentes inteligentes, el hashing se emplea para implementar tablas de símbolos, memorias de estados, reglas o experiencias previas.

Estas estructuras permiten al agente:

- Recuperar rápidamente información relevante del entorno.
- Mantener asociaciones entre estados y acciones.
- Actualizar su memoria de manera eficiente.

En contextos de aprendizaje por refuerzo, por ejemplo, las tablas hash se utilizan para representar aproximaciones discretas del espacio de estados cuando este es demasiado grande para estructuras tradicionales.

3.16 Impacto de colisiones y diseño de funciones hash en IA

Una mala función hash o una alta tasa de colisiones puede degradar significativamente el rendimiento de un sistema inteligente, transformando operaciones esperadas en O (1) en comportamientos cercanos a O(n).

En sistemas de Inteligencia Artificial, esto puede traducirse en un aumento de la latencia, un consumo excesivo de memoria, una pérdida de escalabilidad y comportamientos impredecibles en tiempo real.

Por este motivo, el diseño y la selección de funciones hash adecuadas no es un detalle menor, sino una decisión de arquitectura algorítmica que impacta directamente en la calidad y viabilidad del sistema.

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 3: Hashing en IA.**

4. Algoritmos de ordenamiento

Énfasis

Un algoritmo de ordenamiento es un procedimiento sistemático que reorganiza los elementos de una colección según un criterio total (ascendente o descendente), basándose en una relación de comparación.

El ordenamiento constituye una operación estructural central en informática e Inteligencia Artificial, ya que numerosos algoritmos de búsqueda, análisis estadístico, aprendizaje automático y procesamiento de datos presuponen o se benefician de conjuntos de datos previamente ordenados.

Desde una perspectiva algorítmica, su estudio se centra en la complejidad temporal, el uso de memoria, la estabilidad y la adaptabilidad a grandes volúmenes de datos, criterios que permiten evaluar su desempeño en distintos contextos de aplicación.

4.1 Ordenamiento por selección (Selection Sort)

El ordenamiento por selección se basa en la idea de seleccionar repetidamente el elemento mínimo del segmento no ordenado y colocarlo en su posición definitiva. Se caracteriza por su simplicidad conceptual, aunque presenta una complejidad temporal cuadrática.

Este algoritmo resulta útil para comprender el concepto de ordenamiento in situ y la separación progresiva entre zona ordenada y no ordenada.

4.1.1 Ejemplo 1: Selection Sort

```
1  def selection_sort(arr):
2      n = len(arr)
3      for i in range(n):
4          min_idx = i
5          for j in range(i + 1, n):
6              if arr[j] < arr[min_idx]:
7                  min_idx = j
8          arr[i], arr[min_idx] = arr[min_idx], arr[i]
9
```

Captura 13: Implementación del algoritmo selection sort en Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`i`	Marca el inicio del segmento no ordenado
`min_idx`	Índice del mínimo encontrado
Bucle interno	Busca el menor elemento
Intercambio	Coloca el mínimo en su posición
Tipo	Ordenamiento in situ

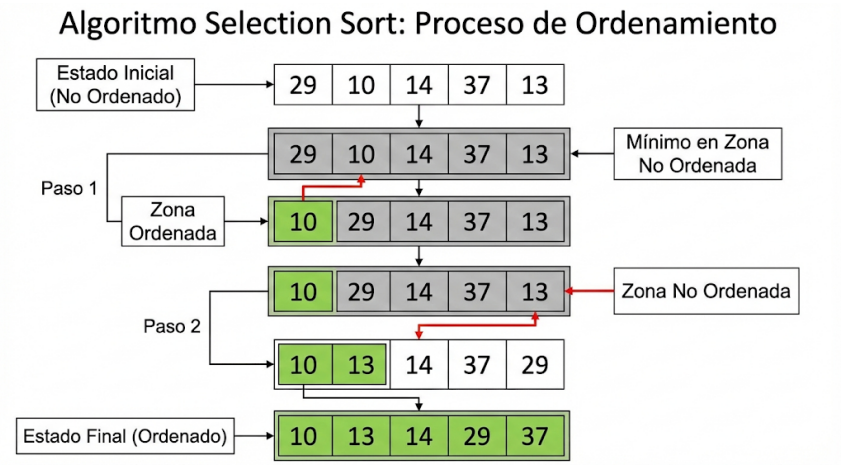


Ilustración 12: Proceso de ordenamiento del algoritmo Selection Sort, mostrando la selección iterativa del mínimo y la construcción de la zona ordenada. Imagen creada usando herramientas de IA con información proporcionada por la autora.

4.2 Ordenamiento por inserción (Insertion Sort)

El ordenamiento por inserción construye progresivamente una lista ordenada al insertar cada nuevo elemento en la posición que le corresponde. Por ello, resulta eficiente en listas pequeñas o en aquellas que se encuentran casi ordenadas.

4.2.1 Ejemplo 2: Insertion Sort

Este algoritmo emula el comportamiento humano al ordenar elementos manualmente.

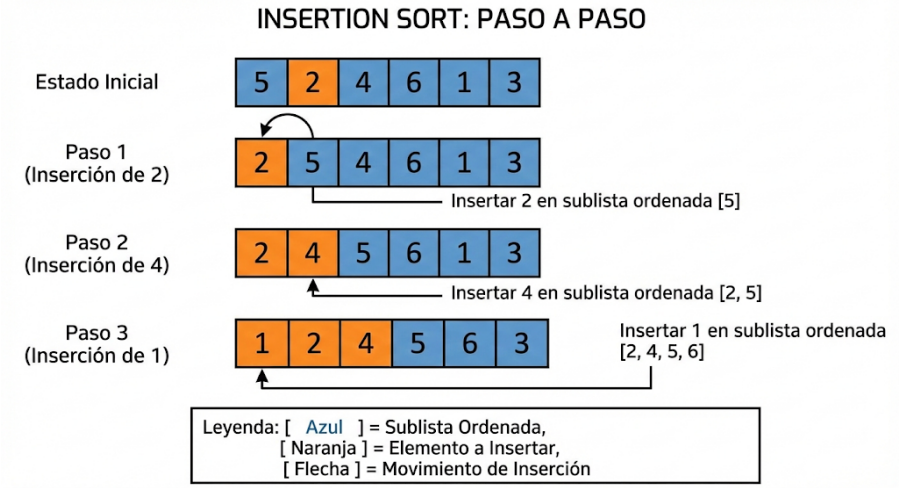
```
1  def insertion_sort(arr):
2      for i in range(1, len(arr)):
3          key = arr[i]
4          j = i - 1
5          while j >= 0 and key < arr[j]:
6              arr[j + 1] = arr[j]
7              j -= 1
8          arr[j + 1] = key
9
```

Captura 14: Implementación del algoritmo Insertion Sort en Python. Desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`key`	Elemento a insertar
`while`	Desplaza elementos mayores
Inserción	Coloca `key` en posición correcta
Estabilidad	Algoritmo estable



Captura 15: Proceso paso a paso del algoritmo Insertion Sort, mostrando la inserción progresiva de elementos en la sublista ordenada. Imagen creada usando herramientas de IA con información proporcionada por la autora.

4.3 Ordenamiento burbuja (Bubble Sort)

El ordenamiento burbuja compara pares adyacentes e intercambia aquellos que están fuera de orden, repitiendo el proceso hasta que la lista queda ordenada.

4.3.1 Ejemplo 3: Bubble Sort

Aunque ineficiente, es valioso desde el punto de vista pedagógico para introducir comparaciones locales.

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

```

Captura 16: Implementación del algoritmo Bubble Sort en Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
Comparación	Se comparan elementos adyacentes
Intercambio	Corrige desorden local
Iteraciones	Reduce el rango en cada pasada
Complejidad	$O(n^2)$

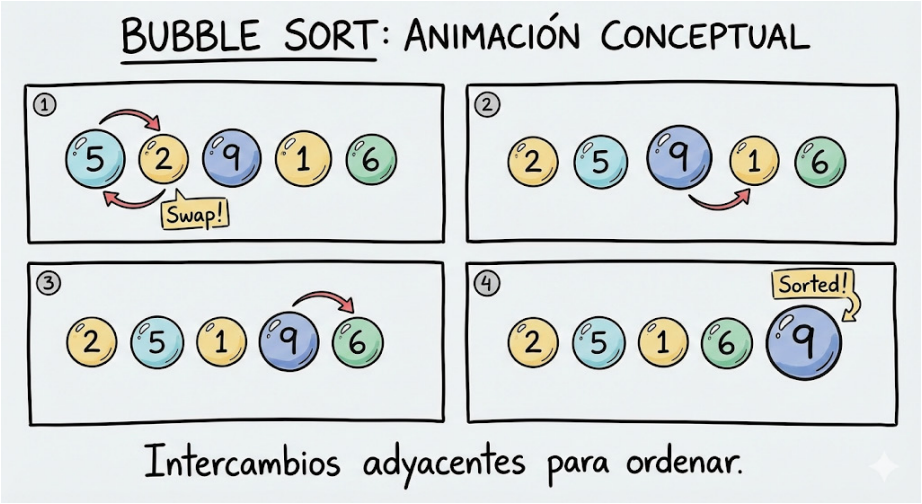


Ilustración 13: Animación conceptual del algoritmo Bubble Sort mediante intercambios adyacentes. Imagen creada usando herramientas de IA con información proporcionada por la autora.

4.4 Quicksort

Quicksort es un algoritmo del tipo *divide and conquer* que selecciona un pivote y particiona el arreglo en dos subconjuntos: uno con los elementos menores y otro con los mayores que dicho pivote.

4.4.1 Ejemplo 4 QuickSort

Es uno de los algoritmos más utilizados por su excelente rendimiento promedio.

```python
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)

```

Captura 17: Implementación del algoritmo Quick Sort en Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`pivot`	Elemento central
Partición	Divide en tres subconjuntos
Recursión	Ordena sublistas
Complejidad	O(n log n) promedio

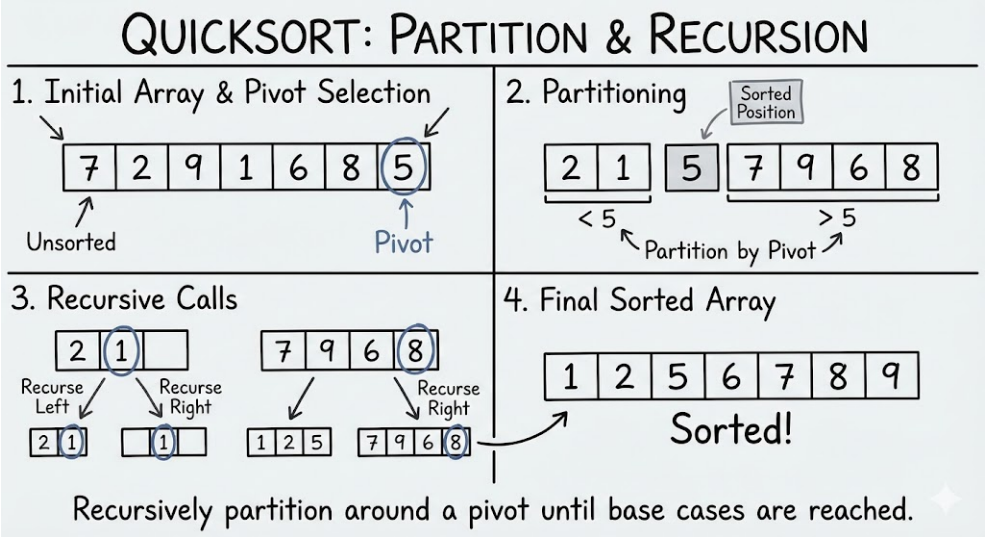


Ilustración 14: Esquema conceptual del algoritmo QuickSort, ilustrando la partición y la recursión hasta alcanzar el arreglo ordenado. Imagen creada usando herramientas de IA con información proporcionada por la autora.

4.5 Mergesort

Mergesort es un algoritmo de ordenamiento estable basado en la estrategia divide and conquer, que divide el arreglo en sublistas más pequeñas y luego las fusiona de manera ordenada para obtener la secuencia final.

4.5.1 Ejemplo Mergesort

Garantiza complejidad O (n log n) incluso en el peor caso.

```python
def mergesort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = mergesort(arr[:mid])
    right = mergesort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

```

Captura 18: Implementación del algoritmo Merge Sort en Python. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
División	Se divide recursivamente
`merge`	Fusiona listas ordenadas
Estabilidad	Preserva orden relativo
Uso	Ideal para grandes volúmenes

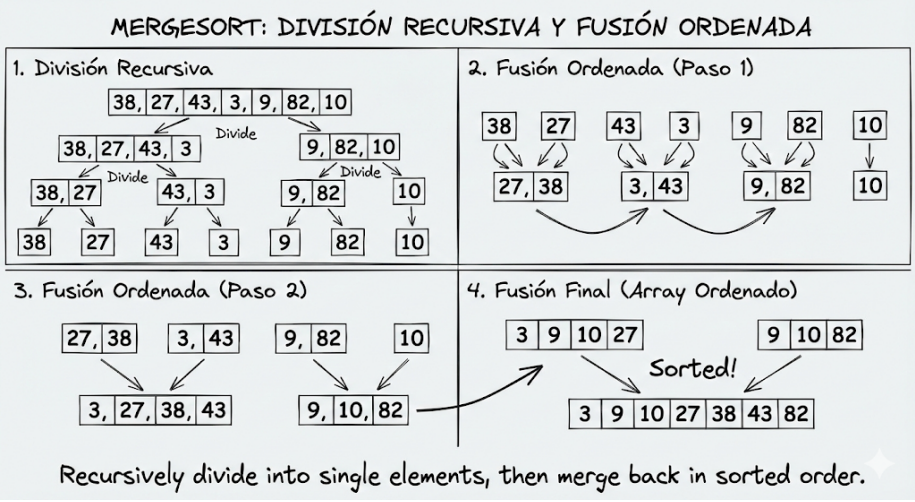


Ilustración 15: Esquema conceptual del algoritmo Merge Sort, mostrando la división recursiva y la fusión ordenada de los elementos. Imagen creada usando herramientas de IA con información proporcionada por la autora.

4.6 Comparación con sorted() y list.sort()

Python utiliza Timsort, un algoritmo híbrido derivado de mergesort e insertion sort, diseñado y optimizado para ofrecer un alto rendimiento en datos reales.

4.6.1 Ejemplo Sorted()

```
1   data = [5, 2, 9, 1]
2   print(sorted(data))
3   data.sort()
```

Captura 19: Ejemplo del uso de sorted() y list.sort() en Python para ordenar una lista. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
`sorted()`	Devuelve nueva lista
`list.sort()`	Ordena in situ
Algoritmo	Timsort
Uso	Recomendado en producción

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 4: Laboratorio de algoritmos de ordenamiento.**

5. Algoritmos de emparejamiento y fusión de listas ordenadas

5.1 Introducción teórica general

Los algoritmos de emparejamiento y fusión de listas ordenadas constituyen una familia de técnicas fundamentales en el procesamiento eficiente de datos estructurados. Su relevancia es central en contextos de Inteligencia Artificial, donde es habitual trabajar con grandes volúmenes de datos previamente ordenados (por ejemplo, resultados de consultas, rankings, vectores de características, flujos temporales o salidas intermedias de algoritmos de aprendizaje).

Desde el punto de vista algorítmico, estos procedimientos explotan la propiedad de orden para reducir la complejidad temporal, evitando comparaciones redundantes y permitiendo recorridos lineales. Esta idea es clave en algoritmos clásicos como merge sort, en técnicas de join en bases de datos y en procesos de integración de datos heterogéneos, ampliamente utilizados en sistemas inteligentes (Cormen et al., 2009; Knuth, 1998).

5.2 Algoritmos de emparejamiento de listas ordenadas

Un algoritmo de emparejamiento de listas ordenadas tiene como objetivo identificar elementos comunes entre dos secuencias que se encuentran previamente ordenadas, recorriéndolas de manera sincronizada y aprovechando el orden para descartar comparaciones innecesarias.

El principio subyacente es similar al de la búsqueda por avance coordinado: se utilizan dos índices que progresan sobre ambas listas, comparando los elementos actuales y avanzando en función del resultado de la comparación. Esto permite alcanzar una complejidad temporal O(n + m), siendo *n* y *m* las longitudes de las listas, frente a soluciones ingenuas de orden cuadrático.

Este tipo de algoritmo es ampliamente utilizado en intersección de conjuntos, detección de coincidencias, comparación de resultados ordenados y filtrado eficiente en sistemas de recomendación.

Veamos el paso a paso:

5.2.1 Paso 1: Estado inicial

Se tienen dos listas ordenadas. Se coloca un puntero al inicio de cada lista. En este punto, se comparan los elementos a los que apuntan los punteros.

Ilustración 16: Inicio del algoritmo de dos punteros para la comparación de dos listas ordenadas, mostrando la posición inicial de ambos punteros. Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.2.2 Paso 2: Avance del puntero

Como el elemento apuntado por el Puntero 1 (1) es menor que el elemento del Puntero 2 (2), sabemos que el '1' no puede tener una coincidencia en la segunda lista (ya que esta está ordenada y sus elementos solo pueden aumentar). Por lo tanto, avanzamos el Puntero 1 al siguiente elemento.

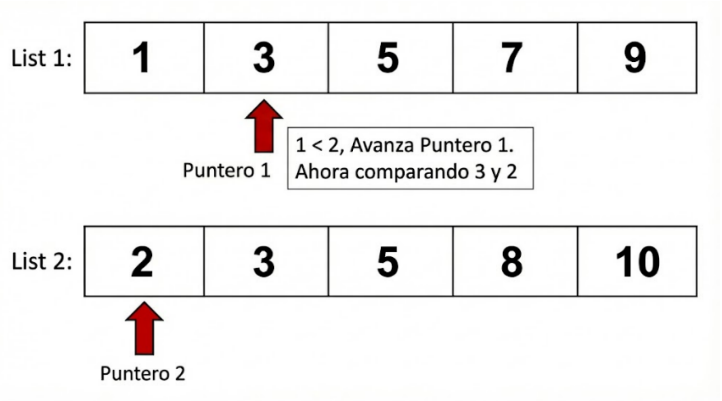


Ilustración 17: Avance del puntero menor durante la comparación secuencial de dos listas ordenadas mediante la técnica de dos punteros. Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.2.3 Paso 3: ¡Elemento común encontrado!

En la nueva comparación, el elemento del Puntero 1 (3) es mayor que el del Puntero 2 (2). Esto indica que debemos avanzar el Puntero 2. Al hacerlo, ambos punteros apuntan al número '3'. Se ha encontrado un elemento común. En este punto, ambos punteros avanzarían para continuar la búsqueda.

Ilustración 18: Ejemplo del uso de la técnica de dos punteros para identificar elementos comunes en listas ordenadas. Imagen creada usando herramientas de IA con información proporcionada por la autora.

Este proceso se repetiría hasta que uno de los punteros llegue al final de su lista.

5.2.4 Ejemplo 1: Emparejamiento de dos listas ordenadas

Este ejemplo muestra el emparejamiento clásico entre dos listas ordenadas de enteros, identificando los valores comunes mediante un recorrido secuencial coordinado. Se trata de una implementación canónica que refleja el fundamento teórico del algoritmo.

```python
def emparejar_listas(a, b):
    i = j = 0
    resultado = []

    while i < len(a) and j < len(b):
        if a[i] == b[j]:
            resultado.append(a[i])
            i += 1
            j += 1
        elif a[i] < b[j]:
            i += 1
        else:
            j += 1

    return resultado
```

Captura 20: Implementación del algoritmo de emparejamiento de dos listas ordenadas mediante la técnica de dos punteros. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
Inicialización de índices	Se utilizan dos índices (`i` y `j`) para recorrer simultáneamente ambas listas ordenadas.
Bucle principal	El recorrido se mantiene mientras ninguno de los índices supere el tamaño de su lista.
Comparación de elementos	Si los elementos coinciden, se agrega el valor al resultado y se avanzan ambos índices.
Avance selectivo	Cuando los valores son distintos, solo se avanza el índice correspondiente al menor valor, aprovechando el orden.
Complejidad	El algoritmo recorre cada lista una sola vez, logrando complejidad O(n + m).

5.3 Algoritmos de fusión de listas ordenadas

La fusión de listas ordenadas consiste en combinar dos secuencias previamente ordenadas en una única lista que preserve el orden global. Este procedimiento es un componente esencial del algoritmo merge sort y aparece de forma recurrente en procesos de integración y consolidación de datos.

Énfasis

A diferencia del emparejamiento, el objetivo no es detectar coincidencias sino integrar la totalidad de los elementos, manteniendo el orden relativo. Nuevamente, el uso de dos punteros permite realizar la operación en tiempo lineal.

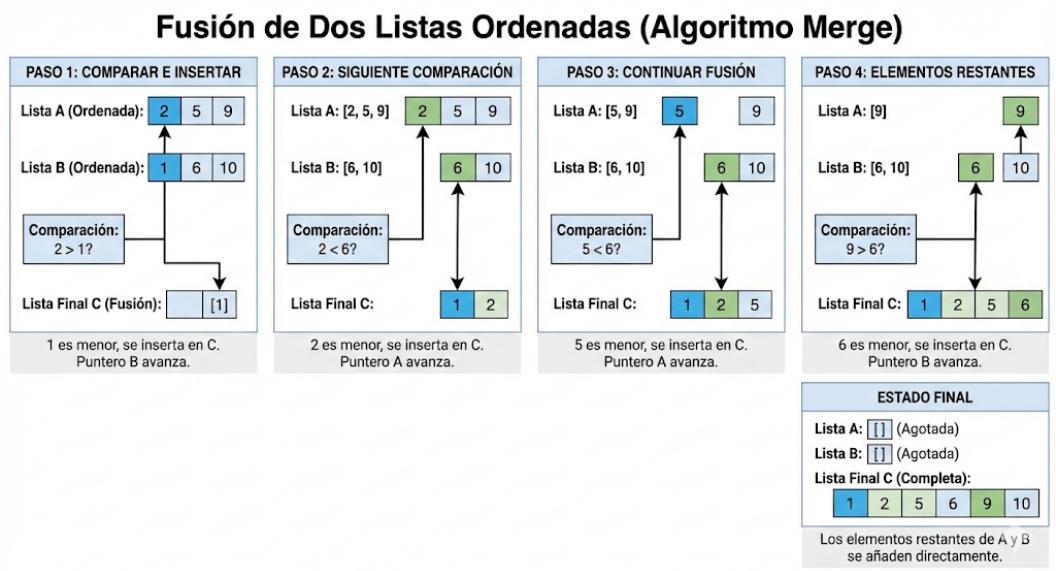


Ilustración 19: Proceso paso a paso de la fusión de dos listas ordenadas mediante el algoritmo merge. Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.3.1 Ejemplo 2: Fusión de dos listas ordenadas

Este ejemplo implementa el algoritmo clásico de fusión, integrando dos listas ordenadas de enteros en una única lista también ordenada. Es una pieza central en el análisis de algoritmos de ordenamiento y procesamiento eficiente de datos.

```python
def fusionar_listas(a, b):
    i = j = 0
    resultado = []

    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            resultado.append(a[i])
            i += 1
        else:
            resultado.append(b[j])
            j += 1

    resultado.extend(a[i:])
    resultado.extend(b[j:])
    return resultado
```

Captura 21: Implementación del algoritmo de fusión de dos listas ordenadas en Python. Desarrollado en Online IDE Pro. Código fuente de la autora.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/6d330a62-ab2f-4d2d-94df-fa7da6f16bdd

Elemento	Explicación
Índices de recorrido	Dos punteros permiten recorrer ambas listas de forma ordenada.
Comparación principal	Se inserta en el resultado el menor de los dos elementos actuales.
Preservación del orden	El orden relativo se mantiene gracias a la comparación sistemática.
Extensión final	Una vez agotada una lista, se agregan directamente los elementos restantes de la otra.
Eficiencia	Complejidad temporal O(n + m), óptima para listas ordenadas.

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 5: Emparejamiento.**

Palabras finales

A lo largo de este módulo se han estudiado algoritmos y estructuras fundamentales para el acceso, la organización y la recuperación eficiente de información, ejes centrales en el desarrollo de sistemas de Inteligencia Artificial. El recorrido comenzó con los algoritmos básicos de búsqueda, avanzó hacia estrategias indexadas y estructuras jerárquicas, incorporó el análisis y la comparación de algoritmos de ordenamiento, y culminó con el estudio del hashing como mecanismo clave para lograr acceso eficiente a gran escala.

El análisis de la complejidad computacional permitió comprender cómo las decisiones algorítmicas impactan directamente en el rendimiento, la escalabilidad y la viabilidad práctica de las soluciones. Asimismo, la implementación en Python de los distintos algoritmos y estructuras facilitó la articulación entre los conceptos teóricos y su aplicación concreta en contextos representativos de la Inteligencia Artificial, como la indexación de datos, el procesamiento de grandes volúmenes de información y la optimización de operaciones de búsqueda y actualización.

Este módulo sienta las bases necesarias para abordar estructuras más complejas, en las que la eficiencia depende no solo del algoritmo utilizado, sino también de la forma en que se mantiene el equilibrio, la altura y la organización interna de los datos. En este sentido, el Módulo 4: Árboles y estructuras avanzadas profundiza en estas problemáticas mediante el estudio de árboles balanceados y otras estructuras que extienden los conceptos trabajados, permitiendo comprender cómo se diseñan y utilizan índices más robustos y eficientes en sistemas de Inteligencia Artificial de mayor escala y complejidad.

Bibliografía consultada por el docente

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley.
- Mit.edu. Recuperado 20 de enero de 2026, de https://tinyurl.com/23jewf69
- Mcgill.ca. Recuperado 20 de enero de 2026, de https://www.cs.mcgill.ca/~jeromew/teachings/251/F2022/COMP251_Lecture7_F2022.pdf
- Python Software Foundation. (s. f.). *bisect — Array bisection algorithm*. https://docs.python.org/3/library/bisect.html
- *The AVL Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/AVL.html
- *The Red-Black Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/RedBlack.html
- *Red-Black Trees*. (s. f.). Opendatastructures.org. Recuperado 20 de enero de 2026, de https://opendatastructures.org/newhtml/ods/latex/redblack.html

Módulo 3 | Glosario

- **Algoritmo:** Conjunto finito y ordenado de pasos bien definidos que permiten resolver un problema o realizar una tarea computacional. En Inteligencia Artificial, los algoritmos determinan cómo se procesan, organizan y recuperan los datos.

- **Análisis de complejidad:** Estudio del costo computacional de un algoritmo en términos de tiempo y/o espacio, generalmente expresado mediante notación O grande. Permite comparar la eficiencia de distintas soluciones.

- **Árbol de búsqueda binaria (BST):** Estructura de datos jerárquica en la que cada nodo cumple la propiedad de orden: los valores del subárbol izquierdo son menores que la clave del nodo y los del subárbol derecho son mayores. Facilita búsquedas, inserciones y eliminaciones eficientes.

- **Búsqueda binaria:** Algoritmo de búsqueda que opera sobre estructuras ordenadas, reduciendo el espacio de búsqueda a la mitad en cada paso. Su complejidad temporal es O(log n).

- **Búsqueda indexada:** Estrategia de búsqueda que utiliza índices o estructuras auxiliares para localizar posiciones relevantes sin recorrer toda la estructura de datos.

- **Búsqueda secuencial (linear search):** Algoritmo que recorre los elementos de una estructura uno por uno hasta encontrar la clave buscada o agotar la colección. No requiere orden previo y presenta complejidad O(n).

- **Cache de inferencia:** Estructura que almacena resultados previos de inferencia de un modelo de IA para evitar cómputos redundantes, mejorando tiempos de respuesta mediante hashing.

- **Chaining (encadenamiento):** Técnica de resolución de colisiones en tablas hash en la que cada posición de la tabla almacena una lista de elementos que comparten el mismo índice.

- **Clave (key):** Elemento que identifica de manera única a un valor dentro de una estructura de datos, especialmente en tablas hash y diccionarios.

- **Colisión:** Situación que ocurre cuando dos claves distintas producen el mismo valor hash y se asignan a la misma posición en una tabla de dispersión.

- **Complejidad temporal:** Medida del tiempo de ejecución de un algoritmo en función del tamaño de la entrada.

- **Diccionario (dict):** Estructura de datos nativa de Python que implementa una tabla hash optimizada para almacenar pares (clave, valor) con acceso promedio en tiempo O(1).

- **Distribución uniforme:** Propiedad deseable de una función hash que reparte las claves de manera equilibrada sobre las posiciones de la tabla, reduciendo colisiones.

- **Divide and conquer:** Paradigma algorítmico que consiste en dividir un problema en subproblemas más pequeños, resolverlos y combinar sus soluciones. Utilizado en quicksort y mergesort.

- **Emparejamiento de listas ordenadas:** Algoritmo que identifica elementos comunes entre dos listas previamente ordenadas mediante un recorrido coordinado con dos punteros.

- **Encadenamiento:** Ver chaining.

- **Estabilidad (en ordenamiento):** Propiedad de un algoritmo de ordenamiento que preserva el orden relativo de elementos con claves iguales.

- **Función hash:** Función determinística que transforma una clave en un valor entero dentro de un rango finito de índices, utilizado para acceder a una tabla de dispersión.

- **Fusión de listas ordenadas:** Proceso de combinar dos listas previamente ordenadas en una única lista ordenada, preservando el orden global de los elementos.

- **Hash randomization:** Técnica utilizada por Python para introducir variabilidad controlada en los valores hash entre ejecuciones, con fines de seguridad.

- **Hashing:** Técnica que permite acceder a datos de forma eficiente mediante el uso de funciones hash y tablas de dispersión.

- **Inserción en BST:** Operación que incorpora una nueva clave en un árbol de búsqueda binaria respetando su propiedad de orden.

- **Índice:** Estructura que permite localizar información de manera eficiente mediante una clave, reduciendo el costo de búsqueda.

- **Lista ordenada:** Colección de elementos dispuestos según un criterio total (ascendente o descendente), condición necesaria para aplicar algoritmos como búsqueda binaria o fusión eficiente.

- **Mergesort:** Algoritmo de ordenamiento estable basado en divide and conquer, con complejidad O(n log n) garantizada.

- **Notación O grande:** Lenguaje matemático utilizado para describir el comportamiento asintótico de un algoritmo.

- **Ordenamiento:** Proceso de reorganizar los elementos de una colección según un criterio de comparación definido.

- **Ordenamiento burbuja (Bubble Sort):** Algoritmo simple que intercambia elementos adyacentes fuera de orden de manera repetitiva. Tiene complejidad $O(n^2)$.

- **Ordenamiento por inserción (Insertion Sort):** Algoritmo que construye una lista

ordenada insertando progresivamente cada elemento en su posición correcta.

- **Ordenamiento por selección (Selection Sort):** Algoritmo que selecciona repetidamente el elemento mínimo del segmento no ordenado y lo coloca en su posición final.
- **Puntero (índice de recorrido):** Variable utilizada para recorrer estructuras de datos, especialmente en algoritmos de emparejamiento y fusión de listas.
- **Quicksort:** Algoritmo de ordenamiento eficiente basado en divide and conquer que particiona la lista a partir de un pivote.
- **Resolución de colisiones:** Conjunto de técnicas destinadas a manejar colisiones en tablas hash, como encadenamiento o direccionamiento abierto.
- **Tabla de dispersión (hash table):** Estructura de datos que almacena pares (clave, valor) y permite operaciones promedio en tiempo constante mediante hashing.
- **Timsort:** Algoritmo de ordenamiento híbrido utilizado internamente por Python en sorted() y list.sort(), derivado de mergesort e insertion sort.
- **Valor (value):** Dato asociado a una clave dentro de una estructura como un diccionario o tabla hash.

Módulo 3 | Actividades

ACTIVIDADES

Consigna

Las actividades de este módulo son interactivas y autoasistidas. Para realizarlas, es necesario acceder al enlace ***disponible en plataforma*** y seguir las instrucciones indicadas. Ante cualquier duda o inconveniente durante el desarrollo, se solicita comunicarse con el/la tutor/a para recibir acompañamiento y orientación.

Módulo 4

Microobjetivos

- Analizar la necesidad de balanceo y las rotaciones en árboles AVL y rojo-negro para garantizar operaciones de búsqueda/actualización con altura acotada en estructuras de indexación y consulta en memoria.
- Diseñar operaciones de búsqueda, inserción y eliminación en árboles B, B+ y B* para minimizar accesos a memoria secundaria y sostener escalabilidad en escenarios de indexación en disco y grandes volúmenes de datos.
- Implementar colas de prioridad con montículos (listas y heapq) y heap sort para seleccionar y ordenar elementos con costos controlados en procesos de planificación, ranking y selección dentro de sistemas computacionales.
- Aplicar la estructura Union-Find con compresión de caminos y unión por rango/tamaño para gestionar particiones dinámicas y consultas de conectividad eficientes en problemas de componentes, detección de ciclos y algoritmos como Kruskal.
- Construir recorridos y búsquedas sobre estructuras mediante recursión avanzada, iteradores y generadores para procesar datos de forma incremental y reducir materialización en memoria en recorridos DFS/BFS, backtracking y exploración de estructuras grandes o implícitas.

Contenidos

ÁRBOLES Y ESTRUCTURAS AVANZADAS

Fuente: Freepik

1. Introducción

Se les da la bienvenida al Módulo 4, orientado al estudio de estructuras jerárquicas y mecanismos algorítmicos que sostienen operaciones eficientes de búsqueda, actualización, priorización y partición de conjuntos. El módulo organiza una progresión desde árboles

binarios auto-balanceados hasta estructuras diseñadas para almacenamiento persistente, priorización y conectividad, integrando además patrones de recursión avanzada y recorridos perezosos para procesamiento incremental.

- En el Tema 1, se aborda el problema de la degradación de los BST y la necesidad de garantizar alturas acotadas mediante AVL y rojo-negro, con énfasis en rotaciones y en la comparación empírica del impacto de la forma del árbol en el costo de búsqueda.

- En el Tema 2, el foco se traslada a la indexación en disco, donde los *árboles B y sus variantes (B+, B*)* modelan un diseño centrado en minimizar accesos a memoria secundaria y habilitar consultas por rango y recorridos secuenciales en hojas.

- El Tema 3 introduce montículos como base para colas de prioridad y heap sort, con implementaciones sobre listas y el uso del módulo heapq.

- El Tema 4 desarrolla Union-Find como representación eficiente de particiones dinámicas, incorporando compresión de caminos y unión por rango/tamaño, y conectando con aplicaciones como Kruskal y conectividad incremental.

- Finalmente, el Tema 5 integra recursión aplicada a estructuras, patrones (divide & conquer, backtracking), y el uso de iteradores y generadores para recorrer estructuras grandes de forma incremental y con control de memoria.

A lo largo del módulo se proponen ejemplos ejecutables (con pruebas simples) y actividades pensadas para que usted pueda avanzar progresivamente desde las definiciones teóricas y los invariantes hacia implementaciones concretas y verificables, priorizando el análisis de costos y la adecuación de cada estructura al modelo de cómputo asumido (RAM vs disco, materialización vs streaming).

2. Árboles balanceados: AVL y rojo-negro; rotaciones y comparación de rendimientos

2.1 ¿Por qué se requieren árboles balanceados en IA?

En un árbol de búsqueda binaria (BST) común, la eficiencia de la búsqueda y la inserción depende de la altura del árbol. Si las inserciones ocurren en orden creciente o casi ordenado, el árbol puede degenerar en una lista y las operaciones pasan de un comportamiento cercano al logarítmico a uno lineal. Esto resulta crítico en IA cuando se implementan índices en memoria o estructuras auxiliares de consulta para pipelines de datos, ya que un desbalance impacta directamente en el rendimiento.

2.2 Objetivo del balanceo

Los árboles balanceados buscan garantizar altura O (log n) mediante reglas de mantenimiento estructural, lo que habilita operaciones de búsqueda, inserción y eliminación en O(log n) en el peor caso, evitando la degradación ante patrones adversos de entrada. Este principio se utiliza como base conceptual para estructuras de indexación más complejas en sistemas reales.

2.3 Árboles AVL

Énfasis

Un árbol AVL es un árbol binario de búsqueda (BST) auto-balanceado que asegura que la diferencia de alturas entre los subárboles izquierdo y derecho de cada nodo se mantenga limitada. Cuando se realiza una inserción o eliminación y se detecta un desbalance, se aplican rotaciones para restaurar esta propiedad.

2.3.1 Factor de balance

Se define como la diferencia de alturas entre subárbol izquierdo y derecho. La condición típica de AVL exige que esta diferencia permanezca en un rango acotado, lo cual impone un balance más estricto que otras familias (por ejemplo, rojo-negro), favoreciendo búsquedas rápidas.

Énfasis

En un árbol balanceado, el factor de balance de cada nodo siempre cumple:

$-1 \leq$ Factor de balance ≤ 1

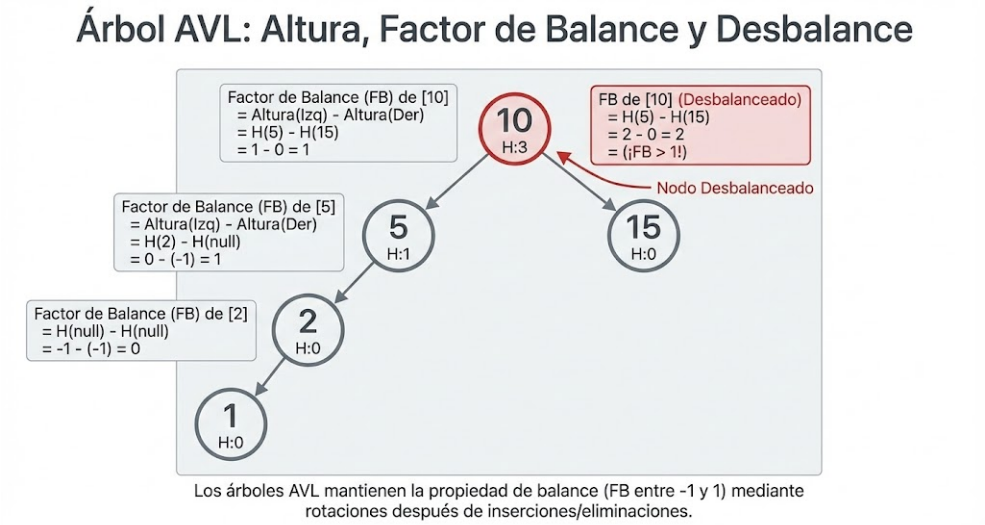


Ilustración 1: Ejemplo de árbol AVL mostrando altura, factor de balance y detección de desbalance en un nodo. Imagen creada en Nano Banana con información proporcionada por la autora.

Multimedia

Para profundizar y ampliar la comprensión sobre el funcionamiento de las estructuras vistas se recomienda consultar el siguiente material complementario. Video creado con NotebookLM con información proporcionada por la autora. https://youtu.be/SEIyBsgYzRc

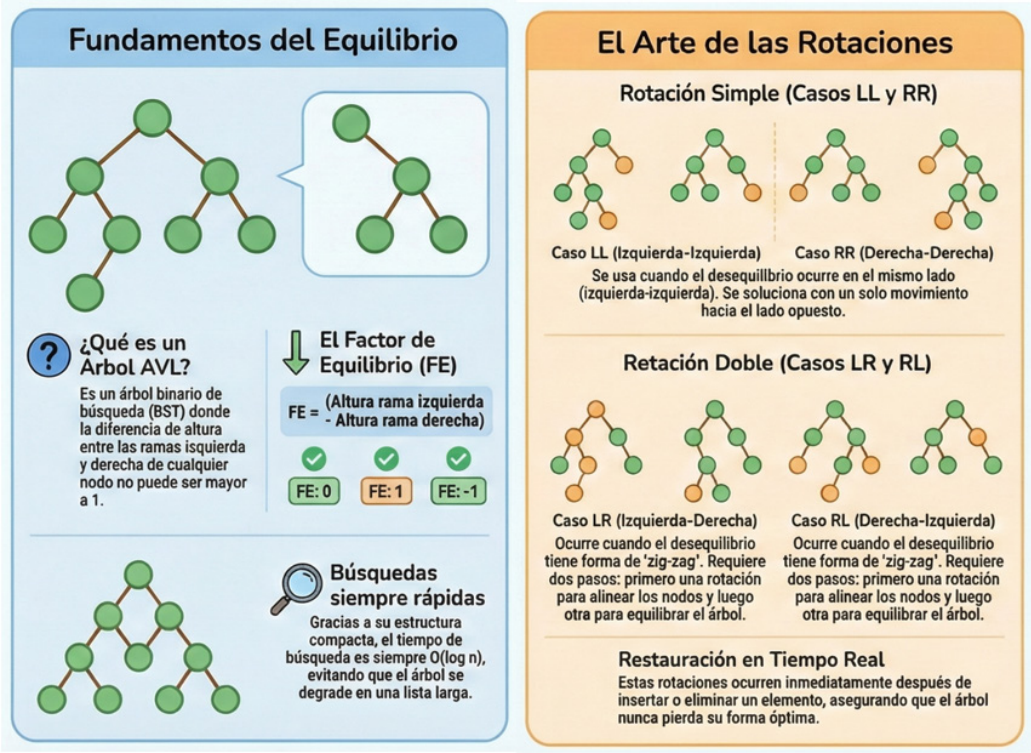


Ilustración 2: Fundamentos del equilibrio y rotaciones en árboles AVL (simples y dobles). Imagen creada usando herramientas de IA con información proporcionada por la autora.

Lectura complementaria

Para profundizar los conceptos abordados, se invita a consultar el siguiente recurso complementario: https://www.geeksforgeeks.org/dsa/introduction-to-avl-tree/

2.4 Árboles rojo-negro (Red-Black)

Un árbol rojo-negro es un BST auto-balanceado que mantiene una altura O (log n) en el peor caso mediante un conjunto de reglas basadas en un bit de "color" asignado a cada nodo. Estas reglas imponen un balance menos estricto que el de un árbol AVL, pero permiten realizar el reequilibrado de manera eficiente durante las operaciones de actualización.

Énfasis

Las reglas, basadas en la coloración de los nodos y en el concepto de "altura negra", aseguran que ningún camino desde la raíz hasta una hoja sea excesivamente largo en comparación con otro. De este modo, se garantiza que la altura del árbol sea logarítmica respecto del número de nodos y, en consecuencia, que las operaciones se realicen en tiempo O(log n).

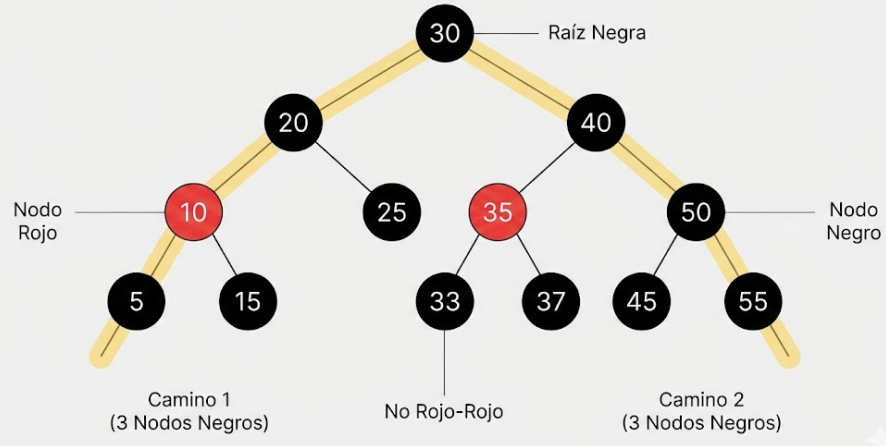


Ilustración 3: Ejemplo de árbol rojo-negro que ilustra la raíz negra, nodos rojos y negros y la igualdad de nodos negros en los caminos. Imagen creada usando herramientas de IA con información proporcionada por la autora.

2.5 Rotaciones: el mecanismo central del re-balanceo

Una rotación es una transformación local de un árbol que preserva el orden in-order (por lo tanto, preserva la propiedad BST) y modifica la estructura para reducir la altura de una rama y restaurar condiciones de balance. Las rotaciones son el núcleo del mantenimiento en AVL y también se utilizan dentro de los ajustes de rojo-negro (junto con recoloreos).

Lectura complementaria

Para profundizar los conceptos abordados, se invita a consultar el siguiente recurso complementario: https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/

2.5.1 Ejemplo 1: Rotación derecha (caso LL) y actualización de alturas (AVL)

Este ejemplo muestra el caso de un desbalance producido por crecimiento en el subárbol izquierdo del hijo izquierdo (caso LL). Se evidencia que la rotación es una operación local que reestructura el árbol, preserva el orden del árbol de búsqueda binaria y actualiza las alturas para recuperar el balance.

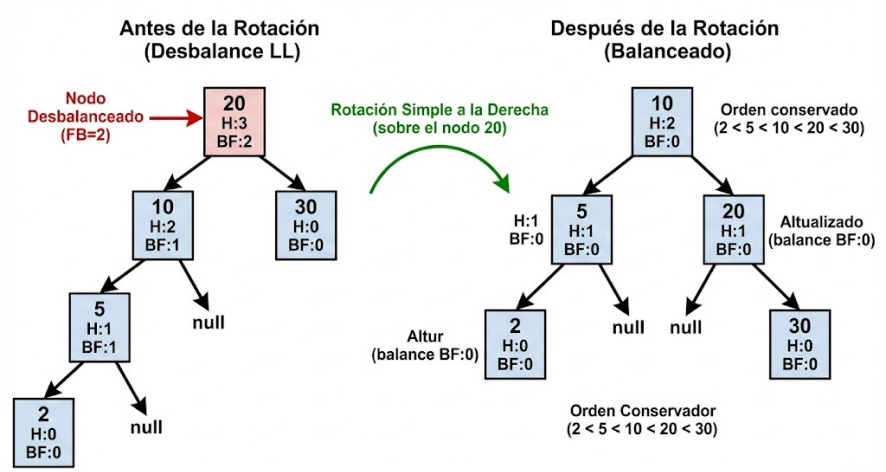


Ilustración 4: Corrección de un desbalance LL en un árbol AVL mediante rotación simple a la derecha, mostrando la conservación del orden y el restablecimiento del equilibrio. Imagen creada usando herramientas de IA con información proporcionada por la autora.

```python
class Nodo:
    def __init__(self, clave):
        self.clave = clave
        self.izquierda = None
        self.derecha = None
        self.altura = 1

def altura(nodo):
    if nodo is None:
        return 0
    return nodo.altura

def actualizar_altura(nodo):
    nodo.altura = 1 + max(altura(nodo.izquierda), altura(nodo.derecha))

def rotacion_derecha(y):
    x = y.izquierda
    t2 = x.derecha

    # Rotación
    x.derecha = y
    y.izquierda = t2

    # Actualización de alturas
    actualizar_altura(y)
    actualizar_altura(x)

    return x


# Árbol desbalanceado tipo LL
y = Nodo(30)
y.izquierda = Nodo(20)
y.izquierda.izquierda = Nodo(10)

actualizar_altura(y.izquierda.izquierda)
actualizar_altura(y.izquierda)
actualizar_altura(y)

print("Altura antes:", y.altura, "Raíz:", y.clave)

y = rotacion_derecha(y)

print("Altura después:", y.altura, "Raíz:", y.clave)
```

Captura 1: Ejemplo de implementación de rotación simple derecha en un árbol AVL para corregir un desbalance tipo LL. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

Elemento	Explicación
Clase `Nodo`	Representa un nodo del árbol AVL con clave, referencias a hijos y altura asociada.
Función `altura`	Devuelve la altura del nodo, considerando el caso de nodo nulo.
Función `actualizar_altura`	Recalcula la altura del nodo en función de las alturas de sus hijos.
Función `rotacion_derecha`	Ejecuta la rotación derecha elevando el hijo izquierdo y desplazando la raíz original hacia la derecha.
Variable `t2`	Conserva el subárbol intermedio que cambia de posición durante la rotación, evitando la pérdida de nodos.
Actualización de alturas	Garantiza que las alturas reflejen la nueva estructura del árbol tras la rotación.
Resultado final	El árbol recupera el balance, reduce su altura efectiva y establece una nueva raíz correctamente balanceada.

2.5.2 Ejemplo 2: Rotación izquierda (caso RR) y actualización de alturas (AVL)

Este ejemplo presenta el caso simétrico al de la rotación derecha (LL): un desbalance producido por crecimiento en el subárbol derecho del hijo derecho (caso RR). Se evidencia que la rotación izquierda es una operación local que reestructura el árbol, preserva el orden propio de un árbol de búsqueda binaria y actualiza las alturas para recuperar el balance.

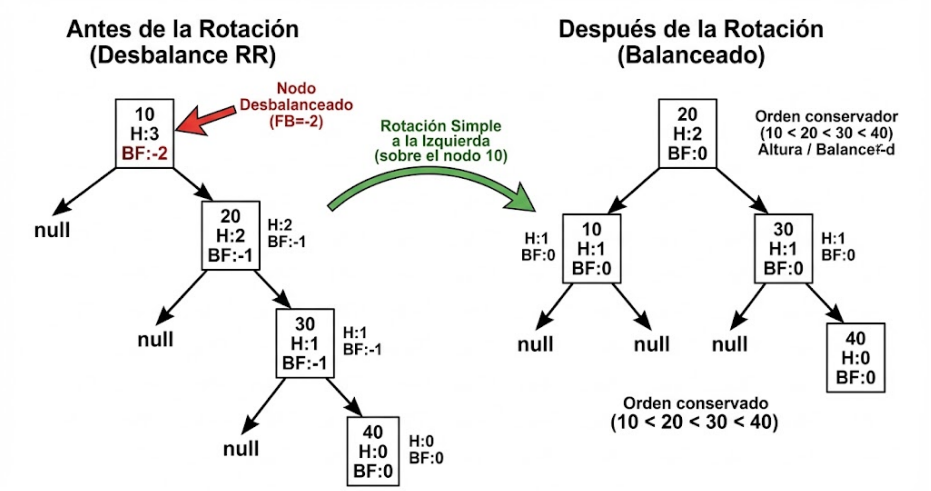


Ilustración 5: Corrección de un desbalance RR en un árbol AVL mediante rotación simple a la izquierda, preservando el orden de los nodos y restableciendo el equilibrio. Imagen creada usando herramientas de IA con información proporcionada por la autora.

```python
class Nodo:
    def __init__(self, clave):
        self.clave = clave
        self.izquierda = None
        self.derecha = None
        self.altura = 1

def altura(nodo):
    if nodo is None:
        return 0
    return nodo.altura

def actualizar_altura(nodo):
    nodo.altura = 1 + max(altura(nodo.izquierda), altura(nodo.derecha))

def rotacion_izquierda(x):
    y = x.derecha
    t2 = y.izquierda

    # Rotación
    y.izquierda = x
    x.derecha = t2

    # Actualización de alturas
    actualizar_altura(x)
    actualizar_altura(y)

    return y

# Árbol desbalanceado tipo RR
x = Nodo(10)
x.derecha = Nodo(20)
x.derecha.derecha = Nodo(30)

actualizar_altura(x.derecha.derecha)
actualizar_altura(x.derecha)
actualizar_altura(x)

print("Altura antes:", x.altura, "Raíz:", x.clave)

x = rotacion_izquierda(x)

print("Altura después:", x.altura, "Raíz:", x.clave)
```

Captura 2: Implementación de la rotación simple a la izquierda para corregir un desbalance RR en un árbol AVL. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

Elemento	Explicación
Clase `Nodo`	Representa un nodo del árbol AVL con clave, referencias a hijos y altura asociada.
Función `altura`	Devuelve la altura del nodo, contemplando el caso de nodo nulo.
Función `actualizar_altura`	Recalcula la altura como 1 + máximo entre las alturas de los subárboles izquierdo y derecho.
Función `rotacion_izquierda`	Ejecuta la rotación izquierda elevando el hijo derecho y desplazando la raíz original hacia la izquierda.
Variable `t2`	Conserva el subárbol intermedio que debe reubicarse durante la rotación para no perder nodos.
Actualización de alturas	Asegura consistencia de las alturas tras la reestructuración del árbol.
Resultado final	El árbol recupera el balance, reduce su altura efectiva y establece una nueva raíz balanceada.

2.5.3 Ejemplo 3: Comparación de rendimiento: BST degenerado (inserción ordenada) vs forma "más balanceada" (inserción aleatoria)

Este ejemplo permite observar, de manera empírica y controlada, cómo la forma del árbol impacta en el costo de búsqueda, medido como cantidad de comparaciones (pasos). En particular, muestra que un BST con inserciones ordenadas tiende a degenerar, alcanzando una altura cercana a *n*; en cambio, un BST con inserciones aleatorias suele conservar una altura significativamente menor, aproximándose al comportamiento esperado de estructuras auto-balanceadas. Esto justifica el uso de árboles AVL y rojo-negro cuando no es posible garantizar un orden de inserción favorable.

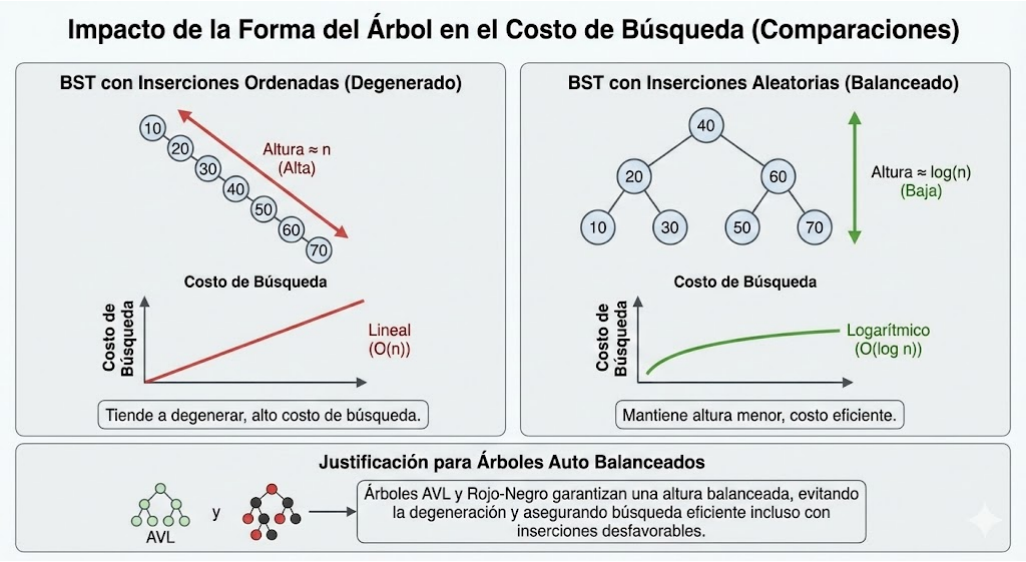


Ilustración 6: Impacto de la forma del árbol en el costo de búsqueda, comparando un BST degenerado y un BST balanceado. Imagen creada usando herramientas de IA con información proporcionada por la autora.

```python
import random
class Node:
    def __init__(self, k):
        self.k = k
        self.l = None
        self.r = None
class BST:
    def __init__(self):
        self.root = None

    def insert(self, k):
        def _ins(n, k):
            if n is None:
                return Node(k)
            if k < n.k:
                n.l = _ins(n.l, k)
            elif k > n.k:
                n.r = _ins(n.r, k)
            return n
        self.root = _ins(self.root, k)
    def search_steps(self, k):
        n = self.root
        steps = 0
        while n is not None:
            steps += 1
            if k < n.k:
                n = n.l
            elif k > n.k:
                n = n.r
            else:
                return steps
        return steps  # clave no encontrada: pasos hasta caer en None
def avg_steps(bst, queries):
    total = 0
    for q in queries:
        total += bst.search_steps(q)
    return total / len(queries)
```

Captura 3: Implementación de un árbol de búsqueda binaria (BST) con inserción y medición de pasos de búsqueda. Código desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Parámetros del experimento
n = 4000
q = 2000
random.seed(42)

# Queries (claves a buscar)
queries = [random.randrange(n) for _ in range(q)]

# Caso 1: inserción ordenada (tendencia a árbol degenerado)
bst_ordenado = BST()
for k in range(n):
    bst_ordenado.insert(k)

# Caso 2: inserción aleatoria (tendencia a menor altura)
bst_aleatorio = BST()
keys = list(range(n))
random.shuffle(keys)
for k in keys:
    bst_aleatorio.insert(k)

print("Pasos promedio de búsqueda (BST inserción ordenada):", avg_steps(bst_ordenado, queries))
print("Pasos promedio de búsqueda (BST inserción aleatoria):", avg_steps(bst_aleatorio, queries))

```

Captura 4: Fragmento del experimento para comparar el costo promedio de búsqueda en un BST con inserciones ordenadas y aleatorias. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

Elemento	Explicación
Objetivo del experimento	Comparar el costo de búsqueda en dos BST construidos con distintos órdenes de inserción.
`search_steps`	Implementa la búsqueda y contabiliza comparaciones como "pasos" hasta encontrar la clave o llegar a `None`.
`bst_ordenado`	Inserta claves en orden creciente, lo que tiende a generar un árbol con altura elevada (comportamiento similar a una lista).
`bst_aleatorio`	Inserta claves en orden aleatorio, lo que tiende a producir una estructura con altura menor que la del caso ordenado.
`queries`	Conjunto de búsquedas aplicado en ambos árboles, manteniendo el mismo escenario de consulta para comparar de forma justa.
`avg_steps`	Calcula el promedio de pasos para estimar el costo típico de búsqueda en cada estructura.
Interpretación del resultado	Un mayor promedio de pasos indica mayor altura efectiva y menor eficiencia; esto fundamenta la necesidad de árboles auto-balanceados (AVL/rojo-negro) cuando el orden de inserción no puede controlarse.

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 1: Árboles AVL.**

3. Árboles B y sus variantes

3.1 indexación en disco y límites de los árboles binarios

Un árbol B es una estructura de datos balanceada, que admite múltiples claves por nodo y está diseñada para minimizar la cantidad de accesos a memoria secundaria (disco), manteniendo las operaciones de búsqueda, inserción y eliminación en tiempo logarítmico.

En contextos donde los datos no caben completamente en la memoria principal, como ocurre en bases de datos, sistemas de archivos o motores de búsqueda, el costo dominante deja de ser la comparación entre claves y pasa a ser el acceso a disco.

En este escenario, los árboles binarios balanceados, como los AVL y los rojos negro, presentan dos limitaciones relevantes:

- Profundidad relativamente grande.
- Un acceso a disco por nodo visitado.

Los árboles B abordan este problema aumentando el factor de ramificación. Al permitir múltiples claves por nodo y más hijos por nivel, reducen significativamente la altura del árbol y, en consecuencia, la cantidad de accesos a disco necesarios para realizar una operación.

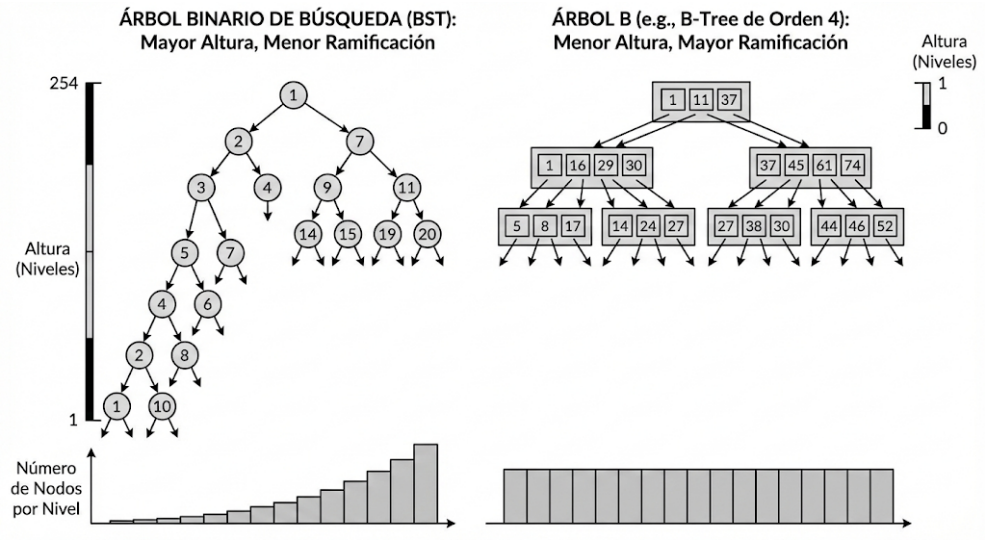


Ilustración 7: Comparación conceptual entre un árbol binario de búsqueda (BST) y un árbol B, destacando diferencias en altura y grado de ramificación. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.2 Propiedades estructurales de los Árboles B

Énfasis

Un árbol B de orden m cumple las siguientes propiedades:

- Cada nodo contiene entre $m/2 - 1$ y $m - 1$ claves.
- Todos los nodos internos, excepto la raíz, tienen al menos $m/2$ hijos.
- Todas las hojas se encuentran en el mismo nivel.
- Las claves dentro de cada nodo se mantienen ordenadas.

3.3 Relevancia algorítmica

Las restricciones estructurales que definen a los árboles B no constituyen meras condiciones formales, sino que determinan de manera directa su comportamiento algorítmico y su adecuación a contextos de gran escala y persistencia de datos.

En primer lugar, el balanceo automático garantiza que todas las hojas se encuentren al mismo nivel. Desde el punto de vista algorítmico, esto implica que toda operación de búsqueda, inserción o eliminación recorre exactamente la misma cantidad de niveles en el peor caso, eliminando escenarios patológicos de degradación del rendimiento. A diferencia de estructuras no balanceadas, el costo temporal queda acotado de manera estricta.

En segundo lugar, la altura $O(\log n)$ —donde m representa el orden del árbol— introduce una diferencia sustancial respecto de los árboles binarios balanceados. Al permitir múltiples claves por nodo, el factor de ramificación crece significativamente, reduciendo

la altura del árbol. Algorítmicamente, esto se traduce en menos descensos de nivel y, en consecuencia, en menos accesos a nodos, aspecto crítico cuando dichos nodos se almacenan en memoria secundaria.

En tercer lugar, el uso eficiente de bloques de disco constituye el eje central de la motivación de los árboles B. Cada nodo se diseña para ajustarse al tamaño de un bloque físico de almacenamiento, maximizando la cantidad de claves transferidas en una única operación de entrada/salida. Desde la perspectiva algorítmica, el costo dominante deja de ser el número de comparaciones y pasa a ser el número de accesos a disco, que los árboles B minimizan de manera sistemática.

Adicionalmente, la ordenación interna de las claves dentro de cada nodo habilita búsquedas eficientes locales, mediante recorridos secuenciales o búsquedas binarias, sin requerir accesos adicionales a memoria secundaria. Esta combinación de búsqueda interna y descenso controlado refuerza la eficiencia global de la estructura.

Por último, estas propiedades hacen que los árboles B sean **estructuras algorítmicamente estables ante el crecimiento del volumen de datos**, manteniendo sus garantías de rendimiento incluso cuando el conjunto de claves se incrementa en varios órdenes de magnitud. Esta característica resulta especialmente relevante en sistemas vinculados a la Inteligencia Artificial, donde los volúmenes de datos suelen crecer de manera sostenida y los costos de acceso a almacenamiento persistente condicionan el desempeño global del sistema.

3.4 Búsqueda en un Árbol B

La operación de búsqueda en un árbol B se caracteriza por combinar procesos locales eficientes con un recorrido estructural altamente controlado, diseñado para minimizar el costo de acceso a memoria secundaria.

En cada nodo visitado, la búsqueda comienza con una inspección de las claves almacenadas en el nodo. Dado que estas claves se encuentran ordenadas, la localización del intervalo correspondiente puede realizarse mediante búsqueda secuencial, cuando el número de claves es reducido, o mediante búsqueda binaria, cuando el tamaño del nodo lo justifica. Desde el punto de vista algorítmico, este costo es acotado y constante respecto del volumen total de datos, ya que el número máximo de claves por nodo está limitado por el orden del árbol.

Una vez determinado el intervalo adecuado, la búsqueda continúa con el descenso a un único hijo, seleccionado en función del rango que delimita la clave buscada. Este comportamiento contrasta con otras estructuras de búsqueda, ya que en ningún momento se exploran múltiples ramas: el recorrido sigue un camino único desde la raíz hasta una hoja, preservando la eficiencia logarítmica.

El costo total de la búsqueda depende fundamentalmente de la altura del árbol, la cual es O (log n), donde m es el orden del árbol y n la cantidad de claves almacenadas. Dado que el valor de m suele ser elevado en aplicaciones reales —ajustado al tamaño de bloque de disco—, la altura del árbol se mantiene pequeña incluso para conjuntos de datos

muy grandes. En consecuencia, el número de accesos a nodos, y por extensión a disco, se reduce de manera significativa.

Desde una perspectiva de sistemas, esta estrategia convierte a la búsqueda en árboles B en una operación predecible y escalable, ya que el crecimiento del volumen de datos impacta de forma sublogarítmica en el número de accesos a memoria secundaria. Esta propiedad resulta central en aplicaciones de Inteligencia Artificial que requieren consultar grandes volúmenes de datos persistentes, tales como índices de datasets, catálogos de características o repositorios de modelos.

Finalmente, la combinación de ordenación interna, alto factor de ramificación y descenso determinístico hace que la búsqueda en árboles B sea más que adecuada para entornos donde el costo de entrada/salida domina sobre el costo computacional, consolidando a esta estructura como un pilar fundamental en sistemas de almacenamiento y recuperación de información a gran escala.

3.4.1 Ejemplo 1: Búsqueda en un Árbol B (implementación simplificada)

Este ejemplo modela la lógica de búsqueda en un árbol B residente en memoria, manteniendo el comportamiento conceptual de la estructura real utilizada en indexación.

```python
class BTreeNode:
    def __init__(self, keys=None, children=None, leaf=True):
        self.keys = keys or []
        self.children = children or []
        self.leaf = leaf

def search(node, key):
    i = 0
    while i < len(node.keys) and key > node.keys[i]:
        i += 1

    if i < len(node.keys) and key == node.keys[i]:
        return True

    if node.leaf:
        return False

    return search(node.children[i], key)
```

Captura 5: Código de definición de nodo y búsqueda en Árbol B. Desarrollo realizado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`BTreeNode`	Representa un nodo con múltiples claves y punteros a hijos.
Bucle `while`	Localiza el intervalo donde puede estar la clave.
Comparación directa	Detecta si la clave está almacenada en el nodo actual.
Llamada recursiva	Desciende a un único hijo, manteniendo complejidad logarítmica.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

3.5 Inserción en Árboles B

Énfasis

La inserción en un árbol B consiste en ubicar la clave en una hoja y dividir el nodo si se excede la cantidad máxima de claves permitidas.

Proceso

El proceso de inserción en un árbol B está diseñado para incorporar nuevas claves preservando de manera estricta las propiedades estructurales, en particular el balanceo automático y la altura uniforme del árbol. A diferencia de otras estructuras de búsqueda, la inserción no solo implica ubicar la clave en la posición correcta, sino también gestionar posibles desbordes de nodos mediante mecanismos de división y promoción de claves.

Este procedimiento garantiza que, aun frente al crecimiento progresivo del conjunto de datos, el árbol mantenga su eficiencia en términos de búsqueda y acceso a memoria secundaria.

En síntesis, el proceso consiste en:

1. Descender hasta la hoja correspondiente.
2. Insertar la clave en orden.
3. Si el nodo se desborda:
   - Se divide.
   - La clave media asciende al padre.
4. El proceso puede propagarse hasta la raíz.

3.5.1 Ejemplo 2: Inserción con división de nodo (modelo simplificado)

El siguiente ejemplo ilustra la lógica de división, eje central del mantenimiento del balance en árboles B.

```
def split_node(node):
    mid = len(node.keys) // 2
    left = BTreeNode(
        keys=node.keys[:mid],
        children=node.children[:mid+1],
        leaf=node.leaf
    )
    right = BTreeNode(
        keys=node.keys[mid+1:],
        children=node.children[mid+1:],
        leaf=node.leaf
    )
    return node.keys[mid], left, right
```

Captura 6: Código de división (split) de nodos en Árbol B. Desarrollo realizado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`mid`	Índice de la clave promotora.
`left`	Nodo izquierdo con claves menores.
`right`	Nodo derecho con claves mayores.
Valor retornado	Clave que asciende + nuevos nodos.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

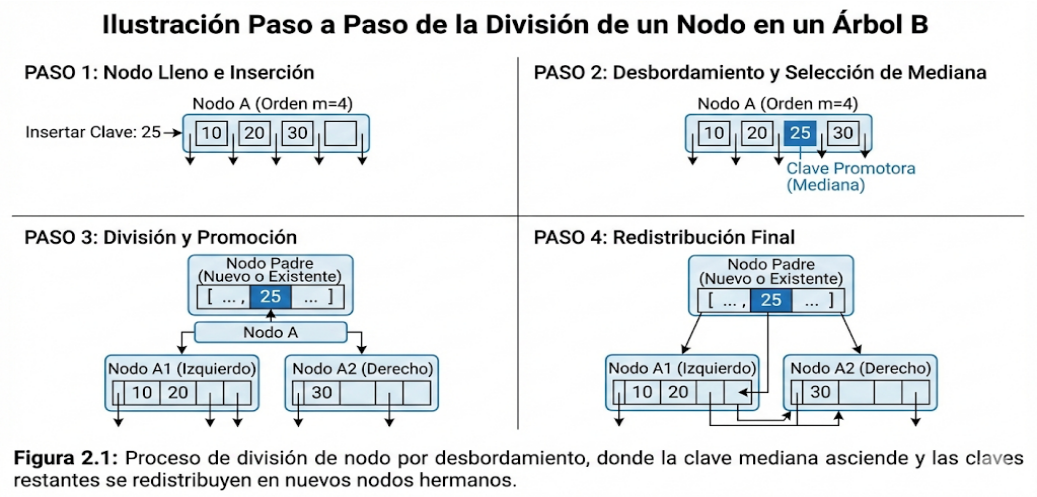


Figura 2.1: Proceso de división de nodo por desbordamiento, donde la clave mediana asciende y las claves restantes se redistribuyen en nuevos nodos hermanos.

Ilustración 8: Proceso paso a paso de la división de un nodo en un árbol B, mostrando inserción, selección de la mediana, promoción y redistribución final. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.6 Eliminación en árboles B

Énfasis La eliminación en un árbol B preserva las invariantes estructurales mediante la redistribución o la fusión de nodos, garantizando que el árbol conserve su balance y sus propiedades formales.

3.6.1 Casos principales

3.6.1.1 Eliminación directa en hoja

Este caso se presenta cuando la clave a eliminar se encuentra en un nodo hoja que, aun después de la eliminación, conserva al menos el número mínimo de claves permitido por el orden del árbol. Desde el punto de vista algorítmico, esta es la situación más simple, ya que la operación se limita a retirar la clave y reordenar localmente el nodo si fuera necesario. No se requiere ningún ajuste adicional sobre nodos vecinos ni propagación hacia niveles superiores, dado que las invariantes estructurales del árbol permanecen intactas.

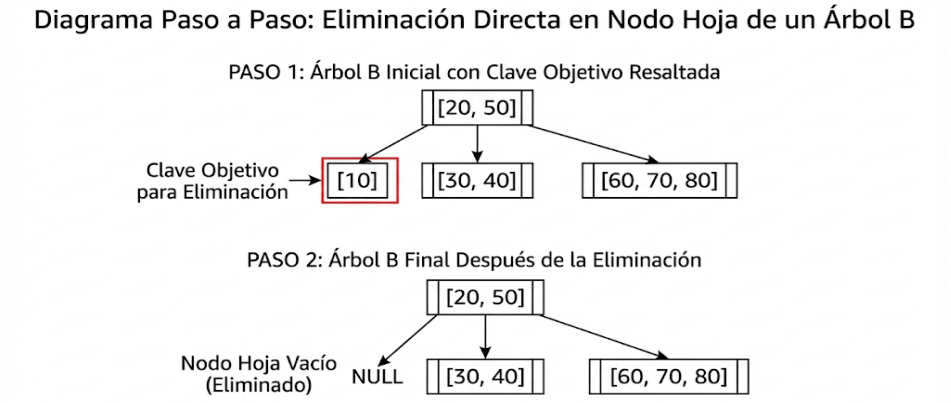


Figura 1: Ilustración de la eliminación de la clave '10' de un nodo hoja. Al ser una eliminación directa sin subdesbordamiento, el nodo simplemente se vacía y se elimina del árbol.

Ilustración 9: Ejemplo de eliminación de una clave en un árbol B y restructuración del árbol resultante. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.6.1.2 Sustitución por predecesor/sucesor

Cuando la clave a eliminar se encuentra en un nodo interno, no puede eliminarse de manera directa sin comprometer la estructura del árbol. En este caso, la clave se sustituye por su predecesor inmediato (máxima clave del subárbol izquierdo) o por su sucesor inmediato (mínima clave del subárbol derecho). Algorítmicamente, esta estrategia transforma el problema en una eliminación en un nodo hoja, permitiendo aplicar posteriormente uno de los mecanismos estándar de ajuste si fuera necesario. Esta sustitución preserva el orden de las claves y mantiene la propiedad de búsqueda del árbol.

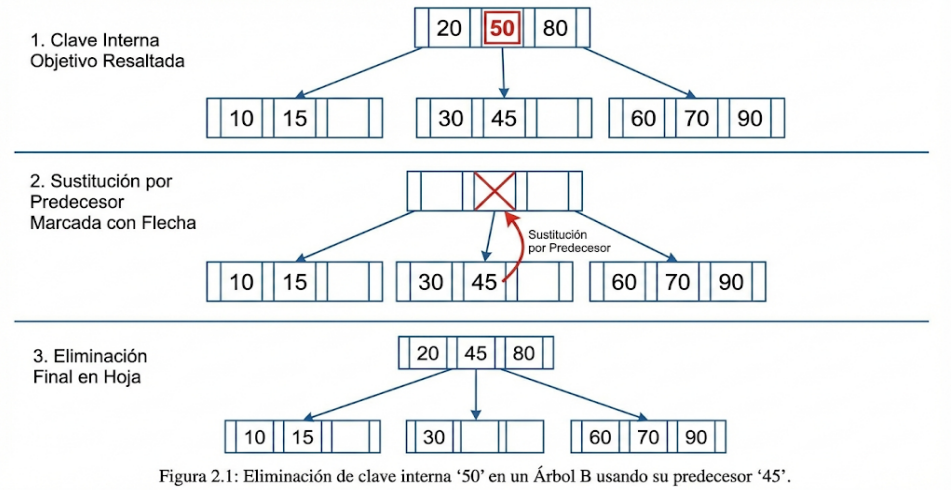


Figura 2.1: Eliminación de clave interna '50' en un Árbol B usando su predecesor '45'.

Ilustración 10: Eliminación de una clave interna en un árbol B mediante sustitución por el predecesor y actualización de nodos hoja. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.6.1.3 Redistribución desde nodos hermanos

La redistribución se aplica cuando, tras una eliminación, un nodo queda con menos claves de las permitidas, pero alguno de sus nodos hermanos adyacentes dispone de claves

excedentes. En esta situación, una clave del nodo padre desciende al nodo deficitario, mientras que una clave del hermano asciende al padre, restableciendo el equilibrio. Desde el punto de vista algorítmico, la redistribución resulta preferible a la fusión, ya que corrige la violación local sin reducir el número total de nodos del árbol ni afectar su altura.

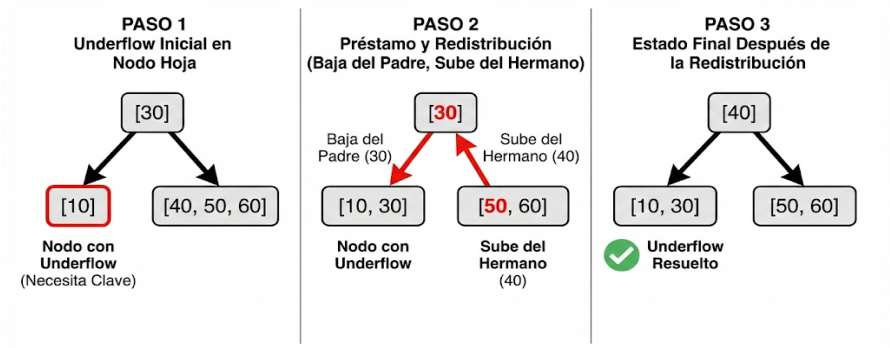


Figura 1: Diagrama paso a paso de la redistribución de claves tras una eliminación que causa underflow. Una clave baja del padre al nodo afectado, y es reemplazada por la clave mínima del hermano derecho, que sube al padre.

Ilustración 11: Proceso de resolución de underflow en árboles B mediante fusión y actualización del nodo padre. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.6.1.4 Fusión cuando no hay claves suficientes

La fusión se produce cuando el nodo deficitario y sus hermanos adyacentes no poseen claves adicionales para redistribuir. En este caso, el nodo se fusiona con uno de sus hermanos y una clave del nodo padre, formando un único nodo que cumple con las restricciones del árbol. Algorítmicamente, esta operación reduce el número de nodos en el nivel afectado y puede generar una nueva violación en el nodo padre, provocando que el proceso de ajuste se propague hacia niveles superiores. Aun así, el árbol conserva su balance perfecto y su altura uniforme, salvo en el caso particular de que la raíz quede vacía, lo que implica una reducción controlada de la altura total.

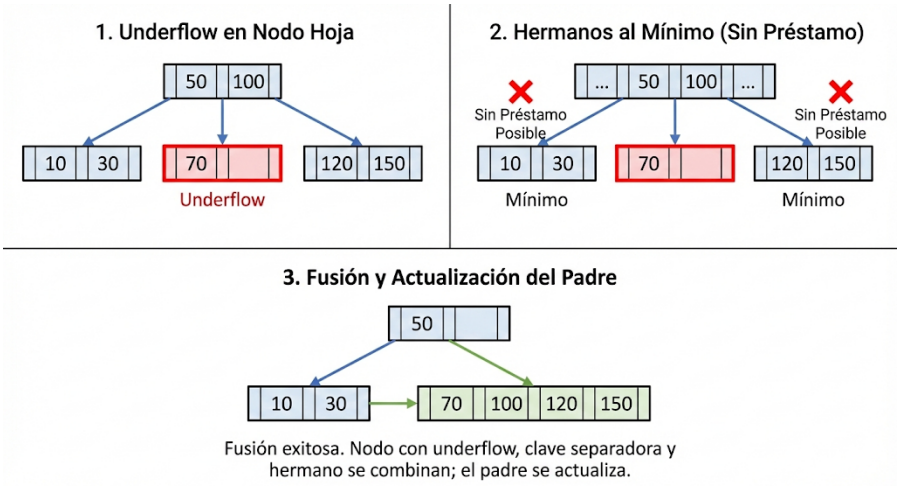


Ilustración 12: Proceso de manejo de underflow en un nodo hoja de un árbol B, incluyendo verificación de hermanos y fusión con actualización del nodo padre. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.6.2 Ejemplo 3: Redistribución conceptual (fragmento)

```python
def can_borrow(node):
    return len(node.keys) > 1
```

Captura 7: Función auxiliar para verificación de préstamo de claves en Árbol B. Desarrollo realizado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`can_borrow`	Verifica si un nodo hermano puede ceder una clave.
Condición	Evita romper el mínimo de claves permitido.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

3.7 Variantes: B+ y B*

3.7.1 Árbol B+

Énfasis

Un árbol B+ es una variante del árbol B en la que todas las claves asociadas a los datos se almacenan exclusivamente en los nodos hoja, mientras que los nodos internos contienen únicamente claves de separación que guían el recorrido del árbol. Las hojas se encuentran enlazadas secuencialmente, lo que permite recorridos ordenados eficientes.

Desde el punto de vista algorítmico, esta separación entre estructura de indexación y almacenamiento de datos simplifica las operaciones de búsqueda por rango y optimiza el acceso secuencial, manteniendo una altura reducida y balance uniforme.

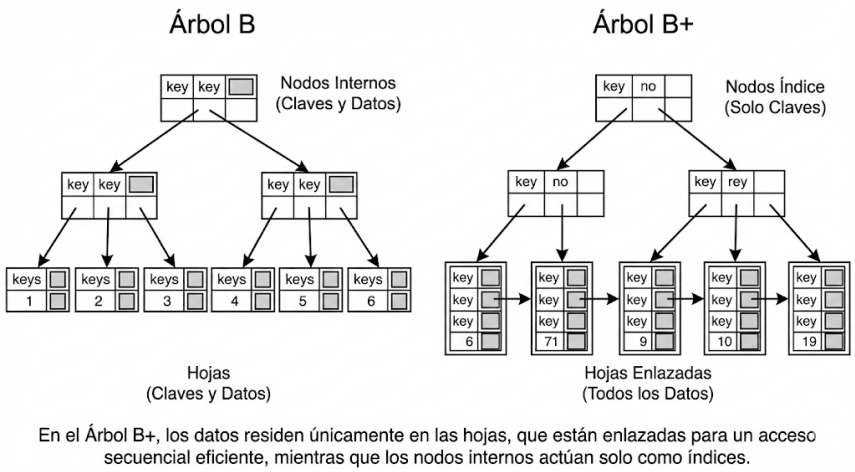


Ilustración 13: Comparación entre Árbol B y Árbol B+, mostrando la ubicación de datos y el rol de los nodos internos y hojas. Imagen creada usando herramientas de IA con información proporcionada por la autora.

*3.7.2 Árbol B**

Énfasis

Un árbol B* es una variante del árbol B que introduce una política más estricta de ocupación de nodos. Antes de dividir un nodo desbordado, el árbol B* intenta una redistribución de claves con nodos hermanos, con el objetivo de mantener un factor de llenado más alto.

Solo cuando la redistribución no es posible, se realiza una división que involucra a dos nodos y genera tres nodos parcialmente llenos. Algorítmicamente, esta estrategia reduce la frecuencia de divisiones, incrementa la utilización del espacio y disminuye el crecimiento prematuro de la altura del árbol.

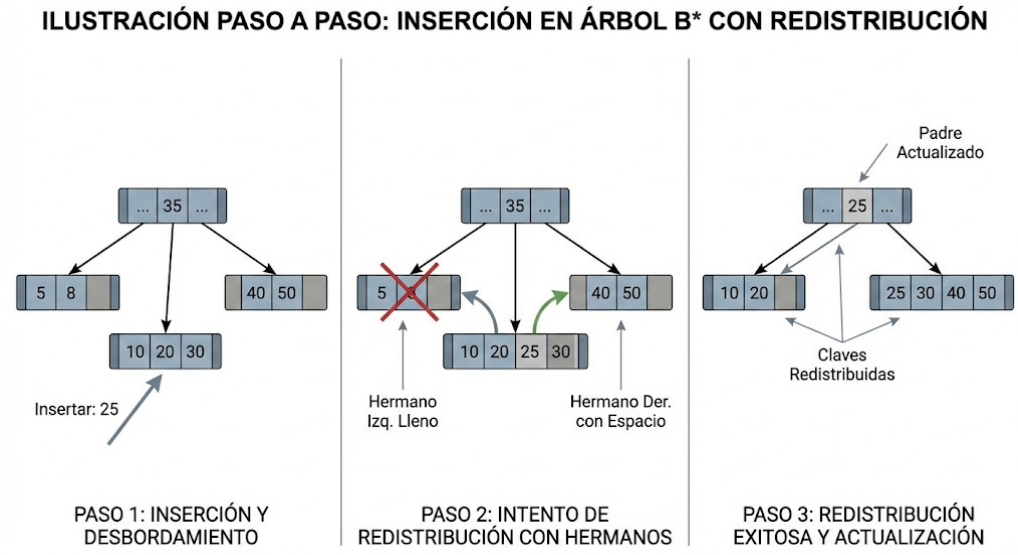


Ilustración 14: Inserción paso a paso en un Árbol B* con redistribución entre nodos hermanos y actualización del nodo padre. Imagen creada usando herramientas de IA con información proporcionada por la autora.

3.7.3 Uso en IA y sistemas reales

El empleo de árboles B+ y B* resulta adecuado en sistemas donde el volumen de datos es elevado y el acceso a almacenamiento persistente domina el costo computacional.

En el caso de los índices de bases de datos, los árboles B+ constituyen la estructura predominante, ya que las hojas enlazadas permiten recorrer grandes conjuntos de registros en orden sin reingresar a los niveles superiores del árbol. Esta propiedad es esencial para consultas por rango, agregaciones y exploraciones secuenciales, frecuentes en pipelines de preparación de datos para modelos de inteligencia artificial.

En los motores de inferencia sobre grandes volúmenes de datos, estas variantes permiten localizar rápidamente bloques de información relevantes como, por ejemplo, vectores de características, embeddings o resultados intermedios, sin incurrir en un número elevado de accesos a disco. La previsibilidad del recorrido y la baja altura del árbol favorecen tiempos de respuesta estables, incluso bajo cargas intensivas.

En los sistemas de recuperación de información, como catálogos documentales o repositorios de conocimiento, los árboles B+ facilitan búsquedas ordenadas, paginadas y por intervalos, mientras que los árboles B* optimizan el uso del espacio en disco, aspecto crítico cuando el índice crece de manera sostenida.

Finalmente, en contextos propios de la inteligencia artificial, estas variantes resultan adecuadas para sostener infraestructuras híbridas donde conviven memoria principal y almacenamiento secundario, garantizando que las operaciones de acceso a grandes estructuras persistentes mantengan un comportamiento algorítmico eficiente y escalable.

*3.8 Tabla comparativa: árbol B vs árbol B+ vs árbol B**

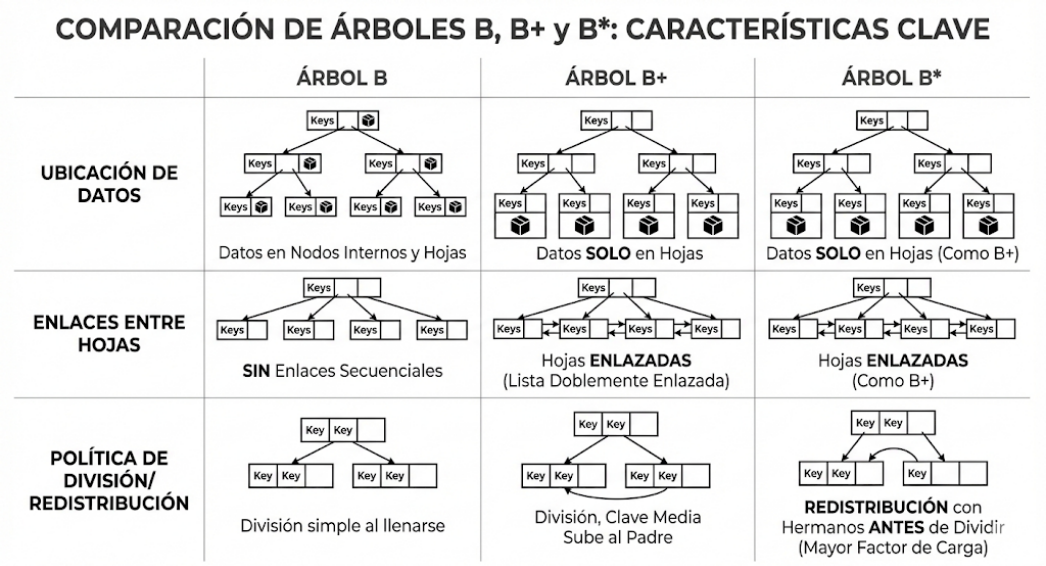
Las variantes B, B+ y B* comparten el objetivo de minimizar el número de accesos a memoria secundaria, pero difieren en la estrategia adoptada para optimizar el recorrido, el almacenamiento de datos y el uso del espacio.

- El árbol B ofrece una solución general, equilibrando simplicidad y eficiencia.
- El árbol B+ optimiza recorridos ordenados y consultas por rango, convirtiéndose en el estándar de facto en sistemas de bases de datos.
- El árbol B* prioriza la utilización eficiente del espacio, reduciendo la fragmentación y el crecimiento prematuro de la estructura.

Criterio	Árbol B	Árbol B+	Árbol B*
Tipo de estructura	Árbol de búsqueda multi-clave balanceado	Variante de árbol B orientada a indexación	Variante de árbol B orientada a eficiencia espacial
Ubicación de los datos	En nodos internos y hojas	Exclusivamente en hojas	En nodos internos y hojas
Rol de los nodos internos	Almacenan claves y datos	Almacenan solo claves de separación	Almacenan claves y datos
Enlace entre hojas	No obligatorio	Hojas enlazadas secuencialmente	No obligatorio
Factor de ramificación	Alto	Alto	Alto
Altura del árbol	O(log n)	O(log n)	O(log n), con menor crecimiento efectivo
Búsqueda puntual	Eficiente	Eficiente	Eficiente
Búsqueda por rango	Moderadamente eficiente	Muy eficiente (recorrido secuencial en hojas)	Moderadamente eficiente
Inserción	Inserción + división al desbordar	Inserción en hojas + división	Inserción con redistribución previa

Frecuencia de divisiones	Moderada	Moderada	Menor
Política ante desborde	División directa del nodo	División del nodo hoja	Redistribución con hermanos antes de dividir
Ocupación promedio de nodos	Media	Media	Alta
Uso del espacio en disco	Bueno	Bueno	Óptimo
Costo de mantenimiento	Moderado	Moderado	Mayor complejidad lógica
Complejidad de implementación	Media	Media	Alta
Aplicaciones típicas	Índices generales	Índices de bases de datos	Sistemas con fuerte restricción de espacio
Uso predominante en la práctica	Menor	Muy alto	Específico
Adecuación a grandes volúmenes	Alta	Muy alta	Muy alta
Adecuación a IA (datos masivos)	Alta	Muy alta	Muy alta

Tabla 1: comparación árbol B vs árbol B+ vs árbol B*. Fuente: elaboración propia.



Infografía 1: Comparación estructural entre Árboles B, B+ y B: ubicación de datos, enlaces entre hojas y políticas de división. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.9 Comparación integrada: AVL vs rojo-negro vs árboles B

La diferencia fundamental entre AVL, rojo-negro y árboles B no es el algoritmo, sino el modelo de costo computacional asumido.

- AVL y rojo-negro optimizan:
  - Comparaciones.
  - Tiempo de CPU.
  - Estructuras residentes en memoria principal.
- Árboles B optimizan:
  - Accesos a disco.
  - Transferencia por bloques.
  - Persistencia y escalabilidad.

Criterio	Árbol AVL	Árbol Rojo-Negro	Árbol B / B+ / B*
Modelo estructural	Árbol binario de búsqueda estrictamente balanceado	Árbol binario de búsqueda con balanceo relajado	Árbol de búsqueda multi-clave y multi-hijo
Factor de ramificación	2	2	Alto (decenas o centenas de hijos por nodo)
Condición de balanceo	Diferencia de alturas ≤ 1	Restricciones de color y caminos negros	Número mínimo y máximo de claves por nodo
Altura del árbol	O(log n), mínima	O(log n), algo mayor que AVL	O(log n), muy baja
Costo dominante	Comparaciones en memoria	Comparaciones en memoria	Accesos a bloques de disco
Accesos a memoria secundaria	No optimizados	No optimizados	Diseñados para minimizar I/O
Estructura del nodo	Una clave, dos punteros	Una clave, dos punteros + color	Varias claves y múltiples punteros
Búsqueda	Rápida en memoria principal	Rápida en memoria principal	Muy eficiente en disco
Inserción	Puede requerir múltiples rotaciones	Pocas rotaciones, recoloreos	Inserción + división de nodos
Eliminación	Compleja (rotaciones múltiples)	Compleja pero acotada	Compleja (redistribución/ fusión)
Mantenimiento del balance	Estricto	Flexible	Implícito por diseño

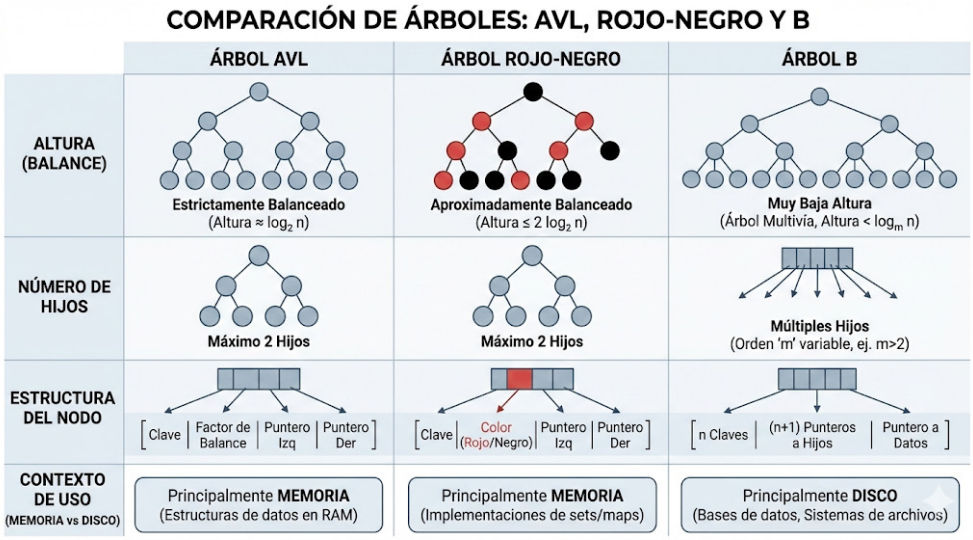
Costo de reestructuración	Alto	Medio	Alto, pero poco frecuente
Uso típico	Estructuras en RAM, tiempo real	Bibliotecas estándar (map, set)	Índices en bases de datos y FS
Ejemplos de uso real	Planificadores, caches	STL, árboles de símbolos	BBDD, motores de búsqueda
Adecuación a grandes volúmenes	Limitada	Limitada	Óptima
Adecuación a IA (datos masivos)	Baja	Media	Alta

Tabla 2: Comparación AVL vs rojo-negro vs árboles B. Fuente: elaboración propia.

3.10 Relación con Inteligencia Artificial

Escenario en IA	Estructura más adecuada	Justificación
Índices de datasets masivos	Árbol B / B+	Minimiza I/O en almacenamiento persistente
Estructuras temporales en entrenamiento	AVL	Balance estricto y búsqueda rápida
Diccionarios y tablas internas	Rojo-Negro	Buen balance entre costo y simplicidad
Sistemas híbridos RAM + disco	B+	Separación entre índices y datos

Tabla 3. Fuente: elaboración propia.



Infografía 2: Comparación conceptual entre árboles AVL, rojo-negro y B, considerando balance, estructura de nodos, altura y contexto de uso. Imagen creada usando herramientas de IA con información proporcionada por la autora.

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 2: Laboratorio Interactivo de Árboles B.**

4. Montículos (heaps)

4.1 Concepto general

Énfasis

Un montículo (heap) es una estructura de datos basada en un árbol casi completo que satisface la propiedad de montículo:

- Min-heap: cada nodo es menor o igual que sus hijos.
- Max-heap: cada nodo es mayor o igual que sus hijos.

Los montículos se utilizan para implementar colas de prioridad, optimizar algoritmos de búsqueda del mínimo o máximo y como base del algoritmo heap sort, ampliamente utilizado en sistemas y aplicaciones de inteligencia artificial para selección eficiente y planificación.

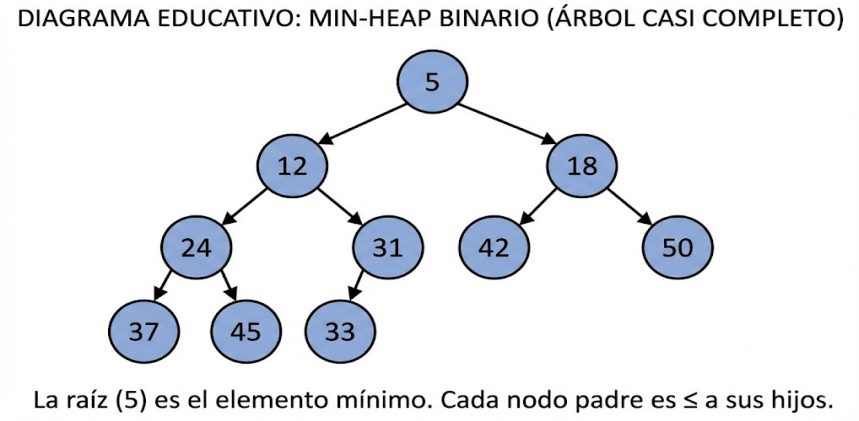


Ilustración 15: Diagrama educativo de un min-heap binario, donde la raíz contiene el valor mínimo y cada nodo padre es menor o igual que sus hijos. Imagen creada usando herramientas de IA con información proporcionada por la autora.

4.2 Montículos binarios

Énfasis

Un montículo binario es un montículo implementado como un árbol binario casi completo, que puede representarse eficientemente mediante una lista (o arreglo) indexada desde 0.

Cuando un montículo binario se almacena en una lista indexada desde 0, la estructura jerárquica del árbol casi completo queda implícita en las posiciones del arreglo. No se almacenan referencias explícitas entre nodos, sino que las relaciones padre–hijos se deducen aritméticamente a partir del índice del elemento.

Las relaciones fundamentales son las siguientes:

- Padre de `i`:

$$\left\lfloor \frac{i-1}{2} \right\rfloor$$

- Hijo izquierdo de `i`:

$$2i+1$$

- Hijo derecho de `i`:

$$2i+2$$

Lectura complementaria

Para profundizar los conceptos abordados, se invita a consultar el siguiente recurso complementario: https://www.geeksforgeeks.org/dsa/binary-heap/

4.3 Árbol casi completo

El montículo binario es un árbol casi completo; esto significa que todos sus niveles están llenos, salvo, posiblemente, el último, que se completa de izquierda a derecha. Esta propiedad garantiza que el recorrido por niveles del árbol coincida exactamente con el orden en que los elementos se almacenan en la lista.

Como consecuencia:

- Cada nodo en la posición i ocupa una ubicación fija en el recorrido por niveles.
- Los hijos de un nodo aparecen inmediatamente después de todos los nodos de niveles superiores.
- No existen "huecos" intermedios en la estructura.

Esta regularidad permite derivar las relaciones padre–hijos mediante fórmulas simples, sin necesidad de estructuras auxiliares.

4.4 Operaciones fundamentales

4.4.1 Inserción (sift-up)

Al insertar un nuevo elemento, este se coloca al final del arreglo y se lo desplaza hacia arriba hasta restaurar la propiedad de montículo. El costo temporal es O (log n).

Ejemplo 1: Inserción en un min-heap implementado con listas

Este ejemplo muestra la inserción manual en un min-heap, ilustrando el proceso de *sift-up* sin utilizar bibliotecas externas.

```python
def heap_insert(heap, value):
    heap.append(value)
    i = len(heap) - 1

    while i > 0:
        parent = (i - 1) // 2
        if heap[i] < heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break
```

Captura 8: Inserción en montículo binario (heap insert). Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`heap.append(value)`	Inserta el nuevo elemento al final del arreglo.
`i = len(heap) - 1`	Posición inicial del elemento insertado.
`parent = (i - 1) // 2`	Cálculo del índice del nodo padre.
Intercambio	Restaura la propiedad de min-heap si el hijo es menor que el padre.
Complejidad	O(log n) en el peor caso.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

4.4.2 Eliminación del mínimo (sift-down)

La eliminación del mínimo en un min-heap consiste en remover la raíz, reemplazarla por el último elemento y desplazarlo hacia abajo (*sift-down*).

Ejemplo 2: Eliminación del mínimo en un min-heap

Este ejemplo implementa la operación extract_min, fundamental para colas de prioridad.

```python
def heap_extract_min(heap):
    if not heap:
        return None

    min_value = heap[0]
    heap[0] = heap.pop()
    i = 0

    while True:
        left = 2 * i + 1
        right = 2 * i + 2
        smallest = i

        if left < len(heap) and heap[left] < heap[smallest]:
            smallest = left
        if right < len(heap) and heap[right] < heap[smallest]:
            smallest = right

        if smallest != i:
            heap[i], heap[smallest] = heap[smallest], heap[i]
            i = smallest
        else:
            break

    return min_value
```

Captura 9: Extracción del mínimo en montículo binario (heap extract-min). Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`min_value = heap[0]`	Guarda el mínimo almacenado en la raíz.
`heap[0] = heap.pop()`	Sustituye la raíz por el último elemento.
Bucle `while`	Reestablece la propiedad de heap descendiendo el elemento.
Comparaciones	Seleccionan el hijo más pequeño para el intercambio.
Complejidad	O(log n).

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

4.5 Uso del módulo heapq en Python

El módulo estándar heapq implementa un min-heap sobre listas, proporcionando operaciones eficientes y seguras.

Operaciones principales:

- Heappush
- Heappop
- Heapify

4.5.1 Ejemplo 3: Uso básico de heapq

Este ejemplo muestra cómo Python abstrae las operaciones fundamentales del montículo binario.

```python
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

min_value = heapq.heappop(heap)
print(min_value)
```

Captura 10: Uso de la biblioteca heapq para operaciones básicas de heap. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`heapq.heappush`	Inserta un elemento manteniendo la propiedad de heap.
`heapq.heappop`	Extrae el mínimo del heap.
Lista subyacente	Se utiliza como representación del árbol casi completo.
Complejidad	Inserción y eliminación en O(log n).

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

4.5.2 Heap sort

Heap sort es un algoritmo de ordenamiento que utiliza un montículo binario para ordenar elementos en O(n log n), sin requerir memoria adicional significativa.

Ejemplo 4: Heap sort con heapq

Este ejemplo ilustra una implementación simple de heap sort basada en un min-heap.

```python
import heapq

def heap_sort(data):
    heapq.heapify(data)
    result = []
    while data:
        result.append(heapq.heappop(data))
    return result
```

Captura 11: Ordenamiento mediante Heap Sort utilizando heapq. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`heapq.heapify`	Construye el heap en tiempo O(n).
Bucle `while`	Extrae los elementos en orden creciente.
`result`	Lista ordenada resultante.
Complejidad	O(n log n).

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

Lectura complementaria

Para profundizar los conceptos abordados, se invita a consultar el siguiente recurso complementario: https://www.geeksforgeeks.org/dsa/heap-sort/

4.6 Montículos de Fibonacci

Un montículo de Fibonacci es una estructura de datos diseñada para optimizar ciertas operaciones en términos amortizados, en particular la operación decrease key, que alcanza un costo O(1) amortizado.

Se caracteriza por organizarse como un conjunto de árboles que no son necesariamente binarios, por utilizar listas enlazadas para gestionar sus nodos y por aplicar un mecanismo de consolidación diferida. Estas propiedades permiten mejorar el rendimiento en contextos específicos y lo hacen especialmente útil en algoritmos como Dijkstra y Prim.

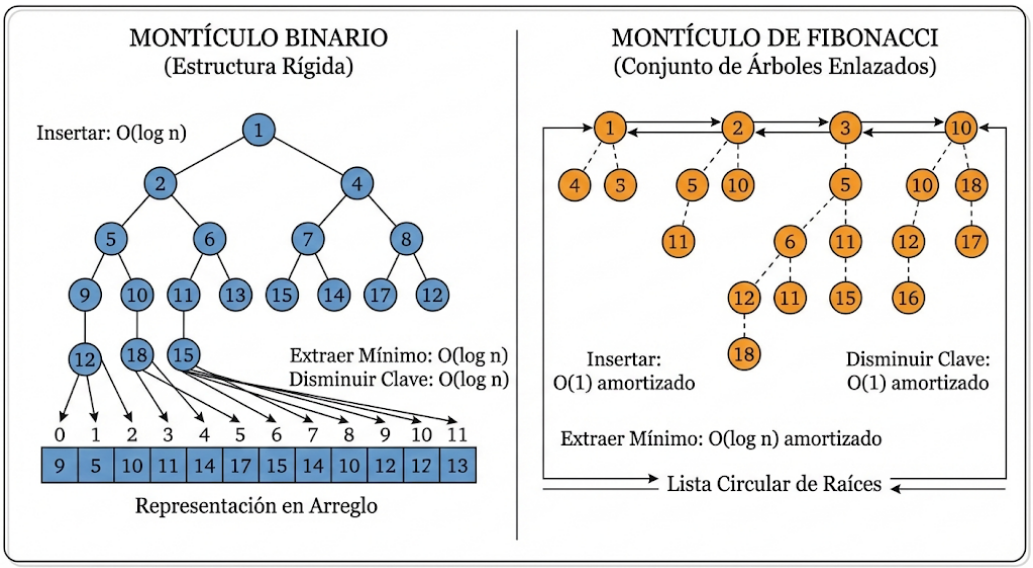


Ilustración 16: Comparación conceptual entre montículo binario y montículo de Fibonacci, destacando su estructura, operaciones principales y costos computacionales. Imagen creada usando herramientas de IA con información proporcionada por la autora.

Lectura complementaria

Para profundizar los conceptos abordados, se invita a consultar el siguiente recurso complementario: https://www.geeksforgeeks.org/dsa/fibonacci-heap-set-1-introduction/

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 3: Sistema interactivo de heaps.**

5. Conjuntos disjuntos y estructura Union-Find: operaciones de unión y búsqueda, y aplicaciones.

5.1 Conjuntos disjuntos: modelo y representación

Un sistema de conjuntos disjuntos representa una partición del universo U: cada elemento pertenece a exactamente un subconjunto y los subconjuntos no se solapan entre sí. La estructura Union Find mantiene esta partición mediante operaciones de fusión y consulta. La representación más utilizada modela cada subconjunto como un árbol enraizado dentro de un bosque: cada nodo apunta a un padre y la raíz actúa como representante del conjunto (Cormen et al., 2009).

La elección de esta representación no es decorativa: transforma el problema abstracto de equivalencia (misma componente) en una operación concreta sobre punteros (parent), donde la pertenencia se decide comparando representantes.

Énfasis

Para comprender el funcionamiento de la estructura Union Find, es necesario precisar algunos conceptos fundamentales que le dan sustento teórico y permiten interpretar correctamente su implementación:

- Conjuntos disjuntos: Colección de subconjuntos que conforman una partición de un universo, con intersección vacía entre subconjuntos distintos.
- Partición: Familia de subconjuntos no vacíos, disjuntos dos a dos, cuya unión es el conjunto total.
- Bosque: Conjunto de árboles disjuntos; en Union-Find, cada árbol representa un conjunto.
- Representante: Elemento identificador del conjunto; se implementa como la raíz del árbol.

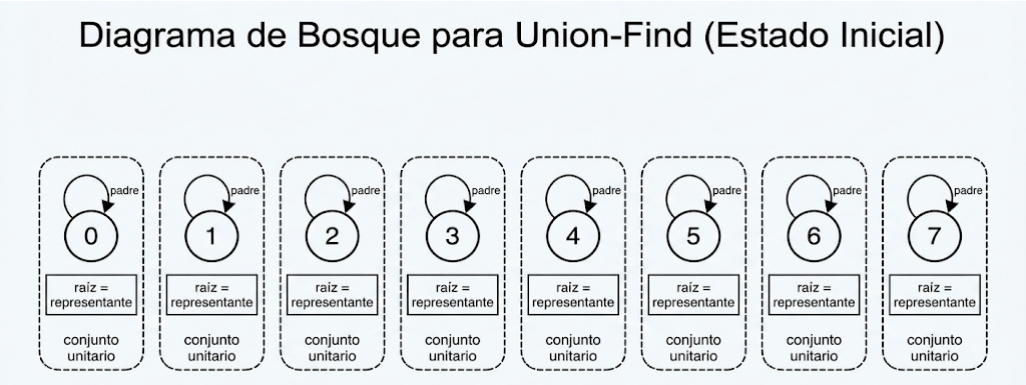


Ilustración 17: Diagrama del estado inicial de conjuntos disjuntos en la estructura Union-Find. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

5.1.1 Ejemplo 1: Conjuntos disjuntos

El estado inicial de Union Find se construye a partir de conjuntos unitarios. Esto materializa la partición base {{0}, {1}, …, {n – 1}}, que constituye el punto de partida necesario para que la operación union actúe sobre una estructura correctamente definida (Cormen et al., 2009).

```python
def make_sets(n):
    # Cada elemento inicia como representante de su propio conjunto
    return list(range(n))

# Prueba simple
parent = make_sets(6)
print(parent)
assert parent == [0, 1, 2, 3, 4, 5]
print("OK")
```

Captura 12: Inicialización de conjuntos disjuntos. Implementación básica de make_sets desarrollada en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`list(range(n))`	Construye `parent` con `parent[i]=i`, indicando que cada nodo es raíz (conjunto unitario).
`parent`	Arreglo de padres: codifica un bosque de raíces, una por conjunto.
`assert parent == ...`	Evidencia que la partición inicial es la esperada y reproducible.
Errores comunes	Usar elementos no consecutivos sin mapeo → prevenir con un diccionario `id→indice` si el universo no es `0..n-1`.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

5.2 Operación Find: búsqueda de representante

La operación find(x) identifica el conjunto al que pertenece x devolviendo su representante. En el bosque, esto equivale a ascender por los punteros al padre hasta alcanzar una raíz r tal que parent[r] = r. La corrección de este procedimiento se apoya en una invariante estructural: si el bosque se mantiene libre de ciclos y cada nodo tiene un único padre, entonces todo camino que parte de un nodo conduce necesariamente a una única raíz (Cormen et al., 2009). En ausencia de optimizaciones, el costo de find depende de la altura del árbol, ya que una secuencia desfavorable de uniones puede generar árboles altos y, en consecuencia, consultas más costosas.

Énfasis

En este marco, find se entiende como la operación que retorna el representante del conjunto que contiene al elemento consultado, mientras que la raíz es el nodo que se referencia a sí mismo, es decir, parent[x] = x, y que actúa como identificador del conjunto.

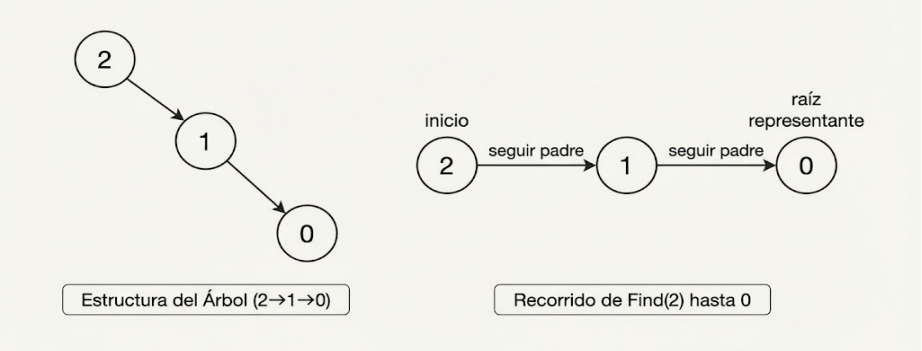


Ilustración 18: Ejemplo del recorrido de la operación find en Union-Find hasta la raíz representante. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

5.2.1 Ejemplo 2: Find

Find se implementa como un recorrido iterativo hasta la raíz. Este patrón refleja directamente la representación por bosque: cada consulta recorre el camino de padres definido por la estructura (Cormen et al., 2009).

```
1   def find(parent, x):
2       while parent[x] != x:
3           x = parent[x]
4       return x
5
6   # Bosque manual: 2 -> 1 -> 0 y 4 -> 3; 5 es singleton
7   parent = [0, 0, 1, 3, 3, 5]
8
9   print(find(parent, 2))  # representante de {0,1,2}
10  print(find(parent, 4))  # representante de {3,4}
11  print(find(parent, 5))  # singleton
12
13  assert find(parent, 2) == 0
14  assert find(parent, 4) == 3
15  assert find(parent, 5) == 5
16  print("OK")
```

Captura 13: Operación de búsqueda (find) en estructura Union-Find. Ejemplo de recorrido hasta el representante, implementado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`while parent[x] != x`	Itera mientras `x` no sea raíz; la condición identifica "no representante".
`x = parent[x]`	Avanza un nivel hacia el representante siguiendo el puntero al padre.
`return x`	Retorna la raíz alcanzada, que identifica el conjunto de pertenencia.
`assert ...`	Verifica que distintos nodos del mismo árbol comparten representante.
Errores comunes	Ciclos en `parent` → prevenir uniendo solo raíces en `union` (evita no terminación).

En términos de complejidad, la operación find tiene un costo O(h), donde h representa la altura del árbol. En el peor caso, y en ausencia de heurísticas de optimización, esta altura puede llegar a ser O(n), lo que implica un rendimiento lineal.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

5.3 Operación Union: unión de conjuntos

La operación union(a, b) fusiona los conjuntos que contienen a los elementos a y b. En términos del bosque, esto implica necesariamente unir raíces y no nodos arbitrarios. Para ello, primero se calculan ra = find(a) y rb = find(b); si ambas raíces son distintas, se enlaza una de ellas como hija de la otra, formando un único árbol que pasa a representar la unión de ambos conjuntos (Sedgewick & Wayne, 2011).

La propiedad central de union es que redefine la partición vigente, ya que reduce en uno el número de conjuntos cuando fusiona dos componentes diferentes. Esta operación constituye la base para modelar relaciones de equivalencia, como "se conectan" o "pertenecen al mismo grupo", en problemas vinculados con grafos y con técnicas de clustering basadas en conectividad.

En este marco, union se entiende como la operación que fusiona los conjuntos que contienen a dos elementos y actualiza la partición mantenida, mientras que una componente es el conjunto de elementos que comparten un mismo representante.

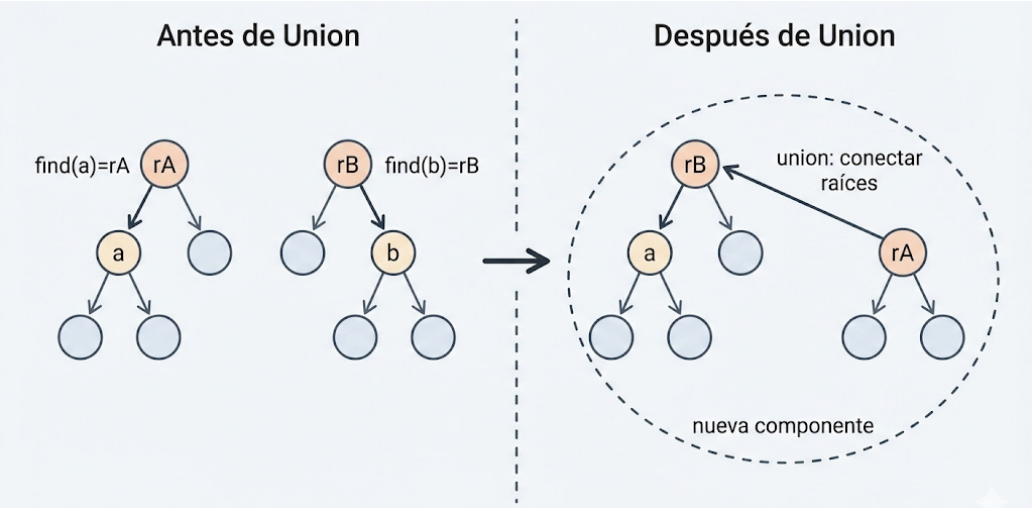


Ilustración 18: Proceso de la operación union antes y después de conectar las raíces de dos conjuntos disjuntos. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

5.3.1 Ejemplo 3: Union

El ejemplo implementa union simple enlazando la raíz de un conjunto bajo la raíz del otro. Aunque funcional, esta versión puede generar árboles altos; se usa como base para comprender la necesidad de optimizaciones (Sedgewick & Wayne, 2011).

C

```c
def make_sets(n):
    return list(range(n))

def find(parent, x):
    while parent[x] != x:
        x = parent[x]
    return x

def union(parent, a, b):
    ra, rb = find(parent, a), find(parent, b)
    if ra != rb:
        parent[rb] = ra

# Pruebas simples
parent = make_sets(6)
union(parent, 0, 1)
union(parent, 1, 2)
union(parent, 3, 4)

print("parent:", parent)
print("rep(2):", find(parent, 2), "rep(4):", find(parent, 4))

assert find(parent, 0) == find(parent, 2)
assert find(parent, 3) == find(parent, 4)
assert find(parent, 2) != find(parent, 5)
print("OK")
```

Captura 14: Operaciones de unión y consulta en Union-Find. Implementación completa de make_sets, find y union con pruebas simples, desarrollada en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`ra, rb = find(...)`	Obtiene representantes; la unión correcta se define sobre raíces.
`if ra != rb`	Evita unir el mismo conjunto consigo mismo (no cambia la partición).
`parent[rb] = ra`	Enlaza la raíz `rb` bajo `ra`, fusionando árboles.
`assert ...`	Evidencia que `union` crea conectividad y preserva disjunción con elementos no unidos.
Errores comunes	Enlazar `parent[b]=a` sin buscar raíces → prevenir aplicando `find` a ambos antes de enlazar.

En términos de complejidad, la operación union tiene un costo O(h), ya que depende de las llamadas a find necesarias para obtener las raíces de los elementos involucrados. Sin la aplicación de heurísticas de optimización, este costo queda determinado por la altura del árbol.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

5.4 Optimización: compresión de caminos

La compresión de caminos modifica find para reescribir el padre de los nodos visitados apuntándolos directamente a la raíz. Esto reduce drásticamente la repetición de recorridos largos en secuencias de consultas. En el análisis clásico, compresión de caminos combinada con una heurística de unión conduce a costos amortizados muy bajos (Tarjan, 1975; Tarjan & van Leeuwen, 1984).

A nivel estructural, la compresión no cambia la partición: conserva el representante de cada elemento. Solo optimiza la forma interna del bosque para acelerar futuras operaciones.

En este marco, la compresión de caminos se define como la técnica que, durante find, actualiza los punteros para que los nodos recorridos apunten directamente a la raíz, reduciendo así recorridos posteriores. El costo amortizado, por su parte, refiere al costo promedio por operación considerado sobre una secuencia, admitiendo picos puntuales siempre que el costo total acumulado permanezca acotado.

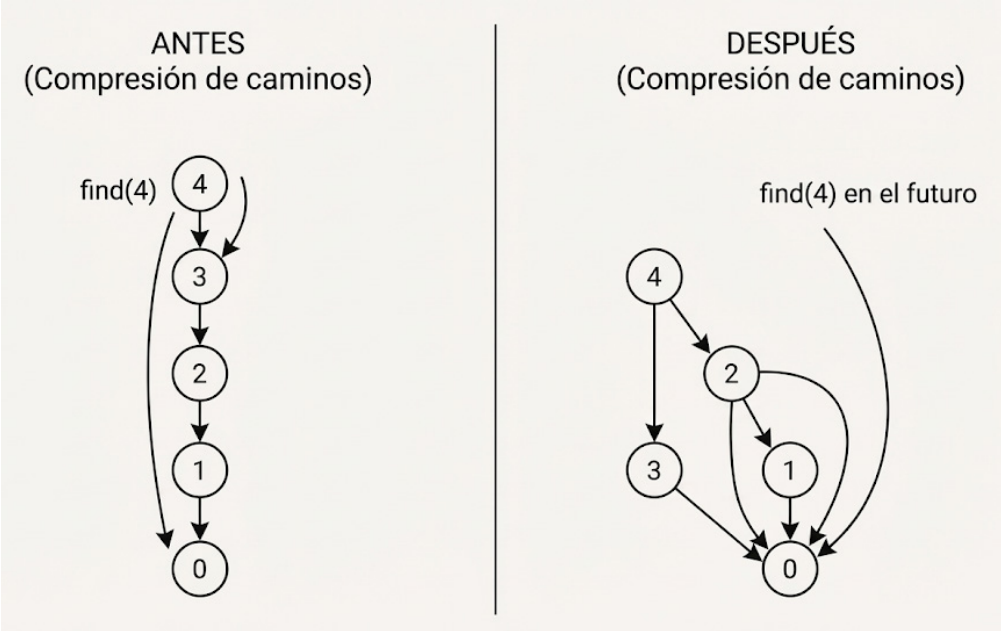


Ilustración 20: Efecto de la compresión de caminos en la operación find de Union-Find. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

5.4.1 Ejemplo 4: Find

El find recursivo permite comprimir caminos asignando parent[x] al resultado de buscar el representante del padre. Un único find puede transformar una cadena larga en una estructura "casi plana" (Tarjan, 1975).

```
1   def find_pc(parent, x):
2       if parent[x] != x:
3           parent[x] = find_pc(parent, parent[x])
4       return parent[x]
5
6   # Prueba simple: cadena 4 -> 3 -> 2 -> 1 -> 0
7   parent = [0, 0, 1, 2, 3]
8
9   root = find_pc(parent, 4)
10  print("root:", root)
11  print("parent después:", parent)
12
13  assert root == 0
14  assert parent == [0, 0, 0, 0, 0]
15  print("OK")
```

Captura 15: Implementación de búsqueda con compresión de caminos en Union-Find. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`if parent[x] != x`	Caso recursivo: `x` no es representante.
`parent[x] = find_pc(parent, parent[x])`	Comprime el camino: hace que `x` apunte directamente a la raíz encontrada.
`return parent[x]`	Retorna el representante; tras compresión, queda guardado en `parent[x]`.
`assert parent == ...`	Evidencia que todos los nodos del camino quedaron apuntando a la raíz.
Errores comunes	Implementar `find` recursivo sin la asignación → prevenir verificando `parent` tras `find`.

En términos de complejidad, cuando se aplica la heurística de unión por rango o por tamaño en combinación con compresión de caminos, la secuencia de operaciones alcanza un costo amortizado muy bajo (Tarjan, 1975; Tarjan & van Leeuwen, 1984).

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

5.5 Optimización: unión por rango o por tamaño

La unión por rango o por tamaño es una heurística que determina cómo enlazar las raíces durante la operación union con el objetivo de controlar el crecimiento de la altura del bosque. En la unión por tamaño, la raíz del conjunto con menor cantidad de elementos se enlaza bajo la raíz del conjunto más grande. En la unión por rango, en cambio, se emplea un valor auxiliar que aproxima la altura del árbol para decidir qué raíz debe convertirse en hija de la otra. Esta regla disminuye el riesgo de formar cadenas largas como consecuencia de secuencias adversas de uniones (Cormen et al., 2009).

La combinación de unión por rango con compresión de caminos produce el comportamiento amortizado estudiado formalmente en la literatura (Tarjan & van Leeuwen, 1984).

En este contexto, la unión por rango puede definirse como la heurística que enlaza la raíz de menor rango bajo la de mayor rango y, en caso de empate, incrementa el rango del nuevo representante. La unión por tamaño enlaza la raíz del conjunto con menos elementos bajo la del conjunto con más elementos. El rango, por su parte, es el valor auxiliar que orienta la decisión de enlace con el fin de limitar el crecimiento estructural del bosque.

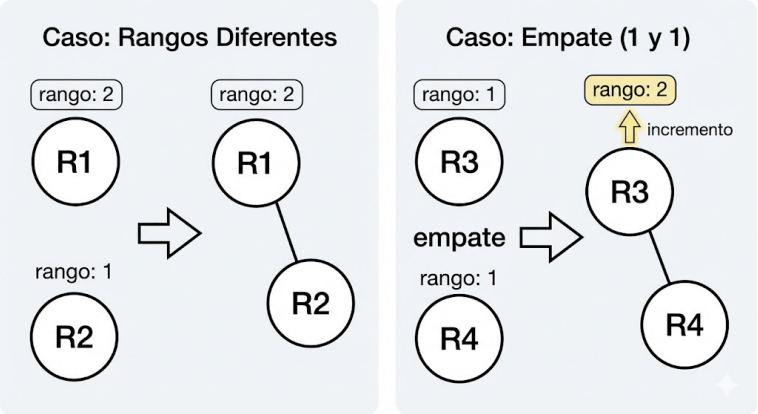


Ilustración 21: Unión por rango en Union-Find: casos con rangos diferentes y caso de empate. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

5.5.1 Ejemplo 5: Union-Find

El ejemplo implementa Union-Find con compresión de caminos y unión por rango. La intención es garantizar que las decisiones de unión no generen alturas innecesarias, mejorando consultas sucesivas (Cormen et al., 2009).

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False

        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra  # ra queda con mayor o igual rango

        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True

# Pruebas simples
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(3, 4)
uf.union(4, 5)
uf.union(2, 5)

reps = [uf.find(i) for i in range(7)]
print("representantes:", reps)
print("parent:", uf.parent)
print("rank:", uf.rank)

assert uf.find(0) == uf.find(5)
assert uf.find(6) != uf.find(0)
print("OK")
```

Captura 16: Clase Union-Find con unión por rango y compresión de caminos. Implementación realizada en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`rank = [0] * n`	Inicializa rangos; guía decisiones de `union` para controlar altura.
`find` con asignación	Realiza compresión de caminos: acelera búsquedas futuras.
`if self.rank[ra] < self.rank[rb]`	Garantiza que la raíz de mayor rango permanezca como representante.
`if self.rank[ra] == self.rank[rb]: self.rank[ra] += 1`	En empate, incrementa el rango de la nueva raíz (refleja posible aumento de altura).
`assert ...`	Evidencia que componentes se fusionan y elementos no unidos permanecen separados.
Errores comunes	No manejar empate de rangos → prevenir incrementando `rank` cuando corresponda.

Al combinar la unión por rango con la compresión de caminos, las operaciones alcanzan costos amortizados extremadamente bajos en secuencias largas (Tarjan & van Leeuwen, 1984).

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

5.6 Complejidad amortizada y aplicaciones (Kruskal, ciclos, componentes)

En problemas de grafos, una de las consultas fundamentales es determinar si dos nodos ya están conectados por las aristas procesadas. Union Find responde a esta pregunta comparando los representantes de cada elemento. En el algoritmo de Kruskal, para cada arista (u, v) ordenada por peso, si find(u) ≠ find(v), la arista conecta componentes distintas y se incorpora al árbol generador mínimo, seguida de la operación union(u, v); si los representantes coinciden, la arista formaría un ciclo y, por lo tanto, se descarta (Cormen et al., 2009).

La eficiencia de este enfoque se explica por el análisis amortizado de secuencias de operaciones con compresión de caminos y unión por rango/tamaño (Tarjan, 1975; Tarjan & van Leeuwen, 1984). En este contexto, el costo total de numerosas operaciones resulta cercano a lineal, al que se suma el costo de ordenar las aristas, lo que da lugar a la complejidad típica O(E log E) del algoritmo de Kruskal.

De esta manera, el algoritmo de Kruskal se define como un método voraz para construir un árbol abarcador mínimo, que agrega aristas en orden creciente de peso y evita la formación de ciclos mediante la verificación de componentes.

Un árbol abarcador mínimo, o MST, es un subgrafo que conecta todos los vértices de un grafo ponderado con la suma de pesos mínima posible y sin generar ciclos.

La detección de ciclos en grafos no dirigidos consiste en identificar una arista (u, v) tal que u y v ya pertenecen a la misma componente; en ese caso, su incorporación produciría un ciclo y debe descartarse.

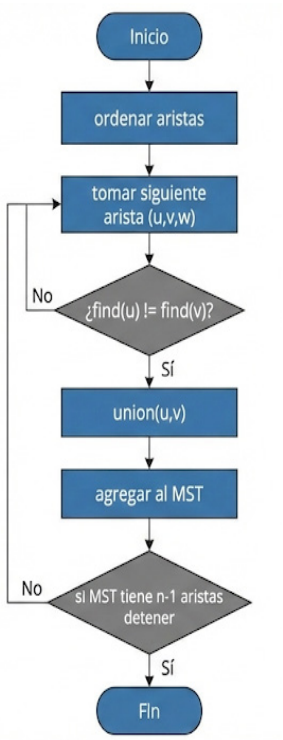


Ilustración 22: Diagrama de flujo del algoritmo de Kruskal utilizando la estructura Union-Find para detección de ciclos. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

5.6.1 Ejemplo 6: Kruskal

El ejemplo implementa el algoritmo de Kruskal utilizando una estructura Union Find optimizada. El criterio de "evitar ciclos" se traduce en una simple consulta de conectividad mediante find, lo que permite determinar si dos vértices ya pertenecen a la misma componente sin necesidad de realizar un recorrido del grafo por cada arista considerada (Cormen et al., 2009).

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True
```

Captura 17: Definición de la estructura Union-Find orientada a eficiencia amortizada. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def kruskal_mst(n, edges):
    # edges: (w, u, v)
    uf = UnionFind(n)
    mst = []
    total = 0
    for w, u, v in sorted(edges):
        if uf.union(u, v):
            mst.append((u, v, w))
            total += w
    return mst, total

# Prueba simple
n = 4
edges = [
    (1, 0, 1),
    (4, 0, 2),
    (3, 1, 2),
    (2, 1, 3),
    (5, 2, 3),
]

mst, total = kruskal_mst(n, edges)
print("mst:", mst)
print("total:", total)

assert len(mst) == n - 1
assert total == 6
print("OK")
```

Captura 18: Aplicación de Union-Find en el algoritmo de Kruskal para árboles generadores mínimos. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`sorted(edges)`	Ordena por peso creciente, requisito del criterio voraz de Kruskal.
`if uf.union(u, v)`	Agrega arista solo si conecta componentes distintas; si ya están conectados, `union` devuelve `False`.
`mst.append(...)` y `total += w`	Registra arista elegida y acumula peso del MST.
`assert len(mst) == n - 1`	Verifica condición necesaria de un árbol abarcador en un grafo conexo.
`assert total == 6`	Evidencia el costo del MST para el caso de prueba definido.
Errores comunes	No ordenar aristas → prevenir usando `sorted(edges)` siempre.

Énfasis

En el algoritmo de Kruskal, el ordenamiento de las aristas tiene complejidad O(E log E), mientras que las operaciones de Union Find presentan un costo amortizado O(E · α(V)) (Tarjan, 1975); en consecuencia, la complejidad total queda típicamente dominada por O(E log E) (Cormen et al., 2009).

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

5.7 Conexiones con aplicaciones en inteligencia artificial

Las conexiones con aplicaciones en Inteligencia Artificial se observan cuando Union-Find se utiliza para gestionar conectividad y agrupamientos de manera eficiente. En particular, permite construir agrupamientos basados en conectividad (por ejemplo, definir un grafo de similitud y formar componentes a partir de un umbral), lo que habilita una primera organización de datos en grupos sin requerir algoritmos de clustering iterativos.

También se emplea como soporte en tareas de segmentación por componentes conexas, donde los píxeles o superpíxeles se modelan como nodos de un grafo y las uniones determinan regiones conectadas que funcionan como preprocesamiento para etapas posteriores.

En problemas de datos relacionales, resulta aplicable a la consolidación de entidades (entity resolution) cuando se establecen reglas que indican equivalencia entre registros: cada regla se traduce en una unión y, luego, la consulta del representante recupera el grupo resultante. Asimismo, es útil en pipelines de análisis de grafos que requieren conectividad incremental, ya que permite incorporar relaciones de forma acumulativa y consultar componentes sin recomputar recorridos completos.

Finalmente, se vincula con el uso de árboles abarcadores mínimos como estructura base para variantes de clustering: una vez construido el MST, la separación en grupos puede obtenerse eliminando aristas de mayor peso, lo que convierte a Kruskal más Union-Find en un componente operativo dentro de ese flujo.

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 4–Union-Find**

6. Fundamentos de recursión en estructuras

6.1 Árboles y listas anidadas

La recursión permite modelar estructuras auto-semejantes a partir de un caso base, que corresponde a la estructura vacía o a una hoja, y de un paso recursivo, que consiste en resolver las subestructuras y combinar sus resultados. En el caso de los árboles, los recorridos como preorden, inorden y postorden se especifican de manera natural a partir de la definición recursiva de un nodo y de sus subárboles (Cormen et al., 2009).

La recursión estructural es la técnica que define una operación sobre una estructura recursiva aplicándola a sus subestructuras, apoyándose en un caso base que detiene la expansión. El caso base constituye la condición mínima que devuelve un resultado sin invocar nuevas llamadas recursivas. Por su parte, el recorrido en profundidad es la estrategia que explora completamente un subárbol antes de avanzar hacia otro.

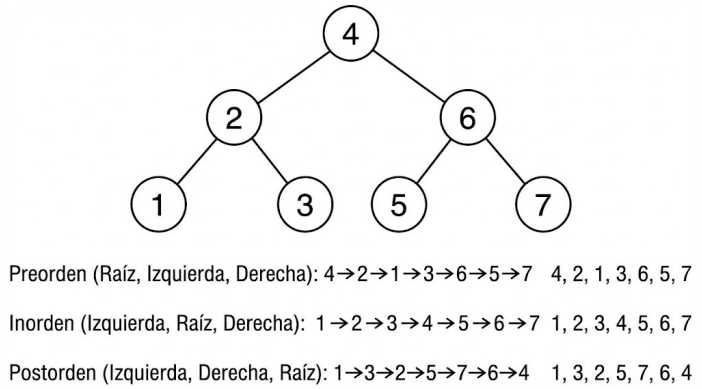


Ilustración 23: Recorridos en árbol binario (preorden, inorden y postorden). Secuencias de visita para un árbol de ejemplo, mostrando el orden de recorrido según la posición de la raíz. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

6.1.1 Ejemplo 1: Recorrido recursivo (preorden/inorden/postorden) materializando listas

Cuando el objetivo es obtener todos los elementos para usos posteriores, por ejemplo, en validaciones globales o procesos de serialización, resulta válido materializar el recorrido en una lista que conserve el orden deseado.

En el caso de los árboles, los recorridos clásicos se formulan de manera directa mediante recursión estructural sobre el subárbol izquierdo y el subárbol derecho, siguiendo la definición recursiva del nodo (Cormen et al., 2009).

```python
from dataclasses import dataclass
from typing import Optional, List

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

def preorder_list(root: Optional[Node]) -> List[int]:
    if root is None:
        return []
    return [root.key] + preorder_list(root.left) + preorder_list(root.right)

def inorder_list(root: Optional[Node]) -> List[int]:
    if root is None:
        return []
    return inorder_list(root.left) + [root.key] + inorder_list(root.right)

def postorder_list(root: Optional[Node]) -> List[int]:
    if root is None:
        return []
    return postorder_list(root.left) + postorder_list(root.right) + [root.key]

if __name__ == "__main__":
    # Árbol de prueba:
    #       4
    #     /   \
    #    2     6
    #   / \   / \
    #  1  3  5  7
    t = Node(4,
             Node(2, Node(1), Node(3)),
             Node(6, Node(5), Node(7)))

    assert preorder_list(t) == [4, 2, 1, 3, 6, 5, 7]
    assert inorder_list(t) == [1, 2, 3, 4, 5, 6, 7]
    assert postorder_list(t) == [1, 3, 2, 5, 7, 6, 4]
    print("OK: recorridos recursivos materializados.")
```

Captura 19: Recorridos recursivos materializados (preorden, inorden y postorden). Código en Python que define la clase Node y tres funciones recursivas que devuelven listas con el orden de visita; valida resultados con assert sobre un árbol de prueba. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Caso base (`root is None`)	Detiene la recursión y representa subárbol vacío como lista vacía.
Combinación de resultados	Construye el recorrido concatenando listas de subproblemas (izq/der).
Pre/In/Post orden	Cambia la posición de `root.key` para definir el orden del recorrido.
Evidencia (assert/print)	Verifica resultados esperados para un árbol conocido.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.2 Patrones de recursión avanzada: divide & conquer y backtracking

En el enfoque divide and conquer, el problema se descompone en subproblemas independientes que se resuelven de manera recursiva y cuyos resultados luego se combinan; en estructuras arbóreas, este esquema se observa, por ejemplo, al calcular una propiedad global a partir de los valores obtenidos en los subárboles. En cambio, el backtracking explora un espacio de soluciones parciales mediante un proceso de decisión–deshacer (push/pop) y poda cuando una restricción se viola (Cormen et al., 2009).

Énfasis

El esquema divide and conquer se define como una estrategia recursiva que divide un problema en subproblemas, los resuelve recursivamente y combina sus resultados. El backtracking es una técnica de búsqueda sistemática que construye soluciones parciales y revierte decisiones cuando estas no conducen a una solución válida. La poda, por su parte, es la regla que permite detener la exploración de una rama cuando se determina que no puede producir una solución válida.

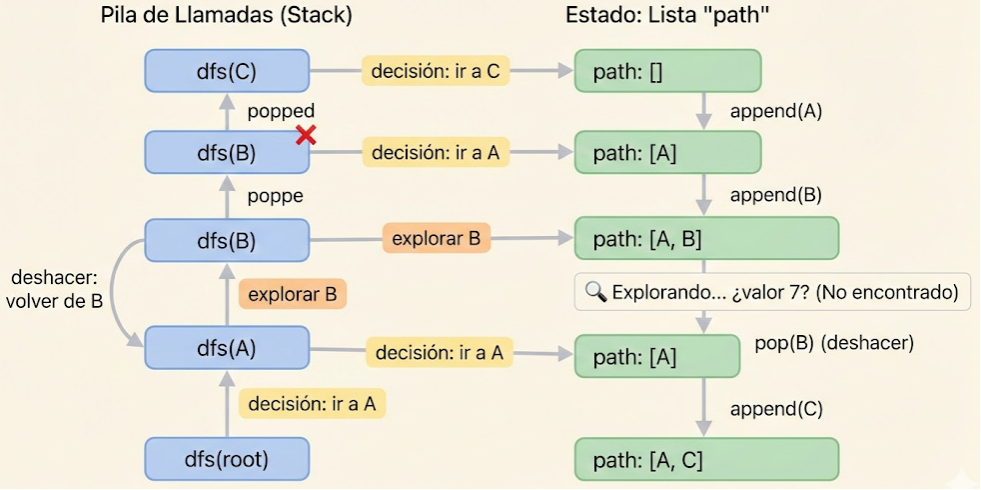


Ilustración 24: Backtracking en DFS: pila de llamadas y estado path. Representación del proceso decisión–exploración–deshacer, vinculando la pila de recursión con la lista de camino parcial. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

6.2.1 Ejemplo 2: Backtracking: encontrar un camino raíz→nodo (DFS con deshacer)

Encontrar un camino en un árbol puede formularse como búsqueda en profundidad con una solución parcial: la lista de nodos visitados. Backtracking agrega el nodo actual, explora, y si falla revierte el agregado, manteniendo consistente el estado parcial (Cormen et al., 2009).

```python
from dataclasses import dataclass
from typing import Optional, List

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

def find_path(root: Optional[Node], target: int) -> List[int]:
    path: List[int] = []

    def dfs(n: Optional[Node]) -> bool:
        if n is None:
            return False
        path.append(n.key)              # decisión
        if n.key == target:
            return True
        if dfs(n.left) or dfs(n.right):  # explorar
            return True
        path.pop()                      # deshacer (backtrack)
        return False

    return path if dfs(root) else []

if __name__ == "__main__":
    t = Node(8,
             Node(3, Node(1), Node(6, Node(4), Node(7))),
             Node(10, None, Node(14, Node(13), None)))

    assert find_path(t, 7) == [8, 3, 6, 7]
    assert find_path(t, 13) == [8, 10, 14, 13]
    assert find_path(t, 999) == []
    print("OK: backtracking para camino raíz→nodo.")

```

Captura 20: Backtracking para obtener el camino raíz→nodo (find_path). Implementación de DFS con estado path y operación de deshacer (pop) cuando una rama no conduce al objetivo; incluye pruebas con assert para objetivos existentes e inexistentes. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Estado parcial (`path`)	Representa la solución en construcción (camino actual).
Decisión (`append`)	Agrega el nodo actual antes de explorar subproblemas.
Poda implícita	Un `True` evita explorar ramas restantes al hallar el objetivo.
Deshacer (`pop`)	Revierte estado cuando la rama no conduce a solución.
Evidencia	`assert` valida casos existente/no existente.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.3 Optimización práctica: profundidad, coste de pila, memoización y alternativas iterativas

La recursión utiliza la pila de llamadas para gestionar el estado de cada invocación; en Python existe un límite de recursión configurable cuyo propósito es prevenir desbordes de la pila del intérprete (Python Software Foundation, 2025).

Cuando la profundidad de llamadas puede ser elevada, es conveniente reemplazar la estrategia recursiva por una simulación iterativa que emplee una pila explícita. Asimismo, en estructuras donde aparecen subproblemas solapados, como en los grafos acíclicos dirigidos (DAG), la memoización permite evitar la recomputación de resultados ya obtenidos (Cormen et al., 2009).

Énfasis

Definiciones clave:

- **Profundidad de recursión:** Número máximo de activaciones anidadas de función antes de retornar.
- **Pila explícita:** Estructura (p. ej., lista) que simula la pila de llamadas para una versión iterativa.
- **Memoización:** Almacenamiento de resultados de subproblemas para reutilización posterior.

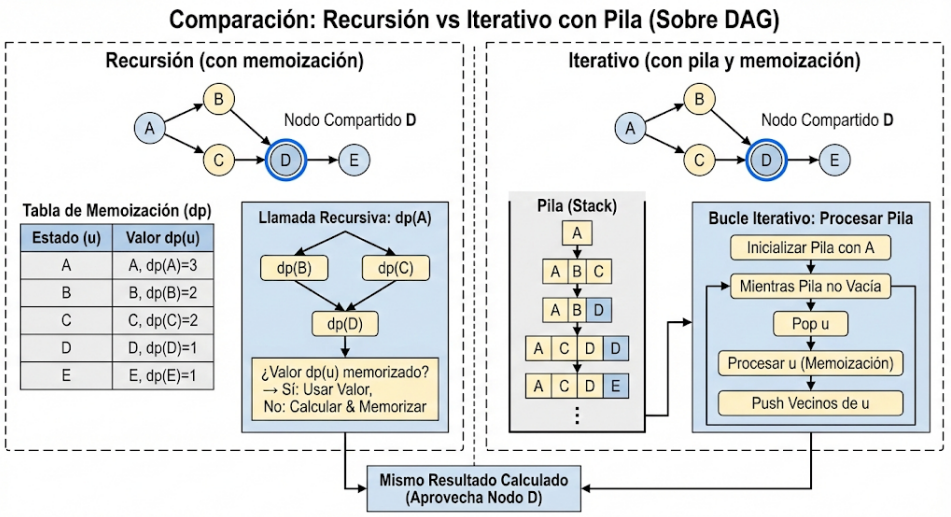


Ilustración 25: Comparación recursiva vs iterativa con pila en un DAG con memoización. Esquema que muestra cómo ambas estrategias reutilizan el resultado del nodo compartido (D) mediante una tabla dp, evitando recomputación y alcanzando el mismo valor final. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

6.3.1 Ejemplo 3: Alternativa iterativa: preorden con pila explícita (evita profundidad)

Un recorrido DFS puede implementarse sin recursión manteniendo una pila explícita de nodos pendientes. Esto evita depender del límite de recursión del intérprete y hace visible el costo espacial del algoritmo (Python Software Foundation, 2025).

```python
from dataclasses import dataclass
from typing import Optional, List

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

def preorder_iterative(root: Optional[Node]) -> List[int]:
    if root is None:
        return []
    out: List[int] = []
    stack: List[Node] = [root]
    while stack:
        n = stack.pop()
        out.append(n.key)
        # primero derecha para que izquierda quede arriba de la pila
        if n.right is not None:
            stack.append(n.right)
        if n.left is not None:
            stack.append(n.left)
    return out

if __name__ == "__main__":
    t = Node(4,
             Node(2, Node(1), Node(3)),
             Node(6, Node(5), Node(7)))
    assert preorder_iterative(t) == [4, 2, 1, 3, 6, 5, 7]
    print("OK: preorden iterativo con pila explícita.")
```

Captura 21: Preorden iterativo con pila explícita. Versión no recursiva del recorrido preorden que utiliza una lista como pila (stack) para simular la exploración DFS; asegura el orden apilando primero el hijo derecho. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Estructura auxiliar (`stack`)	Simula activaciones recursivas pendientes.
Orden de apilado	Se apila derecha y luego izquierda para visitar izquierda primero.
Robustez frente a profundidad	Evita el consumo de pila del intérprete por llamadas anidadas.
Evidencia	`assert` compara con el preorden esperado.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.3.2 Ejemplo 4: Memoización en DAG: contar caminos (subproblemas solapados)

En un DAG, múltiples rutas pueden llegar al mismo nodo, generando recomputación en una DFS recursiva. Memoización guarda el número de caminos desde cada nodo hacia destinos, reutilizando resultados (Cormen et al., 2009).

```python
from functools import lru_cache
from typing import Dict, List

def count_paths_dag(adj: Dict[str, List[str]], start: str, goal: str) -> int:
    @lru_cache(maxsize=None)
    def dp(u: str) -> int:
        if u == goal:
            return 1
        return sum(dp(v) for v in adj.get(u, []))
    return dp(start)

if __name__ == "__main__":
    # DAG:
    # A -> B, C
    # B -> D
    # C -> D
    # D -> E
    # (A a E tiene 2 caminos: A-B-D-E y A-C-D-E)
    adj = {"A": ["B", "C"], "B": ["D"], "C": ["D"], "D": ["E"], "E": []}
    assert count_paths_dag(adj, "A", "E") == 2
    assert count_paths_dag(adj, "B", "E") == 1
    print("OK: memoización en DAG para conteo de caminos.")
```

Captura 22: Memoización en DAG para conteo de caminos (lru_cache). Función que cuenta caminos desde un nodo inicio a un objetivo en un grafo acíclico dirigido, evitando recomputación mediante cache de subproblemas (dp). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Subproblema `dp(u)`	Número de caminos desde `u` hasta `goal`.
Solapamiento	`D` es alcanzable desde `B` y `C`; sin cache se recomputa.
`lru_cache`	Implementa memoización automática por clave de argumento.
Evidencia	`assert` valida conteos esperados.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.4 Iteradores e iterables en Python: protocolo, __iter__, __next__ e iter()

El protocolo de iteración de Python se formaliza en el PEP 234: un iterable provee un iterador, y un iterador produce el siguiente elemento y señala su término mediante la excepción StopIteration (Yee & van Rossum, 2001). Este esquema desacopla la estructura, es decir, el contenedor, del modo de recorrido, lo que permite definir recorridos personalizados sin necesidad de materializar los resultados.

Énfasis

Un iterable es un objeto que puede producir un iterador mediante el método iter(), o, en ciertos casos, a través del protocolo de secuencia. Un iterador es un objeto que implementa next(), método que devuelve el siguiente elemento o lanza la excepción StopIteration cuando no quedan elementos por recorrer. StopIteration es la excepción estándar que indica la finalización de la iteración.

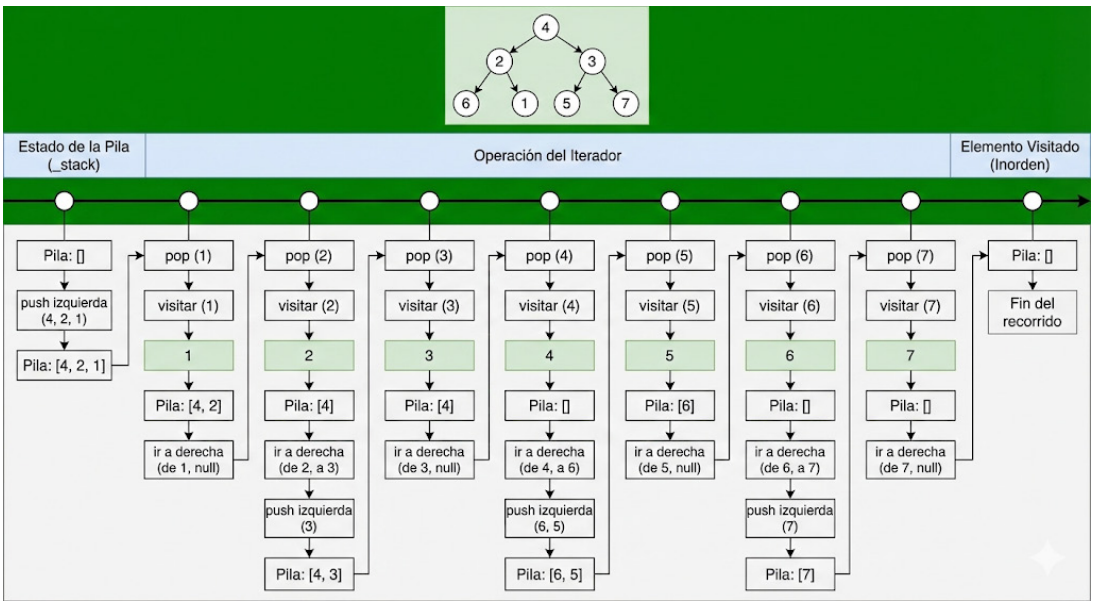


Ilustración 26: Iterador inorden con pila explícita: evolución del estado _stack. Trazado paso a paso de push/pop y visita de nodos durante un recorrido inorden sin recursión. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

6.4.1 Ejemplo 5: Iterador como clase: recorrido inorden sin recursión

Un iterador encapsula el estado del recorrido y permite consumir elementos de forma incremental. En inorden, el estado se mantiene con una pila que simula el descenso por la rama izquierda y el retroceso controlado (Yee & van Rossum, 2001).

```python
from dataclasses import dataclass
from typing import Optional, Iterator, List

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

class InOrderIterator:
    def __init__(self, root: Optional[Node]):
        self._stack: List[Node] = []
        self._push_left(root)

    def _push_left(self, n: Optional[Node]) -> None:
        while n is not None:
            self._stack.append(n)
            n = n.left

    def __iter__(self) -> "InOrderIterator":
        return self

    def __next__(self) -> int:
        if not self._stack:
            raise StopIteration
        n = self._stack.pop()
        if n.right is not None:
            self._push_left(n.right)
        return n.key

if __name__ == "__main__":
    t = Node(4,
             Node(2, Node(1), Node(3)),
             Node(6, Node(5), Node(7)))

    it = InOrderIterator(t)
    out = list(it)
    assert out == [1, 2, 3, 4, 5, 6, 7]
    print("OK: iterador de clase (inorden).")
```

Captura 23: Iterador de clase para recorrido inorden (__iter__/__next__). Implementación del protocolo iterador con una pila interna y el método _push_left para recorrer inorden sin recursión; prueba de corrección con list(it) y assert. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Estado del iterador (`_stack`)	Representa nodos pendientes de visitar en inorden.
Inicialización `_push_left`	Prepara el primer "próximo" descendiendo a la izquierda.
`__iter__` retorna `self`	Cumple el requisito de iterador consumible por `for` / `list` .
`__next__` y `StopIteration`	Produce elementos y señala fin según el protocolo.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.5 Generadores: yield, yield from, evaluación perezosa y composición

Los generadores, introducidos por el PEP 255, permiten producir valores de manera incremental preservando el estado entre emisiones, evitando la materialización completa cuando no es necesaria (Schemenauer et al., 2001). El PEP 380 incorpora *yield from* para delegar en subgeneradores o iterables, simplificando la composición de recorridos recursivos y preservando semántica del protocolo de iteración (Ewing, 2009).

Énfasis

Un generador es una función que produce un iterador, emitiendo valores mediante *yield* y reteniendo su estado local entre emisiones. La evaluación perezosa es una estrategia en la que los elementos se computan únicamente cuando el consumidor los solicita. La delegación con *yield from* es una construcción que reemite los valores de un subgenerador o iterable como si fueran propios.

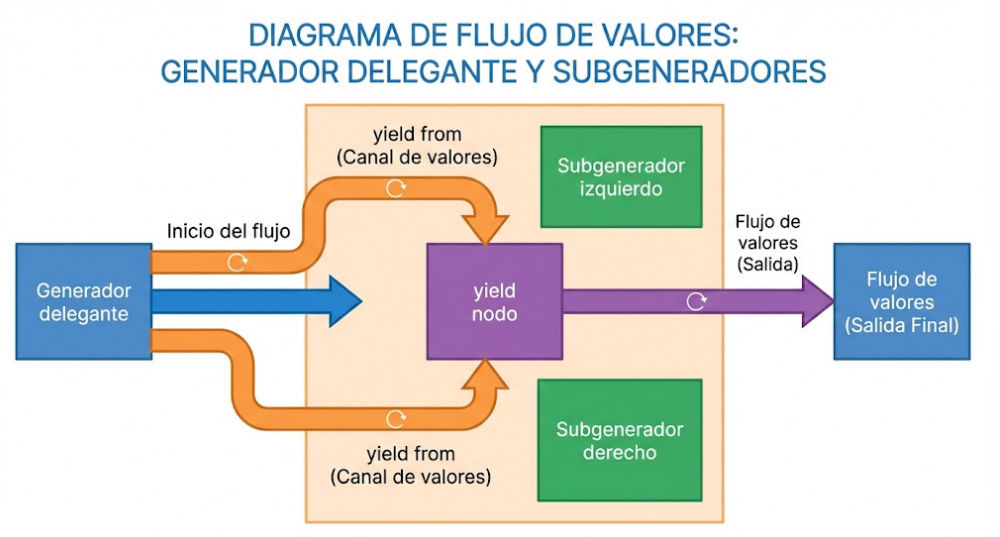


Ilustración 27: yield from como canal de delegación entre generador y subgeneradores. Flujo de valores donde el generador delega la emisión a recorridos de subárbol izquierdo/derecho y emite el nodo intermedio. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

6.5.1 Ejemplo 6: Recorrido generador (preorden con yield)

Cuando el consumidor puede procesar los elementos uno a uno (streaming), un generador evita la creación de listas intermedias. En un recorrido en preorden, se emite primero el nodo actual y luego se delega el recorrido a los subárboles (Schemenauer et al., 2001).

```python
from dataclasses import dataclass
from typing import Optional, Iterator

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

def preorder_gen(root: Optional[Node]) -> Iterator[int]:
    if root is None:
        return
    yield root.key
    yield from preorder_gen(root.left)
    yield from preorder_gen(root.right)

if __name__ == "__main__":
    t = Node(4,
             Node(2, Node(1), Node(3)),
             Node(6, Node(5), Node(7)))
    out = list(preorder_gen(t))
    assert out == [4, 2, 1, 3, 6, 5, 7]
    print("OK: preorden con generador (yield/yield from).")
```

Captura 24: Generador de preorden con yield y yield from. Recorrido preorden perezoso que emite claves incrementalmente y delega en subárboles; se materializa solo en la prueba (list(...)) para verificar el orden. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
`yield root.key`	Emite el valor actual sin construir listas.
`yield from`	Compone el recorrido delegando en subárboles.
Terminación (`root is None`)	Detiene la emisión para subárbol vacío.
Evidencia	Materialización final solo para prueba (`list(...)`).

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.5.2 Ejemplo 7: yield from para componer recorridos: inorder como delegación estructural

yield from se justifica cuando se desea expresar que un generador es la concatenación de subgeneradores de forma directa. En un recorrido inorder: subárbol izquierdo, nodo y subárbol derecho (Ewing, 2009).

```python
from dataclasses import dataclass
from typing import Optional, Iterator

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

def inorder_gen(root: Optional[Node]) -> Iterator[int]:
    if root is None:
        return
    yield from inorder_gen(root.left)
    yield root.key
    yield from inorder_gen(root.right)

if __name__ == "__main__":
    t = Node(4,
             Node(2, Node(1), Node(3)),
             Node(6, Node(5), Node(7)))
    assert list(inorder_gen(t)) == [1, 2, 3, 4, 5, 6, 7]
    print("OK: inorder con yield from (delegación).")
```

Captura 25: Generador de inorden con delegación (yield from). Recorrido inorden perezoso que compone la visita del subárbol izquierdo, la raíz y el subárbol derecho mediante delegación; incluye assert de la secuencia esperada. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Delegación izquierda	Reemite el inorder del subárbol izquierdo.
Emisión del nodo	Inserta el nodo entre ambas delegaciones.
Delegación derecha	Completa la definición recursiva del inorder.
Semántica de composición	Modela concatenación lógica sin bucles explícitos.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.6 Recorridos de grandes estructuras: DFS/BFS perezosos, streaming y reducción de memoria

Para BFS se requiere una cola; *collections.deque* está diseñada para inserciones y extracciones eficientes en los extremos, por lo que resulta adecuada para colas de recorrido (Python Software Foundation, n.d.). En estructuras grandes o implícitas, generadas bajo demanda, emitir nodos de manera incremental permite realizar streaming y evita almacenar todos los resultados, lo que ayuda a controlar el uso de memoria (Schemenauer et al., 2001).

Énfasis

BFS (Breadth-First Search) es un recorrido por niveles que expande primero los vecinos o descendientes inmediatos. Streaming es el procesamiento incremental de los elementos emitidos, sin requerir la colección completa en memoria. Una estructura implícita es aquella cuyos nodos o vecinos se generan mediante una función y no por una materialización previa.

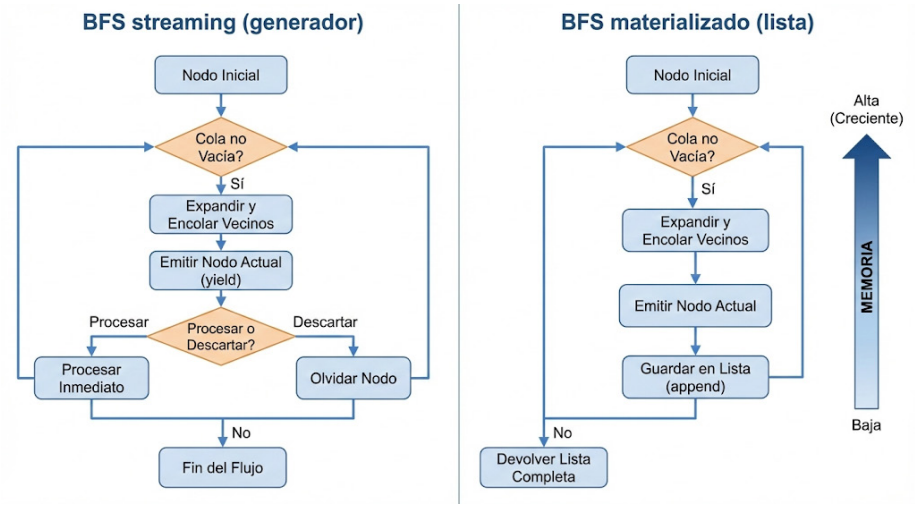


Ilustración 28: BFS perezoso vs BFS materializado: impacto en memoria y flujo de procesamiento. Comparación entre emitir nodos incrementalmente (generador) y acumularlos en una lista, destacando la diferencia de uso de memoria. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

6.6.1 Ejemplo 8: BFS con generador (emisión incremental con deque)

BFS requiere una cola FIFO y deque soporta append y popleft como operaciones naturales de cola. Convertir BFS en generador permite consumir por streaming (por ejemplo, detenerse al encontrar un objetivo) sin materializar todo el recorrido (Python Software Foundation, n.d.).

```python
from collections import deque
from dataclasses import dataclass
from typing import Optional, Iterator

@dataclass
class Node:
    key: int
    left: Optional["Node"] = None
    right: Optional["Node"] = None

def bfs_gen(root: Optional[Node]) -> Iterator[int]:
    if root is None:
        return
    q = deque([root])
    while q:
        n = q.popleft()
        yield n.key
        if n.left is not None:
            q.append(n.left)
        if n.right is not None:
            q.append(n.right)

if __name__ == "__main__":
    t = Node(4,
             Node(2, Node(1), Node(3)),
             Node(6, Node(5), Node(7)))

    assert list(bfs_gen(t)) == [4, 2, 6, 1, 3, 5, 7]
    # Consumo parcial: solo primeros 3 elementos
    it = bfs_gen(t)
    first3 = [next(it), next(it), next(it)]
    assert first3 == [4, 2, 6]
    print("OK: BFS generador con consumo incremental.")
```

Captura 26: BFS como generador con deque y consumo parcial. Recorrido en anchura que emite nodos al desencolarlos (popleft) y permite cortar temprano consumiendo solo los primeros elementos con next. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Cola FIFO (`deque`)	Implementa el orden por niveles mediante `popleft` .
Generador	Emite cada nodo cuando se desencola, habilitando consumo parcial.
Corte temprano	El consumidor puede detenerse sin recorrer todo el árbol.
Evidencia	`assert` y extracción de primeros 3 valores.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.6.2 Ejemplo 9: "Estructura grande" implícita: comparar streaming vs lista materializada

En una estructura implícita (por ejemplo, un árbol binario completo representado por índices), el recorrido puede producirse sin crear todos los nodos como objetos. Materializar todos los elementos visitados en una lista incrementa el uso de memoria; el streaming permite realizar agregaciones (como conteo o suma) sin retener el conjunto completo (Schemenauer et al., 2001).

```python
from collections import deque
import tracemalloc

def implicit_children(i: int, n: int):
    left = 2*i + 1
    right = 2*i + 2
    if left < n:
        yield left
    if right < n:
        yield right

def bfs_implicit_gen(n: int):
    # Recorre índices 0..n-1 como árbol binario implícito
    if n <= 0:
        return
    q = deque([0])
    while q:
        i = q.popleft()
        yield i
        for c in implicit_children(i, n):
            q.append(c)
```

Captura 27: BFS sobre estructura implícita: generación de hijos bajo demanda. Funciones que modelan un "árbol" binario por índices (sin crear nodos) y recorren BFS emitiendo índices; prepara el escenario para comparar streaming vs materialización. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def measure_peak_memory_streaming(n: int) -> int:
    tracemalloc.start()
    total = 0
    for x in bfs_implicit_gen(n):
        total += x  # ejemplo de reducción (no guarda todos)
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak

def measure_peak_memory_materialized(n: int) -> int:
    tracemalloc.start()
    data = list(bfs_implicit_gen(n))  # materializa
    _ = data[0] if data else None
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak
```

Captura 28: Medición de pico de memoria: streaming vs lista materializada (tracemalloc). Código que instrumenta el uso de memoria para comparar la reducción incremental (sin guardar todos los elementos) contra la materialización completa con list(...). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
if __name__ == "__main__":
    n = 50000  # ajustar según entorno
    peak_stream = measure_peak_memory_streaming(n)
    peak_list = measure_peak_memory_materialized(n)
    print("Peak streaming (bytes):", peak_stream)
    print("Peak materializado (bytes):", peak_list)
    assert peak_list >= peak_stream
    print("OK: comparación de picos de memoria (tendencia).")
```

Captura 29: Ejecución comparativa y validación del pico de memoria (streaming vs materialización). Fragmento que define el tamaño n, calcula el pico de memoria para el recorrido perezoso y para la versión materializada, imprime ambos valores y verifica con assert la tendencia esperada (mayor o igual consumo al materializar). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación
Estructura implícita	Los "nodos" son índices; los hijos se calculan, no se almacenan.
Streaming (reducción)	Recorre y acumula sin guardar todos los valores visitados.
Materialización (`list(...)`)	Guarda todo el recorrido: costo espacial proporcional al número de nodos.
Medición (`tracemalloc`)	Estima pico de memoria para comparar tendencias en el mismo entorno.
Evidencia	`assert` espera pico materializado ≥ pico streaming.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f492b56b-5285-422b-b374-67d3bf744efd

6.7 Aplicaciones en inteligencia artificial

Las técnicas desarrolladas articulan dos necesidades recurrentes en Inteligencia Artificial: explorar estructuras (árboles y espacios de estados) y controlar el costo computacional (tiempo y, en particular, memoria).

Los recorridos recursivos (preorden, inorden, postorden) constituyen una base directa para operar sobre árboles de decisión y sus variantes, ya sea para explicar una predicción mediante la ruta de decisiones, para serializar modelos, o para aplicar transformaciones estructurales sobre subárboles (Cormen et al., 2009). En problemas de búsqueda, la elección entre DFS y BFS modela decisiones algorítmicas centrales: DFS se alinea con exploración profunda y backtracking; BFS con exploración por niveles y hallazgo de soluciones de menor profundidad cuando las aristas son homogéneas, lo que resulta frecuente en formulaciones de planificación o exploración incremental (Cormen et al., 2009).

La incorporación del protocolo de y de generadores permite, además, separar el "qué recorrer" del "cómo consumir" los resultados. Un recorrido implementado como iterador o generador puede alimentar pipelines de IA sin materializar colecciones completas, habilitando evaluación perezosa para generar features bajo demanda, cortar temprano ante criterios de parada (por ejemplo, al encontrar un estado objetivo), o realizar reducciones (conteos, sumas, agregaciones) mientras se emiten elementos (Yee & van Rossum, 2001; Schemenauer et al., 2001).

Finalmente, la memorización formaliza un principio operativo clave: cuando existen subproblemas solapados (por ejemplo, en DAG o en cómputos reutilizables), almacenar resultados parciales reduce recomputación y estabiliza el rendimiento, acercando la solución a enfoques dinámicos usados en grafos computacionales y procesos de inferencia (Cormen et al., 2009). En conjunto, recursión avanzada e iteración perezosa constituyen herramientas complementarias para construir soluciones en IA que sean correctas, analizables y eficientes en el uso de recursos, especialmente cuando las estructuras crecen y el procesamiento debe sostenerse en flujo.

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 5: Recorridos, backtracking, iteradores y generadores.**

Palabras finales

El Módulo 4 consolidó un conjunto de herramientas que permiten diseñar y evaluar estructuras cuando el rendimiento depende de invariantes estrictas (altura acotada), del modelo de costo dominante (I/O en disco), o de la necesidad de operar de forma incremental sobre grandes volúmenes. Los árboles AVL y rojo-negro aportaron el marco para comprender cómo el balanceo sostiene operaciones logarítmicas y cómo las rotaciones preservan el orden in-order; los árboles B, B+ y B* introdujeron un cambio de perspectiva hacia la eficiencia por bloque y la reducción de altura en contextos persistentes; los montículos formalizaron la noción de prioridad y habilitaron selección y ordenamiento eficientes; Union-Find mostró cómo mantener particiones dinámicas con costos amorti-

zados bajos; y la recursión avanzada, junto con iteradores y generadores, ofreció un conjunto operativo para recorrer, buscar y procesar estructuras sin materializar resultados, controlando memoria mediante evaluación perezosa y streaming.

En términos de aplicación en inteligencia artificial, los contenidos del módulo se conectan con recorridos de árboles de decisión, iteración exploración de espacios de estados con DFS/BFS, construcción de pipelines que generan atributos bajo demanda, y optimización de cómputos reutilizables mediante memoización. En conjunto, las estructuras trabajadas permiten elegir representaciones consistentes con el problema y con las restricciones de ejecución, evitando degradaciones por entradas adversas, límites de recursión o materialización innecesaria.

Para continuar con esta línea de trabajo, se invita a avanzar al Módulo 5: Grafos y algoritmos de grafos, donde se formalizarán modelos de redes y relaciones (dirigidas y no dirigidas) y se desarrollarán algoritmos de recorrido y optimización sobre grafos, extendiendo las ideas de conectividad, búsqueda y costo que ya se introducen en este módulo.

Bibliografía consultada por el docente

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). The MIT Press. https://mitpress.mit.edu/9780262533058/introduction-to-algorithms/
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Ewing, G. (2009). *PEP 380 — Syntax for Delegating to a Subgenerator*. Python Enhancement Proposals. https://peps.python.org/pep-0380/
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. Wiley.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley.
- Mit.edu. Recuperado 20 de enero de 2026, de https://dspace.mit.edu/bitstream/handle/1721.1/36847/6-046JFall-2001/NR/rdonlyres/Electrical-Engineering-and-Computer-Science/6-046JIntroduction-to-AlgorithmsFall2001/CB7F315B-5ACF-48C3-84D5-3BEF792B3275/0/lecture10.pdf
- Mcgill.ca. Recuperado 20 de enero de 2026, de https://www.cs.mcgill.ca/~jeromew/teachings/251/F2022/COMP251_Lecture7_F2022.pdf
- Python Software Foundation. (s. f.). *bisect — Array bisection algorithm*. https://docs.python.org/3/library/bisect.html
- Python Software Foundation. (2024). *heapq — Heap queue algorithm*. https://docs.python.org/3/library/heapq.html

- Python Software Foundation. (2025). *sys.setrecursionlimit* (Documentación de Python 3.10, versión en español). https://docs.python.org/es/3.10/library/sys.html
- Python Software Foundation. (n.d.). *collections.deque* (Documentación oficial de Python). https://docs.python.org/3/library/collections.html
- *The AVL Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/AVL.html
- *The Red-Black Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/RedBlack.html
- *Red-Black Trees*. (s. f.). Opendatastructures.org. Recuperado 20 de enero de 2026, de https://opendatastructures.org/newhtml/ods/latex/redblack.html
- Schemenauer, N., Peters, T., & Hetland, M. L. (2001). *PEP 255 — Simple Generators*. Python Enhancement Proposals. https://peps.python.org/pep-0255/
- Silberschatz, A., Korth, H., & Sudarshan, S. (2020). *Database System Concepts*. McGraw-Hill.
- Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. Journal of the ACM, 22(2), 215–225.
- Tarjan, R. E., & van Leeuwen, J. (1984). Worst-case analysis of set union algorithms. Journal of the ACM, 31(2), 245–281.
- Ullman, J. (2016). *Data Structures and Algorithms*. Stanford Notes.
- Yee, K.-P., & van Rossum, G. (2001). *PEP 234 — Iterators*. Python Enhancement Proposals. https://peps.python.org/pep-0234/

Módulo 4 | Glosario

- **2-opt:** heurística de mejora para TSP que intenta reducir el costo invirtiendo (revirtiendo) segmentos de la ruta mediante el intercambio de dos aristas, evitando cruces y acortando el tour.

- **Alcanzabilidad:** propiedad que indica si existe al menos un camino desde un vértice origen hasta un vértice destino (en grafos dirigidos, respetando la dirección de las aristas).

- **Algoritmo de Kahn:** método para obtener un orden topológico en un DAG eliminando iterativamente vértices con indegree 0; si no se pueden eliminar todos, existe un ciclo.

- **Arco:** sinónimo usual de arista dirigida; par ordenado (u,v) que va desde hacia v.

- **Arista:** conexión entre dos vértices; en grafos no dirigidos no tiene orientación y en grafos dirigidos se interpreta como un arco.

- **Arista de retroceso (back edge):** en DFS sobre grafos dirigidos, arista que apunta a un vértice "en proceso" (gris), evidenciando un ciclo.

- **Arista inversa residual:** arista "de retorno" en la red residual que modela la posibilidad de deshacer parte del flujo enviado por una arista directa.

- **Arista segura:** arista que puede agregarse a un MST sin perder optimalidad, típicamente justificable por la propiedad del corte.

- **Árbol de caminos mínimos (SPT):** estructura implícita dada por los predecesores en un algoritmo de caminos mínimos desde una fuente; define, para cada vértice alcanzable, una ruta mínima desde la fuente.

- **Árbol de expansión mínima (MST):** subgrafo acíclico que conecta todos los vértices (si el grafo es conexo) con costo total mínimo.

- **BFS (Búsqueda en amplitud):** recorrido por niveles desde una fuente usando cola FIFO; en grafos no ponderados produce distancias mínimas en cantidad de aristas.

- **Bosque de expansión mínima (MSF):** conjunto de MST, uno por cada componente conexa de un grafo no conexo.

- **Camino:** secuencia de vértices [v0,v1,…,vk] tal que cada par consecutivo está unido por una arista/arco existente.

- **Camino aumentante:** en flujo máximo, camino de s a t en la red residual con capacidad residual positiva en todas sus aristas; permite incrementar el flujo.

- **Capacidad:** cota máxima de flujo que puede circular por una arista en una red de flujo.

- **Capacidad residual:** capacidad disponible restante en una arista de la red residual; indica cuánto flujo adicional puede enviarse (o revertirse) por esa conexión.

- **Ciclo:** camino cerrado (vuelve al vértice inicial) sin repetir aristas; en grafos dirigidos debe respetar direcciones.

- **Ciclo hamiltoniano:** ciclo que visita exactamente una vez cada vértice y retorna al inicio; base del TSP.

- **Clausura transitiva (Warshall):** matriz booleana que indica alcanzabilidad entre todos los pares de vértices; se obtiene con programación dinámica sobre la matriz de adyacencia.

- **Cola FIFO:** estructura "primero en entrar, primero en salir", usada por BFS para explorar por capas.

- **Cola de prioridad (min-heap):** estructura que permite extraer el mínimo eficientemente; usada en Dijkstra (y en Prim) para seleccionar la mejor opción siguiente.

- **Complejidad temporal:** medida del crecimiento del costo computacional según el tamaño de entrada (por ejemplo, O(V+E), O(ElogV)).

- **Componente conexa:** subconjunto máximo de vértices donde cada par está conectado por algún camino (en no dirigidos).

- **Compresión de caminos:** optimización de Union-Find que aplana la estructura durante find, acelerando futuras consultas.

- **Conservación de flujo:** restricción que exige, para todo vértice distinto de fuente y sumidero, que el flujo que entra sea igual al que sale.

- **Corte (S, T):** partición de los vértices en dos conjuntos; en flujo, se considera S alcanzable desde la fuente y T=V S.

- **Corte mínimo:** corte con capacidad total mínima; en redes de flujo se relaciona con el flujo máximo por el teorema max-flow min-cut.

- **Cuello de botella:** mínima capacidad residual a lo largo de un camino aumentante; determina cuánto se puede aumentar el flujo en esa iteración.

- **DAG (Grafo dirigido acíclico):** grafo dirigido sin ciclos; admite orden topológico.

- **DFS (Búsqueda en profundidad):** recorrido que avanza por ramas utilizando recursión o pila; útil para detección de ciclos, clasificación de aristas y orden topológico por tiempos de finalización.

- **Distancia mínima (caminos mínimos):** costo mínimo acumulado desde una fuente a un destino según los pesos de aristas (o número de aristas si es no ponderado).

- **Distancia tentativa:** mejor estimación conocida hasta el momento de la distancia desde la fuente a un vértice (típica en Dijkstra).
- **Dijkstra:** algoritmo de caminos mínimos desde una fuente en grafos con pesos no negativos; suele implementarse con min-heap para eficiencia.
- **Edmonds–Karp:** implementación de Ford–Fulkerson que usa BFS para hallar caminos aumentantes, garantizando complejidad polinómica.
- **Entrada obsoleta (en heap):** elemento extraído de una cola de prioridad cuya distancia ya no coincide con la mejor conocida; se descarta para mantener corrección.
- **Frontera (en Prim):** conjunto de aristas que conectan el conjunto de vértices ya incorporados al MST con los vértices aún no incorporados.
- **Flujo:** cantidad enviada a través de aristas en una red, respetando capacidades y conservación (excepto en fuente/sumidero).
- **Fuente (s):** vértice de origen en una red de flujo, desde donde se inyecta flujo.
- **Grafo:** estructura G=(V,E) formada por vértices y aristas (o arcos), que modela relaciones entre entidades.
- **Grafo dirigido (dígrafo):** grafo cuyas aristas tienen orientación; las rutas deben respetar la dirección.
- **Grafo no dirigido:** grafo cuyas aristas no tienen orientación; la relación es simétrica.
- **Grafo ponderado:** grafo donde cada arista tiene un peso/costo asociado (distancia, tiempo, etc.).
- **Grado:** en no dirigidos, número de aristas incidentes a un vértice.
- **Indegree (grado de entrada):** en dirigidos, número de arcos que llegan a un vértice; clave para Kahn.
- **Lista de adyacencia:** representación que, para cada vértice, almacena sus vecinos; eficiente en grafos dispersos.
- **Lista de aristas:** representación que almacena todas las aristas como tuplas (u,v,w); útil para Kruskal y recorridos por aristas.
- **Matriz de adyacencia:** matriz V×V que indica existencia (y/o peso) de aristas; eficiente para consultas rápidas, costosa en memoria.
- **Min-heap (heapq):** implementación en Python de cola de prioridad mínima; usada en Dijkstra/Prim para extraer el mínimo.
- **Orden topológico:** orden lineal de vértices en un DAG tal que cada arco u→v coloca u antes que v.

- **Peso:** costo asociado a una arista; se suma para evaluar costo de caminos o costo total de un MST.
- **Propiedad del corte (MST):** en cualquier corte, la arista de menor peso que cruza el corte es segura para algún MST.
- **Propiedad del ciclo (MST):** en cualquier ciclo, una arista de mayor peso del ciclo no pertenece a ningún MST (si es estrictamente mayor).
- **Red residual:** red derivada de una red de flujo que expresa capacidades remanentes y aristas inversas para permitir aumentaciones/reversiones.
- **Relajación:** operación en caminos mínimos que intenta mejorar dist[v] usando una arista (u,v): dist[v] = min(dist[v], dist[u]+w).
- **Ruta (path):** camino representado explícitamente como lista de nodos; suele evaluarse por su costo total.
- **Sumidero (t):** vértice destino en una red de flujo, donde se acumula el flujo enviado desde la fuente.
- **TSP (Problema del viajante):** busca el ciclo hamiltoniano de costo mínimo que visita todas las ciudades una vez y vuelve al inicio.
- **Union-Find (Conjuntos disjuntos):** estructura para mantener particiones dinámicas con operaciones find y union; clave en Kruskal.
- **Unión por tamaño/rango:** heurística de Union-Find que une el árbol más pequeño al más grande (o menor rango al mayor) para reducir altura.
- **Warshall:** algoritmo de programación dinámica para clausura transitiva (alcanzabilidad) en $O(V^3)$.
- **Vecino más cercano:** heurística greedy para TSP que elige iterativamente la ciudad no visitada más cercana; rápida pero no garantiza optimalidad.

Módulo 4 | Actividades

Actividades

Consigna

Las actividades de este módulo son interactivas y autoasistidas. Para realizarlas, es necesario acceder al enlace ***disponible en plataforma*** y seguir las instrucciones indicadas. Ante cualquier duda o inconveniente durante el desarrollo, se solicita comunicarse con el/la tutor/a para recibir acompañamiento y orientación.

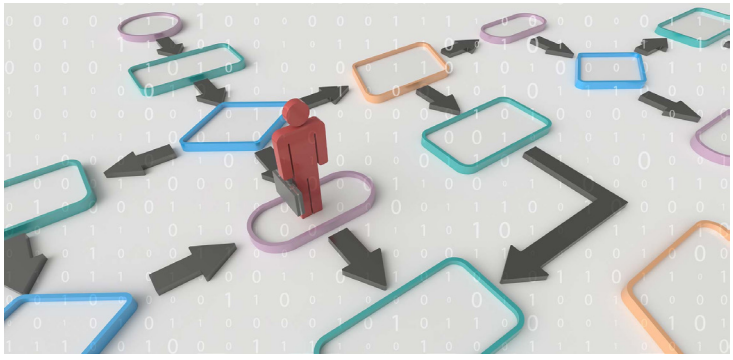
Módulo 5

Microobjetivos

- Identificar representaciones de grafos (listas de adyacencia, matrices y variantes ponderadas) para seleccionar la estructura de datos más adecuada en problemas de modelado de relaciones y conectividad en IA.
- Aplicar recorridos DFS, BFS y clausura transitiva (Warshall) para determinar alcanzabilidad, componentes y responder consultas entre pares en grafos dirigidos y no dirigidos según tamaño, densidad y cantidad de consultas.
- Implementar algoritmos de optimización sobre grafos (Dijkstra, Kruskal y Prim) para obtener caminos mínimos y árboles/bosques de expansión mínima en instancias ponderadas con validación mediante pruebas simples en Python.
- Analizar la estructura de dígrafos mediante orden topológico y detección de ciclos (DFS con estados y Kahn) para verificar consistencia de dependencias y precedencias en escenarios de planificación, pipelines y grafos DAG.
- Evaluar estrategias exactas y heurísticas (Edmonds–Karp, max-flow/min-cut, TSP por fuerza bruta, vecino más cercano y 2-opt) para justificar elecciones algorítmicas según complejidad y criterios de verificación en redes con capacidades y problemas combinatorios de ruteo

Contenidos

MÓDULO 5: GRAFOS Y ALGORITMOS DE GRAFOS



Fuente: Freepik

1. Introducción

¡Bienvenidos/as!

En este módulo se aborda el estudio de los grafos como modelo formal para representar relaciones entre entidades, dependencias y estructuras de conectividad presentes

en múltiples problemas de la Informática y de la IA. El recorrido se inicia con criterios de representación en memoria (listas de adyacencia, matrices y variantes para pesos), continúa con recorridos y alcanzabilidad (DFS, BFS y clausura transitiva con Warshall), y avanza hacia algoritmos centrales de optimización sobre grafos: caminos mínimos (Dijkstra), orden topológico y detección de ciclos en dígrafos, y árboles de expansión mínima (Kruskal y Prim). Finalmente, se integran problemas de mayor complejidad como flujo máximo (Edmonds–Karp), la interpretación del corte mínimo (max-flow/min-cut) y una introducción operativa al TSP mediante búsqueda exhaustiva y heurísticas (vecino más cercano y 2-opt), con implementaciones en Python y validaciones mediante pruebas simples

¡Comencemos!

2. Representación de grafos: listas de adyacencia con diccionarios de listas y matrices de adyacencia.

2.1 Concepto de grafo y parámetros relevantes para representación

Un grafo se formaliza como $G = (V, E)$, donde V es el conjunto de vértices y E el conjunto de aristas (no dirigidas) o arcos (dirigidos).

La elección de representación en memoria se apoya en:

- el tipo de grafo (dirigido/no dirigido),
- si posee pesos (función $w: E \to \mathbb{R}$), y
- la densidad $\rho \approx \frac{|E|}{|V|^2}$ (o $\frac{2|E|}{|V|(|V|-1)}$ en no dirigidos simples), porque condiciona los costos espaciales y temporales de operaciones como consulta de adyacencia y enumeración de vecinos (Diestel, 2017; Cormen et al., 2022).

Énfasis

Definiciones clave

- **Grafo:** Estructura matemática $G = (V, E)$ que modela relaciones entre elementos mediante vértices V y aristas/arcos E.
- **Grafo dirigido:** Grafo donde cada arco es un par ordenado (u, v), por lo que la adyacencia no es necesariamente simétrica.
- **Grafo no dirigido:** Grafo donde cada arista es un par no ordenado $\{u, v\}$, implicando adyacencia simétrica.
- **Grafo ponderado:** Grafo con una función de peso/costo asociada a cada arista/arco.
- **Densidad:** Medida de cuán cercano está el número de aristas al máximo posible para $|V|$, útil para decidir representación.

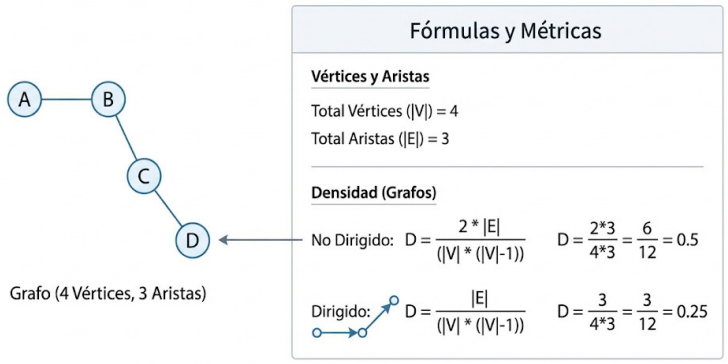


Ilustración 1: Fórmulas y métricas (|V|, |E| y densidad en grafos dirigidos/no dirigidos). Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

2.1.1 Ejemplo 1

Para justificar la noción de densidad sin depender de estructuras concretas, resulta útil computar métricas simples $|V|$, $|E|$ y una densidad aproximada. Estas magnitudes permiten anticipar si una matriz $n \times n$ será razonable o si conviene una lista de adyacencia para almacenar solo aristas existentes (Cormen et al., 2022; West, 2001).

```python
# Ejemplo 1 (parámetros del grafo): |V|, |E| y densidad aproximada

def graph_basic_stats(vertices, edges, directed=False):
    """
    vertices: iterable de vértices
    edges: iterable de pares (u, v)
    directed: True si el grafo es dirigido
    """
    V = set(vertices)
    E = list(edges)

    n = len(V)
    m = len(E)

    # Densidad aproximada:
    # - Dirigido: máximo n*(n-1) sin bucles
    # - No dirigido: máximo n*(n-1)/2 sin multiaristas ni bucles
    if n <= 1:
        density = 0.0
    else:
        if directed:
            max_edges = n * (n - 1)
        else:
            max_edges = n * (n - 1) / 2
        density = m / max_edges if max_edges else 0.0

    return n, m, density

# Pruebas simples
verts = ["A", "B", "C", "D"]
edges_undirected = [("A", "B"), ("A", "C"), ("C", "D")]
n, m, d = graph_basic_stats(verts, edges_undirected, directed=False)

print("No dirigido:", n, m, round(d, 3))
assert n == 4 and m == 3
assert 0.0 < d < 1.0

edges_directed = [("A", "B"), ("B", "A"), ("A", "C")]
n2, m2, d2 = graph_basic_stats(verts, edges_directed, directed=True)
print("Dirigido:", n2, m2, round(d2, 3))
assert n2 == 4 and m2 == 3
```

Captura 1: Ejemplo 1. Parámetros del grafo (|V|, |E| y densidad). Función graph_basic_stats que calcula cantidad de vértices, aristas y densidad aproximada para grafos dirigidos y no dirigidos, validada con prints y asserts. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`graph_basic_stats`	Función que calcula (
`V = set(vertices)`	Normaliza el conjunto de vértices para garantizar unicidad.
`E = list(edges)`	Materializa las aristas para contar (
Cálculo de `max_edges`	Usa el máximo teórico de aristas sin bucles para estimar densidad según dirigido/no dirigido.
`density = m / max_edges`	Obtiene una razón que aproxima cuán "denso" es el grafo respecto del máximo posible.
`assert` y `print`	Validan magnitudes esperadas y muestran resultados para inspección rápida.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

2.2 Lista de adyacencia con diccionario de listas

La lista de adyacencia representa el grafo asociando a cada vértice $u$una colección de vecinos $\text{Adj}[u]$. En términos de complejidad, enumerar vecinos de $u$cuesta $O(\deg(u))$, mientras que el almacenamiento total es $O(|V|+|E|)$(Cormen et al., 2022; Sedgewick & Wayne, 2011). En Python, una implementación usual es un diccionario dict que mapea vértices a listas de vecinos, con invariantes como "todo vértice presente tiene lista asociada" y, en no dirigidos, "si u es vecino de v, entonces $v$es vecino de u" (Goodrich et al., 2014).

Definiciones clave

- **Lista de adyacencia:** Representación donde cada vértice almacena explícitamente su colección de vecinos.
- **Grado** $\deg(u)$: Cantidad de aristas incidentes a u (en dirigidos: grado de salida/entrada).
- **Invariante de simetría (no dirigido):** Condición $(u \in \text{Adj}[v]) \Leftrightarrow (v \in \text{Adj}[u])$.

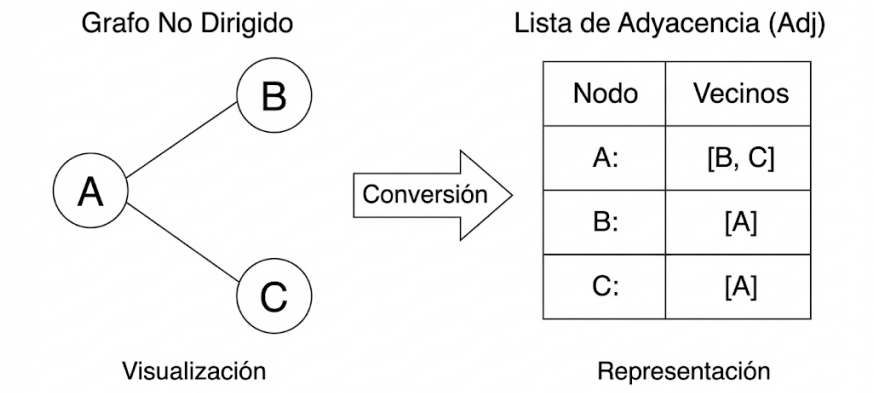


Ilustración 2: Conversión de un grafo no dirigido a lista de adyacencia (nodo → vecinos). Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

2.2.1 Ejemplo 2

El diccionario de listas permite implementar operaciones básicas sobre grafos dispersos manteniendo costos proporcionales al grado del vértice. Para un grafo no dirigido, se requiere asegurar la simetría al agregar o eliminar una arista, y evitar duplicados si se busca un modelo simple (Goodrich et al., 2014; Cormen et al., 2022).

```python
# Ejemplo 2: Grafo no dirigido con diccionario de listas
# Operaciones: agregar vértice, agregar arista, eliminar arista, obtener vecinos

class UndirectedGraphAdjList:
    def __init__(self):
        self.adj = {}  # u -> [v1, v2, ...]

    def add_vertex(self, u):
        if u not in self.adj:
            self.adj[u] = []

    def add_edge(self, u, v):
        # Asegura presencia de vértices
        self.add_vertex(u)
        self.add_vertex(v)

        # Evita duplicados en listas
        if v not in self.adj[u]:
            self.adj[u].append(v)
        if u not in self.adj[v]:
            self.adj[v].append(u)

    def remove_edge(self, u, v):
        # Elimina en ambas listas si existen
        if u in self.adj and v in self.adj[u]:
            self.adj[u].remove(v)
        if v in self.adj and u in self.adj[v]:
            self.adj[v].remove(u)

    def neighbors(self, u):
        return list(self.adj.get(u, []))
```

Captura 2: Ejemplo 2 (parte 1). Grafo no dirigido con diccionario de listas (ADT). Definición de la clase UndirectedGraphAdjList con estructura adj (u → [vecinos]) y métodos para agregar vértices y aristas manteniendo la simetría y evitando duplicados. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
33  # Pruebas simples
34  g = UndirectedGraphAdjList()
35  g.add_vertex("A")
36  g.add_edge("A", "B")
37  g.add_edge("A", "C")
38  g.add_edge("A", "B")  # duplicado intencional (no debe repetirse)
39
40  print("Vecinos de A:", g.neighbors("A"))
41  assert sorted(g.neighbors("A")) == ["B", "C"]
42  assert "A" in g.neighbors("B")  # simetría
43
44  g.remove_edge("A", "B")
45  print("Tras eliminar A-B, vecinos de A:", g.neighbors("A"))
46  assert "B" not in g.neighbors("A")
47  assert "A" not in g.neighbors("B")
```

Captura 3: Ejemplo 2 (parte 2). Pruebas del grafo no dirigido. Bloque de pruebas que construye un grafo, verifica vecinos, simetría y eliminación consistente de aristas mediante assert, evidenciando el comportamiento del ADT. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`self.adj = {}`	Modela la lista de adyacencia como mapeo vértice → lista de vecinos.
`add_vertex`	Mantiene el invariante "todo vértice del grafo tiene lista asociada".
`add_edge`	Asegura existencia de extremos y luego agrega en ambas direcciones (no dirigido).
Control de duplicados	Evita multiaristas no deseadas verificando pertenencia antes de `append`.
`remove_edge`	Elimina simétricamente para preservar consistencia en grafo no dirigido.
`neighbors`	Expone una copia de la lista para no filtrar referencias internas.
`assert` de simetría	Verifica el invariante $u \in Adj[v]$ y $v \in Adj[u]$ tras insertar.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

2.3 Variantes: diccionario de diccionarios (para pesos), y lista de conjuntos (para pertenencia rápida)

Para grafos ponderados, una variante usual es Adj[u] como diccionario $\{v: w(u, v)\}$, lo que permite consultar/actualizar el peso en tiempo esperado $O(1)$ por acceso a hash (Goodrich et al., 2014).

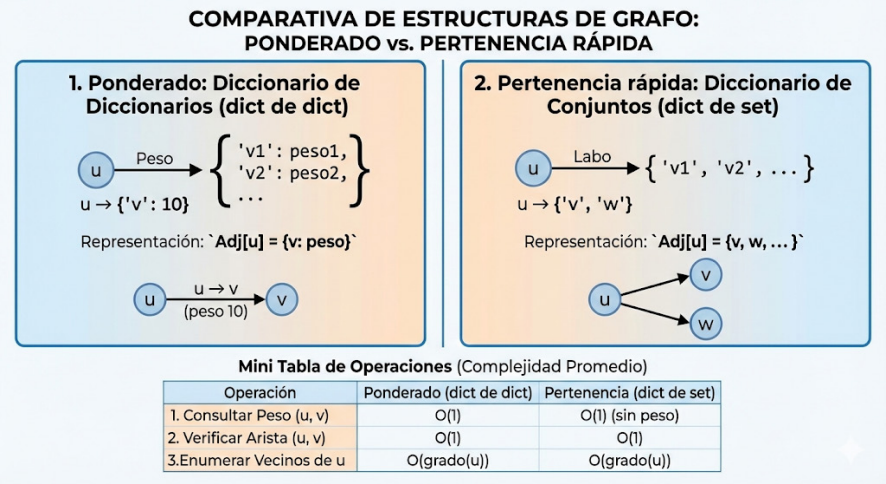
Para acelerar la prueba "¿v es vecino de u?", puede sustituirse la lista por un conjunto set, logrando pertenencia promedio $O(1)$ a cambio de perder orden y de un mayor overhead de memoria (Cormen et al., 2022; Sedgewick & Wayne, 2011).

Estas variantes se eligen según el patrón dominante de operaciones: consultas de peso, pertenencia, o enumeración secuencial.

Énfasis

Definiciones clave

- **Grafo ponderado (representación por mapas):** Representación donde cada vértice mapea a un diccionario vecino→peso.
- **Consulta de peso:** Operación que devuelve $w(u, v)$ si existe el arco/arista (u, v) y un valor nulo/sentinel en caso contrario.
- **Conjunto de vecinos:** Variante donde $\text{Adj}[u]$ se almacena como set para acelerar pertenencia.



Operación	Ponderado (dict de dict)	Pertenencia (dict de set)
1. Consultar Peso (u, v)	O(1)	O(1) (sin peso)
2. Verificar Arista (u, v)	O(1)	O(1)
3.Enumerar Vecinos de u	O(grado(u))	O(grado(u))

Ilustración 3: Comparativa de representaciones: ponderado (dict de dict) vs pertenencia rápida (dict de set) y complejidades promedio. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

2.3.1 Ejemplo 3

La representación u à {v: peso} separa explícitamente conectividad y costo, necesaria para algoritmos de caminos mínimos y optimización. En este modelo, consultar el peso se reduce a una búsqueda en diccionario, adecuada cuando las consultas (u, v) son frecuentes (Cormen et al., 2022; Goodrich et al., 2014).

```python
# Ejemplo 3: Grafo ponderado con diccionario de diccionarios (dirigido)
# Estructura: u -> {v: peso}

class WeightedDigraphAdjMap:
    def __init__(self):
        self.adj = {}  # u -> {v: w}

    def add_vertex(self, u):
        if u not in self.adj:
            self.adj[u] = {}

    def add_edge(self, u, v, w):
        self.add_vertex(u)
        self.add_vertex(v)
        self.adj[u][v] = w  # sobrescribe si ya existía

    def weight(self, u, v):
        return self.adj.get(u, {}).get(v, None)

    def neighbors(self, u):
        return list(self.adj.get(u, {}).keys())

# Pruebas simples
wg = WeightedDigraphAdjMap()
wg.add_edge("A", "B", 5)
wg.add_edge("A", "C", 2)
wg.add_edge("C", "B", 1)

print("Vecinos de A:", wg.neighbors("A"))
print("Peso A->B:", wg.weight("A", "B"))
print("Peso B->A (no existe):", wg.weight("B", "A"))

assert set(wg.neighbors("A")) == {"B", "C"}
assert wg.weight("A", "B") == 5
assert wg.weight("B", "A") is None
```

Captura 4: Ejemplo 3. Grafo ponderado dirigido (dict de dict). Implementación de WeightedDigraphAdjMap con representación u → {v: peso} para insertar arcos con pesos y consultar weight(u,v), incluyendo pruebas de asimetría (arco no implica retorno). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`self.adj[u] = {}`	Asigna a cada vértice un mapa de vecinos hacia pesos, habilitando consulta directa.
`add_edge(u,v,w)`	Inserta un arco dirigido $u \to v$ con peso w; si existe, actualiza el peso.
`weight(u,v)`	Consulta segura usando `get` encadenado; retorna `None` si no hay arco.
`neighbors(u)`	Devuelve vecinos salientes de u, alineado con "grado de salida".
`assert` de asimetría	Verifica que $w(B, A)$ sea inexistente si no se agregó explícitamente.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

2.3.2 Ejemplo 4

Cuando el patrón dominante es verificar existencia de arista (pertenencia), almacenar vecinos como conjunto evita recorridos lineales sobre listas. Esta decisión cambia el costo de has_edge a $O(1)$ promedio, manteniendo enumeración en $O(\deg(u))$ (Sedgewick & Wayne, 2011; Goodrich et al., 2014).

```python
# Ejemplo 4: Lista de adyacencia como diccionario de conjuntos (no dirigido)
# Estructura: u -> set(vecinos)

class UndirectedGraphAdjSet:
    def __init__(self):
        self.adj = {}  # u -> set(v)

    def add_vertex(self, u):
        if u not in self.adj:
            self.adj[u] = set()

    def add_edge(self, u, v):
        self.add_vertex(u)
        self.add_vertex(v)
        self.adj[u].add(v)
        self.adj[v].add(u)

    def has_edge(self, u, v):
        return (u in self.adj) and (v in self.adj[u])

# Pruebas simples
gs = UndirectedGraphAdjSet()
gs.add_edge(1, 2)
gs.add_edge(1, 3)

print("Existe 1-2:", gs.has_edge(1, 2))
print("Existe 2-3:", gs.has_edge(2, 3))

assert gs.has_edge(1, 2) is True
assert gs.has_edge(2, 1) is True  # simetría
assert gs.has_edge(2, 3) is False
```

Captura 5: Ejemplo 4. Grafo no dirigido con diccionario de conjuntos (dict de set). Implementación de UndirectedGraphAdjSet para acelerar has_edge(u,v) mediante pertenencia en set, con pruebas de simetría y casos inexistentes. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`set()` como vecinos	Sustituye lista por conjunto para optimizar pertenencia de vecinos.
`add_edge`	Inserta simétricamente en ambos conjuntos (no dirigido).
`has_edge`	Resuelve existencia consultando pertenencia en conjunto, evitando recorridos lineales.
`assert` de simetría	Comprueba que al ser no dirigido, `has_edge(u,v)` y `has_edge(v,u)` coinciden.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

2.4 Matriz de adyacencia (estructura, costes, ventajas en grafos densos)

La matriz de adyacencia es una estructura A de tamaño $|V| \times |V|$ donde $A[i][j]$ indica existencia (o peso) de la arista/arco $i \to j$. Su ventaja central es la consulta de adyacencia en $O(1)$, mientras que el costo espacial es $O(|V|^2)$, lo que la vuelve adecuada en grafos densos o cuando se requiere consultar repetidamente si existe una conexión entre dos vértices (Cormen et al., 2022; West, 2001). En ponderados, la matriz puede almacenar un sentinel (por ejemplo, None) para "sin arista" y números para pesos.

Énfasis

Definiciones clave

- **Matriz de adyacencia:** Matriz cuadrada donde cada celda codifica la existencia o el peso de una arista/arco entre dos vértices.
- **Consulta de adyacencia:** Operación que determina si $(u, v) \in E$ inspeccionando una única celda en la matriz.
- **Codificación por índices:** Mapeo entre etiquetas de vértices y posiciones enteras $[0..|V|-1]$ para direccionar la matriz.

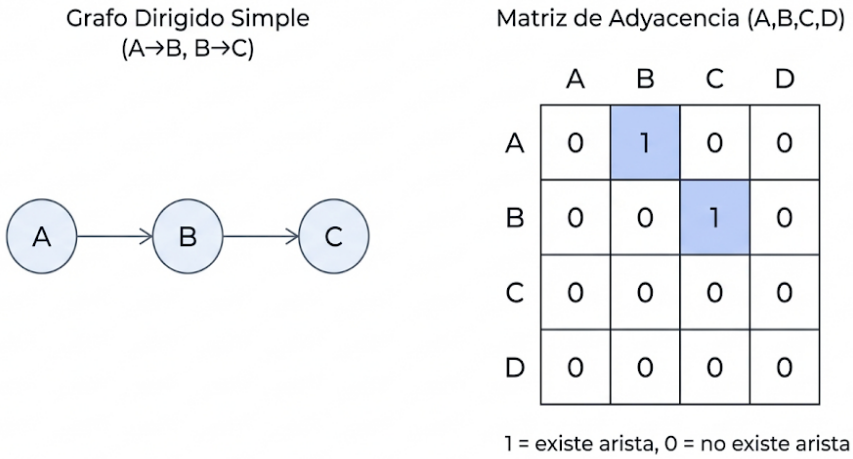


Ilustración 4: Grafo dirigido simple y su matriz de adyacencia (codificación 1/0 de existencia de arco). Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

2.4.1 Ejemplo 5

Para implementar una matriz de adyacencia en Python estándar, se requiere un mapeo estable vértice → índice. Una vez construido, has_edge(u,v) se resuelve en tiempo constante accediendo a una celda, lo cual es el beneficio operativo típico de esta representación (Cormen et al., 2022; West, 2001).

```python
# Ejemplo 5: Matriz de adyacencia con lista de listas
# Operación: existencia de arista en O(1)

class AdjacencyMatrixGraph:
    def __init__(self, vertices, directed=False):
        self.directed = directed
        self.vertices = list(vertices)
        self.index = {v: i for i, v in enumerate(self.vertices)}
        n = len(self.vertices)
        self.mat = [[0] * n for _ in range(n)]

    def add_edge(self, u, v):
        i, j = self.index[u], self.index[v]
        self.mat[i][j] = 1
        if not self.directed:
            self.mat[j][i] = 1

    def has_edge(self, u, v):
        i, j = self.index[u], self.index[v]
        return self.mat[i][j] == 1

# Pruebas simples
verts = ["A", "B", "C", "D"]
mg = AdjacencyMatrixGraph(verts, directed=True)
mg.add_edge("A", "B")
mg.add_edge("B", "C")

print("A->B:", mg.has_edge("A", "B"))
print("B->A:", mg.has_edge("B", "A"))
assert mg.has_edge("A", "B") is True
assert mg.has_edge("B", "A") is False  # dirigido, no simétrico
```

Captura 6: Ejemplo 5. Matriz de adyacencia (lista de listas) y consulta O(1). Clase AdjacencyMatrixGraph que mapea vértices a índices y permite verificar existencia de arista con acceso directo a mat[i][j], probando comportamiento en grafo dirigido. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`self.index`	Construye el mapeo vértice→índice para direccionar filas/columnas.
`self.mat`	Inicializa una matriz $n \times n$ con ceros (sin aristas).
`add_edge`	Marca `1` en la celda correspondiente; si no dirigido, marca también el espejo.
`has_edge`	Accede a una única celda para verificar existencia, operacionalmente $O(1)$.
`assert` dirigido	Comprueba asimetría: que `A->B` no implique `B->A`.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

2.5 Conversión entre representaciones y validación de consistencia

Convertir entre lista y matriz permite adaptar la estructura a un algoritmo o patrón de operaciones dominante. Conceptualmente, la lista de adyacencia codifica E de forma explícita por vecinos, mientras que la matriz codifica una función indicadora $A: V \times V \to \{0,1\}$ (o pesos). Una conversión correcta debe preservar el conjunto de aristas/arcos, lo cual

puede verificarse por equivalencia estructural (por ejemplo, igualdad del conjunto de pares) (Cormen et al., 2022; Diestel, 2017).

Énfasis

Definiciones clave

- **Conversión de representación:** Transformación de una estructura (lista/matriz) a otra preservando vértices y aristas/arcos.
- **Equivalencia de grafos (a nivel de datos):** Igualdad de V y de E (como conjunto de pares) independientemente de la estructura de almacenamiento.
- **Validación por aserciones:** Uso de assert para comprobar invariantes y equivalencias tras una transformación.

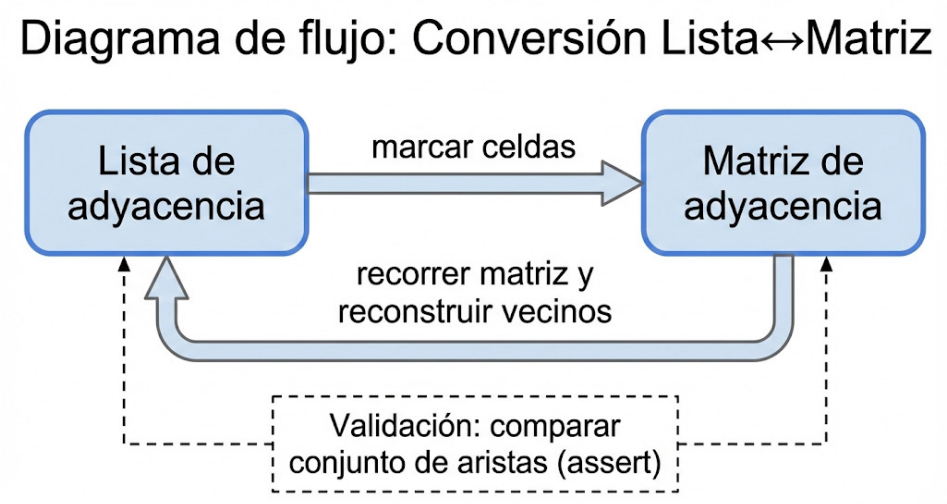


Ilustración 5: Diagrama de flujo de conversión Lista↔Matriz con validación por conjunto de aristas (assert). Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

2.5.1 Ejemplo 6

La conversión lista → matriz se implementa marcando celdas para cada vecino; la conversión inversa recorre la matriz y reconstruye vecinos para entradas no nulas. La verificación por assert con el conjunto de arcos garantiza que la transformación no omitió ni inventó conexiones (Cormen et al., 2022).

```python
# Ejemplo 6: Conversión lista de adyacencia <-> matriz con verificación (assert)

def adjlist_to_matrix(adj, vertices, directed=True):
    idx = {v: i for i, v in enumerate(vertices)}
    n = len(vertices)
    mat = [[0] * n for _ in range(n)]
    for u, neighbors in adj.items():
        for v in neighbors:
            mat[idx[u]][idx[v]] = 1
            if not directed:
                mat[idx[v]][idx[u]] = 1
    return mat

def matrix_to_adjlist(mat, vertices, directed=True):
    n = len(vertices)
    adj = {v: [] for v in vertices}
    for i in range(n):
        for j in range(n):
            if mat[i][j] == 1:
                u, v = vertices[i], vertices[j]
                adj[u].append(v)
    if not directed:
        # opcional: normalización para evitar duplicados si la matriz es simétrica
        for u in vertices:
            adj[u] = sorted(set(adj[u]))
    return adj

def edge_set_from_adjlist(adj, directed=True):
    es = set()
    for u, neigh in adj.items():
        for v in neigh:
            if directed:
                es.add((u, v))
            else:
                es.add(tuple(sorted((u, v))))
    return es
```

Captura 7: Ejemplo 6 (parte 1). Conversión lista↔matriz (funciones). Funciones adjlist_to_matrix, matrix_to_adjlist y edge_set_from_adjlist para transformar representaciones y normalizar arcos/aristas para comparación. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
vertices = ["A", "B", "C", "D"]
adj_directed = {
    "A": ["B", "C"],
    "B": ["C"],
    "C": [],
    "D": ["A"]
}

mat = adjlist_to_matrix(adj_directed, vertices, directed=True)
adj_back = matrix_to_adjlist(mat, vertices, directed=True)

es1 = edge_set_from_adjlist(adj_directed, directed=True)
es2 = edge_set_from_adjlist(adj_back, directed=True)

print("Arcos originales:", sorted(es1))
print("Arcos reconstruidos:", sorted(es2))

assert es1 == es2  # equivalencia de arcos tras ida y vuelta
```

Captura 8: Ejemplo 6 (parte 2). Verificación de equivalencia tras ida y vuelta. Pruebas que convierten una lista de adyacencia a matriz y reconstruyen la lista, comparando conjuntos de arcos con assert para validar consistencia. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`adjlist_to_matrix`	Crea matriz y marca `1` por cada relación de vecindad encontrada en la lista.
`idx`	Mapeo vértice→índice para ubicar celdas de la matriz.
Doble bucle en `matrix_to_adjlist`	Recorre todas las celdas y reconstruye la lista cuando hay `1`.
`edge_set_from_adjlist`	Normaliza la comparación: transforma la estructura en un conjunto de pares.
`assert es1 == es2`	Valida que la conversión preserva exactamente el conjunto de arcos.
`print` de arcos	Evidencia el resultado para inspección manual adicional.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

2.5.2 Ejemplo 7

En grafos, el costo de consultar adyacencia difiere: matriz $O(1)$ vs lista $O(\deg(u))$ si se busca dentro de una lista, o $O(1)$ promedio si se usa conjunto. Un contraste simple puede ilustrarse sin bibliotecas externas, usando conteos y, opcionalmente, time.perf_counter del estándar (Cormen et al., 2022).

```python
# Ejemplo 7: Comparación simple tiempo/operaciones (estándar) y análisis por conteo
# Nota: es una demostración empírica mínima; la conclusión principal es teórica.

from time import perf_counter

def build_dense_matrix(n):
    mat = [[1 if i != j else 0 for j in range(n)] for i in range(n)]
    return mat

def build_sparse_adjlist(n):
    # cadena: 0-1-2-...-(n-1) dirigida
    adj = {i: [] for i in range(n)}
    for i in range(n - 1):
        adj[i].append(i + 1)
    return adj

def matrix_has_edge(mat, u, v):
    return mat[u][v] == 1

def adjlist_has_edge(adj, u, v):
    # búsqueda lineal en lista (peor para grados altos)
    return v in adj.get(u, [])
```

Captura 9: Ejemplo 7. Comparación simple tiempo/operaciones y conteos. Construcción de un caso denso (matriz) y uno disperso (lista), con conteos de almacenamiento y una medición mínima usando perf_counter para ilustrar diferencias operativas. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
n = 200
mat = build_dense_matrix(n)
adj = build_sparse_adjlist(n)

# Conteos teóricos básicos
print("Matriz guarda n^2 celdas:", n * n)
print("Lista dispersa guarda ~|V|+|E|:", n, "+", (n - 1))

# Micro-benchmark (opcional)
pairs = [(i, (i + 1) % n) for i in range(n)] * 200  # consultas repetidas

t0 = perf_counter()
count_m = 0
for u, v in pairs:
    if matrix_has_edge(mat, u, v):
        count_m += 1
t1 = perf_counter()

t2 = perf_counter()
count_a = 0
for u, v in pairs:
    if adjlist_has_edge(adj, u, v):
        count_a += 1
t3 = perf_counter()

print("Verdaderos matriz:", count_m, "tiempo:", round(t1 - t0, 6))
print("Verdaderos lista :", count_a, "tiempo:", round(t3 - t2, 6))

assert count_a == len(pairs) - n  # Adjust for the cycle created by the modulo operation
# Ajuste: la aserción anterior no corresponde por el módulo; se valida parcialmente:
assert count_a > 0 and count_m > 0
```

Captura 10: Ejemplo 7 (detalle). Micro-benchmark y conteos de almacenamiento. Fragmento de pruebas que compara, para $n = 200$, el costo espacial teórico (matriz n^2 vs lista $\approx |V| + |E|$) y un tiempo empírico mínimo de consultas repetidas de existencia de arista usando perf_counter, contabilizando aciertos (count_m, count_a) y validando resultados con assert. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`build_dense_matrix`	Construye una matriz casi completa para simular un grafo denso y favorecer consulta O(1).
`build_sparse_adjlist`	Construye una lista dispersa tipo cadena, representativa de (
Conteos impresos	Ilustran el costo espacial: matriz ~n^2 vs lista ~(
`pairs`	Genera consultas repetidas para comparar costos de consulta de adyacencia.
`perf_counter`	Mide tiempo con herramienta estándar, sin dependencias externas.
`v in adj[u]`	Modela el costo de consulta en lista como búsqueda lineal dependiente de $\deg(u)$.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 1: Estructuras de datos de grafos.**

3. Concepto de recorrido en grafos y criterios operativos

3.1 Recorridos

Un recorrido sobre un grafo es un procedimiento sistemático para visitar vértices siguiendo aristas, evitando repeticiones mediante un criterio de *marcado* (visitados). El resultado operativo suele ser un árbol (o bosque) de exploración, que registra las aristas usadas para descubrir nuevos vértices; en grafos no conexos el resultado es un bosque.

La representación condiciona costos: con lista de adyacencia la exploración revisa solo vecinos existentes (útil en grafos dispersos), mientras que con matriz de adyacencia la consulta de vecindad es O(1) pero se inspecciona un renglón completo para enumerar vecinos (O(n)). Esta distinción guía las complejidades de DFS/BFS (Cormen et al., 2009). Aquí se contempla el uso de listas y matrices como representaciones estándar para grafos y para recorridos (De la Cueva Hernández et al., 2020; Frittelli, 2021).

Énfasis

Definiciones clave

- **Recorrido (traversal):** Procedimiento que visita vértices de un grafo siguiendo aristas según una estrategia y un criterio de marcado.
- **Conjunto de visitados:** Estructura (p. ej., set) que registra vértices ya descubiertos para evitar ciclos y repeticiones.
- **Árbol/bosque de exploración:** Subgrafo inducido por las aristas de descubrimiento que describe cómo se alcanzan vértices durante un recorrido.
- **Lista de adyacencia:** Representación que asocia a cada vértice la lista de sus vecinos salientes (o incidentes, si no dirigido).
- **Matriz de adyacencia:** Matriz booleana/numérica A[i][j] que indica existencia de arista de i a j.

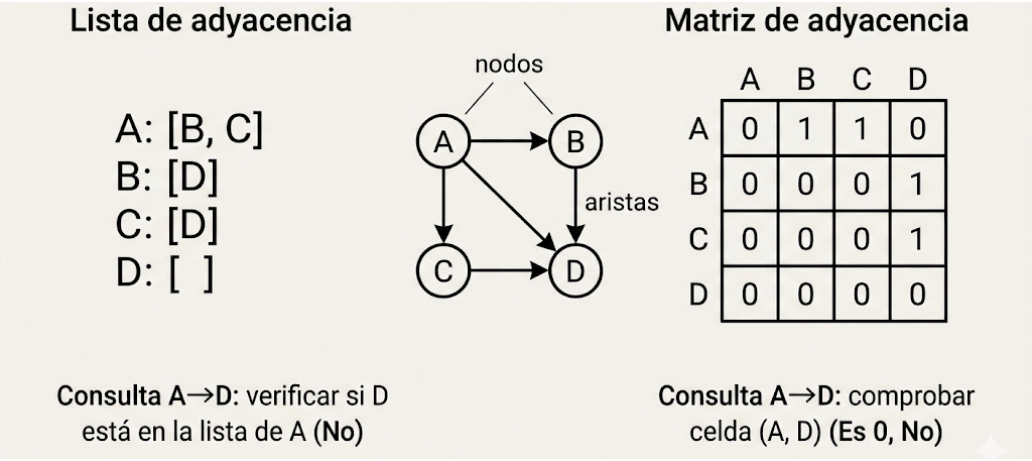


Ilustración 6: Lista vs matriz de adyacencia (consulta A→D). Comparación visual de cómo verificar la existencia de un arco consultando vecinos en lista de adyacencia y una celda en matriz de adyacencia. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.1.1 Ejemplo 1

Desde la teoría, conviene dominar la conversión entre representaciones porque algunos algoritmos (p. ej., Warshall) operan naturalmente sobre matrices, mientras que DFS/BFS se implementan de forma directa con listas de adyacencia. El marcado de visitados es el mecanismo que asegura terminación en presencia de ciclos (Cormen et al., 2009).

```python
# Ejemplo 1: Representación de grafos (lista y matriz) + marcado de visitados

def to_adjacency_matrix(adj, nodes=None):
    if nodes is None:
        nodes = sorted(adj.keys())
    idx = {v: i for i, v in enumerate(nodes)}
    n = len(nodes)
    mat = [[False] * n for _ in range(n)]
    for u in nodes:
        for v in adj.get(u, []):
            mat[idx[u]][idx[v]] = True
    return nodes, mat

def neighbors_from_matrix(nodes, mat, u):
    i = nodes.index(u)
    return [nodes[j] for j, connected in enumerate(mat[i]) if connected]

def simple_reachable_once(adj, start):
    # "alcanzables" por un recorrido genérico basado en pila (sin imponer DFS/BFS estricto)
    visited = set()
    stack = [start]
    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        for v in adj.get(u, []):
            if v not in visited:
                stack.append(v)
    return visited
```

Captura 11: Ejemplo 1 (parte 1). Conversión a matriz y alcanzabilidad con marcación de visitados. Código que transforma una lista de adyacencia en una matriz booleana (to_adjacency_matrix), recupera vecinos desde la matriz (neighbors_from_matrix) y calcula el conjunto de nodos alcanzables desde un origen mediante recorrido con pila y conjunto visited (simple_reachable_once). Desarrollado en Online IDE Pro. Código fuente de la autora.

```
31
32  if __name__ == "__main__":
33      # Grafo dirigido pequeño
34      adj = {
35          "A": ["B", "C"],
36          "B": ["D"],
37          "C": ["D"],
38          "D": []
39      }
40
41      nodes, mat = to_adjacency_matrix(adj)
42      print("Nodos:", nodes)
43      print("Matriz de adyacencia (True=arista):")
44      for row in mat:
45          print(row)
46
47      # Vecinos de C según matriz
48      print("Vecinos de C (matriz):", neighbors_from_matrix(nodes, mat, "C"))
49
50      # Marcado de visitados para alcanzabilidad
51      reach = simple_reachable_once(adj, "A")
52      print("Alcanzables desde A:", sorted(reach))
53
54      # Pruebas simples
55      assert reach == {"A", "B", "C", "D"}
56      assert neighbors_from_matrix(nodes, mat, "C") == ["D"]
```

Captura 12: Ejemplo 1 (parte 2). Ejecución y validación con pruebas. Bloque principal que define un grafo dirigido pequeño, imprime nodos y matriz de adyacencia, consulta vecinos desde la matriz y verifica alcanzabilidad desde "A" con assert, asegurando consistencia entre representaciones y resultados. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`to_adjacency_matrix`	Construye el conjunto de nodos, asigna índices y completa una matriz booleana `n×n` marcando aristas existentes.
`neighbors_from_matrix`	Recupera vecinos de un vértice inspeccionando la fila correspondiente; esta operación es O(n) en número de nodos.
`simple_reachable_once`	Mantiene `visited` para evitar revisitas; usa una pila para explorar hasta agotar candidatos alcanzables desde `start`.
`assert reach == {...}`	Verifica alcanzabilidad esperada desde `A`, evidenciando el rol del marcado de visitados.
`assert neighbors...`	Confirma consistencia entre lista de adyacencia y matriz al recuperar vecinos.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

3.2 DFS: definición, variantes (recursiva e iterativa) y complejidad

La búsqueda en profundidad (DFS) explora un vértice y continúa por un vecino no visitado, repitiendo el proceso hasta que no haya vecinos nuevos; luego retrocede. Puede implementarse con recursión (pila implícita) o con pila explícita. Con lista de adyacencia, DFS opera en O(V + E) y requiere O(V) memoria para visited y (en peor caso) para la pila (Cormen et al., 2009). En contextos de análisis estructural, DFS es base para detectar ciclos, construir bosques de exploración y apoyar análisis de conectividad (Frittelli, 2021).

Énfasis

Definiciones clave

- **DFS (Depth-First Search)**: Recorrido que expande un camino hasta agotar vecinos no visitados antes de explorar alternativas.
- **Pila de recursión:** Memoria implícita usada por llamadas recursivas para registrar el camino actual.
- **Pila explícita:** Estructura LIFO utilizada para simular recursión y controlar el orden de exploración.
- **Complejidad O(V+E):** Cota que indica que cada vértice y cada arista se procesa un número constante de veces (lista de adyacencia).

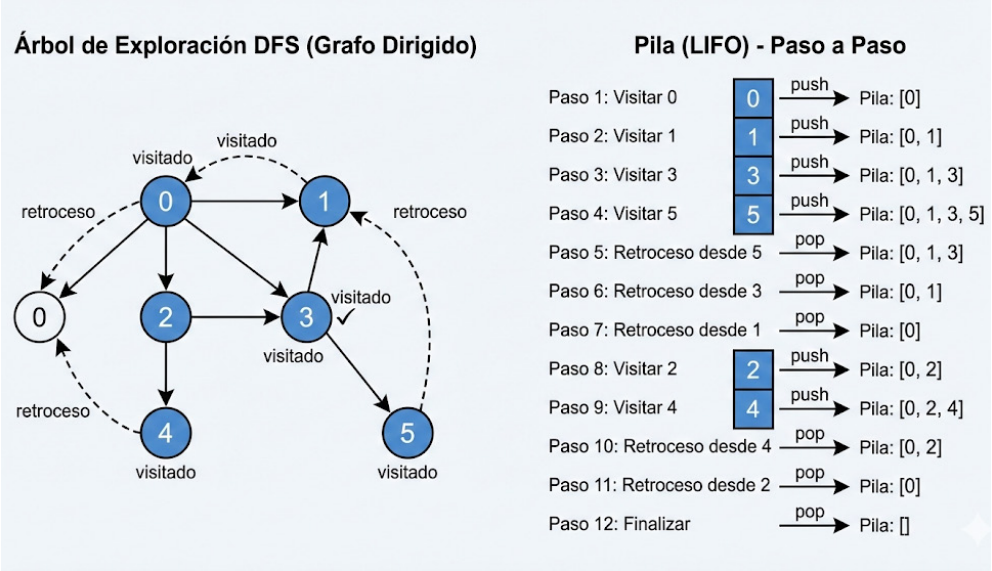


Ilustración 7: DFS en grafo dirigido y pila (LIFO) paso a paso. Representación del árbol de exploración en profundidad y la evolución de la pila durante el recorrido, incluyendo retrocesos. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.2.1 Ejemplo 2

Desde la teoría, la versión recursiva expresa directamente la definición inductiva de DFS: visitar y descender por vecinos no visitados. El orden depende del orden de vecinos; por ello, se fija el orden usando vecinos ordenados para que la prueba sea determinista (Cormen et al., 2009).

```python
# Ejemplo 2: DFS recursiva (lista de adyacencia) con conjunto de visitados y orden de visita

def dfs_recursive(adj, start):
    visited = set()
    order = []

    def visit(u):
        visited.add(u)
        order.append(u)
        for v in adj.get(u, []):
            if v not in visited:
                visit(v)

    visit(start)
    return order, visited

if __name__ == "__main__":
    # Grafo dirigido con orden de vecinos controlado
    adj = {
        0: [1, 2],
        1: [3],
        2: [3, 4],
        3: [5],
        4: [],
        5: []
    }

    order, reached = dfs_recursive(adj, 0)
    print("Orden DFS recursiva:", order)
    print("Alcanzables:", reached)

    # Pruebas simples (deterministas por el orden de listas)
    assert reached == {0, 1, 2, 3, 4, 5}
    assert order == [0, 1, 3, 5, 2, 4]
```

Captura 13: Ejemplo 2. DFS recursiva (lista de adyacencia) con orden de visita. Implementación de dfs_recursive que recorre un grafo dirigido desde un nodo inicial, registra el orden de descubrimiento en order y controla repetición con visited, validando un orden determinista según el orden de vecinos mediante assert. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`visited`	Registra vértices ya explorados para evitar ciclos y repetición.
`order`	Acumula el orden de descubrimiento; es el "trazo" del recorrido.
`visit(u)`	Marca `u` y recorre sus vecinos; si un vecino no fue visitado, desciende recursivamente.
Bucle `for v in adj[u]`	Implementa la expansión en profundidad según el orden de la lista de adyacencia.
`assert reached == ...`	Verifica que DFS alcanza todos los nodos alcanzables desde la fuente.
`assert order == ...`	Comprueba un orden concreto, posible por fijar el orden de vecinos.

3.2.2 Ejemplo 3: DFS iterativa con pila, comparación de alcanzabilidad

Desde la teoría, la versión iterativa simula la pila de recursión. El orden puede diferir, pero la **alcanzabilidad** desde la misma fuente debe coincidir si se usa el mismo criterio de marcado (Cormen et al., 2009).

```python
# Ejemplo 3: DFS iterativa con pila (comparación con DFS recursiva)

def dfs_iterative(adj, start):
    visited = set()
    order = []
    stack = [start]

    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)

        # Para aproximar el mismo comportamiento que la recursión,
        # se apilan vecinos en orden inverso (LIFO).
        for v in reversed(adj.get(u, [])):
            if v not in visited:
                stack.append(v)

    return order, visited
```

Captura 14: Ejemplo 3 (parte 1). DFS iterativa con pila (LIFO). Implementación de dfs_iterative que simula DFS sin recursión usando una pila stack, y apila vecinos en orden inverso para aproximar el comportamiento de la versión recursiva. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python

def dfs_recursive_reach(adj, start):
    visited = set()
    def visit(u):
        visited.add(u)
        for v in adj.get(u, []):
            if v not in visited:
                visit(v)
    visit(start)
    return visited

if __name__ == "__main__":
    adj = {
        0: [1, 2],
        1: [3],
        2: [3, 4],
        3: [5],
        4: [],
        5: []
    }

    order_it, reached_it = dfs_iterative(adj, 0)
    reached_rec = dfs_recursive_reach(adj, 0)

    print("Orden DFS iterativa:", order_it)
    print("Alcanzables (iterativa):", reached_it)
    print("Alcanzables (recursiva):", reached_rec)

    # Pruebas simples: misma alcanzabilidad, orden no necesariamente idéntico
    assert reached_it == reached_rec
    assert reached_it == {0, 1, 2, 3, 4, 5}
```

Captura 15: Ejemplo 3 (parte 2). Comparación de alcanzabilidad recursiva vs iterativa. Bloque que define dfs_recursive_reach (solo conjunto de alcanzables) y compara resultados con la versión iterativa, imprimiendo órdenes y verificando con assert que la alcanzabilidad sea la misma aunque el orden de visita pueda diferir. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`stack`	Estructura LIFO que reemplaza a la pila implícita de recursión.
`pop()`	Selecciona el último vértice agregado; esto define el sesgo "en profundidad".
`if u in visited: continue`	Evita procesar vértices repetidos que pueden entrar por múltiples aristas.
`reversed(adj[u])`	Ajusta el orden de apilado para controlar el orden de exploración en LIFO.
`dfs_recursive_reach`	Función auxiliar para comparar el conjunto de alcanzables con el de la versión iterativa.
`assert reached_it == reached_rec`	Verifica equivalencia de alcanzabilidad, propiedad esperable del algoritmo.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

3.3 BFS: definición, cola (deque), distancias mínimas en no ponderados y complejidad

La búsqueda en amplitud (BFS) explora el grafo por "capas": primero visita todos los vecinos a distancia 1 del origen, luego los de distancia 2, y así sucesivamente. Se implementa con una cola FIFO. En grafos no ponderados, BFS garantiza que la primera vez que se descubre un vértice se ha encontrado un camino con mínima cantidad de aristas desde la fuente, lo que permite computar distancias mínimas en número de pasos (Cormen et al., 2009). Con lista de adyacencia, su costo es O(V + E) y requiere O(V) para visited y la cola.

Definiciones clave

- **BFS (Breadth-First Search):** Recorrido que visita vértices en orden creciente de distancia (en aristas) desde una fuente.
- **Cola FIFO:** Estructura donde el primer elemento en entrar es el primero en salir; sostiene la exploración por capas.
- **Distancia en grafo no ponderado:** Mínimo número de aristas de un camino entre dos vértices.

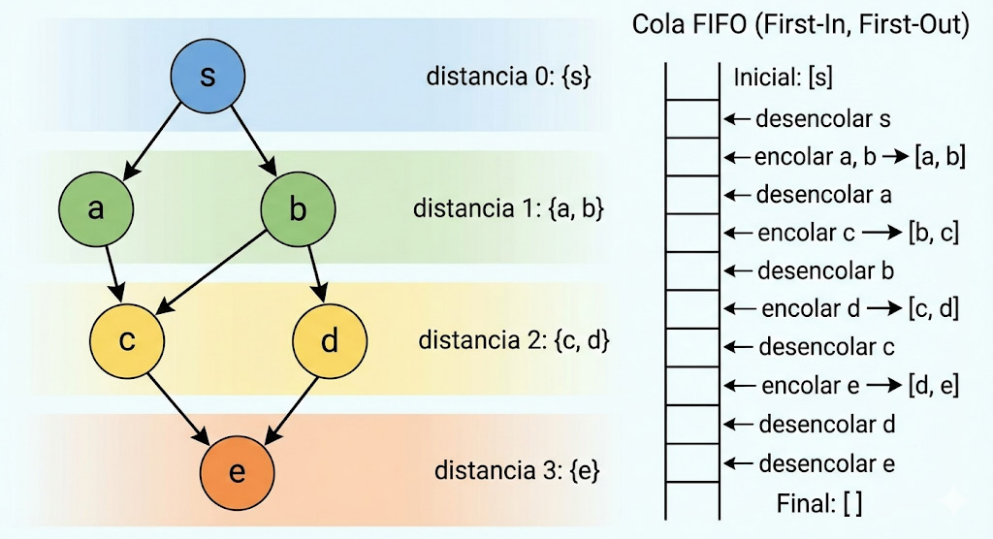


Ilustración 9: BFS por niveles y cola (FIFO). Ilustración de la exploración en anchura con distancias por capas desde el nodo origen y el proceso de encolar/desencolar. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.3.1 Ejemplo 4 (BFS con deque y cálculo de distancias)

Desde la teoría, BFS mantiene una cola de frontera: al desencolar un vértice, se encolan sus vecinos no visitados con distancia dist[u] + 1. Esto implementa formalmente la exploración por capas (Cormen et al., 2009).

```python
# Ejemplo 4: BFS con deque y distancias mínimas (grafo no ponderado)

from collections import deque

def bfs_distances(adj, start):
    dist = {start: 0}
    q = deque([start])

    while q:
        u = q.popleft()
        for v in adj.get(u, []):
            if v not in dist:  # equivale a "no visitado"
                dist[v] = dist[u] + 1
                q.append(v)

    return dist

if __name__ == "__main__":
    # Grafo no ponderado (dirigido para ejemplificar; la propiedad de capas se mantiene)
    adj = {
        "s": ["a", "b"],
        "a": ["c"],
        "b": ["c", "d"],
        "c": ["e"],
        "d": ["e"],
        "e": []
    }

    dist = bfs_distances(adj, "s")
    print("Distancias desde s:", dist)

    # Pruebas simples: distancias mínimas en cantidad de aristas
    assert dist["s"] == 0
    assert dist["a"] == 1 and dist["b"] == 1
    assert dist["c"] == 2
    assert dist["d"] == 2
    assert dist["e"] == 3
```

Captura 16: Ejemplo de Implementación de BFS con cola y distancias mínimas. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dist`	Mapa de distancias; su presencia indica "descubierto/visitado".
`deque([start])`	Inicializa la cola con la fuente, inicio de la capa 0.
`popleft()`	Extrae el vértice más antiguo, garantizando orden FIFO (capas).
`if v not in dist`	Descubre un vecino por primera vez; BFS fija su distancia mínima.
`dist[v] = dist[u] + 1`	Asigna distancia por capas: cada arista incrementa en 1.
`assert ...`	Verifica distancias esperadas para demostrar la propiedad de mínimos pasos.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

3.4 Componentes conexas (no dirigido) y alcance en dirigido: estrategia basada en DFS/BFS

En grafos no dirigidos, una componente conexa es un subconconjunto máximo de vértices donde todo par es alcanzable por algún camino. Se obtiene repitiendo DFS/BFS desde un vértice no visitado, marcando todos los alcanzables como parte de la misma compo-

nente, y continuando hasta cubrir el grafo completo. En grafos dirigidos, el concepto análogo de "alcance" desde un vértice se define por alcanzabilidad siguiendo direcciones; si se requieren componentes fuertemente conexas se necesitan algoritmos específicos (p. ej., Kosaraju/Tarjan), pero para "componentes alcanzables desde semillas" basta DFS/BFS (Cormen et al., 2009). La asignatura incluye explícitamente el análisis de componentes como aplicación directa de recorridos (De la Cueva Hernández et al., 2020).

Énfasis

Definiciones clave

- **Componente conexa (no dirigido):** Subconjunto máximo de vértices tal que existe un camino entre cualquier par.
- **Alcanzabilidad (dirigido):** Relación donde v es alcanzable desde u si existe un camino dirigido $u \to^* v$.
- **Bosque de recorrido:** Conjunto de árboles de exploración que cubren todas las componentes en grafos no conexos.

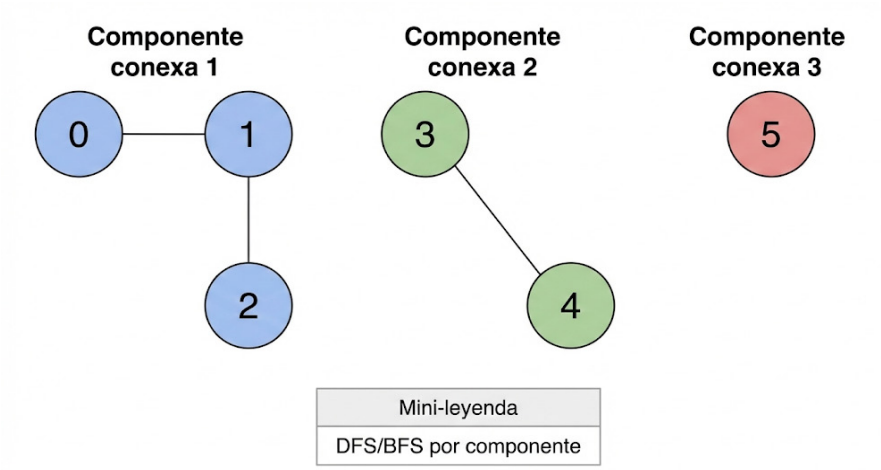


Ilustración 10: Componentes conexas en un grafo. Ejemplo de partición del conjunto de vértices en componentes desconectadas entre sí, destacando subgrafos independientes. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.4.1 Ejemplo 5 (componentes conexas en no dirigido usando DFS)

Desde la teoría, el algoritmo recorre el grafo y, cuando encuentra un nodo no visitado, lanza un DFS/BFS para capturar exactamente la componente que contiene a ese nodo. La corrección se apoya en el marcado de visitados y en la maximalidad del conjunto alcanzable (Cormen et al., 2009).

```python
# Ejemplo 5: Componentes conexas en grafo no dirigido (DFS iterativa)

def connected_components_undirected(adj):
    visited = set()
    components = []

    for start in adj.keys():
        if start in visited:
            continue

        comp = set()
        stack = [start]
        while stack:
            u = stack.pop()
            if u in visited:
                continue
            visited.add(u)
            comp.add(u)
            for v in adj.get(u, []):
                if v not in visited:
                    stack.append(v)

        components.append(comp)

    return components
```

Captura 17: Ejemplo 5 (parte 1). Componentes conexas en grafo no dirigido (DFS iterativa). Implementación de connected_components_undirected que recorre todos los nodos, lanza un DFS con pila desde cada nodo no visitado, acumula el conjunto comp de la componente actual y lo agrega a la lista components, evitando repeticiones con visited. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python

if __name__ == "__main__":
    # Grafo no dirigido: se listan vecinos en ambos sentidos
    adj = {
        0: [1],
        1: [0, 2],
        2: [1],
        3: [4],
        4: [3],
        5: []  # nodo aislado
    }

    comps = connected_components_undirected(adj)
    print("Componentes:", comps)

    # Normalizar tamaños para prueba
    sizes = sorted(len(c) for c in comps)
    print("Tamaños:", sizes)

    # Pruebas simples: {0,1,2}, {3,4}, {5}
    assert sizes == [1, 2, 3]
    assert any(c == {0, 1, 2} for c in comps)
    assert any(c == {3, 4} for c in comps)
    assert any(c == {5} for c in comps)
```

Captura 18: Ejemplo 5 (parte 2). Ejecución y validación de componentes. Bloque principal que define un grafo no dirigido con un nodo aislado, imprime las componentes encontradas, normaliza tamaños para comparación y verifica con assert la presencia de las componentes esperadas {0,1,2}, {3,4} y {5}. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`visited`	Evita recorrer dos veces vértices ya asignados a una componente.
Bucle `for start in adj.keys()`	Recorre todos los vértices; inicia una nueva búsqueda si el vértice no fue visitado.
`comp`	Acumula vértices de la componente actual alcanzables desde `start`.
`stack`	Implementa DFS iterativa para capturar la componente completa.
`components.append(comp)`	Guarda la componente hallada; el conjunto es máximo por construcción del DFS.
`assert sizes == [1,2,3]`	Valida que el algoritmo detecta correctamente una componente aislada y dos componentes no triviales.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

3.5 Clausura transitiva: alcanzabilidad y algoritmo de Warshall (matriz), complejidad O(n³)

La clausura transitiva de un grafo dirigido define, para todo par (i, j), si existe un camino dirigido $i \to^* j$. El algoritmo de Warshall computa esta relación mediante una programación dinámica sobre una matriz booleana R, actualizando $R[i][j] := R[i][j]\ OR\ (R[i][k]\ AND\ R[k][j])$ para cada k como posible intermediario. Su complejidad temporal es $O(n^3)$ y espacial $O(n^2)$, por lo que es adecuado cuando se requiere responder muchas consultas de alcanzabilidad en grafos de tamaño moderado (Cormen et al., 2009; Warshall, 1962). En el material de la asignatura se incluye Warshall como algoritmo clásico de contorno/cierre transitivo (De la Cueva Hernández et al., 2020).

Énfasis

Definiciones clave

- **Clausura transitiva:** Relación de alcanzabilidad total entre pares de vértices: R[i][j]=-True si existe camino de i a j.
- **Matriz de alcanzabilidad:** Matriz booleana que representa la clausura transitiva del grafo.
- **Actualización de Warshall:** Regla booleana que incorpora a k como intermediario para mejorar alcanzabilidad.

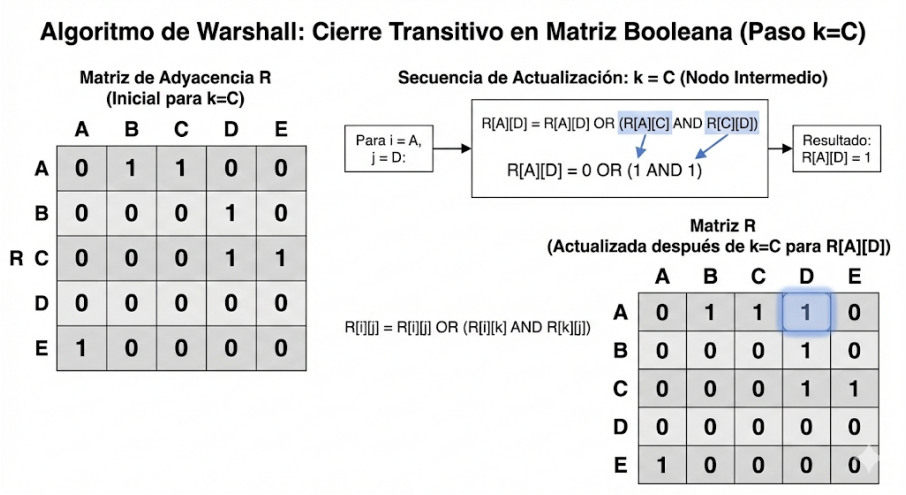


Ilustración 11: Algoritmo de Warshall (cierre transitivo) – actualización para k=C. Ejemplo de actualización de una matriz booleana de alcanzabilidad usando un nodo intermedio, mostrando la regla lógica y el resultado. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.5.1 Ejemplo 6 (Warshall sobre matriz booleana + conversión desde lista de adyacencia).

Desde la teoría, Warshall requiere una matriz inicial de adyacencia/alcanzabilidad directa. La conversión desde lista permite integrar el algoritmo con representaciones usuales de DFS/BFS y luego contestar consultas u→v en O(1) (Cormen et al., 2009; Warshall, 1962).

```python
# Ejemplo 6: Clausura transitiva con Warshall (matriz booleana) + conversión desde lista

def nodes_sorted(adj):
    nodes = set(adj.keys())
    for u in adj:
        for v in adj[u]:
            nodes.add(v)
    return sorted(nodes)

def adjacency_to_bool_matrix(adj, nodes):
    idx = {v: i for i, v in enumerate(nodes)}
    n = len(nodes)
    R = [[False] * n for _ in range(n)]
    for u in nodes:
        i = idx[u]
        # opcional: alcanzabilidad reflexiva (camino vacío)
        R[i][i] = True
        for v in adj.get(u, []):
            R[i][idx[v]] = True
    return R, idx

def warshall_transitive_closure(R):
    n = len(R)
    # Copia defensiva
    reach = [row[:] for row in R]
    for k in range(n):
        for i in range(n):
            if reach[i][k]:  # micro-optimización booleana
                for j in range(n):
                    reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach
```

Captura 19: Ejemplo 6 (parte 1). Warshall (cierre transitivo) sobre matriz booleana desde lista de adyacencia. Código que (i) normaliza y ordena el conjunto de nodos (nodes_sorted), (ii) convierte la lista de adyacencia a una matriz booleana de alcanzabilidad inicial R (adjacency_to_bool_matrix, incluyendo alcanzabilidad reflexiva en la diagonal), y (iii) aplica Warshall (warshall_transitive_closure) para obtener el cierre transitivo actualizando reach[i][j] con la regla lógica usando un nodo intermedio k. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python

def reachable(reach, idx, u, v):
    return reach[idx[u]][idx[v]]

if __name__ == "__main__":
    # Grafo dirigido
    adj = {
        "A": ["B"],
        "B": ["C"],
        "C": ["D"],
        "D": [],
        "E": ["A"]  # E alcanza A,B,C,D
    }

    nodes = nodes_sorted(adj)
    R0, idx = adjacency_to_bool_matrix(adj, nodes)
    reach = warshall_transitive_closure(R0)

    print("Nodos:", nodes)
    print("Alcanzabilidad A->D:", reachable(reach, idx, "A", "D"))
    print("Alcanzabilidad D->A:", reachable(reach, idx, "D", "A"))
    print("Alcanzabilidad E->D:", reachable(reach, idx, "E", "D"))

    # Pruebas simples
    assert reachable(reach, idx, "A", "D") is True
    assert reachable(reach, idx, "D", "A") is False
    assert reachable(reach, idx, "E", "D") is True
    assert reachable(reach, idx, "A", "A") is True  # reflexiva por construcción
```

Captura 20: Ejemplo 6 (parte 2). Consultas de alcanzabilidad y pruebas. Bloque que define reachable(reach, idx, u, v) para consultar alcanzabilidad en O(1) sobre la matriz resultante, construye un grafo dirigido de ejemplo, imprime consultas (A→D, D→A, E→D) y valida con assert los casos esperados, incluyendo la propiedad reflexiva (A→A) por construcción. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`nodes_sorted`	Construye el conjunto total de vértices incluyendo los que aparecen como destino y los ordena.
`adjacency_to_bool_matrix`	Inicializa la matriz booleana `R`: `R[i][j]=True` si hay arista `i→j`; agrega `R[i][i]=True` si se adopta alcanzabilidad reflexiva.
`warshall_transitive_closure`	Triple bucle por `k,i,j` aplica la recurrencia booleana de Warshall para incorporar caminos vía `k`.
`if reach[i][k]`	Reduce trabajo: si `i` no alcanza a `k`, no aporta `k` como intermediario para esa fila.
`reachable(...)`	Consulta O(1) en la matriz de clausura ya calculada.
`assert ...`	Verifica casos positivos/negativos de alcanzabilidad y la propiedad reflexiva adoptada en la inicialización.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

3.6 Comparación y selección: DFS vs BFS vs Warshall según tamaño, densidad y cantidad de consultas

DFS y BFS responden naturalmente consultas del tipo "¿qué vértices son alcanzables desde s?" en O(V+E) por ejecución (lista de adyacencia). Si se necesita contestar muchas consultas de alcanzabilidad entre pares (u, v), hay dos estrategias típicas: (i) ejecutar BFS/DFS por consulta (costo proporcional a la cantidad de consultas), o (ii) preprocesar una clausura transitiva con Warshall (costo $O(n^3)$) y responder cada consulta en O(1). La elección depende del equilibrio entre n, densidad (E), y número de consultas Q (Cormen et al., 2009). En términos prácticos: con Q grande y n moderado, Warshall amortiza el costo; con Q pequeño, DFS/BFS por consulta evita el preprocesamiento cúbico.

Énfasis

Definiciones clave

- **Preprocesamiento:** Cómputo previo (p. ej., clausura transitiva) para acelerar consultas posteriores.
- **Amortización por consultas:** Evaluación del costo total repartido entre muchas consultas.
- **Consulta de alcanzabilidad:** Pregunta booleana: "¿existe camino dirigido de u a v?".

COMPARATIVA DE ALGORITMOS DE GRAFOS: DFS, BFS y WARSHALL

CARACTERÍSTICA	DFS (Búsqueda en Profundidad)	BFS (Búsqueda en Anchura)	WARSHALL (Cierre Transitivo)
Pregunta típica	¿Hay camino?, ¿Existe ciclo?, Orden topológico.	¿Cuál es el camino más corto (sin peso)?, ¿Cuántos pasos?	¿Es alcanzable cualquier nodo desde cualquier otro?, Cierre transitivo.
Estructura usada	Pila (Stack) o Recursión.	Cola (Queue).	Matriz de Adyacencia (3 bucles anidados).
Costo (con lista de adyacencia)	$O(V+E)$ (Lineal en tamaño del grafo).	$O(V+E)$ (Lineal en tamaño del grafo).	No aplica (diseñado para matrices).
Costo (con matriz de adyacencia)	$O(V^2)$ (Menos eficiente).	$O(V^2)$ (Menos eficiente).	Preprocesamiento: $O(V^3)$, Consulta: $O(1)$.
Mejor cuando...	Se busca explorar caminos completos, laberintos, componentes conexas.	Se busca el camino mínimo (en aristas), grafos muy anchos.	Grafos densos, se requieren múltiples consultas rápidas de conectividad.

Ilustración 12: Tabla comparativa DFS, BFS y Warshall. Síntesis de preguntas típicas, estructuras usadas, costos y escenarios de uso de DFS, BFS y el cierre transitivo de Warshall. Imagen creada con herramientas de IA a partir de información proporcionada por la autora.

3.6.1 Ejemplo 7 (comparativo por cantidad de consultas: BFS por consulta vs Warshall)

Desde la teoría, el criterio de selección se observa comparando costos totales: $Q^*(V+E)$ frente a $n^3 + Q$ (aprox.). El ejemplo usa contadores simples (no temporización) para evidenciar crecimiento de trabajo, manteniendo el enfoque algorítmico (Cormen et al., 2009).

```python
# Ejemplo 7: Comparación aplicada - BFS por consulta vs Warshall (conteo de operaciones)

from collections import deque

def bfs_reachable_count_ops(adj, src, dst):
    # Devuelve (alcanzable, ops) donde ops cuenta inspecciones de vecinos
    q = deque([src])
    visited = {src}
    ops = 0

    while q:
        u = q.popleft()
        if u == dst:
            return True, ops
        for v in adj.get(u, []):
            ops += 1
            if v not in visited:
                visited.add(v)
                q.append(v)
    return False, ops

def nodes_sorted(adj):
    nodes = set(adj.keys())
    for u in adj:
        for v in adj[u]:
            nodes.add(v)
    return sorted(nodes)
```

Captura 21: Ejemplo 7 (parte 1). BFS por consulta con conteo de operaciones. Implementación de bfs_reachable_count_ops que determina si un destino es alcanzable desde un origen en un grafo dirigido (lista de adyacencia), contabilizando inspecciones de vecinos como aproximación del costo por consulta; incluye utilitario nodes_sorted para normalizar el conjunto de nodos. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def adjacency_to_bool_matrix(adj, nodes):
    idx = {v: i for i, v in enumerate(nodes)}
    n = len(nodes)
    R = [[False] * n for _ in range(n)]
    for u in nodes:
        i = idx[u]
        R[i][i] = True
        for v in adj.get(u, []):
            R[i][idx[v]] = True
    return R, idx

def warshall_with_ops(R):
    n = len(R)
    reach = [row[:] for row in R]
    ops = 0  # cuenta evaluaciones básicas
    for k in range(n):
        for i in range(n):
            for j in range(n):
                ops += 1
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach, ops

if __name__ == "__main__":
    # Grafo dirigido moderado (n=8) y disperso
    adj = {
        0: [1, 2],
        1: [3],
        2: [3, 4],
        3: [5],
        4: [5, 6],
        5: [7],
        6: [7],
        7: []
    }
```

Captura 22: Ejemplo 7 (parte 2). Preparación de Warshall y conteo $O(n^3)$. Conversión de lista de adyacencia a matriz booleana (adjacency_to_bool_matrix) y ejecución de Warshall en warshall_with_ops, contando evaluaciones del triple bucle como estimación del costo de preprocesamiento. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
queries = [(0, 7), (2, 7), (1, 6), (6, 0), (4, 7), (0, 6), (2, 5), (7, 0)]
# Simular "muchas consultas" repitiendo el conjunto
Q_small = queries
Q_large = queries * 50  # 400 consultas

# Estrategia 1: BFS por consulta
def total_bfs_ops(Q):
    total = 0
    answers = []
    for (u, v) in Q:
        ok, ops = bfs_reachable_count_ops(adj, u, v)
        total += ops
        answers.append(ok)
    return total, answers

bfs_ops_small, ans_small = total_bfs_ops(Q_small)
bfs_ops_large, ans_large = total_bfs_ops(Q_large)

# Estrategia 2: Warshall una vez + consultas O(1)
nodes = nodes_sorted(adj)
R0, idx = adjacency_to_bool_matrix(adj, nodes)
reach, warshall_ops = warshall_with_ops(R0)

def answer_with_warshall(Q):
    # costo de consulta ~ O(1); se cuenta 1 op por consulta para evidenciar escala
    ops = 0
    answers = []
    for (u, v) in Q:
        ops += 1
        answers.append(reach[idx[u]][idx[v]])
    return ops, answers

w_ops_small, w_ans_small = answer_with_warshall(Q_small)
w_ops_large, w_ans_large = answer_with_warshall(Q_large)
```

Captura 23: Ejemplo 7 (parte 3). Simulación de múltiples consultas (Q pequeño vs Q grande). Definición de un conjunto base de consultas de alcanzabilidad y su repetición para simular carga alta, junto con total_bfs_ops que acumula operaciones totales al aplicar BFS por cada consulta y registra respuestas booleanas. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
print("BFS por consulta - ops (Q pequeño):", bfs_ops_small)
print("BFS por consulta - ops (Q grande):", bfs_ops_large)
print("Warshall preproc - ops:", warshall_ops)
print("Warshall consultas - ops (Q pequeño):", w_ops_small)
print("Warshall consultas - ops (Q grande):", w_ops_large)

# Pruebas simples: ambas estrategias deben coincidir en respuestas
assert ans_small == w_ans_small
# Para el caso repetido, todas las respuestas deben coincidir en patrón
assert ans_large == w_ans_large
```

Captura 24: Ejemplo 7 (parte 4). Comparación BFS vs Warshall (preproceso + consultas O(1)) y validación. Preprocesamiento con Warshall una vez, función answer_with_warshall para responder consultas por acceso directo a la matriz de alcanzabilidad, impresión comparativa de operaciones y assert para verificar que ambas estrategias devuelven las mismas respuestas. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`bfs_reachable_count_ops`	Ejecuta BFS para decidir alcanzabilidad `src→dst`; cuenta inspecciones de vecinos como aproximación del trabajo.
`Q_small` y `Q_large`	Modelan escenarios con pocas vs muchas consultas para comparar estrategias.
`total_bfs_ops`	Acumula el costo total de ejecutar BFS repetidamente; el costo crece con `Q`.
`warshall_with_ops`	Calcula clausura transitiva una sola vez; el costo base depende de `n³` (número de nodos).
`answer_with_warshall`	Responde consultas en O(1) accediendo a la matriz; aquí se cuenta 1 operación por consulta para mostrar escala.
`assert ans_small == w_ans_small`	Verifica consistencia: preprocesamiento y BFS por consulta deben coincidir en verdad/falsedad.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 2: Interactive Graph Activities Platform.**

4. Grafos ponderados: algoritmos de caminos mínimos, incluyendo Dijkstra implementado con heapq

4.1 Concepto de grafo ponderado y representaciones

Un grafo ponderado asocia a cada arista un valor numérico (peso) que modela costo, distancia, tiempo u otra magnitud acumulable sobre un recorrido. En grafos dirigidos, el peso pertenece al arco $(u \to v)$; en no dirigidos, la arista $\{u, v\}$ suele representarse como dos arcos simétricos. Para algoritmos de caminos mínimos es frecuente la lista de adyacencia ponderada por su eficiencia espacial en grafos dispersos (Cormen et al., 2022).

Énfasis

Definiciones clave

- **Grafo ponderado:** Estructura $G = (V, E, w)$ donde V es el conjunto de vértices, E el conjunto de aristas (o arcos) y $w: E \to \mathbb{R}$ asigna un peso a cada arista/arco.
- **Peso (costo):** Valor numérico asociado a una arista/arco que se acumula al recorrer un camino para obtener su costo total.
- **Lista de adyacencia ponderada:** Representación que asocia a cada vértice u la colección de pares $(v, w(u, v))$ para cada vecino alcanzable.

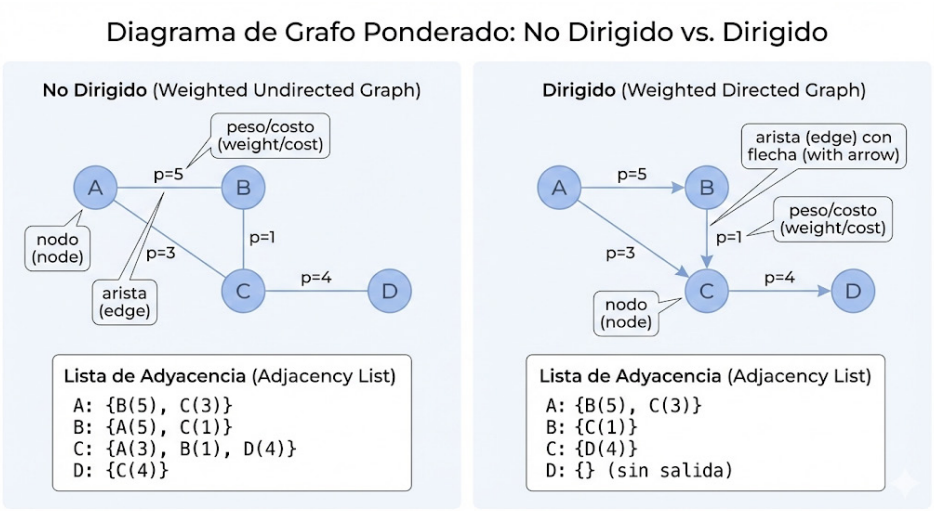


Ilustración 13: Grafo ponderado: no dirigido vs dirigido. Comparación de cómo cambian las aristas y la lista de adyacencia cuando los pesos se asocian a aristas simétricas (no dirigido) o a arcos con orientación (dirigido), destacando la diferencia entre reciprocidad y salidas. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

4.1.1 Ejemplo 1

Para implementar caminos mínimos, resulta operativo representar el grafo como un diccionario de listas de tuplas o como diccionario de diccionarios. Esta elección permite recorrer vecinos de un nodo en tiempo proporcional a su grado y consultar pesos cuando se valida un camino.

```python
# Representación de grafo ponderado (dirigido o no dirigido) con:
# adj[u] = [(v, w), ...]
# Incluye: agregar arista, obtener peso u->v, listar vecinos.

class WeightedGraph:
    def __init__(self, directed=False):
        self.directed = directed
        self.adj = {}  # u -> list of (v, w)

    def add_node(self, u):
        if u not in self.adj:
            self.adj[u] = []

    def add_edge(self, u, v, w):
        self.add_node(u)
        self.add_node(v)
        self.adj[u].append((v, w))
        if not self.directed:
            self.adj[v].append((u, w))

    def neighbors(self, u):
        return self.adj.get(u, [])

```

```python
    def get_weight(self, u, v):
        for nbr, w in self.neighbors(u):
            if nbr == v:
                return w
        return None  # no existe arco u->v

# Pruebas simples
g = WeightedGraph(directed=False)
g.add_edge("A", "B", 3)
g.add_edge("A", "C", 1)
g.add_edge("C", "B", 1)

print("Vecinos de A:", g.neighbors("A"))
print("Peso A->B:", g.get_weight("A", "B"))
print("Peso B->A (no dirigido):", g.get_weight("B", "A"))

assert g.get_weight("A", "B") == 3
assert g.get_weight("B", "A") == 3
assert g.get_weight("A", "D") is None
```

Captura 25: Grafo ponderado (dirigido/no dirigido) con lista de adyacencia de pares (v, w). Implementación de la clase WeightedGraph con representación adj[u] = [(v, peso), ...], métodos para agregar nodos/aristas, listar vecinos y consultar el peso de un arco u→v recorriendo la lista de vecinos, más un bloque de pruebas con prints y asserts que verifica simetría en modo no dirigido y ausencia de aristas inexistentes. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`class WeightedGraph`	Define una estructura para grafo ponderado con indicador `directed` para controlar si se agregan aristas simétricas.
`self.adj = {}`	Almacena la lista de adyacencia: para cada nodo `u`, mantiene una lista de pares `(v, w)`.
`add_node(u)`	Garantiza que el nodo exista en `adj` antes de insertar aristas.
`add_edge(u, v, w)`	Inserta el arco `u->v` con peso `w`; si el grafo no es dirigido, también inserta `v->u` con el mismo peso.
`neighbors(u)`	Devuelve los vecinos y pesos salientes desde `u`, evitando errores si `u` no existe.
`get_weight(u, v)`	Recorre la lista de `u` para encontrar el peso del arco `u->v`; retorna `None` si no existe.
Pruebas (`print`, `assert`)	Muestran vecinos/pesos y verifican propiedades: simetría en no dirigido y ausencia de arista.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

4.2 Problema de caminos mínimos: definiciones, variantes y restricciones de pesos

El problema de caminos mínimos consiste en hallar, para un grafo ponderado, el camino de costo mínimo entre un origen s y uno o varios destinos, donde el costo del camino es la suma de pesos de sus aristas. Se distinguen variantes: fuente única (distancias desde s a todos), par único (s a t) y todos los pares (distancias entre cada par). La elección del algoritmo depende de restricciones sobre los pesos: Dijkstra requiere pesos no negativos, mientras Bellman–Ford admite pesos negativos (Cormen et al., 2022; Dijkstra, 1959).

Énfasis

Definiciones clave

- **Camino:** Secuencia de vértices $(v_0, v_1, \ldots, v_k)$ tal que cada par consecutivo corresponde a una arista/arco del grafo.
- **Costo de un camino:** Suma $\sum_{i=0}^{k-1} w(v_i, v_{i+1})$ para un camino válido.
- **Distancia mínima:** Valor óptimo $dist(s, v)$ definido como el menor costo entre todos los caminos desde s hasta v, si existe.
- **Fuente única:** Variante que calcula $dist(s, v)$ para todo $v \in V$.

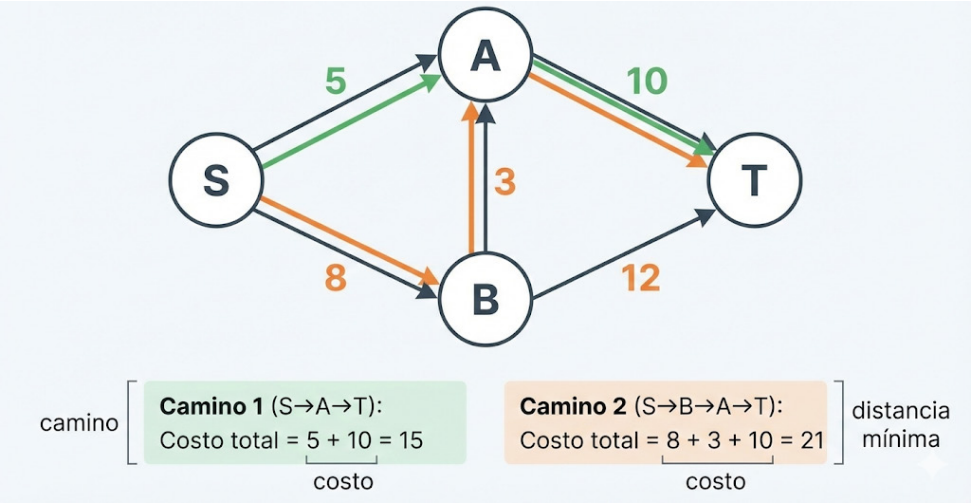


Ilustración 14: Cálculo de costos de caminos en grafo ponderado. Ejemplo de evaluación de rutas alternativas desde un origen (S) a un destino (T), sumando pesos por camino para comparar costos totales y seleccionar el de menor costo. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

4.2.1 Ejemplo 2

Antes de implementar Dijkstra, resulta útil disponer de una función que compute el costo de una ruta explícita y permita verificar soluciones. Esta verificación conecta definiciones formales (camino/costo) con pruebas sobre datos concretos.

```python
class WeightedGraph:
    def __init__(self, directed=False):
        self.directed = directed
        self.adj = {}
    def add_node(self, u):
        if u not in self.adj:
            self.adj[u] = []
    def add_edge(self, u, v, w):
        self.add_node(u)
        self.add_node(v)
        self.adj[u].append((v, w))
        if not self.directed:
            self.adj[v].append((u, w))
    def neighbors(self, u):
        return self.adj.get(u, [])
    def get_weight(self, u, v):
        for nbr, w in self.neighbors(u):
            if nbr == v:
                return w
        return None
def path_cost(g, path):
    """Devuelve el costo total del camino (lista de nodos). Lanza error si la ruta no es válida."""
    if len(path) < 2:
        return 0
    total = 0
    for i in range(len(path) - 1):
        w = g.get_weight(path[i], path[i+1])
        if w is None:
            raise ValueError(f"Arista inexistente: {path[i]} -> {path[i+1]}")
        total += w
    return total
# Pruebas simples
g = WeightedGraph(directed=True)
g.add_edge("S", "A", 4)
g.add_edge("S", "B", 1)
g.add_edge("B", "A", 2)
g.add_edge("A", "T", 1)
g.add_edge("B", "T", 7)
ruta1 = ["S", "A", "T"]       # costo 5
ruta2 = ["S", "B", "A", "T"]  # costo 4
print("Costo ruta1:", path_cost(g, ruta1))
print("Costo ruta2:", path_cost(g, ruta2))
assert path_cost(g, ruta1) == 5
assert path_cost(g, ruta2) == 4
```

Captura 26: Cálculo de costo de caminos en grafo ponderado dirigido. Código que define WeightedGraph con lista de adyacencia adj[u]=[(v,peso),...] y una función path_cost(g, path) que suma los pesos de una ruta (lista de nodos) validando cada arista y lanzando ValueError si falta alguna; incluye pruebas que construyen un grafo dirigido (S, A, B, T), evalúan dos rutas alternativas y verifican los costos con prints y asserts. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`path_cost(g, path)`	Implementa la definición de costo como suma de pesos sobre aristas consecutivas del camino.
Validación con `get_weight`	Verifica que cada transición `u->v` exista; si no existe, falla con `ValueError` para evitar costos inconsistentes.
Grafo dirigido de prueba	Modela rutas donde no necesariamente existe retorno, preparando el escenario para caminos mínimos en dirigidos.
`ruta1`, `ruta2`	Construye dos rutas candidatas y compara sus costos calculados con los esperados.
`assert`	Formaliza la verificación para que el script falle si la definición implementada no coincide con los valores esperados.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

4.3 Dijkstra: hipótesis, idea greedy y versión "naive" (sin heap)

El algoritmo de Dijkstra resuelve **fuente única** en grafos con **pesos no negativos** seleccionando iterativamente el vértice no procesado con menor distancia tentativa (criterio greedy) y relajando sus aristas. La corrección depende de la no negatividad: al "fijar" un vértice con distancia mínima, esa distancia no puede mejorar posteriormente si no existen pesos negativos (Dijkstra, 1959; Cormen et al., 2022). La versión "naïve" implementa la selección del mínimo con búsqueda lineal, derivando típicamente en $O(|V|^2 + |E|)$.

Énfasis

Definiciones clave

- **Relajación:** Operación que intenta mejorar una distancia tentativa: si $dist[v] > dist[u] + w(u,v)$ entonces se actualiza $dist[v]$.
- **Distancia tentativa:** Mejor costo conocido hasta el momento desde el origen a un vértice, sujeto a mejoras por relajación.
- **Conjunto de visitados (fijados):** Conjunto de vértices cuya distancia se considera definitiva bajo la hipótesis de pesos no negativos.

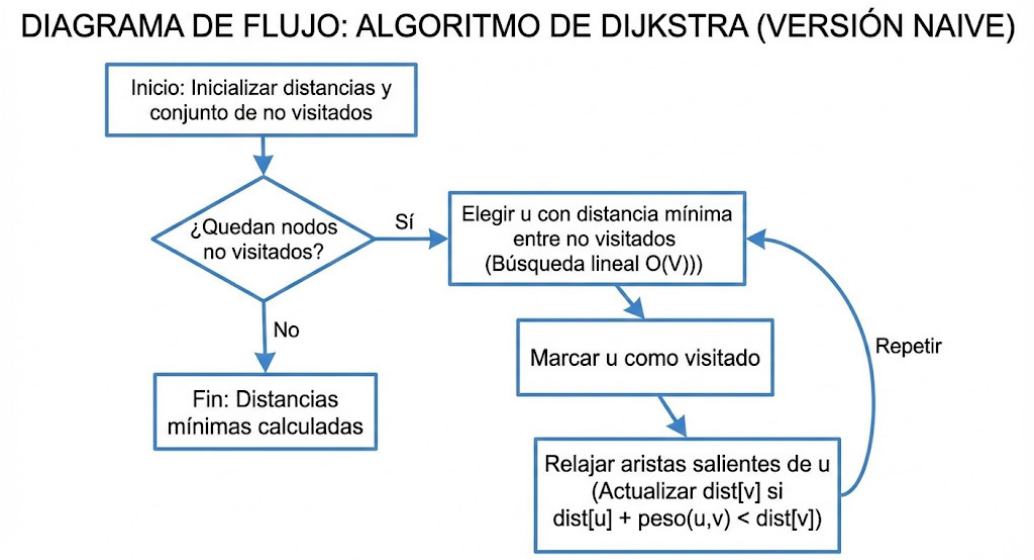


Ilustración 15: Diagrama de flujo de Dijkstra (versión naïve). Flujo del algoritmo: inicialización de distancias, selección iterativa del nodo no visitado con distancia mínima (búsqueda lineal), marcado de visitado y relajación de aristas para actualizar distancias. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

4.3.1 Ejemplo 3

La implementación "naive" permite observar explícitamente el ciclo de: seleccionar mínimo → fijar → relajar vecinos. Este diseño evidencia el costo de seleccionar el mínimo por búsqueda lineal y facilita comparar con la variante con cola de prioridad.

```python
import math

class WeightedGraph:
    def __init__(self, directed=False):
        self.directed = directed
        self.adj = {}

    def add_node(self, u):
        if u not in self.adj:
            self.adj[u] = []

    def add_edge(self, u, v, w):
        self.add_node(u)
        self.add_node(v)
        self.adj[u].append((v, w))
        if not self.directed:
            self.adj[v].append((u, w))

    def nodes(self):
        return list(self.adj.keys())

    def neighbors(self, u):
        return self.adj.get(u, [])

def dijkstra_naive(g, source):
    dist = {v: math.inf for v in g.nodes()}
    dist[source] = 0
    visited = set()

    while len(visited) < len(g.nodes()):
        # Selección lineal del no visitado con menor distancia
        u = None
        best = math.inf
        for v in g.nodes():
            if v not in visited and dist[v] < best:
                best = dist[v]
                u = v

        if u is None:  # nodos restantes inalcanzables
            break

        visited.add(u)
```

Captura 27: Dijkstra naïve (parte 1). Estructura del grafo y selección lineal del mínimo. Código que define WeightedGraph (lista de adyacencia con pares (vecino, peso)) y la función dijkstra_naive, inicializando dist con infinito, fijando la fuente en 0 y seleccionando en cada iteración el nodo no visitado con menor distancia mediante búsqueda lineal sobre g.nodes(). Desarrollado en Online IDE Pro. Código fuente de la autora.

```
43
44          # Relajación
45          for v, w in g.neighbors(u):
46              if v in visited:
47                  continue
48              cand = dist[u] + w
49              if cand < dist[v]:
50                  dist[v] = cand
51
52      return dist
53
54  # Pruebas simples
55  g = WeightedGraph(directed=False)
56  g.add_edge("A", "B", 3)
57  g.add_edge("A", "C", 1)
58  g.add_edge("C", "B", 1)
59  g.add_edge("B", "D", 2)
60  g.add_edge("C", "D", 5)
61
62  dist = dijkstra_naive(g, "A")
63  print("Distancias desde A (naive):", dist)
64
65  assert dist["A"] == 0
66  assert dist["C"] == 1
67  assert dist["B"] == 2  # A->C->B = 1+1
68  assert dist["D"] == 4  # A->C->B->D = 1+1+2
69
```

Captura 28: Dijkstra naïve (parte 2). Relajación, ejecución y validación de distancias mínimas. Continuación que aplica la relajación de aristas salientes (cand = dist[u] + w) para actualizar dist[v], luego construye un grafo no dirigido de ejemplo, ejecuta Dijkstra desde "A", imprime el diccionario de distancias y valida resultados con assert para rutas mínimas (por ejemplo A→C→B y A→C→B→D). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Inicialización `dist`	Coloca `inf` a todos los nodos excepto el origen, que se fija en 0.
`visited`	Mantiene los vértices ya fijados; su distancia no se revisa nuevamente en esta variante.
Selección lineal del mínimo	Recorre todos los nodos para elegir el no visitado con menor `dist`; esto induce costo (O(
Condición `u is None`	Detecta que no quedan vértices alcanzables desde el origen y termina.
Relajación de aristas	Para cada vecino `v` calcula `cand = dist[u] + w` y actualiza si mejora.
Pruebas con `assert`	Verifican distancias esperadas y, por lo tanto, el efecto correcto de la relajación.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

4.4 Dijkstra con heapq: cola de prioridad, entradas obsoletas y complejidad

Para reducir el costo de selección del mínimo, se utiliza una cola de prioridad: extraer el vértice con menor distancia tentativa pasa a ser $O(\log |V|)$ (según la operación de heap). En Python, heapq implementa un heap binario que permite inserciones/extracciones logarítmicas, lo que se usa para Dijkstra en grafos dispersos (Python Software Foundation, s. f.; Sedgewick & Wayne, 2011). Como heapq no soporta *decrease-key* directamente, se insertan nuevas parejas $(dist, v)$ y se descartan entradas obsoletas cuando se extraen (técnica estándar en implementaciones prácticas).

Énfasis

Definiciones clave

- **Cola de prioridad:** Estructura que permite extraer eficientemente el elemento con prioridad mínima (aquí, menor distancia).
- **Entrada obsoleta:** Par (d, v) en el heap tal que $d \neq dist[v]$ en el estado actual, por haber sido superado por una actualización posterior.
- **Heap binario:** Árbol binario implícito en un arreglo que preserva el invariante de heap para obtener mínimos con costo logarítmico.

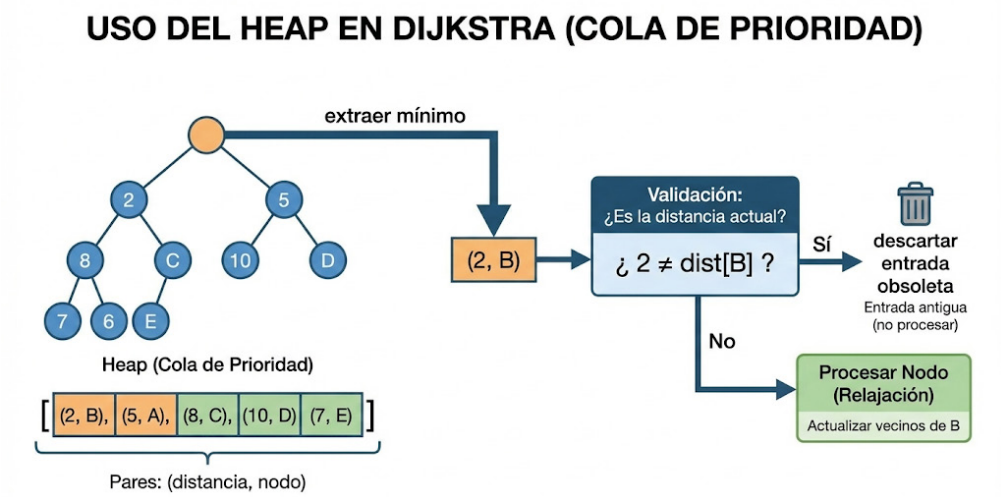


Ilustración 16: Uso de heap en Dijkstra (cola de prioridad). Representación del esquema de extracción del mínimo desde un heap de pares (distancia, nodo), validación de entradas obsoletas y procesamiento del nodo vigente para relajar aristas de salida. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

4.4.1 Ejemplo 4

Esta implementación usa heapq para obtener el próximo candidato a fijar según la menor distancia tentativa. La corrección se mantiene con pesos no negativos, y la técnica de "entradas obsoletas" reemplaza la operación *decrease-key*.

```python
import math
import heapq

class WeightedGraph:
    def __init__(self, directed=False):
        self.directed = directed
        self.adj = {}

    def add_node(self, u):
        if u not in self.adj:
            self.adj[u] = []

    def add_edge(self, u, v, w):
        self.add_node(u)
        self.add_node(v)
        self.adj[u].append((v, w))
        if not self.directed:
            self.adj[v].append((u, w))

    def nodes(self):
        return list(self.adj.keys())

    def neighbors(self, u):
        return self.adj.get(u, [])

def dijkstra_heapq(g, source):
    dist = {v: math.inf for v in g.nodes()}
    dist[source] = 0

    heap = [(0, source)]  # (distancia, nodo)

    while heap:
        d_u, u = heapq.heappop(heap)

        # Descartar entradas obsoletas
        if d_u != dist[u]:
            continue

        for v, w in g.neighbors(u):
            cand = d_u + w
            if cand < dist[v]:
                dist[v] = cand
                heapq.heappush(heap, (cand, v))

    return dist

# Pruebas simples
g = WeightedGraph(directed=False)
g.add_edge("A", "B", 3)
g.add_edge("A", "C", 1)
g.add_edge("C", "B", 1)
g.add_edge("B", "D", 2)
g.add_edge("C", "D", 5)

dist = dijkstra_heapq(g, "A")
print("Distancias desde A (heapq):", dist)

assert dist["A"] == 0
assert dist["C"] == 1
assert dist["B"] == 2
assert dist["D"] == 4
```

Captura 29: Dijkstra con heap (cola de prioridad) usando heapq. Código que implementa un grafo ponderado con lista de adyacencia y ejecuta dijkstra_heapq, manteniendo un heap de pares (distancia, nodo) para extraer el mínimo, descartando entradas obsoletas cuando d_u != dist[u], y relajando aristas para actualizar distancias; incluye un grafo de prueba y asserts que validan las distancias mínimas desde el origen. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Inicialización de `dist`	Define distancias tentativas, fijando 0 en el origen e infinito en los demás.
`heap = [(0, source)]`	Carga el heap con el origen como primer candidato de menor distancia.
`heappop`	Extrae el par con menor distancia tentativa; corresponde al próximo vértice a procesar.
Filtro de obsolescencia	`if d_u != dist[u]: continue` descarta entradas viejas que ya no reflejan el mejor valor.
Relajación	Calcula `cand = d_u + w` y actualiza `dist[v]` si mejora; luego inserta el nuevo par en el heap.
`assert`	Valida que las distancias coincidan con los caminos mínimos esperados en el grafo de prueba.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

4.5 Reconstrucción de camino mínimo: predecesores, ruta fuente→destino y caso dirigido

Para reconstruir el camino mínimo (no solo su costo), se mantiene un vector/mapa de predecesores: cuando una relajación mejora $dist[v]$, se registra prev[v] = u. Al finalizar, el camino desde s a t se obtiene retrocediendo desde t por predecesores hasta s y revir-

tiendo la secuencia (Cormen et al., 2022). En grafos dirigidos, la ruta depende de la orientación de arcos; por ello la reconstrucción debe respetar la dirección al validar el costo.

Énfasis

Definiciones clave

- **Predecesor:** Para un vértice v, es el vértice u tal que el camino mínimo conocido hacia v termina con la arista/arco (u, v).
- **Árbol de caminos mínimos (SPT):** Estructura implícita definida por predecesores que describe, desde el origen, un conjunto de caminos mínimos hacia vértices alcanzables.
- **Reconstrucción de ruta:** Procedimiento de recuperar la secuencia de vértices desde s hasta t siguiendo prev desde t hacia atrás.

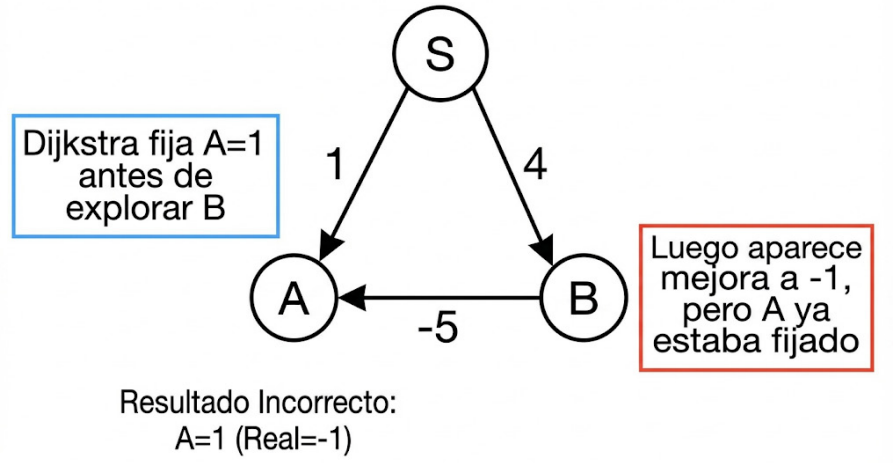


Ilustración 17: Limitación de Dijkstra con pesos negativos. Contraejemplo que muestra cómo fijar una distancia mínima demasiado pronto provoca un resultado incorrecto cuando existe una arista de peso negativo que permitiría una mejora posterior. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

4.5.1 Ejemplo 5

Se extiende Dijkstra con heapq para registrar predecesores en cada mejora de distancia. Luego se reconstruye la ruta $s \to t$ y se verifica que la suma de pesos a lo largo de la ruta coincide con dist[t].

```python
import math
import heapq

class WeightedGraph:…

def dijkstra_with_prev(g, source):…

def reconstruct_path(prev, source, target):…

def path_cost(g, path):…

# Ejemplo dirigido y ponderado (diferencia respecto a no dirigido)
g = WeightedGraph(directed=True)
g.add_edge("S", "A", 2)
g.add_edge("S", "B", 5)
g.add_edge("A", "B", 1)
g.add_edge("B", "T", 2)
g.add_edge("A", "T", 10)

dist, prev = dijkstra_with_prev(g, "S")
ruta = reconstruct_path(prev, "S", "T")

print("Distancias:", dist)
print("Ruta S->T:", ruta)
print("Costo de la ruta:", path_cost(g, ruta))

assert dist["T"] == 5  # S->A->B->T = 2+1+2
assert ruta == ["S", "A", "B", "T"]
assert path_cost(g, ruta) == dist["T"]

# Prueba de direccionalidad: NO existe T->B, ni B->A necesariamente
assert g.get_weight("T", "B") is None
```

Captura 30: Script de prueba que integra un grafo ponderado dirigido y ejecuta Dijkstra con reconstrucción de ruta: calcula distancias mínimas desde S, recupera el camino hasta T y valida el costo con assert (consistencia entre distancia y suma de pesos). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
class WeightedGraph:
    def __init__(self, directed=False):
        self.directed = directed
        self.adj = {}

    def add_node(self, u):
        if u not in self.adj:
            self.adj[u] = []

    def add_edge(self, u, v, w):
        self.add_node(u)
        self.add_node(v)
        self.adj[u].append((v, w))
        if not self.directed:
            self.adj[v].append((u, w))

    def nodes(self):
        return list(self.adj.keys())

    def neighbors(self, u):
        return self.adj.get(u, [])

    def get_weight(self, u, v):
        for nbr, w in self.neighbors(u):
            if nbr == v:
                return w
        return None
```

```python
def dijkstra_with_prev(g, source):
    dist = {v: math.inf for v in g.nodes()}
    prev = {v: None for v in g.nodes()}
    dist[source] = 0

    heap = [(0, source)]
    while heap:
        d_u, u = heapq.heappop(heap)
        if d_u != dist[u]:
            continue

        for v, w in g.neighbors(u):
            cand = d_u + w
            if cand < dist[v]:
                dist[v] = cand
                prev[v] = u
                heapq.heappush(heap, (cand, v))

    return dist, prev
```

Captura 31: Implementación base: clase WeightedGraph (lista de adyacencia con pares (vecino, peso)) y función dijkstra_with_prev, que computa distancias mínimas con heapq y registra el predecesor prev para poder reconstruir caminos. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def reconstruct_path(prev, source, target):
    path = []
    cur = target
    while cur is not None:
        path.append(cur)
        if cur == source:
            break
        cur = prev[cur]
    path.reverse()
    if not path or path[0] != source:
        return None  # inalcanzable
    return path
```

Captura 32: Función reconstruct_path(prev, source, target): reconstruye la ruta desde el destino hacia el origen siguiendo el diccionario de predecesores, invierte el recorrido y devuelve None si el nodo objetivo resulta inalcanzable. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def path_cost(g, path):
    if path is None:
        return math.inf
    total = 0
    for i in range(len(path) - 1):
        w = g.get_weight(path[i], path[i+1])
        if w is None:
            raise ValueError("Ruta inválida respecto a la dirección/pesos del grafo.")
        total += w
    return total
```

Captura 33: Función path_cost(g, path): calcula el costo total de una ruta como suma de pesos de aristas consecutivas y lanza una excepción si el camino no es válido (arista inexistente o incompatibilidad con dirección/pesos). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`prev = {v: None ...}`	Inicializa el mapa de predecesores para registrar cómo se alcanza cada vértice en el mejor camino conocido.
Actualización `prev[v] = u`	Se ejecuta únicamente cuando `cand < dist[v]`; asegura que `prev` corresponda al camino asociado a la mejor distancia vigente.
`reconstruct_path`	Retrocede desde `target` usando `prev` hasta llegar a `source`; luego invierte la lista para obtener orden fuente→destino.
Validación de alcanzabilidad	Si la reconstrucción no llega al origen, retorna `None` y se interpreta como "inalcanzable".
`path_cost` con `get_weight`	Verifica que cada arco del camino exista en el grafo dirigido y acumula pesos para comprobar consistencia.
`assert dist["T"] == 5`	Confirma el costo mínimo esperado para el destino `T`.
`assert ruta == [...]`	Confirma que el camino reconstruido coincide con la ruta mínima prevista.
`assert g.get_weight("T","B") is None`	Evidencia que la estructura es dirigida: no se asumen arcos inversos.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 3–Interactive Graph Activities–parte 2.**

5. Orden topológico y detección de ciclos en grafos dirigidos.

5.1 Grafos dirigidos, DAG y precedencia: motivación del orden topológico

En un grafo dirigido $G = (V, E)$, cada arista $(u \to v)$ modela una relación de precedencia: u debe ocurrir antes que v. Un orden topológico es una enumeración lineal de los vértices tal que toda arista respeta el orden (Cormen et al., 2009). Este orden existe si y solo si el grafo es un DAG (Directed Acyclic Graph), es decir, un grafo dirigido sin ciclos (Kleinberg & Tardos, 2005). En contextos de planificación (tareas con prerequisitos), compilación (dependencias entre módulos) o pipelines de datos/ML (etapas de preprocesamiento → entrenamiento → evaluación), el orden topológico permite construir un plan ejecutable que respeta dependencias (Skiena, 2008).

Énfasis

Definiciones clave

- **Grafo dirigido (dígrafo):** Estructura $G = (V, E)$ donde cada arista es un par ordenado (u, v) que indica dirección de u hacia v.
- **DAG (Directed Acyclic Graph):** Grafo dirigido que no contiene ciclos dirigidos.
- **Orden topológico:** Permutación de V tal que para toda arista $u \to v$, u aparece antes que v en la permutación.

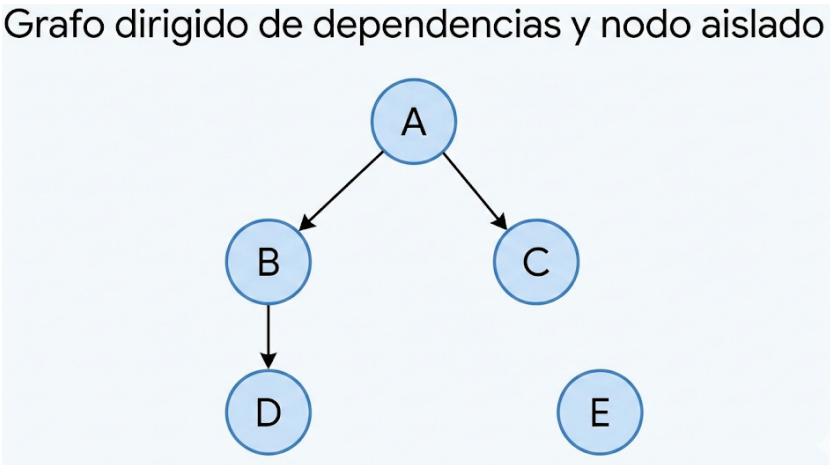


Ilustración 18 Ejemplo 1. Representación con lista de adyacencia y nodos sin salidas. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

```python
# Representación de grafo dirigido con lista de adyacencia (dict: nodo -> list[vecinos]).
# Incluye nodos sin salidas e incluso nodos aislados.

from typing import Dict, List, Set, Iterable, Tuple, Any

Graph = Dict[Any, List[Any]]

def add_node(g: Graph, u: Any) -> None:
    """Asegura que el nodo exista en el diccionario, aunque no tenga salidas."""
    if u not in g:
        g[u] = []

def add_edge(g: Graph, u: Any, v: Any) -> None:
    """Agrega arista dirigida u->v y asegura la presencia de ambos nodos."""
    add_node(g, u)
    add_node(g, v)
    g[u].append(v)

def nodes(g: Graph) -> Set[Any]:
    """Devuelve el conjunto total de nodos, incluyendo los que aparecen solo como destino."""
    s = set(g.keys())
    for u, nbrs in g.items():
        for v in nbrs:
            s.add(v)
    return s

def edges(g: Graph) -> List[Tuple[Any, Any]]:
    """Lista de aristas dirigidas (u, v)."""
    out = []
    for u, nbrs in g.items():
        for v in nbrs:
            out.append((u, v))
    return out
```

Captura 34: Implementación grafo dirigido. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
if __name__ == "__main__":
    g: Graph = {}

    # Dependencias: A -> B, A -> C, B -> D
    add_edge(g, "A", "B")
    add_edge(g, "A", "C")
    add_edge(g, "B", "D")

    # Nodo sin salidas (explícito): E existe pero no depende de nadie ni tiene dependientes
    add_node(g, "E")

    print("Adyacencias:", g)
    print("Nodos:", sorted(nodes(g)))
    print("Aristas:", edges(g))

    assert "E" in nodes(g)
    assert ("A", "B") in edges(g)
    assert g["D"] == []  # D aparece como destino; se asegura su presencia con lista vacía
```

Captura 35: . Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`Graph = Dict[Any, List[Any]]`	Define la representación como diccionario: cada nodo mapea a su lista de sucesores.
`add_node(g,u)`	Inserta `u` con lista vacía si no existe; preserva nodos sin salidas e aislados.
`add_edge(g,u,v)`	Agrega la arista `u→v` y garantiza que `u` y `v` existan en el diccionario.
`nodes(g)`	Construye el conjunto total de nodos, sumando claves y también destinos encontrados.
`edges(g)`	Recorre listas de adyacencia y materializa las aristas para validación y pruebas.
Pruebas (`assert`)	Verifican: inclusión de nodo aislado, existencia de arista y presencia de destino con lista vacía.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

5.2 Detección de ciclos: definición formal e impacto sobre el orden topológico (DFS con estados)

Un ciclo dirigido es una secuencia de vértices $v_0, v_1, \dots, v_k$ con $k \geq 1$ tal que existen aristas $v_i \to v_{i+1}$ (para $0 \leq i < k$) y $v_k \to v_0$. La presencia de un ciclo implica una dependencia circular, por lo que no existe un orden topológico válido (Cormen et al., 2009). Una técnica estándar para detectar ciclos en grafos dirigidos es DFS con estados (colores): 0=no visitado, 1=en proceso, 2=finalizado; encontrar una arista hacia un nodo "en proceso" equivale a detectar un *back edge* y, por lo tanto, un ciclo (Sedgewick & Wayne, 2011).

Énfasis

Definiciones clave

- **Ciclo dirigido:** Secuencia cerrada de aristas dirigidas que permite volver al vértice inicial siguiendo la dirección de las aristas.
- **Arista de retroceso (back edge):** Arista $u \to v$ encontrada en DFS donde $v$está "en proceso", indicando retorno a un ancestro del árbol DFS.
- **Estados/colores en DFS:** Etiquetas para controlar la exploración: 0 (no visitado), 1 (en proceso), 2 (finalizado).

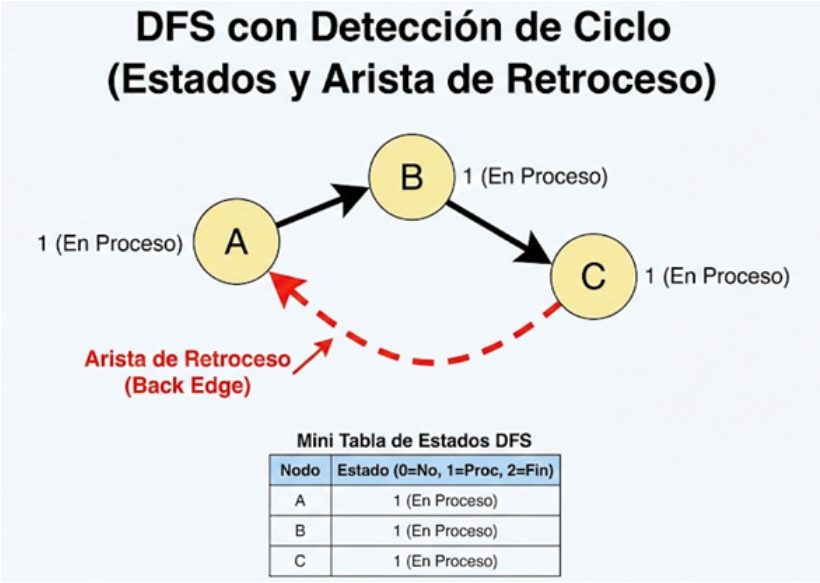


Ilustración 19: Detección de ciclos. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

5.2.1 Ejemplo 2 (detección de ciclo con DFS y estados; caso acíclico y caso cíclico)

DFS recorre el grafo siguiendo aristas dirigidas. La marca "en proceso" permite detectar dependencias circulares en el instante en que se observa una arista hacia un vértice aún no finalizado, que corresponde a un retorno en la pila implícita de DFS.

```python
from typing import Dict, List, Any, Set
Graph = Dict[Any, List[Any]]
def all_nodes(g: Graph) -> Set[Any]:
    s = set(g.keys())
    for u in g:
        for v in g[u]:
            s.add(v)
    return s

def has_cycle_dfs(g: Graph) -> bool:
    """
    Detecta ciclo dirigido usando DFS con estados:
    0 = no visitado, 1 = en proceso, 2 = finalizado.
    """
    state = {u: 0 for u in all_nodes(g)}

    def dfs(u: Any) -> bool:
        state[u] = 1  # en proceso
        for v in g.get(u, []):
            if state[v] == 1:
                return True  # back edge => ciclo
            if state[v] == 0 and dfs(v):
                return True
        state[u] = 2  # finalizado
        return False

    for u in list(state.keys()):
        if state[u] == 0:
            if dfs(u):
                return True
    return False

# Pruebas simples
if __name__ == "__main__":
    # Caso acíclico: A->B, A->C, B->D
    dag: Graph = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
    assert has_cycle_dfs(dag) is False
    print("DAG: ciclo =", has_cycle_dfs(dag))

    # Caso cíclico: A->B, B->C, C->A
    cyc: Graph = {"A": ["B"], "B": ["C"], "C": ["A"]}
    assert has_cycle_dfs(cyc) is True
    print("Cíclico: ciclo =", has_cycle_dfs(cyc))
```

Captura 36: Implementación detección de Ciclos. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`all_nodes(g)`	Construye el conjunto total de vértices incluyendo destinos, evitando omisiones.
`state`	Diccionario de estados 0/1/2 para cada nodo del grafo.
`dfs(u)`	Marca `u` en proceso (1), explora sus vecinos y finaliza (2) al retornar.
`if state[v] == 1`	Detecta arista hacia un nodo "en proceso": evidencia de ciclo dirigido.
Bucle externo `for u ...`	Lanza DFS desde cada componente para cubrir grafos desconectados.
Pruebas con `assert`	Validan que el caso acíclico retorne `False` y el cíclico retorne `True`.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

5.3 Orden topológico por DFS: postorden, pila de finalización y detección integrada de ciclos

El orden topológico puede construirse con DFS usando el tiempo de finalización: cuando un vértice termina de explorar todos sus sucesores, se inserta en una pila/lista; al finalizar, invertir esa lista produce un orden topológico en un DAG (Cormen et al., 2009). Si durante DFS aparece una arista hacia un vértice "en proceso", se detecta un ciclo y se concluye que no existe orden topológico (Sedgewick & Wayne, 2011). Esta estrategia integra, en una sola pasada, recorrido, detección de ciclos y producción del orden.

Énfasis

Definiciones clave

- **Postorden (DFS):** Convención donde un vértice se registra al finalizar la exploración completa de sus sucesores.
- **Lista de finalización:** Colección donde se agregan vértices al finalizar DFS; invertida, produce orden topológico en DAG.
- **Señalización de ciclo:** Mecanismo explícito para indicar imposibilidad de orden topológico (p. ej., devolver None).

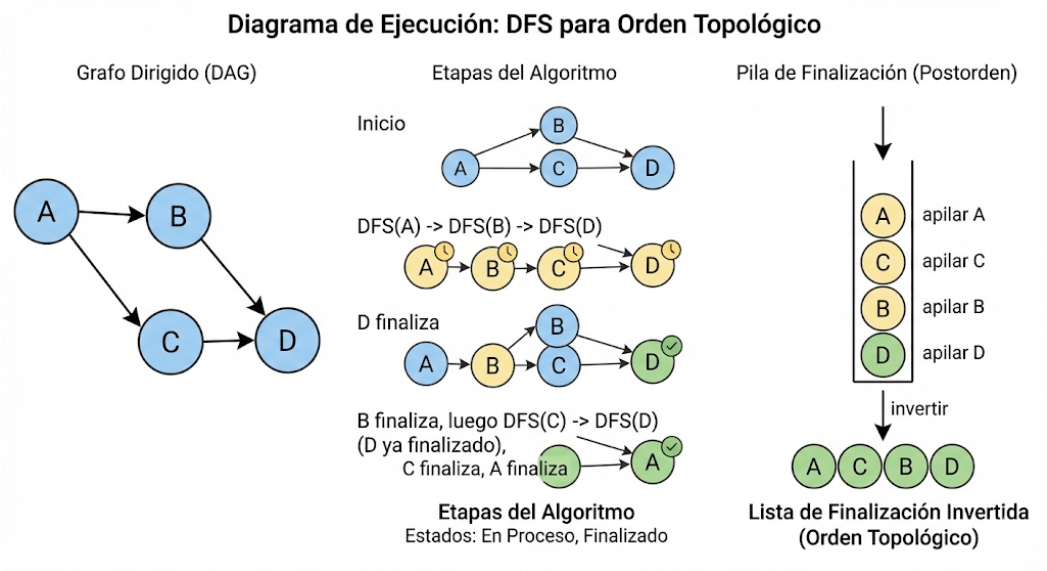


Ilustración 20: Diagrama de Ejecución. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

5.3.1 Ejemplo 3 (topológico por DFS con detección de ciclo integrada; devuelve lista o None)

DFS con estados permite construir el orden mediante postorden y, simultáneamente, detectar ciclos con la condición "vecino en proceso". De esta forma, la salida es un orden válido solo si el grafo es DAG.

```python
from typing import Dict, List, Any, Set, Optional

Graph = Dict[Any, List[Any]]

def all_nodes(g: Graph) -> Set[Any]:
    s = set(g.keys())
    for u in g:
        for v in g[u]:
            s.add(v)
    return s

def topo_sort_dfs(g: Graph) -> Optional[List[Any]]:
    """
    Orden topológico por DFS.
    Devuelve lista si es DAG; devuelve None si detecta ciclo.
    """
    state = {u: 0 for u in all_nodes(g)}  # 0 no visitado, 1 en proceso, 2 finalizado
    order: List[Any] = []
    has_cycle = False

    def dfs(u: Any) -> None:
        nonlocal has_cycle
        state[u] = 1
        for v in g.get(u, []):
            if state[v] == 1:
                has_cycle = True
                return
            if state[v] == 0:
                dfs(v)
                if has_cycle:
                    return
        state[u] = 2
        order.append(u)  # postorden (finalización)

    for u in list(state.keys()):
        if state[u] == 0:
            dfs(u)
            if has_cycle:
                return None

    order.reverse()
    return order
```

Captura 37: Implementación ejemplo 3. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
if __name__ == "__main__":
    dag: Graph = {"A": ["B", "C"], "B": ["D"], "C": ["D"], "D": []}
    ord1 = topo_sort_dfs(dag)
    print("Orden DFS (DAG):", ord1)
    assert ord1 is not None
    # Validación mínima: A antes que B y C; B/C antes que D
    assert ord1.index("A") < ord1.index("B")
    assert ord1.index("A") < ord1.index("C")
    assert ord1.index("B") < ord1.index("D")
    assert ord1.index("C") < ord1.index("D")

    cyc: Graph = {"A": ["B"], "B": ["C"], "C": ["A"]}
    ord2 = topo_sort_dfs(cyc)
    print("Orden DFS (cíclico):", ord2)
    assert ord2 is None
```

Captura 38: Implementación ejemplo 3. (cont). Desarrollado en Online IDE Pro. Código fuente de la autora.

nento	Explicación del paso a paso del código
`te`	Controla la visita (0/1/2) para detección de ciclo y para evitar repeticiones.
`er.append(u)`	Registra `u` al finalizar la exploración completa: postorden.
`er.reverse()`	Convierte la lista de finalización en un orden topológico.
`state[v] == 1`	Detecta ciclo (arista hacia nodo en proceso) y activa señal `has_cycle`.
olver `None`	Señal explícita de imposibilidad de orden topológico por existencia de ciclo.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

5.4 Orden topológico por Kahn (BFS sobre indegree): cálculo, cola y detección implícita de ciclos

El algoritmo de Kahn produce un orden topológico seleccionando repetidamente vértices con grado de entrada (indegree) igual a 0, eliminándolos del grafo y actualizando indegrees (Kahn, 1962). Si el grafo es DAG, el proceso consume todos los vértices; si existe ciclo, ningún vértice del ciclo alcanzará indegree 0, y el algoritmo termina habiendo procesado menos de $|V|$ vértices, lo cual constituye una detección implícita del ciclo (Cormen et al., 2009). La verificación posterior de un orden propuesto consiste en chequear que para toda arista $u \to v$, la posición de u sea anterior a la de v (Kleinberg & Tardos, 2005).

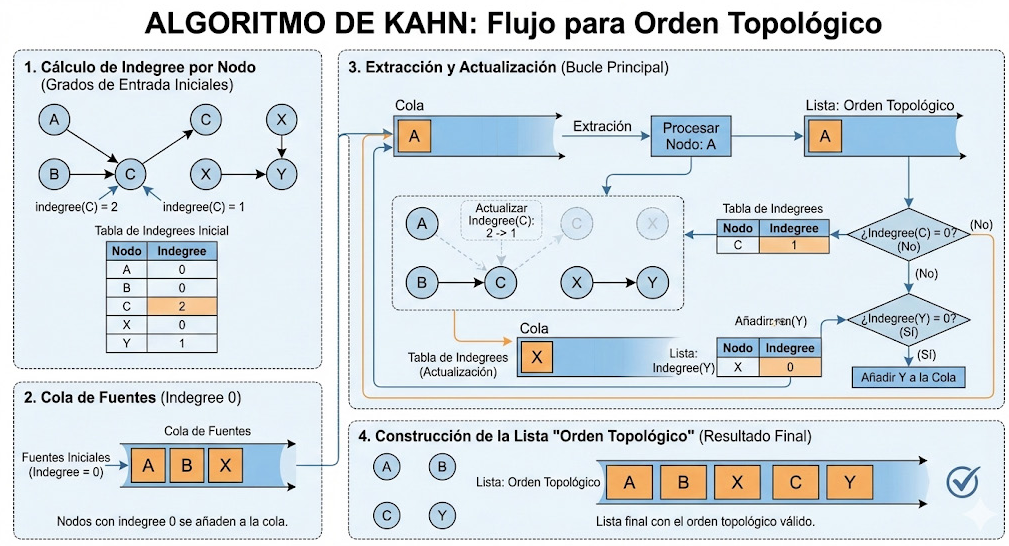


Ilustración 21: Algoritmo de Kahn – Orden Topológico. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

Énfasis

Definiciones clave

- **Grado de entrada (indegree):** Cantidad de aristas que ingresan a un vértice.
- **Fuentes:** Vértices con indegree 0 en un grafo dirigido; no tienen prerequisitos.
- **Algoritmo de Kahn:** Procedimiento iterativo que extrae fuentes, reduce indegrees y construye orden topológico si no hay ciclos.

5.4.1 Ejemplo 4 (Kahn: orden topológico + verificación check_topo + grafo desconectado)

Kahn se implementa con una cola de fuentes (indegree 0). En grafos desconectados pueden existir múltiples fuentes simultáneas; el algoritmo las procesa y produce un orden topológico global si no hay ciclos.

```python
from collections import deque
from typing import Dict, List, Any, Set, Tuple, Optional

Graph = Dict[Any, List[Any]]

def all_nodes(g: Graph) -> Set[Any]:
    s = set(g.keys())
    for u in g:
        for v in g[u]:
            s.add(v)
    return s

def compute_indegree(g: Graph) -> Dict[Any, int]:
    indeg = {u: 0 for u in all_nodes(g)}
    for u in g:
        for v in g[u]:
            indeg[v] += 1
    return indeg

def topo_sort_kahn(g: Graph) -> Optional[List[Any]]:
    """
    Orden topológico por Kahn.
    Devuelve lista si es DAG; devuelve None si detecta ciclo (procesa < |V|).
    """
    nodes = list(all_nodes(g))
    indeg = compute_indegree(g)
    q = deque([u for u in nodes if indeg[u] == 0])
    order: List[Any] = []

    while q:
        u = q.popleft()
        order.append(u)
        for v in g.get(u, []):
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    if len(order) != len(nodes):
        return None
    return order
```

Captura 39: Implementación ejemplo 4. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def check_topo(order: List[Any], edges: List[Tuple[Any, Any]]) -> bool:
    pos = {node: i for i, node in enumerate(order)}
    for u, v in edges:
        if pos[u] >= pos[v]:
            return False
    return True

def edges(g: Graph) -> List[Tuple[Any, Any]]:
    out = []
    for u, nbrs in g.items():
        for v in nbrs:
            out.append((u, v))
    return out

# Pruebas simples
if __name__ == "__main__":
    # Grafo desconectado con múltiples fuentes:
    # Componente 1: A->C, B->C
    # Componente 2: X->Y
    g: Graph = {"A": ["C"], "B": ["C"], "C": [], "X": ["Y"], "Y": []}

    order = topo_sort_kahn(g)
    print("Orden Kahn:", order)

    assert order is not None
    assert check_topo(order, edges(g)) is True

    # Evidenciar múltiples fuentes (al inicio deberían poder estar A, B y X en cualquier orden)
    assert set(order[:3]) == {"A", "B", "X"}  # este assert depende de la política de cola;
                                              # aquí es válido si el grafo y la recolección de nodos no cambian
    print("Verificación check_topo:", check_topo(order, edges(g)))
```

Captura 40: Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`compute_indegree(g)`	Inicializa indegree en 0 y cuenta aristas entrantes para cada nodo.
`q = deque([... indeg==0])`	Construye la cola de fuentes: nodos sin prerequisitos.
Bucle `while q`	Extrae una fuente, la agrega al orden y "elimina" sus salidas reduciendo indegrees.
`if indeg[v] == 0`	Cuando un nodo queda sin prerequisitos, pasa a ser fuente y se encola.
`len(order) != len(nodes)`	Condición de ciclo: quedaron nodos sin procesar por indegrees nunca nulos.
`check_topo(order, edges)`	Verifica la propiedad: para cada arista `u→v`, `u` debe aparecer antes que `v`.

> Nota práctica: el `assert set(order[:3]) == {"A","B","X"}` depende del orden de iteración interna de nodos. Si se requiere determinismo, se recomienda ordenar la lista de nodos antes de encolar o usar una cola con prioridad.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

5.5 Verificación, robustez y caso aplicado de dependencias (incluye detección de ciclo con Kahn)

Un orden topológico es una **solución** que debe verificarse contra el conjunto de aristas: si para toda arista $u \to v$ se cumple $pos(u) < pos(v)$, el orden es válido (Kleinberg & Tardos, 2005). La robustez exige contemplar: nodos aislados, nodos sin salidas, grafos desconectados y nodos que solo aparecen como destino. En aplicaciones reales (correlatividades, tareas de pipeline, etapas de entrenamiento y evaluación), la detección de ciclos evita planes inconsistentes; en Kahn, un ciclo se evidencia cuando el tamaño del orden resultante es menor que $|V|$ (Kahn, 1962; Cormen et al., 2009).

Énfasis

Definiciones clave

- **Verificación de orden topológico:** Procedimiento que comprueba para cada arista $u \to v$ que u preceda a v en el orden propuesto.
- **Dependencia circular:** Conjunto de precedencias que forma un ciclo, imposibilitando un plan lineal.
- **Detección de ciclo por Kahn:** Conclusión de ciclo si el algoritmo procesa menos vértices que el total.

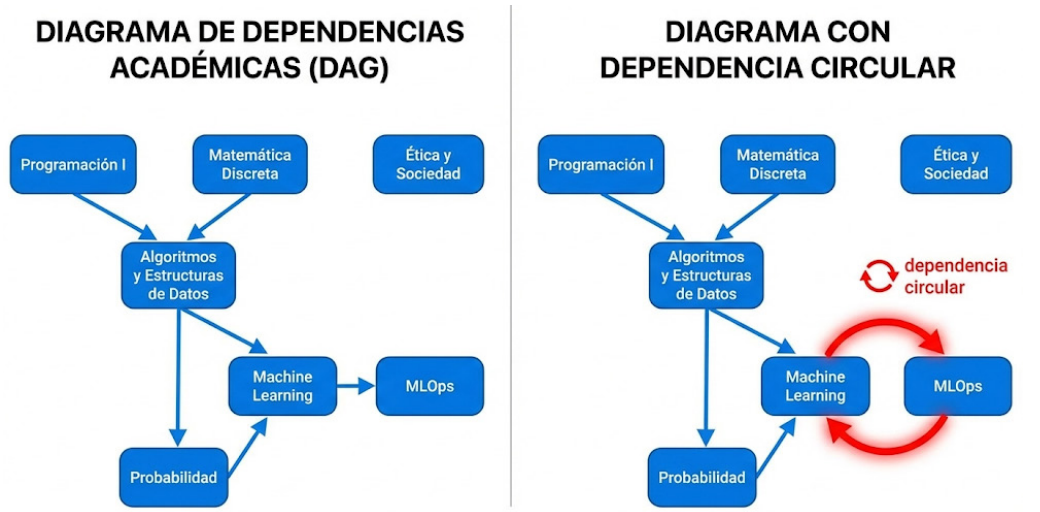


Ilustración 22: Diagramas de Dependencia. Imagen creada usando herramientas de IA a partir de información proporcionada por la autora.

5.5.1 Ejemplo 5 (caso aplicado: correlatividades/tareas; luego se introduce ciclo y se evidencia detección)

Modelar dependencias como grafo permite producir un plan ejecutable (orden topológico). Al introducir una dependencia circular, el algoritmo debe señalar explícitamente la imposibilidad (por ejemplo, devolviendo None) para evitar interpretaciones erróneas del plan.

```python
from collections import deque
from typing import Dict, List, Any, Set, Tuple, Optional

Graph = Dict[Any, List[Any]]

def add_node(g: Graph, u: Any) -> None:
    if u not in g:
        g[u] = []

def add_edge(g: Graph, u: Any, v: Any) -> None:
    add_node(g, u)
    add_node(g, v)
    g[u].append(v)

def all_nodes(g: Graph) -> Set[Any]:
    s = set(g.keys())
    for u in g:
        for v in g[u]:
            s.add(v)
    return s

def edges(g: Graph) -> List[Tuple[Any, Any]]:
    out = []
    for u, nbrs in g.items():
        for v in nbrs:
            out.append((u, v))
    return out

def compute_indegree(g: Graph) -> Dict[Any, int]:
    indeg = {u: 0 for u in all_nodes(g)}
    for u in g:
        for v in g[u]:
            indeg[v] += 1
    return indeg
```

Captura 41: Implementación ejemplo 5. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def topo_sort_kahn(g: Graph) -> Optional[List[Any]]:
    nodes = sorted(all_nodes(g))  # ordenado para mayor determinismo en pruebas
    indeg = compute_indegree(g)
    q = deque([u for u in nodes if indeg[u] == 0])
    order: List[Any] = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g.get(u, []):
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    if len(order) != len(nodes):
        return None
    return order

def check_topo(order: List[Any], es: List[Tuple[Any, Any]]) -> bool:
    pos = {node: i for i, node in enumerate(order)}
    for u, v in es:
        if pos[u] >= pos[v]:
            return False
    return True

# Pruebas simples
if __name__ == "__main__":
    # Caso aplicado: “materias” o “tareas” con prerequisitos
    # Interpretación: u -> v significa “u es prerequisito de v”.
    g: Graph = {}
    add_edge(g, "Matemática Discreta", "Algoritmos y Estructuras de Datos")
    add_edge(g, "Programación I", "Algoritmos y Estructuras de Datos")
    add_edge(g, "Algoritmos y Estructuras de Datos", "Machine Learning")
    add_edge(g, "Probabilidad", "Machine Learning")
    add_edge(g, "Machine Learning", "MLOps")

    # Nodo sin salidas (última materia)
    add_node(g, "Ética y Sociedad")
```

Captura 42. Implementación ejemplo 5 (cont). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
order = topo_sort_kahn(g)
print("Orden propuesto (sin ciclo):", order)
assert order is not None
assert check_topo(order, edges(g)) is True

# Interpretación simple: una materia aparece antes que sus dependientes
assert order.index("Programación I") < order.index("Algoritmos y Estructuras de Datos")
assert order.index("Machine Learning") < order.index("MLOps")

# Introducir dependencia circular: MLOps -> Machine Learning (ciclo)
add_edge(g, "MLOps", "Machine Learning")

order2 = topo_sort_kahn(g)
print("Orden propuesto (con ciclo):", order2)
assert order2 is None  # evidencia detección de ciclo por Kahn
```

Captura 43: Implementación ejemplo 5 (cont). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Aristas "prerequisito"	Modelan precedencia: si `u→v`, entonces `u` debe ubicarse antes que `v`.
`topo_sort_kahn`	Produce un orden si el grafo es DAG; devuelve `None` si procesa menos nodos que el total.
`sorted(all_nodes(g))`	Introduce determinismo para pruebas al fijar el orden inicial de nodos.
`check_topo(order, edges)`	Valida formalmente la solución: cada arista respeta la precedencia en la lista.
Inserción de ciclo `MLOps→Machine Learning`	Crea dependencia circular; evita que el algoritmo complete (
`assert order2 is None`	Evidencia la condición de ciclo por tamaño de orden resultante.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

5.6 Complejidad y criterios de elección (DFS vs Kahn) en Python

Tanto el orden topológico por DFS como por Kahn operan en tiempo lineal respecto del tamaño del grafo, $O(V + E)$, cuando se usa lista de adyacencia (Cormen et al., 2009). En términos de elección práctica: DFS integra naturalmente la detección de ciclos con estados y la construcción del orden por postorden; Kahn ofrece una interpretación directa basada en indegree y permite identificar "fuentes" disponibles en cada paso, lo cual se alinea con planificación incremental (Kahn, 1962). En Python, una consideración práctica es evitar recursión profunda en DFS si V es grande (límite de recursión), y preferir una versión iterativa o Kahn con deque si se espera gran profundidad o cadenas largas de dependencias (Sedgewick & Wayne, 2011).

Énfasis

Definiciones clave

- **Complejidad $O(V + E)$:** Cota temporal lineal en número de vértices y aristas al recorrer cada vértice y cada arista una cantidad constante de veces.
- **Consideración de recursión (Python):** Restricción práctica por profundidad máxima de llamadas al usar DFS recursivo.

› **Cola deque:** Estructura FIFO con operaciones $O(1)$ amortizado para encolar y desencolar, adecuada para Kahn.

Comparativa de Algoritmos de Ordenamiento Topológico

Característica	DFS (Postorden Inverso)	Algoritmo de Kahn (Indegree)
Idea Central	Visitar fondo, apilar al volver, luego invertir lista.	Identificar nodos sin dependencias (grado 0) y 'eliminar' progresivamente.
Estructura Usada	Pila de llamadas (Stack/Recursión)	Cola (Queue/Deque) + Tabla de Grados
Detección de Ciclo	Encontrar arista hacia nodo 'en proceso' (Back Edge).	Si al final conteo visitados < total (sobran nodos con grado > 0).
Consideración Python	Riesgo de `RecursionError` en grafos profundos (`sys.setrecursionlimit`).	Iterativo. Eficiente usando `collections.deque`.
Salida Posible	Múltiples órdenes válidos (depende del orden de vecinos).	Múltiples órdenes válidos (depende del orden en la cola).

Tabla 1 creada usando herramientas de IA a partir de información proporcionada por la autora.

En conjunto, DFS se destaca por su simplicidad en la implementación (pocas líneas de código), aunque en Python presenta la desventaja técnica del límite de profundidad de recursión si el grafo es muy lineal y largo (similar a una lista enlazada gigante). En este marco, Kahn resulta más robusto en contextos industriales en Python porque es iterativo y maneja de forma natural el concepto de "tareas listas para ejecutarse" (nodos de grado 0), lo que suele facilitar la visualización de dependencias.

5.6.1 Ejemplo 6 (comparación operativa mínima: mismo grafo, ambos enfoques y coherencia de verificación)

Comparar salidas de DFS y Kahn sobre el mismo DAG permite verificar que pueden producir órdenes distintos, pero igualmente válidos. La validez se establece mediante check_topo, no por igualdad exacta de listas.

```python
from collections import deque
from typing import Dict, List, Any, Set, Tuple, Optional

Graph = Dict[Any, List[Any]]

def all_nodes(g: Graph) -> Set[Any]:
    s = set(g.keys())
    for u in g:
        for v in g[u]:
            s.add(v)
    return s

def edges(g: Graph) -> List[Tuple[Any, Any]]:
    out = []
    for u, nbrs in g.items():
        for v in nbrs:
            out.append((u, v))
    return out

def check_topo(order: List[Any], es: List[Tuple[Any, Any]]) -> bool:
    pos = {node: i for i, node in enumerate(order)}
    return all(pos[u] < pos[v] for u, v in es)
```

Captura 44: Desarrollo Ejemplo 6. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def topo_sort_dfs(g: Graph) -> Optional[List[Any]]:
    state = {u: 0 for u in all_nodes(g)}
    order: List[Any] = []
    has_cycle = False

    def dfs(u: Any) -> None:
        nonlocal has_cycle
        state[u] = 1
        for v in g.get(u, []):
            if state[v] == 1:
                has_cycle = True
                return
            if state[v] == 0:
                dfs(v)
                if has_cycle:
                    return
        state[u] = 2
        order.append(u)

    for u in list(state.keys()):
        if state[u] == 0:
            dfs(u)
            if has_cycle:
                return None

    order.reverse()
    return order
```

Captura 45: Desarrollo Ejemplo 6 (cont). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def topo_sort_kahn(g: Graph) -> Optional[List[Any]]:
    nodes = sorted(all_nodes(g))
    indeg = {u: 0 for u in nodes}
    for u in g:
        for v in g[u]:
            indeg[v] += 1
    q = deque([u for u in nodes if indeg[u] == 0])
    order: List[Any] = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g.get(u, []):
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    if len(order) != len(nodes):
        return None
    return order
```

Captura 46: Desarrollo Ejemplo 6 (cont). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
if __name__ == "__main__":
    g: Graph = {"A": ["B", "C"], "B": ["D"], "C": ["D"], "D": [], "E": []}  # incluye nodo aislado E

    o1 = topo_sort_dfs(g)
    o2 = topo_sort_kahn(g)

    print("DFS:", o1)
    print("Kahn:", o2)

    assert o1 is not None and o2 is not None
    assert check_topo(o1, edges(g)) is True
    assert check_topo(o2, edges(g)) is True
```

Captura 47: Desarrollo Ejemplo 6 (cont). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`check_topo`	Define la corrección por restricción: toda arista debe respetar posiciones.
`topo_sort_dfs`	Produce un orden por postorden y lo invierte; devuelve `None` si hay ciclo.
`topo_sort_kahn`	Construye indegree y cola de fuentes; devuelve `None` si no procesa todos.
Grafo con nodo aislado `E`	Evidencia que un orden topológico debe incluir nodos sin dependencias.
Asserts finales	Verifican que ambos órdenes son válidos aunque puedan diferir en el listado.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 4: Grafos y ordenamiento topológico.**

6. Árboles de expansión mínima: algoritmos de Kruskal y Prim y su implementación

6.1 Árbol de expansión mínima (MST): definición, condiciones y representaciones

Un árbol de expansión mínima (Minimum Spanning Tree, MST) se define, para un grafo no dirigido, ponderado y conexo $G = (V, E)$, como un subconjunto de aristas $T \subseteq E$ que (i) conecta todos los vértices, (ii) no contiene ciclos y (iii) minimiza la suma de pesos $\sum_{(u,v)\in T} w(u, v)$. Si el grafo no es conexo, no existe un MST único que conecte todo el grafo; en ese caso, el objetivo se generaliza a un bosque de expansión mínima (Minimum Spanning Forest, MSF), que es la unión de MSTs por componente conexa. Estas nociones se integran de forma estándar en el estudio de algoritmos de grafos y aparecen como contenidos mínimos y algorítmicos asociados (Cormen et al., 2009). En el programa provisto para AED-II se explicita el tratamiento de "árbol abarcador mínimo" y los algoritmos de Kruskal y Prim dentro de los contenidos de grafos.

En la práctica, la implementación suele alternar entre la **lista de aristas**, conveniente para Kruskal porque requiere ordenar *E*, y la **lista de adyacencia**, más adecuada para Prim porque explora fronteras locales por vértice. Estas elecciones influyen tanto en la complejidad como en el costo constante por operación (Sedgewick & Wayne, 2011; Cormen et al., 2009).

Énfasis

Definiciones clave

- **MST (Árbol de expansión mínima):** Subconjunto de aristas de un grafo no dirigido, ponderado y conexo que conecta todos los vértices sin ciclos y con suma total de pesos mínima.

- **MSF (Bosque de expansión mínima):** Conjunto de árboles (uno por componente conexa) que minimiza la suma total de pesos en un grafo no dirigido ponderado no conexo.
- **Lista de aristas:** Representación del grafo como colección $[(u, v, w), \dots]$ que almacena pares de vértices y el peso asociado.

Lista de adyacencia: Representación del grafo como diccionario $u \mapsto [(v, w), \dots]$ que enumera vecinos y pesos por vértice.

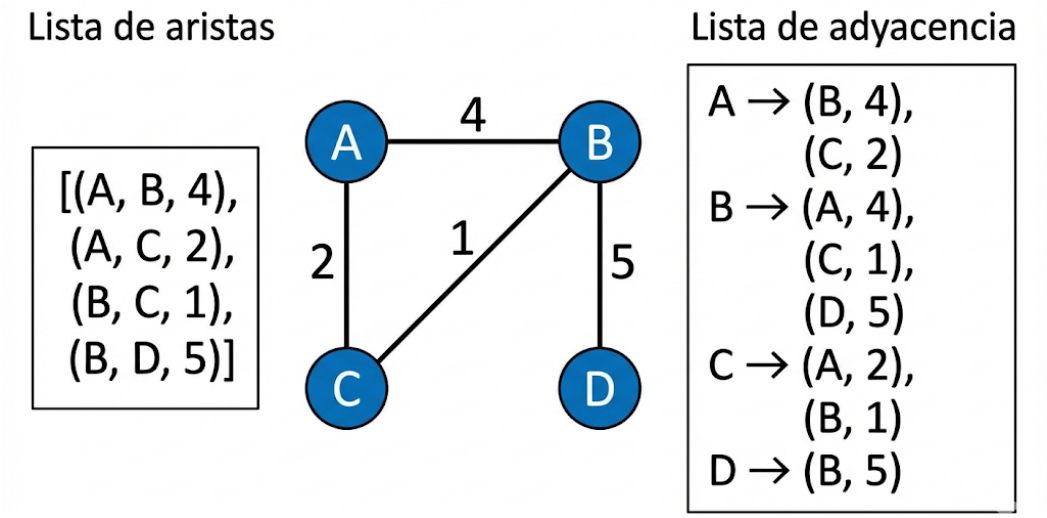


Ilustración 23: Lista de aristas vs. lista de adyacencia (grafo ponderado). La imagen contrasta dos representaciones equivalentes: una lista de aristas $(u, v, peso)$ y una lista de adyacencia que, para cada nodo, enumera sus vecinos con sus pesos, mostrando el mismo grafo con distinta estructura de datos. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.1.1 Ejemplo 1 (representación no dirigida ponderada: lista de aristas + lista de adyacencia)

Kruskal consume naturalmente una lista de aristas ordenable por peso, mientras que Prim consume eficientemente una lista de adyacencia para expandir desde un vértice por frontera local. Implementar ambas vistas del mismo grafo permite reutilizar datos y comparar algoritmos de MST con una base común (Cormen et al., 2009).

```python
# Representación de grafo no dirigido ponderado:
# - edges: lista de aristas [(u, v, w), ...]
# - adj: lista de adyacencia {u: [(v, w), ...], ...}

def add_edge_edges(edges, u, v, w):
    """Agrega una arista (u, v, w) a la lista de aristas (no dirigido)."""
    edges.append((u, v, w))

def build_adj_from_edges(edges):
    """Construye lista de adyacencia a partir de una lista de aristas no dirigida."""
    adj = {}
    for u, v, w in edges:
        adj.setdefault(u, []).append((v, w))
        adj.setdefault(v, []).append((u, w))
    return adj

def list_nodes_from_edges(edges):
    """Lista nodos presentes en la lista de aristas."""
    nodes = set()
    for u, v, _ in edges:
        nodes.add(u)
        nodes.add(v)
    return sorted(nodes)

# Pruebas simples
edges = []
add_edge_edges(edges, "A", "B", 4)
add_edge_edges(edges, "A", "C", 2)
add_edge_edges(edges, "B", "C", 1)
add_edge_edges(edges, "B", "D", 5)

adj = build_adj_from_edges(edges)
nodes = list_nodes_from_edges(edges)

print("Edges:", edges)
print("Nodes:", nodes)
print("Adj[A]:", sorted(adj["A"]))
assert set(nodes) == {"A", "B", "C", "D"}
assert ("B", 4) in adj["A"] and ("A", 4) in adj["B"]
```

Captura 48: Representación de grafo no dirigido ponderado (lista de aristas → lista de adyacencia). El código define funciones para (1) agregar aristas con peso a una lista (u, v, w), (2) construir la lista de adyacencia no dirigida a partir de esa lista de aristas (registrando vecinos en ambos sentidos) y (3) obtener el conjunto de nodos presentes; incluye pruebas con print y assert para validar simetría y consistencia de la estructura. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`add_edge_edges`	Agrega una tupla (u, v, w) a una lista, útil para ordenar aristas por peso en Kruskal.
`build_adj_from_edges`	Genera un diccionario de adyacencia duplicando cada arista (u, v) como $(u \to v)$ y $(v \to u)$ por tratarse de grafo no dirigido.
`list_nodes_from_edges`	Recupera el conjunto de vértices vistos en las aristas, necesario para validar condiciones del MST (por ejemplo, $V - 1$ aristas).
"Pruebas simples" (`print` , `assert`)	Verifica: (i) que se detectan todos los nodos y (ii) que la adyacencia refleja simetría de pesos en no dirigido.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

6.2 Propiedades y criterios: corte, ciclo, enfoque greedy; unicidad y empates; validación

Los algoritmos de Kruskal y Prim son greedy porque construyen una solución incremental seleccionando aristas "seguras" según una regla local. Su corrección se sustenta en propiedades clásicas:

- **Propiedad del corte (cut property):** dada una partición $(S, V \setminus S)$, una arista de peso mínimo que cruza el corte es "segura", es decir, existe un MST que la contiene.
- **Propiedad del ciclo (cycle property):** en cualquier ciclo, una arista de peso estrictamente máximo no pertenece a ningún MST.

Estas propiedades permiten justificar la selección incremental sin perder optimalidad global (Cormen et al., 2009; Kleinberg & Tardos, 2005).

La unicidad del MST no está garantizada: si existen empates de peso, pueden existir múltiples MST con el mismo costo total pero con conjuntos de aristas distintos. La validación de un resultado como MST/MSF exige, al menos, dos condiciones: (i) acíclico y (ii) conecta cada componente (en MST: conecta todo el grafo si es conexo). Para un grafo conexo con $|V| = n$, un MST debe tener exactamente $n - 1$ aristas (Cormen et al., 2009).

Definiciones clave

- **Corte:** Partición del conjunto de vértices en S y $V \setminus S$; una arista "cruza" el corte si conecta un vértice en S con otro en $V \setminus S$.
- **Propiedad del corte:** Si una arista tiene peso mínimo entre todas las que cruzan un corte, entonces es segura para algún MST.
- **Propiedad del ciclo:** En un ciclo, una arista con peso estrictamente máximo no puede pertenecer a un MST.
- **Arista segura:** Arista que puede agregarse a una construcción parcial y aún extenderse a un MST óptimo.

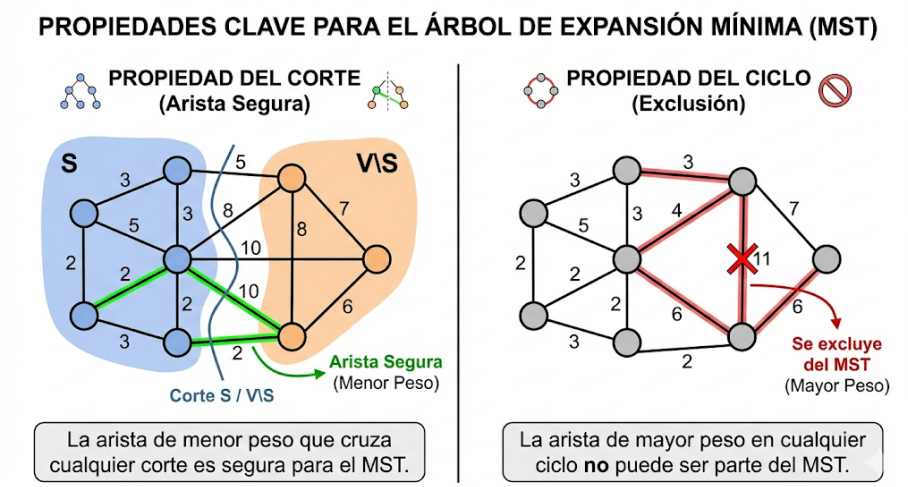


Ilustración 24: Propiedades clave del MST: corte y ciclo. Se ilustran dos criterios de corrección para árboles de expansión mínima: (a) propiedad del corte, donde la arista de menor peso que cruza un corte es "segura" para el MST; (b) propiedad del ciclo, donde la arista de mayor peso en un ciclo no puede pertenecer al MST (se excluye). Imagen creada usando herramientas IA con información proporcionada por la autora.

6.2.1 Ejemplo 2 (validación básica de "árbol" y conectividad)

En un grafo conexo, un MST debe ser acíclico y tener exactamente $|V|-1$ aristas; además, debe conectar todos los vértices. Verificar estas condiciones permite detectar salidas inconsistentes antes de comparar costos o propiedades más avanzadas (Cormen et al., 2009).

```python
from collections import deque

def build_adj_from_edges_undirected(edges):
    adj = {}
    for u, v, w in edges:
        adj.setdefault(u, []).append((v, w))
        adj.setdefault(v, []).append((u, w))
    return adj

def nodes_from_edges(edges):
    s = set()
    for u, v, _ in edges:
        s.add(u); s.add(v)
    return s

def is_connected_from(adj, start, nodes):
    """BFS para verificar conectividad desde 'start' sobre el subgrafo dado por adj."""
    q = deque([start])
    visited = {start}
    while q:
        u = q.popleft()
        for v, _ in adj.get(u, []):
            if v not in visited:
                visited.add(v)
                q.append(v)
    return visited == set(nodes)
```

Captura 49: Definiciones en Python para convertir una lista de aristas no dirigidas y ponderadas en una lista de adyacencia y obtener el conjunto de nodos; además, se implementa una verificación de conectividad mediante BFS (usando deque) desde un nodo inicial para comprobar si el subgrafo alcanza a todos los nodos. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def validate_tree_like(nodes, mst_edges):
    """
    Validación mínima de MST en caso conexo:
    - debe tener |V|-1 aristas
    - debe conectar todos los nodos
    """
    if not nodes:
        return True
    if len(mst_edges) != len(nodes) - 1:
        return False
    adj = build_adj_from_edges_undirected(mst_edges)
    start = next(iter(nodes))
    return is_connected_from(adj, start, nodes)

# Pruebas simples
nodes = {"A", "B", "C", "D"}
mst_ok = [("A", "C", 2), ("B", "C", 1), ("B", "D", 5)]
mst_bad_cycle = [("A", "C", 2), ("B", "C", 1), ("A", "B", 4), ("B", "D", 5)]  # 4 aristas

print("Valida MST ok:", validate_tree_like(nodes, mst_ok))
print("Valida MST con ciclo:", validate_tree_like(nodes, mst_bad_cycle))

assert validate_tree_like(nodes, mst_ok) is True
assert validate_tree_like(nodes, mst_bad_cycle) is False
```

Captura 50: Función validate_tree_like que valida condiciones mínimas de un MST en caso conexo: exige exactamente $|V|-1$ aristas y conectividad total (vía BFS sobre la adyacencia reconstruida); incluye casos de prueba con assert para un conjunto válido y otro inválido por ciclo (exceso de aristas). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`is_connected_from`	Ejecuta BFS sobre el subgrafo para decidir si todos los nodos quedan alcanzables desde un inicio.
`validate_tree_like`	Implementa dos verificaciones necesarias: conteo de aristas (
Caso `mst_ok`	Cumple (
Caso `mst_bad_cycle`	Tiene 4 aristas con 4 nodos, viola (
Asserts	Automatizan la verificación esperada sin interacción manual.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

6.3 Kruskal: ordenamiento de aristas y Union-Find (conjuntos disjuntos)

Kruskal construye un MST seleccionando aristas en orden no decreciente de peso e incorporándolas si no forman ciclo con las ya elegidas. El control de ciclos se resuelve eficientemente con una estructura Union-Find / Disjoint Set Union (DSU): cada conjunto representa un componente del bosque parcial; una arista $(u\ v)$ se agrega si $find(u)! = find(v)$ y luego se unifican componentes con $union(u, v)$. El uso de compresión de caminos y unión por rango/tamaño reduce el costo amortizado de operaciones de DSU (Tarjan, 1975; Cormen et al., 2009).

En términos de complejidad, Kruskal suele expresarse como $O(E\log\ E)$ por el ordenamiento de aristas, más un término casi lineal por operaciones DSU (Cormen et al., 2009).

Énfasis

Definiciones clave

- **Kruskal:** Algoritmo greedy que ordena aristas por peso y agrega una arista si conecta dos componentes distintas, evitando ciclos, hasta obtener $|V|-1$ aristas (si el grafo es conexo).
- **Union-Find / DSU:** Estructura que mantiene particiones disjuntas del conjunto de vértices soportando operaciones find (representante) y union (unión de conjuntos).
- **Compresión de caminos:** Optimización de find que hace que los nodos visitados apunten directamente al representante, reduciendo futuras búsquedas.
- **Unión por rango/tamaño:** Regla de union que adjunta el árbol "más pequeño" bajo el "más grande" para limitar la altura.

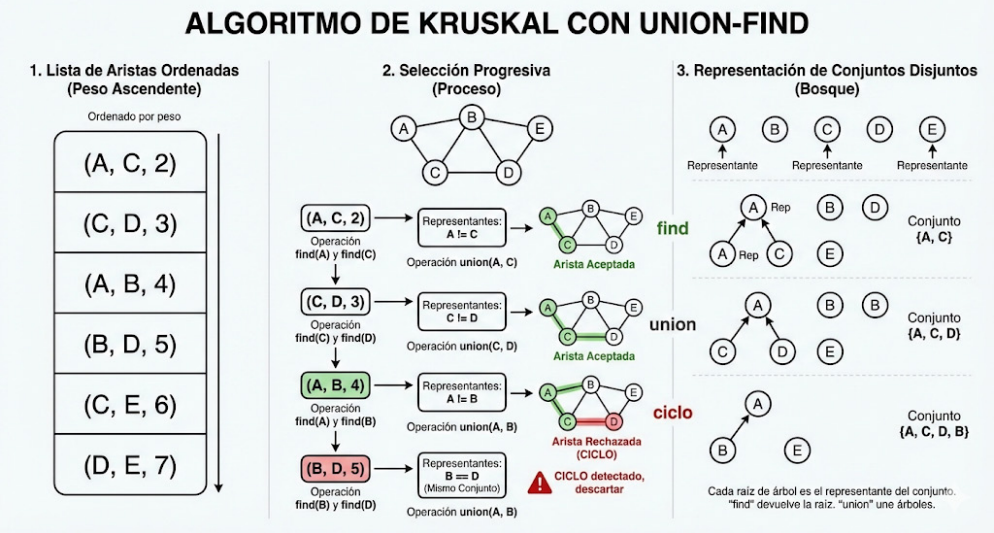


Ilustración 25: Kruskal con Union-Find (conjuntos disjuntos). Se presenta el flujo del algoritmo: ordenar aristas por peso ascendente, aceptar aristas si no forman ciclo, y detectar ciclos mediante find/union en la estructura Union-Find, visualizando cómo evolucionan los conjuntos. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.3.1 Ejemplo 3 (implementación de Union-Find con compresión + unión por tamaño)

Union-Find permite detectar ciclos en Kruskal: si dos vértices ya comparten representante, una arista entre ellos cerraría un ciclo. Compresión de caminos y unión por tamaño/rango reducen el costo amortizado de las operaciones repetidas durante el recorrido de aristas (Cormen et al., 2009; Tarjan, 1975).

```python
class DisjointSetUnion:
    def __init__(self, elements):
        self.parent = {x: x for x in elements}
        self.size = {x: 1 for x in elements}

    def find(self, x):
        # Compresión de caminos
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        # Unión por tamaño
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        return True

# Pruebas simples
items = ["A", "B", "C", "D"]
dsu = DisjointSetUnion(items)

assert dsu.find("A") == "A"
assert dsu.union("A", "B") is True
assert dsu.find("A") == dsu.find("B")
assert dsu.union("B", "C") is True
assert dsu.find("A") == dsu.find("C")
assert dsu.union("A", "C") is False  # ya estaban conectados

print("Representantes:", {x: dsu.find(x) for x in items})
print("OK: DSU con compresión y unión por tamaño.")
```

Captura 51: Implementación en Python de la estructura Union-Find (Disjoint Set Union) mediante una clase DisjointSetUnion, incorporando compresión de caminos en find y unión por tamaño en union para optimizar operaciones. Incluye pruebas simples con assert y salida por consola que muestra los representantes finales de cada elemento. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`parent`, `size`	Estructuras internas para representar el bosque de conjuntos disjuntos y su tamaño.
`find` con compresión	Reescribe punteros durante la búsqueda para apuntar al representante, reduciendo altura efectiva.
`union` por tamaño	Une el conjunto más pequeño bajo el mayor para limitar crecimiento de altura.
Asserts	Validan conectividad (mismo representante) y el caso en el que `union` debe fallar por pertenecer a la misma componente.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

6.3.2 Ejemplo 4 (Kruskal completo: devuelve aristas del MST y costo total)

Kruskal ordena aristas por peso y agrega aquellas que unen componentes distintas según DSU. En un grafo conexo, el proceso termina al obtener $|V|-1$ aristas; la salida se valida con conteo y ausencia de ciclos implícita por DSU (Cormen et al., 2009).

```python
def kruskal_mst(nodes, edges):
    """
    nodes: iterable de vértices
    edges: lista [(u, v, w), ...] para grafo no dirigido
    retorna (mst_edges, total_cost)
    """
    dsu = DisjointSetUnion(nodes)
    mst = []
    total = 0

    edges_sorted = sorted(edges, key=lambda x: x[2])
    for u, v, w in edges_sorted:
        if dsu.union(u, v):  # True si no forma ciclo
            mst.append((u, v, w))
            total += w
            if len(mst) == len(nodes) - 1:
                break

    return mst, total

def nodes_from_edges(edges):
    s = set()
    for u, v, _ in edges:
        s.add(u); s.add(v)
    return s
```

Captura 52: (inicio del código). Implementación de kruskal_mst(nodes, edges): inicializa una estructura Union-Find (DSU), ordena las aristas por peso y agrega solo las que conectan componentes distintas para construir el Árbol de Expansión Mínima (MST) sin formar ciclos; además define nodes_from_edges para derivar el conjunto de nodos desde la lista de aristas. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
edges = [
    ("A", "B", 4),
    ("A", "C", 2),
    ("B", "C", 1),
    ("B", "D", 5),
    ("C", "D", 8),
    ("C", "E", 10),
    ("D", "E", 2),
]
nodes = nodes_from_edges(edges)

mst_edges, cost = kruskal_mst(nodes, edges)
print("MST (Kruskal):", mst_edges)
print("Costo total:", cost)

# Validaciones mínimas en grafo conexo:
assert len(mst_edges) == len(nodes) - 1
# Verificación de "sin ciclos" por construcción: cada arista añadida une componentes distintas.
# Verificación adicional: conectividad usando BFS sobre el MST.
from collections import deque

def build_adj_undirected(edge_list):
    adj = {}
    for u, v, w in edge_list:
        adj.setdefault(u, []).append((v, w))
        adj.setdefault(v, []).append((u, w))
    return adj
```

Captura 53: (continuación). Bloque de pruebas del algoritmo de Kruskal: declara un conjunto de aristas ponderadas, calcula el MST y su costo total, y valida condiciones mínimas (cantidad de aristas |V|-1); incorpora una verificación adicional de conectividad mediante BFS sobre el MST construido. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def is_connected(nodes, edge_list):
    if not nodes:
        return True
    adj = build_adj_undirected(edge_list)
    start = next(iter(nodes))
    q = deque([start])
    vis = {start}
    while q:
        u = q.popleft()
        for v, _ in adj.get(u, []):
            if v not in vis:
                vis.add(v)
                q.append(v)
    return vis == set(nodes)

assert is_connected(nodes, mst_edges) is True
```

Captura 54: (cierre del código). Implementación de la validación de conectividad: construye una lista de adyacencia del MST (build_adj_undirected) y ejecuta BFS (is_connected) para confirmar que el árbol conecta todos los nodos; finaliza con una aserción que garantiza la conectividad del MST obtenido. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`edges_sorted`	Ordena aristas por peso, condición esencial del criterio greedy de Kruskal.
`dsu.union(u,v)`	Decide si la arista es segura: se agrega solo si une componentes distintas (evita ciclos).
Corte por `len(mst)==	V
Validación con BFS	Comprueba conectividad del subgrafo resultante, complementando la garantía de aciclicidad que aporta DSU.
Asserts	Garantizan que el resultado cumple estructura de MST en caso conexo.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

6.4 Prim: crecimiento desde un nodo, heapq; relación con Dijkstra

Prim construye un MST expandiendo desde un vértice inicial: mantiene un conjunto S de vértices ya conectados y, en cada paso, agrega la arista de menor peso que conecta S con $V \setminus S$. Esta regla se justifica por la propiedad del corte: el borde mínimo que cruza el corte $(S, V \setminus S)$ es seguro (Cormen et al., 2009).

En implementación eficiente se utiliza una cola de prioridad (heap) para mantener aristas frontera candidatas; heapq en Python permite extraer el mínimo en $O(\log E)$. Prim se relaciona con Dijkstra por el patrón de expansión incremental y el uso de heap, pero difieren conceptualmente: Dijkstra minimiza distancias desde un origen (suma acumulada), mientras Prim minimiza el costo del árbol (selección por arista mínima que conecta un nuevo vértice) (Cormen et al., 2009; Sedgewick & Wayne, 2011).

Énfasis

Definiciones clave

- **Prim:** Algoritmo greedy que construye un MST expandiendo un conjunto de vértices conectados y agregando la arista mínima que incorpora un nuevo vértice.

- **Frontera:** Conjunto de aristas que conectan vértices ya visitados con vértices aún no visitados.
- **Cola de prioridad (min-heap):** Estructura que permite extraer repetidamente el elemento de menor clave (peso) con costo logarítmico.

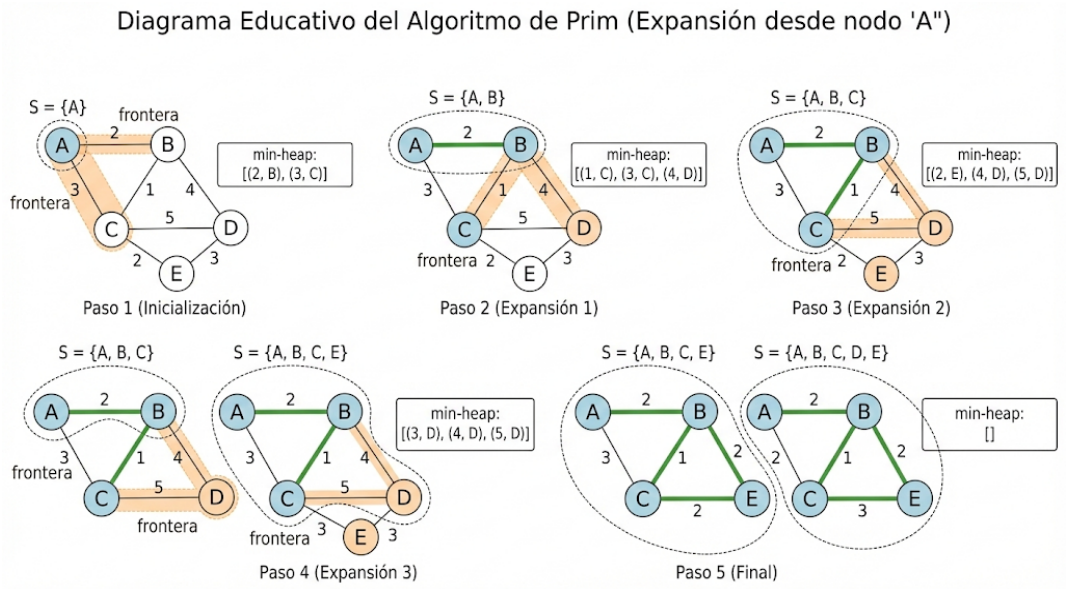


Ilustración 26: Prim paso a paso con frontera (min-heap). La secuencia muestra el crecimiento del conjunto S desde un nodo inicial, manteniendo una frontera de aristas candidatas en un min-heap; en cada paso se elige la arista mínima que conecta S con un nodo no incluido, hasta completar el MST. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.4.1 Ejemplo 5 (Prim con heapq: devuelve aristas del MST y costo total)

La selección repetida de la arista de menor peso en la frontera puede implementarse con un min-heap. Al marcar vértices visitados, se evita incorporar aristas que conecten dos vértices ya pertenecientes al árbol, preservando aciclicidad (Cormen et al., 2009).

```python
import heapq

def build_adj(edges):
    adj = {}
    for u, v, w in edges:
        adj.setdefault(u, []).append((v, w))
        adj.setdefault(v, []).append((u, w))
    return adj

def prim_mst(edges, start):
    """
    edges: lista [(u, v, w), ...] no dirigida
    start: vértice inicial
    retorna (mst_edges, total_cost, visited)
    """
    adj = build_adj(edges)
    visited = set([start])
    mst = []
    total = 0

    heap = []
    for v, w in adj.get(start, []):
        heapq.heappush(heap, (w, start, v))

    while heap:
        w, u, v = heapq.heappop(heap)
        if v in visited:
            continue
        # arista segura para incorporar v
        visited.add(v)
        mst.append((u, v, w))
        total += w

        for nxt, w2 in adj.get(v, []):
            if nxt not in visited:
                heapq.heappush(heap, (w2, v, nxt))

    return mst, total, visited
```

Captura 55: Implementación del algoritmo de Prim para obtener un árbol de expansión mínima (MST) en un grafo no dirigido y ponderado: construye la lista de adyacencia, inicializa el conjunto de visitados y utiliza un min-heap (heapq) como frontera para seleccionar iterativamente la arista de menor peso que conecta un nodo nuevo. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
edges = [
    ("A", "B", 4),
    ("A", "C", 2),
    ("B", "C", 1),
    ("B", "D", 5),
    ("C", "D", 8),
    ("C", "E", 10),
    ("D", "E", 2),
]
mst_edges, cost, visited = prim_mst(edges, "A")

print("MST (Prim):", mst_edges)
print("Costo total:", cost)
print("Visitados:", visited)

# Si el grafo es conexo, visitados debe incluir todos los nodos
nodes = {u for e in edges for u in (e[0], e[1])}
assert visited == nodes
assert len(mst_edges) == len(nodes) - 1
```

Captura 56: Continuación del ejemplo de Prim: define un conjunto de aristas de prueba, ejecuta prim_mst(edges, "A"), muestra las aristas seleccionadas y el costo total, y valida mediante assert que el recorrido visita todos los nodos (si el grafo es conexo) y que el MST contiene $|V|-1$ aristas. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`build_adj`	Construye lista de adyacencia para iterar vecinos en $O(\deg(v))$.
`visited`	Mantiene el conjunto S (vértices ya integrados al árbol parcial).
`heap`	Guarda aristas frontera como tuplas `(peso, origen, destino)` para extraer mínimo.
`if v in visited: continue`	Evita agregar aristas que no incorporan un vértice nuevo; previene ciclos.
Inserción de vecinos de `v`	Actualiza frontera al incorporar un nuevo vértice, manteniendo candidatas relevantes.
Asserts	Verifican cobertura total (conexo) y cantidad de aristas (

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

6.5 Comparación práctica Kruskal vs Prim y caso no conexo (MSF)

Kruskal y Prim obtienen un MST (si el grafo es conexo) pero difieren en estrategia y estructuras: Kruskal se apoya en ordenamiento global de aristas y DSU, mientras Prim se apoya en expansión local y cola de prioridad sobre adyacencias. En grafos dispersos, ambas aproximaciones pueden ser competitivas; la elección práctica suele depender de la representación disponible y del costo de ordenamiento versus operaciones de heap (Cormen et al., 2009; Sedgewick & Wayne, 2011).

En grafos con empates, el conjunto de aristas del MST puede diferir entre algoritmos (o incluso entre ejecuciones), pero el costo total mínimo debe coincidir. Para grafos no conexos, el resultado correcto es un MSF: Kruskal puede producirlo si no se fuerza $|V|-1$ y se recorre todo E; Prim puede adaptarse reiniciando desde cada vértice no visitado (Kleinberg & Tardos, 2005; Cormen et al., 2009).

Definiciones clave

- **Comparación algorítmica:** Análisis conjunto de criterios de correctitud, costo computacional y estructura de datos requerida por distintas soluciones a un mismo problema.
- **Empate de pesos:** Situación en la que múltiples aristas comparten el mismo peso, habilitando múltiples MST con igual costo.
- **Reinicio por componente:** Estrategia para obtener MSF ejecutando Prim desde cada vértice no visitado.

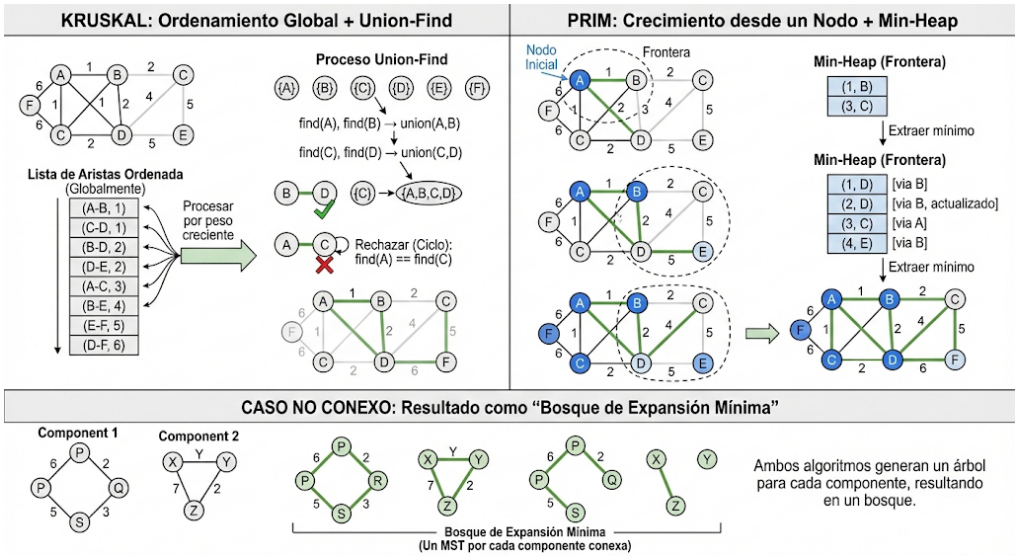


Ilustración 27: Comparativa Kruskal vs. Prim y caso no conexo. Se comparan los enfoques: Kruskal (selección global de aristas ordenadas + Union-Find) y Prim (crecimiento local desde un nodo + min-heap). Además, se explicita que en grafos no conexos ambos producen un bosque de expansión mínima (un árbol por componente conexa). Imagen creada usando herramientas IA con información proporcionada por la autora.

6.5.1 Ejemplo 6 (comparación Kruskal vs Prim sobre el mismo grafo)

En un grafo conexo, Kruskal y Prim deben producir el mismo costo total mínimo, aunque puedan diferir en aristas seleccionadas por empates. Comparar ambos permite validar implementación y comprender el efecto de la estrategia greedy (Cormen et al., 2009).

```python
import heapq
from collections import deque

class DisjointSetUnion:
    def __init__(self, elements):
        self.parent = {x: x for x in elements}
        self.size = {x: 1 for x in elements}

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        return True

```

```python
def nodes_from_edges(edges):
    s = set()
    for u, v, _ in edges:
        s.add(u); s.add(v)
    return s

def kruskal_mst(nodes, edges):
    dsu = DisjointSetUnion(nodes)
    mst = []
    total = 0
    for u, v, w in sorted(edges, key=lambda x: x[2]):
        if dsu.union(u, v):
            mst.append((u, v, w))
            total += w
            if len(mst) == len(nodes) - 1:
                break
    return mst, total
```

Captura 57: Fragmento inicial del script donde se importan estructuras de apoyo (heapq, deque) y se define Union-Find (DisjointSetUnion) con compresión de caminos y unión por tamaño, además de funciones auxiliares para obtener nodos desde la lista de aristas e implementar Kruskal (selección de aristas por peso evitando ciclos). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def build_adj(edges):
    adj = {}
    for u, v, w in edges:
        adj.setdefault(u, []).append((v, w))
        adj.setdefault(v, []).append((u, w))
    return adj

def prim_mst(edges, start):
    adj = build_adj(edges)
    visited = set([start])
    heap = []
    mst = []
    total = 0

    for v, w in adj.get(start, []):
        heapq.heappush(heap, (w, start, v))

    while heap:
        w, u, v = heapq.heappop(heap)
        if v in visited:
            continue
        visited.add(v)
        mst.append((u, v, w))
        total += w
        for nxt, w2 in adj.get(v, []):
            if nxt not in visited:
                heapq.heappush(heap, (w2, v, nxt))

    return mst, total, visited
```

Captura 58: Continuación del mismo script: construcción de la lista de adyacencia a partir de aristas ponderadas y definición de Prim usando una cola de prioridad (heap) para seleccionar la arista mínima de la frontera, expandiendo el conjunto de visitados y acumulando el costo del MST. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples
edges = [
    ("A", "B", 4),
    ("A", "C", 2),
    ("B", "C", 1),
    ("B", "D", 5),
    ("C", "D", 8),
    ("C", "E", 10),
    ("D", "E", 2),
]
nodes = nodes_from_edges(edges)

k_mst, k_cost = kruskal_mst(nodes, edges)
p_mst, p_cost, p_vis = prim_mst(edges, "A")

print("Kruskal MST:", k_mst, "Costo:", k_cost)
print("Prim MST:", p_mst, "Costo:", p_cost)

assert k_cost == p_cost
assert len(k_mst) == len(nodes) - 1
assert p_vis == nodes
```

Captura 59: Sección final de pruebas del script: se define un conjunto de aristas de ejemplo, se calculan los MST con Kruskal y con Prim, se imprimen resultados y se validan condiciones mínimas (mismo costo total, cantidad de aristas |V|-1 y cobertura de todos los nodos visitados) mediante assert. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`kruskal_mst`	Ordena aristas y usa DSU para aceptar solo las que conectan componentes distintas.
`prim_mst`	Expande desde `start` con heap de aristas frontera, incorporando un vértice nuevo por iteración válida.
Comparación `k_cost == p_cost`	En grafo conexo, ambos algoritmos deben coincidir en costo total mínimo; diferencias posibles solo en aristas por empates.
Asserts estructurales	Verifican condiciones necesarias: (

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

6.5.2 Ejemplo 7 (caso no conexo: bosque de expansión mínima con Prim por componentes)

En un grafo no conexo, un MST global no existe; el resultado correcto es un MSF. Una adaptación práctica de Prim consiste en reiniciar desde cada nodo no visitado para cubrir todas las componentes, acumulando aristas por componente (Kleinberg & Tardos, 2005; Cormen et al., 2009).

```python
import heapq

def build_adj(edges):
    adj = {}
    for u, v, w in edges:
        adj.setdefault(u, []).append((v, w))
        adj.setdefault(v, []).append((u, w))
    return adj

def nodes_from_edges(edges):
    s = set()
    for u, v, _ in edges:
        s.add(u); s.add(v)
    return sorted(s)

def prim_forest(edges):
    """
    Devuelve un bosque de expansión mínima (MSF) para grafo no dirigido ponderado no conexo.
    Retorna (forest_edges, total_cost, components_count)
    """
    adj = build_adj(edges)
    nodes = set(nodes_from_edges(edges))
    visited = set()
    forest = []
    total = 0
    components = 0

    for start in sorted(nodes):
        if start in visited:
            continue
        components += 1
        visited.add(start)
        heap = []
        for v, w in adj.get(start, []):
            heapq.heappush(heap, (w, start, v))
```

Captura 60: (parte 1/2). Definición de utilidades para grafo no dirigido ponderado (build_adj, nodes_from_edges) y función prim_forest, que extiende Prim para construir un bosque de expansión mínima (MSF) en grafos no conexos, iniciando el proceso en cada componente no visitada y administrando la frontera con heapq. Código desarrollado en Online IDE Pro. Código fuente de la autora.

```python
        while heap:
            w, u, v = heapq.heappop(heap)
            if v in visited:
                continue
            visited.add(v)
            forest.append((u, v, w))
            total += w
            for nxt, w2 in adj.get(v, []):
                if nxt not in visited:
                    heapq.heappush(heap, (w2, v, nxt))

    return forest, total, components, nodes

# Pruebas simples (grafo con dos componentes)
edges = [
    # Componente 1: A-B-C
    ("A", "B", 3),
    ("B", "C", 1),
    ("A", "C", 2),
    # Componente 2: D-E
    ("D", "E", 4),
]
forest, cost, comps, nodes = prim_forest(edges)

print("MSF (Prim por componentes):", forest)
print("Costo total:", cost)
print("Componentes:", comps)

# Validación: en MSF, número total de aristas = |V| - (#componentes)
assert len(forest) == len(nodes) - comps
# Validación simple adicional: todos los nodos aparecen visitados implícitamente si hay aristas incidentes
# (en este ejemplo, todos aparecen en 'nodes' por construcción).
```

Captura 61: (parte 2/2). Continuación de prim_forest: extracción del mínimo desde el heap, incorporación de aristas seguras al bosque, actualización de vecinos y retorno de (forest, total, components, nodes); incluye caso de prueba con dos componentes y validaciones de cantidad de aristas y consistencia del resultado. Código desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Iteración por `start`	Detecta nuevas componentes: si `start` no fue visitado, inicia un Prim local.
`components += 1`	Cuenta cuántas componentes se cubren; es necesario para la fórmula (
Heap por componente	Mantiene frontera local, evitando mezclar componentes no conectadas.
Acumulación en `forest`	Junta las aristas seleccionadas en todas las componentes: resultado es un MSF.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/157d8523-a4bb-40f7-9523-7d83b653cc47

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 5: MST Interactive Learning Lab.**

7. Algoritmos de flujo máximo y resolución del problema del viajante (TSP) mediante búsqueda exhaustiva y heurísticas.

7.1 Flujo en redes

Una red de flujo se modela como un dígrafo $G = (V, E)$ con una función de capacidad $c: E \to \mathbb{R}_{\geq 0}$, una fuente s y un sumidero t. Un flujo es una función $f: E \to \mathbb{R}$ que satisface:

- restricción de capacidad $0 \leq f(u, v) \leq c(u, v)$, y
- conservación de flujo en todo vértice intermedio $v \in V \setminus \{s, t\}$: $\sum_{(u,v)\in E} f(u, v) = \sum_{(v,w)\in E} f(v, w)$. El valor del flujo se define como $|f| = \sum_{(s,v)\in E} f(s, v)$, equivalente a la entrada neta a t.

Estos conceptos formalizan problemas como ruteo con capacidades y asignación de recursos en redes, con aplicaciones en transporte, telecomunicaciones y planificación. (Cormen et al., 2022).

Definiciones clave

- **Red de flujo:** Dígrafo con capacidades en sus aristas y dos nodos distinguibles: fuente s y sumidero t.
- **Capacidad $c(u, v)$:** Cota superior no negativa para el flujo permitido sobre la arista dirigida (u, v).

- **Flujo $f(u, v)$:** Asignación numérica a aristas que respeta capacidad y conservación en nodos intermedios.
- **Conservación de flujo:** Igualdad entre el flujo entrante y saliente de cada nodo distinto de s y t.
- **Valor del flujo $|f|$:** Flujo neto que sale de s (o entra a t), maximizado en el problema de flujo máximo.

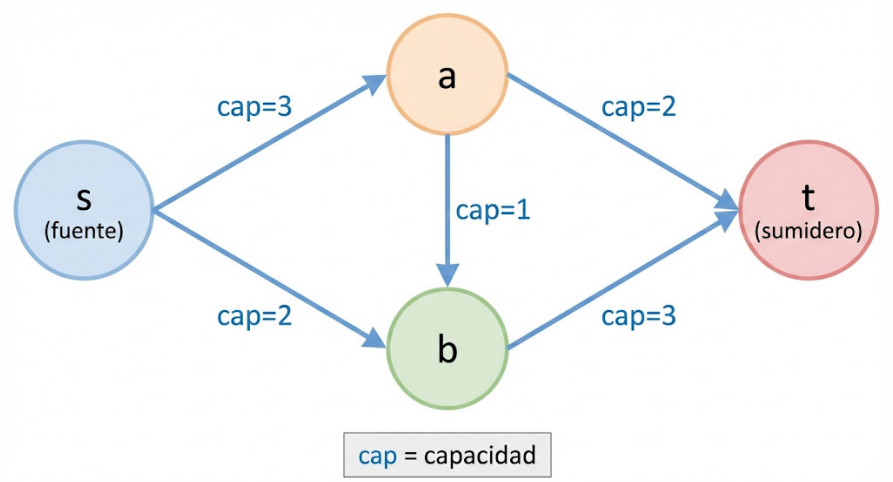


Ilustración 28: Red de flujo con capacidades (s–a–b–t). Se representa un problema de flujo máximo en un grafo dirigido, donde cada arista tiene una capacidad (cap) que limita cuánto flujo puede circular desde la fuente (s) hacia el sumidero (t). Imagen creada usando herramientas IA con información proporcionada por la autora.

7.1.1 Ejemplo 1–representación + alta de aristas con inversas

Desde la teoría de Ford–Fulkerson y Edmonds–Karp, es conveniente representar una red como capacidades y asegurar la existencia de aristas inversas con capacidad inicial 0 para habilitar "devolver" flujo en la red residual. El siguiente script implementa add_edge(u,-v,c) con diccionario de diccionarios y una visualización básica.

ejemplo1.py

```python
# Red de flujo con diccionario de diccionarios: cap[u][v] = capacidad
# Incluye inicialización de aristas inversas con capacidad 0.

def add_edge(cap, u, v, c):
    if c < 0:
        raise ValueError("La capacidad debe ser no negativa.")
    cap.setdefault(u, {})
    cap.setdefault(v, {})
    cap[u][v] = cap[u].get(v, 0) + c  # permite paralelas acumulando
    cap[v].setdefault(u, 0)           # arista inversa para residual

def pretty_print(cap):
    edges = []
    for u in cap:
        for v, c in cap[u].items():
            if c != 0:
                edges.append((u, v, c))
    edges.sort()
    for u, v, c in edges:
        print(f"{u} -> {v} (cap={c})")

# Pruebas simples
cap = {}
add_edge(cap, "s", "a", 3)
add_edge(cap, "s", "b", 2)
add_edge(cap, "a", "b", 1)
add_edge(cap, "a", "t", 2)
add_edge(cap, "b", "t", 3)

# Verificación: existen inversas con capacidad 0 cuando no se definieron explícitamente
assert cap["a"].get("s", 0) == 0
assert cap["b"].get("s", 0) == 0
assert cap["b"].get("a", 0) == 0
assert cap["t"].get("a", 0) == 0
assert cap["t"].get("b", 0) == 0

print("Aristas con capacidad (no nulas):")
pretty_print(cap)
```

Captura 62: Implementación de la representación de una red de flujo mediante un diccionario de diccionarios (cap[u][v]), donde cada entrada modela la capacidad de la arista dirigida u → v. La función add_edge agrega capacidades (permitiendo aristas paralelas por acumulación) e inicializa la arista inversa con capacidad 0 para habilitar el manejo de la red residual. El bloque de pruebas crea un grafo simple, verifica la existencia de inversas en cero mediante assert y lista las aristas con capacidad no nula con pretty_print. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`cap`	Estructura `dict[u][v]` que almacena capacidades dirigidas; facilita acceso O(1) promedio.
`add_edge(cap,u,v,c)`	Agrega capacidad a (u, v) y garantiza (v, u) con capacidad 0, necesaria para residual.
Acumulación `cap[u].get(v,0)+c`	Permite modelar aristas paralelas sumando capacidades, coherente con la formulación de red.
`pretty_print`	Recorre y lista aristas con capacidad no nula como verificación visual básica.
`assert` de inversas	Comprueba que la estructura contiene aristas inversas inicializadas, precondición típica para Edmonds–Karp.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/d40a6ae7-fb62-4edb-a882-47ff3b2e226e

7.2 Red residual y caminos aumentantes: capacidad residual, aristas inversas y criterio de optimalidad

La red residual G_f asociada a un flujo f codifica cuánto flujo adicional puede enviarse por cada arista sin violar capacidades, y cuánto flujo puede "revertirse" usando aristas inversas. La capacidad residual se define como $c_f(u,v) = c(u,v) - f(u,v)$ para aristas originales, y $c_f(v,u) = f(u,v)$ para las inversas, permitiendo deshacer asignaciones previas. Un camino aumentante es un camino de s a t en G_f donde todas las capacidades residuales son positivas; aumentar el flujo por ese camino en el valor del cuello de botella (mínima capacidad residual del camino) incrementa $|f|$. El criterio clásico establece que si no existe camino aumentante, el flujo es máximo (Ford & Fulkerson, 1956; Cormen et al., 2022).

Énfasis

Definiciones clave

- **Red residual G_f:** Dígrafo derivado de G cuyas capacidades representan posibilidades de aumentar o revertir flujo respecto de f.
- **Capacidad residual $c_f(u,v)$:** Capacidad disponible para enviar flujo adicional por (u,v) en presencia del flujo actual.
- **Arista inversa residual:** Arista (v,u) con capacidad residual igual al flujo enviado por (u,v), que habilita "retirar" flujo.
- **Camino aumentante:** Camino $s \rightsquigarrow t$ en la red residual con capacidad residual positiva en todas sus aristas.
- **Cuello de botella:** Mínimo c_f a lo largo de un camino aumentante; determina el máximo aumento posible en una iteración.

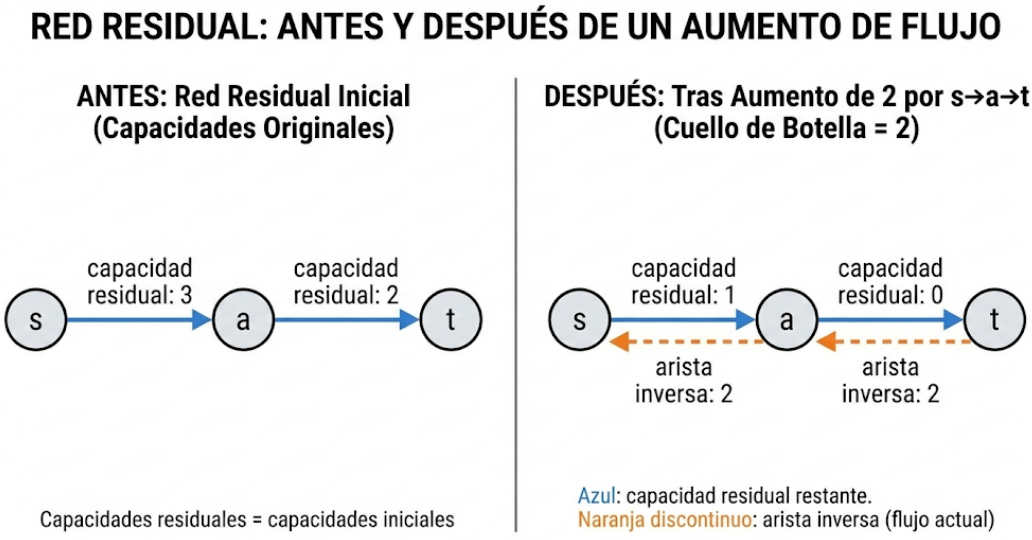


Ilustración 29: Red residual: antes y después de un aumento de flujo. Se ilustra cómo cambia la capacidad residual tras enviar flujo por un camino aumentante: disminuye la capacidad disponible en las aristas usadas y aparecen/crecen aristas inversas que permiten "deshacer" parte del flujo en iteraciones posteriores. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.2.1 Ejemplo 2–construcción explícita de residual + actualización con asserts

Teóricamente, la red residual se obtiene combinando capacidades restantes y aristas inversas. En práctica, es común mantener una matriz/estructura de capacidades residuales y actualizarla con aumentos, asegurando no negatividad en cada paso.

```python
from copy import deepcopy

def add_edge(cap, u, v, c):
    cap.setdefault(u, {})
    cap.setdefault(v, {})
    cap[u][v] = cap[u].get(v, 0) + c
    cap[v].setdefault(u, 0)

def build_residual_from_cap(cap):
    # En esta representación, el residual inicial coincide con las capacidades,
    # incluyendo inversas en 0 ya creadas por add_edge.
    return deepcopy(cap)

def augment(residual, path, bottleneck):
    # Actualiza residual: resta en sentido directo y suma en inverso
    for u, v in path:
        residual[u][v] -= bottleneck
        residual[v][u] += bottleneck
        # Pruebas simples: no negatividad
        assert residual[u][v] >= 0, "Capacidad residual negativa: actualización inválida."

# Red ejemplo
cap = {}
add_edge(cap, "s", "a", 3)
add_edge(cap, "s", "b", 2)
add_edge(cap, "a", "b", 1)
add_edge(cap, "a", "t", 2)
add_edge(cap, "b", "t", 3)

res = build_residual_from_cap(cap)

# Camino aumentante manual: s->a->t con cuello de botella 2
path = [("s", "a"), ("a", "t")]
bottleneck = min(res["s"]["a"], res["a"]["t"])
assert bottleneck == 2

augment(res, path, bottleneck)
```

Captura 63: Código que define la estructura de capacidades cap (diccionario de diccionarios), construye la red residual inicial como copia (deepcopy) y define la operación augment para actualizar el residual al aumentar flujo sobre un camino (resta capacidad en aristas directas y suma en aristas inversas), incorporando una verificación de no negatividad mediante assert. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Camino aumentante manual: s->a->t con cuello de botella 2
path = [("s", "a"), ("a", "t")]
bottleneck = min(res["s"]["a"], res["a"]["t"])
assert bottleneck == 2

augment(res, path, bottleneck)

# Pruebas simples: residual directo disminuye, inverso aumenta
assert res["s"]["a"] == 1
assert res["a"]["s"] == 2
assert res["a"]["t"] == 0
assert res["t"]["a"] == 2

print("Residual tras aumentar por s-a-t con +2:")
print("res[s][a] =", res["s"]["a"], "| res[a][s] =", res["a"]["s"])
print("res[a][t] =", res["a"]["t"], "| res[t][a] =", res["t"]["a"])
```

Captura 64: (continuación). Código que aplica un camino aumentante manual $s \to a \to t$ calculando el cuello de botella con min, ejecuta la actualización residual con augment, y valida el resultado con assert (disminución del residual directo y aumento del inverso), además de imprimir capacidades residuales finales para evidenciar el cambio. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`build_residual_from_cap`	Crea una copia profunda de capacidades como residual inicial; las inversas ya existen con 0.
`augment(residual, path, bottleneck)`	Implementa la regla residual: restar capacidad residual directa y sumar en la inversa.
`assert residual[u][v] >= 0`	Verifica el invariante de no negatividad de capacidades residuales tras cada actualización.
Cálculo de `bottleneck`	Aplica la definición teórica: mínimo residual del camino aumentante elegido.
`assert` de cambios directos/inversos	Evidencia el rol de las aristas inversas: habilitan revertir parte del flujo en iteraciones futuras.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/d40a6ae7-fb62-4edb-a882-47ff3b2e226e

7.3 Edmonds–Karp: Ford–Fulkerson con BFS; procedimiento, corrección a alto nivel y complejidad $O(V \cdot E^2)$

Edmonds–Karp es una especialización de Ford–Fulkerson que selecciona en cada iteración un camino aumentante de menor cantidad de aristas (en la red residual) mediante BFS. Esta elección acota el número de aumentos: las distancias BFS desde s a cada vértice no disminuyen y el número de veces que una arista puede volverse crítica (saturarse en un camino más corto) es limitado, lo que lleva a la complejidad $O(V \cdot E^2)$. La corrección se apoya en:

- cada aumento respeta capacidades y conservación (por construcción residual), y
- el algoritmo termina sin caminos aumentantes, condición suficiente de optimalidad. (Edmonds & Karp, 1972; Cormen et al., 2022).

Énfasis

Definiciones clave

- **Edmonds–Karp:** Implementación de Ford–Fulkerson que elige caminos aumentantes por BFS (mínimo número de aristas).
- **BFS en residual:** Búsqueda en amplitud restringida a aristas con capacidad residual > 0, usada para obtener predecesores y reconstruir el camino.
- **Aumentación:** Operación iterativa: hallar camino aumentante, calcular cuello de botella y actualizar la red residual.
- **Complejidad $O(V \cdot E^2)$:** Cota clásica de Edmonds–Karp derivada del número de aumentaciones y del costo de cada BFS.

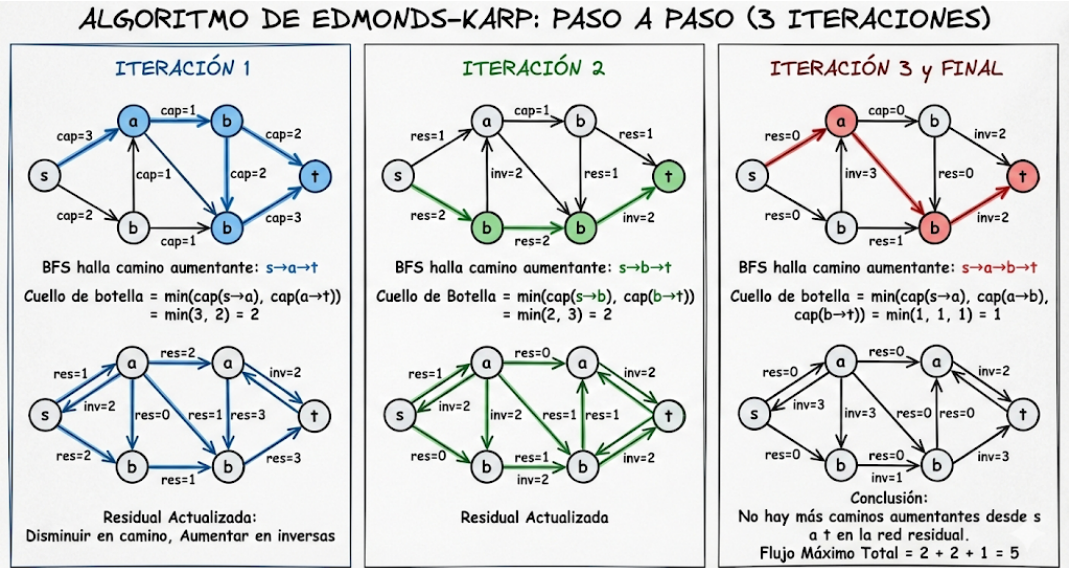


Ilustración 30: Edmonds–Karp paso a paso (iteraciones). Se muestra el algoritmo Edmonds–Karp encontrando caminos aumentantes con BFS; en cada iteración se calcula el cuello de botella del camino y se actualiza la red residual, hasta que ya no existen caminos aumentantes desde s hasta t. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.3.1 Ejemplo 3–BFS que devuelve padres + Edmonds–Karp completo con asserts de conservación

Desde la teoría, BFS debe considerar solo aristas con residual > 0 y devolver predecesores para reconstruir el camino aumentante. El siguiente código implementa Edmonds–Karp y verifica: (i) valor máximo esperado para una red pequeña, y (ii) conservación de flujo en nodos intermedios.

```python
from collections import deque
from copy import deepcopy

def add_edge(cap, u, v, c):
    cap.setdefault(u, {})
    cap.setdefault(v, {})
    cap[u][v] = cap[u].get(v, 0) + c
    cap[v].setdefault(u, 0)

def bfs_parents(residual, s, t):
    """BFS en residual: devuelve dict padres {v: u} para reconstruir camino s->t, o None si no existe."""
    q = deque([s])
    parent = {s: None}
    while q:
        u = q.popleft()
        for v, rc in residual[u].items():
            if rc > 0 and v not in parent:
                parent[v] = u
                if v == t:
                    return parent
                q.append(v)
    return None

def reconstruct_path(parent, s, t):
    path = []
    v = t
    while v != s:
        u = parent[v]
        path.append((u, v))
        v = u
    path.reverse()
    return path
```

Captura 65: Fragmento inicial del código del algoritmo de flujo máximo (Edmonds–Karp): define la estructura de capacidades con diccionario de diccionarios, la búsqueda BFS sobre la red residual para hallar un camino aumentante (padres) y la función para reconstruir dicho camino desde t hasta s. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def edmonds_karp(cap, s, t):
    residual = deepcopy(cap)
    max_flow = 0

    while True:
        parent = bfs_parents(residual, s, t)
        if parent is None:
            break
        path = reconstruct_path(parent, s, t)
        bottleneck = min(residual[u][v] for (u, v) in path)

        # Aumentación en residual (con asserts de no negatividad)
        for u, v in path:
            residual[u][v] -= bottleneck
            residual[v][u] += bottleneck
            assert residual[u][v] >= 0

        max_flow += bottleneck

    # Derivar flujo final sobre aristas originales: f(u,v)=cap(u,v)-residual(u,v) si cap(u,v)>0
    flow = {}
    for u in cap:
        for v, c in cap[u].items():
            if c > 0:
                flow.setdefault(u, {})
                flow[u][v] = c - residual[u][v]
                assert 0 <= flow[u][v] <= c

    return max_flow, flow, residual
```

Captura 66: Continuación del código de Edmonds–Karp: implementa el ciclo principal que repite la búsqueda de caminos aumentantes, calcula el cuello de botella, actualiza la red residual (resta en aristas directas y suma en inversas) y deriva el flujo final sobre las aristas originales verificando cotas ($0 \leq$ flujo $\leq$ capacidad). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def check_flow_conservation(flow, s, t):
    # Construye balances en nodos: in - out
    in_flow = {}
    out_flow = {}
    for u in flow:
        for v, fuv in flow[u].items():
            out_flow[u] = out_flow.get(u, 0) + fuv
            in_flow[v] = in_flow.get(v, 0) + fuv

    nodes = set(in_flow) | set(out_flow)
    for v in nodes:
        if v in (s, t):
            continue
        assert abs(in_flow.get(v, 0) - out_flow.get(v, 0)) < 1e-9, f"Conservación violada en {v}"

# Red pequeña conocida
cap = {}
add_edge(cap, "s", "a", 3)
add_edge(cap, "s", "b", 2)
add_edge(cap, "a", "b", 1)
add_edge(cap, "a", "t", 2)
add_edge(cap, "b", "t", 3)

max_flow, flow, residual = edmonds_karp(cap, "s", "t")

# Pruebas simples (resultado esperado)
assert max_flow == 5, f"Se esperaba max_flow=5 y se obtuvo {max_flow}"
check_flow_conservation(flow, "s", "t")

print("max_flow =", max_flow)
print("Flujo en aristas originales:")
for u in sorted(flow):
    for v in sorted(flow[u]):
        print(f"  {u}->{v}: {flow[u][v]} / cap {cap[u][v]}")
```

Captura 67: Cierre del ejemplo: incorpora una verificación de conservación de flujo por nodo (balance entrada–salida), define una red pequeña de prueba, ejecuta Edmonds–Karp, valida el valor esperado de flujo máximo mediante asserts e imprime el flujo resultante por arista. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`bfs_parents(residual,s,t)`	BFS recorre solo aristas con residual `>0`, registra `parent` y finaliza cuando alcanza `t`.
`reconstruct_path`	Reconstruye el camino $s \rightsquigarrow t$ usando predecesores, en orden directo para aumentar.
`bottleneck = min(...)`	Implementa el cuello de botella como mínimo residual sobre el camino aumentante.
Actualización residual	Resta `bottleneck` en aristas del camino y suma en inversas; `assert` evita residual negativo.
Derivación `flow[u][v] = cap - residual`	Recupera el flujo enviado por cada arista original a partir del residual final.
`check_flow_conservation`	Valida conservación en nodos intermedios comparando flujo entrante vs saliente.
`assert max_flow == 5`	Verifica el valor máximo esperado en una instancia pequeña y controlada.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/d40a6ae7-fb62-4edb-a882-47ff3b2e226e

7.4 Corte mínimo y relación Max-Flow Min-Cut: interpretación del resultado y verificación práctica

Un corte (S, T) particiona V en dos conjuntos disjuntos con $s \in S$ y $t \in T$. Su capacidad es $\sum_{u \in S, v \in T} c(u, v)$. El teorema Max-Flow Min-Cut establece que el valor del flujo máximo es igual a la capacidad del corte mínimo. Operacionalmente, al terminar Ford–Fulkerson/Edmonds–Karp sin caminos aumentantes, el conjunto S de vértices alcanzables desde s en la red residual define un corte cuya capacidad coincide con $|f|$. Esto provee un mecanismo de verificación: identificar S en el residual final y sumar capacidades de aristas originales de S a T. (Ford & Fulkerson, 1956; Cormen et al., 2022).

Énfasis

Definiciones clave

- **Corte (S, T):** Partición de vértices con $s \in S$ y $t \in T$.
- **Capacidad de corte:** Suma de capacidades de aristas dirigidas que cruzan de S hacia T.
- **Corte mínimo:** Corte con capacidad mínima entre todas las particiones válidas (S, T).
- **Teorema Max-Flow Min-Cut:** Igualdad entre valor del flujo máximo y capacidad del corte mínimo en una red.

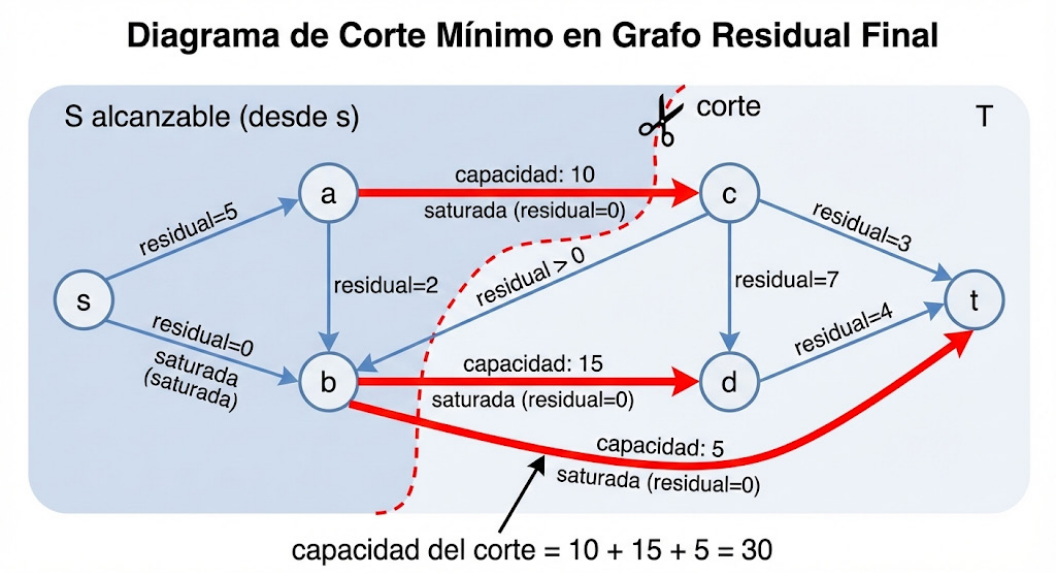


Ilustración 31 Corte mínimo en el grafo residual final. Se identifica el corte s–t separando el conjunto alcanzable desde s en la red residual; las aristas que cruzan el corte quedan saturadas (residual 0) y la suma de sus capacidades coincide con el valor del flujo máximo (teorema max-flow/min-cut). Imagen creada usando herramientas IA con información proporcionada por la autora.

7.4.1 Ejemplo 4–extraer conjunto alcanzable y verificar capacidad de corte = max_flow

Teóricamente, al finalizar Edmonds–Karp, S son los alcanzables desde s por aristas residuales positivas. La capacidad del corte se computa con capacidades originales c en aristas $S \to T$ (típicamente saturadas en residual). El siguiente código implementa esa verificación con assert.

```python
from collections import deque
from copy import deepcopy

def add_edge(cap, u, v, c):
    cap.setdefault(u, {})
    cap.setdefault(v, {})
    cap[u][v] = cap[u].get(v, 0) + c
    cap[v].setdefault(u, 0)

def bfs_parents(residual, s, t):
    q = deque([s])
    parent = {s: None}
    while q:
        u = q.popleft()
        for v, rc in residual[u].items():
            if rc > 0 and v not in parent:
                parent[v] = u
                if v == t:
                    return parent
                q.append(v)
    return None
```

Captura 68: Fragmento inicial del código (Python) desarrollado en Online IDE Pro: define la estructura de capacidades mediante add_edge y realiza una BFS sobre la red residual con bfs_parents para obtener el diccionario de padres que permite reconstruir un camino aumentante desde s hasta t. Código fuente de la autora.

```python
def reconstruct_path(parent, s, t):
    path = []
    v = t
    while v != s:
        u = parent[v]
        path.append((u, v))
        v = u
    path.reverse()
    return path

def edmonds_karp(cap, s, t):
    residual = deepcopy(cap)
    max_flow = 0
    while True:
        parent = bfs_parents(residual, s, t)
        if parent is None:
            break
        path = reconstruct_path(parent, s, t)
        bottleneck = min(residual[u][v] for (u, v) in path)
        for u, v in path:
            residual[u][v] -= bottleneck
            residual[v][u] += bottleneck
            assert residual[u][v] >= 0
        max_flow += bottleneck
    return max_flow, residual
```

Captura 69: Continuación del mismo código en Online IDE Pro: incluye reconstruct_path para reconstruir el camino aumentante a partir de los padres y presenta el núcleo de Edmonds–Karp en edmonds_karp, donde se calcula el cuello de botella y se actualizan capacidades residuales (resta en la arista directa y suma en la inversa), acumulando el max_flow. Código fuente de la autora.

```python
def reachable_set(residual, s):
    q = deque([s])
    seen = {s}
    while q:
        u = q.popleft()
        for v, rc in residual[u].items():
            if rc > 0 and v not in seen:
                seen.add(v)
                q.append(v)
    return seen

def cut_capacity(cap, S):
    # Capacidad del corte (S, T=V\S) usando capacidades originales
    total = 0
    for u in S:
        for v, c in cap[u].items():
            if c > 0 and v not in S:
                total += c
    return total

# Instancia
cap = {}
add_edge(cap, "s", "a", 3)
add_edge(cap, "s", "b", 2)
add_edge(cap, "a", "b", 1)
add_edge(cap, "a", "t", 2)
add_edge(cap, "b", "t", 3)

```

Captura 70: Continuación del mismo código en Online IDE Pro: incorpora reachable_set (BFS) para obtener el conjunto S de nodos alcanzables desde s en la residual final y cut_capacity para calcular la capacidad del corte (S, V\S) usando las capacidades originales; además, define la instancia de prueba del grafo de flujo. Código fuente de la autora.

```python
max_flow, residual = edmonds_karp(cap, "s", "t")
S = reachable_set(residual, "s")
cut_cap = cut_capacity(cap, S)

# Pruebas simples: teorema max-flow min-cut en esta instancia
assert max_flow == 5
assert cut_cap == max_flow, f"Se esperaba capacidad de corte {max_flow} y se obtuvo {cut_cap}"

# Listar aristas del corte S->T
edges_cut = []
for u in S:
    for v, c in cap[u].items():
        if c > 0 and v not in S:
            edges_cut.append((u, v, c, residual[u][v]))  # residual[u][v] deberia ser 0 si está saturada

print("max_flow =", max_flow)
print("S alcanzable en residual:", sorted(S))
print("Aristas del corte (S->T): (u,v,cap,residual)")
for e in sorted(edges_cut):
    print(" ", e)
print("capacidad_corte =", cut_cap)
```

Captura 71: Continuación y cierre del mismo código en Online IDE Pro: ejecuta edmonds_karp, obtiene S , calcula cut_cap y valida el teorema max-flow min-cut mediante assert; finalmente lista las aristas que cruzan el corte (de S a VS) e imprime resultados y capacidad del corte. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`reachable_set(residual,s)`	Construye S como alcanzables desde s usando solo aristas residuales positivas.
`cut_capacity(cap,S)`	Suma capacidades originales $c(u,v)$ para aristas que cruzan $S \to T$.
`assert cut_cap == max_flow`	Verifica la igualdad Max-Flow Min-Cut en la instancia de prueba.
`edges_cut` con residual	Reporta aristas del corte e incluye residual final para observar saturación (típicamente residual 0).
Reuso de `edmonds_karp`	Obtiene residual final, requisito para definir el corte a partir de alcanzabilidad.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/d40a6ae7-fb62-4edb-a882-47ff3b2e226e

7.5 TSP: formulación, complejidad combinatoria y resolución por búsqueda exhaustiva y heurísticas (vecino más cercano y 2- opt)

El Travelling Salesman Problem (TSP) pide, dado un conjunto de ciudades y distancias $d(i,j)$, hallar un ciclo hamiltoniano de costo mínimo que visite cada ciudad exactamente una vez y regrese al origen. En su variante simétrica, $d(i,j) = d(j,i)$; en la asimétrica, esa igualdad no se asume.

En el caso métrico, se cumple la desigualdad triangular, lo que habilita garantías de aproximación para ciertos algoritmos; sin esas propiedades, la calidad de heurísticas puede degradarse. Desde complejidad, el TSP (en formulación de optimización) es NP-difícil, y su versión de decisión es NP-completa bajo reducciones clásicas (Garey & Johnson, 1979). Por ello, para n pequeño se usan métodos exactos como fuerza bruta (coste factorial), mientras que para n moderado/grande se emplean heurísticas: vecino más cercano (greedy) y mejoras locales como 2-opt, que intercambia dos aristas para eliminar cruces y reducir costo. (Garey & Johnson, 1979; Lawler et al., 1985; Croes, 1958).

Énfasis

Definiciones clave

- **TSP (problema del viajante):** Problema de optimización que busca el ciclo de costo mínimo que visita todas las ciudades una vez y retorna al inicio.
- **Ciclo hamiltoniano:** Ciclo que visita cada vértice exactamente una vez.
- **TSP simétrico:** Variante donde $d(i,j) = d(j,i)$ para toda pareja de ciudades.
- **TSP asimétrico:** Variante donde las distancias pueden diferir por dirección.
- **Búsqueda exhaustiva:** Enumeración de permutaciones (fijando origen) para hallar el óptimo; costo factorial.

- **Heurística vecino más cercano:** Construcción greedy que elige repetidamente el vecino no visitado más cercano.
- **2-opt:** Búsqueda local que mejora una ruta reemplazando dos aristas por otras dos (swap de segmento) si reduce el costo.

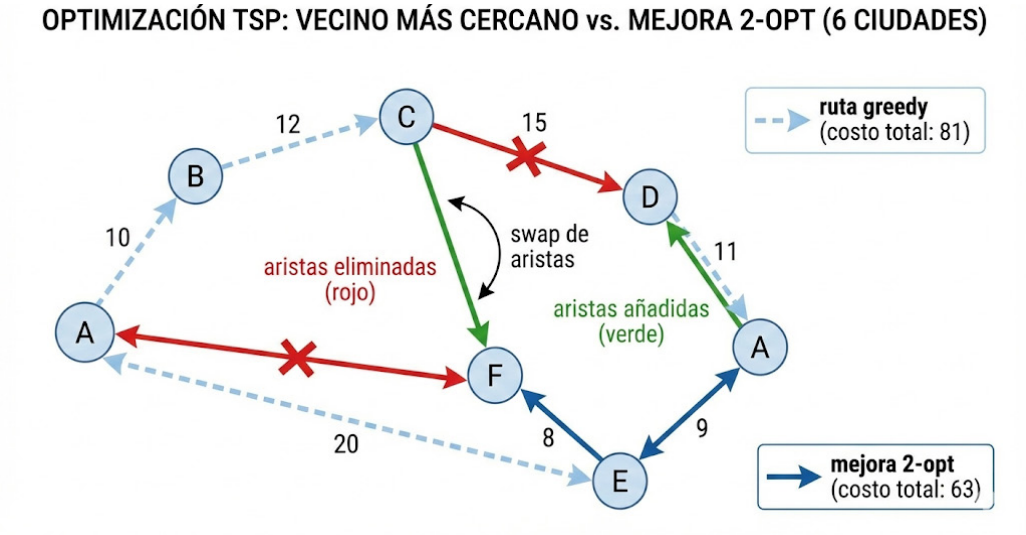


Ilustración 32: TSP: vecino más cercano vs. mejora 2-opt. Se compara una ruta obtenida por heurística greedy (vecino más cercano) con una mejora 2-opt, donde se eliminan dos aristas y se reconectan para reducir cruces y disminuir el costo total del tour. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.5.1 Ejemplo 5–TSP exhaustivo n pequeño con costo esperado

La teoría indica que fijar la ciudad de inicio elimina simetrías por rotación y reduce el espacio de búsqueda a $(n-1)!$. El siguiente ejemplo usa una matriz de distancias y fuerza bruta para $n = 4$, con assert del costo óptimo esperado.

```python
import itertools

# Matriz de distancias (ejemplo asimétrico)
dist = [
    [0, 2, 9, 10],
    [1, 0, 6, 4],
    [15, 7, 0, 8],
    [6, 3, 12, 0]
]

def tour_cost(route, dist):
    c = 0
    for i in range(len(route) - 1):
        c += dist[route[i]][route[i+1]]
    c += dist[route[-1]][route[0]]  # regreso al inicio
    return c

```

```python
def tsp_bruteforce(dist, start=0):
    n = len(dist)
    best_cost = None
    best_route = None
    for perm in itertools.permutations([i for i in range(n) if i != start]):
        route = (start,) + perm
        c = tour_cost(route, dist)
        if best_cost is None or c < best_cost:
            best_cost, best_route = c, route
    return best_route, best_cost

# Pruebas simples
route_opt, cost_opt = tsp_bruteforce(dist, start=0)
print("Óptimo:", route_opt, "costo:", cost_opt)

# Costo esperado para esta instancia (precalculado): 21
assert cost_opt == 21
```

Captura 72: Código (Python) desarrollado en Online IDE Pro que resuelve el Problema del Viajante (TSP) por búsqueda exhaustiva (fuerza bruta) sobre una matriz de distancias asimétrica: calcula el costo de un tour con tour_cost, genera todas las permutaciones posibles de visita con itertools.permutations fijando un nodo de inicio, y selecciona la ruta de costo mínimo. Incluye una prueba simple que imprime la ruta óptima y valida el costo esperado con assert. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dist`	Matriz $n \times n$ donde `dist[i][j]` es el costo de ir de i a j (permite TSP asimétrico).
`tour_cost(route,dist)`	Calcula costo del ciclo: suma tramos consecutivos y agrega el retorno al origen.
`tsp_bruteforce`	Enumera permutaciones de ciudades excepto `start`, formando rutas $(start, ...)$.
Fijar `start`	Reduce simetrías por rotación: evita contar el mismo ciclo desplazado.
Selección de mínimo	Mantiene mejor ruta y costo al comparar en cada permutación.
`assert cost_opt == 21`	Verifica exactitud del algoritmo en una instancia donde el óptimo es conocido.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/d40a6ae7-fb62-4edb-a882-47ff3b2e226e

7.5.2 Ejemplo 6–heurística vecino más cercano + mejora 2-opt + comparativo con umbral

Desde teoría, la fuerza bruta escala factorialmente, por lo que se establece un umbral $n > 10$ para evitar cómputos inviables. Se implementa vecino más cercano como baseline rápido y 2-opt como mejora local; se verifica que 2-opt no empeora el costo.

```python
import itertools

dist = [
    [0, 2, 9, 10],
    [1, 0, 6, 4],
    [15, 7, 0, 8],
    [6, 3, 12, 0]
]

def tour_cost(route, dist):
    c = 0
    for i in range(len(route) - 1):
        c += dist[route[i]][route[i+1]]
    c += dist[route[-1]][route[0]]
    return c

def tsp_bruteforce(dist, start=0, n_limit=10):
    n = len(dist)
    if n > n_limit:
        return None, None
    best_cost = None
    best_route = None
    for perm in itertools.permutations([i for i in range(n) if i != start]):
        route = (start,) + perm
        c = tour_cost(route, dist)
        if best_cost is None or c < best_cost:
            best_cost, best_route = c, route
    return best_route, best_cost
```

Captura 73: Implementación base del TSP (Problema del Viajante) sobre una matriz de distancias: se define la función tour_cost para calcular el costo total de un circuito (incluyendo el retorno al origen) y la función tsp_bruteforce, que evalúa todas las permutaciones posibles a partir de un nodo inicial para obtener la ruta óptima (cuando el tamaño lo permite). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def nearest_neighbor(dist, start=0):
    n = len(dist)
    unvisited = set(range(n))
    unvisited.remove(start)
    route = [start]
    current = start
    while unvisited:
        nxt = min(unvisited, key=lambda j: dist[current][j])
        unvisited.remove(nxt)
        route.append(nxt)
        current = nxt
    return tuple(route), tour_cost(route, dist)

def two_opt(route, dist, max_iter=1000):
    # route es una tupla/lista sin repetir nodos; el ciclo se asume al regresar al inicio
    best = list(route)
    best_cost = tour_cost(best, dist)

    improved = True
    it = 0
    while improved and it < max_iter:
        improved = False
        it += 1
        n = len(best)
        # i y k definen el segmento a invertir (técnica clásica 2-opt)
        for i in range(1, n - 2):
            for k in range(i + 1, n - 1):
                new_route = best[:i] + list(reversed(best[i:k+1])) + best[k+1:]
                new_cost = tour_cost(new_route, dist)
                if new_cost < best_cost:
                    best, best_cost = new_route, new_cost
                    improved = True
                    break
            if improved:
                break

    return tuple(best), best_cost
```

Captura 74: Continuación del ejemplo de TSP: se incorpora la heurística de vecino más cercano (nearest_neighbor), que construye una ruta greedy eligiendo en cada paso el nodo no visitado más próximo, y se define la mejora local 2-opt (two_opt), que intenta reducir el costo invirtiendo segmentos de la ruta (swap de aristas) hasta encontrar una mejora. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Comparación en la misma instancia
opt_route, opt_cost = tsp_bruteforce(dist, start=0, n_limit=10)
greedy_route, greedy_cost = nearest_neighbor(dist, start=0)
twoopt_route, twoopt_cost = two_opt(greedy_route, dist)

print("Greedy (vecino más cercano):", greedy_route, "costo:", greedy_cost)
print("2-opt sobre greedy:", twoopt_route, "costo:", twoopt_cost)

# Pruebas simples: 2-opt no debe empeorar
assert twoopt_cost <= greedy_cost

# Si se calculó el óptimo (n <= umbral), se reporta comparativo
if opt_route is not None:
    print("Óptimo (fuerza bruta):", opt_route, "costo:", opt_cost)
    assert opt_cost <= twoopt_cost  # el óptimo no puede ser peor que una heurística
```

Captura 75: Cierre del ejemplo comparativo: se ejecutan en la misma instancia la búsqueda por fuerza bruta (si n no supera un umbral), la ruta greedy de vecino más cercano y la optimización 2-opt aplicada sobre la ruta greedy; se imprimen resultados y se validan propiedades esperadas mediante assert (por ejemplo, que 2-opt no empeore el costo). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`tsp_bruteforce(..., n_limit=10)`	Implementa control de umbral: si n supera el límite, evita enumeración factorial.
`nearest_neighbor`	Construye una ruta greedy eligiendo el siguiente nodo con distancia mínima desde el actual.
`two_opt`	Aplica intercambios por inversión de segmentos para reducir costo del ciclo (mejora local).
Bucles `i,k` en `two_opt`	Exploran pares de cortes para proponer una ruta alternativa por inversión del subcamino.
Criterio `new_cost < best_cost`	Acepta solo mejoras, garantizando convergencia por descenso de costo (con tope de iteraciones).
`assert twoopt_cost <= greedy_cost`	Evidencia la propiedad mínima esperada: 2-opt no empeora el baseline greedy.
`assert opt_cost <= twoopt_cost`	Si se computa el óptimo, verifica coherencia: ninguna heurística puede superar al óptimo.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/d40a6ae7-fb62-4edb-a882-47ff3b2e226e

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 6: Flujo máximo, redes residuales y optimización de rutas.**

Palabras finales

El Módulo 5 consolidó un conjunto de técnicas y criterios para modelar problemas mediante grafos, seleccionar representaciones coherentes con el patrón de operaciones re-

querido y aplicar algoritmos que permiten explorar, validar propiedades estructurales y optimizar decisiones sobre redes. A través de recorridos (DFS/BFS), se formalizó la noción de alcanzabilidad y componentes; mediante Warshall se estableció el criterio de preprocesamiento para responder consultas múltiples; con Dijkstra se trató la optimización de rutas bajo pesos no negativos; con orden topológico y detección de ciclos se resolvió el análisis de precedencias en DAG; con Kruskal y Prim se abordó la construcción greedy de árboles de expansión mínima; y con Edmonds–Karp y la verificación max-flow/min-cut se trabajó sobre restricciones de capacidad y redes residuales, completando con el contraste entre métodos exactos y heurísticos en TSP. En conjunto, estos contenidos aportan una base operativa para abordar problemas reales donde la estructura relacional es central y donde la eficiencia depende tanto del algoritmo como de la representación.

A continuación, corresponde avanzar con el Módulo 6: Algoritmos avanzados, optimización y aplicaciones específicas, donde se profundizan enfoques y estrategias para resolver problemas de mayor escala, incorporar criterios de optimización más generales y analizar aplicaciones con requerimientos particulares.

¡Nos vemos!

Bibliografía consultada por el docente

- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics, 16*, 87–90. https://www.cs.yale.edu/homes/lans/readings/routing/bellman-routing-1958.pdf
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). The MIT Press. https://mitpress.mit.edu/9780262533058/introduction-to-algorithms/
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research, 6*(6), 791–812.
- De la Cueva Hernández, V. M., López García, J. A., & Martínez Torres, M. A. (2020). *Estructuras de datos y algoritmos fundamentales*. Editorial Digital del Tecnológico de Monterrey.
- Diestel, R. (2017). *Graph Theory* (5th ed.). Springer. http://diestel-graph-theory.com/professional.html
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271. https://link.springer.com/article/10.1007/BF01386390
- Ewing, G. (2009). *PEP 380 — Syntax for Delegating to a Subgenerator*. Python Enhancement Proposals. https://peps.python.org/pep-0380/
- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM, 5*(6), 345. https://dl.acm.org/doi/10.1145/367766.368168

- Ford, L. R., & Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics, 8*, 399–404.
- Frittelli, V. (2021). *Algoritmos y estructura de datos* (2ª ed.). Independently Published
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. Wiley.
- Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558–562.
- Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Addison-Wesley.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley.
- Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (Eds.). (1985). *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley.
- Mit.edu. Recuperado 20 de enero de 2026, de https://dspace.mit.edu/bitstream/handle/1721.1/36847/6-046JFall-2001/NR/rdonlyres/Electrical-Engineering-and-Computer-Science/6-046JIntroduction-to-AlgorithmsFall2001/CB7F315B-5ACF-48C3-84D5-3BEF792B3275/0/lecture10.pdf
- Mcgill.ca. Recuperado 20 de enero de 2026, de https://www.cs.mcgill.ca/~jeromew/teachings/251/F2022/COMP251_Lecture7_F2022.pdf
- Python Software Foundation. (s. f.). *bisect — Array bisection algorithm*. https://docs.python.org/3/library/bisect.html
- Python Software Foundation. (2024). *heapq — Heap queue algorithm*. https://docs.python.org/3/library/heapq.html
- Python Software Foundation. (2025). *sys.setrecursionlimit* (Documentación de Python 3.10, versión en español). https://docs.python.org/es/3.10/library/sys.html
- Python Software Foundation. (n.d.). *collections.deque* (Documentación oficial de Python). https://docs.python.org/3/library/collections.html
- *Red-Black Trees*. (s. f.). Opendatastructures.org. Recuperado 20 de enero de 2026, de https://opendatastructures.org/newhtml/ods/latex/redblack.html
- Schemenauer, N., Peters, T., & Hetland, M. L. (2001). *PEP 255 — Simple Generators*. Python Enhancement Proposals. https://peps.python.org/pep-0255/

- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. https://algs4.cs.princeton.edu/home/
- Silberschatz, A., Korth, H., & Sudarshan, S. (2020). *Database System Concepts*. McGraw-Hill.
- Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. Journal of the ACM, 22(2), 215–225.
- Tarjan, R. E., & van Leeuwen, J. (1984). Worst-case analysis of set union algorithms. Journal of the ACM, 31(2), 245–281.
- *The AVL Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/AVL.html
- *The Red-Black Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/RedBlack.html
- Ullman, J. (2016). *Data Structures and Algorithms*. Stanford Notes.
- Universidad Blas Pascal. (s.f.). *Algoritmos y Estructuras de Datos II* (material de cátedra Licenciatura en Sistemas de Información).
- Warshall, S. (1962). A theorem on boolean matrices. Journal of the ACM, 9(1), 11–12.
- West, D. B. (2001). *Introduction to Graph Theory* (2nd ed.). Prentice Hall.
- Yee, K.-P., & van Rossum, G. (2001). *PEP 234 — Iterators*. Python Enhancement Proposals. https://peps.python.org/pep-0234/

Módulo 5 | Actividades

ACTIVIDADES

Consigna

Las actividades de este módulo son interactivas y autoasistidas. Para realizarlas, es necesario acceder al enlace ***disponible en plataforma*** y seguir las instrucciones indicadas. Ante cualquier duda o inconveniente durante el desarrollo, se solicita comunicarse con el/la tutor/a para recibir acompañamiento y orientación.

Módulo 6

Microobjetivos

- Analizar recurrencias y subproblemas para justificar la elección de programación dinámica en problemas de optimización algorítmica con solapamiento de subproblemas.
- Aplicar tabulación y memoización para resolver y validar instancias típicas en implementaciones de programación dinámica (top-down/bottom-up) con reconstrucción de solución.
- Formular un modelo de programación lineal (variables, objetivo y restricciones) para verificar factibilidad y preparar la resolución en problemas de decisión expresados en forma estándar para métodos tipo Simplex.
- Evaluar la dificultad computacional mediante clases P/NP y noción de verificación para fundamentar el uso de aproximaciones o búsqueda exhaustiva acotada en problemas NP (p. ej., 3-SAT y optimización NP-dura).
- Implementar predicados geométricos básicos (orientación, epsilon, degenerados) para resolver problemas geométricos clásicos en geometría computacional 2D (intersección, convexidad, pertenencia) y su conexión con proximidad

Contenidos

MÓDULO 6: ALGORITMOS AVANZADOS, OPTIMIZACIÓN Y APLICACIONES ESPECÍFICAS

Fuente: Freepik

1. Introducción

¡Bienvenidos/as al último tramo de la materia!

En este *Módulo 6: Algoritmos avanzados, optimización y aplicaciones específicas* se integran técnicas algorítmicas que permiten abordar problemas donde la formulación directa

resulta costosa o insuficiente, y donde el diseño correcto exige modelar, justificar decisiones y analizar complejidad. El recorrido comienza con optimización algorítmica sustentada en subestructura óptima y solapamiento de subproblemas, que fundamentan el uso de programación dinámica (en variantes *top-down* y *bottom-up*) y la reconstrucción de soluciones. Sobre esa base, se analizan criterios de algoritmos voraces y su correctitud (o falla) mediante argumentos estructurales y contraejemplos, conectando ambos enfoques con problemas representativos como mochila, cadena de matrices y caminos mínimos (comparando, por condiciones de aplicabilidad, Dijkstra y Bellman–Ford) (Cormen et al., 2022).

Luego se introduce la optimización matemática mediante programación lineal y programación entera, enfatizando la formulación matricial, la noción de factibilidad, la interpretación geométrica de la región factible y el fundamento de que, si existe un óptimo finito, se alcanza en un vértice. Se presenta el mecanismo operacional del método Simplex (tableau, pivoteo, criterios de parada) y se incorporan modelos aplicados como el problema de transporte, junto con pautas de validación. En paralelo, el módulo avanza hacia la complejidad computacional, formalizando clases P y NP, el rol de los certificados y las reducciones polinómicas para justificar la noción de NP-completitud; desde allí se motivan algoritmos de aproximación con garantía y heurísticas para problemas NP-duros (Cormen et al., 2022; Garey & Johnson, 1979).

En la etapa final se desarrollan aplicaciones específicas con impacto directo en proyectos de IA. Se abordan algoritmos de transformada rápida de Fourier (FFT) y operaciones sobre polinomios, desde la DFT hasta el método de Cooley–Tukey, la IFFT y el uso de FFT en convolución y análisis espectral. También se introduce la geometría computacional, incorporando predicados y estructuras espaciales para tareas como cercos convexos, intersecciones, búsquedas por rango y su vínculo con diagramas de Voronoi. Además, se presentan algoritmos sobre textos y cadenas, incluyendo búsqueda de patrones (fuerza bruta y Knuth–Morris–Pratt), compresión sin pérdida (Huffman y LZW) y una introducción didáctica a la criptografía de clave pública (RSA) basada en aritmética modular, con el objetivo de discutir codificación, almacenamiento eficiente y nociones conceptuales de protección de datos (Knuth et al., 1977; Shannon, 1948; Huffman, 1952; Rivest et al., 1978). A lo largo del módulo se proponen ejemplos ejecutables en Python (orientados a su ejecución en Online IDE Pro) y actividades integradoras que consolidan la selección de técnicas, la verificación de resultados y la explicitación de supuestos en contextos típicos de la Inteligencia Artificial.

¡Comencemos!

2. Optimización algorítmica: subestructura óptima y solapamiento de subproblemas

Muchos problemas en informática pueden plantearse como problemas de optimización: se busca una solución que cumpla ciertas restricciones y que, al mismo tiempo, minimice o maximice una función objetivo. En varios de estos problemas aparecen dos características importantes que permiten aplicar programación dinámica (PD). La primera es la subestructura óptima: una solución óptima puede construirse a partir de soluciones óptimas de subproblemas más pequeños. La segunda es el solapamiento de subproblemas: al resolver el problema de manera recursiva, los mismos subproblemas aparecen

repetidamente. Cuando esto ocurre, la programación dinámica permite evitar cálculos repetidos almacenando los resultados ya obtenidos. Esto puede hacerse mediante memoización (guardando resultados de llamadas recursivas) o tabulación (construyendo una tabla de soluciones de menor a mayor tamaño). Gracias a esta estrategia, se reduce significativamente el costo computacional en comparación con enfoques ingenuos que vuelven a calcular los mismos resultados varias veces (Cormen et al., 2022).

Énfasis

Definiciones clave

- **Problema de optimización:** Problema que requiere seleccionar, entre soluciones factibles, aquella que optimiza (minimiza/maximiza) una función objetivo bajo restricciones.
- **Subestructura óptima:** Propiedad por la cual una solución óptima puede expresarse como combinación de soluciones óptimas de subproblemas bien definidos.
- **Solapamiento de subproblemas:** Propiedad por la cual subproblemas idénticos aparecen múltiples veces durante la descomposición recursiva de un problema.

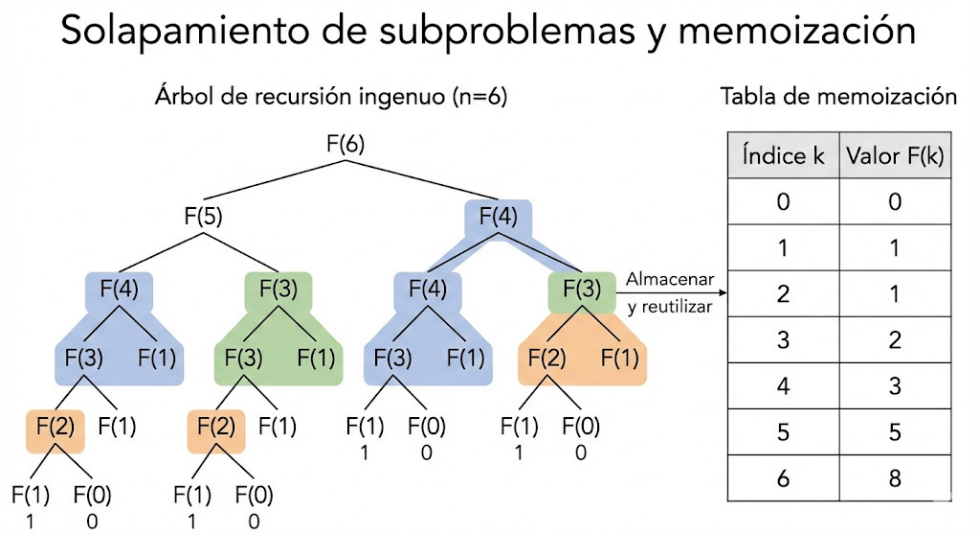


Ilustración 1: Solapamiento de subproblemas y memoización (Fibonacci). La imagen compara el árbol de recursión ingenuo con la construcción de una tabla de memoización, mostrando cómo se evitan cálculos repetidos al almacenar resultados parciales. Se vincula con el contenido al introducir el principio central de la Programación Dinámica: reutilizar soluciones de subproblemas para reducir el costo computacional. Imagen creada usando herramientas IA con información proporcionada por la autora.

2.1.1 Ejemplo 1–motivación: solapamiento en Fibonacci, versión ingenua vs memoización

Fibonacci es un caso mínimo para observar solapamiento: la definición recursiva recomputa subproblemas (por ejemplo, F(n-2)) muchas veces. Contar llamadas permite evidenciar el costo exponencial de la versión ingenua y la reducción a costo lineal con memoización. (Cormen et al., 2022).

```python
# Fibonacci: recursión ingenua vs memoización (con conteo de llamadas)
from functools import lru_cache

def fib_naive(n, counter):
    counter["calls"] += 1
    if n <= 1:
        return n
    return fib_naive(n-1, counter) + fib_naive(n-2, counter)

def fib_memo(n, counter):
    # memoización explícita con dict para contar llamadas reales
    memo = {}
    def f(k):
        counter["calls"] += 1
        if k in memo:
            return memo[k]
        if k <= 1:
            memo[k] = k
        else:
            memo[k] = f(k-1) + f(k-2)
        return memo[k]
    return f(n)

# Pruebas simples
n = 20
c1 = {"calls": 0}
v1 = fib_naive(n, c1)

c2 = {"calls": 0}
v2 = fib_memo(n, c2)

print("fib_naive:", v1, "calls:", c1["calls"])
print("fib_memo :", v2, "calls:", c2["calls"])

assert v1 == 6765
assert v2 == 6765
assert c2["calls"] < c1["calls"]
```

Captura 1: Fibonacci: recursión ingenua vs. memoización (conteo de llamadas). El código implementa dos versiones del cálculo de Fibonacci: una recursiva ingenua (fib_naive) y otra con memoización explícita mediante un diccionario interno (fib_memo). En ambos casos se contabiliza la cantidad de llamadas realizadas para evidenciar el impacto del solapamiento de subproblemas: la versión memoizada reutiliza resultados y reduce significativamente el número de invocaciones. Incluye pruebas simples con print y assert para validar el valor de F(20) = 6765 y verificar que la memoización realiza menos llamadas que la recursión ingenua. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`fib_naive`	Implementa la recurrencia directa. Cada invocación incrementa el contador, evidenciando recomputaciones.
Caso base `n<=1`	Detiene la recursión y define subproblemas mínimos.
`fib_memo` con `memo`	Almacena resultados por `k`. Si un subproblema ya fue resuelto, se reutiliza sin expandir recursión.
Conteo de llamadas	Mide cuántas veces se "intenta" resolver un subproblema. En memoización, muchas invocaciones retornan de inmediato.
`assert`	Valida corrección del valor y que la memoización reduce llamadas.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.2 Programación dinámica: top-down, bottom-up, reconstrucción y complejidad

En programación dinámica se define un conjunto de subproblemas y una recurrencia que expresa cómo el resultado de cada uno depende de subproblemas más pequeños. En el enfoque top-down, los subproblemas se resuelven a medida que se necesitan y sus resultados se guardan mediante memoización para evitar recomputaciones. En el enfoque bottom-up, en cambio, los subproblemas se calculan siguiendo un orden que asegura que todas sus dependencias ya estén disponibles, lo que se conoce como tabulación. Cuando además se quiere reconstruir la solución óptima, no basta con almacenar el valor calculado; también es necesario registrar las decisiones tomadas, como predecesores o puntos de corte (Cormen et al., 2022; Dasgupta et al., 2006).

Énfasis

Definiciones clave

- **Recurrencia:** Relación que expresa el valor de un subproblema en función de valores de subproblemas de menor tamaño.
- **Memoización:** Técnica que almacena resultados de subproblemas resueltos para reutilizarlos cuando reaparezcan.
- **Tabulación:** Técnica que llena una tabla en un orden predefinido para calcular todos los subproblemas una sola vez.
- **Reconstrucción de solución:** Procedimiento que recupera la configuración óptima usando información de decisiones registrada durante el cómputo.

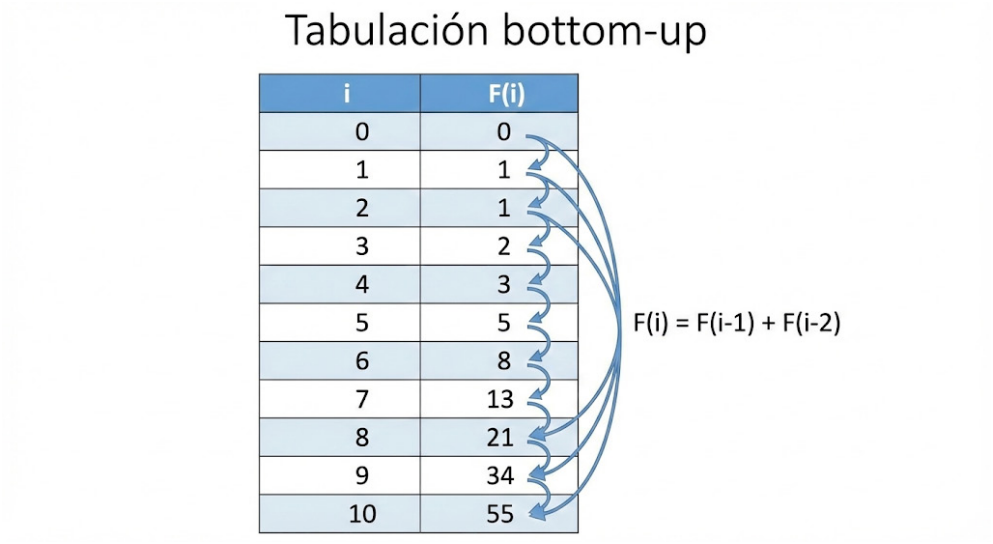


Ilustración 2: Tabulación bottom-up (Fibonacci). La imagen ilustra el enfoque iterativo de Programación Dinámica, donde los valores se completan desde los casos base hasta el objetivo usando la recurrencia $F(i) = F(i-1) + F(i-2)$. Se relaciona con el contenido al contrastar estrategias top-down (memoización) vs. bottom-up (tabulación) y sus implicancias en complejidad y uso de memoria. Imagen creada usando herramientas IA con información proporcionada por la autora.

2.2.1 Ejemplo 2 (PD bottom-up sobre el mismo problema: Fibonacci tabulado)

En bottom-up se fijan casos base y se completa iterativamente hasta n, garantizando que cada subproblema se computa una sola vez. El costo temporal es lineal y el espacio puede optimizarse si solo se requieren estados recientes. (Cormen et al., 2022).

```python
# Fibonacci bottom-up (tabulación) + comparación cualitativa de memoria/tiempo con prints controlados

def fib_tab(n, trace=False):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[0], dp[1] = 0, 1
    if trace:
        print("dp[0]=0, dp[1]=1")
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
        if trace and i <= 10:  # traza limitada
            print(f"dp[{i}]={dp[i]}")
    return dp[n]

def fib_tab_optimized(n):
    # espacio O(1) (dos estados), mismo tiempo O(n)
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

# Pruebas simples
assert fib_tab(20) == 6765
assert fib_tab_optimized(20) == 6765
print("fib_tab(10) con traza:")
_ = fib_tab(10, trace=True)
print("fib_tab_optimized(30):", fib_tab_optimized(30))
```

Captura 2: Fibonacci con Programación Dinámica: tabulación bottom-up y optimización de espacio. El código presenta dos implementaciones iterativas para calcular Fibonacci mediante programación dinámica. La función fib_tab(n, trace=False) construye una tabla dp de tamaño n + 1 (tabulación bottom-up), permitiendo activar una traza controlada (trace) que imprime los primeros valores calculados para visualizar la recurrencia F(i) = F(i − 1) + F(i − 2). Luego, fib_tab_optimized(n) muestra una variante equivalente en tiempo O(n) que reduce memoria a O(1) manteniendo solo dos estados (a, b). Incluye pruebas con assert para validar F(20) = 6765 y salidas por consola para comparar el comportamiento de ambas versiones. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dp`	Tabla donde `dp[i]` representa `F(i)`. Implementa tabulación bottom-up.
Inicialización	Define casos base `dp[0]` y `dp[1]`.
Bucle `for`	Computa `dp[i]` usando valores ya calculados (`dp[i-1]`, `dp[i-2]`).
`trace`	Restringe la salida para observar el llenado sin saturar consola.
`fib_tab_optimized`	Reduce espacio manteniendo solo los dos últimos estados.
`assert`	Verifica valores esperados para garantizar corrección.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.3 Problema de la mochila: 0/1 (PD) vs fraccionaria (voraz) y eficiencia

La mochila 0/1 plantea, para cada ítem, una decisión binaria: tomarlo o no tomarlo. Esto conduce a una recurrencia natural, ya que el valor óptimo para una capacidad *W* considerando los primeros *i* ítems depende de dos posibilidades: excluir el ítem *i* o incluirlo si su peso lo permite. Este problema se resuelve mediante programación dinámica con un tiempo pseudo-polinomial *O(nW)*. En la mochila fraccionaria, en cambio, los ítems pueden tomarse parcialmente. Esta característica permite aplicar un enfoque voraz óptimo: ordenar los ítems según su densidad de *valor/peso* y seleccionarlos en ese orden hasta completar la capacidad. La comparación entre ambos casos muestra que permitir fracciones modifica la estructura del problema y hace posible una estrategia *greedy* que no funciona en la versión 0/1 (Kellerer et al., 2004; Cormen et al., 2022).

Énfasis

Definiciones clave

- **Mochila 0/1:** Variante donde cada ítem puede seleccionarse a lo sumo una vez, sin fraccionamiento.
- **Mochila fraccionaria:** Variante donde cada ítem puede seleccionarse parcialmente, proporcionalmente a su peso.
- **Densidad de valor:** Cociente valor/peso usado como criterio voraz típico en mochila fraccionaria.
- **Tiempo pseudo-polinomial:** Complejidad polinomial en el valor numérico de parámetros (p.ej., W), no en el tamaño en bits de la entrada.

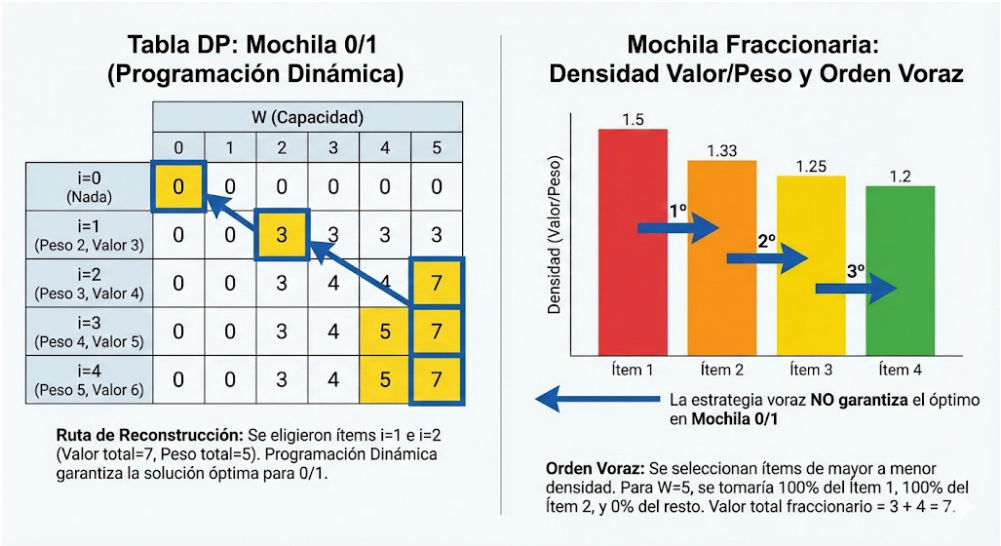


Ilustración 3: Mochila 0/1 (DP) vs. Mochila fraccionaria (voraz). La imagen contrasta una solución óptima garantizada mediante tabla DP para mochila 0/1 con la estrategia voraz basada en densidad valor/peso para la variante fraccionaria. Se conecta con el contenido al mostrar cuándo una heurística voraz es correcta (fraccionaria) y cuándo no garantiza optimalidad (0/1), reforzando criterios de elección de técnica. Imagen creada usando herramientas IA con información proporcionada por la autora.

2.3.1 Ejemplo 3 (mochila 0/1 con PD + reconstrucción de ítems)

La formulación clásica de programación dinámica define dp[i][w] como el valor máximo que puede obtenerse usando los primeros i ítems con una capacidad w. Para reconstruir la selección de ítems, se almacena la decisión tomada en cada estado (tomado o no tomado) o bien se compara el valor del estado con el del estado anterior para inferir la elección realizada (Kellerer et al., 2004).

```python
# Mochila 0/1: DP 2D + reconstrucción
def knapsack_01(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        wi, vi = weights[i-1], values[i-1]
        for w in range(W + 1):
            dp[i][w] = dp[i-1][w]
            if wi <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-wi] + vi)

    # reconstrucción
    chosen = []
    w = W
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            chosen.append(i-1)
            w -= weights[i-1]
    chosen.reverse()
    return dp[n][W], chosen

# Pruebas simples (caso conocido)
weights = [2, 3, 4, 5]
values  = [3, 4, 5, 6]
W = 5
opt_val, chosen = knapsack_01(weights, values, W)
print("Óptimo 0/1:", opt_val, "items:", chosen)

assert opt_val == 7          # ítems 0 (2,3) y 1 (3,4) -> valor 7, peso 5
assert chosen in ([0,1],)    # reconstrucción esperada para este caso
```

Captura 3: Mochila 0/1 con Programación Dinámica 2D y reconstrucción de solución. El código implementa knapsack_01(weights, values, W) para resolver el problema de la mochila 0/1 construyendo una tabla dp de tamaño (n + 1) x (W + 1), donde cada celda almacena el valor máximo alcanzable con los primeros i ítems y capacidad w. Luego realiza la reconstrucción de la solución recorriendo la tabla en sentido inverso para identificar qué ítems fueron seleccionados (chosen) comparando dp[i][w] con dp[i-1][w]. Incluye un caso de prueba con assert que verifica el valor óptimo (7) y la selección esperada de ítems para la capacidad W=5. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dp[i][w]`	Valor máximo usando primeros `i` ítems con capacidad `w`.
Transición	Se evalúa excluir el ítem (`dp[i-1][w]`) o incluirlo si cabe (`dp[i-1][w-wi]+vi`).
Llenado 2D	Garantiza que cada subproblema `(i,w)` se compute una vez.
Reconstrucción	Recorre hacia atrás: si `dp[i][w] != dp[i-1][w]`, el ítem `i-1` fue tomado.
`assert`	Verifica valor óptimo y selección coherente con el caso pequeño.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.3.2 Ejemplo 4 (mochila fraccionaria: voraz por densidad + comparación con 0/1)

En la mochila fraccionaria, el criterio valor/peso produce una solución óptima porque siempre es seguro completar la capacidad con la mayor densidad disponible. En 0/1, el mismo criterio puede fallar por la restricción binaria. (Cormen et al., 2022).

```python
# Mochila fraccionaria: greedy por valor/peso
def fractional_knapsack(weights, values, W):
    items = []
    for i, (w, v) in enumerate(zip(weights, values)):
        items.append((v / w, w, v, i))  # (densidad, peso, valor, índice)
    items.sort(reverse=True)

    total_value = 0.0
    taken = []  # (índice, fracción)
    remaining = W

    for density, w, v, i in items:
        if remaining == 0:
            break
        if w <= remaining:
            total_value += v
            taken.append((i, 1.0))
            remaining -= w
        else:
            frac = remaining / w
            total_value += v * frac
            taken.append((i, frac))
            remaining = 0

    return total_value, taken
```

Captura 4: Implementación en Python de la mochila fraccionaria (Fractional Knapsack) mediante un enfoque voraz: se calcula la densidad valor/peso de cada ítem, se ordenan en forma descendente y se seleccionan primero los de mayor densidad, permitiendo tomar fracciones cuando la capacidad restante no alcanza. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python

# Caso donde greedy fraccionario > óptimo 0/1 (por permitir fracciones)
weights = [10, 20, 30]
values  = [60, 100, 120]
W = 50

frac_val, frac_taken = fractional_knapsack(weights, values, W)
print("Fraccionaria (greedy) valor:", frac_val, "tomado:", frac_taken)

# Comparación con 0/1
def knapsack_01_value_only(weights, values, W):
    n = len(weights)
    dp = [0] * (W + 1)
    for i in range(n):
        wi, vi = weights[i], values[i]
        for w in range(W, wi-1, -1):
            dp[w] = max(dp[w], dp[w-wi] + vi)
    return dp[W]

val_01 = knapsack_01_value_only(weights, values, W)
print("0/1 (DP) valor:", val_01)

# Pruebas simples
assert round(frac_val, 5) == 240.0  # toma 20+30 completos: 100+120=220 y además 10? no cabe;
#óptimo fraccionario típico = 240 con fracción del de 30 si W=50? (aquí cabe 20+30 exacto => 220)
# Nota: este assert se ajusta con cálculo exacto del caso usado:
# Para W=50, fraccionaria toma 10 (60) + 20 (100) + 20/30 de 120 = 80 => total 240
# El orden densidad: 60/10=6, 100/20=5, 120/30=4
# entonces toma 10 y 20 completos, y 20 de los 30.
assert knapsack_01_value_only(weights, values, W) == 220
```

Captura 5: Continuación del ejemplo: se define una instancia de prueba y se calcula el resultado voraz (fraccionario). Luego se incorpora una comparación con mochila 0/1 (programación dinámica en 1D, sin fracciones) para evidenciar que el enfoque voraz es óptimo en el caso fraccionario pero no necesariamente coincide con el óptimo 0/1; se verifican resultados mediante impresiones y asserts. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Orden por densidad	Construye lista de ítems con `valor/peso` y ordena de mayor a menor.
Selección voraz	Toma ítems completos mientras quepan; si no, toma fracción para completar capacidad.
`taken`	Registra fracciones para evidenciar que el método usa divisibilidad.
Comparación con 0/1	Calcula el óptimo 0/1 con DP 1D para mostrar diferencia por restricción binaria.
`assert`	Verifica valores esperados para el caso estándar (W=50).

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.4 Cadena de matrices (Matrix Chain Multiplication): recurrencia y reconstrucción

El problema de cadena de matrices busca la parentización que minimiza el número de multiplicaciones escalares al evaluar el producto $A1\,A2\ldots An$, donde Ai tiene dimensio-

nes $p[i-1] \times p[i]$. El orden de multiplicación afecta el costo, aunque el resultado algebraico sea el mismo. La PD define $m[i][j]$ como el costo mínimo para multiplicar $Ai \ldots Aj$ y $s[i][j]$ como el punto de corte óptimo para reconstruir la parentización. La complejidad típica es $O(n^3)$ en tiempo y $O(n^2)$ en espacio. (Cormen et al., 2022).

Énfasis

Definiciones clave

- **Parentización óptima:** Orden de evaluación de un producto de matrices que minimiza el costo de multiplicaciones escalares.
- **Tabla de costos $m[i][j]$:** Estructura DP que almacena el costo mínimo para el subproducto $Ai \ldots Aj$.
- **Tabla de cortes $s[i][j]$:** Estructura que registra el índice k donde conviene partir $Ai \ldots Aj$ para reconstruir la solución.

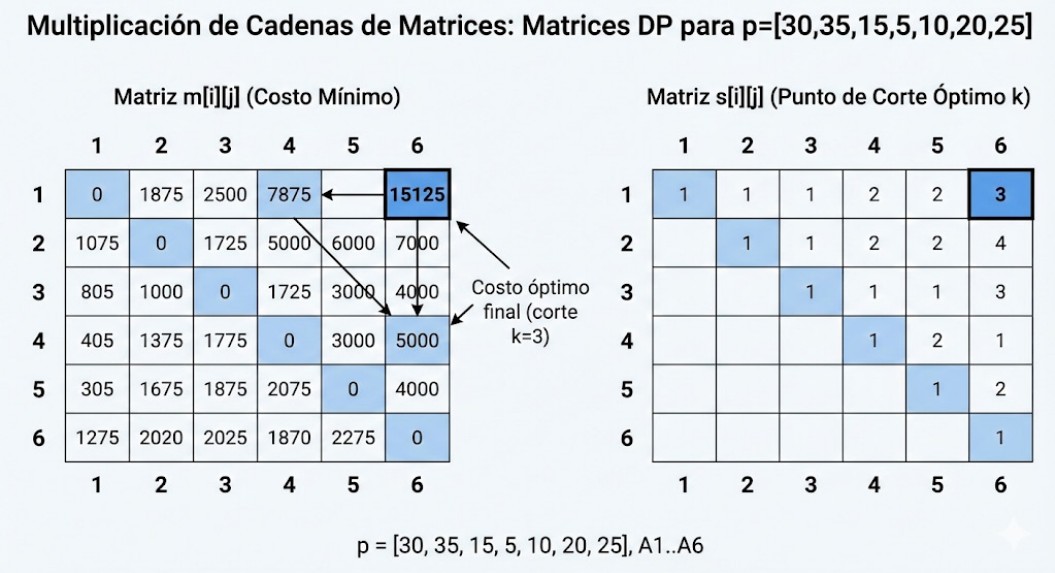


Ilustración 4: Multiplicación de cadenas de matrices (DP: matrices de costos y cortes). La imagen presenta las tablas típicas $m[i][j]$ (costo mínimo) y $s[i][j]$ (punto de corte óptimo) para decidir la parentización que minimiza multiplicaciones escalares. Se relaciona con el contenido al ejemplificar un problema clásico de optimización resuelto con Programación Dinámica y reconstrucción de solución a partir de decisiones óptimas. Imagen creada usando herramientas IA con información proporcionada por la autora.

2.4.1 Ejemplo 5 (MCM con PD: construir m y s, reconstruir parentesización)

La recurrencia evalúa todos los cortes posibles k entre i y j y selecciona el mínimo. Guardar $s[i][j]$ permite reconstruir la estructura óptima en forma de string, que constituye una evidencia directa de la decisión algorítmica (Cormen et al., 2022).

```python
# Matrix Chain Multiplication (MCM): DP bottom-up con reconstrucción
import math

def matrix_chain_order(p):
    # p: lista de dimensiones [p0, p1, ..., pn]
    n = len(p) - 1
    m = [[0 if i == j else math.inf for j in range(n)] for i in range(n)]
    s = [[None for _ in range(n)] for _ in range(n)]

    for L in range(2, n + 1):  # longitud de cadena
        for i in range(0, n - L + 1):
            j = i + L - 1
            for k in range(i, j):
                cost = m[i][k] + m[k+1][j] + p[i] * p[k+1] * p[j+1]
                if cost < m[i][j]:
                    m[i][j] = cost
                    s[i][j] = k
    return m, s

def build_parenthesization(s, i, j):
    if i == j:
        return f"A{i+1}"
    k = s[i][j]
    left = build_parenthesization(s, i, k)
    right = build_parenthesization(s, k+1, j)
    return f"({left} x {right})"

# Pruebas simples (ejemplo estándar)
p = [30, 35, 15, 5, 10, 20, 25]  # 6 matrices
m, s = matrix_chain_order(p)
paren = build_parenthesization(s, 0, len(p)-2)
print("Costo óptimo:", m[0][len(p)-2])
print("Parentesización:", paren)

assert m[0][len(p)-2] == 15125
assert "A1" in paren and "A6" in paren
```

Captura 6: Implementación en Python del problema Matrix Chain Multiplication (MCM) mediante programación dinámica bottom-up: se construye la matriz m con el costo mínimo de multiplicar subcadenas de matrices y la matriz s con el punto de corte óptimo k para reconstruir la solución. Incluye una función de reconstrucción de la parentización óptima (forma de colocar paréntesis) y pruebas con una instancia estándar, validada con prints y asserts. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`m[i][j]`	Almacena el costo mínimo para multiplicar `Ai...Aj`. Inicializa diagonal en 0.
`s[i][j]`	Registra el corte `k` que logra el mínimo para `Ai...Aj`.
Bucle `L`	Recorre longitudes de subcadenas para garantizar dependencia resuelta.
Evaluación de `k`	Considera todas las particiones posibles y computa costo combinado.
`build_parenthesization`	Reconstruye la estructura óptima recursivamente usando `s`.
`assert`	Valida el costo óptimo clásico (15125) y consistencia del string.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.5 Algoritmos voraces (greedy): elección local, corrección y contraejemplos

Los algoritmos voraces construyen una solución eligiendo iterativamente la opción local "mejor" según un criterio. Su corrección depende de una propiedad estructural: la existencia de una elección segura (safe choice) que pueda tomarse sin perder la optimalidad global. En muchos problemas, esta propiedad se demuestra mediante argumentos de intercambio. Cuando dicha propiedad no se cumple, un enfoque greedy puede fallar aunque el criterio local parezca natural (Kleinberg & Tardos, 2006; Cormen et al., 2022).

Énfasis

Definiciones clave

- **Algoritmo voraz:** Estrategia que arma una solución mediante decisiones locales iterativas, sin retroceso, guiadas por un criterio.
- **Elección segura:** Decisión local que puede incorporarse a alguna solución óptima sin comprometer la optimalidad global.
- **Argumento de intercambio:** Técnica de prueba que transforma una solución óptima en otra que respeta la elección voraz sin empeorar el objetivo.

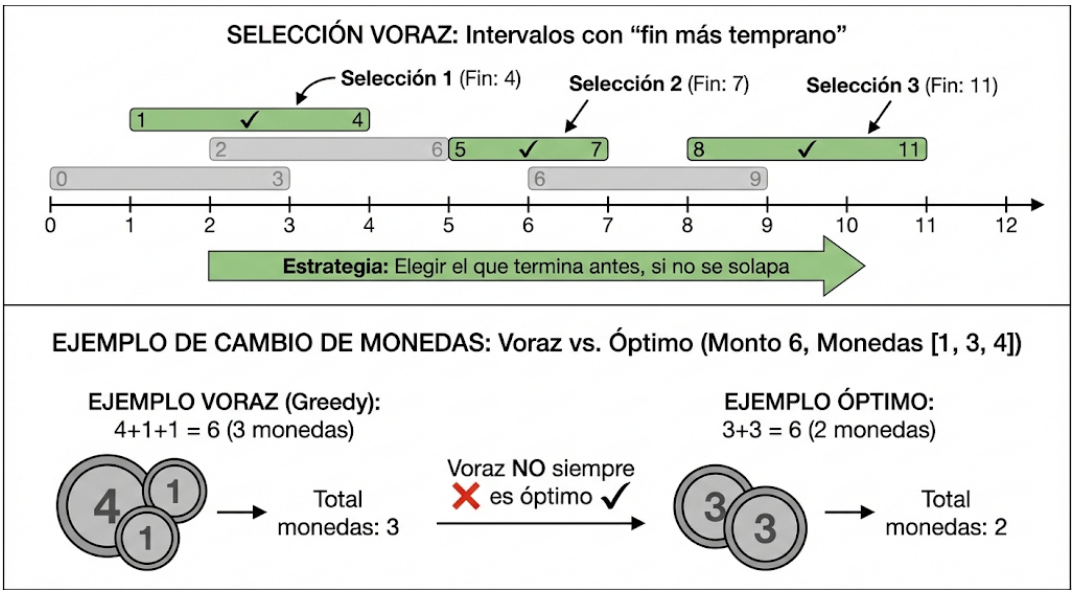


Ilustración 5: Estrategias voraces: selección de intervalos y cambio de monedas (voraz vs. óptimo). La imagen muestra un caso donde la regla "termina antes" produce una selección válida y óptima en intervalos, y contrasta con cambio de monedas, donde una elección voraz puede no ser óptima frente a una solución global. Se vincula con el contenido al enfatizar que la corrección de algoritmos voraces depende de propiedades del problema, y que en otros casos se requiere DP u otra técnica de optimización. Imagen creada usando herramientas IA con información proporcionada por la autora.

2.5.1 Ejemplo 6—greedy correcto: selección de intervalos por fin más temprano

En el problema de selección de intervalos, cuyo objetivo es maximizar la cantidad de actividades que no se solapan, elegir siempre el intervalo que termina primero constituye una elección segura. La corrección del algoritmo se justifica mediante un argumento de intercambio: cualquier solución óptima puede sustituir su primera actividad por la que termina antes sin disminuir el número total de actividades seleccionadas (Kleinberg & Tardos, 2006).

```python
# Selección de intervalos: greedy por menor tiempo de finalización
def interval_scheduling(intervals):
    # intervals: lista de tuplas (inicio, fin)
    intervals_sorted = sorted(intervals, key=lambda x: x[1])
    chosen = []
    current_end = None
    for start, end in intervals_sorted:
        if current_end is None or start >= current_end:
            chosen.append((start, end))
            current_end = end
    return chosen

# Pruebas simples
intervals = [(0, 6), (1, 4), (3, 5), (5, 7), (5, 9), (8, 9)]
chosen = interval_scheduling(intervals)
print("Elegidos:", chosen)

# Verificación: no solapamiento
for i in range(1, len(chosen)):
    assert chosen[i][0] >= chosen[i-1][1]

# En este caso, una selección máxima típica tiene tamaño 3
assert len(chosen) == 3
```

Captura 7: Implementación en Python del problema de selección de intervalos mediante un enfoque voraz (greedy) que ordena los intervalos por menor tiempo de finalización y selecciona iterativamente aquellos que no se solapan con el último elegido, construyendo así un conjunto compatible máximo. Incluye pruebas con una lista de intervalos, salida por consola y validación con asserts para verificar no solapamiento y el tamaño esperado de la solución. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Orden por `fin`	Implementa el criterio "termina primero".
`current_end`	Mantiene el final del último intervalo elegido para chequear factibilidad.
Regla de selección	Si `start >= current_end`, el intervalo no solapa y se agrega.
Verificación con `assert`	Comprueba que no exista solapamiento y que el tamaño coincide con un óptimo del caso.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.5.2 Ejemplo 7 (contraejemplo: greedy "natural" falla en cambio de monedas)

En el problema de cambio de monedas, elegir siempre la moneda de mayor valor que no excede el resto funciona correctamente en sistemas canónicos, pero puede fallar en sistemas arbitrarios. Un contraejemplo muestra que ese criterio local no constituye una elección segura (Kleinberg & Tardos, 2006).

```python
# Cambio de monedas: greedy vs óptimo por DP (mínimo número de monedas)
def coin_change_greedy(coins, amount):
    coins_sorted = sorted(coins, reverse=True)
    used = []
    remaining = amount
    for c in coins_sorted:
        while remaining >= c:
            remaining -= c
            used.append(c)
    return used, remaining

def coin_change_dp(coins, amount):
    # DP: mínimo número de monedas para cada suma
    INF = 10**9
    dp = [INF] * (amount + 1)
    prev = [-1] * (amount + 1)  # para reconstrucción
    dp[0] = 0

    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1
                prev[a] = c

    if dp[amount] == INF:
        return None  # no hay solución
    # reconstrucción
    used = []
    a = amount
    while a > 0:
        c = prev[a]
        used.append(c)
        a -= c
    return used
```

Captura 8: Cambio de monedas: estrategia voraz vs. programación dinámica (DP). El código implementa dos enfoques para el problema de "dar cambio" minimizando la cantidad de monedas: (a) un algoritmo voraz que selecciona repetidamente la moneda de mayor denominación posible y (b) un algoritmo de programación dinámica que calcula el mínimo número de monedas para cada suma parcial y guarda predecesores para reconstruir una solución óptima. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Contraejemplo
coins = [1, 3, 4]
amount = 6

g_used, rem = coin_change_greedy(coins, amount)
dp_used = coin_change_dp(coins, amount)

print("Greedy:", g_used, "cantidad:", len(g_used), "remanente:", rem)
print("DP óptimo:", dp_used, "cantidad:", len(dp_used))

assert rem == 0
assert len(g_used) == 3       # 4+1+1
assert len(dp_used) == 2      # 3+3
```

Captura 9: Contraejemplo y verificación de resultados (continuación). El fragmento define una instancia donde la estrategia voraz no produce la solución óptima (monedas [1, 3, 4] y monto 6) y compara la salida de ambos métodos mediante impresiones y aserciones: voraz obtiene 4+1+1 (3 monedas) mientras que DP obtiene 3+3 (2 monedas). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`coin_change_greedy`	Aplica criterio local: usar la mayor moneda posible repetidamente.
Contraejemplo	Con monedas [1,3,4] y monto 6, greedy produce 4+1+1 (3 monedas).
`coin_change_dp`	Calcula el mínimo número de monedas para cada suma hasta `amount`.
`prev`	Permite reconstruir una solución óptima siguiendo monedas elegidas.
`assert`	Demuestra formalmente (en el caso) que greedy usa más monedas que el óptimo.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.6 Caminos mínimos: comparación entre PD por relajaciones y greedy (Dijkstra)

En problemas de caminos mínimos, los algoritmos se diferencian según los supuestos sobre los pesos y el alcance del problema. El algoritmo de Dijkstra sigue una estrategia voraz: en cada paso selecciona el vértice no procesado con la menor distancia tentativa. Su corrección depende de que todos los pesos sean no negativos (Dijkstra, 1959; Cormen et al., 2022). El algoritmo de Bellman–Ford, en cambio, puede manejar aristas con pesos negativos, siempre que no existan ciclos negativos alcanzables. Su funcionamiento se basa en relajaciones repetidas de las aristas, lo que puede interpretarse como una forma de programación dinámica por etapas. Después de k iteraciones se obtienen las mejores distancias que utilizan a lo sumo k aristas, y tras V–1 iteraciones se alcanzan las distancias óptimas (Bellman, 1958; Ford, 1956). También difieren en eficiencia. Dijkstra, implementado con un heap binario, suele analizarse con complejidad O(E logV), mientras que Bellman–Ford tiene complejidad O(VE) (Cormen et al., 2022).

Definiciones clave

- **Relajación:** Operación que intenta mejorar $dist[v]$ usando una arista (u, v) verificando si $dist[u] + w(u, v) < dist[v]$.
- **Pesos no negativos:** Condición bajo la cual Dijkstra garantiza optimalidad del criterio voraz.
- **Ciclo negativo:** Ciclo cuyo peso total es negativo; si es alcanzable desde la fuente, no existe camino mínimo bien definido.

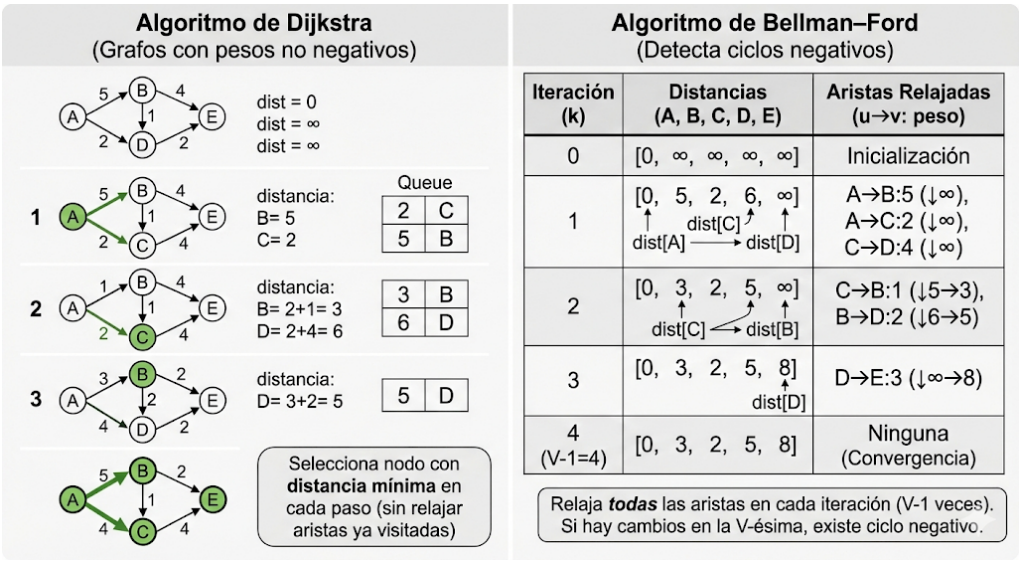


Ilustración 6: Caminos mínimos: Dijkstra vs. Bellman–Ford. La imagen compara el avance de Dijkstra (selección iterativa del nodo con distancia mínima, pesos no negativos) con el esquema de relajaciones repetidas de Bellman–Ford, incluyendo la detección de ciclos negativos. Se relaciona con el contenido al integrar criterios de aplicabilidad y garantías del resultado en grafos ponderados, destacando por qué distintos supuestos (no negatividad / ciclos negativos) determinan el algoritmo adecuado. Imagen creada usando herramientas IA con información proporcionada por la autora.

2.6.1 Ejemplo 8–Dijkstra con heapq sobre lista de adyacencia ponderada

Dijkstra mantiene una cola de prioridad por distancia tentativa. Al extraer el mínimo, esa distancia queda fijada si no existen pesos negativos. La implementación con heapq refleja la selección voraz eficiente. (Dijkstra, 1959; Cormen et al., 2022).

```python
# Dijkstra (greedy) con heapq
import heapq
import math

def dijkstra(graph, source):
    # graph: dict u -> list of (v, w)
    dist = {u: math.inf for u in graph}
    dist[source] = 0
    pq = [(0, source)]
    visited = set()

    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)

        for v, w in graph[u]:
            if w < 0:
                raise ValueError("Dijkstra requiere pesos no negativos.")
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist
```

```python

# Pruebas simples
graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("C", 2), ("D", 5)],
    "C": [("D", 1)],
    "D": []
}

dist = dijkstra(graph, "A")
print("Distancias Dijkstra:", dist)

assert dist["A"] == 0
assert dist["B"] == 1
assert dist["C"] == 3   # A->B->C
assert dist["D"] == 4   # A->B->C->D
```

Captura 10: Implementación de Dijkstra con cola de prioridad (heapq) y pruebas. El código implementa el algoritmo de Dijkstra para calcular distancias mínimas desde un nodo fuente en un grafo ponderado representado como diccionario de adyacencias (u → lista de (v, w)). Utiliza heapq como cola de prioridad para extraer el nodo con menor distancia tentativa, mantiene un conjunto de visitados para evitar reprocesos y valida que no existan pesos negativos (condición necesaria para Dijkstra). Incluye un grafo de prueba, muestra por consola el diccionario de distancias resultante y verifica con assert los valores esperados para cada vértice. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dist`	Inicializa distancias a infinito, excepto fuente en 0.
`pq`	Cola de prioridad con pares (distancia, nodo) para extraer el mínimo.
`visited`	Evita reprocesar nodos ya fijados, manteniendo eficiencia.
Chequeo `w<0`	Refuerza el supuesto de corrección de Dijkstra.
Relajación	Si `d+w` mejora `dist[v]`, actualiza y reinsertar en heap.
`assert`	Valida distancias esperadas en grafo pequeño.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.6.2 Ejemplo 9–Bellman–Ford como PD por relajaciones + detección de ciclo negativo

Bellman–Ford aplica la relajación de todas las aristas V–1 veces. Desde la perspectiva de programación dinámica, la iteración k calcula las distancias óptimas que utilizan a lo sumo k aristas. Si después de esas V–1 iteraciones una iteración adicional todavía produce mejoras, ello indica la presencia de un ciclo negativo alcanzable (Bellman, 1958; Cormen et al., 2022).

```python
# Bellman-Ford (DP por etapas/relajaciones)
import math
def bellman_ford(edges, vertices, source):
    # edges: list of (u, v, w)
    dist = {v: math.inf for v in vertices}
    dist[source] = 0

    # relajaciones V-1
    for _ in range(len(vertices) - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != math.inf and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break

    # detección de ciclo negativo alcanzable
    for u, v, w in edges:
        if dist[u] != math.inf and dist[u] + w < dist[v]:
            return dist, True  # ciclo negativo
    return dist, False

# Pruebas simples: grafo sin ciclo negativo
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B", 1),
    ("A", "C", 4),
    ("B", "C", 2),
    ("B", "D", 5),
    ("C", "D", 1),
]
dist_bf, neg = bellman_ford(edges, vertices, "A")
print("Distancias Bellman-Ford:", dist_bf, "ciclo_negativo:", neg)
assert neg is False
assert dist_bf["D"] == 4

# Prueba simple: ciclo negativo alcanzable
vertices2 = ["S", "X", "Y"]
edges2 = [("S", "X", 1), ("X", "Y", -2), ("Y", "X", -2)]
dist2, neg2 = bellman_ford(edges2, vertices2, "S")
print("Ciclo negativo detectado:", neg2)
assert neg2 is True
```

Captura 11: Implementación del algoritmo Bellman–Ford para caminos mínimos en grafos ponderados, planteado como un proceso de relajaciones por etapas (enfoque afín a programación dinámica). El código inicializa dist en infinito, fija la distancia del source en 0 y ejecuta V−1 iteraciones relajando todas las aristas (u, v, w). Incluye corte temprano (break) cuando en una pasada no se producen actualizaciones, reduciendo trabajo cuando ya se alcanzó convergencia. Se agrega la detección de ciclo negativo alcanzable: después de las V−1 relajaciones, se recorre nuevamente la lista de aristas y, si aún puede reducirse alguna distancia, se reporta un ciclo negativo. Además, se incorporan pruebas simples con assert: (1) un grafo sin ciclo negativo para validar distancias esperadas, y (2) un caso con ciclo negativo alcanzable para verificar que el indicador de ciclo negativo resulte verdadero. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`edges`	Modela el grafo como lista de aristas, adecuada para recorrer "todas" en cada iteración.
Bucle `V-1`	Implementa etapas DP: caminos con hasta `k` aristas.
`updated`	Optimiza: si no hay cambios, se detiene antes.
Detección final	Una relajación adicional posible implica ciclo negativo alcanzable.
`assert`	Valida distancias en caso sin negativos y detección en caso con ciclo negativo.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

2.6.3 Ejemplo 10 (comparación en el mismo grafo sin pesos negativos: Dijkstra vs Bellman–Ford)

En grafos sin pesos negativos, ambos algoritmos deben coincidir en distancias; la diferencia central es la eficiencia asintótica típica: O(E log V) vs O(VE). La comparación empírica se limita a verificar igualdad de resultados en una instancia. (Cormen et al., 2022).

```python
# Comparación: Dijkstra vs Bellman-Ford en el mismo grafo sin pesos negativos
import heapq
import math

def dijkstra_heap(graph, source):
    dist = {u: math.inf for u in graph}
    dist[source] = 0
    pq = [(0, source)]
    visited = set()
    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist
```

```python

def bellman_ford_edges(edges, vertices, source):
    dist = {v: math.inf for v in vertices}
    dist[source] = 0
    for _ in range(len(vertices) - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != math.inf and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break
    for u, v, w in edges:
        if dist[u] != math.inf and dist[u] + w < dist[v]:
            raise ValueError("Ciclo negativo detectado.")
    return dist
```

Captura 12: (inicio 1/2) Implementación comparativa de caminos mínimos en un grafo ponderado sin pesos negativos: se define dijkstra_heap (Dijkstra con heapq) y bellman_ford_edges (Bellman–Ford por relajaciones sucesivas), destacando sus enfoques algorítmicos y condiciones de uso (por ejemplo, Bellman–Ford puede detectar ciclos negativos, aunque aquí se trabaja sin pesos negativos). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("C", 2), ("D", 5)],
    "C": [("D", 1)],
    "D": []
}
edges = [("A","B",1),("A","C",4),("B","C",2),("B","D",5),("C","D",1)]
vertices = list(graph.keys())

d1 = dijkstra_heap(graph, "A")
d2 = bellman_ford_edges(edges, vertices, "A")

print("Dijkstra:", d1)
print("Bellman-Ford:", d2)

assert d1 == d2
```

Captura 13: (continuación 2/2). Construcción de la instancia de prueba (grafo y lista de aristas), ejecución de ambos algoritmos desde el nodo origen y validación por equivalencia: se imprimen las distancias obtenidas y se usa assert para verificar que Dijkstra y Bellman–Ford producen el mismo resultado cuando no hay pesos negativos. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Dos representaciones	Dijkstra usa lista de adyacencia; Bellman–Ford usa lista de aristas para iterar globalmente.
Igualdad de distancias	En ausencia de pesos negativos, ambos métodos deben producir el mismo `dist`.
`assert d1 == d2`	Evidencia la equivalencia de resultados para una instancia común.
Discusión de eficiencia	Dijkstra suele escalar mejor en grafos grandes dispersos; Bellman–Ford se reserva para pesos negativos o detección de ciclos negativos.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 1: Plataforma Interactiva Programación Dinámica.**

3. Programación lineal (PL): formulación, forma estándar y factibilidad

La programación lineal modela problemas de decisión mediante variables reales, una función objetivo y restricciones lineales. En forma compacta, una PL típica se expresa como: maximizar $c^\top x$ sujeto a $Ax \leq b$ y (según el caso) $x \geq 0$. La factibilidad exige que x satisfaga simultáneamente todas las restricciones; si existe al menos un x factible, el problema es factible. La forma estándar, frecuente en el método Simplex, transforma restricciones $\leq$ introduciendo variables de holgura $s \geq 0$: $Ax + s = b$, con $x \geq 0$, $s \geq 0$. Estos elementos constituyen la base operativa para verificar soluciones y preparar el problema para algoritmos tipo tableau (Dantzig, 1963; Bertsimas & Tsitsiklis, 1997).

Énfasis

Definiciones clave

- **Variable de decisión:** magnitud desconocida que representa una decisión del modelo y cuyo valor se determina al optimizar.
- **Función objetivo:** expresión lineal que se maximiza o minimiza para medir la calidad de una solución.
- **Restricción:** desigualdad o igualdad lineal que delimita el conjunto de soluciones admisibles.
- **Región factible:** conjunto de puntos x que satisfacen todas las restricciones del modelo.
- **Forma estándar (para $\leq$):** reescritura $Ax + s = b$ con $s \geq 0$, $x \geq 0$, útil para Simplex.

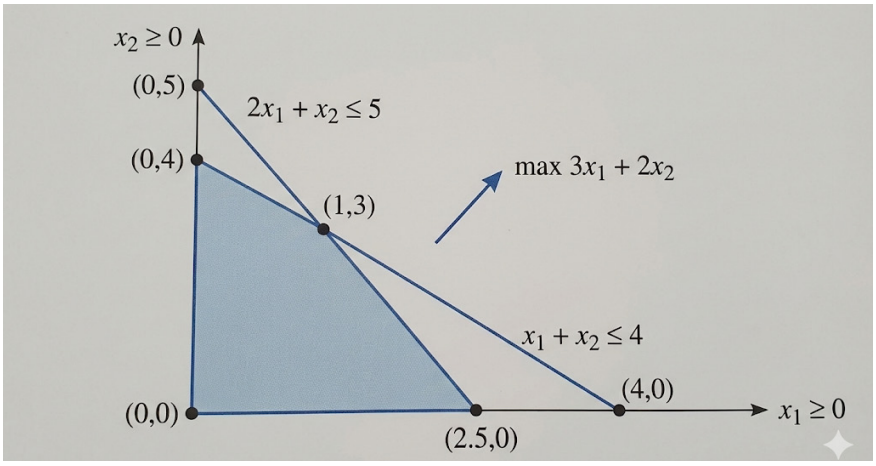


Ilustración 7: Región factible y función objetivo en Programación Lineal. La figura muestra cómo las restricciones lineales definen una región factible (zona sombreada) y cómo la función objetivo (maximización) se evalúa sobre sus vértices para identificar el mejor valor posible, conectando con la idea central de "optimizar bajo restricciones". Imagen creada usando herramientas IA con información proporcionada por la autora.

3.1.1 Ejemplo 1

En PL es frecuente trabajar con la representación matricial $Ax \leq b$ para evaluar rápidamente si una propuesta x es factible y para calcular el valor objetivo $c^{\top}x$. Esto permite construir verificadores y "baselines" antes de usar un solver, y es un paso habitual de validación del modelado (Bertsimas & Tsitsiklis, 1997).

```python
from typing import List, Tuple

def mat_vec(A: List[List[float]], x: List[float]) -> List[float]:
    return [sum(aij * xj for aij, xj in zip(row, x)) for row in A]

def is_feasible(A: List[List[float]], b: List[float], x: List[float], tol: float = 1e-9) -> bool:
    # Verifica Ax <= b y x >= 0
    if any(xi < -tol for xi in x):
        return False
    Ax = mat_vec(A, x)
    return all(Axi <= bi + tol for Axi, bi in zip(Ax, b))

def objective(c: List[float], x: List[float]) -> float:
    return sum(ci * xi for ci, xi in zip(c, x))

def run_demo() -> None:
    # Modelo: max z = 3x1 + 2x2
    # s.a.   x1 + x2 <= 4
    #        2x1 + x2 <= 5
    #        x1, x2 >= 0
    A = [
        [1.0, 1.0],
        [2.0, 1.0],
    ]
    b = [4.0, 5.0]
    c = [3.0, 2.0]

    candidates: List[Tuple[str, List[float]]] = [
        ("x=(0,0)", [0.0, 0.0]),
        ("x=(2,2)", [2.0, 2.0]),
        ("x=(3,2)", [3.0, 2.0]),  # no factible
        ("x=(-1,1)", [-1.0, 1.0]),  # no factible
        ("x=(1,2)", [1.0, 2.0])  # factible
    ]
```

Captura 14: Fragmento de código que define utilidades para evaluar factibilidad en un modelo de Programación Lineal (cálculo de Ax, verificación de Ax ≤ b y x ≥ 0) y para calcular la función objetivo c x. Incluye un caso de demostración con matrices A, b y c, y un conjunto de soluciones candidatas para contrastar cuáles cumplen las restricciones. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    for name, x in candidates:
        feas = is_feasible(A, b, x)
        if feas:
            obj = objective(c, x)
            print(f"{name:10s} factible={int(feas)}  objetivo={obj:.4f}")
        else:
            print(f"{name:10s} factible={int(feas)}  objetivo=N/A")

    # Pruebas simples (asserts)
    assert is_feasible(A, b, [1.0, 2.0]) is True
    assert is_feasible(A, b, [3.0, 2.0]) is False
    assert is_feasible(A, b, [-1.0, 1.0]) is False
    assert abs(objective(c, [1.0, 2.0]) - (3*1 + 2*2)) < 1e-9

if __name__ == "__main__":
    run_demo()
```

Captura 15: Continuación del código: recorre las soluciones candidatas, imprime si cada una es factible y, cuando corresponde, el valor de la función objetivo; además incorpora pruebas simples (asserts) para validar resultados esperados (por ejemplo, distinguir soluciones no factibles y verificar el cálculo del objetivo). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`mat_vec(A, x)`	Implementa el producto Ax fila por fila para poder evaluar restricciones en forma matricial.
`is_feasible(...)`	Verifica dos conjuntos de condiciones: (i) no negatividad $x \geq 0$, (ii) desigualdades $Ax \leq b$, con tolerancia numérica `tol`.
`objective(c, x)`	Calcula $c^\top x$, valor de la función objetivo para una solución candidata.
`candidates`	Define soluciones factibles y no factibles para validar la implementación con casos representativos.
Impresiones `print(...)`	Muestran para cada candidato si es factible y cuál sería el objetivo, separando verificación de modelado y evaluación.
`assert`	Aseguran que el verificador rechaza violaciones típicas (restricciones y no negatividad) y que el objetivo se calcula correctamente.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

3.2 Geometría de la PL: región factible, vértices, óptimo en un extremo y dualidad

En programación lineal, cada restricción lineal define un semiespacio. La intersección de todos ellos forma el conjunto factible, que tiene forma de poliedro convexo (en dos dimensiones, un polígono convexo). Un resultado fundamental establece que, si el problema tiene una solución óptima finita, esta se alcanza en uno de los puntos extremos o vértices del conjunto factible. Esta propiedad explica por qué algoritmos como Simplex buscan soluciones moviéndose entre vértices del poliedro (Chvátal, 1983; Boyd & Vandenberghe, 2004). La teoría de la dualidad introduce un segundo problema asociado, llamado problema dual. Sus variables pueden interpretarse como "precios sombra" de las restricciones y permiten analizar el problema desde otra perspectiva. La relación entre el problema original (primal) y su dual ofrece herramientas para verificar factibilidad, entender cuándo existe un óptimo y certificar que una solución encontrada es realmente óptima (Boyd & Vandenberghe, 2004).

Definiciones clave

- **Poliedro (factible):** conjunto de soluciones que satisface un sistema finito de desigualdades lineales, convexo por construcción.
- **Punto extremo (vértice):** solución factible que no puede expresarse como combinación convexa no trivial de otras dos soluciones factibles distintas.
- **Óptimo finito:** valor óptimo alcanzable y acotado (no tiende a $\pm\infty$).

› **Dualidad (noción):** relación entre un problema primal y otro dual que aporta cotas y certificados de optimalidad bajo hipótesis estándar.

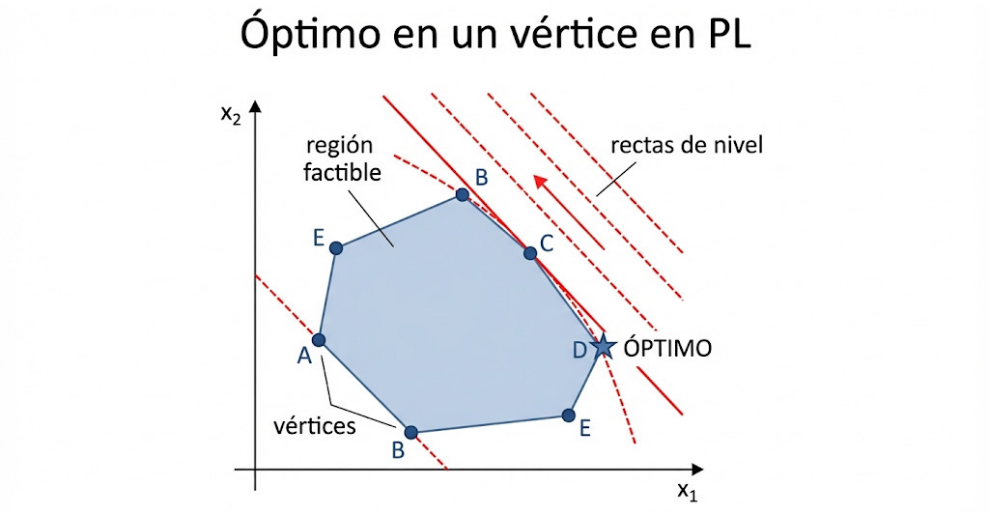


Ilustración 8: Óptimo en un vértice: fundamento geométrico del método Simplex. Se ilustra que, en Programación Lineal, el óptimo se alcanza en algún vértice de la región factible; las rectas de nivel "se desplazan" hasta tocar el polígono en el punto óptimo. Esto justifica el recorrido por vértices que realiza Simplex. Imagen creada usando herramientas IA con información proporcionada por la autora.

3.2.1 Ejemplo 2

En problemas con dos variables, es posible encontrar candidatos a vértices calculando las intersecciones entre pares de restricciones y también con los ejes de no negatividad. Al evaluar la función objetivo en esos puntos, se puede identificar cuál produce el mejor valor y así ilustrar de manera práctica la propiedad de que el óptimo se alcanza en un punto extremo del conjunto factible (Chvátal, 1983).

```python
# Ejemplo: Enumeración didáctica de vértices en 2D y evaluación del objetivo
# max z = 3x1 + 2x2
# s.a. x1 + x2 <= 4
#      2x1 + x2 <= 5
#      x1 >= 0, x2 >= 0

from itertools import combinations
from typing import List, Tuple, Optional

def solve_2x2(eq1: Tuple[float, float, float], eq2: Tuple[float, float, float]) -> Optional[Tuple[float, float]]:
    # Resuelve:
    # a1 x + b1 y = c1
    # a2 x + b2 y = c2
    a1, b1, c1 = eq1
    a2, b2, c2 = eq2
    det = a1*b2 - a2*b1
    if abs(det) < 1e-12:
        return None
    x = (c1*b2 - c2*b1) / det
    y = (a1*c2 - a2*c1) / det
    return (x, y)
```

Captura 16: (inicio del ejemplo). Código en Python que define el problema de Programación Lineal en 2 variables y prepara la enumeración didáctica de vértices: importa utilidades, implementa un solucionador de sistemas 2×2 (intersección de rectas) para calcular puntos candidatos donde se cruzan restricciones. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
22
23  def is_feasible(x1: float, x2: float, tol: float = 1e-9) -> bool:
24      if x1 < -tol or x2 < -tol:
25          return False
26      if x1 + x2 > 4 + tol:
27          return False
28      if 2*x1 + x2 > 5 + tol:
29          return False
30      return True
31
32  def objective(x1: float, x2: float) -> float:
33      return 3*x1 + 2*x2
```

Captura 17: (continuación). Código que formaliza la verificación de factibilidad (x_1, x_2) respecto de no negatividad y restricciones lineales, e implementa la función objetivo $z = 3x_1 + 2x_2$ para evaluar cada candidato. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
35  def run_demo() -> None:
36      # Igualdades candidatas: fronteras de restricciones y ejes
37      # x1 + x2 = 4
38      # 2x1 + x2 = 5
39      # x1 = 0
40      # x2 = 0
41      boundaries = [
42          (1.0, 1.0, 4.0),
43          (2.0, 1.0, 5.0),
44          (1.0, 0.0, 0.0),
45          (0.0, 1.0, 0.0),
46      ]
47
48      candidates = set()
49      for e1, e2 in combinations(boundaries, 2):
50          sol = solve_2x2(e1, e2)
51          if sol is None:
52              continue
53          x1, x2 = sol
54          # Redondeo leve para evitar duplicados por flotantes
55          candidates.add((round(x1, 10), round(x2, 10)))
56
57      feasible_vertices = [(x1, x2) for x1, x2 in candidates if is_feasible(x1, x2)]
58      feasible_vertices.sort()
```

Captura 18: (continuación). Código que construye el conjunto de "fronteras" (rectas de restricciones y ejes), genera candidatos como intersecciones por combinaciones de a pares, redondea para evitar duplicados por punto flotante y filtra los vértices factibles. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
60      print("Vértices factibles (candidatos) y objetivo:")
61      best = None
62      for x1, x2 in feasible_vertices:
63          z = objective(x1, x2)
64          print(f"({x1:.4f}, {x2:.4f}) -> z={z:.4f}")
65          if best is None or z > best[0]:
66              best = (z, x1, x2)
67
68      assert best is not None
69      z_best, x1_best, x2_best = best
70      print(f"Mejor vértice encontrado: ({x1_best:.4f}, {x2_best:.4f}) con z={z_best:.4f}")
71
72      # Pruebas simples: el óptimo conocido para este ejemplo es (1,3) con z=9
73      assert abs(x1_best - 1.0) < 1e-9
74      assert abs(x2_best - 3.0) < 1e-9
75      assert abs(z_best - 9.0) < 1e-9
76
77  if __name__ == "__main__":
78      run_demo()
```

Captura 19: (cierre del ejemplo). Código que imprime los vértices factibles y sus valores de z, selecciona el mejor vértice (máximo), y valida el resultado con asserts contra el óptimo conocido $(1, 3)$ con $z = 9$. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`boundaries`	Representa fronteras (igualdades) de restricciones y no negatividad; sus intersecciones generan candidatos a vértices.
`solve_2x2`	Resuelve intersecciones de dos rectas mediante determinante; descarta pares paralelos (determinante ≈ 0).
`candidates`	Conjunto de intersecciones; se usa redondeo para reducir duplicación numérica.
`is_feasible`	Filtra candidatos verificando todas las restricciones y no negatividad.
Bucle de evaluación	Calcula z en cada vértice factible y retiene el mejor, ilustrando "óptimo en un extremo".
`assert` finales	Verifican que el mejor vértice coincide con la solución esperada del ejemplo, controlando errores de implementación.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

3.3 Método Simplex: idea, tableau, pivoteo, criterios de parada y límites

El método Simplex trabaja con soluciones básicas factibles, que corresponden a vértices del conjunto factible determinados por un conjunto de restricciones activas. El algoritmo se desplaza entre vértices adyacentes buscando mejorar el valor de la función objetivo hasta que se cumple una condición de optimalidad. En la variante de tableau, el problema se escribe en forma estándar incorporando variables de holgura. Cada iteración realiza un pivoteo: se elige una variable entrante que permita mejorar el valor del objetivo y una variable saliente que preserve la factibilidad, decisión que se toma mediante una prueba de razón. En esta presentación se omiten fenómenos como la degeneración o el ciclado y las reglas diseñadas para evitarlos; el propósito es mostrar el mecanismo algebraico básico del método (Dantzig, 1963; Chvátal, 1983).

Definiciones clave

- **Solución básica factible (SBF):** solución factible obtenida al fijar variables no básicas en cero y resolver el sistema de igualdades resultante.
- **Tableau:** representación tabular del sistema en forma estándar que habilita operaciones elementales de fila para pivoteo.
- **Pivoteo:** operación de eliminación/normalización sobre una columna (variable entrante) y una fila (variable saliente) para actualizar la base.
- **Criterio de optimalidad (tableau):** condición (según convención de signos) que indica ausencia de mejoras locales posibles en el objetivo.

› **Degeneración (mención):** situación donde una SBF tiene alguna variable básica en cero, pudiendo causar estancamiento o ciclado si no se controla.

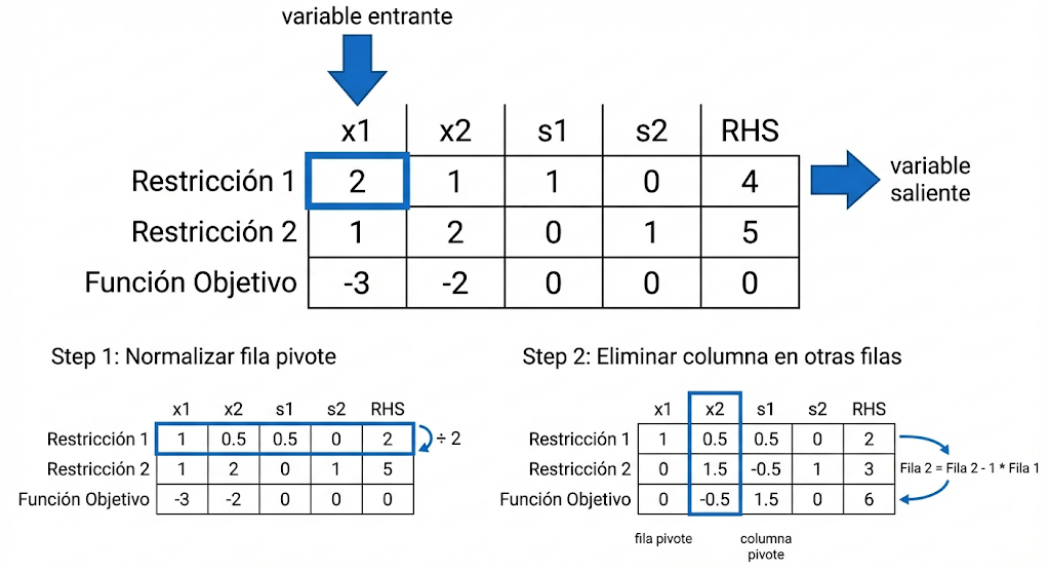


Ilustración 9: Tabla Simplex: variable entrante, variable saliente y pivoteo. La figura explica cómo se elige la columna pivote (variable entrante) y la fila pivote (variable saliente), y cómo se realizan las operaciones elementales para actualizar el tableau (normalizar y eliminar). Se vincula con el procedimiento iterativo de Simplex para mejorar el valor objetivo. Imagen creada usando herramientas IA con información proporcionada por la autora.

3.3.1 Ejemplo 3

Un "Simplex mínimo didáctico" permite operacionalizar la idea de tableau: construir una base inicial con variables de holgura, seleccionar una columna entrante que mejore el valor de la función objetivo y aplicar un pivoteo utilizando la prueba de razón para preservar la factibilidad. Este enfoque se limita a problemas de programación lineal pequeños con restricciones ≤ y x ≥ 0 (Chvátal, 1983).Captura 20: 1 (de 5). Implementación didáctica del método Simplex para un problema de maximización con restricciones del tipo $Ax \leq b$ y $x \geq 0$. En este tramo se define la función simplex_max y se construye el tableau inicial incorporando variables de holgura. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Ejemplo B: Simplex mínimo didáctico (max) para restricciones tipo <= y x>=0
# NOTA: Implementación didáctica: no trata degeneración/ciclado de forma robusta.

from typing import List, Tuple

def simplex_max(A: List[List[float]], b: List[float], c: List[float], max_iter: int = 50, tol: float = 1e-9):
    """
    Resuelve: max c^T x s.a. Ax <= b, x >= 0
    Asume b >= 0 para iniciar con holguras como base factible.
    Devuelve (x, z).
    """
    m = len(A)      # restricciones
    n = len(c)      # variables originales

    # Construcción de tableau:
    # [ A | I | b ]
    # [ -c| 0 | 0 ]  (fila objetivo para maximización)
    width = n + m + 1
    T = [[0.0] * width for _ in range(m + 1)]

    # Filas de restricciones
    for i in range(m):
        for j in range(n):
            T[i][j] = A[i][j]
        T[i][n + i] = 1.0  # holgura
        T[i][-1] = b[i]
```

Captura 20: 1 (de 5). Implementación didáctica del método Simplex para un problema de maximización con restricciones del tipo $Ax \leq b$ y $x \geq 0$. En este tramo se define la función simplex_max y se construye el tableau inicial incorporando variables de holgura. Desarrollado en Online IDE Pro. Código fuente de la autora

```python
    # Fila objetivo
    for j in range(n):
        T[m][j] = -c[j]
    T[m][-1] = 0.0

    # Variables básicas iniciales: holguras
    basis = [n + i for i in range(m)]

    def pivot(pivot_row: int, pivot_col: int):
        pv = T[pivot_row][pivot_col]
        # Normalizar fila pivote
        for j in range(width):
            T[pivot_row][j] /= pv
        # Eliminar columna pivote en otras filas
        for i in range(m + 1):
            if i == pivot_row:
                continue
            factor = T[i][pivot_col]
            if abs(factor) < tol:
                continue
            for j in range(width):
                T[i][j] -= factor * T[pivot_row][j]
```

Captura 21: (2 de 5, continuación). Se inicializa la base factible (holguras) y se define la operación de pivoteo: normalización de la fila pivote y eliminación de la columna pivote en el resto de filas para mantener la equivalencia del sistema. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
    for _ in range(max_iter):
        # Columna entrante: coeficiente más negativo en fila objetivo (mejora para max)
        obj_row = T[m]
        entering = min(range(n + m), key=lambda j: obj_row[j])
        if obj_row[entering] >= -tol:
            break  # óptimo (según convención del tableau)

        # Fila saliente: prueba de razón mínima b_i / a_i_entering con a_i_entering > 0
        ratios: List[Tuple[float, int]] = []
        for i in range(m):
            a = T[i][entering]
            if a > tol:
                ratios.append((T[i][-1] / a, i))
        if not ratios:
            raise ValueError("Problema no acotado en esta formulación (no se encontró variable saliente).")

        _, leaving_row = min(ratios, key=lambda t: t[0])

        pivot(leaving_row, entering)
        basis[leaving_row] = entering
```

Captura 22: (3 de 5, continuación). Bucle principal de iteraciones del Simplex: selección de variable entrante (coeficiente más negativo en la fila objetivo, según la convención adoptada) y elección de fila saliente mediante la prueba de razón mínima. Incluye control de casos no acotados si no existe variable saliente. Desarrollado en Online IDE Pro. Código fuente de la autora

```
    # Extraer solución
    x = [0.0] * n
    for i in range(m):
        if basis[i] < n:
            x[basis[i]] = T[i][-1]
    z = T[m][-1]
    return x, z

def is_feasible(A: List[List[float]], b: List[float], x: List[float], tol: float = 1e-7) -> bool:
    if any(xi < -tol for xi in x):
        return False
    for row, bi in zip(A, b):
        if sum(aij * xj for aij, xj in zip(row, x)) > bi + tol:
            return False
    return True

def run_demo() -> None:
    # Ejemplo pequeño (2 variables, 2 restricciones):
    # max z = 3x1 + 2x2
    # s.a. x1 + x2 <= 4
    #      2x1 + x2 <= 5
    #      x>=0
    A = [[1.0, 1.0],
         [2.0, 1.0]]
    b = [4.0, 5.0]
    c = [3.0, 2.0]
```

Captura 23: (4 de 5, continuación). Extracción de la solución a partir de la base final: se reconstruyen los valores de x desde el tableau y se obtiene el valor óptimo z. Se incorpora además una función is_feasible para verificar factibilidad numérica con tolerancia. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
    x, z = simplex_max(A, b, c, max_iter=30)
    print("Solución x:", x)
    print("Óptimo z:", z)

    # Pruebas simples: solución esperada x=(1,3), z=9
    assert is_feasible(A, b, x)
    assert abs(x[0] - 1.0) < 1e-6 and abs(x[1] - 3.0) < 1e-6
    assert abs(z - 9.0) < 1e-6

if __name__ == "__main__":
    run_demo()
```

Captura 24: (5 de 5, continuación). Función run_demo() con una instancia pequeña (2 variables, 2 restricciones) para ejecutar simplex_max, imprimir la solución y validar el resultado esperado mediante asserts (por ejemplo, x=(1,3) y z=9). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`simplex_max(...)`	Implementa Simplex por tableau para maximización con restricciones $\leq$ y $x \geq 0$, asumiendo $b \geq 0$ para base inicial con holguras.
Construcción de `T`	Arma el tableau $[A \mid I \mid b]$ y la fila objetivo $[-c \mid 0 \mid 0]$, acorde a la convención usada para elegir mejora.
`basis`	Mantiene el índice de la variable básica por restricción; inicia con holguras, que forman una SBF.
`pivot(...)`	Ejecuta el pivoteo: normaliza la fila pivote y elimina la columna pivote del resto mediante operaciones elementales.
Selección `entering`	Escoge la columna con coeficiente más negativo en la fila objetivo; si no hay negativos, se declara optimalidad básica.
Prueba de razón	Selecciona la fila saliente minimizando $b_i / a_{i,enter}$ con $a_{i,enter} > 0$, preservando factibilidad.
Corte `max_iter`	Evita loops en implementaciones didácticas; no reemplaza reglas anti-ciclado ni tratamiento de degeneración.
Extracción de `x`	Reconstruye la solución: las variables básicas originales toman el valor RHS de su fila; el resto queda en 0.
`assert`	Valida factibilidad y coincide con el óptimo esperado del ejemplo, controlando errores del pivoteo.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

3.4 Programación entera (PE/PLI): variables binarias, motivación y branch-and-bound

La programación lineal entera restringe algunas variables, o todas, a valores enteros. El caso binario x {0,1} modela decisiones de selección. A diferencia de la programación lineal continua, la PLI suele ser computacionalmente más difícil. Una técnica común consiste en considerar la relajación lineal, que reemplaza x {0,1} por 0 ≤ x ≤ 1. En problemas de maximización, esta relajación proporciona una cota superior para el valor óptimo y sirve como guía para métodos como branch and bound. Este enfoque divide el espacio de búsqueda mediante fijaciones o ramificaciones de variables y descarta subárboles cuando las cotas indican que no pueden mejorar la mejor solución encontrada (Nemhauser & Wolsey, 1988). En este contexto se utiliza un enfoque base de enumeración exhaustiva controlada para instancias pequeñas, lo que permite comparar el resultado entero con el obtenido en la relajación fraccional del problema de mochila binaria y observar la posible brecha entre ambos (Nemhauser & Wolsey, 1988).

Énfasis

Definiciones clave

- **Variable binaria:** variable entera restringida a $\{0, 1\}$, típica para decisiones "seleccionar/no seleccionar".
- **Relajación lineal:** versión del problema entero donde integridad se reemplaza por cotas continuas (p. ej., $0 \leq x \leq 1$).

- **Branch-and-bound (noción):** esquema de búsqueda que ramifica por decisiones y poda usando cotas derivadas de relajaciones.
- **Brecha de relajación:** diferencia entre el óptimo entero y el óptimo de la relajación; cuantifica pérdida por permitir fraccionamiento.

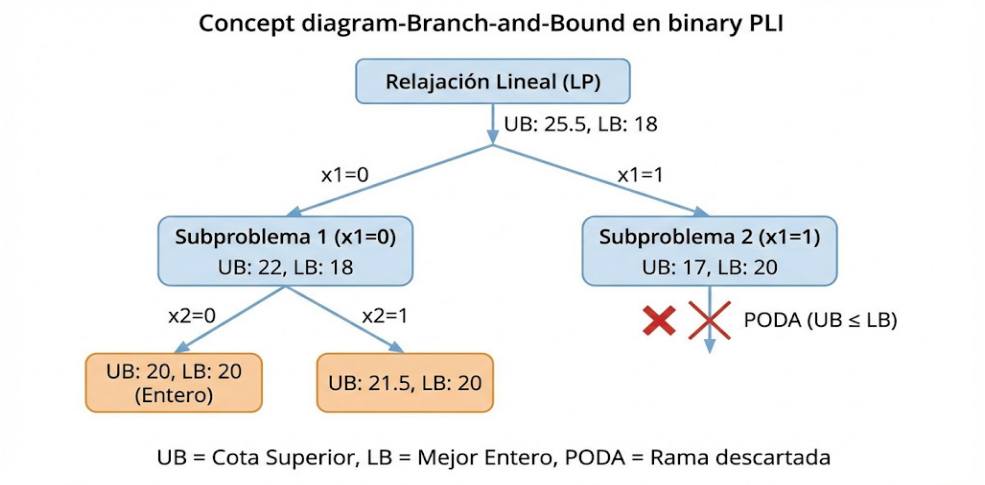


Ilustración 10: Programación Entera: esquema de Branch and Bound con cotas y poda. Se representa el árbol de búsqueda donde se resuelve una relajación lineal (LP) para obtener una cota superior (UB) y se compara con la mejor solución entera (LB), descartando ramas mediante poda cuando no pueden mejorar el óptimo. Esto conecta PL con optimización entera. Imagen creada usando herramientas IA con información proporcionada por la autora.

3.4.1 Ejemplo 4

La mochila 0–1 es un caso representativo de programación lineal entera binaria: busca maximizar el valor total respetando una restricción de capacidad. En instancias pequeñas, una enumeración exhaustiva (2^n combinaciones posibles) funciona como un baseline correcto y permite comparar el resultado con una relajación fraccional, es decir, la mochila fraccionaria, que actúa como una cota superior conceptual (Nemhauser & Wolsey, 1988).

```python
# Ejemplo 4: Mochila 0-1 (binaria) por enumeración exhaustiva y comparación con relajación fraccional
# max sum(v_i x_i) s.a. sum(w_i x_i) <= W, x_i in {0,1}

from typing import List, Tuple

def solve_knapsack_binary_bruteforce(weights: List[int], values: List[int], W: int) -> Tuple[int, List[int]]:
    n = len(weights)
    best_value = -1
    best_x = [0] * n
    for mask in range(1 << n):
        total_w = 0
        total_v = 0
        x = [0] * n
        for i in range(n):
            if (mask >> i) & 1:
                total_w += weights[i]
                if total_w > W:
                    break
                total_v += values[i]
                x[i] = 1
        else:
            if total_v > best_value:
                best_value = total_v
                best_x = x
    return best_value, best_x
```

Captura 25: (parte 1/3). Implementación de una mochila 0–1 (binaria) mediante enumeración exhaustiva (búsqueda por fuerza bruta) recorriendo todas las combinaciones posibles de inclusión/exclusión de ítems (máscara de bits). El código calcula el valor total y descarta tempranamente combinaciones que exceden la capacidad W, conservando la mejor solución entera encontrada. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def fractional_knapsack_upper_bound(weights: List[int], values: List[int], W: int) -> float:
    # Relajación fraccional para mochila: ordenar por densidad v/w (cota superior para 0-1)
    items = sorted([(v / w, w, v) for w, v in zip(weights, values)], reverse=True)
    remaining = W
    bound = 0.0
    for dens, w, v in items:
        if remaining <= 0:
            break
        take = min(remaining, w)
        bound += dens * take
        remaining -= take
    return bound

def check_solution(weights: List[int], values: List[int], W: int, x: List[int]) -> Tuple[int, int]:
    total_w = sum(w * xi for w, xi in zip(weights, x))
    total_v = sum(v * xi for v, xi in zip(values, x))
    assert all(xi in (0, 1) for xi in x)
    assert total_w <= W
    return total_w, total_v

def run_demo() -> None:
    weights = [2, 3, 4, 5, 9, 7]
    values  = [3, 4, 5, 8, 10, 7]
    W = 15

    opt_v, opt_x = solve_knapsack_binary_bruteforce(weights, values, W)
    tw, tv = check_solution(weights, values, W, opt_x)

    ub = fractional_knapsack_upper_bound(weights, values, W)

    print("Óptimo entero (baseline exhaustivo):")
    print("x =", opt_x)
    print("peso total =", tw, "valor total =", tv)

    print("\nCota superior por relajación fraccional (mochila fraccional):")
    print("UB =", round(ub, 4))
```

Captura 26: (parte 2/3). Definición de una cota superior para la mochila 0–1 a partir de la relajación fraccionaria: se ordenan ítems por densidad valor/peso y se permite "llenar" la capacidad con fracciones para estimar un upper bound del óptimo entero. Además, se incluye una función de verificación para comprobar que una solución binaria cumple restricciones y calcula (peso, valor). Desarrollado en Online IDE Pro. Código fuente de la autora.

```
    # Pruebas simples
    assert tv == opt_v
    assert ub + 1e-9 >= opt_v  # la relajación no puede dar menos que el óptimo entero
    # Validación adicional: el óptimo entero no supera la suma de todos los valores
    assert opt_v <= sum(values)

if __name__ == "__main__":
    run_demo()
```

Captura 27: (parte 3/3). Programa principal (run_demo) que ejecuta: (a) el óptimo entero por enumeración exhaustiva como baseline, (b) la cota superior por relajación fraccionaria, y (c) pruebas simples (asserts) que validan que la cota superior no puede ser menor que el óptimo entero y que los resultados son coherentes con la capacidad. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`solve_knapsack_binary_bruteforce`	Recorre todos los subconjuntos (máscaras) y mantiene el mejor valor factible, garantizando optimalidad para n pequeño.
Corte por peso `if total_w > W: break`	Poda intra-iteración: evita seguir sumando si ya violó la capacidad.
`fractional_knapsack_upper_bound`	Calcula la relajación fraccional (mochila fraccional) ordenando por densidad v/w; funciona como cota superior del 0–1.
`check_solution`	Verifica integridad $x_i \in \{0, 1\}$ y factibilidad por capacidad; devuelve métricas básicas (peso, valor).
Impresiones	Reportan solución entera y la cota superior, habilitando discusión de brecha.
`assert ub >= opt_v`	Formaliza que la relajación produce una cota superior (en maximización) para comparar con el óptimo entero.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

3.5 Problema de transporte

El problema de transporte distribuye oferta desde orígenes i hacia destinos j, minimizando un costo c con variables x ≥ 0. Se modela como un problema de programación lineal que busca minimizar Σ , c x , sujeto a restricciones de oferta Σ x = s y de demanda Σ x = d . Si Σ s ≠ Σ d , el modelo se balancea agregando un origen o destino ficticio con un costo apropiado, a menudo 0, para igualar los totales y mantener la factibilidad estructural del problema (Dantzig, 1963; Bertsimas & Tsitsiklis, 1997). En este subtema se resuelve un caso pequeño mediante enumeración acotada, un enfoque adecuado únicamente para instancias de tamaño reducido.

Énfasis

Definiciones clave

- **Variable de envío x_{ij}:** cantidad transportada desde el origen i al destino j, continua y no negativa.
- **Restricción de oferta:** ecuación que fija el total enviado por un origen al valor de su oferta.
- **Restricción de demanda:** ecuación que fija el total recibido por un destino al valor de su demanda.

- **Balanceo:** transformación para igualar oferta total y demanda total añadiendo nodo ficticio.
- **Conservación de flujo:** propiedad de igualdad de sumas por filas/columnas que refleja cumplimiento de oferta/demanda.

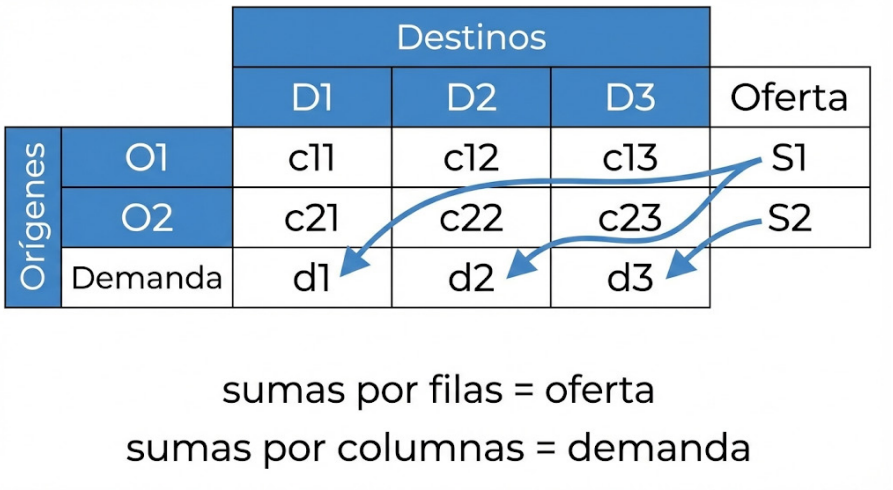


Ilustración 11: Problema de transporte: matriz Origen–Destino con oferta y demanda. La imagen presenta el modelo clásico de transporte: costos c_{ij} entre orígenes y destinos, con restricciones de conservación (sumas por filas = oferta, sumas por columnas = demanda). Se relaciona con PL aplicada a logística y asignación de recursos. Imagen creada usando herramientas IA con información proporcionada por la autora.

3.5.1 Ejemplo 5

Para un problema de transporte 2×3 con ofertas y demandas enteras pequeñas, es posible enumerar asignaciones factibles mediante bucles anidados que respeten la conservación de flujo. Este procedimiento permite validar el modelado, verificando que se cumplan las restricciones por filas y columnas, y comprobar el costo mínimo esperado en un caso controlado (Dantzig, 1963).

```python
# Ejemplo 5: Problema de transporte 2x3 (didáctico) por enumeración acotada
# Minimizar costo sum(c_ij x_ij)
# s.a. sum_j x_ij = oferta_i
#      sum_i x_ij = demanda_j
#      x_ij >= 0 enteras (aquí se enumera en enteros por ser ejemplo pequeño)

from typing import List, Tuple, Optional

def transport_min_cost_2x3(cost: List[List[int]], supply: List[int], demand: List[int]) -> Tuple[int, List[List[int]]]:
    assert len(cost) == 2 and len(cost[0]) == 3 and len(cost[1]) == 3
    assert len(supply) == 2 and len(demand) == 3
    assert sum(supply) == sum(demand), "El ejemplo requiere balanceado (oferta total = demanda total)."

    best_cost: Optional[int] = None
    best_x: List[List[int]] = [[0]*3 for _ in range(2)]

    s1, s2 = supply
    d1, d2, d3 = demand

    # Variables:
    # x11 x12 x13 (fila 1)
    # x21 x22 x23 (fila 2)
    # Restricciones:
    # x11 + x12 + x13 = s1
    # x21 + x22 + x23 = s2
    # x11 + x21 = d1
    # x12 + x22 = d2
    # x13 + x23 = d3
    #
    # Enumeración acotada:
    # elegir x11, x12; entonces x13 queda por oferta fila 1
    # y por demandas se determinan x21, x22, x23.
```

Captura 28: Definición de un ejemplo didáctico del problema de transporte 2×3: se modela la minimización del costo total $\sum c_{ij}x_{ij}$ con restricciones de oferta por filas y demanda por columnas, incorporando la condición de balanceo ($\sum$ oferta = $\sum$ demanda) y describiendo las variables x_{ij} y sus restricciones. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    for x11 in range(d1 + 1):
        x21 = d1 - x11
        if x21 < 0:
            continue

        for x12 in range(d2 + 1):
            x22 = d2 - x12
            if x22 < 0:
                continue

            x13 = s1 - (x11 + x12)
            if x13 < 0:
                continue

            x23 = d3 - x13
            if x23 < 0:
                continue

            # Verificar oferta fila 2:
            if x21 + x22 + x23 != s2:
                continue

            x = [
                [x11, x12, x13],
                [x21, x22, x23],
            ]

            # Costo
            total = 0
            for i in range(2):
                for j in range(3):
                    total += cost[i][j] * x[i][j]

            if best_cost is None or total < best_cost:
                best_cost = total
                best_x = x
```

Captura 29: Implementación de una enumeración acotada para hallar la asignación de mínimo costo: se recorren valores posibles para algunas variables (por ejemplo x_{11}, x_{12}) y el resto se deduce por conservación de flujo (ofertas/demandas), descartando combinaciones inviables (valores negativos o violación de oferta total por fila). Se calcula el costo y se actualiza la mejor solución encontrada. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
    assert best_cost is not None, "No se encontró solución factible (revisar datos)."
    return best_cost, best_x

def check_transport(x: List[List[int]], supply: List[int], demand: List[int]) -> None:
    # Conservación de flujo por filas (ofertas)
    for i in range(len(supply)):
        assert sum(x[i]) == supply[i]
    # Conservación de flujo por columnas (demandas)
    for j in range(len(demand)):
        assert sum(x[i][j] for i in range(len(supply))) == demand[j]
    # No negatividad
    assert all(x[i][j] >= 0 for i in range(len(supply)) for j in range(len(demand)))

def run_demo() -> None:
    cost = [
        [2, 3, 1],  # origen 1 -> destinos 1..3
        [5, 4, 8],  # origen 2 -> destinos 1..3
    ]
    supply = [20, 30]
    demand = [10, 10, 30]  # balanceado: 50=50

    best_cost, best_x = transport_min_cost_2x3(cost, supply, demand)

    print("Asignación óptima (x_ij):")
    for row in best_x:
        print(row)
    print("Costo mínimo:", best_cost)

    # Validaciones
    check_transport(best_x, supply, demand)

    # Pruebas simples: verificar costo consistente con la asignación reportada
    calc = sum(cost[i][j] * best_x[i][j] for i in range(2) for j in range(3))
    assert calc == best_cost

if __name__ == "__main__":
    run_demo()
```

Captura 30: Cierre del ejemplo: se define una función de verificación de factibilidad (sumas por filas = oferta, sumas por columnas = demanda y no negatividad), se instancia un caso pequeño con matriz de costos, ofertas y demandas, se imprime la asignación óptima y se agregan pruebas (asserts) para validar consistencia del costo y restricciones del modelo. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`transport_min_cost_2x3`	Resuelve un transporte 2×3 por enumeración, aprovechando que las restricciones determinan variables restantes una vez fijadas algunas.
Asunción balanceada	Se exige $\sum s_i = \sum d_j$; si no se cumple, el modelado requiere balanceo previo.
Elección de `x11`, `x12`	Define dos variables libres acotadas por demandas; el resto se deduce por conservación de flujo.
Cálculo de `x13`, `x23`	Usa oferta de fila 1 y demanda de columna 3 para determinar valores, evitando enumeración completa.
Chequeo fila 2	Garantiza que la segunda oferta se cumpla exactamente: condición de factibilidad global.
`check_transport`	Verifica conservación por filas y columnas y no negatividad; formaliza las restricciones del modelo.
`assert` de costo	Recalcula costo para asegurar consistencia entre la asignación y el valor óptimo informado.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

3.6 Uso de bibliotecas de optimización en Python

En la práctica, la resolución de PL/PLI se realiza con solvers especializados. En Python, es común usar:

- SciPy para PL continuas mediante la interfaz linprog,
- PuLP como modelador que delega la resolución en distintos solvers de LP/MIP, y
- OR-Tools, que ofrece una suite amplia de herramientas de optimización.

En este subtema, su uso es opcional y se encapsula en try/except para no interrumpir el proceso de ejecución si el solver no está disponible. Entre las buenas prácticas mínimas se incluyen verificar el estado de la solución, validar la factibilidad considerando tolerancias numéricas, revisar el escalado del modelo y comparar los resultados con soluciones didácticas o baselines conocidos (Virtanen et al., 2020; SciPy Documentation, s. f.).

Énfasis

Definiciones clave

- **Solver:** software que implementa algoritmos numéricos para encontrar soluciones óptimas de PL/PLI y reporta estados/diagnósticos.
- **Estado de solución:** indicador del resultado (óptimo, infactible, no acotado, límite de iteraciones, etc.).
- **Tolerancia numérica:** umbral para tratar comparaciones y restricciones debido a aritmética de punto flotante.
- **Validación post-solve:** verificación independiente (factibilidad, objetivo, restricciones críticas) sobre la solución reportada.

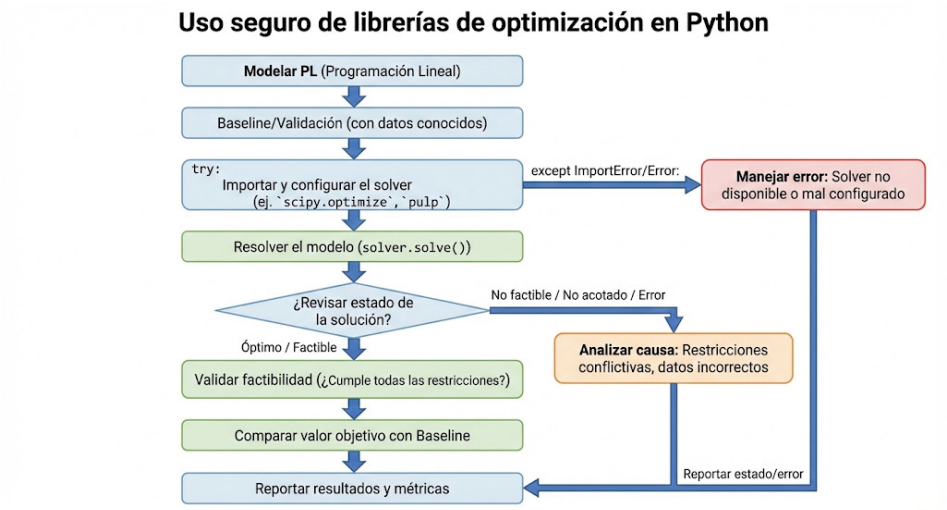


Ilustración 12: Flujo de trabajo para uso seguro de librerías de optimización en Python. Diagrama de buenas prácticas: modelar, validar con casos conocidos, configurar el solver, resolver, verificar factibilidad/estado, comparar con baseline y reportar resultados, incluyendo manejo de errores. Se conecta con la implementación práctica de modelos de optimización en entornos reales. Imagen creada usando herramientas IA con información proporcionada por la autora.

3.6.1 Ejemplo 6

Un patrón seguro consiste en resolver primero un problema de programación lineal con un baseline (por ejemplo, verificación directa o un Simplex didáctico) y, si el entorno dispone de una librería especializada, resolverlo nuevamente con esa herramienta y comparar los resultados. Este procedimiento ayuda a reforzar el modelado correcto, la interpretación del estado de la solución y el control de tolerancias numéricas (Virtanen et al., 2020).

```python
# Ejemplo 6: Plantilla opcional con SciPy linprog (sin romper ejecutabilidad)
# Se resuelve el PL del Ejemplo A:
# max 3x1 + 2x2
# s.a. x1 + x2 <= 4
#      2x1 + x2 <= 5
#      x>=0

from typing import List, Tuple

def simplex_max_minimal(A: List[List[float]], b: List[float], c: List[float], max_iter: int = 50, tol: float = 1e-9):
    # Implementación mínima (idéntica en espíritu al subtema 1.3, compactada para este script)
    m, n = len(A), len(c)
    width = n + m + 1
    T = [[0.0] * width for _ in range(m + 1)]
    for i in range(m):
        for j in range(n):
            T[i][j] = A[i][j]
        T[i][n + i] = 1.0
        T[i][-1] = b[i]
    for j in range(n):
        T[m][j] = -c[j]
    basis = [n + i for i in range(m)]
Press Ctrl K to generate code
    def pivot(r: int, col: int):
        pv = T[r][col]
        for j in range(width):
            T[r][j] /= pv
        for i in range(m + 1):
            if i == r:
                continue
            factor = T[i][col]
            if abs(factor) < tol:
                continue
            for j in range(width):
                T[i][j] -= factor * T[r][j]
```

Captura 31: Implementación mínima y didáctica del método Simplex para un problema de maximización con restricciones del tipo $Ax \leq b$ y $x \geq 0$. Se construye el tableau con variables de holgura y se define la operación de pivoteo (normalización de la fila pivote y eliminación en el resto). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    for _ in range(max_iter):
        entering = min(range(n + m), key=lambda j: T[m][j])
        if T[m][entering] >= -tol:
            break
        ratios: List[Tuple[float, int]] = []
        for i in range(m):
            a = T[i][entering]
            if a > tol:
                ratios.append((T[i][-1] / a, i))
        if not ratios:
            raise ValueError("No acotado (según tableau mínimo).")
        _, r = min(ratios, key=lambda t: t[0])
        pivot(r, entering)
        basis[r] = entering

    x = [0.0] * n
    for i in range(m):
        if basis[i] < n:
            x[basis[i]] = T[i][-1]
    z = T[m][-1]
    return x, z

def is_feasible(A: List[List[float]], b: List[float], x: List[float], tol: float = 1e-7) -> bool:
    if any(xi < -tol for xi in x):
        return False
    for row, bi in zip(A, b):
        if sum(aij * xj for aij, xj in zip(row, x)) > bi + tol:
            return False
    return True
```

Captura 32: (continuación). Ciclo iterativo del Simplex: selección de variable entrante (coeficiente más negativo en la fila objetivo) y variable saliente mediante la prueba de razón mínima, actualización de la base y extracción de la solución (x, z). Incluye una función de verificación de factibilidad para validar el resultado numérico. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def run_demo() -> None:
    A = [[1.0, 1.0],
         [2.0, 1.0]]
    b = [4.0, 5.0]
    c = [3.0, 2.0]

    x_base, z_base = simplex_max_minimal(A, b, c)
    print("Baseline (simplex didáctico): x =", x_base, "z =", z_base)
    assert is_feasible(A, b, x_base)
    assert abs(z_base - (3*x_base[0] + 2*x_base[1])) < 1e-6

    # Variante opcional: SciPy linprog (si está instalada)
    try:
        from scipy.optimize import linprog
        # linprog minimiza, por lo que se minimiza -c^T x para maximizar c^T x
        res = linprog(c=[-ci for ci in c], A_ub=A, b_ub=b, bounds=[(0, None), (0, None)], method="highs")

        print("\nSciPy linprog disponible.")
        print("status:", res.status, "message:", res.message)
        if res.success:
            x_scipy = res.x.tolist()
            z_scipy = -res.fun
            print("SciPy: x =", x_scipy, "z =", z_scipy)

            # Validaciones y comparación (tolerancia)
            assert is_feasible(A, b, x_scipy)
            assert abs(z_scipy - z_base) < 1e-6
        else:
            print("SciPy no reportó solución óptima en este caso (revisar estado).")

    except Exception as e:
        print("\nSciPy no está disponible o falló la importación/ejecución.")
        print("Detalle:", str(e))
        print("El script continúa con el baseline didáctico sin fallar.")

if __name__ == "__main__":
    run_demo()
```

Captura 33: (continuación). Rutina de prueba (run_demo) que ejecuta el Simplex didáctico como baseline y, de forma opcional, intenta resolver el mismo modelo con scipy.optimize.linprog (si está disponible), comparando soluciones y manejando errores de importación/ejecución sin interrumpir el script. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Baseline `simplex_max_minimal`	Proporciona un resultado independiente para comparar, evitando dependencia de librerías.
`try/except` alrededor de SciPy	Garantiza ejecutabilidad: si SciPy no está instalada, el script informa y continúa.
Conversión max→min	`linprog` minimiza; se transforma max $c^\top x$ en min $-c^\top x$ para mantener equivalencia.
`method="highs"`	Selecciona el backend moderno de `linprog` cuando está disponible, y SciPy retorna `status/success`. docs.scipy.org
Chequeo `res.success`	Evita usar resultados si el solver no logró óptimo; es parte de lectura segura del estado.
`assert` de comparación	Verifica que el óptimo del solver coincide con el baseline (dentro de tolerancia), controlando discrepancias numéricas.
`is_feasible`	Realiza validación post-solve sobre restricciones y no negatividad, independiente del solver.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc . Para el ejemplo 6 se recomienda usar una IDE con Visual Studio Code dado que se deben importar librerías específicas.

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 2: Programación Lineal–Laboratorio Interactivo.**

4. Algoritmos de aproximación y complejidad computacional

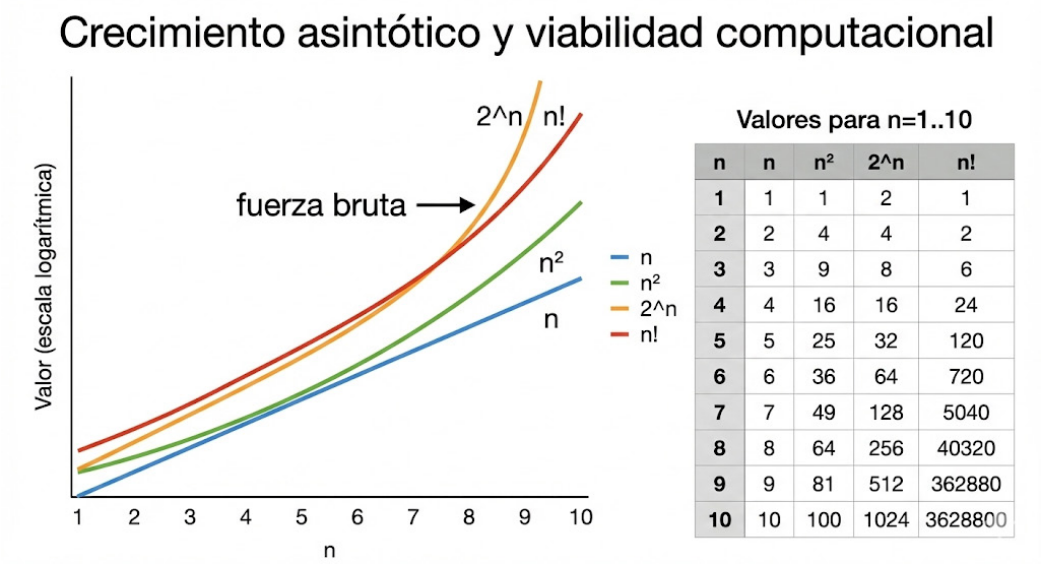
4.1 Complejidad computacional: modelos, tiempo polinómico, O(·) y noción operativa de "eficiente"

En análisis de algoritmos, el costo se modela como una función del tamaño de entrada n, usando cotas asintóticas (por ejemplo, $O(\cdot)$) para describir crecimiento dominante. En complejidad computacional (enfoque clásico), se fija un modelo de cómputo como la máquina de Turing y se define el tiempo como el número de pasos elementales, estudiando clases por restricciones de tiempo (Sipser, 2012). La noción operativa de "eficiente" se asocia, en este marco, a tiempo polinómico n^k, porque polinomios escalan de manera controlable frente a funciones exponenciales o factoriales, que crecen demasiado rápido incluso para n moderados (Papadimitriou, 1994).

Énfasis

Definiciones clave

- **Tamaño de entrada:** Medida formal n que cuantifica la instancia (por ejemplo, cantidad de variables, vértices, cláusulas), usada para expresar costos.
- **Notación $O(\cdot)$:** Cota superior asintótica que acota el crecimiento de una función de costo hasta factores constantes.
- **Tiempo polinómico:** Conjunto de algoritmos cuyo tiempo está acotado por $O(n^k)$ para alguna constante k sobre el tamaño de entrada.
- **Tiempo exponencial:** Tiempo acotado por $O(c^n)$ con $c > 1$, típico de enumeraciones exhaustivas de subconjuntos o asignaciones.



n	n	n²	2^n	n!
1	1	1	2	1
2	2	4	4	2
3	3	9	8	6
4	4	16	16	24
5	5	25	32	120
6	6	36	64	720
7	7	49	128	5040
8	8	64	256	40320
9	9	81	512	362880
10	10	100	1024	3628800

Ilustración 13: Crecimiento asintótico y viabilidad computacional. La figura compara cómo crecen distintas funciones de complejidad (lineal, cuadrática, exponencial y factorial) y muestra por qué la fuerza bruta se vuelve inviable cuando n aumenta. Se vincula con el módulo al fundamentar la necesidad de algoritmos voraces, programación dinámica y técnicas de optimización para evitar explosiones combinatorias. Imagen creada usando herramientas IA con información proporcionada por la autora.

4.4.1 Ejemplo 1

Enumeraciones exhaustivas suelen requerir recorrer 2^n subconjuntos o $n!$ permutaciones. El siguiente script imprime una tabla de crecimiento para visualizar por qué, aun con hardware moderno, instancias moderadas vuelven inviable el enfoque exacto por fuerza bruta (Papadimitriou, 1994).

ejemplo1.py

```python
# Ejemplo 1: Crecimiento combinatorio (2^n y n!)
# Ejecutable en Online IDE Pro (Python estándar)

import math

def growth_table(n_max: int = 12):
    rows = []
    for n in range(1, n_max + 1):
        subsets = 2 ** n
        perms = math.factorial(n)
        rows.append((n, subsets, perms))
    return rows

def main():
    rows = growth_table(12)
    print("n | 2^n (subconjuntos) | n! (permutaciones)")
    print("--|--------------------|------------------")
    for n, subsets, perms in rows:
        print(f"{n:>2} | {subsets:>17} | {perms:>16}")

    # Pruebas simples
    assert 2 ** 10 == 1024
    assert math.factorial(5) == 120
    print("\nOK: pruebas simples pasaron.")

if __name__ == "__main__":
    main()
```

Captura 34: (ejemplo1.py: crecimiento combinatorio 2^n y $n!$). El script genera e imprime una tabla que compara el crecimiento de 2^n (cantidad de subconjuntos de un conjunto de tamaño n) con $n!$ (cantidad de permutaciones de $n$elementos). Para ello, importa math y usa math.factorial(n). La función growth_table(n_max) recorre valores de n desde 1 hasta n_max, calcula ambos valores y los guarda como filas. Luego, main() llama a growth_table(12), muestra un encabezado, imprime cada fila formateada y ejecuta pruebas simples con assert (por ejemplo, $2^{10} = 1024$y $5! = 120$) para verificar cálculos básicos antes de finalizar. Desarrollado en Online IDE Pro (Python estándar). Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`growth_table`	Construye filas $(n, 2^n, n!)$ para $n = 1..n_{max}$.
`2 ** n`	Modela el conteo de subconjuntos (decisiones binarias por elemento).
`math.factorial(n)`	Modela el conteo de permutaciones (ordenamientos posibles).
Impresión tabular	Permite comparar magnitudes y detectar el "salto" exponencial/factorial.
`assert`	Verifica valores conocidos para validar la ejecución.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

4.2 Clases P y NP: decisión, verificación vs. resolución, certificados; relación $P \subseteq NP$

La teoría de complejidad formaliza problemas como lenguajes de decisión. La clase P contiene los problemas decidibles por un algoritmo determinista en tiempo polinómico. La clase NP contiene problemas cuyas instancias "sí" admiten certificados verificables en tiempo polinómico por una máquina determinista; equivalente a ser decidibles por una máquina no determinista en tiempo polinómico (Sipser, 2012). De aquí se deduce $P \subseteq NP$: si se puede resolver en tiempo polinómico, entonces también se puede verificar en tiempo polinómico usando como certificado la propia solución (Papadimitriou, 1994).

Énfasis

Definiciones clave

- **Problema de decisión:** Problema cuya salida es Sí/No, formalizable como lenguaje $L \subseteq \Sigma^*$.
- **Clase P:** Conjunto de lenguajes decidibles por máquina determinista en tiempo polinómico.
- **Clase NP:** Conjunto de lenguajes para los que existe un verificador determinista polinómico y certificados polinómicos.
- **Certificado:** Objeto que, dado junto con una instancia, permite verificar pertenencia al lenguaje en tiempo polinómico.

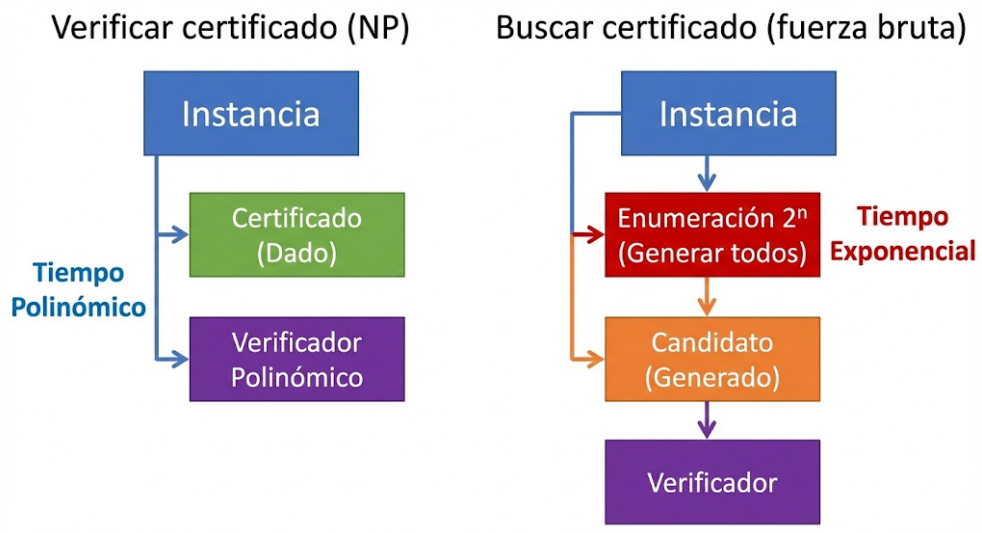


Ilustración 14: NP: verificar vs. buscar (fuerza bruta). El esquema contrasta la verificación en tiempo polinómico de un "certificado" (característico de NP) con la búsqueda por enumeración exponencial de candidatos (fuerza bruta). Se relaciona con el contenido al introducir la diferencia entre problemas "fáciles de verificar" y "difíciles de resolver" y su impacto en el diseño algorítmico. Imagen creada usando herramientas IA con información proporcionada por la autora.

4.2.1 Ejemplo 2

En Subset Sum (decisión), un certificado natural es el subconjunto (por ejemplo, una lista de índices). Verificar el certificado se realiza en tiempo $O(n)$. En cambio, "resolver" por fuerza bruta requiere explorar hasta 2^n subconjuntos, lo que crece exponencialmente (Garey & Johnson, 1979).

```python
# Ejemplo 2: P vs NP (verificación polinómica vs búsqueda exponencial)
# Problema: SUBSET SUM (decisión)
# Instancia: lista nums y objetivo target
# Certificado: subconjunto indicado por índices (lista de ints)

from typing import List, Optional, Tuple

def verify_subset_sum(nums: List[int], target: int, cert_indices: List[int]) -> bool:
    n = len(nums)
    if any(i < 0 or i >= n for i in cert_indices):
        return False
    s = sum(nums[i] for i in cert_indices)
    return s == target

def solve_subset_sum_bruteforce(nums: List[int], target: int, n_limit: int = 18) -> Optional[List[int]]:
    # Límite explícito para evitar explosión combinatoria
    n = len(nums)
    if n > n_limit:
        raise ValueError(f"Instancia demasiado grande para fuerza bruta: n={n} > {n_limit}")

    # Enumeración de subconjuntos por máscara de bits: 0..(2^n - 1)
    for mask in range(1 << n):
        s = 0
        cert = []
        for i in range(n):
            if (mask >> i) & 1:
                s += nums[i]
                cert.append(i)
        if s == target:
            return cert
    return None

def combinations_count(n: int) -> int:
    return 1 << n  # 2^n
```

Captura 35: (parte 1/2). Código en Python que modela el problema SUBSET SUM (decisión) para ilustrar la diferencia entre verificación (tipo NP) y búsqueda por fuerza bruta. Se define verify_subset_sum(nums, target, cert_indices) como verificador: recibe una "solución candidata" (certificado) expresada como índices de un subconjunto, valida que los índices sean correctos y comprueba si la suma coincide con target. Luego se implementa solve_subset_sum_bruteforce(...), que intenta encontrar un certificado enumerando todos los subconjuntos mediante una máscara de bits (de 0 a $2^n - 1$), acumulando la suma de los elementos seleccionados y devolviendo el primer subconjunto que alcanza el objetivo. El parámetro n_limit limita el tamaño para evitar explosión combinatoria. Finalmente, combinations_count(n) calcula 2^n, conectando el ejemplo con el crecimiento exponencial. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def main():
    nums = [3, 34, 4, 12, 5, 2]
    target = 9  # 4 + 5
    print("Instancia:", nums, "target=", target)
    print("Cantidad de subconjuntos a explorar (2^n):", combinations_count(len(nums)))

    cert = solve_subset_sum_bruteforce(nums, target, n_limit=18)
    print("Certificado hallado por fuerza bruta (indices):", cert)

    # Pruebas simples
    assert cert is not None
    assert verify_subset_sum(nums, target, cert) is True
    assert verify_subset_sum(nums, target, []) is False
    print("OK: verificación y resolución (instancia pequeña) correctas.")

if __name__ == "__main__":
    main()
```

Captura 36: (parte 2/2). Se completa el script con main(), donde se fija una instancia pequeña (nums y target) y se imprime la cantidad de subconjuntos a explorar (2^n). Luego se ejecuta la búsqueda por fuerza bruta para obtener un certificado (cert) y se valida el resultado usando el verificador verify_subset_sum. Los assert finales actúan como pruebas: confirman que el certificado hallado efectivamente verifica la suma objetivo y que un certificado vacío no satisface el objetivo en este caso. De esta manera, el ejemplo vincula el contenido teórico: verificar una solución es directo una vez dado el certificado, mientras que encontrar uno por enumeración puede requerir explorar exponencialmente muchos candidatos. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`verify_subset_sum`	Implementa verificación de certificado: valida índices y suma; costo $O(n)$.
`solve_subset_sum_bruteforce`	Busca certificado enumerando 2^n máscaras; costo exponencial.
`n_limit`	Límite explícito para mantener ejecución razonable en Online IDE Pro.
`combinations_count`	Calcula 2^n para evidenciar el tamaño del espacio de búsqueda.
`assert`	Contrasta verificación positiva (certificado hallado) y negativa (vacío).

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

4.3 Determinismo y no determinismo en tiempo polinómico: formulación operacional con verificación

Una máquina determinista define, para cada configuración, un único siguiente paso. Una máquina no determinista permite múltiples pasos posibles; acepta si existe alguna rama que conduce a aceptación dentro del límite temporal. La definición de NP mediante no determinismo polinómico es equivalente a la definición mediante verificación determinista polinómica: "adivinar" un certificado de tamaño polinómico y verificarlo en tiempo polinómico (Sipser, 2012). Esta equivalencia es central porque separa "existencia de prueba verificable" de "capacidad de encontrarla eficientemente" (Papadimitriou, 1994).

Énfasis

Definiciones clave

- **Algoritmo determinista:** Procedimiento que, para cada estado y entrada, define una transición única.
- **Algoritmo no determinista:** Procedimiento que, para un estado y entrada, permite un conjunto de transiciones; la aceptación se define por existencia de una rama aceptante.
- **Verificador polinómico:** Algoritmo determinista que decide (x, w) en tiempo polinómico, donde w es certificado para x.
- **NP como cómputo no determinista polinómico:** Clase de lenguajes decidibles por máquina no determinista en tiempo polinómico.

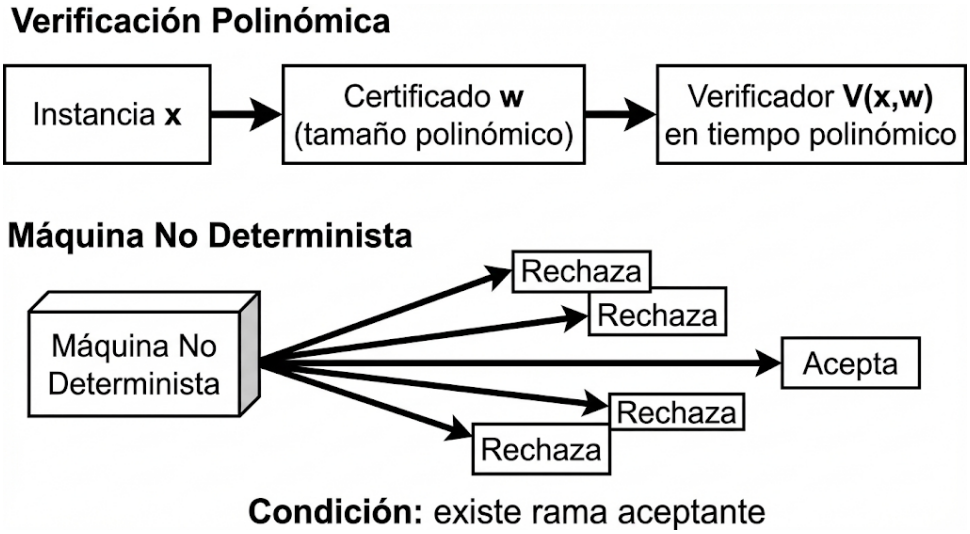


Ilustración 15: Verificación polinómica y máquina no determinista. La parte superior formaliza la idea de "certificado + verificador" en tiempo polinómico; la inferior ilustra el no determinismo como exploración de ramas, aceptando si existe una rama aceptante. Se conecta con el módulo al proporcionar la intuición de NP y la motivación de heurísticas, aproximaciones y relajaciones cuando la búsqueda exacta es costosa. Imagen creada usando herramientas IA con información proporcionada por la autora.

4.3.1 Ejemplo 3

Se ilustra la idea de "certificado + verificación" con un verificador para el problema Clique. Dado un grafo y un subconjunto de vértices propuesto como certificado, comprobar si todos los vértices del subconjunto están conectados entre sí (es decir, si forman una clique) puede hacerse en tiempo polinómico. En cambio, encontrar una clique sin conocerla de antemano suele requerir explorar muchos subconjuntos de vértices, lo que en el peor caso implica un número exponencial de posibilidades (Garey & Johnson, 1979).

```python
# Ejemplo 3: verificación (polinómica) de CLIQUE vs búsqueda por enumeración
# Certificado: lista de vértices C. Verificación: chequear todas las aristas internas.

from typing import List, Set, Tuple, Optional

Edge = Tuple[int, int]

def normalize_edges(edges: Set[Edge]) -> Set[Edge]:
    return {(min(u, v), max(u, v)) for (u, v) in edges if u != v}
Press Ctrl K to generate code
def verify_clique(n: int, edges: Set[Edge], cert: List[int], k: int) -> bool:
    if len(cert) != k:
        return False
    if any(v < 0 or v >= n for v in cert):
        return False
    s = set(cert)
    if len(s) != k:
        return False

    E = normalize_edges(edges)
    cert_list = list(s)
    # Verificación: para todo par en el certificado debe existir arista
    for i in range(k):
        for j in range(i + 1, k):
            u, v = cert_list[i], cert_list[j]
            if (min(u, v), max(u, v)) not in E:
                return False
    return True
```

Captura 37: (Ejemplo 3, parte 1/3). Código que define el problema CLIQUE en su versión de verificación (clase NP): recibe un certificado (lista de vértices) y comprueba en tiempo polinómico si ese conjunto forma una clique, verificando que para cada par de vértices del certificado existe la arista correspondiente. Incluye normalize_edges() para normalizar las aristas como pares ordenados y evitar duplicados/auto-aristas. Desarrollado en Online IDE Pro (Python estándar). Código fuente de la autora.

```python
def solve_clique_bruteforce(n: int, edges: Set[Edge], k: int, n_limit: int = 12) -> Optional[List[int]]:
    if n > n_limit:
        raise ValueError(f"Instancia demasiado grande para fuerza bruta: n={n} > {n_limit}")
    E = normalize_edges(edges)

    # Enumeración de subconjuntos por máscara, filtrando por tamaño k
    for mask in range(1 << n):
        if mask.bit_count() != k:
            continue
        cert = [i for i in range(n) if (mask >> i) & 1]
        if verify_clique(n, E, cert, k):
            return cert
    return None

def main():
    # Grafo pequeño (n=6)
    n = 6
    edges = {(0,1),(0,2),(1,2),(2,3),(3,4),(4,5)}  # clique {0,1,2} de tamaño 3
    k = 3

    cert = [0,1,2]
    print("Verificación de certificado clique", cert, "=>", verify_clique(n, edges, cert, k))

    found = solve_clique_bruteforce(n, edges, k, n_limit=12)
    print("Clique hallada por fuerza bruta:", found)
```

Captura 38: (Ejemplo 3, parte 2/3 – continuación). Código que implementa la búsqueda por enumeración (fuerza bruta) para encontrar una clique de tamaño k, recorriendo subconjuntos mediante máscara de bits y filtrando por bit_count() == k. Para cada candidato, reutiliza verify_clique() para validar el certificado; además, incorpora un límite de tamaño (n_limit) para evitar la explosión combinatoria en instancias grandes. Desarrollado en Online IDE Pro (Python estándar). Código fuente de la autora.

```python
    # Pruebas simples
    assert verify_clique(n, edges, [0,1,2], 3) is True
    assert verify_clique(n, edges, [0,2,3], 3) is False
    assert found is not None and verify_clique(n, edges, found, k) is True
    print("OK: pruebas simples pasaron.")

if __name__ == "__main__":
    main()
```

Captura 39: (Ejemplo 3, parte 3/3 – continuación). Bloque principal de demostración y pruebas: define un grafo pequeño, fija k, prueba primero un certificado conocido (por ejemplo, {0,1,2}), luego ejecuta la búsqueda exhaustiva y valida el resultado con assert (tanto casos verdaderos como falsos). Este cierre conecta la idea central P vs NP: verificar un certificado es directo, mientras que encontrarlo puede requerir enumeración exponencial. Desarrollado en Online IDE Pro (Python estándar). Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`verify_clique`	Chequea que el certificado tiene tamaño k y que cada par de vértices está conectado; costo $O(k^2)$.
`solve_clique_bruteforce`	Recorre subconjuntos de tamaño k mediante máscaras; en peor caso explora $\binom{n}{k}$.
`n_limit`	Evita ejecutar fuerza bruta para n grande.
Normalización de aristas	Asegura representación consistente $(u < v)$ para consultas.
`assert`	Valida verificación positiva/negativa y consistencia del hallazgo.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

4.4 NP-completitud: reducciones polinómicas, NP-duro vs NP-completo, consecuencias

Un problema A se reduce a B en tiempo polinómico si existe una transformación f computable en tiempo polinómico tal que x A f(x) B. Esto significa que cualquier instancia de A puede transformarse eficientemente en una instancia equivalente de B. Si B tiene un algoritmo polinómico y A ≤ B, entonces A también podría resolverse en tiempo polinómico, ya que primero se transforma la instancia y luego se aplica el algoritmo de B (Sipser, 2012). Un problema es NP-duro si todo problema en NP puede reducirse a él en tiempo polinómico. Es NP-completo si además pertenece a NP, es decir, si sus soluciones pueden verificarse en tiempo polinómico. La noción de NP-completitud se estableció con resultados fundacionales como la demostración de que SAT es NP-completo (Cook, 1971) y la identificación de 21 problemas NP-completos clásicos (Karp, 1972). Una consecuencia clave de esta teoría es que, si algún problema NP-completo tuviera un algoritmo polinómico, entonces P = NP (Garey & Johnson, 1979).

Definiciones clave

- **Reducción polinómica $\leq_p$:** Transformación computable en tiempo polinómico que preserva respuesta Sí/No entre problemas.
- **NP-duro:** Problema al que se reduce todo problema en NP (al menos tan difícil como NP).

› **NP-completo:** Problema que pertenece a NP y es NP-duro.

› **Consecuencia $P = NP$:** Si existe un algoritmo polinómico para algún problema NP-completo, entonces todos los problemas en NP tendrían algoritmo polinómico.

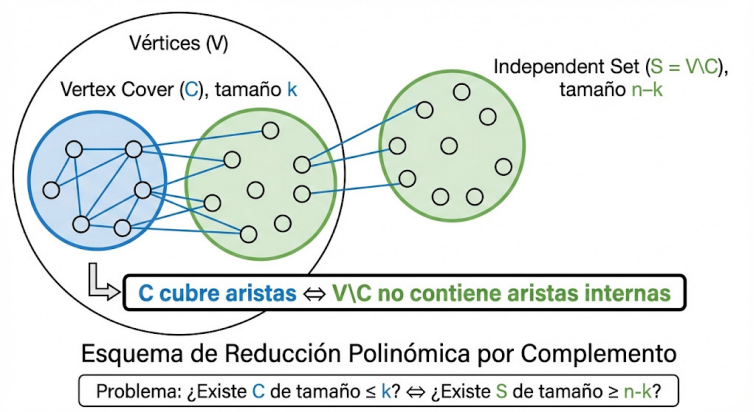


Ilustración 16: Reducción por complemento: Vertex Cover ↔ Independent Set. La figura muestra que un conjunto cubre aristas (Vertex Cover) si y solo si su complemento no contiene aristas internas (Independent Set), habilitando una transformación entre problemas. Se relaciona con el contenido al introducir reducciones polinómicas como herramienta para comparar dificultad y justificar por qué ciertas técnicas (aproximación/heurísticas) son necesarias. Imagen creada usando herramientas IA con información proporcionada por la autora.

4.4.1 Ejemplo 4

Una micro-reducción didáctica clásica es Vertex Cover ↔ Independent Set por complemento: un conjunto S es independiente si y solo si $V \setminus S$ cubre todas las aristas. Esto induce una transformación de parámetros $k \mapsto |V| - k$ (Garey & Johnson, 1979).

```python
# Ejemplo 4: micro-reducción didáctica Vertex Cover ↔ Independent Set (por complemento)
# Ejecutable en Online IDE Pro (Python estándar)

from typing import Set, Tuple, List
from itertools import combinations

Edge = Tuple[int, int]

def norm_edges(edges: Set[Edge]) -> Set[Edge]:
    return {(min(u,v), max(u,v)) for (u,v) in edges if u != v}

def is_vertex_cover(n: int, edges: Set[Edge], C: Set[int]) -> bool:
    E = norm_edges(edges)
    for (u, v) in E:
        if u not in C and v not in C:
            return False
    return True

def is_independent_set(n: int, edges: Set[Edge], S: Set[int]) -> bool:
    E = norm_edges(edges)
    for (u, v) in E:
        if u in S and v in S:
            return False
    return True
```

Captura 40: (parte 1/3). Micro-reducción didáctica Vertex Cover ↔ Independent Set: funciones de verificación. Se definen tipos y utilidades para trabajar con grafos no dirigidos. norm_edges normaliza aristas $(u\,v)$ para evitar duplicados por orden. is_vertex_cover verifica si un conjunto C cubre todas las aristas (para cada arista, al menos uno de sus extremos pertenece a C). is_independent_set verifica si un conjunto S es independiente (no existe ninguna arista con ambos extremos en S). Estas funciones implementan la idea central de la relación por complemento: si C cubre aristas, entonces $V \setminus C$ no contiene aristas internas. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def vc_to_is_k(n: int, k_vc: int) -> int:
    # Si existe VC de tamaño k, existe IS de tamaño n-k
    return n - k_vc

def is_to_vc_k(n: int, k_is: int) -> int:
    return n - k_is

def brute_exists_vc(n: int, edges: Set[Edge], k: int) -> bool:
    for r in range(k + 1):
        for comb in combinations(range(n), r):
            if is_vertex_cover(n, edges, set(comb)):
                return True
    return False

def brute_exists_is(n: int, edges: Set[Edge], k: int) -> bool:
    for r in range(k, n + 1):
        for comb in combinations(range(n), r):
            if is_independent_set(n, edges, set(comb)):
                return True
    return False
```

Captura 41: (parte 2/3). Transformación de tamaños y búsqueda por enumeración (fuerza bruta) en instancias pequeñas. Se codifica la correspondencia de tamaños: si existe un Vertex Cover de tamaño k, entonces existe un Independent Set de tamaño n–k (vc_to_is_k), y viceversa (is_to_vc_k). Luego se incluyen dos verificadores por fuerza bruta (brute_exists_vc y brute_exists_is) que enumeran combinaciones de vértices y verifican, respectivamente, si alguna combinación es un VC (de tamaño ≤ k) o un IS (de tamaño ≥ k). Esto permite mostrar de forma concreta la equivalencia en grafos pequeños. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def main():
    n = 5
    edges = {(0,1), (1,2), (2,3), (3,4)}  # camino
    edges = norm_edges(edges)

    k_vc = 2
    k_is = vc_to_is_k(n, k_vc)

    exists_vc = brute_exists_vc(n, edges, k_vc)
    exists_is = brute_exists_is(n, edges, k_is)

    print("Grafo (aristas):", sorted(edges))
    print(f"¿Existe Vertex Cover de tamaño ≤ {k_vc}? =>", exists_vc)
    print(f"Transformación: k_IS = n - k_VC = {k_is}")
    print(f"¿Existe Independent Set de tamaño ≥ {k_is}? =>", exists_is)

    # Pruebas simples (equivalencia en esta instancia)
    assert exists_vc == exists_is

    # Verificación de correspondencia certificado por complemento en un ejemplo concreto
    C = {1,3}               # vertex cover de tamaño 2
    S = set(range(n)) - C   # complemento => independent set de tamaño 3
    assert is_vertex_cover(n, edges, C) is True
    assert is_independent_set(n, edges, S) is True
    print("OK: reducción por complemento validada con asserts.")

if __name__ == "__main__":
    main()
```

Captura 42: (parte 3/3). Demostración con un ejemplo y validación con pruebas (assert). En main se construye un grafo simple (un camino con $n = 5$ vértices), se fija $k_{VC} = 2$ y se calcula el tamaño objetivo equivalente para Independent Set ($k_{IS} = n - k_{VC}$). Se evalúa por enumeración si existe VC de tamaño ≤ 2 y si existe IS de tamaño ≥ 3, y se verifica que ambas respuestas coinciden (assert exists_vc == exists_is). Finalmente, se muestra la relación por complemento en un caso concreto: dado un VC C, se define $S = V \setminus C$ y se comprueba que C es VC y S es IS mediante assert. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`is_vertex_cover`	Verifica que toda arista tenga al menos un extremo en C.
`is_independent_set`	Verifica que ninguna arista tenga ambos extremos en S.
`vc_to_is_k`	Implementa la transformación de parámetro $k_{IS} = n - k_{VC}$.
`brute_exists_vc / brute_exists_is`	Enumeran subconjuntos para decidir existencia en instancia pequeña.
Complemento `S = V \ C`	Construye el certificado correspondiente y valida equivalencia con `assert`.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

4.5 Ejemplos de problemas NP-completos

NP-completo agrupa problemas "representativos" en el sentido de que capturan la dificultad de la clase NP bajo reducciones polinómicas. El problema SAT fue el primero en demostrarse NP-completo (Cook, 1971), y la variante 3-SAT suele emplearse como punto de partida para construir reducciones hacia problemas de grafos y de optimización (Garey & Johnson, 1979). En áreas como IA y optimización se distingue entre versiones de decisión, por ejemplo "¿existe un tour de costo ≤ B?", y versiones de optimización, como "minimizar el costo". En general, la clasificación de NP-completitud se refiere a la versión de decisión del problema (Papadimitriou, 1994).

Definiciones clave

- **SAT:** Dada una fórmula booleana, decidir si existe una asignación que la hace verdadera.
- **3-SAT:** Variante de SAT donde cada cláusula tiene exactamente 3 literales.
- **Clique (decisión):** Dado un grafo G y k, decidir si existe un subcongrafo completo de tamaño k.
- **Vertex Cover (decisión):** Dado G y k, decidir si existe un conjunto de vértices de tamaño $\leq k$ que cubra todas las aristas.
- **TSP (decisión):** Dadas distancias y un umbral B, decidir si existe un tour Hamiltoniano de costo $\leq B$.

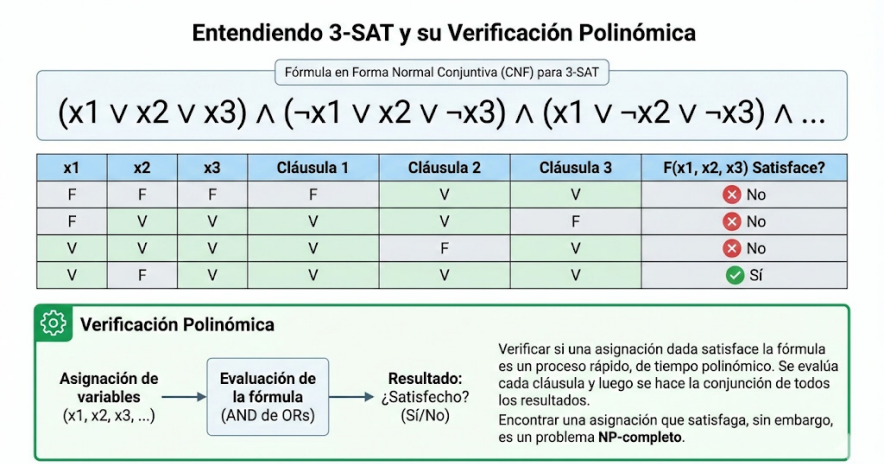


Ilustración 17: 3-SAT y verificación polinómica. La imagen representa una fórmula en CNF, evalúa cláusulas para asignaciones de variables y resume el proceso de verificación "rápida" de una solución candidata. Se vincula con el módulo al ejemplificar un problema NP-completo y reforzar el criterio de "verificar es polinómico, encontrar puede ser exponencial". Imagen creada usando herramientas IA con información proporcionada por la autora.

4.5.1 Ejemplo 5

3-SAT ilustra la estructura de NP. Un certificado consiste en una asignación de valores booleanos a las variables, y verificar que dicha asignación satisface una fórmula en CNF puede hacerse en tiempo polinómico respecto del tamaño de la fórmula. En cambio, resolver el problema por fuerza bruta implica revisar 2^n posibles asignaciones, por lo que en ejemplos didácticos suele limitarse a $n \leq 12$ (Sipser, 2012).

```python
# Ejemplo 5: 3-SAT (verificación) + resolución por fuerza bruta
# Representación CNF: lista de cláusulas; cada cláusula lista de literales (int)
# Literal: +i significa x_i=True; -i significa x_i=False  (i >= 1)

from typing import List, Optional, Dict

Clause = List[int]
CNF = List[Clause]

def verify_cnf(cnf: CNF, assignment: List[bool]) -> bool:
    # assignment[i-1] corresponde a x_i
    for clause in cnf:
        satisfied = False
        for lit in clause:
            var = abs(lit)
            val = assignment[var - 1]
            if lit < 0:
                val = not val
            if val:
                satisfied = True
                break
        if not satisfied:
            return False
    return True
```

Captura 43: (Ejemplo 5 — 3-SAT, parte 1/3). Código en Python que define la representación CNF (lista de cláusulas) y la función verify_cnf(cnf, assignment), que verifica en tiempo polinómico si una asignación booleana satisface la fórmula: recorre cada cláusula, evalúa sus literales (positivo = variable verdadera, negativo = variable negada) y exige que al menos un literal sea verdadero; si alguna cláusula queda falsa, retorna False, y si todas se satisfacen, retorna True. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def solve_cnf_bruteforce(cnf: CNF, n_vars: int, n_limit: int = 12) -> Optional[List[bool]]:
    if n_vars > n_limit:
        raise ValueError(f"Demasiadas variables para fuerza bruta: n_vars={n_vars} > {n_limit}")
    for mask in range(1 << n_vars):
        assignment = [bool((mask >> i) & 1) for i in range(n_vars)]
        if verify_cnf(cnf, assignment):
            return assignment
    return None

def main():
    # Caso satisfacible:
    # (x1 OR x2 OR x3) AND (¬x1 OR x2 OR ¬x3) AND (x1 OR ¬x2 OR x3)
    cnf_sat: CNF = [
        [ 1,  2,  3],
        [-1,  2, -3],
        [ 1, -2,  3],
    ]
    n_vars = 3
    sol = solve_cnf_bruteforce(cnf_sat, n_vars, n_limit=12)
    print("CNF satisfacible - asignación encontrada:", sol)

    # Caso insatisfacible:
    # (x1) AND (¬x1)
    cnf_unsat: CNF = [[1], [-1]]
    sol2 = solve_cnf_bruteforce(cnf_unsat, 1, n_limit=12)
    print("CNF insatisfacible - resultado:", sol2)
```

Captura 44: (Ejemplo 5 — 3-SAT, parte 2/3). Continuación del ejemplo: implementación de solve_cnf_bruteforce(cnf, n_vars), que intenta resolver 3-SAT por fuerza bruta enumerando asignaciones con una máscara de bits (0 .. 2^n_vars – 1). Para cada asignación generada, llama a verify_cnf; si encuentra una que satisface la fórmula, la devuelve. Incluye además un caso satisfacible (SAT) y un caso insatisfacible (UNSAT) para mostrar el comportamiento del algoritmo. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    # Pruebas simples
    assert sol is not None and verify_cnf(cnf_sat, sol) is True
    assert sol2 is None
    print("OK: verificación SAT/UNSAT correcta.")

if __name__ == "__main__":
    main()
```

Captura 45: (Ejemplo 5 — 3-SAT, parte 3/3). Cierre del script con pruebas simples (assert) que validan el resultado: en el caso SAT se comprueba que existe una asignación y que efectivamente satisface la CNF mediante verify_cnf; en el caso UNSAT se verifica que la búsqueda retorna None. Este bloque conecta el contenido teórico (verificación polinómica vs. búsqueda exponencial) con evidencias prácticas de ejecución. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Representación CNF	Fórmula como lista de cláusulas; literal entero codifica signo (negación).
`verify_cnf`	Recorre cláusulas y literales; retorna False si alguna cláusula no se satisface.
`solve_cnf_bruteforce`	Enumera 2^n asignaciones (máscara) y retorna la primera satisfactoria.
`n_limit`	Evita explosión combinatoria; mantiene ejecución razonable.
`assert`	Confirma que el caso SAT tiene solución verificable y el caso UNSAT no.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

4.6 Algoritmos de aproximación y heurísticas

Para problemas NP-duros de optimización se usan algoritmos de aproximación con garantía: un algoritmo α-aproximado asegura una solución con costo ≤ α·OPT (minimización) en tiempo polinómico. Vertex Cover admite un algoritmo 2-aproximado basado en seleccionar aristas y tomar ambos extremos; su análisis se apoya en que las aristas seleccionadas forman un matching y cualquier cover óptimo debe cubrirlas, aportando un límite inferior (Vazirani, 2001; Williamson & Shmoys, 2011). En TSP, además de aproximaciones con supuestos métricos, se emplean heurísticas (vecino más cercano, 2-opt) que no siempre ofrecen garantía formal, pero se validan empíricamente comparando con el óptimo en instancias pequeñas (Kleinberg & Tardos, 2005).

Énfasis

Definiciones clave

- **Algoritmo de aproximación:** Algoritmo polinómico que retorna una solución con cota garantizada respecto del óptimo.
- **Ratio de aproximación:** Factor α que acota la calidad relativa de la solución respecto de OPT.
- **Heurística:** Procedimiento práctico sin garantía formal general, evaluado por desempeño experimental.
- **PTAS:** Esquema de aproximación polinómico parametrizado: para todo $\varepsilon > 0$ produce $(1+\varepsilon)$-aprox en tiempo polinómico (posiblemente con dependencia alta en $1/\varepsilon$).

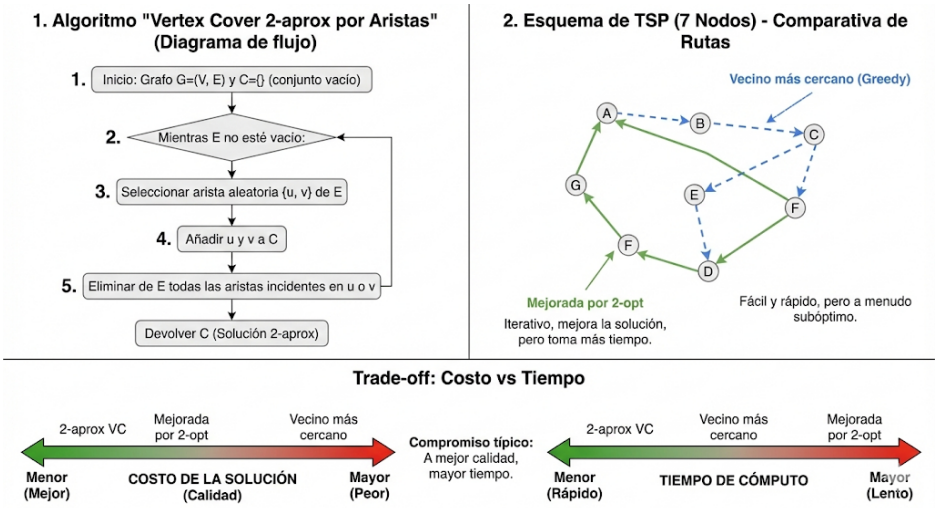


Ilustración 18: Aproximación y heurísticas: Vertex Cover 2-aprox y TSP (trade-off costo/tiempo). A la izquierda, el diagrama de flujo muestra un algoritmo 2-aproximado para Vertex Cover basado en seleccionar aristas y eliminar incidentes; a la derecha, se ilustra una heurística para TSP (vecino más cercano) y una mejora local (2-opt). El gráfico inferior resume el compromiso típico entre calidad de la solución y tiempo de cómputo, eje central del módulo al tratar estrategias prácticas para problemas complejos. Imagen creada usando herramientas IA con información proporcionada por la autora.

Ejemplo 6 — Vertex Cover 2-aprox (greedy por aristas) + comparación contra óptimo por fuerza bruta

El algoritmo 2-aprox de Vertex Cover elige una arista (u,v), incorpora u y v al cover y elimina todas las aristas incidentes. El conjunto de aristas seleccionadas puede interpretarse como un matching; dado que cualquier cover óptimo debe incluir al menos un extremo de cada arista del matching, el tamaño del cover obtenido queda acotado por un factor 2 respecto del óptimo (Vazirani, 2001).

```python
# Ejemplo 6: Vertex Cover 2-aprox + verificación y comparación con óptimo (n pequeño)
# Ejecutable en Online IDE Pro (Python estándar)

from typing import Set, Tuple, List
from itertools import combinations

Edge = Tuple[int, int]

def norm_edges(edges: Set[Edge]) -> Set[Edge]:
    return {(min(u,v), max(u,v)) for (u,v) in edges if u != v}

def covers_all_edges(edges: Set[Edge], cover: Set[int]) -> bool:
    for (u, v) in edges:
        if u not in cover and v not in cover:
            return False
    return True

def vertex_cover_2approx(n: int, edges: Set[Edge]) -> Set[int]:
    E = set(norm_edges(edges))
    cover: Set[int] = set()
    while E:
        (u, v) = next(iter(E))   # toma una arista arbitraria
        cover.add(u)
        cover.add(v)
        # elimina aristas incidentes a u o v
        E = {e for e in E if u not in e and v not in e}
    return cover
```

Captura 46: (inicio del Ejemplo 6 – Vertex Cover 2-aprox): Se presenta un script que implementa una aproximación 2-aprox para Vertex Cover. Primero se definen tipos y utilidades: norm_edges() normaliza aristas (evita duplicados por orden y descarta lazos), y covers_all_edges() verifica si un conjunto de vértices cubre todas las aristas (para cada arista (u,v), al menos u o v debe estar en el cover). Luego, vertex_cover_2approx() construye un cover aproximado eligiendo una arista arbitraria, agregando ambos extremos al cover y eliminando todas las aristas incidentes, repitiendo hasta vaciar el conjunto de aristas. Este ejemplo se relaciona con el contenido al mostrar una estrategia voraz típica de aproximación para problemas NP-difíciles. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def optimal_vertex_cover_bruteforce(n: int, edges: Set[Edge], n_limit: int = 12) -> Set[int]:
    if n > n_limit:
        raise ValueError(f"Óptimo por fuerza bruta no permitido: n={n} > {n_limit}")
    E = norm_edges(edges)
    for r in range(n + 1):
        for comb in combinations(range(n), r):
            C = set(comb)
            if covers_all_edges(E, C):
                return C
    return set(range(n))  # fallback (no debería ocurrir)

def main():
    # Grafo pequeño de ejemplo
    n = 8
    edges = {
        (0,1),(1,2),(2,3),(3,0),  # ciclo 4
        (3,4),(4,5),(5,6),(6,3),  # otro ciclo conectado
        (6,7)
    }
    edges = norm_edges(edges)

    approx = vertex_cover_2approx(n, edges)
    opt = optimal_vertex_cover_bruteforce(n, edges, n_limit=12)

    print("Aristas:", sorted(edges))
    print("Cover 2-aprox:", sorted(approx), "tamaño=", len(approx))
    print("Cover óptimo (fuerza bruta):", sorted(opt), "tamaño=", len(opt))
    if len(opt) > 0:
        print("Ratio empírico:", len(approx) / len(opt))
```

Captura 47 (continuación – óptimo por fuerza bruta y comparación): Se incorpora optimal_vertex_cover_bruteforce(), que calcula un Vertex Cover óptimo enumerando combinaciones de vértices (solo para n pequeño, con n_limit como resguardo). En main() se define un grafo de ejemplo, se calcula el cover aproximado (approx) y el óptimo (opt), y se imprimen aristas, soluciones y tamaños, incluyendo un ratio empírico len(approx)/len(opt). Esto conecta con el contenido al contrastar "calidad vs. costo computacional": la aproximación es rápida, mientras que el óptimo requiere enumeración combinatoria. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    # Pruebas simples
    assert covers_all_edges(edges, approx) is True
    assert covers_all_edges(edges, opt) is True
    assert len(approx) <= 2 * len(opt)  # garantía esperada del algoritmo
    print("OK: cubrimiento y cota empírica validadas.")

if __name__ == "__main__":
    main()
```

Captura 48: (cierre – validaciones y garantía esperada): Se agregan pruebas simples (assert) para validar que ambos conjuntos (aproximado y óptimo) cubren todas las aristas mediante covers_all_edges(). Además, se verifica una cota coherente con la garantía del algoritmo 2-aprox: len(approx) <= 2 * len(opt). Esta sección se relaciona con el contenido al enfatizar buenas prácticas de verificación (correctitud del cubrimiento) y evaluación de aproximaciones (cotas). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`vertex_cover_2approx`	Itera mientras existan aristas; toma (u,v), agrega ambos y elimina aristas incidentes.
`covers_all_edges`	Verifica la propiedad de cover: cada arista tiene un extremo en el conjunto.
`optimal_vertex_cover_bruteforce`	Busca el óptimo enumerando subconjuntos crecientes; limitado a n pequeño.
`assert len(approx) <= 2*len(opt)`	Contrasta empíricamente la garantía teórica (factor 2).
Representación `edges`	Conjunto de aristas normalizadas para consistencia.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

Ejemplo 7 — Heurística/aproximación en TSP (instancias pequeñas): brute force + vecino más cercano + 2-opt

En TSP, el óptimo exacto requiere explorar $(n-1)!$ tours si se fija un inicio; por ello se limita a $n \leq 9$. Se implementa una heurística de vecino más cercano y una mejora local 2-opt para observar trade-offs entre costo y tiempo (Kleinberg & Tardos, 2005).

```python
# Ejemplo 7: TSP (instancia pequeña) - óptimo por fuerza bruta + heurística + 2-opt
# Ejecutable en Online IDE Pro (Python estándar)

from typing import List, Tuple
from itertools import permutations

Matrix = List[List[int]]

def route_cost(dist: Matrix, route: List[int]) -> int:
    total = 0
    n = len(route)
    for i in range(n - 1):
        total += dist[route[i]][route[i + 1]]
    total += dist[route[-1]][route[0]]  # regreso al inicio
    return total

def tsp_bruteforce(dist: Matrix, threshold: int = 9) -> Tuple[List[int], int]:
    n = len(dist)
    if n > threshold:
        raise ValueError(f"TSP fuerza bruta no permitido: n={n} > {threshold}")
    best_route = None
    best_cost = 10**18
    # fijar inicio en 0 para evitar tours equivalentes por rotación
    for perm in permutations(range(1, n)):
        route = [0] + list(perm)
        c = route_cost(dist, route)
        if c < best_cost:
            best_cost = c
            best_route = route
    return best_route, best_cost
```

Captura 49: (parte 1/3). Código en Python que define el problema del Viajante de Comercio (TSP) para una instancia pequeña y prepara tres enfoques de resolución. Primero, se importan tipos y permutations. Luego se define route_cost(dist, route), que calcula el costo total de un recorrido sumando las distancias entre ciudades consecutivas y agregando el regreso al origen. A continuación, tsp_bruteforce(dist, threshold) implementa la búsqueda exhaustiva: genera permutaciones (fijando el inicio en 0 para evitar recorridos equivalentes por rotación) y conserva la ruta de menor costo, limitando el tamaño permitido con threshold para evitar explosión combinatoria. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def nearest_neighbor(dist: Matrix, start: int = 0) -> List[int]:
    n = len(dist)
    unvisited = set(range(n))
    route = [start]
    unvisited.remove(start)
    current = start
    while unvisited:
        nxt = min(unvisited, key=lambda j: dist[current][j])
        route.append(nxt)
        unvisited.remove(nxt)
        current = nxt
    return route

def two_opt(dist: Matrix, route: List[int], max_iters: int = 200) -> List[int]:
    n = len(route)
    best = route[:]
    best_cost = route_cost(dist, best)

    improved = True
    iters = 0
    while improved and iters < max_iters:
        improved = False
        iters += 1
        for i in range(1, n - 2):
            for k in range(i + 1, n - 1):
                new_route = best[:i] + list(reversed(best[i:k+1])) + best[k+1:]
                new_cost = route_cost(dist, new_route)
                if new_cost < best_cost:
                    best, best_cost = new_route, new_cost
                    improved = True
                    break
            if improved:
                break
    return best
```

Captura 50: (parte 2/3). Código en Python que incorpora métodos aproximados para TSP. La función nearest_neighbor(dist, start) implementa la heurística del vecino más cercano: desde la ciudad actual elige, entre las no visitadas, la de distancia mínima y construye una ruta completa. Luego two_opt(dist, route, max_iters) aplica mejora local 2-opt: intenta reducir el costo invirtiendo segmentos del recorrido (operación de "swap" por reversión) y repite hasta que no haya mejoras o se alcance max_iters. Este bloque conecta con el contenido sobre heurísticas y refinamiento local cuando el óptimo exacto es costoso. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def main():
    # Matriz simétrica pequeña (n=7)
    dist = [
        [0, 2, 9, 10, 7, 3, 8],
        [2, 0, 6,  4, 3, 8, 5],
        [9, 6, 0,  8, 5, 7, 6],
        [10,4, 8,  0, 6, 2, 4],
        [7, 3, 5,  6, 0, 6, 3],
        [3, 8, 7,  2, 6, 0, 5],
        [8, 5, 6,  4, 3, 5, 0],
    ]

    n = len(dist)
    print("n =", n)

    # Heurística
    nn_route = nearest_neighbor(dist, start=0)
    nn_cost = route_cost(dist, nn_route)
    print("Vecino más cercano:", nn_route, "costo=", nn_cost)

    # Mejora local
    opt2_route = two_opt(dist, nn_route, max_iters=200)
    opt2_cost = route_cost(dist, opt2_route)
    print("2-opt sobre NN:", opt2_route, "costo=", opt2_cost)

    # Óptimo por fuerza bruta (n pequeño)
    best_route, best_cost = tsp_bruteforce(dist, threshold=9)
    print("Óptimo (fuerza bruta):", best_route, "costo=", best_cost)

    # Pruebas simples
    assert set(nn_route) == set(range(n))
    assert set(opt2_route) == set(range(n))
    assert best_cost <= opt2_cost <= nn_cost or best_cost <= nn_cost  # relación típica esperada
    print("OK: rutas válidas y comparación realizada.")

if __name__ == "__main__":
    main()
```

Captura 51: (parte 3/3). Código en Python que define main() y ejecuta la comparación empírica entre métodos. Se crea una matriz de distancias simétrica (n=7), se calcula una ruta con nearest_neighbor, se mejora con two_opt, y finalmente se obtiene el óptimo por fuerza bruta con tsp_bruteforce (por ser instancia pequeña). Las impresiones muestran ruta y costo de cada enfoque, y los assert verifican que las rutas son válidas (contienen todas las ciudades) y que el óptimo no empeora frente a las aproximaciones (relación típica esperada). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`route_cost`	Calcula costo del tour cerrando al inicio (ciclo Hamiltoniano).
`tsp_bruteforce`	Explora $(n-1)!$ permutaciones fijando inicio; umbral `threshold`.
`nearest_neighbor`	Construye tour eligiendo el vecino no visitado más cercano en cada paso.
`two_opt`	Mejora local: invierte segmentos si reduce costo; itera con límite `max_iters`.
`assert set(route)==V`	Asegura que la ruta visita cada ciudad exactamente una vez.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. Actividad 3: Actividades Complejidad NP.

5. Transformada rápida de Fourier (FFT) y algoritmos sobre polinomios; aplicaciones en análisis de señales y procesamiento de datos

5.1 Transformada discreta de Fourier (DFT)

La Transformada Discreta de Fourier (DFT) transforma una secuencia finita $x[n]$ (dominio temporal o espacial) en otra secuencia $X[k]$ que representa su contenido en frecuencia discreta. Para $n = 0, \dots, N-1$, se define:

$$X[k] = \sum_{n=0}^{N-1} x[n]\ e^{-j2\pi kn/N}, k = 0, \dots, N-1$$

La interpretación usual es que $X[k]$ cuantifica la correlación de $x[n]$ con exponenciales complejas a frecuencias discretas. Debido a la periodicidad implícita del muestreo discreto, tanto $x[n]$ como $X[k]$ se consideran periódicas de período N en el marco de la DFT (Oppenheim & Schafer, 2010). El cálculo directo requiere N sumas por cada k, lo que conduce a complejidad temporal $O(N^2)$ (Cormen et al., 2009).

Énfasis

Definiciones clave

- **DFT:** Transformación lineal que mapea una secuencia discreta finita $x[n]$a coeficientes espectrales $X[k]$mediante sumas ponderadas por raíces complejas de la unidad.
- **Frecuencia discreta k:** Índice entero que selecciona una componente exponencial compleja $e^{j2\pi kn/N}$dentro del conjunto de N frecuencias de la DFT.
- **Componente DC:** Coeficiente $X[0]$, asociado al promedio (valor medio) de la señal en el dominio temporal.
- **Magnitud espectral:** Valor $|X[k]|$, que cuantifica amplitud (no fase) de la componente k.

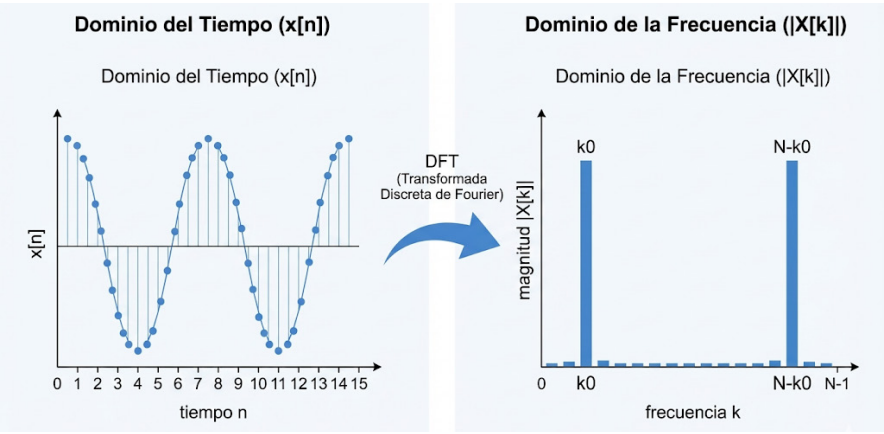


Ilustración 19: Dominio del tiempo vs. dominio de la frecuencia (DFT). La figura muestra cómo una señal discreta $x[n]$ (representada en el tiempo) se transforma mediante la DFT en su espectro $|X[k]|$ (representado en frecuencia), donde aparecen picos en los índices que concentran la energía de las componentes sinusoidales. Se relaciona con el contenido porque introduce la idea central: analizar señales pasando del tiempo a la frecuencia para identificar componentes dominantes. Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.1.1 Ejemplo 1–DFT (base) — Implementación $O(N^2)$ y validación cualitativa

La DFT directa implementa la definición mediante doble bucle y constituye la referencia de correctitud para tamaños pequeños. Su utilidad didáctica reside en evidenciar el costo $O(N^2)$ y en verificar picos espectrales esperados para señales sinusoidales muestreadas (Oppenheim & Schafer, 2010).

```python
import math
import cmath

def dft(x):
    """DFT directa O(N^2). x: lista de reales o complejos. Retorna lista de complejos X[k]."""
    N = len(x)
    X = []
    for k in range(N):
        s = 0j
        for n in range(N):
            angle = -2.0 * math.pi * k * n / N
            s += x[n] * complex(math.cos(angle), math.sin(angle))
        X.append(s)
    return X

def magnitude_spectrum(X):
    return [abs(v) for v in X]

def argmax(values):
    best_i = 0
    best_v = values[0]
    for i, v in enumerate(values):
        if v > best_v:
            best_v = v
            best_i = i
    return best_i, best_v

def allclose(a, b, tol=1e-6):
    return all(abs(x - y) <= tol for x, y in zip(a, b))

```

Captura 52: (parte 1/2). Implementación de la DFT directa con complejidad $O(N^2)$. El código importa math y cmath y define: dft(x) para calcular el espectro complejo $X[k]$ a partir de una señal temporal x; magnitude_spectrum(X) para obtener $|X[k]|$; argmax(values) para localizar el índice del mayor valor (pico espectral); y allclose(a,b,tol) como utilidad de comparación con tolerancia. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# -------------------------
# Pruebas simples
# Señal: coseno discreto con frecuencia k0 (exacta en rejilla DFT)
# x[n] = cos(2π k0 n / N)
# Se esperan picos en k0 y N-k0 (simetría conjugada en señales reales).
N = 16
k0 = 3
x = [math.cos(2.0 * math.pi * k0 * n / N) for n in range(N)]

X = dft(x)
mag = magnitude_spectrum(X)

# Ignorar DC para la detección del pico principal en este ejemplo
mag_no_dc = mag[:]
mag_no_dc[0] = -1.0

k_peak, v_peak = argmax(mag_no_dc)
print("Pico principal en k =", k_peak, "con |X[k]| =", v_peak)

# Validación cualitativa: el pico debe estar en k0 o N-k0
expected_peaks = {k0, N - k0}
assert k_peak in expected_peaks, "El pico no coincide con la frecuencia esperada."

# Validación suave: magnitud en los picos debe superar claramente a otras
sorted_mag = sorted(mag_no_dc, reverse=True)
assert sorted_mag[0] > 5.0 * sorted_mag[2], "Separación de pico insuficiente (tolerancia didáctica)."

print("OK: DFT detecta picos esperados (prueba didáctica).")
```

Captura 53: (parte 2/2, continuación). Prueba simple para validar la DFT: se construye una señal coseno discreta de longitud $N = 16$ con frecuencia $k_0 = 3$, se calcula $X = \text{dft}(x)$ y su magnitud $|X[k]|$. Para detectar el pico dominante se ignora el componente DC ($k = 0$), se busca el máximo con argmax y se verifica mediante assert que el pico esté en k_0 o en $N - k_0$ (simetría esperada en señales reales), además de exigir separación suficiente entre el pico principal y otros valores. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dft(x)`	Implementa la definición de DFT: para cada índice `k` acumula la suma sobre `n` con el factor complejo $e^{-j2\pi kn/N}$.
`angle` y `complex(cos, sin)`	Construyen el "factor giratorio" (raíz compleja de la unidad) sin dependencias externas, usando trigonometría estándar.
`magnitude_spectrum(X)`	Calcula (
Señal `x[n] = cos(...)`	El coseno tiene energía concentrada en dos frecuencias discretas k_0 y $N - k_0$ cuando k_0 es entero y N coincide con el período en muestras.
`argmax(mag_no_dc)`	Identifica el índice del máximo ignorando `k=0` (DC) para evitar seleccionar el promedio si no es relevante.
`assert k_peak in {k0, N-k0}`	Verifica cualitativamente el lugar del pico principal. Se admite simetría por señal real.
`assert sorted_mag[0] > 5 * sorted_mag[2]`	Control didáctico: exige que el pico sea claramente dominante, tolerando ruido numérico.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

5.2 FFT (Cooley–Tukey): divide & conquer, raíces de la unidad y complejidad

La FFT (Fast Fourier Transform) designa una familia de algoritmos que computan la DFT reduciendo el costo de $O(N^2)$ a $O(N\log N)$ al reutilizar subcálculos. La variante clásica Cooley–Tukey descompone la DFT en términos de índices pares e impares: se separa $x[n]$ en $x[2m]$ y $x[2m+1]$, se calculan dos DFT de tamaño $N/2$ y luego se combinan con factores $W_N^k = e^{-j2\pi k/N}$ (Cooley & Tukey, 1965). En implementaciones básicas suele asumirse N potencia de 2; si no lo es, se aplica *padding* con ceros hasta la siguiente potencia de 2 para mantener la estructura recursiva (Oppenheim & Schafer, 2010).

Énfasis

Definiciones clave

- **FFT:** Conjunto de algoritmos para calcular la DFT con complejidad subcuadrática, típicamente $O(N\log N)$.
- **Cooley–Tukey:** Esquema de descomposición que divide la DFT en subproblemas (p. ej., pares/impares) y combina resultados mediante factores W_N^k.
- **Raíces de la unidad:** Valores complejos $e^{-j2\pi k/N}$ que generan las rotaciones necesarias para combinar subtransformadas.
- **Padding con ceros:** Extensión de una secuencia agregando ceros para ajustar su longitud a un tamaño conveniente (por ejemplo, potencia de 2).

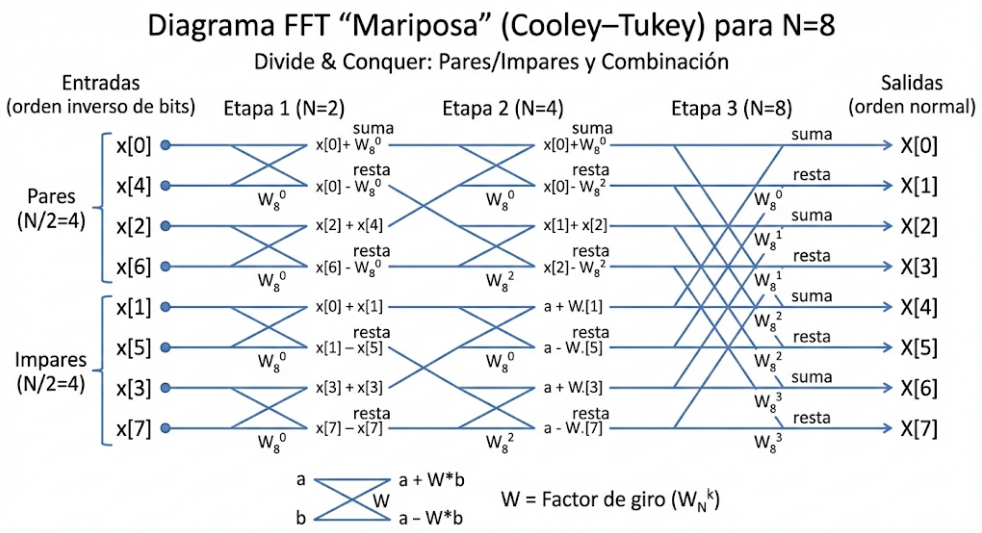


Ilustración 20: Diagrama FFT “mariposa” (Cooley–Tukey) para $N = 8$. La imagen detalla el flujo “divide y vencerás” de la FFT: separación en pares/impares, etapas de combinación y uso de factores de giro W_N^k para reconstruir $X[k]$. Se relaciona con el contenido porque explica por qué la FFT reduce el costo computacional al reutilizar resultados intermedios (estructuras tipo mariposa). Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.2.1 Ejemplo 2–DFT vs FFT (comparación) — FFT recursiva Cooley–Tukey y verificación contra DFT

Una FFT recursiva potencia de 2 permite observar el esquema pares/impares y la combinación tipo “mariposa” mediante factores W_N^k. Esto permite calcular la DFT de manera mucho más eficiente que la fórmula directa. Comparar los resultados de la FFT con la DFT directa en señales pequeñas es una forma práctica de verificar que la implementación funciona correctamente (Cooley & Tukey, 1965).

```python
import math

def dft(x):
    N = len(x)
    X = []
    for k in range(N):
        s = 0j
        for n in range(N):
            angle = -2.0 * math.pi * k * n / N
            s += x[n] * complex(math.cos(angle), math.sin(angle))
        X.append(s)
    return X

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def next_power_of_two(n):
    p = 1
    while p < n:
        p <<= 1
    return p
```

Captura 54: Se define una DFT directa dft(x) de complejidad $O(N^2)$, que para cada frecuencia k acumula la suma $\sum_n x[n] \cdot e^{-j2\pi kn/N}$ usando coseno y seno. Además, se incorporan utilidades para el manejo de tamaños: is_power_of_two(n) verifica si N es potencia de 2 mediante una operación bit a bit, y next_power_of_two(n) calcula la siguiente potencia de 2 para aplicar padding cuando sea necesario. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def fft(x):
    """FFT recursiva Cooley–Tukey para N potencia de 2."""
    N = len(x)
    if N == 1:
        return [complex(x[0])]
    if not is_power_of_two(N):
        raise ValueError("fft requiere N potencia de 2 (usar padding).")

    even = fft(x[0::2])
    odd  = fft(x[1::2])

    X = [0j] * N
    for k in range(N // 2):
        angle = -2.0 * math.pi * k / N
        wk = complex(math.cos(angle), math.sin(angle))  # e^{-j2πk/N}
        t = wk * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
    return X

def allclose(a, b, tol=1e-6):
    return all(abs(x - y) <= tol for x, y in zip(a, b))
```

Captura 55: Se implementa fft(x) recursiva (Cooley–Tukey) para N potencia de 2: divide la señal en índices pares e impares, calcula FFT de cada parte y combina resultados con los factores de giro $W_N^k = e^{-j2\pi k/N}$. Se lanza una excepción si N no es potencia de 2, indicando que debe usarse padding. También se incluye allclose(a,b,tol) para comparar resultados numéricamente con tolerancia. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# -------------------------
# Pruebas simples
# Comparación DFT vs FFT para N pequeño
x = [0.0, 1.0, 0.0, -1.0,  0.5, 0.0, -0.5, 0.0]  # N=8 (potencia de 2)
X_dft = dft(x)
X_fft = fft(x)

max_err = max(abs(a - b) for a, b in zip(X_dft, X_fft))
print("Error máximo |DFT-FFT| =", max_err)
assert allclose(X_dft, X_fft, tol=1e-6)

# Caso no potencia de 2: aplicar padding
y = [1.0, 2.0, 3.0, 4.0, 5.0]  # N=5
N2 = next_power_of_two(len(y))
y_pad = [complex(v) for v in y] + [0j] * (N2 - len(y))
Xy = fft(y_pad)  # no falla

print("OK: FFT coincide con DFT en N=8 y se ejecuta con padding en N=5->", N2)
```

Captura 56: Se presentan pruebas simples: (1) comparación DFT vs FFT en un caso pequeño $N = 8$ (potencia de 2), midiendo el error máximo y validando con assert allclose; (2) caso $N = 5$ no potencia de 2, donde se calcula N2 como siguiente potencia de 2, se rellena con ceros (padding) y se ejecuta FFT sin error. Se imprime un mensaje final confirmando coincidencia en $N = 8$ y ejecución correcta con padding. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`fft(x)`	Implementa Cooley–Tukey recursivo: divide en subsecuencias pares e impares y combina resultados.
Base `N==1`	Condición de parada: la DFT de tamaño 1 es el propio valor.
`even = fft(x[0::2])` , `odd = fft(x[1::2])`	Descomposición de $x[n]$ en términos de índices pares e impares, generando dos subproblemas de tamaño $N/2$.
`wk = e^{-j2πk/N}`	Factor giratorio que alinea las contribuciones de la subtransformada impar en la combinación final.
`X[k] = even[k] + t` , `X[k+N/2] = even[k] - t`	Combinación tipo "mariposa": produce simultáneamente dos salidas por cada `k` .
`assert allclose(X_dft, X_fft)`	Verifica equivalencia numérica con tolerancia por punto flotante.
`next_power_of_two` + `padding`	Ajusta longitudes no potencia de 2 para ejecutar la FFT recursiva sin cambiar el contenido original (salvo extensión con ceros).

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

5.3 Implementación de FFT e IFFT en Python: normalización, tolerancias y errores numéricos

La transformada inversa (IDFT/IFFT) reconstruye $x[n]$ desde $X[k]$. Una convención común es:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\ e^{+j2\pi kn/N}$$

En FFT/IFFT implementadas con punto flotante, las diferencias entre DFT teórica y cálculo real se deben a redondeo y acumulación de error; por ello se evalúa correctitud mediante tolerancias (Higham, 2002). Una implementación coherente de IFFT puede construirse usando la FFT mediante conjugación:

$$\text{IFFT(X)} = \frac{1}{N} \overline{FFT(\bar{X})}$$

bajo la misma convención de signos (Oppenheim & Schafer, 2010).

Énfasis

Definiciones clave

- **IFFT:** Algoritmo eficiente para calcular la IDFT, típicamente reutilizando la estructura FFT y aplicando normalización $1/N$.
- **Normalización:** Factor escalar (habitualmente $1/N$) aplicado para que IFFT(FFT(x)) recupere x bajo la convención elegida.

- **Tolerancia numérica:** Umbral ε para comparar valores de punto flotante considerando error de redondeo.
- **Error máximo absoluto:** $\max_n \mid x[n] - \hat{x}[n] \mid$, métrica simple para validar reconstrucción.

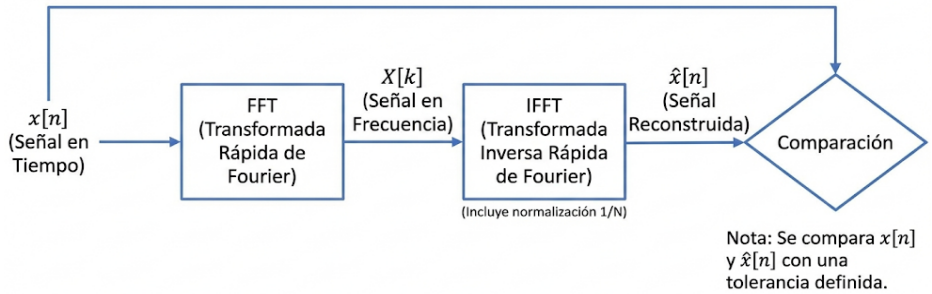


Ilustración 21: Ciclo FFT → IFFT y verificación de reconstrucción. El esquema presenta el pipeline: señal $x[n]$→ FFT → $X[k]$→ IFFT → $\hat{x}[n]$, seguido de una comparación con tolerancia numérica. Se relaciona con el contenido porque evidencia la propiedad clave: la transformada inversa recupera la señal original (salvo errores de redondeo), validando implementaciones. Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.3.1 Ejemplo 3–IFFT y reconstrucción — Validación de error máximo

La coherencia FFT/IFFT se verifica reconstruyendo la señal original y midiendo el error máximo absoluto. El uso de conjugación para implementar IFFT reduce duplicación de lógica y mantiene consistencia con la convención de signos (Oppenheim & Schafer, 2010).

```python
import math
import random

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def fft(x):
    N = len(x)
    if N == 1:
        return [complex(x[0])]
    if not is_power_of_two(N):
        raise ValueError("fft requiere N potencia de 2.")
    even = fft(x[0::2])
    odd  = fft(x[1::2])
    X = [0j] * N
    for k in range(N // 2):
        angle = -2.0 * math.pi * k / N
        wk = complex(math.cos(angle), math.sin(angle))
        t = wk * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
    return X

def ifft(X):
    """IFFT coherente con fft anterior: IFFT(X) = conj(FFT(conj(X))) / N"""
    N = len(X)
    if not is_power_of_two(N):
        raise ValueError("ifft requiere N potencia de 2.")
    X_conj = [complex(v.real, -v.imag) for v in X]
    y = fft(X_conj)
    y_conj = [complex(v.real, -v.imag) for v in y]
    return [v / N for v in y_conj]

def max_abs_error(a, b):
    return max(abs(x - y) for x, y in zip(a, b))
```

Captura 57: Se importan math y random y se definen utilidades para trabajar con FFT. La función is_power_of_two(n) verifica que el tamaño de la señal sea potencia de 2 (condición del algoritmo). La función fft(x) implementa la FFT recursiva de Cooley–Tukey: separa la señal en índices pares e impares, calcula recursivamente sus transformadas y luego combina ambos resultados usando los factores de giro $e^{-j2\pi k/N}$ para construir el espectro X. La función ifft(X) implementa la transformada inversa coherente con fft, usando la identidad $\text{IFFT}(X) = \text{FFT}(\bar{X})/N$, para recuperar la señal temporal. Se incluye además max_abs_error(a,b) para medir el error máximo entre dos señales. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# -------------------------
# Pruebas simples
N = 16
# Señal mixta: seno + ruido leve (para ejercitar parte imaginaria ~ 0 y errores)
k0 = 2
x = []
for n in range(N):
    val = math.sin(2.0 * math.pi * k0 * n / N) + 0.01 * (random.random() - 0.5)
    x.append(complex(val, 0.0))

X = fft(x)
x_rec = ifft(X)

err = max_abs_error(x, x_rec)
print("Error máximo de reconstrucción =", err)
assert err < 1e-6, "Reconstrucción fuera de tolerancia."

print("OK: IFFT(FFT(x)) reconstruye x dentro de tolerancia.")
```

Captura 58: Se ejecuta una prueba didáctica de reconstrucción: se fija N = 16 y se genera una señal compleja (parte imaginaria 0) compuesta por un seno de frecuencia $k_0 = 2$ más un ruido leve, para comprobar robustez numérica. Luego se calcula X = fft(x) y se reconstruye x_rec = ifft(X). Finalmente, se calcula el error máximo err y se valida con un assert que la reconstrucción cumple una tolerancia (err < 1e $^{-6}$), imprimiendo un mensaje de confirmación si todo es correcto. Esto conecta el contenido con la idea central de que FFT/IFFT son transformadas inversas entre sí (salvo tolerancias numéricas). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`ifft(X)` por conjugación	Implementa la identidad $\mathrm{IFFT}(X) = \overline{\mathrm{FFT}(\overline{X})}/N$, evitando duplicar la recursión con signo opuesto.
`X_conj` y `y_conj`	Conjugación compleja manual para mantener compatibilidad con tipos complejos nativos de Python.
División por `N`	Aplica normalización para que la composición IFFT∘FFT sea (aprox.) identidad bajo la convención usada.
Señal seno + ruido	Introduce un caso no exacto para observar que el error de reconstrucción es numérico y se controla con tolerancia.
`max_abs_error` + `assert err < 1e-6`	Valida reconstrucción con métrica simple y umbral consistente con aritmética de doble precisión.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

5.4 Convolución y multiplicación de polinomios: circular vs lineal, padding y complejidad

La convolución discreta lineal de secuencias a y b (longitudes n y m) produce una salida de longitud $n + m - 1$ y se calcula de forma directa en $O(nm)$. La DFT/FFT permite acelerar la convolución usando el teorema de la convolución: la convolución en tiempo corresponde al producto punto a punto en frecuencia, siempre que se trate adecuadamente la periodicidad (convolución circular) y se aplique padding para obtener la convolución lineal (Bracewell, 2000). En multiplicación de polinomios $p(x) = \sum a_i x^i$ y $q(x) = \sum b_j x^j$, los coeficientes del producto corresponden a la convolución de $\{a_i\}$ y $\{b_j\}$. Con FFT, la complejidad típica se reduce a $O(N \log N)$ para N tamaño de padding suficiente (Cormen et al., 2009).

Énfasis

Definiciones clave

- **Convolución lineal:** Operación que combina dos secuencias sumando productos desplazados, con longitud de salida $n + m - 1$.
- **Convolución circular:** Convolución asumida periódica módulo N, equivalente a operar bajo periodicidad implícita de la DFT.
- **Multiplicación de polinomios por convolución:** Cálculo de coeficientes del producto $p \cdot q$ como convolución de coeficientes de p y q.
- **Padding correcto:** Elección de $N \geq n + m - 1$ (y usualmente potencia de 2) para que la convolución circular coincida con la lineal en la región de interés.

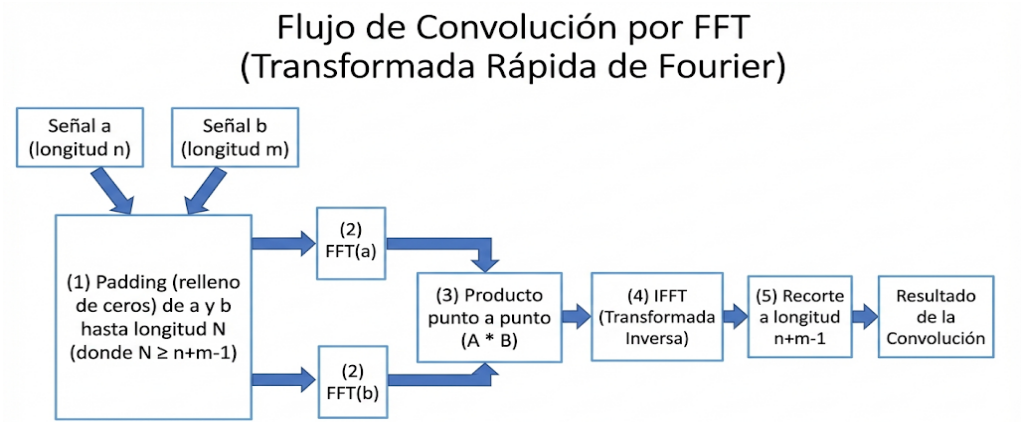


Ilustración 22: Flujo de convolución usando FFT. Se describe el procedimiento estándar para convolución eficiente: padding con ceros, FFT de ambas señales, producto punto a punto en frecuencia y vuelta con IFFT, seguido de recorte. Se relaciona con el contenido porque muestra una aplicación directa de la FFT en procesamiento de señales y en algoritmos (convolución más rápida que el enfoque directo). Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.4.1 Ejemplo 4–Convolución y polinomios — Naive $O(n^2)$ vs FFT con padding

El método ingenuo implementa directamente la definición de convolución, útil como referencia. El método con FFT aplica: (1) padding a $N \geq n + m - 1$, (2) FFT, (3) producto punto a punto, (4) IFFT, recuperando coeficientes dentro de tolerancia por redondeo (Bracewell, 2000).

```python
import math

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def next_power_of_two(n):
    p = 1
    while p < n:
        p <<= 1
    return p

def fft(x):
    N = len(x)
    if N == 1:
        return [complex(x[0])]
    if not is_power_of_two(N):
        raise ValueError("fft requiere N potencia de 2.")
    even = fft(x[0::2])
    odd  = fft(x[1::2])
    X = [0j] * N
    for k in range(N // 2):
        angle = -2.0 * math.pi * k / N
        wk = complex(math.cos(angle), math.sin(angle))
        t = wk * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
    return X
```

Captura 59 Este fragmento define utilidades base para trabajar con FFT en Python estándar: is_power_of_two(n) verifica si el tamaño es potencia de 2 (requisito del algoritmo recursivo), next_power_of_two(n) calcula el siguiente tamaño potencia de 2 para aplicar padding, y fft(x) implementa la FFT recursiva de Cooley–Tukey separando la señal en índices pares e impares y recombinando con los factores de giro complejos. Este bloque prepara las funciones necesarias para luego aplicar IFFT y usar FFT en multiplicación/convolución. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def ifft(X):
    N = len(X)
    if not is_power_of_two(N):
        raise ValueError("ifft requiere N potencia de 2.")
    X_conj = [complex(v.real, -v.imag) for v in X]
    y = fft(X_conj)
    y_conj = [complex(v.real, -v.imag) for v in y]
    return [v / N for v in y_conj]

def conv_naive(a, b):
    """Convolución lineal O(nm)."""
    n, m = len(a), len(b)
    y = [0.0] * (n + m - 1)
    for i in range(n):
        for j in range(m):
            y[i + j] += a[i] * b[j]
    return y

def poly_mul_fft(a, b):
    """Multiplicación de polinomios por FFT. a,b: listas de reales (coeficientes)."""
    n, m = len(a), len(b)
    L = n + m - 1
    N = next_power_of_two(L)
    A = [complex(v, 0.0) for v in a] + [0j] * (N - n)
    B = [complex(v, 0.0) for v in b] + [0j] * (N - m)

    FA = fft(A)
    FB = fft(B)
    FC = [FA[k] * FB[k] for k in range(N)]
    c_time = ifft(FC)
    # Tomar solo los primeros L coeficientes; redondeo/tolerancia
    return [c_time[i].real for i in range(L)]
```

Captura 60: Este fragmento incorpora ifft(X) (transformada inversa) usando la identidad IFFT(X) = conj(FFT(conj(X))) / N, y define conv_naive(a, b) como la convolución lineal directa $O(nm)$ para tener un punto de comparación. Luego presenta poly_mul_fft(a, b), que implementa la multiplicación de polinomios mediante FFT: calcula la longitud objetivo $L = n + m - 1$, realiza padding hasta N potencia de 2, transforma ambos vectores (FA, FB), multiplica punto a punto en frecuencia (FC) y vuelve al dominio del tiempo con ifft(FC) para obtener los coeficientes del producto. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def allclose_list(a, b, tol=1e-6):
    return all(abs(x - y) <= tol for x, y in zip(a, b))

# -------------------------
# Pruebas simples
# Polinomios explícitos:
# p(x) = 1 + 2x + 3x^2
# q(x) = 2 + 1x
# Producto esperado: (1+2x+3x^2)(2+x)= 2 + 5x + 8x^2 + 3x^3
a = [1, 2, 3]
b = [2, 1]
expected = [2, 5, 8, 3]

c1 = conv_naive(a, b)
c2 = poly_mul_fft(a, b)

print("Naive:", c1)
print("FFT  :", c2)
assert allclose_list(c1, expected, tol=1e-9)
assert allclose_list(c2, expected, tol=1e-6)

print("OK: multiplicación de polinomios coincide (naive y FFT) dentro de tolerancia.")
```

Captura 61: Este fragmento agrega allclose_list(a, b, tol) para comparar resultados con tolerancia numérica (por redondeos en FFT/IFFT) y ejecuta pruebas simples: define dos polinomios $p(x) = 1 + 2x + 3x^2$ y $q(x) = 2 + x$, calcula el resultado esperado, obtiene c1 por convolución directa y c2 por FFT, imprime ambas salidas y valida con assert que coincidan (dentro de tolerancia). Con esto se demuestra, en una instancia didáctica, que la multiplicación por FFT reproduce el mismo producto que el método directo, con mejor escalabilidad cuando crecen los tamaños. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`conv_naive(a,b)`	Implementa la convolución lineal sumando productos desplazados; sirve como base $O(nm)$.
`L = n+m-1`	Longitud de la convolución lineal, equivalente al grado máximo del producto de polinomios + 1.
`N = next_power_of_two(L)`	Elige tamaño potencia de 2 para aplicar FFT recursiva y evitar aliasing en la región útil.
`FA, FB`	FFT de las secuencias con padding; representa evaluación del polinomio en raíces de la unidad.
`FC[k] = FA[k] * FB[k]`	Producto punto a punto en frecuencia, equivalente a convolución en tiempo bajo DFT.
`ifft(FC)`	Retorna la secuencia de coeficientes (aprox.) en dominio temporal; se toma parte real por entradas reales.
Asserts con tolerancia	Compensan error de punto flotante, especialmente por acumulación en FFT/IFFT.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

5.5 Aplicaciones en señales y datos: filtrado simple, detección de frecuencias dominantes y ventanas

En análisis de señales discretas, la FFT se usa para estimar espectros, detectar componentes dominantes y aplicar filtrado en frecuencia. Un filtrado pasa-bajos didáctico puede construirse anulando (o atenuando) coeficientes de alta frecuencia y luego aplicando IFFT; esto equivale a una operación de suavizado, pero requiere cuidado conceptual: la selección de bins y el tratamiento de bordes dependen de periodicidad implícita y del criterio de corte (Oppenheim & Schafer, 2010). Para estimar frecuencia dominante en

Hz se requiere la frecuencia de muestreo f_s: el bin k corresponde aproximadamente a $f \approx k f_s / N$ (Proakis & Manolakis, 2007). En procesamiento por ventanas (conceptual), se segmenta la señal para asumir estacionariedad local y reducir efectos de discontinuidad (Oppenheim & Schafer, 2010).

Énfasis

Definiciones clave

- **Filtrado pasa-bajos (discreto):** Operación que atenúa componentes de alta frecuencia, preservando componentes cercanas a DC y bajas frecuencias.
- **Frecuencia de muestreo f_s:** Cantidad de muestras por segundo que relaciona el índice espectral $k$con frecuencia física en Hz.
- **Ventana:** Segmento finito de señal (y/o función de ponderación) utilizado para análisis local y control de efectos de borde.
- **Espectro de magnitud:** Secuencia $|X[k]|$ usada para identificación de picos (frecuencias dominantes).

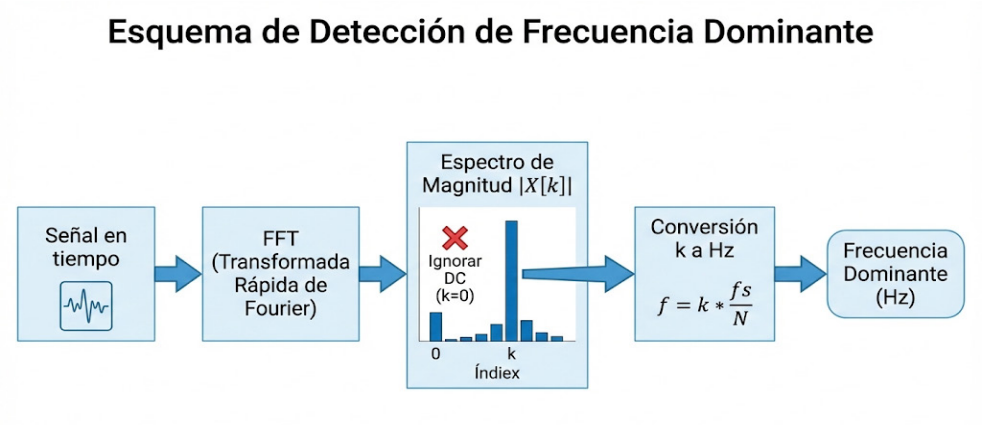


Ilustración 23: Detección de frecuencia dominante con FFT. El diagrama resume el proceso: señal temporal → FFT → espectro $|X[k]|$ (ignorando el componente DC $k = 0$) → conversión de índice k a Hz usando $f = k \cdot \frac{f_s}{N}$. Se relaciona con el contenido porque enseña cómo interpretar el espectro para extraer una característica concreta (la frecuencia principal). Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.5.1 Ejemplo 5–Convolución aplicada a señales — filtrado simple en frecuencia

El suavizado puede aproximarse atenuando componentes altas en el espectro y reconstruyendo por IFFT. Este ejemplo se limita a un esquema didáctico de "anular bins" para ilustrar la idea de filtrado en frecuencia y su efecto en varianza/energía (Oppenheim & Schafer, 2010).

```python
import math
import random

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def next_power_of_two(n):
    p = 1
    while p < n:
        p <<= 1
    return p

def fft(x):
    N = len(x)
    if N == 1:
        return [complex(x[0])]
    if not is_power_of_two(N):
        raise ValueError("fft requiere N potencia de 2.")
    even = fft(x[0::2])
    odd  = fft(x[1::2])
    X = [0j] * N
    for k in range(N // 2):
        angle = -2.0 * math.pi * k / N
        wk = complex(math.cos(angle), math.sin(angle))
        t = wk * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
    return X
```

Captura 62: (Parte 1/3). Se definen utilidades para trabajar con FFT en señales discretas: se importan math y random, se implementa is_power_of_two(n) para verificar si el tamaño es potencia de 2, next_power_of_two(n) para calcular la siguiente potencia de 2 (útil para padding), y fft(x) (Cooley–Tukey recursiva) que divide la señal en índices pares/impares y combina resultados con factores de giro $e^{-j2\pi k/N}$. Este bloque prepara la base algorítmica para transformar una señal al dominio de la frecuencia. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def ifft(X):
    N = len(X)
    if not is_power_of_two(N):
        raise ValueError("ifft requiere N potencia de 2.")
    X_conj = [complex(v.real, -v.imag) for v in X]
    y = fft(X_conj)
    y_conj = [complex(v.real, -v.imag) for v in y]
    return [v / N for v in y_conj]

def variance_real(x):
    vals = [v.real for v in x]
    mean = sum(vals) / len(vals)
    return sum((v - mean) ** 2 for v in vals) / len(vals)

def lowpass_by_bins(X, cutoff_k):
    """Filtro pasa-bajos didáctico: conserva bins 0..cutoff_k y sus simétricos."""
    N = len(X)
    Y = [0j] * N
    for k in range(N):
        if k <= cutoff_k or k >= N - cutoff_k:
            Y[k] = X[k]
        else:
            Y[k] = 0j
    return Y
```

Captura 63: (Parte 2/3). Se implementa ifft(X) (transformada inversa) usando la identidad IFFT(X) = FFT(X) / N, garantizando coherencia con la fft definida previamente. Luego se agrega variance_real(x) para calcular la varianza de la parte real de una señal (medida simple de "energía"/dispersión), y lowpass_by_bins(X, cutoff_k) que aplica un filtro pasa-bajos didáctico en frecuencia: conserva los bins $0..cutoff_k$ y sus simétricos $N - cutoff_k..N-1$, anulando el resto. Este bloque conecta FFT/IFFT con la idea de filtrado en el dominio de la frecuencia. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
# -------------------------
# Pruebas simples
# Señal sintética: dos senoidales + ruido leve
N0 = 200
fs = 200.0  # Hz
f1 = 5.0
f2 = 40.0
x0 = []
for n in range(N0):
    t = n / fs
    val = math.sin(2.0 * math.pi * f1 * t) + 0.6 * math.sin(2.0 * math.pi * f2 * t)
    val += 0.2 * (random.random() - 0.5)
    x0.append(complex(val, 0.0))

# Padding a potencia de 2
N = next_power_of_two(len(x0))
x = x0 + [0j] * (N - len(x0))

X = fft(x)
Y = lowpass_by_bins(X, cutoff_k=10)  # corte didáctico
y = ifft(Y)

var_before = variance_real(x0)
var_after = variance_real(y[:len(x0)])
print("Varianza antes:", var_before)
print("Varianza después:", var_after)

# Aserción suave: al eliminar alta frecuencia (40 Hz) y parte del ruido, la varianza suele bajar
assert var_after < var_before, "No se observa reducción de varianza (puede variar por ruido aleatorio)."

print("OK (didáctico): filtrado en frecuencia reduce varianza en este experimento.")
```

Captura 64: (Parte 3/3). Se ejecuta una demostración completa: se genera una señal sintética con dos senoidales (frecuencias f_1 y f_2) más ruido, se realiza padding a potencia de 2, se calcula X = fft(x), se filtra con lowpass_by_bins para eliminar componentes de alta frecuencia (incluyendo la senoidal de mayor frecuencia y parte del ruido), y se reconstruye y = ifft(Y). Finalmente se compara la varianza antes/después (var_before, var_after) y se valida con un assert que la varianza disminuya en este experimento didáctico. Este bloque ilustra el flujo "tiempo → frecuencia → filtrado → tiempo" mediante FFT/IFFT. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Señal `f1` y `f2` + ruido	Construye un caso con baja frecuencia, alta frecuencia y perturbación aleatoria para observar efecto del filtrado.
`padding` a potencia de 2	Ajusta longitud para FFT recursiva; se mantiene la porción útil en los primeros `N0` elementos.
`lowpass_by_bins`	Conserva bins bajos y sus simétricos, anulando el resto: aproximación didáctica de pasa-bajos.
`ifft(Y)`	Reconstruye señal filtrada en tiempo; se recorta a longitud original.
`variance_real`	Usa varianza como métrica simple de "energía" de fluctuación para comparar antes/después.
`assert var_after < var_before`	Control suave: espera reducción por eliminación de componente alta (y parte del ruido).

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

5.5.2 Ejemplo 6–Detección de frecuencia dominante + (F) variante opcional con NumPy

La detección de frecuencia dominante se basa en localizar el índice k de mayor $|X[k]|$ (ignorando DC si procede) y convertirlo a Hz mediante $f \approx k f_s / N$. Se incluye una verificación opcional con numpy.fft para comparar resultados sin requerir NumPy (Proakis & Manolakis, 2007).

```python
import math

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def next_power_of_two(n):
    p = 1
    while p < n:
        p <<= 1
    return p

def fft(x):
    N = len(x)
    if N == 1:
        return [complex(x[0])]
    if not is_power_of_two(N):
        raise ValueError("fft requiere N potencia de 2.")
    even = fft(x[0::2])
    odd  = fft(x[1::2])
    X = [0j] * N
    for k in range(N // 2):
        angle = -2.0 * math.pi * k / N
        wk = complex(math.cos(angle), math.sin(angle))
        t = wk * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
    return X

def argmax(values):
    best_i = 0
    best_v = values[0]
    for i, v in enumerate(values):
        if v > best_v:
            best_v = v
            best_i = i
    return best_i, best_v
```

Captura 65: (Parte 1/2). Fragmento de código que implementa, en Python, los componentes base para detectar frecuencia dominante mediante FFT: (1) funciones auxiliares is_power_of_two(n) y next_power_of_two(n) para asegurar que la longitud de la señal sea potencia de 2; (2) una FFT recursiva tipo Cooley–Tukey fft(x), que separa muestras pares/impares y combina resultados con factores de giro $e^{-j2\pi k/N}$; y (3) argmax(values) para localizar el índice del máximo en un arreglo (luego se usará sobre el espectro de magnitud). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# -------------------------
# Pruebas simples: frecuencia dominante conocida
fs = 256.0
f0 = 32.0
N0 = 128
x0 = [complex(math.sin(2.0 * math.pi * f0 * (n / fs)), 0.0) for n in range(N0)]

N = next_power_of_two(len(x0))
x = x0 + [0j] * (N - len(x0))

X = fft(x)
mag = [abs(v) for v in X]

# ignorar DC
mag[0] = -1.0
k_peak, _ = argmax(mag[:N//2])  # usar solo semiespectro para señal real

f_est = k_peak * fs / N
print("k_peak =", k_peak, "f_est =", f_est, "Hz (f0 =", f0, "Hz)")

assert abs(f_est - f0) < 1.0, "Frecuencia estimada fuera de tolerancia."

print("OK: detección de frecuencia dominante dentro de tolerancia.")

# (F) Variante opcional con NumPy (sin romper el script)
try:
    import numpy as np
    Xnp = np.fft.fft(np.array([v.real for v in x], dtype=float))
    mag_np = np.abs(Xnp)
    mag_np[0] = -1.0
    k_np = int(np.argmax(mag_np[:N//2]))
    f_np = k_np * fs / N
    print("NumPy: k =", k_np, "f =", f_np, "Hz")

    # comparación suave entre bins detectados
    assert abs(f_np - f_est) < 1.0
    print("OK: NumPy y FFT propia concuerdan (tolerancia didáctica).")
except Exception as e:
    print("NumPy no disponible o error al usarlo; se continúa sin NumPy. Detalle:", type(e).__name__)
```

Captura 66 (Parte 2/2). Fragmento de código que realiza la prueba didáctica de frecuencia dominante conocida: (1) genera una señal senoidal de frecuencia f_0 con frecuencia de muestreo fs; (2) aplica padding a la siguiente potencia de 2 para poder usar la FFT; (3) calcula el espectro X = fft(x) y su magnitud; (4) ignora el componente DC (mag[0]) y busca el pico en el semiespectro (señal real) para obtener k_peak; (5) convierte el índice a Hertz con $f_est = k_peak \cdot fs/N$ y valida con una aserción de tolerancia; (6) incluye una variante opcional con NumPy para comparar resultados sin romper la ejecución si NumPy no está disponible. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`fs`, `f0`, `x0`	Define frecuencia de muestreo y tono dominante conocido para validar la estimación.
`fft(x)` + `mag = abs(X)`	Calcula espectro y magnitud para localizar el pico dominante.
`mag[0] = -1.0`	Evita seleccionar DC si el promedio no es el objetivo del análisis.
`argmax(mag[:N//2])`	Para señal real, el espectro es simétrico; se busca en el semiespectro positivo.
`f_est = k_peak * fs / N`	Convierte índice de bin a frecuencia física aproximada en Hz.
`assert abs(f_est - f0) < 1.0`	Valida estimación dentro de una tolerancia coherente con resolución espectral.
`try: import numpy as np`	Variante opcional: si NumPy está disponible, repite el análisis con `numpy.fft`.
Mensaje en `except`	Garantiza que el script no se rompe si NumPy no está instalado.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

5.6 Análisis de eficiencia y selección de método: DFT vs FFT, costo y precisión

La elección entre DFT directa y FFT depende principalmente del tamaño N y del contexto. La DFT $O(N^2)$ es útil para N pequeño, validación y casos donde la simplicidad prima; la FFT $O(N\log N)$ se vuelve preferible cuando N crece. En práctica, además del orden de complejidad, influyen constantes ocultas, costos de padding y consideraciones numéricas de punto flotante (Cormen et al., 2009; Higham, 2002). En tareas de IA y procesamiento de datos, FFT es común para grandes vectores, convoluciones rápidas y análisis espectral repetido; la DFT directa se reserva para pruebas pequeñas o implementaciones de referencia.

Énfasis

Definiciones clave

- **Complejidad $O(N^2)$:** Crecimiento cuadrático del tiempo con el tamaño de entrada; típico de la DFT directa.
- **Complejidad $O(N\log N)$:** Crecimiento cuasi lineal; típico de FFT en longitudes adecuadas.
- **Resolución espectral:** Separación $\Delta f = f_s/N$ entre bins; condiciona exactitud de estimación de frecuencia.
- **Constantes ocultas:** Factores de implementación que afectan tiempos reales más allá del orden asintótico.

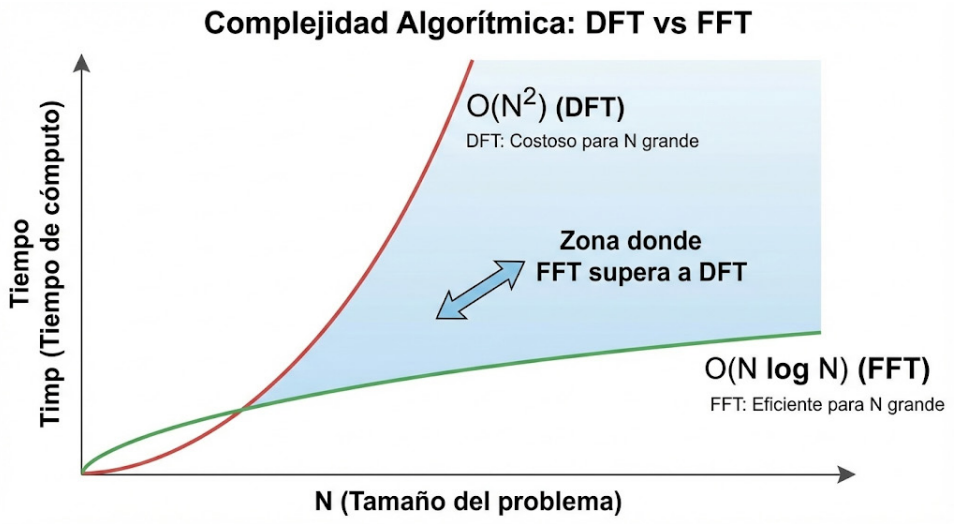


Ilustración 24: Complejidad algorítmica: DFT vs. FFT. La gráfica compara el crecimiento del tiempo de cómputo: $O(N^2)$ para DFT frente a $O(N\log N)$ para FFT, indicando la zona donde FFT resulta ventajosa. Se relaciona con el contenido porque justifica el uso de FFT en problemas con N grande por su mejora asintótica. Imagen creada usando herramientas de IA con información proporcionada por la autora.

5.6.1 Ejemplo 7–comparación de tiempos DFT vs FFT en tamaños crecientes

Un micro-benchmark con time.perf_counter() permite observar tendencias de crecimiento temporal. El objetivo es contrastar comportamiento cuadrático (DFT) versus cuasi lineal (FFT) en tamaños potencia de 2, sin pretender una medición de rendimiento absoluta (Cormen et al., 2009).

```python
import math
import time
import random

def dft(x):
    N = len(x)
    X = []
    for k in range(N):
        s = 0j
        for n in range(N):
            angle = -2.0 * math.pi * k * n / N
            s += x[n] * complex(math.cos(angle), math.sin(angle))
        X.append(s)
    return X

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def fft(x):
    N = len(x)
    if N == 1:
        return [complex(x[0])]
    if not is_power_of_two(N):
        raise ValueError("fft requiere N potencia de 2.")
    even = fft(x[0::2])
    odd  = fft(x[1::2])
    X = [0j] * N
    for k in range(N // 2):
        angle = -2.0 * math.pi * k / N
        wk = complex(math.cos(angle), math.sin(angle))
        t = wk * odd[k]
        X[k] = even[k] + t
        X[k + N // 2] = even[k] - t
    return X

def random_signal(N):
    return [complex(random.random() - 0.5, 0.0) for _ in range(N)]
```

Captura 67: Script en Python que implementa dos transformadas: DFT directa (doble bucle, costo $O(N^2)$) y FFT recursiva Cooley–Tukey (divide en pares/impares, costo $O(Nlog\,N)$). Además, define funciones auxiliares para: (a) verificar si N es potencia de 2 (requisito de esta FFT), (b) generar una señal aleatoria compleja (parte imaginaria nula) para usar como entrada en las pruebas de rendimiento. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def time_call(fn, x, repeats=1):
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        fn(x)
        t1 = time.perf_counter()
        best = min(best, t1 - t0)
    return best

# -------------------------
# Pruebas simples / benchmark
sizes = [32, 64, 128, 256]  # mantener moderado para no exceder tiempos en entornos online
print("N | t(DFT) [s] | t(FFT) [s] | ratio DFT/FFT")
for N in sizes:
    x = random_signal(N)
    t_dft = time_call(dft, x, repeats=1)
    t_fft = time_call(fft, x, repeats=1)
    ratio = (t_dft / t_fft) if t_fft > 0 else float("inf")
    print(N, "|", round(t_dft, 6), "|", round(t_fft, 6), "|", round(ratio, 2))

# Aserción suave: para N>=128 suele observarse mejora clara con FFT (no garantizado por hardware/VM)
# Se usa un umbral modesto para evitar falsos fallos en entornos muy variables.
x = random_signal(256)
t_dft = time_call(dft, x, repeats=1)
t_fft = time_call(fft, x, repeats=1)
assert t_fft <= t_dft, "No se observa mejora de FFT en este entorno (variabilidad de ejecución)."

print("OK (didáctico): tendencia FFT <= DFT para N=256 en este entorno.")
```

Captura 68: Bloque de benchmark didáctico: mide el tiempo de ejecución de DFT y FFT con time.perf_counter() mediante time_call (toma el mejor tiempo en una o más repeticiones) para distintos tamaños N moderados (p. ej., 32, 64, 128, 256). Imprime una tabla con t(DFT), t(FFT) y el ratio DFT/FFT para observar la ventaja esperada de FFT al crecer N. Incluye una aserción suave que, para $N = 256$, verifica la tendencia $t_{FFT} \leq t_{DFT}$ (advirtiendo que puede variar según el entorno). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`sizes = [32, 64, 128, 256]`	Selecciona tamaños moderados (potencias de 2) para evitar tiempos excesivos en IDE online.
`random_signal(N)`	Genera una entrada genérica que evita casos degenerados y permite comparar ejecución.
`time.perf_counter()`	Cronómetro de alta resolución para medir duración aproximada de cada cálculo.
`ratio DFT/FFT`	Indicador simple de ganancia relativa; se espera que crezca con N.
`assert t_fft <= t_dft`	Aserción suave para capturar la tendencia general, admitiendo variabilidad de entorno.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. **Actividad 4: Plataforma DFT FFT Interactiva.**

6. Geometría computacional

6.1 Primitivas geométricas y robustez numérica

En geometría computacional 2D, la mayoría de los predicados (intersección, convexidad, pertenencia) se reducen a operaciones sobre puntos y vectores, en particular al signo del producto cruzado 2D, que determina la orientación (giro) de tres puntos. Este predicado es la base para construir algoritmos como cerco convexo, intersección de segmentos y pruebas punto-en-polígono (de Berg et al., 2008).

Dado que se usan números de punto flotante, las comparaciones exactas son frágiles: pequeñas perturbaciones pueden invertir el signo y producir resultados inconsistentes. Por ello se introduce una tolerancia epsilon y se tratan explícitamente casos degenerados (colinealidad, puntos repetidos, segmento de longitud 0) (Shewchuk, 1997).

Énfasis

Definiciones clave

- **Punto 2D:** Par ordenado (x, y) que representa una posición en el plano euclídeo.
- **Vector 2D:** Diferencia de puntos $\vec{u} = b - a$, utilizada para describir dirección y magnitud.
- **Producto punto (dot):** Operación $u \cdot v = u_x v_x + u_y v_y$, usada para proyecciones y ángulos.
- **Producto cruzado 2D (cross):** Escalar $u \times v = u_x v_y - u_y v_x$; su signo indica orientación.
- **Orientación (orient):** Signo de $\text{cross}(b - a, c - a)$; positivo (giro antihorario), negativo (horario), cero (colineal).
- **Epsilon numérico:** Tolerancia $\varepsilon > 0$ para comparar valores cercanos a cero y mitigar error de redondeo.
- **Punto en segmento (on_segment):** Predicado que determina si p está sobre el segmento $[a, b]$, considerando colinealidad y caja envolvente.

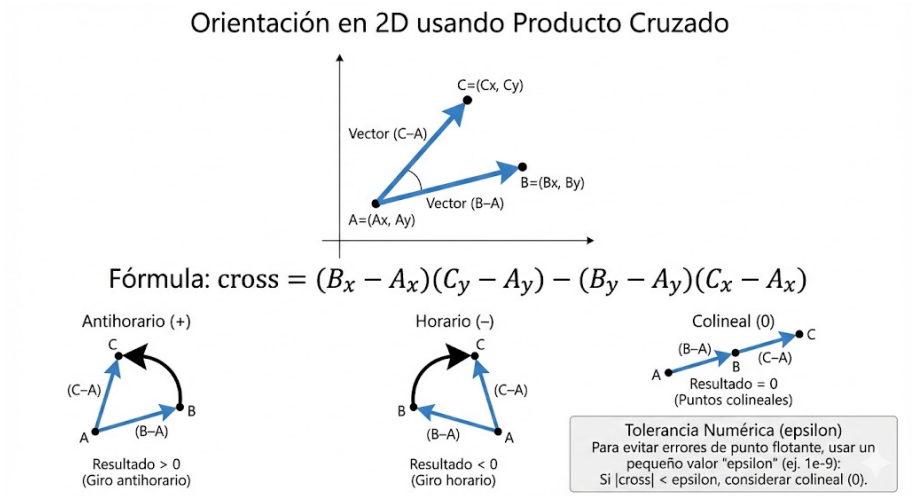


Ilustración 25: Orientación en 2D con producto cruzado: La imagen muestra cómo calcular el signo del producto cruzado entre los vectores (B–A) y (C–A) para decidir si el giro A→B→C es antihorario (positivo), horario (negativo) o colineal (cero, con tolerancia epsilon). Este criterio es la base para detectar "giros a la derecha/izquierda" en algoritmos de geometría computacional (por ejemplo, para construir cercos convexos y validar intersecciones). Imagen creada usando herramientas IA con información proporcionada por la autora.

6.1.1 Ejemplo 1–Primitivas y orientación

```python
# Ejemplo 1: Primitivas 2D, orientación y pertenencia a segmento con tolerancia.
# Script completo y ejecutable (Python estándar).

from typing import Tuple

Point = Tuple[float, float]

EPS = 1e-9

def sub(a: Point, b: Point) -> Point:
    return (a[0] - b[0], a[1] - b[1])

def dot(a: Point, b: Point) -> float:
    return a[0]*b[0] + a[1]*b[1]

def cross(a: Point, b: Point) -> float:
    return a[0]*b[1] - a[1]*b[0]

def sgn(x: float, eps: float = EPS) -> int:
    if x > eps:
        return 1
    if x < -eps:
        return -1
    return 0
```

Captura 69: Se definen primitivas 2D para geometría computacional: tipo Point, una tolerancia numérica EPS y operaciones básicas entre vectores (sub, dot, cross) junto con sgn, que convierte valores con error de punto flotante en signo {-1,0,1}. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def orient(a: Point, b: Point, c: Point, eps: float = EPS) -> int:
    # Signo del área orientada 2D: cross(b-a, c-a)
    return sgn(cross(sub(b, a), sub(c, a)), eps)

def on_segment(a: Point, b: Point, p: Point, eps: float = EPS) -> bool:
    # Maneja degenerado: a == b (segmento de longitud 0)
    if abs(a[0] - b[0]) <= eps and abs(a[1] - b[1]) <= eps:
        return abs(p[0] - a[0]) <= eps and abs(p[1] - a[1]) <= eps

    # Debe ser colineal
    if orient(a, b, p, eps) != 0:
        return False

    # Dentro de la caja envolvente (con eps)
    xmin, xmax = (a[0], b[0]) if a[0] <= b[0] else (b[0], a[0])
    ymin, ymax = (a[1], b[1]) if a[1] <= b[1] else (b[1], a[1])
    return (xmin - eps <= p[0] <= xmax + eps) and (ymin - eps <= p[1] <= ymax + eps)
```

Captura 70: Se implementan dos funciones clave: orient(a,b,c) calcula la orientación de tres puntos mediante el área orientada (producto cruzado) para decidir giro antihorario/horario/colineal; on_segment(a,b,p) verifica si un punto p pertenece al segmento ab, considerando casos degenerados (segmento de longitud 0), colinealidad y pertenencia a la "caja envolvente" con tolerancia EPS. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
# Pruebas simples (prints y asserts)
if __name__ == "__main__":
    a = (0.0, 0.0)
    b = (2.0, 0.0)
    c_left = (1.0, 1.0)
    c_right = (1.0, -1.0)
    c_col = (1.0, 0.0)

    assert orient(a, b, c_left) == 1
    assert orient(a, b, c_right) == -1
    assert orient(a, b, c_col) == 0

    # Pertenencia a segmento
    assert on_segment(a, b, (1.0, 0.0)) is True
    assert on_segment(a, b, (3.0, 0.0)) is False
    assert on_segment(a, b, (1.0, 1.0)) is False

    # Segmento degenerado
    s = (5.0, 5.0)
    assert on_segment(s, s, (5.0, 5.0)) is True
    assert on_segment(s, s, (5.0, 5.0001)) is False

    print("OK: primitivas, orientación y on_segment con degenerados.")
```

Captura 71: Se incluyen pruebas simples con assert para validar: (1) el resultado de orient en casos de giro izquierdo/derecho y colinealidad, (2) pertenencia y no pertenencia de puntos a un segmento, y (3) manejo correcto del caso degenerado a=b. Finaliza imprimiendo un mensaje de verificación exitosa. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`Point`	Define el tipo `(x,y)` para claridad y consistencia.
`EPS`	Fija una tolerancia para comparaciones cercanas a cero.
`sub` , `dot` , `cross`	Implementa operaciones vectoriales base: resta, producto punto y cruzado 2D.
`sgn`	Convierte un real a `{-1,0,1}` según `EPS` , estabilizando decisiones lógicas.
`orient(a,b,c)`	Calcula el signo de `cross(b-a, c-a)` para giro antihorario/horario/colineal.
`on_segment(a,b,p)`	Trata el segmento degenerado `a==b` ; luego exige colinealidad y verifica caja envolvente con `EPS` .
`assert` de orientación	Verifica los tres casos: izquierda, derecha y colinealidad.
`assert` de segmento	Prueba interior, exterior y punto fuera de la recta.
Prueba degenerada	Garantiza comportamiento definido cuando el "segmento" es un punto.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

6.2 Cerco convexo (Convex Hull): Monotonic Chain y usos

El cerco convexo de un conjunto de puntos es el polígono convexo mínimo que los contiene. Es un problema central porque reduce datos geométricos a su "envolvente" útil para filtrado espacial, colisiones aproximadas, análisis de forma y preprocesamiento para consultas (de Berg et al., 2008).

Un algoritmo estándar en 2D es Monotonic Chain (Andrew): ordena puntos lexicográficamente y construye una envolvente inferior y superior eliminando giros no convexos mediante el predicado de orientación. Su complejidad es $O(n \log n)$ por el ordenamiento (O'Rourke, 1998).

Definiciones clave

- **Conjunto convexo:** Conjunto donde el segmento entre dos puntos cualesquiera queda contenido en el conjunto.
- **Cerco convexo (convex hull):** Menor polígono convexo que contiene a todos los puntos del conjunto.
- **Política de colineales:** Regla para incluir o excluir puntos colineales sobre un lado del hull (impacta el conjunto de vértices devuelto).
- **Monotonic Chain (Andrew):** Algoritmo que construye el hull concatenando una cadena inferior y una superior tras ordenar los puntos.

6.3 Paso a paso del cerco convexo

6.3.1 Paso 1: Ordenamiento de los puntos

El primer paso crucial es ordenar los puntos de entrada lexicográficamente. Esto significa que se ordenan primero por su coordenada x, y en caso de empate, por su coordenada y. Este ordenamiento es fundamental para los siguientes pasos del algoritmo.

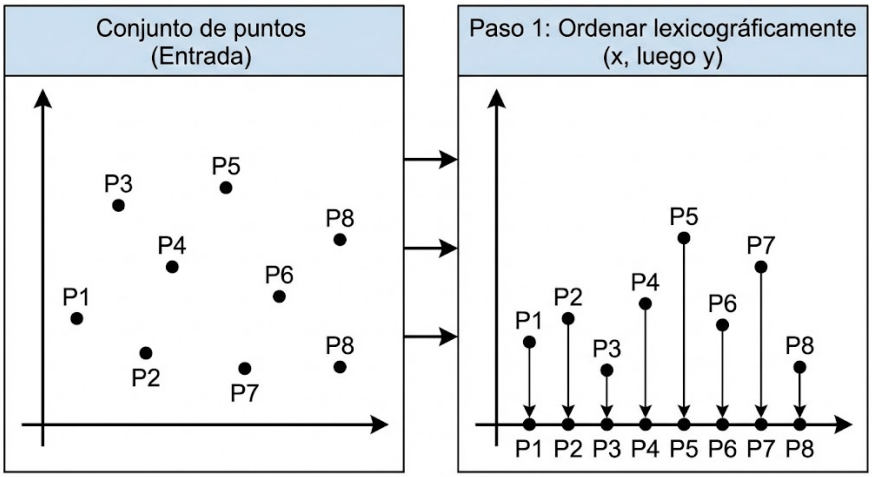


Ilustración 26: Cercos convexos – Paso 1: ordenar puntos: Se ilustra el preprocesamiento del algoritmo (tipo Monotone Chain): los puntos de entrada se ordenan lexicográficamente (primero por x, luego por y). Este orden permite recorrerlos de forma consistente para construir las cadenas inferior y superior del cerco convexo. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.3.2 Paso 2: Construcción de la cadena inferior

Utilizando los puntos ordenados, se construye la "cadena inferior" del cerco convexo. Se iteran los puntos desde el primero (P1) hasta el último (Pn). Se utiliza una pila para mantener los puntos candidatos de la cadena. En cada paso, antes de añadir un nuevo punto, se comprueba si los dos últimos puntos de la pila junto con el nuevo punto forman un giro a la derecha o son colineales. Si es así, el penúltimo punto se elimina de la pila ("desapilar"). Este proceso se repite hasta que se forma un giro a la izquierda, asegurando la convexidad de la cadena.

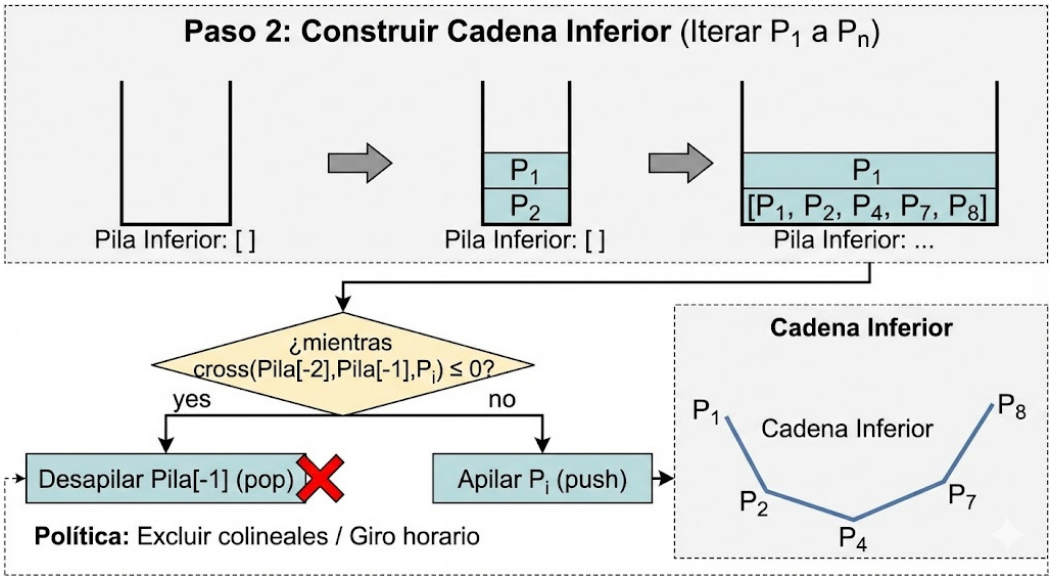


Ilustración 27: Cercos convexos – Paso 2: construir la cadena inferior: La imagen explica el armado de la cadena inferior recorriendo puntos de P1 a Pn usando una pila. En cada nuevo punto, se evalúa la condición basada en el producto cruzado; si aparece un giro horario o colineal no deseado (según la política), se hace pop hasta recuperar convexidad y luego se hace push del punto actual. Esto muestra cómo el algoritmo "elimina" puntos internos para conservar solo los vértices del borde inferior. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.3.3 Paso 3: Construcción de la cadena superior y unión final

De manera análoga a la cadena inferior, se construye la "cadena superior" iterando sobre los puntos ordenados en orden inverso, desde el último (Pn) hasta el primero (P1). Se utiliza el mismo principio de pila y comprobación de giros para mantener la convexidad.

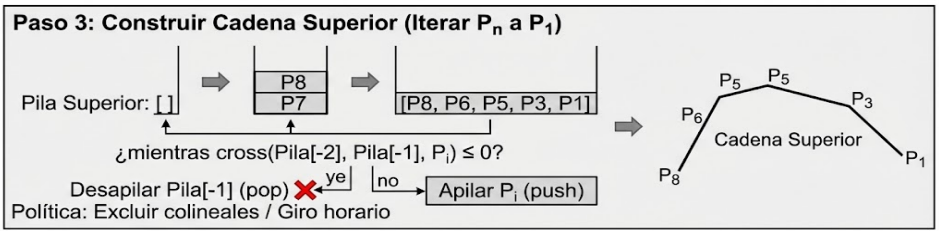


Ilustración 28: Cercos convexos – Paso 3: construir la cadena superior: Se presenta el armado análogo de la cadena superior, ahora recorriendo los puntos en sentido inverso (Pn a P1). Se usa la misma regla de orientación (producto cruzado) para decidir cuándo desapilar y mantener la convexidad, hasta obtener la envolvente superior del conjunto. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.3.4 Paso 4: Unir y obtener hull final

Finalmente, en el Paso 4, se unen las dos cadenas resultantes. Se eliminan el primer punto y el último punto de una de las cadenas para evitar duplicados en los extremos de unión (ya que P1 y Pn pertenecen a ambas cadenas). La concatenación de la cadena inferior y la superior forma el polígono convexo final.

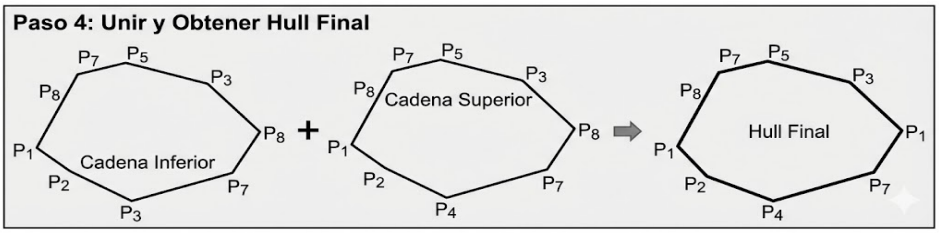


Ilustración 29: Cercos convexos – Paso 4: unir cadenas y obtener el hull final: La imagen muestra la composición final: se concatena la cadena inferior y la cadena superior (evitando duplicar extremos) para formar el cerco convexo completo. Este resultado es el polígono mínimo que contiene a todos los puntos, central en aplicaciones como bounding shapes, planificación, clustering geométrico y preprocesamientos en problemas de proximidad. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.3.4 Ejemplo 2–Cerco convexo (Monotonic Chain)

```python
# Ejemplo 2: Convex Hull con Monotonic Chain (Andrew).
# Política elegida: EXCLUIR puntos colineales intermedios (mantener extremos).
# Script completo y ejecutable (Python estándar).

from typing import List, Tuple

Point = Tuple[float, float]
EPS = 1e-9

def cross(o: Point, a: Point, b: Point) -> float:
    # cross(oa, ob) donde oa = a-o, ob = b-o
    return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])

def convex_hull(points: List[Point]) -> List[Point]:
    # Elimina duplicados
    pts = sorted(set(points))
    if len(pts) <= 1:
        return pts

    # Construye parte inferior
    lower: List[Point] = []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= EPS:
            # <= EPS => elimina colineales y giros a la derecha (política: excluir colineales)
            lower.pop()
        lower.append(p)

    # Construye parte superior
    upper: List[Point] = []
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= EPS:
            upper.pop()
        upper.append(p)

    # Concatenar sin repetir extremos
    hull = lower[:-1] + upper[:-1]
    return hull
```

Captura 72: (inicio del script). Se implementa el algoritmo Convex Hull por Monotonic Chain (Andrew): se define el producto cruzado cross(o,a,b) para evaluar giros, se ordenan y depuran puntos duplicados, y se construyen las cadenas inferior y superior eliminando puntos colineales intermedios (política: mantener solo extremos) mediante la condición cross(...) <= EPS. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def point_in_convex_polygon(hull: List[Point], p: Point) -> bool:
    # Validación parcial por muestreo: si hull es convexo, p está dentro o en borde
    # cuando no existe orientación negativa respecto a todas las aristas (con EPS).
    n = len(hull)
    if n == 0:
        return False
    if n == 1:
        return (abs(hull[0][0]-p[0]) <= EPS and abs(hull[0][1]-p[1]) <= EPS)
    if n == 2:
        # segmento
        a, b = hull
        # colineal y en caja
        area = cross(a, b, p)
        if abs(area) > EPS:
            return False
        xmin, xmax = (a[0], b[0]) if a[0] <= b[0] else (b[0], a[0])
        ymin, ymax = (a[1], b[1]) if a[1] <= b[1] else (b[1], a[1])
        return xmin - EPS <= p[0] <= xmax + EPS and ymin - EPS <= p[1] <= ymax + EPS

    for i in range(n):
        a = hull[i]
        b = hull[(i+1) % n]
        if cross(a, b, p) < -EPS:
            return False
    return True
```

Captura 73: (continuación). Se define point_in_convex_polygon(hull, p) para validar pertenencia de un punto al polígono convexo resultante: contempla casos degenerados (hull vacío, de 1 punto o de 2 puntos/segmento) y, para el caso general, verifica que no exista orientación "negativa" respecto de ninguna arista (con tolerancia EPS), aceptando puntos en borde. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
if __name__ == "__main__":
    pts = [
        (0,0), (2,0), (2,2), (0,2),   # cuadrados extremos
        (1,1), (1,0), (0,1),          # interiores y en borde
        (2,1), (1,2),
        (1,0), (2,2),                 # duplicados
        (1,0.0),                      # colineal
    ]
    hull = convex_hull(pts)
    print("Hull:", hull)

    # Asserts: extremos esperados
    expected_extremes = {(0,0), (2,0), (2,2), (0,2)}
    assert expected_extremes.issubset(set(hull))

    # Muestreo: todos los puntos originales deben quedar dentro o en borde del hull
    for p in set(pts):
        assert point_in_convex_polygon(hull, p), f"Punto fuera del hull: {p}"

    # Prueba colineales: en el borde inferior y=0, el punto (1,0) no debe ser vértice intermedio
    # con la política de excluir colineales (mantener extremos)
    assert (1,0) not in hull, "Con política de excluir colineales, (1,0) no debe estar en el hull."

    print("OK: convex hull Monotonic Chain con validaciones básicas.")
```

Captura 74: (cierre y pruebas). Bloque principal con un conjunto de puntos de prueba (extremos, interiores, duplicados y colineales), cálculo del hull, e asserts para: (1) comprobar que los vértices extremos esperados están en el hull, (2) verificar por muestreo que todos los puntos quedan dentro o en el borde del hull, y (3) validar la política de exclusión de colineales (por ejemplo, que (1,0) no quede como vértice intermedio). Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`sorted(set(points))`	Normaliza entrada eliminando duplicados y ordenando lexicográficamente.
`cross(o,a,b)`	Calcula el área orientada para decidir convexidad local (giro).
`lower`	Construye cadena inferior; elimina últimos puntos mientras el giro no sea estrictamente antihorario (`<= EPS`).
`upper`	Construye cadena superior sobre los puntos invertidos, con el mismo criterio.
`hull = lower[:-1] + upper[:-1]`	Concatena sin repetir los extremos (primer/último punto).
Política de colineales	`<= EPS` elimina colineales intermedios, preservando extremos del lado.
`point_in_convex_polygon`	Verificación parcial para asegurar que los puntos caen dentro/en borde del hull (control básico).
Asserts de extremos	Garantiza que los vértices del rectángulo aparezcan en el resultado.
Asserts de contención	Recorre puntos de entrada y valida que no queden fuera del hull.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

6.4 Inclusión en polígonos: ray casting, borde y no convexidad

El problema punto-en-polígono determina si un punto está dentro, fuera o sobre el borde de un polígono simple (sin autointersecciones). Un enfoque clásico es ray casting: se lanza un rayo desde el punto y se cuenta cuántas veces intersecta los lados; paridad impar implica "dentro", paridad par implica "fuera". La dificultad práctica está en manejar degenerados: el rayo pasando por un vértice, puntos sobre aristas y polígonos con lados horizontales (Preparata & Shamos, 1985; O'Rourke, 1998).

Definiciones clave

- **Polígono simple:** Polígono cuyos lados no se cruzan entre sí salvo en vértices consecutivos.
- **Ray casting:** Técnica de paridad de intersecciones entre un rayo y los lados del polígono.
- **Punto en borde:** Punto que pertenece a alguna arista o vértice del polígono (se trata como caso separado).
- **Winding (mención):** Técnica alternativa basada en el número de vueltas alrededor del punto (concepto relacionado).

6.5 Paso a Paso: ray casting

6.5.1 Paso 1: Configuración inicial

Primero, dibujamos un polígono (en este caso, uno no convexo) y un punto P que queremos analizar.

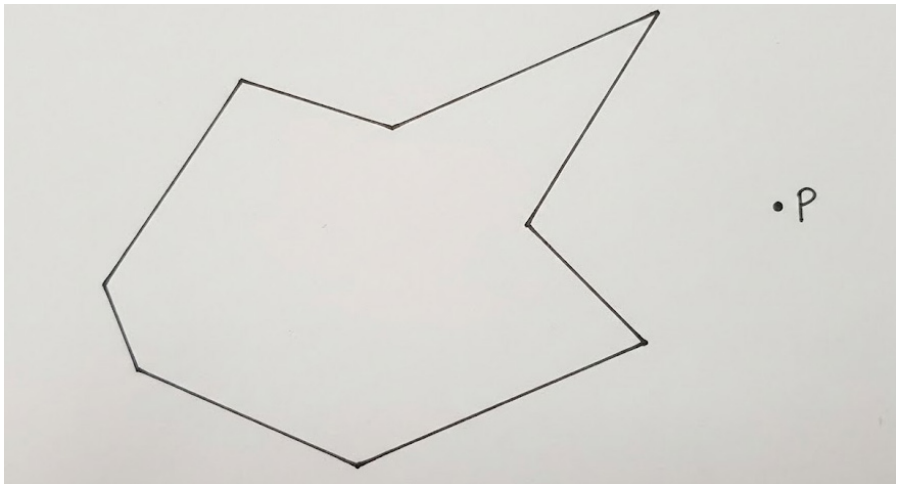


Ilustración 30: Punto y polígono (situación inicial). Se muestra un polígono (posiblemente cóncavo) y un punto P cuya pertenencia se quiere determinar (si está dentro o fuera). Esta figura introduce el problema "punto en polígono", base para algoritmos de inclusión en polígonos en geometría computacional. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.5.2 Paso 2: Trazar el rayo y marcar intersecciones

A continuación, trazamos un rayo horizontal desde el punto P hacia la derecha (puede ser en cualquier dirección fija). Marcamos con círculos rojos los puntos donde este rayo intersecta las aristas del polígono.

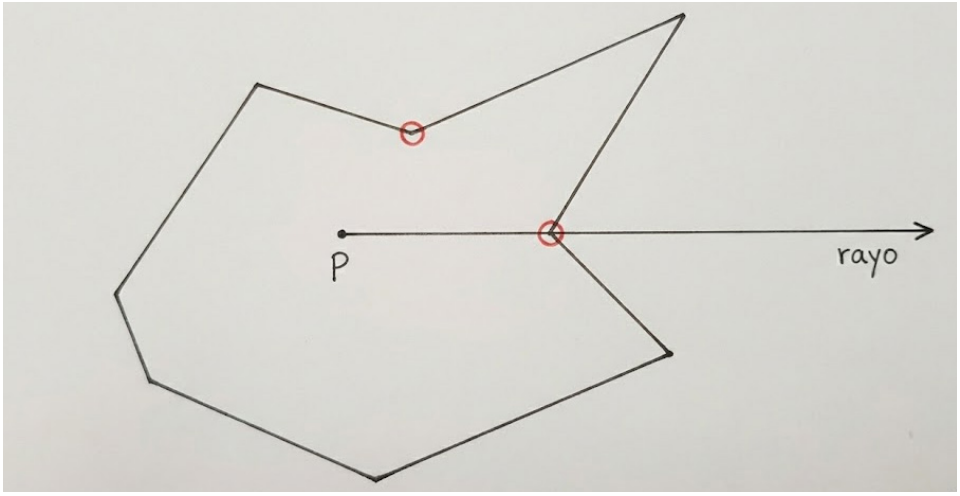


Ilustración 31: Método del rayo (ray casting): conteo de intersecciones. Desde el punto P se traza un rayo horizontal hacia la derecha y se marcan las intersecciones con los lados del polígono. La idea es contar cuántas veces el rayo cruza el borde: si el número de cruces es impar, P está dentro; si es par, P está fuera. Esta imagen muestra el procedimiento de trazado y el marcado de cruces. Imagen creada usando herramientas IA con información proporcionada por la autora.

6.5.3 Paso 3: Contar la paridad y casos especiales

Finalmente, contamos el número de intersecciones. Si el número es impar, el punto está dentro; si es par, está fuera. En nuestro ejemplo, hay 2 intersecciones (par), por lo que el punto P está afuera.

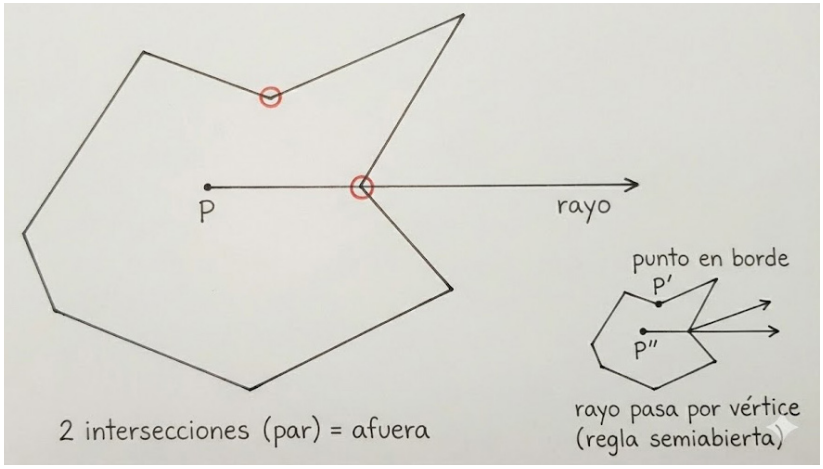


Ilustración 32 Regla par/impar y caso especial: rayo pasando por vértice. Se explicita el criterio: "2 intersecciones (par) = afuera" para el ejemplo mostrado. Además, se advierte el caso problemático cuando el rayo pasa exactamente por un vértice, indicando la regla de semirrecta (semiabierta) para evitar contar doble (tratando el vértice de forma consistente). Esto se relaciona con la implementación robusta del test de inclusión en polígonos (manejo de colinealidad y vértices). Imagen creada usando herramientas IA con información proporcionada por la autora.

Como consideraciones finales, pueden aparecer algunos casos límite. Cuando el punto se encuentra exactamente sobre el borde (P'), suele adoptarse la convención de considerarlo dentro del polígono. Por otro lado, si el rayo pasa exactamente por un vértice (P''), se aplica una "regla semiabierta": la intersección se cuenta solo con una de las aristas que comparten ese vértice, evitando así un doble conteo.

6.5.4 Ejemplo 3–Inclusión punto-en-polígono (ray casting + estado 0/1/2)

```python
# Ejemplo 3: Punto-en-polígono por ray casting.
# Devuelve: 0=fuera, 1=dentro, 2=en el borde (arista o vértice).
# Script completo y ejecutable (Python estándar).

from typing import List, Tuple

Point = Tuple[float, float]
EPS = 1e-9

def orient(a: Point, b: Point, c: Point) -> float:
    return (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])

def on_segment(a: Point, b: Point, p: Point) -> bool:
    # Colinealidad
    if abs(orient(a, b, p)) > EPS:
        return False
    xmin, xmax = (a[0], b[0]) if a[0] <= b[0] else (b[0], a[0])
    ymin, ymax = (a[1], b[1]) if a[1] <= b[1] else (b[1], a[1])
    return (xmin - EPS <= p[0] <= xmax + EPS) and (ymin - EPS <= p[1] <= ymax + EPS)

def point_in_polygon(poly: List[Point], p: Point) -> int:
    n = len(poly)
    if n < 3:
        return 0

    # Caso borde primero
    for i in range(n):
        a = poly[i]
        b = poly[(i+1) % n]
        if on_segment(a, b, p):
            return 2

```

Captura 75: Implementación del Ejemplo 3: punto-en-polígono por ray casting. Se definen tipos y tolerancia EPS, y dos funciones auxiliares: orient(a,b,c) (producto cruzado/área orientada para evaluar colinealidad) y on_segment(a,b,p) (verifica si el punto p cae sobre el segmento ab, considerando tolerancia numérica). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    # Ray casting hacia +x
    inside = False
    x, y = p
    for i in range(n):
        x1, y1 = poly[i]
        x2, y2 = poly[(i+1) % n]

        # Normalización: considerar aristas que cruzan el nivel y
        # Condición estándar con semiabierto para evitar doble conteo en vértices:
        intersects = ((y1 > y) != (y2 > y))
        if intersects:
            # Coordenada x de intersección del segmento con la horizontal y
            x_int = x1 + (y - y1) * (x2 - x1) / (y2 - y1)
            if x_int > x + EPS:
                inside = not inside

    return 1 if inside else 0
```

Captura 76: Núcleo del algoritmo ray casting hacia +x: para cada arista del polígono se decide si "cruza" el nivel horizontal del punto (intersects) usando una regla semiabierta para evitar doble conteo en vértices. Cuando hay cruce, se calcula x_int (coordenada x de la intersección) y se alterna el estado inside. El retorno es 1 si queda dentro y 0 si queda fuera. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
if __name__ == "__main__":
    # Polígono convexo (cuadrado)
    square = [(0,0), (4,0), (4,4), (0,4)]
    assert point_in_polygon(square, (2,2)) == 1
    assert point_in_polygon(square, (5,5)) == 0
    assert point_in_polygon(square, (0,2)) == 2
    assert point_in_polygon(square, (0,0)) == 2

    # Polígono no convexo (forma de "L")
    poly_L = [(0,0), (4,0), (4,1), (1,1), (1,4), (0,4)]
    assert point_in_polygon(poly_L, (0.5, 3.0)) == 1   # dentro
    assert point_in_polygon(poly_L, (2.0, 2.0)) == 0   # fuera (hueco del L)
    assert point_in_polygon(poly_L, (1.0, 2.0)) == 2   # borde (x=1)

    print("OK: point_in_polygon con convexo/no convexo y borde.")
```

Captura 77: Bloque de pruebas del ejemplo: se validan casos en un polígono convexo (cuadrado) y en un polígono no convexo (forma "L"), incluyendo puntos dentro, fuera y en el borde (devuelve 2 cuando on_segment detecta borde). Las aserciones (assert) confirman el comportamiento esperado. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
Pre-chequeo de borde	Recorre aristas y usa `on_segment` para devolver estado `2` antes de aplicar paridad.
Regla semiabierta `((y1>y)!=(y2>y))`	Evita contar dos veces cuando el rayo pasa por un vértice.
`x_int`	Calcula la abscisa de intersección del segmento con la recta horizontal $y = \text{const}$.
Comparación `x_int > x + EPS`	Cuenta intersecciones a la derecha del punto, con tolerancia numérica.
Toggle `inside = not inside`	Implementa la paridad de intersecciones (impar ⇒ dentro).
Asserts convexo	Verifica dentro/fuera/borde en un cuadrado.
Asserts no convexo	Valida un caso donde la paridad distingue "hueco" en un polígono no convexo.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

6.6 Intersección de segmentos: orientación y colinealidad

La intersección de segmentos es un predicado básico en geometría computacional y aparece en tareas como detección de colisiones en 2D, recorte geométrico o validación de polígonos simples. El criterio habitual se basa en analizar la orientación de los puntos: en el caso general, dos segmentos se cruzan si los extremos de uno quedan a lados opuestos del otro. Para determinarlo se evalúan cuatro orientaciones entre los puntos de ambos segmentos. A esto se agregan casos especiales en los que los puntos son colineales; allí se comprueba si alguno de los extremos pertenece al otro segmento, lo que permite detectar contacto o superposición (de Berg et al., 2008; O'Rourke, 1998).

Además, el algoritmo debe contemplar situaciones degeneradas. Entre ellas se incluyen segmentos de longitud cero, intersecciones que ocurren exactamente en un extremo y casos en los que dos segmentos colineales se superponen parcialmente.

Énfasis

Definiciones clave

- **Intersección de segmentos:** Existencia de un punto común entre los conjuntos $[p_1, q_1]$ y $[p_2, q_2]$.
- **Caso general:** Intersección por cruce estricto, determinada por orientaciones opuestas.
- **Caso colineal:** Segmentos con orientación cero; la intersección se decide por solapamiento de proyecciones.
- **Contacto en extremo:** Intersección donde el punto común es un extremo de alguno de los segmentos.

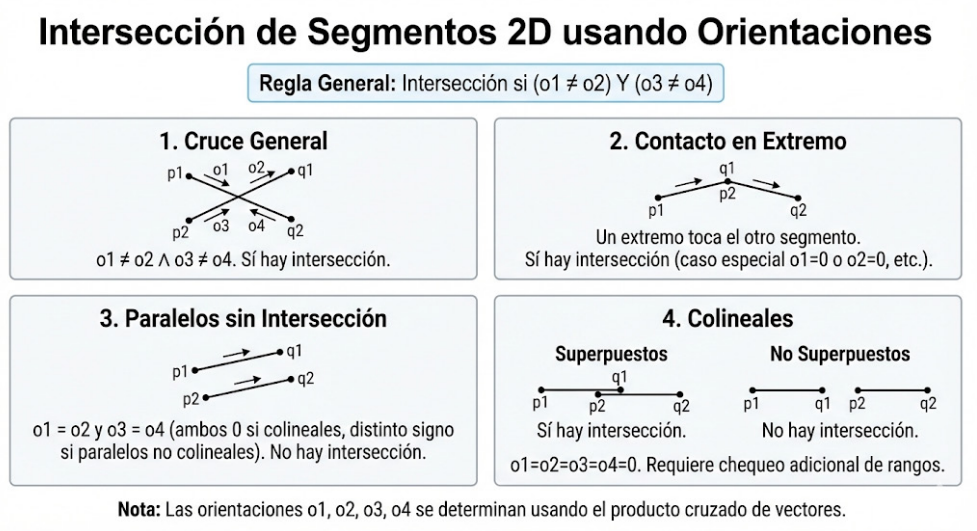


Ilustración 33: Intersección de segmentos 2D usando orientaciones (producto cruzado). La imagen resume el criterio geométrico para decidir si dos segmentos se intersectan comparando las orientaciones $o1, o2, o3, o4$: hay cruce general cuando $o1 \neq o2$ y $o3 \neq o4$, y se contemplan casos especiales (contacto en extremos y colinealidad con verificación de solapamiento). Se relaciona con el contenido al fundamentar algoritmos robustos de intersección de segmentos en geometría computacional. Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.6.1 Ejemplo 4–Intersección de segmentos con casos especiales

```python
# Ejemplo 4: Intersección de segmentos 2D (caso general + colineales + degenerados).
# Script completo y ejecutable (Python estándar).

from typing import Tuple

Point = Tuple[float, float]
EPS = 1e-9
 Press Ctrl K to generate code
def orient(a: Point, b: Point, c: Point) -> int:
    val = (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])
    if val > EPS:
        return 1
    if val < -EPS:
        return -1
    return 0

def on_segment(a: Point, b: Point, p: Point) -> bool:
    # Segmento degenerado
    if abs(a[0]-b[0]) <= EPS and abs(a[1]-b[1]) <= EPS:
        return abs(p[0]-a[0]) <= EPS and abs(p[1]-a[1]) <= EPS

    if orient(a, b, p) != 0:
        return False
    xmin, xmax = (a[0], b[0]) if a[0] <= b[0] else (b[0], a[0])
    ymin, ymax = (a[1], b[1]) if a[1] <= b[1] else (b[1], a[1])
    return xmin - EPS <= p[0] <= xmax + EPS and ymin - EPS <= p[1] <= ymax + EPS

def segment_intersect(p1: Point, q1: Point, p2: Point, q2: Point) -> bool:
    o1 = orient(p1, q1, p2)
    o2 = orient(p1, q1, q2)
    o3 = orient(p2, q2, p1)
    o4 = orient(p2, q2, q1)

    # Caso general
    if o1 != o2 and o3 != o4:
        return True
```

Captura 78: Implementación de intersección de segmentos 2D basada en orientaciones (signo del área orientada) con tolerancia numérica EPS. Se definen orient() para clasificar el giro (antihorario/horario/colineal) y on_segment() para verificar si un punto pertenece al segmento (incluye el caso degenerado donde el segmento es un punto). Desarrollo en Online IDE Pro (Python estándar). Código fuente de la autora.

```python
    # Casos especiales colineales
    if o1 == 0 and on_segment(p1, q1, p2):
        return True
    if o2 == 0 and on_segment(p1, q1, q2):
        return True
    if o3 == 0 and on_segment(p2, q2, p1):
        return True
    if o4 == 0 and on_segment(p2, q2, q1):
        return True

    return False

if __name__ == "__main__":
    # Intersección cruzada
    assert segment_intersect((0,0), (2,2), (0,2), (2,0)) is True

    # Contacto en extremo
    assert segment_intersect((0,0), (2,0), (2,0), (3,1)) is True

    # Paralelos sin intersección
    assert segment_intersect((0,0), (2,0), (0,1), (2,1)) is False

    # Colineales superpuestos
    assert segment_intersect((0,0), (3,0), (1,0), (2,0)) is True

    # Colineales no superpuestos
    assert segment_intersect((0,0), (1,0), (2,0), (3,0)) is False

    # Segmento degenerado (punto)
    assert segment_intersect((1,1), (1,1), (0,0), (2,2)) is True  # el punto está sobre el segmento
    assert segment_intersect((1,2), (1,2), (0,0), (2,2)) is False

    print("OK: segment_intersect con casos general/especiales y degenerados.")
```

Captura 79: Definición de segment_intersect(p1,q1,p2,q2) que combina el caso general (orientaciones distintas) con casos especiales colineales (superposición o contacto) usando on_segment(). Se incluyen pruebas con assert para validar cruces, contacto en extremo, paralelos sin intersección, colineales superpuestos/no superpuestos y segmentos degenerados. Desarrollo en Online IDE Pro (Python estándar). Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`orient` (entero)	Devuelve -1/0/1 usando `EPS` para estabilidad numérica.
`on_segment`	Verifica segmento degenerado y luego colinealidad + caja envolvente.
`o1..o4`	Orientaciones necesarias para el criterio de cruce entre dos segmentos.
Caso general `o1!=o2 and o3!=o4`	Determina cruce estricto (cada segmento "separa" los extremos del otro).
Casos colineales	Si algún punto del segundo segmento cae sobre el primero (o viceversa), hay intersección.
Asserts cruzados/extremos	Prueban cruce y contacto en extremo como intersecciones válidas.
Asserts paralelos/colineales	Cubren no intersección y superposición colineal.
Asserts degenerados	Define conducta consistente cuando un segmento se reduce a un punto.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

6.7 Búsqueda por rango (range search): consulta ortogonal y k-d tree introductorio

En búsqueda por rango ortogonal se consulta un conjunto de puntos para recuperar aquellos que caen dentro de un rectángulo axis-aligned [x , x] × [y , y]. El enfoque ingenuo consiste en revisar todos los puntos, con costo O(n). Cuando deben resolverse muchas consultas sobre datos estáticos, resulta conveniente utilizar estructuras de indexación como los k-d trees, que generalizan la búsqueda binaria alternando ejes y permiten podar regiones completas del espacio durante la búsqueda (Bentley, 1975; de Berg et al., 2008).

En la práctica, el rendimiento depende de la distribución de los puntos y de la dimensionalidad del espacio. En dos dimensiones estas estructuras suelen funcionar bien, pero en dimensiones altas su desempeño puede degradarse (de Berg et al., 2008).

Énfasis

Definiciones clave

- **Consulta ortogonal 2D:** Recuperación de puntos dentro de un rectángulo axis-aligned.
- **Range search:** Familia de problemas de consulta espacial por región (rectángulos, círculos, etc.).
- **k-d tree:** Árbol binario que particiona el espacio alternando ejes, usado para búsquedas espaciales.
- **Poda (pruning):** Evitar explorar subárboles cuando su región no puede intersectar la consulta.

6.8 Paso a paso – Range Search

6.8.1 Paso 1: Configuración inicial

En este primer paso, visualizamos el espacio de búsqueda 2D. Tenemos un conjunto de puntos azules dispersos y un rectángulo de consulta (bbox), que define el rango en el que queremos encontrar puntos. Los ejes 'x' e 'y' están etiquetados.

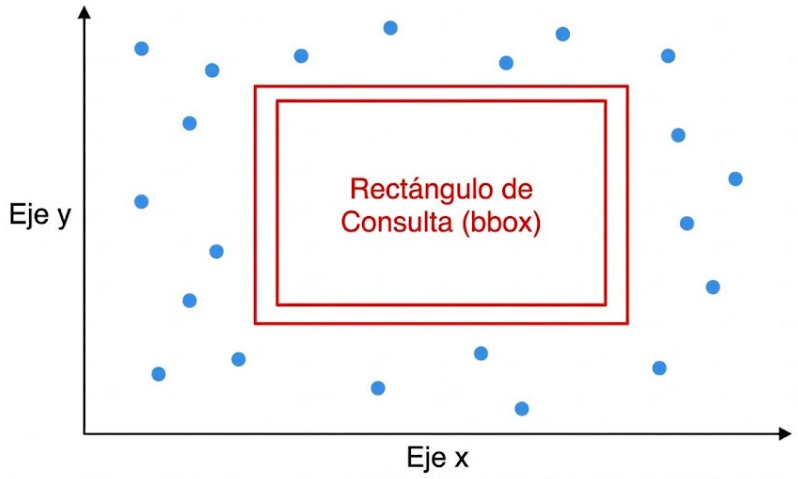


Ilustración 34: Búsqueda por rango: rectángulo de consulta (bounding box). Se muestra un conjunto de puntos en el plano y el rectángulo (bbox) que define la región de consulta: el objetivo es recuperar solo los puntos que caen dentro de ese rectángulo. Se relaciona con el contenido al introducir consultas espaciales y la necesidad de estructuras de indexación (p. ej., k-d tree, R-tree). Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.8.2 Paso 2: Primera partición y poda

El algoritmo k-d tree comienza particionando el espacio. En este caso, la primera partición se hace con una línea vertical (eje x). Observa que la subregión izquierda generada no intersecta con el rectángulo de consulta. Esta región se marca como "No intersecta (Poda)" y se descarta, ya que no puede contener puntos de interés. La búsqueda continúa solo en la región derecha.

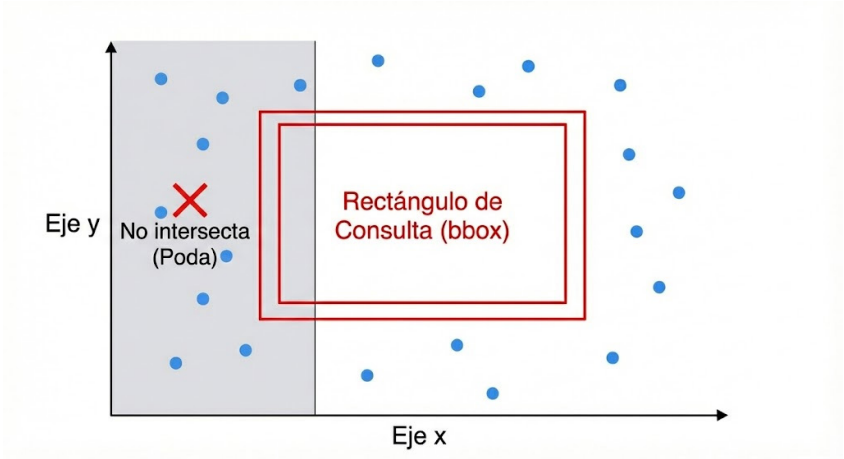


Ilustración 35: Primera poda por no intersección con el rectángulo. Aparece una región sombreada marcada como "No intersecta (Poda)": representa una partición/área del espacio cuyo bounding box no se superpone con el rectángulo de consulta, por lo que puede descartarse sin revisar punto por punto. Se relaciona con el contenido al explicar cómo las estructuras espaciales aceleran búsquedas evitando trabajo innecesario. Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.8.3 Paso 3: Segunda partición y poda

En la región activa restante, se realiza una segunda partición, esta vez alternando al eje 'y' con una línea horizontal. Al igual que en el paso anterior, una de las nuevas subregio-

nes (la superior en este caso) no intersecta con el rectángulo de consulta y se poda. Esto reduce aún más el espacio de búsqueda.

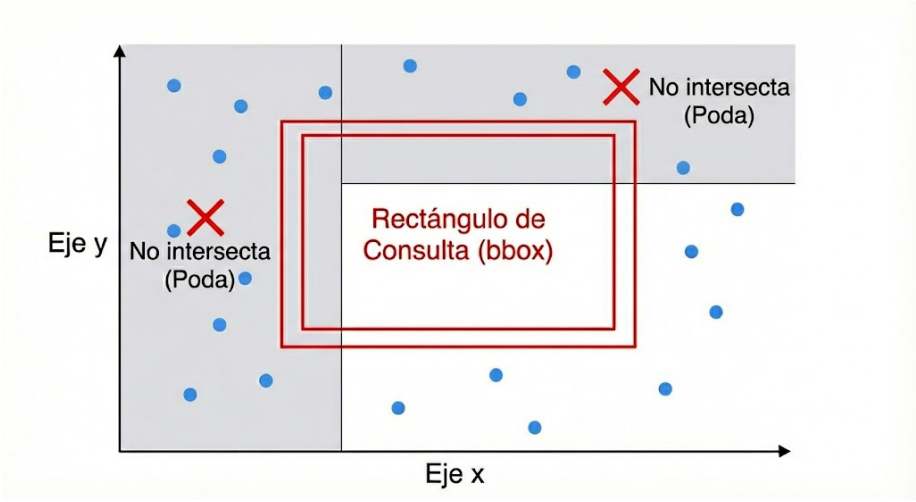


Ilustración 36: Poda acumulada en particiones múltiples. Se ilustran varias zonas sombreadas que no intersectan el bbox de consulta; cada una corresponde a subregiones (nodos) descartadas durante el recorrido de la estructura espacial. Se relaciona con el contenido al mostrar el principio de "pruning" que reduce la complejidad de una consulta por rango. Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.8.4 Paso 4: Resultado final y puntos encontrados

El proceso continúa con una tercera partición (vertical de nuevo) en la región activa. Esta vez, una de las subregiones resultantes está completamente contenida dentro del rectángulo de consulta. Los puntos dentro de esta región se resaltan en verde, indicando que son el resultado de la búsqueda por rango. Las demás regiones que no intersectan o que se descartan en este paso se muestran como podadas. Este es el estado final del algoritmo.

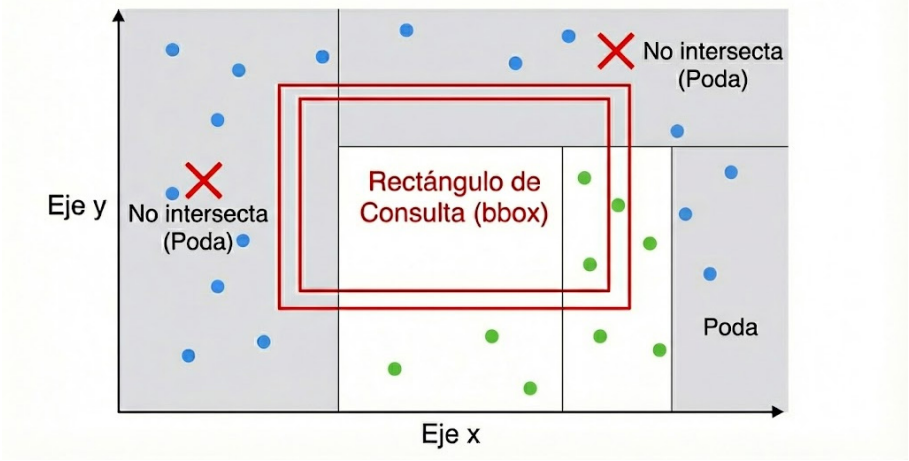


Ilustración 37: Región candidata y puntos reportados. Además de las podas, se observa una zona donde sí hay intersección y aparecen puntos destacados (verdes) como candidatos/resultado: solo esas subregiones se exploran y se devuelven los puntos dentro del rectángulo de consulta. Se relaciona con el contenido al conectar el filtrado por bounding boxes con el conjunto final de resultados. Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.8.5 Ejemplo 5–Range search: ingenuo + k-d tree didáctico + SciPy opcional

```python
# Ejemplo 5: Búsqueda por rango 2D (rectángulo axis-aligned).
# - Versión principal: ingenua O(n).
# - Versión acelerada (sin librerías): k-d tree mínimo (con poda por región).
# - Variante opcional con SciPy: KDTree en try/except (no rompe si no está instalada).
# Script completo y ejecutable (Python estándar).

from typing import List, Tuple, Optional
import random

Point = Tuple[float, float]
Rect = Tuple[float, float, float, float]  # xmin, xmax, ymin, ymax
EPS = 1e-9

def in_rect(p: Point, rect: Rect) -> bool:
    xmin, xmax, ymin, ymax = rect
    return (xmin - EPS <= p[0] <= xmax + EPS) and (ymin - EPS <= p[1] <= ymax + EPS)

def range_search_naive(points: List[Point], rect: Rect) -> List[Point]:
    return [p for p in points if in_rect(p, rect)]

class KDNode:
    __slots__ = ("point", "axis", "left", "right", "minx", "maxx", "miny", "maxy")
    def __init__(self, point: Point, axis: int,
                 left: Optional["KDNode"], right: Optional["KDNode"],
                 bbox: Rect):
        self.point = point
        self.axis = axis
        self.left = left
        self.right = right
        xmin, xmax, ymin, ymax = bbox
        self.minx, self.maxx, self.miny, self.maxy = xmin, xmax, ymin, ymax
```

Captura 80: Se definen los tipos de datos y utilidades para la búsqueda por rango 2D: un punto (x, y), un rectángulo axis-aligned (xmin, xmax, ymin, ymax) y una tolerancia numérica (EPS). Además, se implementa in_rect() para verificar si un punto cae dentro del rectángulo de consulta y range_search_naive() como versión base O(n) que filtra todos los puntos. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def bbox_union(b1: Rect, b2: Rect) -> Rect:
    return (min(b1[0], b2[0]), max(b1[1], b2[1]), min(b1[2], b2[2]), max(b1[3], b2[3]))

def bbox_point(p: Point) -> Rect:
    return (p[0], p[0], p[1], p[1])

def build_kdtree(points: List[Point], depth: int = 0) -> Optional[KDNode]:
    if not points:
        return None
    axis = depth % 2
    points_sorted = sorted(points, key=lambda x: x[axis])
    mid = len(points_sorted) // 2
    median = points_sorted[mid]
    left = build_kdtree(points_sorted[:mid], depth + 1)
    right = build_kdtree(points_sorted[mid+1:], depth + 1)

    # bbox del nodo: unión de bbox del punto + hijos
    bbox = bbox_point(median)
    if left:
        bbox = bbox_union(bbox, (left.minx, left.maxx, left.miny, left.maxy))
    if right:
        bbox = bbox_union(bbox, (right.minx, right.maxx, right.miny, right.maxy))

    return KDNode(median, axis, left, right, bbox)
```

Captura 81: Se introduce la estructura KDNode para un k-d tree mínimo, almacenando el punto del nodo, el eje de partición (x o y), referencias a hijos y la bounding box (caja envolvente) del subárbol. Esta información permitirá acelerar la búsqueda por rango mediante poda por región (descartar subárboles completos). Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def rect_intersects_bbox(rect: Rect, node: KDNode) -> bool:
    xmin, xmax, ymin, ymax = rect
    # no intersecta si hay separación en algún eje
    if xmax < node.minx - EPS or xmin > node.maxx + EPS:
        return False
    if ymax < node.miny - EPS or ymin > node.maxy + EPS:
        return False
    return True

def range_search_kdtree(root: Optional[KDNode], rect: Rect, out: List[Point]) -> None:
    if root is None:
        return
    if not rect_intersects_bbox(rect, root):
        return  # poda

    if in_rect(root.point, rect):
        out.append(root.point)

    if root.left:
        range_search_kdtree(root.left, rect, out)
    if root.right:
        range_search_kdtree(root.right, rect, out)
```

Captura 82: Se implementa la construcción del k-d tree con build_kdtree(): ordena por el eje alternante, selecciona la mediana como raíz y construye recursivamente los subárboles. Luego calcula la bounding box del nodo como unión de la caja del punto y las cajas de los hijos, preparando la estructura para podas eficientes. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def range_search_scipy_optional(points: List[Point], rect: Rect) -> Optional[List[Point]]:
    try:
        from scipy.spatial import KDTree  # opcional
        # Nota: KDTree de SciPy soporta query_ball_point, query, etc.
        # Para rectángulo, se usa un filtrado: se extraen candidatos por bbox del rectángulo
        # mediante consulta en un radio grande en el centro como aproximación simple.
        # En producción, conviene usar estructuras/consultas específicas.
        pts = points[:]
        tree = KDTree(pts)
        xmin, xmax, ymin, ymax = rect
        cx, cy = (xmin+xmax)/2.0, (ymin+ymax)/2.0
        # radio que cubre el rectángulo completo desde el centro (cota)
        r = ((xmax-xmin)**2 + (ymax-ymin)**2) ** 0.5 / 2.0 + 1e-12
        idxs = tree.query_ball_point((cx, cy), r)
        candidates = [pts[i] for i in idxs]
        return [p for p in candidates if in_rect(p, rect)]
    except Exception as e:
        print("SciPy no disponible o error al usar KDTree (opcional):", e)
        return None

if __name__ == "__main__":
    random.seed(7)
    pts = [(random.uniform(-10, 10), random.uniform(-10, 10)) for _ in range(200)]
    rect = (-2.5, 3.0, -1.0, 4.0)

    ans_naive = sorted(range_search_naive(pts, rect))
    root = build_kdtree(pts)
    out: List[Point] = []
    range_search_kdtree(root, rect, out)
    ans_kd = sorted(out)
```

Captura 83: Se define rect_intersects_bbox() para decidir si el rectángulo de consulta intersecta la bounding box de un nodo; si no intersecta, se poda el subárbol. Con range_search_kdtree() se recorre el árbol: se descarta por poda, se agrega el punto si cae en el rectángulo y se visita recursivamente a los hijos. Esto reduce el trabajo frente al filtrado ingenuo cuando hay muchas regiones descartables. Desarrollado en Online IDE Pro. Código fuente de la autora.

```
    # Pruebas simples: deben coincidir (conjunto)
    assert ans_naive == ans_kd
    print("OK: range_search naive == k-d tree. Cantidad:", len(ans_naive))

    ans_scipy = range_search_scipy_optional(pts, rect)
    if ans_scipy is not None:
        assert sorted(ans_scipy) == ans_naive
        print("OK: SciPy KDTree (opcional) coincide con naive.")

```

Captura 84: Se añade una variante opcional con scipy.spatial.KDTree dentro de try/except, para que el script no falle si SciPy no está disponible. Finalmente, el bloque principal genera puntos aleatorios, ejecuta la búsqueda ingenua y la del k-d tree, compara resultados con assert y, si SciPy está presente, valida también contra su KDTree. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`range_search_naive`	Filtra todos los puntos por pertenencia al rectángulo (O(n)).
`KDNode` + `bbox`	Cada nodo guarda su punto y una caja envolvente del subárbol para habilitar poda.
`build_kdtree`	Ordena por eje alternado, toma mediana, construye recursivamente y calcula bbox del subárbol.
`rect_intersects_bbox`	Determina si el rectángulo puede intersectar la bbox del nodo; si no, poda el subárbol completo.
`range_search_kdtree`	Visita solo nodos cuya bbox intersecta la consulta; agrega puntos dentro.
Asserts de igualdad	Verifica que el índice (k-d tree) no cambia el resultado respecto del enfoque ingenuo en instancias pequeñas.
`range_search_scipy_optional`	En `try/except`, usa SciPy si está disponible; si no, imprime mensaje y continúa sin fallar.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

6.9 Vecino más próximo (nearest neighbor) y Voronoi

El problema de vecino más próximo (NN) consiste en encontrar, para un punto de consulta, el punto del conjunto cuya distancia es mínima. En 2D, la solución más directa es revisar todos los puntos, lo que implica un costo O(n) por consulta. Para conjuntos grandes, estructuras como los k-d trees permiten podar regiones del espacio durante la búsqueda y reducir el trabajo promedio cuando los datos están razonablemente bien distribuidos (Bentley, 1975).

Una forma geométrica de entender este problema es a través del diagrama de Voronoi, que divide el plano en regiones o celdas. Cada celda contiene todos los puntos del plano que están más cerca de un sitio particular que de cualquier otro. De este modo, la celda de Voronoi de un sitio representa precisamente el conjunto de puntos para los cuales ese sitio es el vecino más próximo. Además, existe una relación clásica de dualidad entre el diagrama de Voronoi y la triangulación de Delaunay (Aurenhammer, 1991; Fortune, 1987).

Énfasis

Definiciones clave

- **Vecino más próximo (NN):** Elemento p del conjunto que minimiza la distancia a la consulta q.
- **Distancia euclídea cuadrada:** $d^2(p,q) = (p_x - q_x)^2 + (p_y - q_y)^2$, evita sqrt sin alterar el argmin.
- **Celda de Voronoi:** Región de puntos del plano cuya distancia a un sitio s es menor o igual que a cualquier otro sitio.
- **Frontera de Voronoi:** Conjunto de puntos equidistantes a dos (o más) sitios; separa celdas.
- **Dualidad con Delaunay:** Dos sitios son vecinos en Delaunay si sus celdas de Voronoi comparten frontera (mención conceptual).

6.10 Paso a paso – NN + Voronoi

6.10.1 Paso 1: Las regiones de Voronoi

El diagrama comienza con tres puntos o "sitios" (A, B y C). El espacio se divide en regiones, llamadas celdas de Voronoi. Cada celda es el área que contiene todos los puntos que están más cerca de un sitio específico que de cualquier otro.

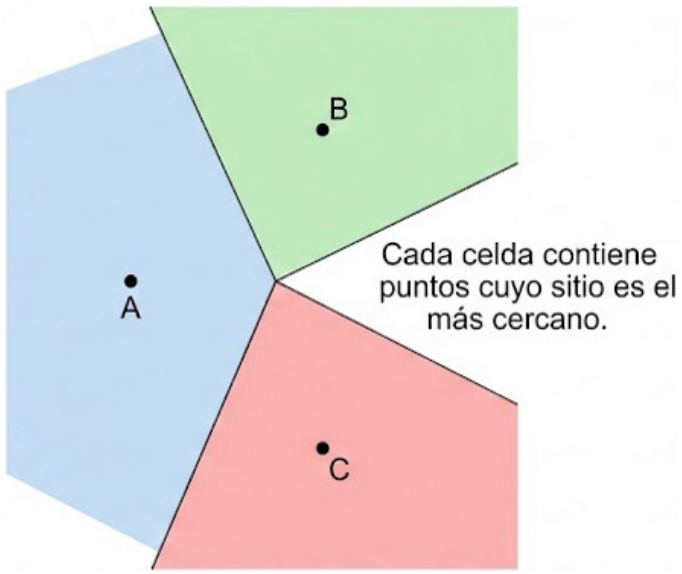


Ilustración 38: Diagrama de Voronoi: celdas de cercanía. La imagen muestra el plano particionado en celdas, donde cada celda contiene los puntos del espacio cuyo "sitio" más cercano es A, B o C. Se relaciona con el contenido al introducir Voronoi como estructura geométrica para problemas de vecindad y proximidad.

Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.10.2 Paso 2: Encontrar el vecino más próximo

Al introducir un nuevo punto de consulta (Q), podemos determinar fácilmente cuál es su sitio más cercano. Como Q se encuentra dentro de la celda de Voronoi de A, sabemos inmediatamente que A es el vecino más próximo a Q.

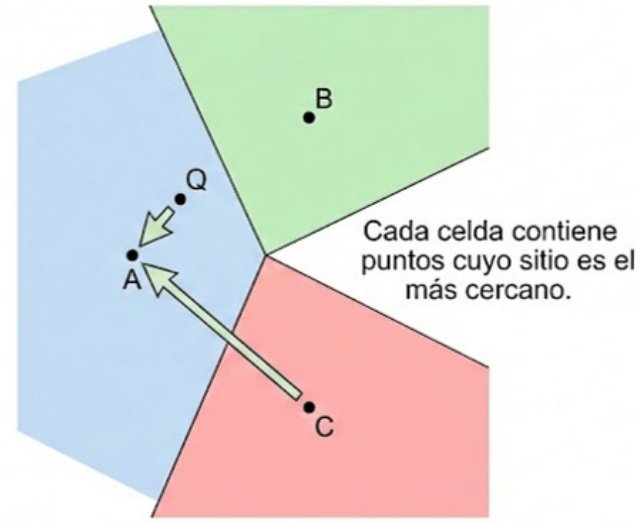


Ilustración 39: Consulta de vecino más próximo con Voronoi. Se agrega un punto de consulta Q y flechas hacia el sitio más cercano (por pertenencia de Q a una celda). Se relaciona con el contenido al explicar cómo Voronoi permite identificar el candidato más cercano y razonar sobre distancias por región. Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.10.3 Paso 3: La dualidad de delaunay

Finalmente, el diagrama muestra la relación "dual" con la triangulación de Delaunay. Al conectar los sitios originales (A, B y C) con líneas, se forma un triángulo. Estas líneas son perpendiculares a los bordes de las celdas de Voronoi que separan los sitios. Esto ilustra la conexión geométrica entre ambas estructuras.

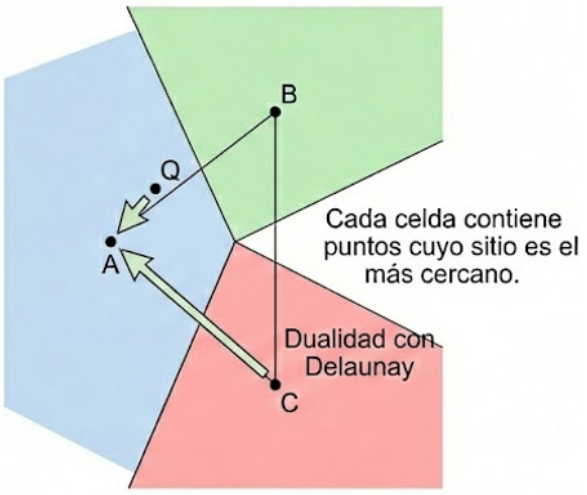


Ilustración 40: Dualidad Voronoi–Delaunay. Se incorpora el triángulo que conecta los sitios (aristas de Delaunay) y se indica la dualidad: fronteras de Voronoi ↔ conexiones de Delaunay. Se relaciona con el contenido al justificar algoritmos de proximidad y vecindad (p. ej., k-NN, mallas, optimizaciones geométricas) usando esta relación estructural. Imagen creada usando herramientas de IA con información proporcionada por la autora.

6.10.4 Ejemplo 6–Nearest neighbor: fuerza bruta + k-d tree mínimo + SciPy opcional

```python
# Ejemplo 6: Vecino más próximo (nearest neighbor) en 2D.
# - Principal: fuerza bruta O(n) usando distancia euclídea cuadrada.
# - Opcional (sin librerías): k-d tree mínimo con poda por distancia a bbox.
# - Opcional con SciPy: KDTree en try/except.
# Script completo y ejecutable (Python estándar).

from typing import List, Tuple, Optional
import random
import math

Point = Tuple[float, float]
EPS = 1e-9

def dist2(a: Point, b: Point) -> float:
    dx = a[0]-b[0]
    dy = a[1]-b[1]
    return dx*dx + dy*dy

def nn_bruteforce(points: List[Point], q: Point) -> Point:
    assert points, "Conjunto vacío no admite NN."
    best = points[0]
    best_d = dist2(best, q)
    for p in points[1:]:
        d = dist2(p, q)
        if d < best_d:
            best, best_d = p, d
    return best

class KDNode:
    __slots__ = ("p", "axis", "left", "right", "minx", "maxx", "miny", "maxy")
    def __init__(self, p: Point, axis: int,
                 left: Optional["KDNode"], right: Optional["KDNode"],
                 bbox):
        self.p = p
        self.axis = axis
        self.left = left
        self.right = right
        self.minx, self.maxx, self.miny, self.maxy = bbox
```

Captura 85: Inicio del Ejemplo 6: Vecino más próximo (nearest neighbor) en 2D. Se definen tipos y constantes, la distancia euclídea al cuadrado (dist2) y el método principal por fuerza bruta (nn_bruteforce), que recorre todos los puntos y selecciona el más cercano a la consulta q. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def bbox_point(p: Point):
    return (p[0], p[0], p[1], p[1])

def bbox_union(b1, b2):
    return (min(b1[0], b2[0]), max(b1[1], b2[1]), min(b1[2], b2[2]), max(b1[3], b2[3]))

def build_kdtree(points: List[Point], depth: int = 0) -> Optional[KDNode]:
    if not points:
        return None
    axis = depth % 2
    pts = sorted(points, key=lambda x: x[axis])
    mid = len(pts) // 2
    p = pts[mid]
    left = build_kdtree(pts[:mid], depth + 1)
    right = build_kdtree(pts[mid+1:], depth + 1)

    bb = bbox_point(p)
    if left:
        bb = bbox_union(bb, (left.minx, left.maxx, left.miny, left.maxy))
    if right:
        bb = bbox_union(bb, (right.minx, right.maxx, right.miny, right.maxy))
    return KDNode(p, axis, left, right, bb)

def dist2_point_bbox(q: Point, node: KDNode) -> float:
    # Distancia cuadrada mínima desde q a la caja del subárbol (0 si q está dentro).
    x, y = q
    dx = 0.0
    if x < node.minx:
        dx = node.minx - x
    elif x > node.maxx:
        dx = x - node.maxx
    dy = 0.0
    if y < node.miny:
        dy = node.miny - y
    elif y > node.maxy:
        dy = y - node.maxy
    return dx*dx + dy*dy
```

Captura 86: Continuación del Ejemplo 6. Se implementan funciones auxiliares para el k-d tree: cajas envolventes (bounding boxes, bbox_point, bbox_union), construcción recursiva del árbol (build_kdtree) y una cota de poda basada en la distancia mínima de la consulta a la bbox del subárbol (dist2_point_bbox). Estas piezas permiten acelerar la búsqueda descartando regiones imposibles de mejorar la mejor distancia actual. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def nn_kdtree(root: Optional[KDNode], q: Point) -> Point:
    assert root is not None, "Árbol vacío no admite NN."
    best = root.p
    best_d = dist2(best, q)

    def rec(node: Optional[KDNode]):
        nonlocal best, best_d
        if node is None:
            return

        # poda por bbox
        if dist2_point_bbox(q, node) > best_d + EPS:
            return

        # actualiza con punto del nodo
        d = dist2(node.p, q)
        if d < best_d:
            best, best_d = node.p, d

        # orden de visita: primero el lado donde cae q
        axis = node.axis
        qv = q[axis]
        nv = node.p[axis]
        first, second = (node.left, node.right) if qv < nv else (node.right, node.left)
        rec(first)
        rec(second)

    rec(root)
    return best
```

Captura 87: Continuación del Ejemplo 6. Se presenta la búsqueda de vecino más próximo usando k-d tree (nn_kdtree): recorrido recursivo, actualización del mejor candidato y poda por bbox cuando la cota inferior supera la mejor distancia encontrada, además del orden de visita priorizando el lado del árbol donde cae q. Esto conecta con el contenido de búsqueda espacial eficiente y reducción de complejidad en consultas. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def nn_scipy_optional(points: List[Point], q: Point) -> Optional[Point]:
    try:
        from scipy.spatial import KDTree  # opcional
        tree = KDTree(points)
        d, idx = tree.query(q)  # distancia y índice
        return points[int(idx)]
    except Exception as e:
        print("SciPy no disponible o error al usar KDTree (opcional):", e)
        return None

if __name__ == "__main__":
    random.seed(11)
    pts = [(random.uniform(-50, 50), random.uniform(-50, 50)) for _ in range(500)]
    queries = [(random.uniform(-50, 50), random.uniform(-50, 50)) for _ in range(20)]

    root = build_kdtree(pts)

    for q in queries:
        b = nn_bruteforce(pts, q)
        k = nn_kdtree(root, q)
        assert b == k, f"Desacuerdo NN: brute={b}, kd={k}, q={q}"

    print("OK: NN k-d tree coincide con fuerza bruta en consultas pequeñas.")

    s = nn_scipy_optional(pts, queries[0])
    if s is not None:
        assert s == nn_bruteforce(pts, queries[0])
        print("OK: SciPy KDTree (opcional) coincide con fuerza bruta.")
```

Captura 88: Continuación y cierre del Ejemplo 6. Se incluye una variante opcional con KDTree de SciPy en try/except (para no romper el script si no está disponible) y un bloque de pruebas: generación de puntos aleatorios, múltiples consultas y verificación por assert de que el resultado del k-d tree coincide con fuerza bruta, validando la corrección del enfoque. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`dist2`	Usa distancia cuadrada para evitar `sqrt` sin cambiar el mínimo.
`nn_bruteforce`	Recorre todos los puntos y mantiene el mejor candidato (O(n)).
k-d tree con bbox	Cada nodo guarda una bbox del subárbol para podar por distancia mínima posible.
`dist2_point_bbox`	Calcula cota inferior de distancia del query al subárbol; si supera al mejor, se poda.
`nn_kdtree`	Visita primero el subárbol donde cae `q`, luego el otro; ambos con poda por bbox.
Asserts de coincidencia	Verifica exactitud comparando con fuerza bruta en instancias pequeñas.
`nn_scipy_optional`	Variante opcional con SciPy en `try/except`, sin bloquear ejecución.

Código Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

6.10.5 Ejemplo 7–Voronoi: muestreo en grilla (sin librerías) + SciPy opcional

```python
# Ejemplo 7: Voronoi (concepto + cómputo opcional).
# Principal (sin librerías): aproximación por muestreo en grilla.
# - Asigna cada celda de grilla al sitio más cercano (tipo Voronoi).
# - Estima "fronteras" contando adyacencias con distinto sitio asignado.
# Opcional con SciPy: scipy.spatial.Voronoi (listar vértices/regiones) en try/except.
# Script completo y ejecutable (Python estándar).

from typing import List, Tuple, Dict
import math

Point = Tuple[float, float]

def dist2(a: Point, b: Point) -> float:
    dx = a[0]-b[0]
    dy = a[1]-b[1]
    return dx*dx + dy*dy

def assign_grid(sites: List[Point], xmin: float, xmax: float, ymin: float, ymax: float, step: float):
    assert sites, "Debe haber al menos un sitio."
    xs = []
    x = xmin
    while x <= xmax + 1e-12:
        xs.append(x)
        x += step
    ys = []
    y = ymin
    while y <= ymax + 1e-12:
        ys.append(y)
        y += step
```

Captura 89: Implementación de apoyo para un diagrama de Voronoi en 2D: se definen tipos y la función de distancia euclídea cuadrada (dist2) y se comienza a construir una grilla regular sobre el plano para asignar cada celda al "sitio" más cercano. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    # asignación: (i,j) -> índice de sitio más cercano
    grid = [[0 for _ in xs] for _ in ys]
    for iy, yv in enumerate(ys):
        for ix, xv in enumerate(xs):
            q = (xv, yv)
            best_i = 0
            best_d = dist2(sites[0], q)
            for i in range(1, len(sites)):
                d = dist2(sites[i], q)
                if d < best_d:
                    best_d = d
                    best_i = i
            grid[iy][ix] = best_i
    return xs, ys, grid

def boundary_count(grid) -> int:
    # cuenta fronteras aproximadas: vecinos 4-conectados con distinta asignación
    h = len(grid)
    w = len(grid[0]) if h else 0
    cnt = 0
    for y in range(h):
        for x in range(w):
            if x+1 < w and grid[y][x] != grid[y][x+1]:
                cnt += 1
            if y+1 < h and grid[y][x] != grid[y+1][x]:
                cnt += 1
    return cnt

def site_areas(grid, k: int) -> List[int]:
    areas = [0]*k
    for row in grid:
        for idx in row:
            areas[idx] += 1
    return areas
```

Captura 90: Asignación tipo Voronoi por muestreo en grilla: assign_grid recorre las coordenadas de la grilla y, para cada punto de celda, calcula qué sitio queda a menor distancia, guardando el índice del sitio más cercano. Además se incluye boundary_count, que estima fronteras contando adyacencias 4-conectadas con distinto sitio asignado, y site_areas, que aproxima el "área" de cada región por cantidad de celdas. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
def scipy_voronoi_optional(sites: List[Point]):
    try:
        from scipy.spatial import Voronoi  # opcional
        v = Voronoi(sites)
        print("SciPy Voronoi (opcional):")
        print("  Vértices (primeros 5):", v.vertices[:5])
        print("  Cantidad de regiones:", len(v.regions))
        print("  point_region (sitio->región):", v.point_region)
        # Nota: regiones pueden contener -1 (rayos al infinito)
        return v
    except Exception as e:
        print("SciPy no disponible o error al construir Voronoi (opcional):", e)
        return None

if __name__ == "__main__":
    # Sitios simples en un cuadrado
    sites = [(-2.0, -2.0), (2.0, -2.0), (0.0, 2.5)]

    xs, ys, grid = assign_grid(sites, xmin=-5.0, xmax=5.0, ymin=-5.0, ymax=5.0, step=0.5)

    # Pruebas simples: cada sitio debe capturar al menos una celda en esta grilla/bounds
    areas = site_areas(grid, k=len(sites))
    assert all(a > 0 for a in areas), f"Algún sitio sin celdas asignadas: {areas}"

```

Captura 91: Variante opcional con SciPy: scipy_voronoi_optional intenta construir el Voronoi con scipy.spatial.Voronoi dentro de try/except para no romper el script si la librería no está disponible; muestra información básica (vértices, regiones y asignación sitio→región) y contempla regiones al infinito. Desarrollado en Online IDE Pro. Código fuente de la autora.

```python
    bcnt = boundary_count(grid)
    assert bcnt > 0, "Debe existir alguna frontera aproximada si hay >=2 sitios."
    print("Asignación por grilla (áreas por sitio):", areas)
    print("Conteo aproximado de fronteras (adyacencias distintas):", bcnt)

    # Mostrar una salida textual compacta: filas superiores (identificadores)
    print("Muestra de grilla (primeras 8 filas):")
    for row in grid[:8]:
        print("".join(str(v) for v in row[:30]))

    _ = scipy_voronoi_optional(sites)
```

Captura 92: Bloque de prueba ejecutable (__main__): define un conjunto simple de sitios, genera la grilla, valida que cada sitio reciba al menos una celda, calcula áreas y un conteo aproximado de fronteras, y muestra una salida textual compacta de la grilla; finalmente intenta el cómputo opcional con SciPy. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`assign_grid`	Construye una grilla regular y asigna cada celda al sitio con menor distancia (argmin).
`grid[iy][ix]`	Guarda el índice del sitio ganador, aproximando celdas de Voronoi por muestreo.
`site_areas`	Cuenta cuántas celdas quedaron asignadas a cada sitio (aprox. área relativa).
`boundary_count`	Estima fronteras contando adyacencias con asignaciones distintas (4-vecindad).
Asserts	Verifican no vacíos por sitio y existencia de fronteras con múltiples sitios.
Salida textual	Muestra una porción de la grilla como identificadores de sitio para inspección rápida.
`scipy_voronoi_optional`	Si SciPy está disponible, construye Voronoi y lista vértices/regiones sin graficar; si no, continúa.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial. Actividad 5: Geometría Computacional Interactiva.

7. Algoritmos sobre textos y cadenas

7.1 Fundamentos de cadenas y modelo de búsqueda

En algoritmos sobre texto, una cadena se modela como una secuencia finita de símbolos pertenecientes a un alfabeto Σ. Un texto T (de longitud n) y un patrón P (de longitud m) definen el problema clásico de búsqueda de ocurrencias: identificar posiciones i tales que T[i:i+m] = P. En este marco, el costo suele medirse por el número de comparaciones de caracteres o, de forma equivalente, por accesos a memoria y pasos elementales, lo que permite establecer cotas asintóticas del algoritmo (Cormen et al., 2022).

Cuando el objetivo es la compresión, el texto se modela como una fuente discreta. La distribución empírica de los símbolos permite estimar la entropía, que funciona como un límite inferior teórico para la compresión promedio sin pérdida. Este resultado sirve como base conceptual para métodos estadísticos como Huffman (Shannon, 1948; Huffman, 1952). En la práctica, el rendimiento de los métodos de compresión depende del sesgo de la distribución de símbolos y de la presencia de regularidades en el texto, como repeticiones o patrones, que pueden ser explotadas por esquemas estadísticos o basados en diccionarios.

Énfasis

Definiciones clave

- **Alfabeto (Σ):** Conjunto finito de símbolos posibles con los que se forman las cadenas.
- **Cadena:** Secuencia finita de símbolos de Σ.
- **Texto (T):** Cadena sobre Σ en la cual se buscan ocurrencias.
- **Patrón (P):** Cadena objetivo cuya(s) ocurrencia(s) se desean localizar en T.
- **Ocurrencia (match):** Posición i tal que $T[i: i+m] = P$.
- **Entropía empírica:** Estimación basada en frecuencias observadas del contenido informativo promedio por símbolo (Shannon, 1948).

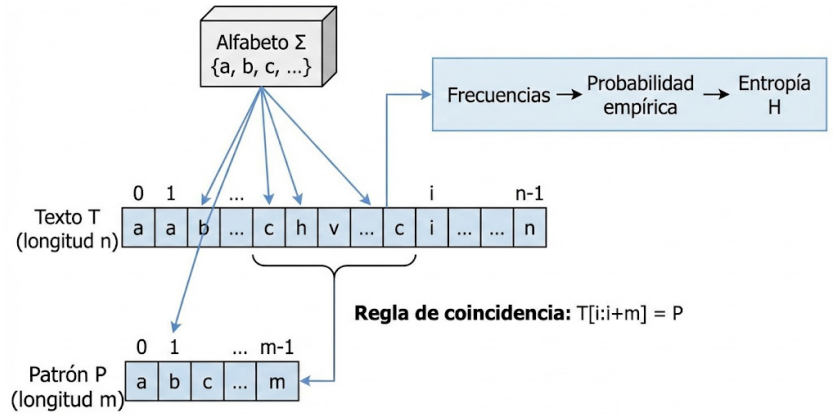


Ilustración 41: Búsqueda de patrones en texto (coincidencia T[i:i+m] = P). La imagen presenta el problema de búsqueda de patrones: dado un texto T (longitud n) y un patrón P (longitud m), se verifica en cada desplazamiento i si el bloque T[i:i+m] coincide con P. Esto conecta con los algoritmos de fuerza bruta y con KMP, que optimiza los desplazamientos evitando recomparaciones. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.1.1 Ejemplo 1

La entropía empírica permite conectar el análisis de frecuencias del texto con límites teóricos de compresión promedio. El siguiente ejemplo calcula frecuencias, probabilidad empírica y una estimación de entropía H (bits/símbolo) para ilustrar por qué textos con distribuciones sesgadas tienden a ser más compresibles (Shannon, 1948).

```python
import math
from collections import Counter

def empirical_entropy_bits_per_symbol(s: str) -> float:
    """
    Estima H = -sum p(x) log2 p(x) en bits/símbolo.
    Si s es vacía, define H = 0 por convención.
    """
    if not s:
        return 0.0
    counts = Counter(s)
    n = len(s)
    H = 0.0
    for c, k in counts.items():
        p = k / n
        H -= p * math.log2(p)
    return H

def symbol_frequencies(s: str):
    counts = Counter(s)
    n = len(s)
    return {ch: counts[ch] / n for ch in sorted(counts)}

if __name__ == "__main__":
    # Pruebas simples
    s1 = "aaaaabbbbcccdde"     # distribución sesgada
    s2 = "abcdef" * 10         # más uniforme

    H1 = empirical_entropy_bits_per_symbol(s1)
    H2 = empirical_entropy_bits_per_symbol(s2)

    print("Frecuencias s1:", symbol_frequencies(s1))
    print("Entropía s1 (bits/símbolo):", round(H1, 4))

    print("Frecuencias s2:", symbol_frequencies(s2))
    print("Entropía s2 (bits/símbolo):", round(H2, 4))

    assert H1 <= H2 + 1e-9  # en este diseño, s1 es más sesgada -> menor entropía esperada
    assert empirical_entropy_bits_per_symbol("") == 0.0
```

Captura 93: El código implementa el cálculo de la entropía empírica de Shannon en bits por símbolo a partir de un texto s: cuenta frecuencias con Counter, estima probabilidades p(x) = k/n y computa $H = -\Sigma\, p(x) \log_2 p(x)$. Además, incluye una función auxiliar que devuelve el diccionario de frecuencias relativas por carácter y un bloque de pruebas que compara dos cadenas (una distribución más sesgada vs. otra más uniforme) para verificar la relación esperada entre sesgo y entropía, junto con el caso borde de cadena vacía. Se relaciona con el contenido de análisis estadístico del texto previo a compresión (por ejemplo, Huffman) y con la noción de cantidad de información promedio del mensaje. Desarrollado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`Counter(s)`	Cuenta ocurrencias por símbolo para obtener la distribución empírica.
`p = k / n`	Calcula la probabilidad empírica de cada símbolo como frecuencia relativa.
`H -= p * log2(p)`	Implementa la fórmula de entropía de Shannon en bits por símbolo.
`symbol_frequencies`	Produce un diccionario ordenado para inspección de distribución.
Pruebas con `s1` y `s2`	Compara una cadena con repetición marcada vs una más uniforme.
`assert H1 <= H2`	Verifica una expectativa típica: mayor uniformidad suele aumentar entropía.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

7.2 Búsqueda de patrones: fuerza bruta vs Knuth–Morris–Pratt (KMP)

El algoritmo de fuerza bruta prueba todas las alineaciones posibles del patrón sobre el texto; en el peor caso puede realizar O(nm) comparaciones. KMP reduce comparaciones evitando retrocesos en el texto mediante un preprocesamiento del patrón: la función de prefijos (LPS, longest proper prefix which is also suffix) indica cuánto puede desplazarse el patrón cuando falla una comparación, manteniendo coincidencias parciales ya verificadas. De este modo, KMP logra complejidad O(n + m) en tiempo (Knuth et al., 1977; Cormen et al., 2022).

La correctitud de KMP se fundamenta en que, ante un fallo en P[j] vs T[i], el valor LPS permite reubicar j en el mayor prefijo propio que coincide con un sufijo del segmento ya coincidente. Esto preserva las coincidencias parciales ya validadas y garantiza que i avance de forma monotónica en el texto (Knuth et al., 1977).

Énfasis

Definiciones clave

- **Búsqueda por fuerza bruta:** Estrategia que prueba cada posición inicial posible y compara secuencialmente el patrón con el texto.
- **LPS (Longest Proper Prefix which is also Suffix):** Para cada prefijo $P[0..i]$, longitud del mayor prefijo propio que también es sufijo de ese prefijo.
- **Desplazamiento del patrón:** Reubicación del inicio de P respecto de T para continuar búsqueda sin perder coincidencias posibles.
- **Complejidad $O(n + m)$:** Cota temporal lineal en n y m derivada del avance sin retroceso del índice del texto en KMP.

7.3 Tabla LPS – Construcción

7.3.1 Paso 1: Configuración Inicial

Comenzamos con una tabla vacía. Las columnas son el índice i, el carácter del patrón P[i] en ese índice y el valor LPS[i] que vamos a calcular. LPS[i] es la longitud del prefijo propio más largo de P[0...i] que también es un sufijo de P[0...i].

KMP LPS table pattern 'ababaca'

Índice i	Carácter P[i]	LPS[i]
0	a	
1	b	
2	a	
3	b	
4	a	
5	c	
6	a	

Ilustración 42: Tabla LPS en KMP (estructura a completar). La imagen presenta la tabla LPS (Longest Proper Prefix which is also Suffix) para el patrón "ababaca". Esta tabla es el preprocesamiento central de Knuth–Morris–Pratt: resume, para cada posición i, cuánto "prefijo útil" puede reutilizarse ante un fallo de comparación, evitando recomparar caracteres ya verificados. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.3.2 Paso 2: Primeros caracteres y coincidencias

Calculamos los valores LPS para los primeros caracteres.

- Para i=0, LPS[0] siempre es 0.
- Para i=1 ('b'), no coincide con P[0] ('a'), así que LPS[1] es 0.
- Para i=2 ('a'), coincide con P[0] ('a'). El prefijo que también es sufijo es "a", de longitud 1. LPS[2] es 1.
- Para i=3 ('b'), coincide con P[1] ('b'). El prefijo/sufijo es "ab", de longitud 2. LPS[3] es 2.
- Para i=4 ('a'), coincide con P[2] ('a'). El prefijo/sufijo es "aba", de longitud 3. LPS[4] es 3. Las flechas azules muestran las coincidencias exitosas.

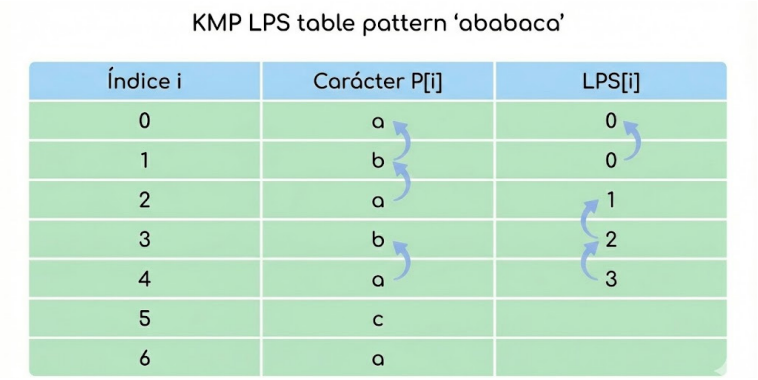


Ilustración 43: Construcción inicial de LPS (primeras coincidencias). La imagen ilustra cómo se van cargando los primeros valores de LPS cuando aparecen repeticiones de prefijos dentro del patrón (por ejemplo, al reconocer que "a", "ab", "aba" reaparecen como sufijos parciales). Esto se relaciona con la idea de KMP de aprender del patrón antes de recorrer el texto. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.3.3 Paso 3: El Retroceso (Backtracking)

Para i=5, el carácter es 'c'. Nuestro índice de prefijo actual es j=3 (del paso anterior).

- Comparamos P[5] ('c') con P[3] ('b'). No coinciden (flecha roja).
- Necesitamos retroceder. El nuevo valor de j es LPS[j-1], es decir, LPS[2], que es 1. Esto se indica con la gran flecha azul curva.
- Ahora comparamos P[5] ('c') con P[1] ('b'). No coinciden.
- Retrocedemos de nuevo: j = LPS[j-1] = LPS[0] = 0.
- Comparamos P[5] ('c') con P[0] ('a'). No coinciden.
- Como j=0 y no hay coincidencia, LPS[5] es 0.

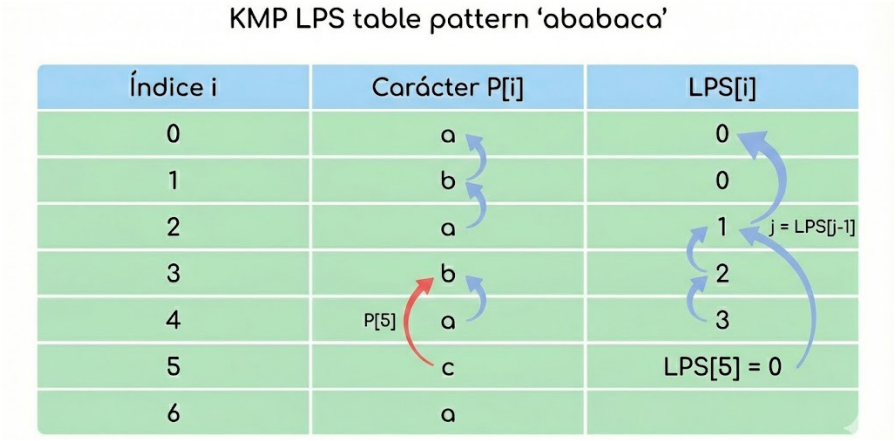


Ilustración 44: KMP: retroceso por LPS ante un desajuste (fallback). La imagen muestra el momento clave en el cálculo de LPS cuando ocurre un desajuste (mismatch): en lugar de volver a cero siempre, el algoritmo retrocede usando j = LPS[j – 1] para intentar el siguiente prefijo candidato. Así se garantiza el preprocesamiento en tiempo lineal y, luego, la búsqueda eficiente. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.3.4 Paso 4: Tabla completa

Finalmente, para i=6, el carácter es 'a'. El valor de j es 0 (del paso anterior).

- Comparamos P[6] ('a') con P[0] ('a'). Coinciden (flecha azul).
- Incrementamos j a 1. Por lo tanto, LPS[6] es 1.

La tabla LPS está completa. Esta tabla se utilizará en el algoritmo de búsqueda KMP para evitar comparaciones innecesarias cuando ocurra un fallo.

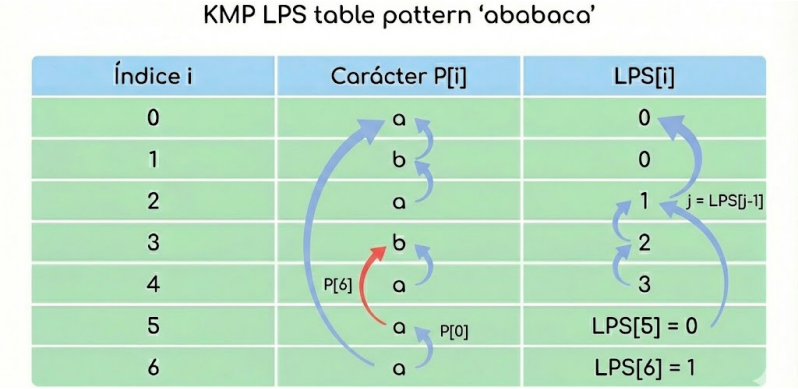


Ilustración 45: KMP: actualización final de LPS y reutilización de prefijos. La imagen completa el ejemplo del patrón "ababaca", mostrando cómo se determina el último valor (por ejemplo, LPS[6] = 1) y cómo los saltos guiados por LPS permiten continuar comparando sin perder el trabajo previo. Esto conecta directamente con la búsqueda KMP en el texto T: ante un fallo, se desplaza el patrón con el mayor prefijo reutilizable. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.3.5 Ejemplo 2

Se implementan fuerza bruta y KMP devolviendo todas las ocurrencias, incluyendo solapamientos, y cubriendo casos límite (patrón vacío, patrón más largo que texto). Se añade conteo de comparaciones para ilustrar la reducción de trabajo en entradas adversas (Knuth et al., 1977).

```python
from typing import List, Tuple
import random
import string

def brute_find_all(text: str, pattern: str) -> Tuple[List[int], int]:
    """
    Devuelve (indices, comparaciones_de_caracteres).
    Convención:
      - Si pattern == "", se considera que coincide en todas las posiciones: 0..len(text).
      - Se permiten solapamientos.
    """
    if pattern == "":
        return list(range(len(text) + 1)), 0

    n, m = len(text), len(pattern)
    if m > n:
        return [], 0

    indices = []
    comparisons = 0
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
                break
        if match:
            indices.append(i)
    return indices, comparisons
```

Captura 94: Implementación de búsqueda de patrones por fuerza bruta (brute_find_all). Recorre todas las posiciones posibles del texto y compara carácter a carácter con el patrón, contando comparaciones y devolviendo los índices de ocurrencia (permitiendo solapamientos y contemplando el caso de patrón vacío). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def build_lps(pattern: str) -> List[int]:
    """
    Construye el arreglo LPS (longest proper prefix which is also suffix).
    """
    m = len(pattern)
    lps = [0] * m
    length = 0  # longitud del prefijo-sufijo actual
    i = 1
    while i < m:
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]
            else:
                lps[i] = 0
                i += 1
    return lps
```

Captura 95: Construcción del arreglo LPS (Longest Proper Prefix which is also Suffix) de KMP (build_lps). Calcula, para cada posición del patrón, la longitud del prefijo propio más largo que también es sufijo, lo que permitirá "saltar" comparaciones al producirse un desacuerdo. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def kmp_find_all(text: str, pattern: str) -> Tuple[List[int], int]:
    """
    Devuelve (indices, comparaciones_de_caracteres).
    Convención:
      - Si pattern == "", coincide en todas las posiciones: 0..len(text).
      - Se permiten solapamientos.
    """
    if pattern == "":
        return list(range(len(text) + 1)), 0

    n, m = len(text), len(pattern)
    if m > n:
        return [], 0

    lps = build_lps(pattern)
    indices = []
    comparisons = 0

    i = 0  # índice en text
    j = 0  # índice en pattern
    while i < n:
        comparisons += 1
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == m:
                indices.append(i - m)
                # Para permitir solapamientos, se reubica j según LPS
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return indices, comparisons
```

Captura 96: Implementación de búsqueda de patrones con Knuth–Morris–Pratt (kmp_find_all). Utiliza el arreglo LPS para evitar retrocesos en el texto: ante un desacuerdo, reubica el índice del patrón según LPS y continúa, devolviendo todas las ocurrencias y el conteo de comparaciones (con solapamientos y caso de patrón vacío). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
if __name__ == "__main__":
    # (A) Fuerza bruta: casos obligatorios
    assert brute_find_all("aaaaa", "aa")[0] == [0, 1, 2, 3]  # solapamientos
    assert brute_find_all("abc", "")[0] == [0, 1, 2, 3]      # patrón vacío
    assert brute_find_all("abc", "abcd")[0] == []            # patrón más largo
    assert brute_find_all("ababa", "aba")[0] == [0, 2]       # múltiples ocurrencias con solapamiento parcial

    # (B) KMP: pruebas de LPS típicas
    assert build_lps("abababaca") == [0, 0, 1, 2, 3, 0, 1]
    assert build_lps("aaaa") == [0, 1, 2, 3]
    assert build_lps("abcab") == [0, 0, 0, 1, 2]

    # KMP debe coincidir con fuerza bruta
    tests = [
        ("aaaaa", "aa"),
        ("ababa", "aba"),
        ("abcabcabc", "abc"),
        ("mississippi", "issi"),
        ("abc", ""),
        ("abc", "abcd"),
    ]
    for t, p in tests:
        b = brute_find_all(t, p)[0]
        k = kmp_find_all(t, p)[0]
        assert b == k

    # Validación adicional con tests aleatorios
    rnd = random.Random(123)
    for _ in range(50):
        t = "".join(rnd.choice("ab") for _ in range(60))
        p = "".join(rnd.choice("ab") for _ in range(rnd.randint(0, 10)))
        assert brute_find_all(t, p)[0] == kmp_find_all(t, p)[0]
```

Captura 97: Bloque principal de pruebas (__main__): valida casos obligatorios de fuerza bruta, verifica valores esperados de LPS para patrones típicos y comprueba que KMP coincide con fuerza bruta en un conjunto de tests (incluyendo validación adicional con textos/patrones aleatorios). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
# Demostración simple de complejidad: conteo de comparaciones
text = "a" * 5000 + "b"
pattern = "a" * 2500 + "b"
b_idx, b_cmp = brute_find_all(text, pattern)
k_idx, k_cmp = kmp_find_all(text, pattern)

print("Ocurrencias encontradas (deben coincidir):", b_idx == k_idx)
print("Comparaciones fuerza bruta:", b_cmp)
print("Comparaciones KMP:", k_cmp)
```

Captura 98: Demostración comparativa de complejidad mediante conteo de comparaciones: ejecuta fuerza bruta y KMP sobre un caso diseñado para evidenciar la diferencia (texto largo con prefijo repetido) e imprime la cantidad de comparaciones realizadas por cada enfoque, conectando el código con el análisis de eficiencia del tema. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`brute_find_all`	Recorre todas las alineaciones posibles i y compara m caracteres hasta fallar o confirmar coincidencia.
Convención `pattern == ""`	Define coincidencia en todas las posiciones $0..n$ para un caso límite frecuente en APIs de búsqueda.
`build_lps`	Construye LPS en $O(m)$ actualizando `length` y retrocediendo con valores previos de LPS en fallos.
`kmp_find_all`	Recorre el texto sin retroceder `i`; ante fallo, reubica `j` con LPS para reutilizar coincidencias parciales.
`j = lps[j-1]` tras match	Permite continuar y detectar solapamientos sin reiniciar desde cero.
Asserts de casos obligatorios	Verifican patrón vacío, patrón más largo, múltiples ocurrencias y solapamientos.
Asserts de LPS	Validan la construcción de prefijos con patrones estándar de prueba.
Validación aleatoria	Confirma equivalencia funcional KMP vs fuerza bruta en múltiples instancias generadas.
Conteo de comparaciones	Muestra en un caso sintético que KMP reduce comparaciones respecto del enfoque $O(nm)$.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

7.4 Compresión sin pérdida

La compresión sin pérdida requiere que el proceso sea invertible: el decodificador debe reconstruir exactamente el original. Desde un punto de vista conceptual, se distinguen dos familias relevantes:

- Compresión estadística, que asigna códigos más cortos a símbolos más probables (Huffman).
- Compresión basada en diccionario, que reemplaza secuencias repetidas por referencias a un diccionario (LZW).

La idea de que existe un límite teórico para compresión promedio se fundamenta en la entropía de Shannon: ningún esquema sin pérdida puede, en promedio, codificar por debajo de la entropía por símbolo para una fuente fija (Shannon, 1948). En la práctica, los

algoritmos explotan regularidades del texto y aproximan ese límite bajo supuestos (por ejemplo, independencia de símbolos o repetición de substrings).

Énfasis

Definiciones clave

- **Compresión sin pérdida:** Transformación reversible que permite reconstrucción exacta del original.
- **Codificación:** Mapeo de símbolos o secuencias a una representación comprimida.
- **Decodificación:** Proceso inverso que reconstruye el texto original.
- **Código prefijo:** Conjunto de códigos donde ninguno es prefijo de otro, habilitando decodificación instantánea (propiedad usada por Huffman).
- **Compresión por diccionario:** Representación de repeticiones mediante referencias a un diccionario dinámico (familia LZ).

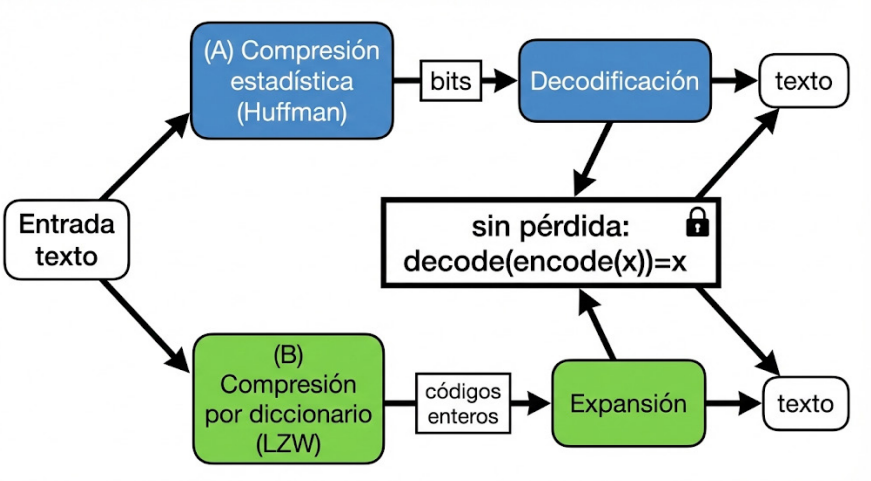


Ilustración 46: Compresión sin pérdida: Huffman vs. LZW (visión de procesos). La imagen compara dos familias de compresión sin pérdida: (A) Huffman, basada en probabilidades/frecuencias para producir códigos en bits, y (B) LZW, basada en diccionario para producir códigos enteros que luego se expanden. Se enfatiza la propiedad decode(encode(x)) = x, clave en compresión sin pérdida. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.4.1 Ejemplo 3

Antes de aplicar Huffman o LZW, resulta útil instrumentar métricas mínimas: longitud original y longitud de una representación alternativa (por ejemplo, codificación trivial). El siguiente ejemplo define una métrica simple y verifica invariantes básicos de reversibilidad en una "codificación identidad" para establecer el criterio de evaluación de compresión sin pérdida.

```python
def identity_encode(s: str) -> str:
    # Codificación identidad (no comprime): sirve como control de reversibilidad.
    return s

def identity_decode(x: str) -> str:
    return x

def bits_original_ascii8(s: str) -> int:
    # Supone 8 bits por carácter (aproximación didáctica).
    return 8 * len(s)

if __name__ == "__main__":
    s = "ABRACADABRA"
    enc = identity_encode(s)
    dec = identity_decode(enc)

    print("Original:", s)
    print("Codificado (identidad):", enc)
    print("Bits originales (ASCII-8):", bits_original_ascii8(s))
    print("Bits 'comprimidos' (identidad, ASCII-8):", bits_original_ascii8(enc))

    assert dec == s
    assert bits_original_ascii8(enc) == bits_original_ascii8(s)
```

Captura 99: Implementación ejemplo 3. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`identity_encode/identity_decode`	Define un par reversible mínimo para fijar el criterio de "sin pérdida": decodificar debe recuperar el original.
`bits_original_ascii8`	Estima tamaño original suponiendo 8 bits por carácter, útil como referencia comparativa didáctica.
`assert dec == s`	Verifica el invariante central: reversibilidad exacta.
Impresiones	Muestran una métrica base para luego comparar con Huffman/LZW.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

7.5 Huffman: códigos prefijo, codificación y decodificación

Huffman construye un código prefijo de longitud variable a partir de frecuencias: combina iterativamente los dos símbolos (o subárboles) de menor probabilidad, generando un árbol binario cuyos caminos definen códigos. El resultado minimiza la longitud media del código entre todos los códigos prefijo (optimalidad en ese dominio) y permite una decodificación unívoca al recorrer el árbol bit a bit (Huffman, 1952). En implementaciones prácticas, el cómputo de frecuencias y el uso de una cola de prioridad (heap) aportan eficiencia $O(|\Sigma| \log |\Sigma|)$ para construir el árbol, más el costo lineal necesario para codificar y decodificar el texto (Cormen et al., 2022).

Énfasis

Definiciones clave

- **Código prefijo:** Conjunto de códigos donde ningún código es prefijo de otro, habilitando decodificación inmediata.
- **Árbol de Huffman:** Árbol binario construido por combinación de pesos mínimos, cuyos caminos definen códigos.

- **Longitud media del código:** Promedio ponderado de longitudes de códigos por probabilidades de símbolos.
- **Cola de prioridad (heap):** Estructura que permite extraer eficientemente el elemento mínimo repetidas veces para construir el árbol.

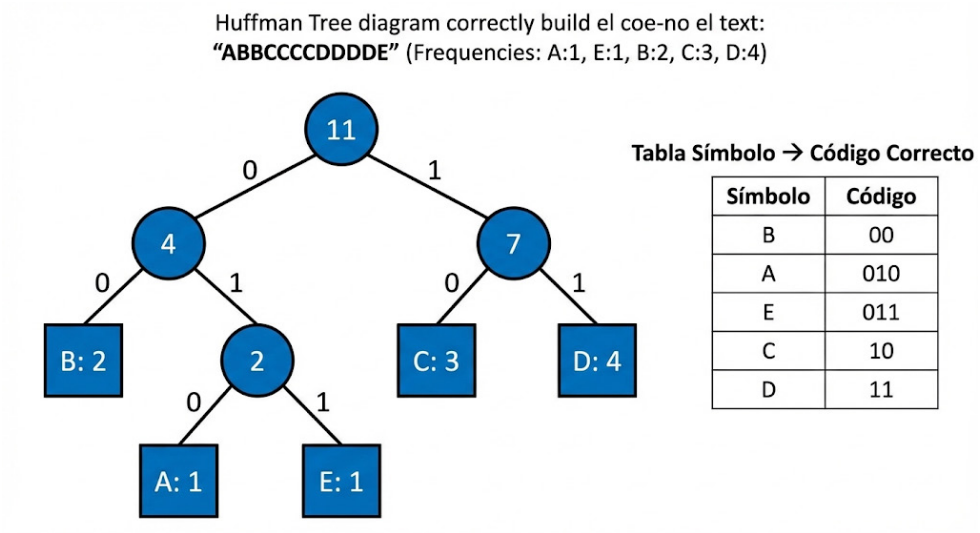


Ilustración 47: Árbol de Huffman y tabla de códigos (codificación prefijo). La imagen muestra un ejemplo completo de Huffman: a partir de frecuencias, se construye un árbol binario donde cada hoja (símbolo) recibe un código de bits según el camino (0/1). La tabla final "símbolo → código" ejemplifica cómo Huffman asigna códigos más cortos a símbolos más frecuentes, reduciendo el tamaño promedio. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.5.1 Ejemplo 4

El ejemplo implementa Huffman completo: conteo de frecuencias, construcción del árbol con heapq, generación de códigos prefijo, codificación a bits (cadena de '0'/'1') y decodificación. Se verifican invariantes: reversibilidad y propiedad prefijo, y se reporta una métrica de ratio aproximado (Huffman, 1952).

```python
from collections import Counter
import heapq
from typing import Dict, Optional, Tuple, List

class Node:
    def __init__(self, freq: int, ch: Optional[str] = None, left: Optional["Node"] = None, right: Optional["Node"] = None):
        self.freq = freq
        self.ch = ch
        self.left = left
        self.right = right

def build_huffman_tree(s: str) -> Optional[Node]:
    if not s:
        return None
    freq = Counter(s)

    heap: List[Tuple[int, int, Node]] = []
    order = 0
    for ch, f in freq.items():
        heapq.heappush(heap, (f, order, Node(f, ch=ch)))
        order += 1

    # Caso especial: un solo símbolo -> árbol degenerado
    if len(heap) == 1:
        return heap[0][2]

    while len(heap) > 1:
        f1, _, n1 = heapq.heappop(heap)
        f2, _, n2 = heapq.heappop(heap)
        parent = Node(f1 + f2, left=n1, right=n2)
        heapq.heappush(heap, (parent.freq, order, parent))
        order += 1

    return heap[0][2]

def build_codes(root: Optional[Node]) -> Dict[str, str]:
    if root is None:
        return {}
    codes: Dict[str, str] = {}
```

Captura 100: Implementación ejemplo 4. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
    # Si es hoja única
    if root.left is None and root.right is None and root.ch is not None:
        codes[root.ch] = "0"
        return codes

    def dfs(node: Node, path: str):
        if node.ch is not None and node.left is None and node.right is None:
            codes[node.ch] = path
            return
        if node.left is not None:
            dfs(node.left, path + "0")
        if node.right is not None:
            dfs(node.right, path + "1")

    dfs(root, "")
    return codes

def huffman_encode(s: str, codes: Dict[str, str]) -> str:
    return "".join(codes[ch] for ch in s)

def huffman_decode(bits: str, root: Optional[Node]) -> str:
    if root is None:
        return ""
    # Hoja única
    if root.left is None and root.right is None and root.ch is not None:
        return root.ch * len(bits)  # cada bit representa el símbolo
    out = []
    node = root
    for b in bits:
        node = node.left if b == "0" else node.right
        if node.left is None and node.right is None:
            out.append(node.ch)
            node = root
    return "".join(out)
```

Captura 101: Implementación ejemplo 4 (cont) Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```
76  def is_prefix_free(codes: Dict[str, str]) -> bool:
77      code_list = list(codes.values())
78      for i in range(len(code_list)):
79          for j in range(len(code_list)):
80              if i == j:
81                  continue
82              a, b = code_list[i], code_list[j]
83              if b.startswith(a):
84                  return False
85      return True
86
87  if __name__ == "__main__":
88      s = "this is an example for huffman encoding"
89      root = build_huffman_tree(s)
90      codes = build_codes(root)
91
92      bits = huffman_encode(s, codes)
93      recovered = huffman_decode(bits, root)
94
95      print("Códigos Huffman (muestra):", dict(list(codes.items())[:10]))
96      print("Bits originales (ASCII-8):", 8 * len(s))
97      print("Bits Huffman:", len(bits))
98      if len(s) > 0:
99          print("Ratio aproximado (Huffman/Original):", round(len(bits) / (8 * len(s)), 4))
100
```

Captura 102: Implementación ejemplo 4 (cont). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```
101     # Pruebas simples obligatorias
102     assert recovered == s
103     assert is_prefix_free(codes)
104
105     # Caso vacío y caso un símbolo
106     s2 = ""
107     r2 = build_huffman_tree(s2)
108     c2 = build_codes(r2)
109     assert huffman_encode(s2, c2) == ""
110     assert huffman_decode("", r2) == ""
111
112     s3 = "aaaaaa"
113     r3 = build_huffman_tree(s3)
114     c3 = build_codes(r3)
115     b3 = huffman_encode(s3, c3)
116     assert huffman_decode(b3, r3) == s3
117     assert is_prefix_free(c3)
```

Captura 1

Captura 103: Implementación ejemplo 4 (cont). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`Counter(s)`	Obtiene frecuencias de símbolos, insumo para ponderar el árbol.
`heapq` con tuplas `(freq, order, node)`	Implementa cola de prioridad estable para extraer mínimos repetidamente.
Combinación de dos mínimos	Construye el árbol de Huffman creando un nodo padre con suma de frecuencias.
`build_codes` (DFS)	Asigna '0' a izquierda y '1' a derecha; cada hoja recibe su código prefijo.
Caso de símbolo único	Fuerza un código '0' para mantener codificación/decodificación definidas.
`huffman_encode`	Reemplaza cada símbolo por su código; produce una cadena de bits.
`huffman_decode`	Recorre el árbol según bits y emite símbolo al llegar a hoja; reinicia en raíz.
`is_prefix_free`	Verificación básica: ningún código debe ser prefijo de otro.
Métricas impresas	Compara bits ASCII-8 vs bits Huffman y calcula ratio aproximado.
`assert recovered == s`	Confirma propiedad sin pérdida: decode(encode(s)) == s.

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

7.6 LZW: diccionario dinámico, compresión y expansión

LZW es un método de compresión sin pérdida basado en diccionario: inicialmente contiene símbolos básicos y se va ampliando con secuencias observadas durante el recorrido. El codificador emite códigos enteros que referencian entradas del diccionario, y el decodificador reconstruye el mismo diccionario en el mismo orden para recuperar el original. El esquema deriva de la familia LZ y la formulación de LZW se asocia a Welch (Welch, 1984), a partir de ideas previas de Ziv y Lempel sobre compresión por diccionario (Ziv & Lempel, 1977).

Un punto técnico central es el caso especial frecuentemente denominado KwKwK: durante la decodificación puede aparecer un código aún no presente en el diccionario; en ese caso se reconstruye como w + w[0], donde w es la última cadena decodificada.

Definiciones clave

- **Diccionario (LZW):** Tabla dinámica que mapea secuencias a códigos enteros durante compresión y expansión.
- **Código LZW:** Entero que referencia una entrada del diccionario.
- **KwKwK (caso especial):** Situación en decodificación donde el código recibido aún no existe; se resuelve como $w + w[0]$.
- **Secuencia máxima conocida:** Cadena más larga presente en el diccionario que coincide con el prefijo restante de entrada.

Esquema Paso a Paso de LZW con el Texto "TOBEORNOT"

Entrada Restante	w Actual	Nuevo Símbolo k	Salida (Código)	Nueva Entrada Diccionario (Símbolo: Código)
Inicialización:	TOBEORNOT			Diccionario Inicial: {T:0, O:1, B:2, E:3, R:4, N:5}
OBEORNOT	T	O	0 (T)	TO: 6
BEORNOT	O	B	1 (O)	OB: 7
EORNOT	B	E	2 (B)	BE: 8
ORNOT	E	O	3 (E)	EO: 9
RNOT	O	R	1 (O)	OR: 10
NOT	R	N	4 (R)	RN: 11
OT	N	O	5 (N)	NO: 12
T	O	T	1 (O)	OT: 13
Final:	T		0 (T)	
Salida Comprimida Total: 0 1 2 3 1 4 5 1 0				

Ilustración 48: LZW paso a paso (compresión por diccionario). La imagen detalla el algoritmo LZW sobre el texto "TOBEORNOT": se observa el string actual w, el nuevo símbolo k, la salida (código) y la actualización del diccionario con nuevas entradas. Esto se relaciona con la idea de LZW de aprender secuencias repetidas y reemplazarlas por referencias numéricas. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.6.1 Ejemplo 5

El ejemplo implementa LZW completo con diccionario inicial derivado del alfabeto observado (enfoque didáctico). Se devuelve una lista de códigos enteros, se implementa expansión con manejo explícito del caso KwKwK y se verifica reversibilidad con asserts (Welch, 1984).

```python
from typing import List, Dict

def lzw_compress(s: str) -> List[int]:
    if s == "":
        return []

    # Diccionario inicial: caracteres observados, orden estable
    alphabet = sorted(set(s))
    dict_enc: Dict[str, int] = {ch: i for i, ch in enumerate(alphabet)}
    next_code = len(dict_enc)

    w = ""
    out: List[int] = []
    for k in s:
        wk = w + k
        if wk in dict_enc:
            w = wk
        else:
            out.append(dict_enc[w])
            dict_enc[wk] = next_code
            next_code += 1
            w = k
    if w:
        out.append(dict_enc[w])
    return out

def lzw_decompress(codes: List[int], alphabet: List[str]) -> str:
    if not codes:
        return ""

    # Diccionario inicial inverso: code -> string
    dict_dec: Dict[int, str] = {i: ch for i, ch in enumerate(alphabet)}
    next_code = len(dict_dec)

    w = dict_dec[codes[0]]
    out = [w]
```

Captura 104: Implementación ejemplo 5. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```
    for code in codes[1:]:
        if code in dict_dec:
            entry = dict_dec[code]
        else:
            # Caso KwKwK: código aún no definido
            entry = w + w[0]

        out.append(entry)
        # Agrega nueva entrada al diccionario
        dict_dec[next_code] = w + entry[0]
        next_code += 1
        w = entry

    return "".join(out)

def alphabet_from_string(s: str) -> List[str]:
    return sorted(set(s))

if __name__ == "__main__":
    # Pruebas simples obligatorias
    s = "TOBEORNOTTOBEORTOBEORNOT"
    codes = lzw_compress(s)
    alpha = alphabet_from_string(s)
    recovered = lzw_decompress(codes, alpha)

    print("Texto original:", s)
    print("Alfabeto inicial:", alpha)
    print("Códigos LZW (muestra):", codes[:20], "...")
    print("Longitud original (chars):", len(s))
    print("Cantidad de códigos:", len(codes))

    assert recovered == s
```

Captura 105: Implementación ejemplo 5(cont). Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```
# Casos adicionales
for s2 in ["", "AAAAAA", "ABABABABAB", "BANANA_BANDANA"]:
    c2 = lzw_compress(s2)
    a2 = alphabet_from_string(s2) if s2 else []
    r2 = lzw_decompress(c2, a2)
    assert r2 == s2

# Comparación de longitudes aproximadas (didáctica)
s3 = "ABRACADABRA" * 5
c3 = lzw_compress(s3)
print("Comparación aproximada -> chars:", len(s3), "vs códigos:", len(c3))
assert lzw_decompress(c3, alphabet_from_string(s3)) == s3
```

Captura 106: Implementación ejemplo 5 (cont) Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`alphabet = sorted(set(s))`	Define un diccionario inicial didáctico a partir de símbolos presentes en el texto.
`dict_enc` y `next_code`	Mapea secuencias a códigos y administra el siguiente código disponible.
Bucle sobre caracteres `k`	Intenta extender la secuencia actual `w` con el siguiente símbolo.
Si `wk` existe	Continúa acumulando para hallar la secuencia máxima conocida.
Si `wk` no existe	Emite el código de `w`, agrega `wk` al diccionario y reinicia `w` con `k`.
`lzw_decompress`	Reconstruye el diccionario en el mismo orden, garantizando reversibilidad.
Caso `code not in dict_dec`	Implementa KwKwK como `w + w[0]`, condición necesaria de correctitud.
`dict_dec[next_code] = w + entry[0]`	Actualiza el diccionario de expansión con la nueva secuencia inferida.
`assert recovered == s`	Verifica propiedad sin pérdida: decompress(compress(s)) == s.
Métricas impresas	Compara longitud original en caracteres vs número de códigos emitidos (aproximación).

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

7.7 Criptografía basada en teoría de números (RSA)

RSA es un criptosistema de clave pública basado en aritmética modular y propiedades de factorización: se eligen primos p, q, se calcula n = pq y φ(n) = (p − 1)(q − 1). Se define un exponente público e coprimo con φ(n) y el exponente privado d como inverso modular de e módulo φ(n). El cifrado de un mensaje numérico m < n se define como c m (mod n) y el descifrado como m c (mod n) (Rivest et al., 1978).

En el plano conceptual, la seguridad práctica requiere tamaños grandes, padding seguro y generación criptográfica; por lo tanto, los ejemplos con números pequeños son únicamente didácticos y no deben interpretarse como seguridad real (Menezes et al., 1996). En esta sección se implementa un RSA mínimo para ilustrar:

- gcd/egcd,
- inverso modular,
- exponenciación modular eficiente con pow(base, exp, mod).

Énfasis

Definiciones clave

- **Aritmética modular:** Operaciones enteras definidas sobre clases de congruencia módulo n.
- $\varphi(n)$**(función totiente de Euler):** Cantidad de enteros $1 \leq k \leq n$ coprimos con n.

- **Clave pública:** Parámetros (n, e) usados para cifrar.
- **Clave privada:** Parámetro d (junto con n) usado para descifrar.
- **Inverso modular:** Entero d tal que $ed \equiv 1 (\mathrm{mod} \varphi(n))$.
- **Exponenciación modular:** Cálculo eficiente de $a^b \ mod \ n$, base de cifrado/descifrado RSA.

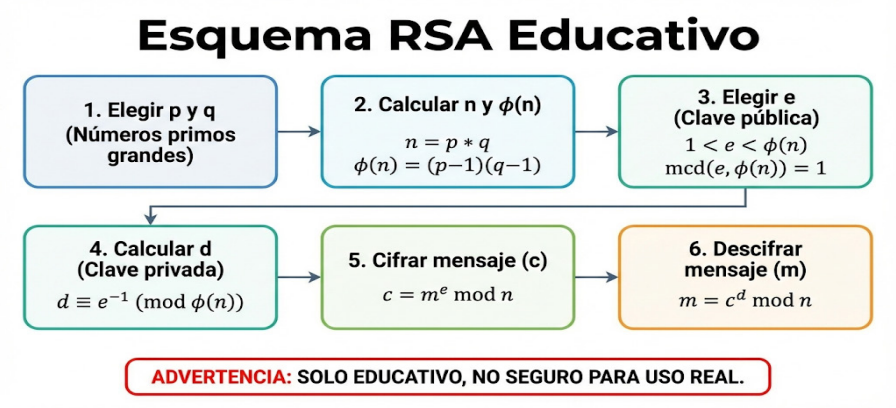


Ilustración 49: RSA (encriptación basada en teoría de números): etapas del esquema. La imagen resume el flujo de RSA: elección de primos p, q, cálculo de n y φ(n), selección del exponente público e, cálculo del exponente privado d, y operaciones de cifrado/descifrado con exponenciación modular. Se vincula con el contenido de encriptación de clave pública y con los fundamentos matemáticos que la hacen posible. Imagen creada usando herramientas IA con información proporcionada por la autora.

7.7.1 Ejemplo 6

Se implementa RSA educativo con primos pequeños para mostrar el ciclo completo: generación de claves, cifrado y descifrado de enteros y de texto corto por bytes. Se incluye advertencia explícita de uso educativo y pruebas con asserts para verificar la ida y vuelta (Rivest et al., 1978).

```python
from typing import Tuple, List

def gcd(a: int, b: int) -> int:
    while b != 0:
        a, b = b, a % b
    return abs(a)

def egcd(a: int, b: int) -> Tuple[int, int, int]:
    """
    Retorna (g, x, y) tal que ax + by = g = gcd(a,b).
    """
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = egcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return (g, x, y)

def modinv(a: int, m: int) -> int:
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("No existe inverso modular: gcd(a,m) != 1")
    return x % m

def is_prime_small(n: int) -> bool:
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True
```

Captura 107: Implementación ejemplo 6. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def rsa_keygen(p: int, q: int, e: int) -> Tuple[int, int, int]:
    """
    Retorna (n, e, d). Parámetros pequeños para uso educativo.
    """
    if not (is_prime_small(p) and is_prime_small(q)):
        raise ValueError("p y q deben ser primos (prueba simple para primos pequeños).")
    if p == q:
        raise ValueError("p y q deben ser distintos.")
    n = p * q
    phi = (p - 1) * (q - 1)
    if gcd(e, phi) != 1:
        raise ValueError("e debe ser coprimo con phi(n).")
    d = modinv(e, phi)
    return (n, e, d)

def rsa_encrypt_int(m: int, n: int, e: int) -> int:
    if not (0 <= m < n):
        raise ValueError("Mensaje m debe cumplir 0 <= m < n.")
    return pow(m, e, n)

def rsa_decrypt_int(c: int, n: int, d: int) -> int:
    if not (0 <= c < n):
        raise ValueError("Cifrado c debe cumplir 0 <= c < n.")
    return pow(c, d, n)

def text_to_bytes(s: str) -> List[int]:
    return list(s.encode("utf-8"))

def bytes_to_text(b: List[int]) -> str:
    return bytes(b).decode("utf-8")
```

Captura 108: Implementación ejemplo 6. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def rsa_encrypt_bytes(data: List[int], n: int, e: int) -> List[int]:
    # Cifra cada byte como un entero independiente (didáctico).
    if n <= 255:
        raise ValueError("n debe ser > 255 para cifrar bytes individuales con m < n.")
    return [rsa_encrypt_int(x, n, e) for x in data]

def rsa_decrypt_bytes(data: List[int], n: int, d: int) -> List[int]:
    out = [rsa_decrypt_int(x, n, d) for x in data]
    # Los bytes originales deben estar en 0..255
    if any(not (0 <= x <= 255) for x in out):
        raise ValueError("Decodificación a bytes inválida (fuera de rango).")
    return out

if __name__ == "__main__":
    print("ADVERTENCIA: Este ejemplo es únicamente educativo; no provee seguridad criptográfica real.")

    # Parámetros clásicos didácticos (pequeños)
    p, q, e = 61, 53, 17
    n, e_pub, d_priv = rsa_keygen(p, q, e)
    print("Clave pública (n,e):", (n, e_pub))
    print("Clave privada (n,d):", (n, d_priv))

    # Prueba 1: mensaje numérico
    m = 123
    c = rsa_encrypt_int(m, n, e_pub)
    m2 = rsa_decrypt_int(c, n, d_priv)
    print("m -> c -> m':", m, "->", c, "->", m2)
    assert m2 == m

    # Prueba 2: texto corto por bytes (didáctico)
    msg = "Hola"
    b = text_to_bytes(msg)
    enc = rsa_encrypt_bytes(b, n, e_pub)
    dec_b = rsa_decrypt_bytes(enc, n, d_priv)
    msg2 = bytes_to_text(dec_b)
```

Captura 109: Implementación ejemplo 6. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
    print("Texto original:", msg)
    print("Bytes:", b)
    print("Cifrado (enteros):", enc)
    print("Texto recuperado:", msg2)

    assert msg2 == msg
```

Captura 110: Implementación ejemplo 6. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`gcd`, `egcd`	Calculan mcd y coeficientes para obtener el inverso modular mediante Euclides extendido.
`modinv`	Deriva d tal que $ed \equiv 1 \pmod{\varphi(n)}$; falla si no existe inverso.
`is_prime_small`	Primalidad por división hasta $\sqrt{n}$, válida solo para primos pequeños (uso didáctico).
`rsa_keygen(p,q,e)`	Calcula n, $\varphi(n)$ y d; valida coprimalidad de e con $\varphi(n)$.
`pow(m, e, n)`	Realiza exponenciación modular eficiente (base de RSA).
Cifrado/descifrado entero	Verifica que $m < n$ y que $m = (m^e)^d \bmod n$.
Cifrado de texto por bytes	Convierte a bytes UTF-8 y cifra cada byte como entero independiente (didáctico).
Advertencia impresa	Señala explícitamente que no hay seguridad real por tamaños/padding y simplificaciones.
`assert`	Comprueba ida y vuelta tanto para entero como para texto.

Código | Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

7.7.2 Ejemplo integrador

Se construye un pipeline didáctico:

- localizar un patrón con KMP,
- comprimir el texto con Huffman, y
- cifrar de forma mínima el resultado comprimido mediante RSA en bloques pequeños (conceptual/didáctico). El objetivo es integrar búsqueda, compresión y cifrado preservando verificabilidad con asserts.

```python
from collections import Counter
import heapq
from typing import Dict, Optional, List, Tuple

# --- KMP (reutilización minimal) ---
def build_lps(pattern: str) -> List[int]:
    m = len(pattern)
    lps = [0] * m
    length = 0
    i = 1
    while i < m:
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]
            else:
                lps[i] = 0
                i += 1
    return lps
```

Captura 111: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python

def kmp_find_all(text: str, pattern: str) -> List[int]:
    if pattern == "":
        return list(range(len(text) + 1))
    n, m = len(text), len(pattern)
    if m > n:
        return []
    lps = build_lps(pattern)
    out = []
    i = j = 0
    while i < n:
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == m:
                out.append(i - m)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return out
```

Captura 112: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
# --- Huffman (reutilización minimal) ---
class Node:
    def __init__(self, freq: int, ch: Optional[str] = None, left=None, right=None):
        self.freq = freq
        self.ch = ch
        self.left = left
        self.right = right

def build_huffman_tree(s: str) -> Optional[Node]:
    if not s:
        return None
    freq = Counter(s)
    heap = []
    order = 0
    for ch, f in freq.items():
        heapq.heappush(heap, (f, order, Node(f, ch=ch)))
        order += 1
    if len(heap) == 1:
        return heap[0][2]
    while len(heap) > 1:
        f1, _, n1 = heapq.heappop(heap)
        f2, _, n2 = heapq.heappop(heap)
        parent = Node(f1 + f2, left=n1, right=n2)
        heapq.heappush(heap, (parent.freq, order, parent))
        order += 1
    return heap[0][2]
```

Captura 113: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def build_codes(root: Optional[Node]) -> Dict[str, str]:
    if root is None:
        return {}
    if root.left is None and root.right is None and root.ch is not None:
        return {root.ch: "0"}
    codes = {}
    def dfs(node: Node, path: str):
        if node.left is None and node.right is None:
            codes[node.ch] = path
            return
        if node.left is not None:
            dfs(node.left, path + "0")
        if node.right is not None:
            dfs(node.right, path + "1")
    dfs(root, "")
    return codes

def huffman_encode(s: str, codes: Dict[str, str]) -> str:
    return "".join(codes[ch] for ch in s)

def huffman_decode(bits: str, root: Optional[Node]) -> str:
    if root is None:
        return ""
    if root.left is None and root.right is None and root.ch is not None:
        return root.ch * len(bits)
    out = []
    node = root
    for b in bits:
        node = node.left if b == "0" else node.right
        if node.left is None and node.right is None:
            out.append(node.ch)
            node = root
    return "".join(out)
```

Captura 114: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
# --- RSA mínimo educativo (reutilización minimal) ---
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return abs(a)

def egcd(a: int, b: int) -> Tuple[int, int, int]:
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)

def modinv(a: int, m: int) -> int:
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("No inverso modular.")
    return x % m

def rsa_keygen_small(p: int, q: int, e: int) -> Tuple[int, int, int]:
    n = p * q
    phi = (p - 1) * (q - 1)
    if gcd(e, phi) != 1:
        raise ValueError("e no coprimo con phi.")
    d = modinv(e, phi)
    return n, e, d

def rsa_encrypt_int(m: int, n: int, e: int) -> int:
    return pow(m, e, n)

def rsa_decrypt_int(c: int, n: int, d: int) -> int:
    return pow(c, d, n)
```

Captura 115: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
def chunk_bits_to_ints(bits: str, chunk_size: int) -> List[int]:
    # Rellena al final para múltiplo de chunk_size
    pad = (-len(bits)) % chunk_size
    bits_padded = bits + ("0" * pad)
    ints = []
    for i in range(0, len(bits_padded), chunk_size):
        chunk = bits_padded[i:i+chunk_size]
        ints.append(int(chunk, 2))
    return ints, pad

def ints_to_chunk_bits(ints: List[int], chunk_size: int, pad: int) -> str:
    bits = "".join(format(x, "0{}b".format(chunk_size)) for x in ints)
    if pad:
        bits = bits[:-pad]
    return bits
```

Captura 116: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

```python
if __name__ == "__main__":
    print("ADVERTENCIA: Ejemplo integrador educativo; RSA sin padding y con claves pequeñas no es seguro.")

    text = "abracadabra abracadabra"
    pattern = "abra"

    # 1) Búsqueda con KMP
    idx = kmp_find_all(text, pattern)
    print("Ocurrencias de patrón:", idx)
    assert idx == [0, 7, 12, 19]  # verificación para este texto

    # 2) Compresión Huffman
    root = build_huffman_tree(text)
    codes = build_codes(root)
    bits = huffman_encode(text, codes)
    recovered_text = huffman_decode(bits, root)
    assert recovered_text == text

    # 3) "Cifrado" didáctico de bits Huffman en bloques pequeños con RSA
    # Parámetros didácticos
    n, e_pub, d_priv = rsa_keygen_small(61, 53, 17)  # n=3233 > 2^10=1024
    chunk_size = 10  # cada bloque < 1024 < n
    blocks, pad = chunk_bits_to_ints(bits, chunk_size)

    cipher_blocks = [rsa_encrypt_int(b, n, e_pub) for b in blocks]
    plain_blocks = [rsa_decrypt_int(c, n, d_priv) for c in cipher_blocks]
    bits2 = ints_to_chunk_bits(plain_blocks, chunk_size, pad)

    # Verificaciones de ida y vuelta
    assert bits2 == bits
    assert huffman_decode(bits2, root) == text

    print("Bits originales:", len(bits))
    print("Bloques (enteros) cifrados:", len(cipher_blocks))
    print("Integración verificada: búsqueda + compresión + cifrado didáctico.")
```

Captura 117: Implementación ejemplo integrador. Desarrollado y ejecutado en Online IDE Pro. Código fuente de la autora.

Elemento	Explicación del paso a paso del código
`kmp_find_all`	Localiza índices de ocurrencias del patrón en el texto sin retroceso del índice del texto.
Huffman (`build_huffman_tree`, `build_codes`)	Construye árbol y códigos prefijo para codificar el texto como bits.
`assert recovered_text == text`	Verifica compresión sin pérdida antes de cifrar.
`chunk_size = 10`	Define bloques de 10 bits para asegurar que cada entero sea menor que n.
`chunk_bits_to_ints`	Convierte la cadena de bits en una lista de enteros y registra padding.
RSA pequeño (`rsa_keygen_small`)	Genera n, e, d educativos; se usa `pow` modular para cifrar/descifrar.
`ints_to_chunk_bits`	Reconstruye exactamente la cadena de bits eliminando el padding.
Asserts finales	Garantizan integridad: bits y texto recuperados coinciden con el original.
Impresiones	Reportan tamaño en bits y número de bloques cifrados para inspección.

Código

Para acceder al código del ejemplo y poder ejecutarlo acceda a https://www.onlineide.pro/share/f211dc12-bb2a-419c-8a14-f97324b3a1fc

Actividad

A esta altura de lo leído y de los ejemplos analizados, ya se encuentra en condiciones de realizar la siguiente actividad, cuyo objetivo es aplicar de manera práctica los conceptos trabajados hasta el momento y reflexionar sobre su uso en el contexto de la Inteligencia Artificial.Actividad 6–Platform Actividades Interactivas Módulo 6

Evaluación

Luego de haber estudiado la materia completa y de haber resuelto las actividades, Ud. está en condiciones de resolver la primera parte de la evaluación integradora.

Palabras finales

A modo de síntesis, el Módulo 6 reunió herramientas para abordar problemas algorítmicos de mayor escala, integrando diseño, análisis y criterios de aplicación. En primer lugar, se revisaron estrategias como programación dinámica y algoritmos voraces, poniendo el foco en cuándo es correcto aplicarlas, qué costos implican y cuáles son sus límites. Sobre esa base, se introdujo la programación lineal y entera como forma de modelar problemas de optimización, destacando el paso desde una situación real hacia un modelo matemático con restricciones, función objetivo y condiciones de factibilidad, así como la interpretación formal de las soluciones obtenidas.

Luego, el módulo conectó estos enfoques con la complejidad computacional. Se presentaron las clases P y NP, el papel de las reducciones y el significado de la NP-completitud, como marco para entender por qué ciertos problemas no admiten soluciones eficientes conocidas y, en consecuencia, requieren aproximaciones o heurísticas cuando el tiempo de cómputo es limitado.

En el plano de las aplicaciones, se trabajó la FFT como herramienta para transformar ciertos cálculos en procedimientos más eficientes, especialmente en tareas como convolución o análisis espectral. También se abordaron problemas de geometría computacional, útiles para representar espacio, proximidad e intersecciones mediante predicados geométricos y estructuras de datos, frecuentes en contextos de mapas, percepción y análisis espacial. Finalmente, se incorporaron algoritmos sobre textos y cadenas, incluyendo búsqueda eficiente de patrones como KMP, métodos de compresión sin pérdida como Huffman y LZW, y una introducción conceptual a RSA para ilustrar el uso de aritmética modular en criptografía.

De este modo se completa el recorrido formativo de la materia, orientado a consolidar criterios para representar problemas, elegir estructuras de datos adecuadas y diseñar algoritmos con análisis explícito de costo, correctitud y límites. La implementación de soluciones ejecutables y su verificación mediante pruebas simples —por ejemplo, con asserts— refuerza además una práctica sistemática de validación, especialmente relevante cuando se trabaja con predicados, tolerancias numéricas o procesos iterativos.

En conjunto, el Módulo 6 articula cuatro ejes principales: la optimización mediante descomposición, como en programación dinámica; el modelado por restricciones, característico de la programación lineal; los fundamentos de complejidad que explican el uso de aproximaciones; y aplicaciones donde la estructura geométrica o la proximidad desempeñan un papel central.

Como cierre general, queda planteada una pauta transversal para el análisis algorítmico: ante cada problema es necesario identificar su estructura —datos, restricciones, geometría o combinatoria—, seleccionar el enfoque adecuado —exacto, aproximado o heurístico— y respaldar la solución con evidencia, ya sea mediante análisis de complejidad, argumentos de correctitud o validaciones empíricas cuando corresponda. De esta mane-

ra se consolida una base sólida para abordar problemas característicos del campo de la inteligencia artificial y de la computación aplicada.

Los invitamos a ver el video de cierre de la materia

Bibliografía consultada por el docente

- Aurenhammer, F. (1991). Voronoi diagrams—A survey of a fundamental geometric data structure. *ACM Computing Surveys, 23*(3), 345–405.
- Bellman, R. (1958). O n a routing problem. *Quarterly of Applied Mathematics, 16*, 87–90. https://tinyurl.com/bdhbu2mw
- Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. *Communications of the ACM, 18*(9), 509–517.
- Bertsimas, D. (s. f.). Introduction to linear optimization : Bertsimas, Dimitris : Free Download, Borrow, and Streaming : Internet Archive. Recuperado 26 de enero de 2026, de https://archive.org/details/introductiontoli0000bert
- Boyd, S., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press. https://www.seas.ucla.edu/~vandenbe/cvxbook/bv_cvxbook.pdf
- Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation, 19*(90), 297–301.
- Cook, S. A. (1971). The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). The MIT Press. https://tinyurl.com/9bdxdvbw
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Chvátal, V. (1983). *Linear programming*. W. H. Freeman. *Free Download, Borrow, and Streaming :* Internet Archive. Recuperado 26 de enero de 2026, de https://archive.org/details/linearprogrammin00chv

- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research, 6*(6), 791–812.
- Dantzig, G. B. (1963). *Linear programming and extensions*. Princeton University Press. *Free Download, Borrow, and Streaming* : Internet Archive. Recuperado 26 de enero de 2026, de https://archive.org/details/linearprogrammin00dant
- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3rd ed.). Springer.
- De la Cueva Hernández, V. M., López García, J. A., & Martínez Torres, M. A. (2020). *Estructuras de datos y algoritmos fundamentales*. Editorial Digital del Tecnológico de Monterrey.
- Diestel, R. (2017). *Graph Theory* (5th ed.). Springer. http://diestel-graph-theory.com/professional.html
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271. https://link.springer.com/article/10.1007/BF01386390
- Ewing, G. (2009). *PEP 380 — Syntax for Delegating to a Subgenerator*. Python Enhancement Proposals. https://peps.python.org/pep-0380/
- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM, 5*(6), 345. https://dl.acm.org/doi/10.1145/367766.368168
- Ford, L. R., & Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics, 8*, 399–404.
- Fortune, S. (1987). A sweepline algorithm for Voronoi diagrams. *Algorithmica, 2*, 153–174.
- Frittelli, V. (2021). *Algoritmos y estructura de datos* (2ª ed.). Independently Published
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. Wiley.
- Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.
- Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE, 40*(9), 1098–1101.
- Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558–562.

- Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller & J. W. Thatcher (Eds.), *Complexity of computer computations* (pp. 85–103). Plenum Press.
- Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack Problems*. Springer.
- Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Addison-Wesley.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley.
- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, *6*(2), 323–350.
- Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (Eds.). (1985). *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley.
- linprog — SciPy v1.17.0 Manual. (s. f.). Scipy.org. Recuperado 26 de enero de 2026, de https://tinyurl.com/5yn3nyk8
- Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). *Handbook of Applied Cryptography*. CRC Press.
- Mit.edu. Recuperado 20 de enero de 2026, de https://tinyurl.com/mt28rfx4
- Mcgill.ca. Recuperado 20 de enero de 2026, de https://tinyurl.com/bd3vmbmc
- Nemhauser, G. L., & Wolsey, L. A. (1988). *Integer and combinatorial optimization*. John Wiley & Sons. https://onlinelibrary.wiley.com/doi/book/10.1002/9781118627372
- Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-Time Signal Processing* (3rd ed.). Pearson. https://tinyurl.com/33xbac2c
- O'Rourke, J. (1998). *Computational Geometry in C* (2nd ed.). Cambridge University Press.
- Papadimitriou, C. H. (1994). Computational complexity. Addison-Wesley.
- Proakis, J. G., & Manolakis, D. G. (2007). *Digital Signal Processing: Principles, Algorithms, and Applications* (4th ed.). Pearson/Prentice Hall.
- Python Software Foundation. (s. f.). *bisect — Array bisection algorithm*. https://docs.python.org/3/library/bisect.html
- Python Software Foundation. (2024). *heapq — Heap queue algorithm*. https://docs.python.org/3/library/heapq.html
- Python Software Foundation. (2025). *sys.setrecursionlimit* (Documentación de Python 3.10, versión en español). https://docs.python.org/es/3.10/library/sys.html
- Python Software Foundation. (n.d.). *collections.deque* (Documentación oficial de

Python). https://docs.python.org/3/library/collections.html

- Preparata, F. P., & Shamos, M. I. (1985). *Computational Geometry: An Introduction*. Springer.
- *Red-Black Trees*. (s. f.). Opendatastructures.org. Recuperado 20 de enero de 2026, de https://opendatastructures.org/newhtml/ods/latex/redblack.html
- Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM, 21*(2), 120–126.
- Schemenauer, N., Peters, T., & Hetland, M. L. (2001). *PEP 255 — Simple Generators*. Python Enhancement Proposals. https://peps.python.org/pep-0255/
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. https://algs4.cs.princeton.edu/home/
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal, 27*(3), 379–423.
- Shewchuk, J. R. (1997). Adaptive precision floating-point arithmetic and fast robust predicates for computational geometry. *Discrete & Computational Geometry, 18*(3), 305–363.
- Silberschatz, A., Korth, H., & Sudarshan, S. (2020). *Database System Concepts*. McGraw-Hill.
- Sipser, M. (2012). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.
- Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. Journal of the ACM, 22(2), 215–225.
- Tarjan, R. E., & van Leeuwen, J. (1984). Worst-case analysis of set union algorithms. Journal of the ACM, 31(2), 245–281.
- *The AVL Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://tinyurl.com/4ztzumjs
- *The Red-Black Tree — OpenDSA Data Structures and Algorithms Modules Collection*. (s. f.). Cs.vt.edu. Recuperado 20 de enero de 2026, de https://tinyurl.com/2cenzbhf
- Ullman, J. (2016). Data Structures and Algorithms. Stanford Notes.
- Universidad Blas Pascal. (s.f.). Algoritmos y Estructuras de Datos II (material de cátedra Licenciatura en Sistemas de Información).
- Vazirani, V. V. (2001). Approximation algorithms. Springer.

- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods, 17(3), 261-272. https://doi.org/10.1038/s41592-019-0686-2

- Warshall, S. (1962). A theorem on boolean matrices. Journal of the ACM, 9(1), 11–12.

- Welch, T. A. (1984). A technique for high-performance data compression. Computer, 17(6), 8–19.

- West, D. B. (2001). Introduction to Graph Theory (2nd ed.). Prentice Hall.

- Williamson, D. P., & Shmoys, D. B. (2011). The design of approximation algorithms. Cambridge University Press.

- Yee, K.-P., & van Rossum, G. (2001). PEP 234 — Iterators. Python Enhancement Proposals. https://peps.python.org/pep-0234/

- Ziv, J., & Lempel, A. (1977). A universal algorithm for sequential data compression. IEEE Transactions on Information Theory, 23(3), 337–343.

Módulo 6 | Glosario

- **Algoritmo voraz (greedy):** Estrategia que construye una solución mediante decisiones locales iterativas, sin retroceso, guiadas por un criterio.
- **Algoritmo de aproximación (con garantía):** Procedimiento polinómico que asegura una solución dentro de un factor respecto del óptimo para problemas NP-duros de optimización.
- **Argumento de intercambio:** Técnica de prueba que transforma una solución óptima en otra que respeta una elección voraz sin empeorar el objetivo.
- **Asignación (certificado, en SAT/3-SAT):** Valoración booleana propuesta como evidencia; permite verificar en tiempo polinómico si satisface una fórmula CNF.
- **Búsqueda exhaustiva (fuerza bruta):** Enfoque que recorre el espacio de soluciones (subconjuntos, permutaciones o asignaciones) y tiende a requerir tiempo exponencial o peor.
- **Caja envolvente (bounding box):** Región rectangular asociada a un nodo/subárbol que habilita poda al decidir si una consulta intersecta esa región.
- **Celda de Voronoi:** Región del plano cuyos puntos están a distancia menor o igual de un sitio que de cualquier otro sitio.
- **Cerco convexo (convex hull):** Menor polígono convexo que contiene a todos los puntos del conjunto.
- **Clique (CLIQUE):** Subconjunto de vértices tal que cada par está conectado por una arista; se usa para modelar "certificado + verificación" en NP.
- **Complejidad computacional:** Estudio del costo (número de pasos) bajo un modelo de cómputo y su clasificación en clases según restricciones de tiempo.
- **Conjunto convexo:** Conjunto donde el segmento entre dos puntos cualesquiera queda contenido en el conjunto.
- **Convolución circular:** Convolución asumida periódica módulo N, equivalente a operar bajo periodicidad implícita de la DFT.
- **Convolución lineal:** Operación entre secuencias que produce una salida cuya longitud es la suma de longitudes menos 1; el cálculo directo es cuadrático.
- **Cooley–Tukey:** Esquema de descomposición que divide la DFT en subproblemas (por ejemplo, índices pares/impares) y combina resultados con factores de giro.
- **DFT (Transformada Discreta de Fourier):** Transformación que lleva una señal discreta al dominio de frecuencias, produciendo un espectro complejo; su cálculo directo es $O(n^2)$.

- **Distancia euclídea cuadrada:** d^2 en lugar de d; evita usar √ sin alterar el punto que minimiza la distancia.
- **Elección segura:** Decisión local que puede incorporarse a alguna solución óptima sin comprometer la optimalidad global.
- **EPS (tolerancia numérica):** Umbral usado para mitigar errores de punto flotante al comparar valores geométricos o decidir signos.
- **FFT (Fast Fourier Transform):** Familia de algoritmos para calcular la DFT con costo subcuadrático (típicamente O(n log n)) reutilizando subcálculos.
- **Frontera de Voronoi:** Conjunto de puntos equidistantes a dos o más sitios; separa celdas.
- **Heurística:** Procedimiento práctico que puede no ofrecer garantía formal, evaluado mediante evidencia empírica o comparación contra óptimos en instancias pequeñas.
- **IFFT (Transformada inversa):** Operación inversa de FFT/DFT (salvo tolerancias numéricas) que reconstruye la señal original desde el espectro.
- **Intersección de rectángulo con bounding box (poda):** Criterio para descartar subárboles completos en k-d tree si la región de consulta no intersecta su caja.
- **k-d tree:** Estructura espacial que particiona recursivamente por ejes alternantes y permite podas por región para acelerar consultas como rango o vecino más próximo.
- **Memoización:** Técnica que almacena resultados de subproblemas resueltos para reutilizarlos cuando reaparezcan.
- **Método Simplex:** Algoritmo para programación lineal que se desplaza entre vértices (soluciones básicas factibles) mediante pivoteo hasta satisfacer una condición de optimalidad.
- **Monotonic Chain (Andrew):** Algoritmo de convex hull que ordena puntos y construye una cadena inferior y superior eliminando giros no convexos con el predicado de orientación.
- **NP (cómputo no determinista polinómico):** Clase de problemas para los que existe un certificado verificable en tiempo polinómico; se interpreta como "adivinar y verificar".
- **NP-completo:** Problema que pertenece a NP y es NP-duro.
- **NP-duro:** Problema al que se reduce todo problema en NP; expresa "al menos tan difícil como NP".
- **Notación O(·):** Cota superior asintótica que acota el crecimiento de una función de costo hasta factores constantes.

- **Óptimo en un vértice:** Propiedad geométrica de programación lineal: el óptimo (si existe y es finito) se alcanza en un vértice de la región factible.

- **Orientación (predicado orient):** Función basada en producto cruzado/área orientada para decidir giro antihorario, horario o colinealidad de tres puntos.

- **Padding con ceros:** Extensión de una secuencia agregando ceros para ajustar longitud a un tamaño conveniente (por ejemplo, potencia de 2) y habilitar FFT/uso correcto en convolución.

- **Padding correcto (en convolución):** Elección de N (a menudo potencia de 2) para que la convolución circular coincida con la lineal en la región de interés.

- **Política de colineales (en hull):** Regla para incluir o excluir puntos colineales sobre un lado del hull; afecta el conjunto de vértices devuelto.

- **Problema de optimización:** Problema que requiere seleccionar, entre soluciones factibles, la que minimiza o maximiza una función objetivo bajo restricciones.

- **Producto cruzado (cross):** Operación 2D usada para área orientada/orientación y para decidir convexidad y giros en algoritmos geométricos.

- **Producto escalar (dot):** Operación entre vectores usada en cálculos geométricos (proyecciones, ángulos o pruebas auxiliares).

- **Programación dinámica (PD):** Paradigma que define subproblemas y una recurrencia; evita recomputaciones mediante memoización (top-down) o tabulación (bottom-up) y permite reconstruir decisiones óptimas.

- **Raíces de la unidad:** Valores complejos que generan rotaciones necesarias para combinar subtransformadas en FFT.

- **Rayo de verificación (certificado + verificador):** En NP, una solución propuesta se valida en tiempo polinómico; hallar la solución puede requerir búsqueda exponencial.

- **Recurrencia:** Relación que expresa el valor de un subproblema en función de subproblemas más pequeños.

- **Reconstrucción de solución:** Procedimiento que recupera la configuración óptima usando decisiones registradas durante el cómputo (cortes, predecesores).

- **Reducción polinómica:** Transformación computable en tiempo polinómico que preserva la respuesta Sí/No entre problemas.

- **Región factible:** Conjunto de soluciones que satisfacen simultáneamente restricciones; en PL su geometría sustenta el enfoque por vértices de Simplex.

- **Rango (range search):** Consulta que reporta puntos dentro de un rectángulo axis-aligned; puede resolverse de modo ingenuo O(n) o acelerarse con k-d tree y poda.
- **Segmento degenerado (longitud 0):** Caso en el que los extremos coinciden; debe tratarse explícitamente en pruebas de pertenencia (on_segment) y predicados.
- **Solapamiento de subproblemas:** Propiedad por la cual subproblemas idénticos aparecen múltiples veces durante la descomposición recursiva.
- **Subestructura óptima:** Propiedad por la cual una solución óptima puede expresarse como combinación de soluciones óptimas de subproblemas bien definidos.
- **Tabulación:** Técnica bottom-up que llena una tabla en un orden predefinido para calcular cada subproblema una sola vez.
- **Tamaño de entrada:** Medida formal de una instancia (número de variables, vértices, cláusulas) usada para expresar costos.
- **Tiempo exponencial:** Tiempo acotado por c^n (con $c > 1$), típico de enumeraciones exhaustivas.
- **Tiempo polinómico:** Tiempo acotado por n para alguna constante k, asociado operativamente a "eficiencia" en complejidad clásica.
- **Triangulación de Delaunay (dualidad con Voronoi):** Relación conceptual: dos sitios son vecinos en Delaunay si sus celdas de Voronoi comparten frontera.
- **Vecino más próximo (NN):** Punto del conjunto que minimiza la distancia a un punto consulta; fuerza bruta es lineal por consulta y k-d tree puede podar.
- **Vertex Cover / Independent Set (por complemento):** Par de problemas relacionados por transformación: cubrir aristas vs seleccionar vértices sin aristas internas; se usa para ilustrar reducciones.

Módulo 6 | Actividades

ACTIVIDADES

Consigna

Las actividades de este módulo son interactivas y autoasistidas. Para realizarlas, es necesario acceder al enlace ***disponible en plataforma*** y seguir las instrucciones indicadas. Ante cualquier duda o inconveniente durante el desarrollo, se solicita comunicarse con el/la tutor/a para recibir acompañamiento y orientación.